CENGAGE Learning

Ann McIver McHoes
Ida M. Flynn

Sistemas Operativos

Sexta edición

Sistemas operativos

Sexta edición

Ann McIver McHoes

Ida M. Flynn

Traducción:

Hugo Villagómez Velázquez
Traductor profesional

Revisión técnica:

M. en C. Jorge Cortés Galicia
Profesor del Departamento de Ingeniería y Sistemas Computacionales
Escuela Superior de Cómputo
Instituto Politécnico Nacional

MATERIAL PROMOCIONAL PROHIBIDA SU VENTA
MATERIAL PROMOCIONAL PROHIBIDA SU VENTA
MATERIAL PROMOCIONAL PROHIBIDA SU VENTA

CENGAGE Learning

Australia • Brasil • Corea • España • Estados Unidos • Japón • México • Reino Unido • Singapur

CENGAGE Learning™

Sistemas operativos.
Sexta edición
Ann McIver McHoes y Ida M. Flynn

Presidente de Cengage Learning Latinoamérica:
Javier Arellano Gutiérrez

Director general México y Centroamérica:
Pedro Turbay Garrido

Director editorial y de producción Latinoamérica:
Raúl D. Zendejas Espejel

Coordinadora editorial:
María Rosas López

Editor:
Sergio R. Cervantes González

Editor de producción:
Timoteo Eliosa García

Diseño de portada:
Mónica Gómez López

Composición tipográfica:
Ediciones OVA

© D.R. 2011 por Cengage Learning Editores, S.A. de C.V., una Compañía de Cengage Learning, Inc.
Corporativo Santa Fe
Av. Santa Fe núm. 505, piso 12
Col. Cruz Manca, Santa Fe
C.P. 05349, México, D.F.
Cengage Learning™ es una marca registrada usada bajo permiso.

DERECHOS RESERVADOS. Ninguna parte de este trabajo amparado por la Ley Federal del Derecho de Autor, podrá ser reproducida, transmitida, almacenada o utilizada en cualquier forma o por cualquier medio, ya sea gráfico, electrónico o mecánico, incluyendo, pero sin limitarse a lo siguiente: fotocopiado, reproducción, escaneo, digitalización, grabación en audio, distribución en Internet, distribución en redes de información o almacenamiento y recopilación en sistemas de información a excepción de lo permitido en el Capítulo III, Artículo 27 de la Ley Federal del Derecho de Autor, sin el consentimiento por escrito de la Editorial.

Traducido del libro
Understanding Operating Systems.
Sixth Edition.
McIver, Ann/Ida M. Flynn
Publicado en inglés por Course Technology/Cengage Learning © 2011
ISBN-13: 978-1-4390-7920-1
ISBN-10: 1-4390-7920-X

Datos para catalogación bibliográfica:
Mc Iver, Ann/Ida M. Flynn
Sistemas operativos.
Sexta Edición
ISBN: 978-607-481-485-9

Visite nuestro sitio en:
http://latinoamerica.cengage.com

Impreso por Artgraph
Av. Peñuelas No. 15-D
Col. San Pedrito Peñuelas
C.P. 76148 Querétaro, Qro.
Impreso en Diciembre de 2010

Impreso en México
1 2 3 4 5 6 7 14 13 12 11

Dedicado a una maestra ganadora de premios y maravillosa amiga: Ida Moretti Flynn; su amor por enseñar sigue vivo.

AMM

Contenido

Parte uno Concepto de sistemas operativos 1

Capítulo 1 Introducción a los sistemas operativos 3

Introducción 4
¿Qué es un sistema operativo? 4
Software del sistema operativo 4
Administración de la memoria principal 6
Administración del procesador 6
Administración de dispositivos 7
Administración de archivos 7
Administración de redes 7
Interface de usuario 7
Cuestiones de cooperación 8
Breve historia del hardware de la computadora 9
Tipos de sistemas operativos 12
Breve historia del desarrollo de los sistemas operativos 14
Década de 1940 14
Década de 1950 16
Década de 1960 18
Década de 1970 19
Década de 1980 20
Década de 1990 21
Década de 2000 22
Hilos 24
Diseño orientado a objetos 25
Conclusión 26
Términos clave 27
Búsquedas de interés 29
Ejercicios 29

Capítulo 2 Administración de la memoria: primeros sistemas 31

Esquema contiguo de usuario único 32
Particiones fijas 34
Particiones dinámicas 36

Asignación del mejor ajuste contra el primer ajuste 38
Desasignación 44
Caso 1: Unión de dos bloques libres 45
Caso 2: Unión de tres bloques libres 46
Caso 3: Desasignación de un bloque aislado 47
Particiones dinámicas reubicables 48
Conclusión 54
Términos clave 54
Búsquedas de interés 56
Ejercicios 56

Capítulo 3 Administración de la memoria: memoria virtual 63

Asignación de memoria paginada 64
Paginación por demanda 71
Políticas y conceptos de remplazo de páginas 76
Primero en entrar, primero en salir 77
Menos usada recientemente 79
Mecánica de la paginación 82
El conjunto de trabajo 84
Asignación de memoria segmentada 86
Asignación de memoria segmentada/paginación por demanda 89
Memoria virtual 92
Memoria caché 94
Conclusión 98
Términos clave 100
Búsquedas de interés 102
Ejercicios 102

Capítulo 4 Administración del procesador 107

Visión general 108
Acerca de las tecnologías multinúcleo 110
Planificación de trabajos contra planificación de procesos 110
Planificador de procesos 111
Estado del trabajo y del proceso 113
Bloques de control de proceso 114
Bloques de contol de proceso y colas 115
Políticas de planificación de procesos 116
Algoritmos de planificación de procesos 118
Primero en llegar, primero en servirse 118
Siguiente trabajo más corto primero 120

Planificación de prioridad 121
Tiempo más corto restante 122
Round robin 124
Colas de multinivel 127
Caso 1: Sin movimiento entre colas 128
Caso 2: Movimiento entre colas 128
Caso 3: Cuanto de tiempo variable por cola 128
Caso 4: Envejecimiento 129
Comentario sobre interrupciones 129
Conclusión 130
Términos clave 131
Búsquedas de interés 134
Ejercicios 134

Capítulo 5 Administración de procesos 139

Bloqueo mutuo 141
Siete casos de bloqueo mutuo 142
Caso 1: Bloqueos mutuos en solicitudes de archivos 142
Caso 2: Bloqueos mutuos en bases de datos 143
Caso 3: Bloqueos mutuos en asignación de dispositivos dedicados 145
Caso 4: Bloqueos mutuos en asignación de dispositivos múltiples 145
Caso 5: Bloqueos en *spooling* 146
Caso 6: Bloqueos en una red 147
Caso 7: Bloqueos en compartición de discos 148
Condiciones para bloqueos 149
Modelado de bloqueos mutuos 150
Estrategias para manejo de bloqueos mutuos 153
Inanición 161
Conclusión 163
Términos clave 164
Búsquedas de interés 165
Ejercicios 165

Capítulo 6 Procesos concurrentes 171

¿Qué es procesamiento paralelo? 172
Evolución de los multiprocesadores 174
Introducción a los procesadores de núcleo múltiple 174
Configuraciones típicas de multiprocesamiento 175

Configuración maestro/esclavo 175
Configuración débilmente acoplada 176
Configuración simétrica 177
Software de sincronización de procesos 178
Prueba e inicio 179
WAIT y SIGNAL 180
Semáforos 180
Cooperación de procesos 183
Productores y consumidores 183
Lectores y escritores 185
Programación concurrente 187
Aplicaciones de programación concurrente 187
Hilos y programación concurrente 190
Estados de un hilo 191
Bloques de control de hilo 193
Lenguajes de programación concurrente 193
Java 194
Conclusión 196
Términos clave 197
Búsquedas de interés 198
Ejercicios 198

Capítulo 7 Administración de dispositivos 203

Tipos de dispositivos 204
Acceso secuencial de medios de almacenamiento 205
Dispositivos de almacenamiento con acceso directo 208
Almacenamiento en disco magnético de cabeza fija 208
Almacenamiento en disco magnético de cabeza móvil 209
Almacenamiento en disco óptico 211
Tecnología CD-DVD 213
Tecnología Blu-ray 214
Almacenamiento en la memoria flash 215
Tiempos de acceso DASD 216
Dispositivos de cabeza fija 216
Dispositivos de cabeza móvil 218
Componentes del subsistema de E/S 219
Comunicación entre dispositivos 222
Administración de solicitudes de E/S 225
Estrategias de posicionamiento del manejador de dispositivos 226
Estrategias de búsqueda: ordenamiento de rotación 230

RAID 232
Nivel cero 234
Nivel uno 234
Nivel dos 236
Nivel tres 236
Nivel cuatro 236
Nivel cinco 237
Nivel seis 238
Niveles RAID anidados 238
Conclusión 239
Términos clave 240
Búsquedas de interés 243
Ejercicios 243

Capítulo 8 Administración de archivos 249

El administrador de archivos 250
Responsabilidades del administrador de archivos 250
Definiciones 251
Interacción con el administrador de archivos 252
Configuración de volumen típica 253
Acerca de los subdirectorios 255
Convenciones de nomenclatura de archivos 256
Organización de archivos 258
Formato de los registros 259
Organización física de los archivos 259
Asignación de almacenamiento físico 263
Almacenamiento contiguo 263
Almacenamiento no contiguo 264
Almacenamiento indexado 265
Métodos de acceso 267
Acceso secuencial 268
Acceso directo 268
Niveles en un sistema de administración de archivos 269
Módulo de verificación de control de acceso 272
Matriz de control de acceso 273
Listas de control de acceso 274
Listas de capacidad 274
Compresión de datos 275
Compresión de texto 275
Otros esquemas de compresión 276
Conclusión 277

Términos clave 277
Búsquedas de interés 279
Ejercicios 280

Capítulo 9 Conceptos de organización de redes 283

Terminología básica 284
Topologías de red 286
En estrella 287
Topología de anillo 287
Topología de bus 289
Topología de árbol 290
Topología híbrida 291
Tipos de redes 292
Red de área local 292
Red de área metropolitana 293
Red de área amplia 293
Red de área local inalámbrica 293
Cuestiones de diseño de software 295
Convenciones de direccionamiento 295
Estrategias de encaminamiento 296
Modelos de conexión 298
Resolución de conflictos 301
Normas de protocolos de transporte 305
Modelo de referencia OSI 305
Modelo TCP/IP 309
Conclusión 311
Términos clave 311
Búsquedas de interés 313
Ejercicios 314

Capítulo 10 Administración de funciones de redes 317

Historia de las redes 318
Comparación de los sistemas operativos de red y distribuidos 318
Desarrollo del DO/S 321
Administración de la memoria 321
Administración de procesos 323
Administración de dispositivos 328
Administración de archivos 330
Administración de redes 334

Desarrollo del NOS 336
Características importantes del NOS 337
Funciones primordiales del NOS 338
Conclusión 339
Términos clave 339
Búsquedas de interés 340
Ejercicios 340

Capítulo 11 Seguridad y ética 343

Rol del sistema operativo en la seguridad 344
Supervivencia del sistema 344
Niveles de protección 345
Respaldo y recuperación 346
Violaciones de seguridad 347
Intromisiones no intencionales 347
Ataques intencionales 348
Protección del sistema 354
Software antivirus 355
Cortafuegos 356
Autenticación 357
Encriptado 359
Administración de contraseñas 361
Construcción de contraseñas 361
Alternativas de contraseñas 363
Ingeniería social 365
Ética 366
Conclusión 367
Términos clave 367
Búsquedas de interés 370
Ejercicios 370

Capítulo 12 Administración del sistema 373

Evaluación de un sistema operativo 374
Cooperación entre componentes 374
Rol de la administración de la memoria 375
Rol de la administración del procesador 375
Rol de la administración de dispositivos 376
Rol de la administración de archivos 378
Rol de la administración de redes 379

Medición del rendimiento del sistema 380
Herramientas de medición 380
Ciclo de realimentación 383
Administración de parches 385
Fundamentos de parches 386
Opciones de software 388
Sincronización del ciclo de parches 388
Monitoreo 388
Contabilidad 391
Conclusión 392
Términos clave 393
Búsquedas de interés 394
Ejercicios 394

Parte dos Sistemas operativos en la práctica 397

Capítulo 13 Sistema operativo UNIX 401

Visión general 402
Historia 402
La evolución de UNIX 404
Objetivos de diseño 405
Administración de la memoria 406
Administración de procesos 408
Tabla de procesos contra tabla del usuario 409
Sincronización 411
Administración de dispositivos 414
Clasificación de dispositivos 414
Controladores de dispositivos 416
Administración de archivos 417
Convenciones de nomenclatura de archivo 418
Listas de directorios 419
Estructuras de datos 422
Interfaz de usuario 423
Archivos de script 425
Redirección 426
Tuberías 427
Filtros 428
Comandos adicionales 429
Conclusión 431
Términos clave 432

Búsquedas de interés 433
Ejercicios 433

Capítulo 14 Sistema operativo MS-DOS 435

Historia 436
Objetivos de diseño 438
Administración de la memoria 440
Asignación de la memoria principal 442
Asignación de bloques de memoria 443
Administración del procesador 444
Administración de procesos 444
Manejadores de interrupciones 445
Administración de dispositivos 446
Administración de archivos 447
Convenciones de nomenclatura de archivos 447
Administración de archivos 448
Interfaz de usuario 452
Archivos por lotes 453
Redirección 454
Filtros 455
Tuberías 455
Comandos adicionales 456
Conclusión 458
Términos clave 458
Búsquedas de interés 459
Ejercicios 460

Capítulo 15 Sistemas operativos Windows 463

Desarrollo de Windows 464
Primeros productos Windows 464
Sistemas operativos para usuarios de hogar y profesionales 465
Sistemas operativos para redes 466
Objetivos de diseño 467
Extensibilidad 467
Portabilidad 468
Confiabilidad 468
Compatibilidad 469
Rendimiento 470

Administración de la memoria 470
Características del modo de usuario 471
Implementación de la memoria virtual 472
Administración del procesador 474
Administración de dispositivos 476
Administración de archivos 480
Administración de redes 483
Servicios de directorio 484
Administración de seguridad 485
Fundamentos de seguridad 486
Terminología de seguridad 486
Interfaz de usuario 488
Conclusión 493
Términos clave 494
Búsquedas de interés 495
Ejercicios 495

Capítulo 16 Sistema operativo Linux 499

Visión general 500
Historia 500
Objetivos de diseño 502
Administración de la memoria 503
Administración del procesador 506
Organización de la tabla de procesos 506
Sincronización de procesos 507
Administración de procesos 507
Administración de dispositivos 508
Clasificaciones de dispositivos 509
Controladores de dispositivos 509
Clases de dispositivos 510
Administración de archivos 511
Estructuras de datos 511
Convenciones de nomenclatura de archivos 512
Listas de directorios 513
Interfaz de usuario 515
Interfaces basadas en órdenes 516
Interfaces gráficas de usuario 516
Monitor de sistema 517
Configuración de servicios 517
Sistema de registros 518
Atajos de teclado 518

Administración de sistemas 519
Conclusión 520
Términos clave 520
Búsquedas de interés 521
Ejercicios 522

Apéndice

A Código ACM de ética y conducta profesional 525

Glosario 529

Bibliografía 559

Índice analítico 563

Prefacio

Este libro explica de manera no tan técnica este tema bastante técnico al plantear los conceptos de los sistemas operativos en una forma que los lectores puedan entender rápidamente.

Para quienes el tema es nuevo, este texto muestra qué son los sistemas operativos, qué hacen, cómo lo hacen, cómo puede evaluarse su rendimiento y cómo se comparan entre sí. A lo largo de todo el texto describimos la función global e indicamos a los lectores dónde encontrar información más detallada, en caso de así desearlo.

Para quienes cuentan con bases técnicas más sólidas, este texto presenta el tema de manera concisa al describir las complejidades de los sistemas operativos sin entrar en detalles complicados. Puede decirse que este libro se queda donde empiezan otros libros sobre sistemas operativos.

Para lograr lo anterior, hemos establecido algunas hipótesis sobre nuestro público. Primero, suponemos que nuestros lectores poseen algunos conocimientos sobre sistemas informáticos. Segundo, suponemos que cuentan con conocimientos de trabajo de los sistemas operativos y la forma en que éstos interactúan con los usuarios. Recomendamos (aunque no lo requerimos) que los lectores conozcan por lo menos un sistema operativo. En algunas partes encontramos necesario incluir ejemplos usando Java o pseudocódigo a fin de ilustrar el funcionamiento interno de los sistemas operativos pero, para los lectores que no conocen lenguajes de computación, para cada ejemplo hemos agregado una descripción en prosa que explica los hechos en términos más conocidos.

Organización y características

Este libro está estructurado para explicar las funciones de un sistema operativo sin tomar en cuenta el hardware en que está instalado. La organización aborda un problema recurrente con los libros de texto sobre tecnologías que continúan evolucionando; es decir, los avances constantes en un tema en desarrollo hacen obsoletos de inmediato a los libros de texto que sólo están dedicados a tales avances. Para abordar este problema hemos dividido el material en dos partes: primero, los conceptos –que no cambian tan rápido– y segundo, las cuestiones específicas de los sistemas operativos –que cambian rápidamente en unos cuantos meses o años. Nuestro objetivo es proporcionar a los lectores la habilidad para aplicar con inteligencia los temas, percibiendo que aunque un comando, o una serie de comandos, usado por un sistema operativo pueda diferir ligeramente de otro comando de otro sistema operativo, sus objetivos son los mismos y las funciones de los sistemas operativos también son las mismas.

Aunque resulta más difícil entender cómo funcionan los sistemas operativos que memorizar los detalles de uno solo, comprender los conceptos generales de un sistema operativo consti-

tuye un logro a plazo más largo. Esta comprensión también es redituable a largo plazo porque permite adaptarse a medida que cambia la tecnología, como, inevitablemente, ocurre. En consecuencia, el objetivo de este libo es proporcionar a los usuarios de sistemas informáticos una base sólida sobre las componentes de los sistemas operativos, sus funciones y objetivos, así como sobre la forma en que interactúan y se interrelacionan.

La parte uno, los primeros 12 capítulos, describe la teoría de los sistemas operativos. Se centra en cada uno de los "administradores" a su vez y muestra la forma en que trabajan juntos. Luego introduce conceptos de organización de redes, seguridad, ética y administración de funciones de redes. La parte dos analiza sistemas operativos reales, la forma en que aplican las teorías presentadas en la parte uno, y cómo se comparan entre sí.

El capítulo 1 proporciona una breve introducción al tema. El plato fuerte del texto comienza en los capítulos 2 y 3 con Administración de la memoria, porque se trata de la componente más simple de explicar de un sistema operativo y se ha vinculado históricamente con los avances de un sistema operativo al otro. En los capítulos 4, 5 y 6 explicamos el rol del administrador del procesador, al analizar en primer término sistemas simples y luego ampliando el análisis para incluir sistemas de multiprocesamiento. Al momento de llegar a la Administración de dispositivos en el capítulo 7 y a la Administración de archivos en el capítulo 8, los lectores ya se han adentrado en los cuatro principales administradores encontrados en todos los sistemas operativos. Los capítulos 9 y 10 presentan términos y conceptos relacionados con redes, y los capítulos 11 y 12 analizan seguridad, ética y algunos de los sacrificios que los diseñadores consideran cuando tratan de satisfacer las necesidades de su población de usuarios.

En cada capítulo se incluyen objetivos de aprendizaje, términos clave y temas de investigación. Para los lectores con orientación técnica, los ejercicios al final de cada capítulo incluyen problemas para estudiantes avanzados. Observe por favor que en algunos ejercicios avanzados se suponen conocimientos de asuntos no presentados en el texto, aunque son satisfactorios para quienes disfrutan los desafíos. Esperamos que algunos lectores con conocimientos más generales los pasen por alto con todo gusto.

En un intento por hacer más accesibles los conceptos, a lo largo de todo el libro hemos incluido ejemplos de la vida real para ilustrar conceptos abstractos. Sin embargo, no debe confundirse nuestro estilo coloquial con nuestro considerable respeto por el tema de estudio. El tema de los sistemas operativos es complicado y no es posible cubrirlo por completo en estas pocas páginas. Así, este libro de texto no pretende ser un tratado profundo de la teoría y las aplicaciones de los sistemas operativos. Este es el enfoque global.

La parte dos presenta cuatro sistemas operativos en el orden en que fueron lanzados por primera vez: UNIX, MS-DOS, Windows y Linux (nuevo con esta edición). Aquí, en cada capítulo se analiza la forma en que un sistema operativo aplica los conceptos presentados en la parte uno y cómo el sistema operativo se compara con los otros. De nuevo, es necesario recalcar que se trata de un análisis general, ya que un examen profundo de un sistema operativo requiere detalles basados en su versión normal actual, lo cual no es posible hacer aquí. Sugerimos encarecidamente que los lectores usen nuestro análisis como una guía, como una base de partida, al comparar las ventajas y las desventajas de un sistema operativo específico y que complementen nuestro trabajo con investigaciones que sean lo más actuales posible.

El texto concluye con varias referencias de apoyo. Los términos importantes en un capítulo se enumeran en la conclusión de éste como términos clave. El extenso glosario al final del libro incluye definiciones breves de cientos de términos usados en estas páginas. La bibliografía puede orientar al lector para que realice investigaciones básicas sobre el tema. Finalmente, el apéndice presenta el código ACM de ética y conducta profesional.

En este texto no se incluye ningún análisis de bases de datos y estructuras de datos, excepto como ejemplos de problemas de sincronización de procesos, ya que éstos se relacionan de manera tangencial con los sistemas operativos y a menudo constituyen el tema de otros cursos. Sugerimos que los lectores empiecen por aprender los fundamentos según se presentan en las páginas siguientes antes de abordar estos temas complicados.

Cambios en la sexta edición

Esta edición presenta muchas actualizaciones de tecnología sobre la quinta edición:

- Nuevas referencias para el sistema operativo Macintosh OS X, el cual está basado en UNIX.
- Numerosos y nuevos ejercicios de tarea en cada capítulo.
- Referencias actualizadas para expandir la influencia de la tecnología inalámbrica.
- Más información de redes a lo largo del texto.
- Se mantiene el énfasis en sistemas de seguridad y administración por lotes.
- Mayor análisis describiendo la administración de procesos múltiples.
- Detalles de actualidad en los capítulos que analizan UNIX, Windows y Linux.
- Nuevos temas de investigación y ejercicios para los estudiantes para los capítulos en UNIX, MS-DOS, Windows y Linux.

Otros cambios a lo largo del texto son aclaraciones editoriales, leyendas expandidas e ilustraciones mejoradas.

Nota para los instructores

En caso de que este texto se use en el aula, se cuenta con los siguientes suplementos:

Electronic Instructor's Manual. El manual del instructor que acompaña este libro de texto incluye material de enseñanza adicional como apoyo para preparar las clases, incluyendo planes de estudio, resúmenes de capítulos, notas técnicas, notas para clases, pruebas rápidas, sugerencias para enseñanza y temas de análisis.

Aprendizaje a distancia. Course Technology se enorgullece en presentar bancos de pruebas en línea en WebCT y Blackboard para proporcionar la experiencia de aprendizaje dinámico más completa posible. Se anima a los instructores a que cubran la mayor parte del curso,

tanto en línea como fuera de línea. Para más información sobre cómo acceder al banco de prueba en línea, es necesario entrar en contacto con el representante de ventas local de Course Technology.

Presentaciones en PowerPoint. En este libro se incluyen diapositivas de Microsoft PowerPoint para cada capítulo. Las diapositivas se incluyen como apoyo de enseñanza para presentaciones en el aula, ya sea para que los estudiantes cuenten con la red para revisar los capítulos o para imprimirlas y distribuirlas en el aula. Los instructores pueden aportar sus propias diapositivas para temas adicionales que presenten a los alumnos.

Soluciones. En el CD-ROM Teaching Tools se presentan soluciones selectas a preguntas de repaso y ejercicios, que también se encuentran en la página *www.course.com* de Course Technology. Las soluciones están protegidas con contraseñas.

Orden de presentación. Hemos diseñado este libro con una estructura modular para algunas opciones de presentación más cómodas, dependiendo de las preferencias del instructor. Por ejemplo, el programa de estudios puede seguir los capítulos tal y como se presentan desde el Capítulo 1 hasta el Capítulo 12 para presentar los conceptos fundamentales que todos los sistemas operativos tienen en común. Usando esta ruta, los estudiantes pueden aprender acerca de la administración de memoria, procesadores, dispositivos, archivos y redes, en ese orden. Una ruta alternativa puede comenzar con el capítulo 1, seguido de la administración de procesadores en los capítulos 4 al 6, luego la administración de memoria en los capítulos 2 y 3, tocando sistemas de seguridad y administración en los capítulos 11 y 12, y finalmente abordar la administración de dispositivos y archivos en los capítulos 7 y 8. Como redes es a menudo tema de otros cursos, los instructores pueden elegir evitar entrar en los capítulos 9 y 10, o bien pueden incluirlos para un mejor tratamiento de los sistemas operativos.

Esperamos que encuentre útil nuestro análisis de ética en el capítulo 11, el que se incluyó como respuesta a las preguntas de los usuarios universitarios del libro, quienes buscan el análisis de este tema en sus lecturas.

En la parte dos examinamos con detalle cuatro sistemas operativos en un intento por mostrar cómo los conceptos de los 12 primeros capítulos son aplicados en un sistema operativo específico. En cada caso el capítulo está estructurado en una forma similar a los capítulos de la parte uno, es decir, analizan la administración de memoria, procesadores, archivos, dispositivos, redes y sistemas. En suma, cada uno incluye una introducción para una o más interfaces de usuario del sistema operativo. Con esta edición, añadimos ejercicios y temas de investigación para cada uno de estos capítulos para ayudar a los estudiantes a explorar cuestiones analizadas en las páginas previas.

Por primera vez se incluyen referencias al sistema operativo Macintosh OSX en el capítulo de UNIX.

Continuamos con la inclusión de MS-DOS a pesar de su antigüedad debido a que los revisores académicos y los usuarios lo pidieron de manera específica, presumiblemente para que los estudiantes puedan aprender las bases de esta interfaz comando-controlador utilizando un simulador Windows.

Si usted tiene sugerencias para ser incluidas en este libro, por favor háganoslas llegar. Aunque estamos un poco apretados de espacio, será un placer considerar las posibilidades.

Reconocimientos

Nuestro agradecimiento es para todos nuestros amigos y colegas, quienes fueron muy generosos con su aliento, consejos y apoyo. Nuestro agradecimiento especial para: Robert Kleinmann, Eleanor Irwin, Charles R. Woratschek, Terri Lennox y Robert Flynn por su ayuda.

También agradecemos especialmente a todo el personal de Course Technology, Brooks/Cole y PWS Publishing, quienes hicieron aportaciones importantes a todas las cinco ediciones de este texto, especialmente Alyssa Pratt, Kallie Swanson, Mike Sugarman y Mary Thomas Stone. Además, a las siguientes personas, que contribuyeron de manera fundamental para esta edición: Jennifer Feltri, Content Project Manager y Sreejit Govindan, Integra.

Apreciamos profundamente los comentarios de los revisores, quienes nos ayudaron a refinar esta edición:

Revisores del proyecto:
Nisheeth Agrawal: Calhoun Community College
Brian Arthur: Mary Baldwin College
Margaret Moore: University of Phoenix

Revisores por capítulos:
Kent Einspahr: Concordia University
Gary Heisler: Lansing Community College
Paul Hemler: Hampden-Sidney College

Y nuestro agradecimiento a la multitud de estudiantes e instructores que han enviado comentarios y sugerencias valiosos desde la publicación de la primera edición en 1991. Por favor, sigan enviándolos.

Ann McIver McHoes, mchoesa@duq.edu

Ida M. Flynn

Parte Uno

Concepto de sistemas operativos

"Así como el trabajo de las abejas melíferas, criaturas que, por una regla en la naturaleza, enseñan el acto de ordenar un reino poblado."

William Shakespeare *(1564-1616; en Enrique V)*

Todos los sistemas operativos poseen ciertas características fundamentales en común: deben administrar la memoria, la capacidad de procesamiento, los dispositivos y periféricos, archivos y redes. En la parte uno de este texto presentamos una visión general de estas características básicas de los sistemas operativos.

- El capítulo 1 presenta el tema.
- Los capítulos 2-3 analizan la administración de la memoria principal.
- Los capítulos 4-6 cubren la administración del procesador.
- El capítulo 7 se centra en la administración de dispositivos.
- El capítulo 8 está dedicado a la administración de archivos.
- Los capítulos 9-10 revisan brevemente la administración de red.
- El capítulo 11 analiza cuestiones de seguridad del sistema.
- El capítulo 12 explora la administración del sistema e interacción de los componentes de los sistemas operativos.

Luego, en la parte dos del texto (capítulos 13 a 16) se abordan sistemas operativos específicos y la forma en que aplican la teoría presentada en la parte uno.

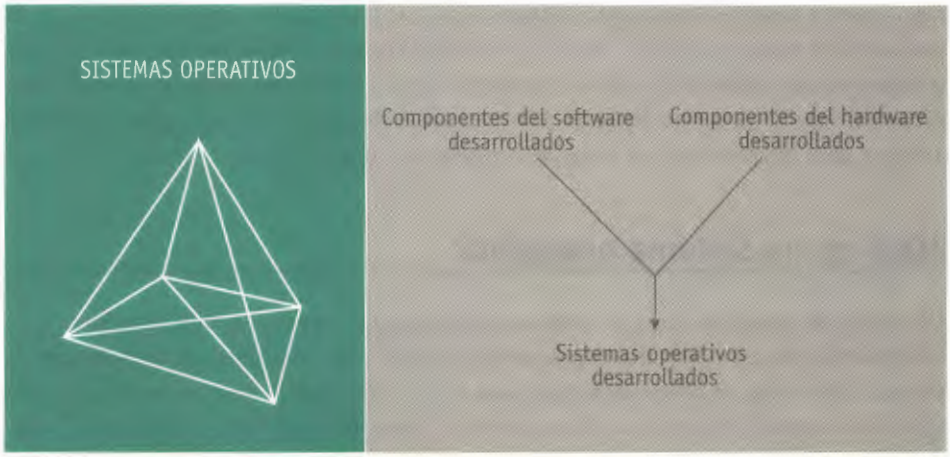
A lo largo de todo nuestro análisis de estos temas bastante técnicos intentamos incluir definiciones de términos que pudieran ser desconocidos para el lector. Sin embargo, no siempre es posible describir una función y definir los términos técnicos sin perder claridad en la explicación. En consecuencia, al final de cada capítulo hemos escrito los términos clave

con las definiciones, y al final del texto se presenta un amplio glosario como referencia. Los asuntos enumerados en la sección Términos clave se muestran en negritas la primera vez que aparecen.

A lo largo del libro hemos mantenido lo más simple posible las descripciones y los ejemplos para introducir al lector en las complejidades del sistema sin enredarlo en detalles técnicos. No obstante, es necesario tomar en cuenta que para casi todos los temas explicados en las páginas siguientes hay más información que puede estudiarse. Nuestro objetivo es presentar el tema y animar al lector para que alcance sus intereses usando otros textos o fuentes primarias en caso de requerir más detalles.

Capítulo 1

Introducción a los sistemas operativos



"Creo que hay un mercado mundial para tal vez cinco computadoras."

Thomas J. Watson *(1874-1956; presidente de IBM 1949-1956)*

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Innovaciones en el desarrollo de sistemas operativos.
- El rol fundamental de un sistema operativo.
- Los administradores de subsistemas de software más importantes y sus funciones.
- Los tipos de hardware de máquina en que funcionan los sistemas operativos.
- Las diferencias entre sistemas operativos por lotes, interactivos, de tiempo real, híbridos e inmersos.
- Multiprocesamiento y su impacto en la evolución del software de los sistemas operativos.
- Virtualización y tendencias en la arquitectura de multinúcleo en sistemas operativos nuevos.

Introducción

Comprender un sistema operativo significa entender el funcionamiento de todo un sistema informático, porque el sistema operativo gestiona todas y cada una de las partes de hardware y software. Este texto explora qué son los sistemas operativos, cómo funcionan, qué hacen y por qué.

Este capítulo describe brevemente la manera en que trabajan los sistemas operativos de uso común y cómo, en general, han evolucionado. Los capítulos siguientes exploran cada componente con mayor profundidad y muestran cómo su función se relaciona con las otras partes del sistema operativo. En otras palabras, usted observará cómo las piezas trabajan en armonía para que el sistema de cómputo funcione sin contratiempos.

¿Qué es un sistema operativo?

Un sistema de cómputo consta de **software** (programas) y **hardware** (la máquina física y sus componentes electrónicos). El **sistema operativo** es la parte fundamental del software, la porción del sistema de cómputo que gestiona todo el hardware y el software. Para ser más específicos, controla todos los archivos, todos los dispositivos, todas las secciones de la memoria principal y todos los nanosegundos del tiempo de procesamiento. Controla quién y cómo puede usar el sistema. En resumen, es el jefe.

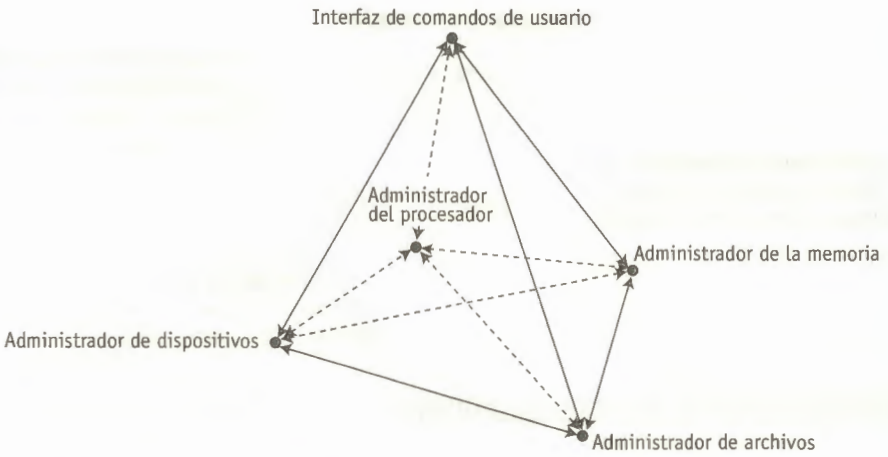
En consecuencia, cada que el usuario envía una orden, el sistema operativo debe asegurarse que ésta se ejecute o, en caso de no ejecutarse, debe arreglárselas para que el usuario reciba un mensaje que explique el error. Recuerde: esto no necesariamente significa que el sistema operativo ejecuta la orden o envía el mensaje de error, sino que controla las partes del sistema que lo hacen.

Software del sistema operativo

La pirámide que se ve en la figura 1.1 es una representación abstracta de un sistema operativo y muestra la manera en que las partes más importantes funcionan juntas.

En la base de la pirámide están los cuatro administradores esenciales de todo sistema operativo: el **administrador de la memoria**, el **administrador del procesador**, el **administrador de dispositivos** y el **administrador de archivos**. De hecho, estos administradores constituyen la base de todos los sistemas operativos y cada uno se analiza en detalle a lo largo de la primera parte de este libro. Cada administrador trabaja estrechamente con los otros administradores y lleva a cabo su papel único sin importar el sistema operativo específico que se analice. En la parte superior de la pirámide está la interfaz de comandos de usuario, desde la cual el usuario emite órdenes al sistema operativo. Éste es un componente único en cada sistema operativo y difiere mucho de un sistema operativo a otro: algunas veces incluso entre versiones diferentes del mismo sistema operativo.

A menos que mencionemos redes o internet, nuestros análisis son válidos para los elementos básicos de los sistemas operativos. Los capítulos 9 y 10 están dedicados a redes.

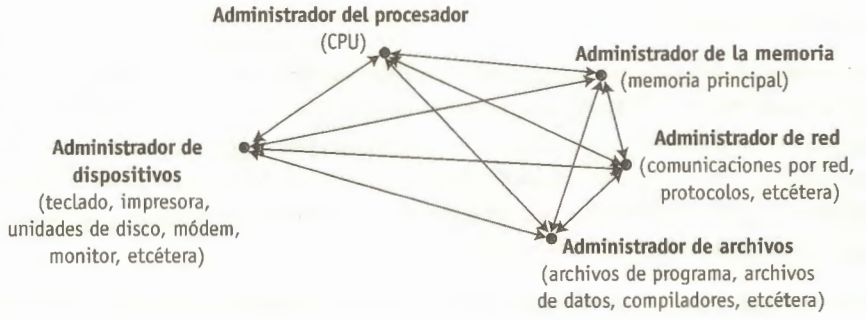


(Figura 1.1)
Este modelo de un sistema operativo no en red muestra cuatro administradores de subsistema que atienden la interfaz de comandos de usuario.

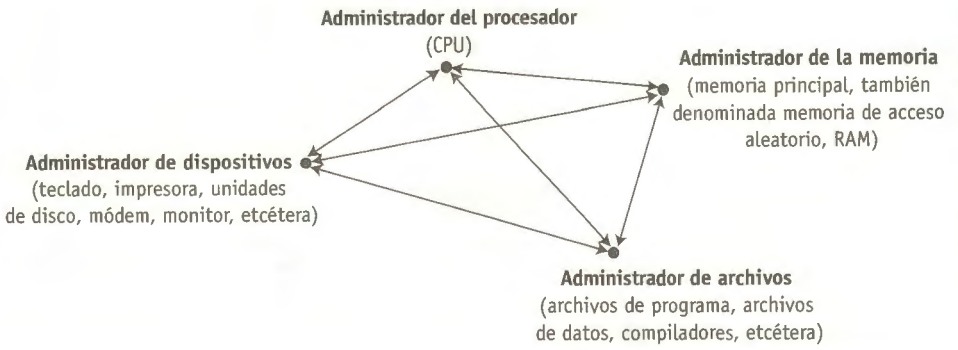
Una red no siempre formaba parte integral de los sistemas operativos; los primeros sistemas eran independientes y toda la capacidad de red estaba montada en la parte superior de sistemas operativos existentes. La mayor parte de los sistemas operativos actuales incorporan de manera rutinaria un **administrador de red** (que se analiza en los capítulos 9 y 10). En la figura 1.2 se muestra la base de la pirámide de un sistema operativo de red.

Sin importar el tamaño o la configuración del sistema, cada uno de los administradores de subsistema, que se muestran en la figura 1.3, debe realizar las siguientes tareas:

- Supervisar sus recursos de manera continua.
- Reforzar las políticas que determinan quién obtiene qué, cuándo y cuánto.
- Asignar el recurso cuando sea conveniente.
- Desasignar el recurso cuando sea conveniente.



(Figura 1.2)
Los sistemas de red cuentan con un administrador de red que asume la responsabilidad de las tareas de red, mientras trabaja en armonía con todos los demás administradores.



(Figura 1.3)
Cada administrador del subsistema en la base de la pirámide asume la responsabilidad de sus propias tareas mientras trabaja en armonía con todos los demás administradores.

Adninistración de la memoria principal

El administrador de la memoria (el tema de los capítulos 2 y 3) está a cargo de la memoria principal, también conocida como RAM, siglas en inglés de Random Access Memory: memoria de acceso aleatorio. El administrador de la memoria verifica la validez de cada solicitud por espacio de memoria y, si es una solicitud legal, asigna una porción de la memoria que aún no está en uso. En un entorno multiusuario, el administrador de la memoria crea una tabla para seguir la pista de quién está usando qué sección de la memoria. Finalmente, cuando llega el momento de reclamar memoria, el administrador de la memoria la desasigna.

RAM es la memoria principal de la computadora y en los primeros sistemas se denominaba "almacenamiento primario".

Una responsabilidad primordial del administrador de la memoria es proteger el espacio en la memoria principal ocupado por el sistema operativo en sí: no puede permitir que ninguna parte de éste sea asignada o modificada accidentalmente.

Administración del procesador

El administrador del procesador (el tema de los capítulos 4 a 6) decide cómo asignar la unidad central de procesamiento (CPU). Una función importante del administrador del procesador es seguir la pista del estado de cada proceso. Aquí, un proceso se define como una instancia de la ejecución de un programa.

El administrador del procesador supervisa si la CPU está ejecutando un proceso o en espera de un comando READ o WRITE para terminar la ejecución. Debido a que maneja las transiciones de los procesos de un estado de ejecución a otro, puede compararse con un controlador de tráfico. Una vez que el administrador del procesador asigna el procesador, establece los registros y tablas necesarios y, cuando ha terminado el trabajo o ha expirado la cantidad máxima de tiempo, reclama el procesador.

Reflexione como sigue: el administrador del procesador tiene dos niveles de responsabilidad. Uno consiste en manejar trabajos en la medida en que ingresan al sistema y el otro es administrar cada proceso dentro de estos trabajos. La primera parte es manejada por el planificador de trabajos, la parte de alto nivel del administrador del procesador, que acepta o rechaza los trabajos de entrada. La segunda parte es manejada por el planificador de procesos, la parte de bajo nivel del administrador del procesador, que es responsable de decidir cuál proceso obtiene la CPU y durante cuánto tiempo.

Administración de dispositivos

El administrador de dispositivos (tema del capítulo 7) supervisa cada dispositivo, canal y unidad de control. Su trabajo consiste en escoger la manera más eficiente para asignar todos los dispositivos del sistema, impresoras, puertos, unidades de disco, etcétera, con base en una política de planificación elegida por los diseñadores del sistema.

El administrador de dispositivos efectúa esto al asignar cada recurso, empezar su operación y, por último, desasignando el dispositivo, poniéndolo a disposición para el siguiente proceso o trabajo.

Adminitración de archivos

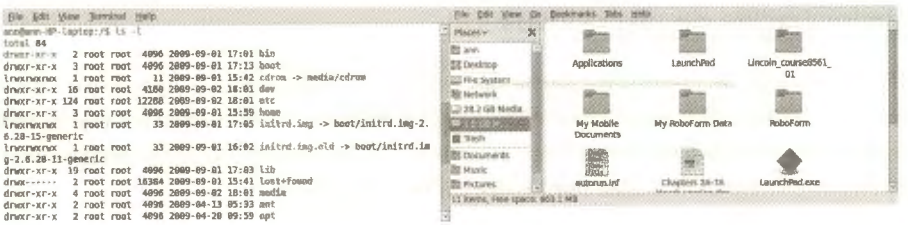
El administrador de archivos (tema del capítulo 8) sigue la pista de todos los archivos en el sistema, incluyendo archivos de datos, archivos de programa, compiladores y aplicaciones. Al usar políticas de acceso predeterminado, refuerza las restricciones sobre quién tiene acceso a qué archivos. El administrador de archivos también controla lo que se permite hacer a los usuarios una vez que tienen acceso a éstos. Por ejemplo, un usuario puede tener acceso de sólo lectura, acceso de lectura y escritura o los permisos para crear y borrar archivos. La administración del control de acceso forma parte fundamental de la administración de archivos. Por último, el administrador de archivos asigna los recursos necesarios y después los desasigna.

Administración de redes

Los sistemas operativos con internet o con capacidad de trabajar en red, tienen un quinto administrador esencial llamado administrador de redes (tema de los capítulos 9 y 10) que proporciona una forma conveniente para que los usuarios participen de los recursos mientras controlan el acceso de los usuarios a los mismos. Estos recursos incluyen hardware (tal como CPUs, áreas de memoria, impresoras, controladores de cinta, módems y controladores de disco) y software (tal como compiladores, programas de aplicación y archivos de datos).

Interface de usuario

Ésta es la parte del sistema operativo que interactúa directamente con el usuario. En el pasado, la interface de usuario consistía de comandos escritos desde un teclado y desplegados en un monitor, como se muestra en la figura 1.4. Ahora, muchos sistemas permiten a los usuarios seleccionar una opción del menú de una lista. La interface de usuario, máquinas de escritorio y formatos varían ampliamente de un sistema operativo a otro, como se muestra en los capítulos 13 a 16 en la parte dos de este libro.



(Figura 1.4)

Dos interfaces de usuario de Linux: una interfaz comando-controlador (izquierda) y una interfaz menú-controlador (derecha).

Cuestiones de cooperación

Sin embargo, no basta que cada administrador realice sus tareas individuales. También debe ser capaz de trabajar en armonía con todos y cada uno de los demás administradores. A continuación se presenta un ejemplo simplificado. Considere que alguien selecciona una opción desde un menú para ejecutar un programa. Los siguientes pasos principales deben ocurrir de manera secuencial:

1. El administrador de dispositivos debe recibir los impulsos eléctricos desde el mouse (o ratón) o el teclado, formar la orden y enviarla a la interfaz de comandos de usuario, donde el administrador del procesador la valida.
2. Luego, el administrador del procesador envía un mensaje de reconocimiento que debe desplegarse en el monitor de video para que el usuario se entere de que el comando ha sido enviado.
3. Cuando el administrador del procesador recibe el comando, determina si el programa debe obtenerse del almacenamiento o ya está en la memoria, y luego notifica este hecho al administrador idóneo.
4. Si el programa está en almacenamiento, el administrador de archivos debe calcular su ubicación exacta en el disco y pasar esta información al administrador de dispositivos, que recupera el programa y lo envía al administrador de la memoria.
5. Después, el administrador de la memoria encuentra espacio para el programa y registra su ubicación exacta en la memoria. Una vez que el programa está en la memoria, el administrador de la memoria debe seguir la pista de su ubicación en la memoria (incluso si se ha movido), así como su avance en la medida en que es ejecutado por el administrador del procesador.
6. Una vez que termina la ejecución del programa, debe enviar un mensaje de terminación al administrador del procesador de modo que el procesador pueda ser asignado al siguiente programa que está esperando en línea.
7. Finalmente, el administrador del procesador debe enviar el mensaje de terminación al administrador de dispositivos, para que éste pueda enviar un mensaje al usuario y actualice la pantalla.

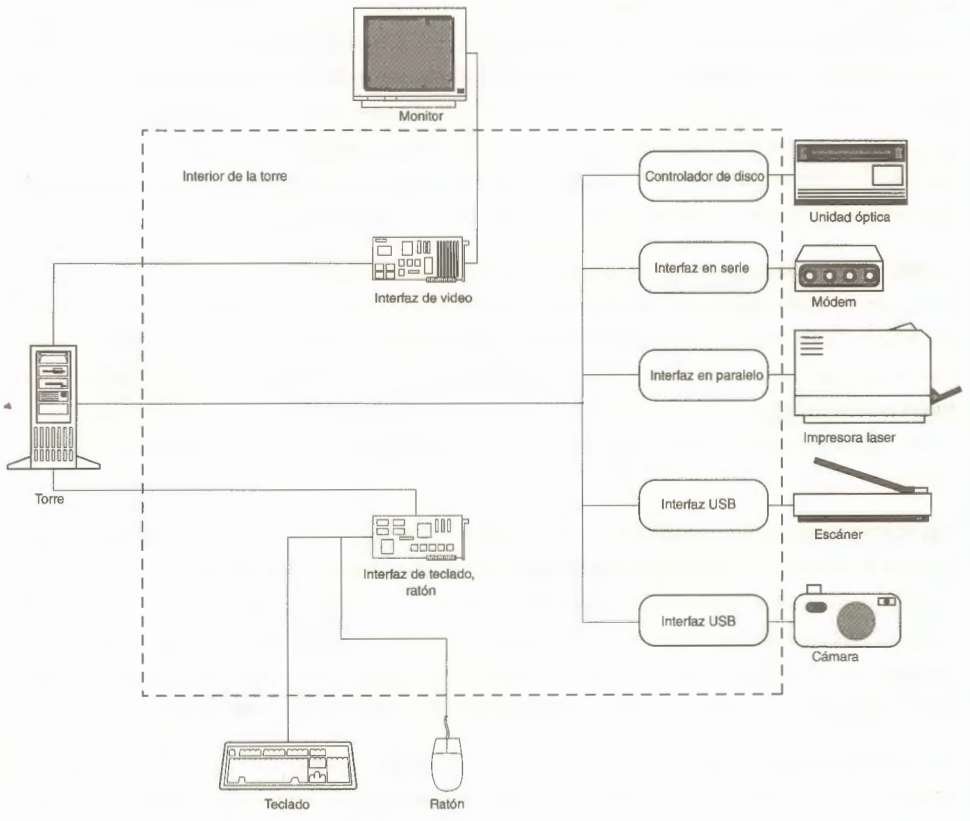
Aunque esta es una demostración bastante simplificada de una operación complicada, ilustra algo de la increíble precisión necesaria para que el sistema operativo pueda trabajar sin contratiempos. Así, aunque durante gran parte de este texto analizaremos cada administrador de manera aislada, ningún administrador único es capaz de efectuar sus tareas sin la cooperación activa de cada una de las otras partes.

Breve historia del hardware de la computadora

Para apreciar el rol del sistema operativo (que es el software), es necesario analizar los aspectos esenciales del hardware del sistema de cómputo, la máquina física y sus componentes electrónicos, incluyendo los circuitos integrados de la memoria, los dispositivos de entrada/salida, los dispositivos de **almacenamiento** y la unidad central de procesamiento (CPU).

- La **memoria principal** (memoria de acceso aleatorio RAM) es donde deben residir los datos y las instrucciones para su procesamiento.
- Los **dispositivos de E/S**, abreviación de dispositivos de entrada/salida, incluyen a todas las unidades periféricas en el sistema, como impresoras, unidades de disco, unidades de CD/DVD, memoria flash, teclados, etcétera.
- La **unidad central de procesamiento** (CPU) es el cerebro con los circuitos (algunas veces denominados el chip) para controlar la interpretación y ejecución de las instrucciones. En esencia, controla la operación de todo el sistema de cómputo, como se ilustra en la figura 1.5. Todas las referencias de almacenamiento, las manipulaciones de datos y las operaciones de E/S son iniciadas o realizadas por la CPU.

Hasta mediados de la década de 1970, las computadoras se clasificaban según su capacidad y precio. Una **computadora central** era una máquina grande en tamaño y capacidad de me-



(Figura 1.5) *Representación lógica de la configuración del hardware de un sistema de cómputo típico. La torre contiene la unidad central de procesamiento, la unidad aritmética y lógica, registros, la memoria caché y la memoria principal, así como controladores e interfaces mostradas con líneas discontinuas.*

moria interna. La IBM 360, introducida en 1964, es un ejemplo clásico de una de las primeras computadoras centrales. La IBM 360 modelo 30 requería una habitación con aire acondicionado de alrededor de 18 pies cuadrados para alojar la CPU, la consola del operador, una impresora, una lectora de tarjetas y una perforadora de tarjetas. La CPU medía 5 pies de alto por 6 pies de ancho, y contaba con una memoria interna de 64 KB (considerada grande en esa época), y su precio era de $200 000 dólares estadunidenses de 1964. Debido a su tamaño y precio en la época, sus aplicaciones estaban generalmente limitadas a grandes centros de cómputo pertenecientes al gobierno federal, universidades y grandes empresas.

La **minicomputadora** fue desarrollada para satisfacer las necesidades de instituciones más pequeñas; aquellas con sólo unas docenas de usuarios. Una de las primeras minicomputadoras fue comercializada por la Digital Equipment Corporation para satisfacer las necesidades de escuelas grandes y pequeñas universidades que empezaron a ofrecer cursos de informática a principios de la década de 1970. (El precio de su PDP-8 era inferior a $18 000.) El tamaño, la capacidad de memoria y el costo de las minicomputadoras eran menores que los de las computadoras centrales. Hoy, las computadoras que se ubican entre las microcomputadoras y las computadoras centrales en cuanto a capacidad a menudo se denominan computadoras de rango intermedio.

La **supercomputadora** fue desarrollada esencialmente para aplicaciones gubernamentales que requerían capacidad masiva y cálculos numéricos rápidos para realizar operaciones militares y predicciones meteorológicas. Las actividades de negocios e industriales comenzaron a interesarse en la tecnología cuando las computadoras masivas se volvieron más rápidas y menos costosas. Un ejemplo típico es la supercomputadora Cray, que cuenta con desde seis hasta miles de procesadores que efectúan hasta 2.4 billones de operaciones de punto flotante por segundo (2.4 teraflops). Las supercomputadoras se usan para una amplia variedad de tareas, desde investigación científica hasta atención al consumidor y desarrollo de productos. Incluso a menudo se utilizaban para realizar cálculos complicados necesarios para crear películas de dibujos animados. Y son de ayuda para las compañías petroleras en su búsqueda de petróleo al analizar cantidades masivas de datos (Stair, 1999).

La **microcomputadora** fue desarrollada a fines de la década de 1970 para ofrecer capacidad de cómputo poco costosa a usuarios individuales. Los primeros modelos contaban con una cantidad revolucionaria de memoria: 64 KB. Físicamente eran menores que las minicomputadoras de esa época, aunque mayores que las microcomputadoras actuales. Finalmente, las microcomputadoras crecieron para dar cabida a software con mayor capacidad y alta velocidad. El rasgo característico de la primera microcomputadora era su condición de usuario único.

Las microcomputadoras poderosas desarrolladas para uso comercial, educativo y gubernamental se denominan **estaciones de trabajo**. En términos generales, las estaciones de trabajo están conectadas entre sí y se usan para atender a usuarios de ingeniería y técnicos que realizan cálculos matemáticos masivos, diseño asistido por computadora (CAD) u otras aplicaciones que requieren una CPU bastante poderosa, grandes cantidades de memoria principal y visualización gráfica de alta resolución para satisfacer sus necesidades.

HP-UX, Sun Solaris y Macintosh OS X son sólo tres de muchos sistemas operativos basados en UNIX.

Los **servidores** son poderosas computadoras que proporcionan servicios especializados a otras computadoras en redes cliente/servidor. Los ejemplos pueden incluir servidores de im-

presión, de internet, de correo electrónico (e-mail), etc.; cada uno desarrolla tareas críticas en la red. Por ejemplo, un servidor de archivos es usualmente una computadora poderosa con una capacidad sustancial de almacenamiento (tan grande como una colección de discos duros), gestiona el almacenamiento de archivos y los recupera de otras computadoras, llamadas clientes, en la red.

(Tabla 1.1)

Breve lista de plataformas y sistemas operativos (enumerados en orden alfabético).

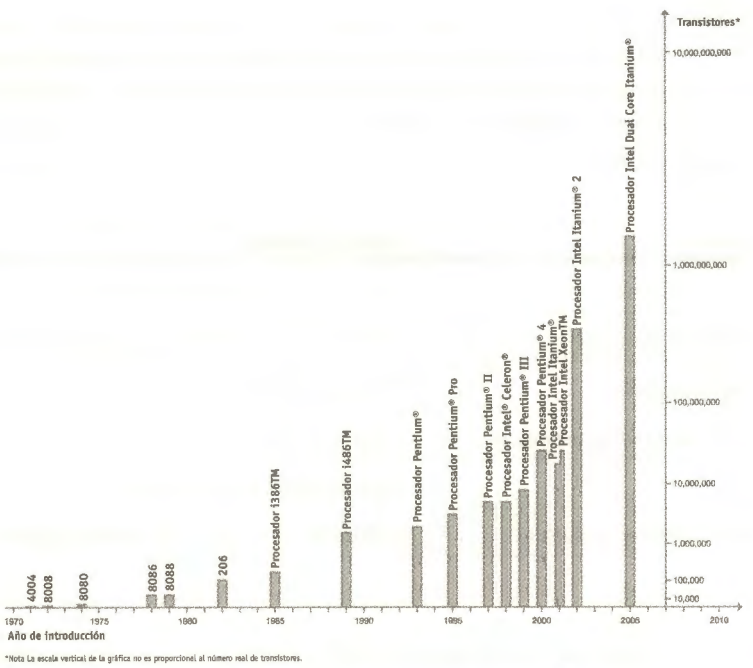
Plataforma	Sistema operativo
Microcomputadoras	Linux, UNIX, (incluye Mac), Windows
Computadoras centrales	IBM Z/390, Linux, IBM OS/400, OpenVMS, UNIX
Supercomputadoras	IRIX, Linux, UNICOS
Estaciones de servicio, servidores	Linux, UNIX, Windows
Redes	Linux, NetWare, UNIX, Windows
Asistentes digitales personales	BlackBerry, Linux, Palm OS, Windows Mobile

En la tabla 1.1 se muestran algunos sistemas operativos típicos para una amplia variedad de plataformas. Desde mediados de la década de 1970, los avances rápidos en la tecnología de las computadoras han desdibujado las características distintivas de primeras máquinas: tamaño físico, costo y capacidad de memoria. Las computadoras centrales simples siguen contando con una gran memoria principal, aunque ahora existen a disposición en gabinetes del tamaño de un escritorio.

Las redes forman parte integral de modernos sistemas de cómputo porque pueden conectar estaciones de servicio, servidores y dispositivos periféricos en sistemas de cómputo integrados. La capacidad de redes se ha vuelto un rasgo estándar en muchos dispositivos de computación: organizadores personales, asistentes digitales personales (PDA), teléfonos celulares y navegadores de red portátiles.

En alguna ocasión las computadoras se clasificaban según su capacidad de memoria: hoy se clasifican según la capacidad de su procesador. Debemos recalcar que estas categorías son relativas y que lo grande hoy se vuelve mediano y luego pequeño en el futuro próximo.

En 1965, el ejecutivo Gordon Moore de Intel observó que cada nuevo chip de procesador contenía aproximadamente el doble de la capacidad de su predecesor, y que cada chip nuevo se lanzaba en un plazo entre 18 y 24 meses después del lanzamiento del chip previo. Moore concluyó que esta tendencia podría provocar que la potencia de las computadoras aumentase exponencialmente sobre periodos relativamente cortos. Conocida ahora como la ley de Moore, mostrada en la figura 1.6, la tendencia ha continuado, sigue siendo extraordinariamente precisa y constituye la base para predicciones en la industria. El chip Intel 4004 en 1971 cuenta con 2300 transistores, mientras que el chip Pentium II veinte años después consta de 7.5 millones, y el procesador Pentium 4 Extreme Edition introducido en 2004 tenía 178 millones de transistores. Con frecuencia la ley de Moore es utilizada por observadores industriales para hacer sus pronósticos de capacidades en los chips.



(Figura 1.6)

El pronóstico de Gordon Moore de 1965 sigue siendo válido por más de tres décadas.

Copyright © 2005 Intel Corporation

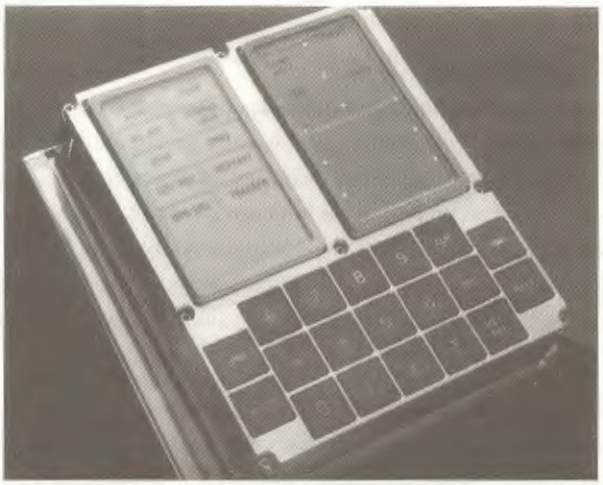
Tipos de sistemas operativos

Los sistemas operativos para computadoras grandes y pequeñas se clasifican en cinco categorías diferenciadas por el tiempo de respuesta y la manera en que los datos se introducen en el sistema: por lotes, interactivos, en tiempo real, híbridos e inmersos.

Los **sistemas por lotes** se remontan a las primeras computadoras, cuando dependían de montones de tarjetas perforadas o carretes de cinta magnética para entrada cuando un trabajo se introducía al ensamblar las tarjetas en un mazo y todo el mazo de tarjetas se hacía pasar por una lectora de tarjetas como grupo: un lote. La eficiencia de un sistema por lotes se mide en **rendimiento**: el número de trabajos completados en una cantidad de tiempo dada (por ejemplo, 550 trabajos por hora).

Los **sistemas interactivos** (también denominados sistemas de tiempo compartido) proporcionan un tiempo de respuesta más rápido que los sistemas por lotes, pero son más lentos que los sistemas en tiempo real que mencionaremos a continuación. Se introdujeron para satisfacer las demandas de usuarios que requerían un tiempo de respuesta rápido al depurar sus programas. El sistema operativo requería el desarrollo de software de tiempo compartido, que permitía que cada usuario interactuara directamente con el sistema de cómputo por medio de comandos introducidos desde una terminal semejante a una máquina de escribir. El sistema operativo proporciona realimentación inmediata al usuario y el tiempo de respuesta puede medirse en fracciones de segundo.

Los **sistemas en tiempo real** se usan en entornos de tiempo crítico donde la confiabilidad es crucial y los datos deben procesarse en un límite de tiempo estricto. El límite de tiempo



(Figura 1.7)

La interfaz de la computadora del Apolo en 1968. El equipo de orientación del Apolo tenía pocas partes móviles y carecía de tubos de vacío, por lo cual era rugoso y compacto.

Cortesía de la NASA

no necesita ser ultra rápido (aunque a menudo lo es), sino que el tiempo de respuesta del sistema debe satisfacer el plazo o se corre el riesgo de sufrir consecuencias importantes. También requieren contar con que las contingencias ocurran con elegancia; es decir, que se preserven las capacidades y datos del sistema tanto como sea posible a fin de facilitar la recuperación. Por ejemplo, los sistemas en tiempo real se usan para vuelos espaciales (como se muestra en la figura 1.7), control de tráfico aéreo, aviones que vuelan por cable, procesos industriales críticos, cierto equipo médico y conmutación telefónica, por mencionar unos cuantos.

Hay dos tipos de sistemas en tiempo real, dependiendo de las consecuencias de no cumplir con el plazo.

- Los sistemas en tiempo real duros corren el riesgo de un fallo total del sistema si no se cumple con el plazo previsto.
- Los sistemas en tiempo real suaves experimentan degradación de rendimiento, pero no fallo total del sistema, como consecuencia de no cumplir con un plazo dado.

Aunque teóricamente es posible convertir un sistema operativo de propósito general en un sistema en tiempo real simplemente al establecer un plazo, la incertidumbre de estos sistemas es incapaz de proporcionar los tiempos de respuesta garantizados que requiere el rendimiento en tiempo real (Dougherty, 1995). En consecuencia, la mayor parte de los sistemas inmersos y los entornos en tiempo real requieren sistemas operativos especialmente diseñados para satisfacer las necesidades en tiempo real.

Los **sistemas híbridos** son una combinación de sistemas por lotes y sistemas interactivos. Parecen ser interactivos porque los usuarios individuales pueden tener acceso al sistema y obtener respuestas rápidas, aunque estos sistemas en realidad aceptan y ejecutan programas por lotes en el entorno cuando la carga interactiva es ligera. Un sistema híbrido aprovecha

el tiempo libre entre el uso con gran demanda del sistema y las ocasiones en que hay baja demanda. Muchos sistemas de cómputo grandes son híbridos.

Los **sistemas inmersos** son computadoras colocadas dentro de otros productos para añadir características y capacidades. Por ejemplo, se encuentran computadoras inmersas en aparatos domésticos, automóviles, reproductores de música digitales, elevadores y marcapasos. En el caso de los automóviles, las computadoras inmersas pueden ayudar con el rendimiento del motor, el frenado y navegación. Por ejemplo, varios proyectos están en vías de implementar "carreteras inteligentes", que pueden alertar a los conductores en automóviles equipados con computadoras inmersas a fin de que elijan rutas alternas cuando hay congestionamientos de tráfico.

Los sistemas operativos para computadoras inmersas son muy diferentes de los sistemas operativos para sistemas de cómputo generales. Cada uno está diseñado para llevar a cabo un conjunto de programas específicos que no son intercambiables entre sistemas. Esto permite que los diseñadores hagan más eficiente el sistema operativo y aprovechen al máximo los recursos limitados de la computadora, como la memoria.

Antes que sea posible usar un sistema operativo de propósito general, como Linux, UNIX o Windows en un sistema inmerso, los diseñadores del sistema deben seleccionar cuáles componentes, del sistema operativo completo, son necesarios en ese entorno particular. La versión final de este sistema operativo incluye sólo los elementos necesarios; cualquier característica o función no deseadas se dejan de lado. En consecuencia, los sistemas operativos con un núcleo pequeño (la parte central del software) y otras funciones que pueden mezclarse y hacerse corresponder para satisfacer los requerimientos de los sistemas inmersos tendrán potencial en este mercado.

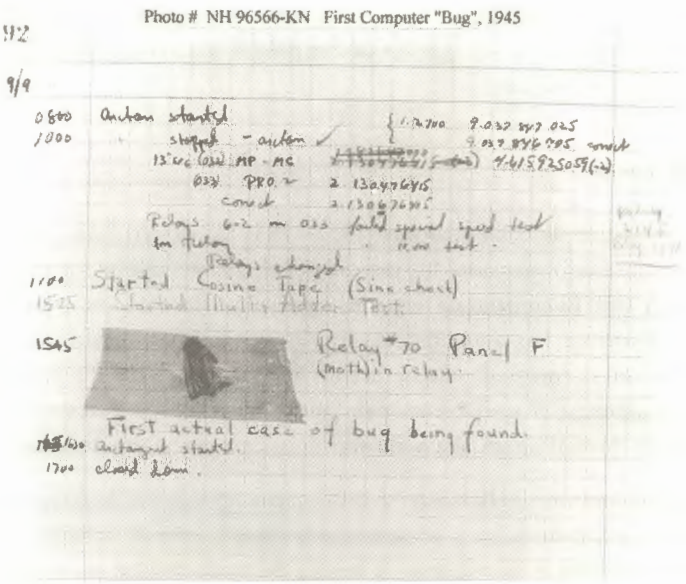
Breve historia del desarrollo de los sistemas operativos

La evolución del software de los sistemas operativos es muy semejante a la evolución del hardware de la computadora que fue diseñado para controlar. A continuación se presenta una visión general muy breve de esta evolución.

Década de 1940

La primera generación de computadoras (1940-1955) surgió en una época de tecnología de tubos de vacío con computadoras del tamaño de un aula. Cada computadora era única en cuanto a estructura y propósito. Había poca necesidad de contar con software de sistema operativo estándar porque el uso de cada computadora estaba restringido a unos cuantos profesionales que trabajaban en aplicaciones matemáticas, científicas y militares, todos los cuales conocían las peculiaridades de su hardware.

Un programa típico incluía todas las instrucciones necesarias para que la computadora efectuara las tareas solicitadas. Proporcionaba direcciones explícitas a la lectora de tarjetas (cuándo empezar, cómo interpretar los datos en las tarjetas, cuándo terminar), la CPU (cómo



(Figura 1.8)

El libro de registro de actividad de la computadora Mark I de la Dra. Grace Hopper en 1945 incluía los restos del primer "bug" en una computadora: una polilla que quedó atrapada en los relés de la computadora, provocando la caída del sistema. El término "bug" es sinónimo de "error" y sigue en uso.

y dónde almacenar las instrucciones en la memoria, qué calcular, dónde encontrar los datos, dónde enviar la salida) y al dispositivo de salida (cuándo empezar, cómo imprimir el producto terminado, cómo formatear la página y cuándo terminar).

Las máquinas eran operadas por los programadores desde la consola principal: era un proceso manual. De hecho, para depurar un programa, los programadores podían detener el procesador, leer el contenido de cada registro, hacer las correcciones en ubicaciones de la memoria y luego reanudar la operación. El primer "bug" (error) fue una polilla atrapada en una computadora de Harvard, como se muestra en la figura 1.8.

Para ejecutar programas, los programadores debían reservar la máquina durante el lapso que estimaban necesario para que la computadora ejecutara el programa. Como resultado, la máquina se subutilizaba. La CPU procesaba datos y realizaba cálculos durante sólo una fracción del tiempo disponible y, de hecho, todo el sistema permanecía ocioso entre las reservaciones.

Con el tiempo, el hardware y el software de la computadora se volvieron más estándares y la ejecución de programas requería menos pasos y menos conocimiento del funcionamiento interno de la computadora. Se desarrollaron compiladores y ensambladores para traducir a código binario los comandos de los lenguajes de alto nivel en evolución que se introducían en términos coloquiales.

Sistemas operativos rudimentarios comenzaron a adquirir forma con la creación de macros, programas de biblioteca, subrutinas normales y programas de utilidad incluían subrutinas de controladores de dispositivos: programas escritos previamente que estandarizaban la forma en que se usaban los dispositivos de entrada y salida.

Los primeros programas estaban en desventaja significativa porque estaban diseñados para usar todos los recursos de manera conservadora a expensas de la comprensibilidad. Eso

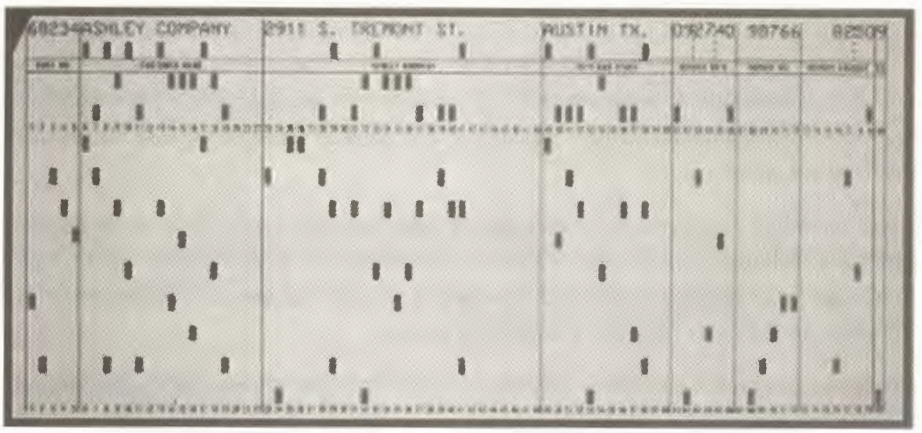
significaba que muchos programas usaban una lógica intrincada que sólo el programador original podía entender, de modo que era casi imposible para cualquier persona depurar o modificar el programa más tarde.

Década de 1950

La segunda generación de computadoras (1955-1965) fue desarrollada para satisfacer las necesidades de un nuevo mercado: los mercados gubernamentales y los investigadores de negocios (figura 1.8). El entorno de negocios impuso mucha más importancia en el costo de la eficacia del sistema. Las computadoras seguían siendo bastante costosas, especialmente en comparación con su equipo de oficina (la IBM 7094 costaba $200 000). En consecuencia, era necesario maximizar el rendimiento para que la inversión para su uso en los negocios mereciera la pena, lo cual significaba aumentar drásticamente el uso del sistema.

Dos mejoras se adoptaron ampliamente: se contrataron operadores de computadora para facilitar la operación de la máquina, y se estableció la planificación de trabajos (o de tareas). La planificación de trabajos es un esquema de mejora de productividad que agrupa los programas con requerimientos semejantes. Por ejemplo, varios programas FORTRAN podían ejecutarse juntos mientras el compilador FORTRAN seguía residiendo en la memoria. O todos los trabajos que usaban la lectora de tarjetas para la entrada podían ejecutarse juntos, y los que usaban la unidad de cinta podían ejecutarse más tarde. Algunos operadores encontraron que la combinación más eficiente era una mezcla de requerimientos de dispositivos de E/S. Es decir, que al mezclar programas con entrada de cinta y programas con entrada de tarjeta, las cintas podían montarse o rebobinarse mientras la lectora de tarjetas estaba ocupada. Una típica tarjeta perforada se muestra en la figura 1.9.

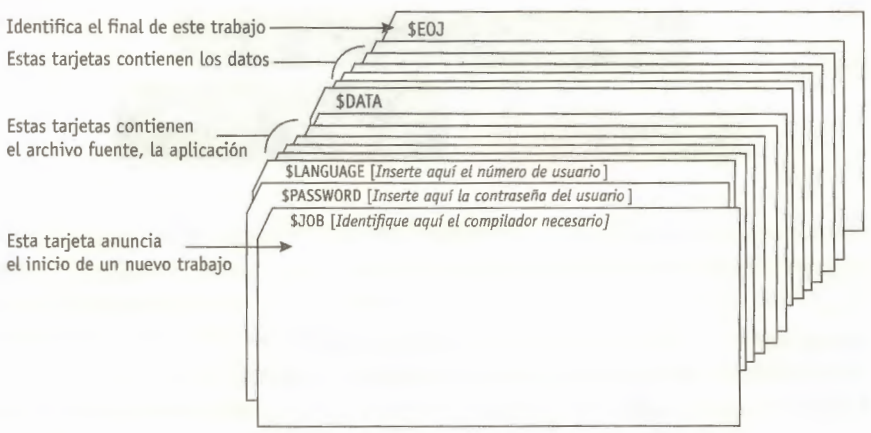
La planificación de trabajos introdujo la necesidad de tarjetas de control, que definían la naturaleza exacta y los requerimientos de cada programa, como se muestra en la figura 1.10. Este fue uno de los primeros usos de un lenguaje de control de trabajos (JCL), que ayudaba al sistema operativo a coordinar y administrar los recursos del sistema al identificar a los usuarios y sus trabajos, así como a especificar los recursos requeridos para ejecutar cada trabajo.



(Figura 1.9)

Cada letra o número impreso a lo largo de la parte superior de la tarjeta perforada es representado por una única combinación de agujeros debajo de ésta.

De ibm.com



(Figura 1.10) *La estructura del programa Lenguaje de Control de Trabajos (llamado JCL) y el orden de las tarjetas perforadas para la computadora DEC-10.*

Pero incluso con técnicas por lotes, la más rápida computadora de segunda generación permitió costosos retrasos de tiempo entre la CPU y los dispositivos de E/S. Por ejemplo, un trabajo con 1600 tarjetas podía requerir 79 segundos para su lectura por la lectora de tarjetas y sólo 5 segundos de tiempo de CPU para su ensamble o compilación. Eso significaba que la CPU permanecía ociosa 94 por ciento del tiempo y ocupada sólo el 6 por ciento del tiempo que dedicaba a ese trabajo, una ineficiencia que resultaba en un pobre uso del sistema completo.

Eventualmente, varios factores ayudaron a mejorar el rendimiento de la CPU.

- Primero, la velocidad de los dispositivos de E/S como unidades de cinta, discos y tambores gradualmente se volvió más rápida.
- Segundo, para usar más del área de almacenamiento disponible en estos dispositivos, los registros se agruparon en bloques antes de su recuperación o almacenamiento. (Denominado "por bloques", esto significa que varios registros lógicos se agrupan dentro de un registro físico y esto se analizará con detalle en el capítulo 7.)
- Tercero, para reducir la discrepancia en velocidad entre la E/S y la CPU, entre éstas se colocó una interfaz denominada unidad de control que actuaba como búfer y es un área de almacenamiento intermedio que funciona como sigue: A medida que el lento dispositivo de entrada lee un registro, la unidad de control coloca cada carácter del registro en el búfer. Cuando el búfer está lleno, todo el registro se transmite rápidamente a la CPU. Para dispositivos de salida, el proceso es justo lo opuesto: la CPU coloca todo el registro en el búfer, que luego es pasado sólo por la unidad de control a la menor tasa requerida por el dispositivo de salida.

  Los búfers de esta generación fueron conceptualmente similares a aquellos que se utilizan actualmente de manera rutinaria por buscadores de internet para suavizar la reproducción de audio y video, como se muestra en la figura 1.11.

  Si una unidad de control tiene más de un búfer, el proceso de E/S puede realizarse aún más rápido. Por ejemplo, si la unidad de control tiene dos búfers, el segundo puede cargarse mientras el primero transmite su contenido hacia o desde la CPU. Idealmente, para cuando se ha transmitido el primero, el segundo ya está listo para actuar y así sucesivamente. De esta manera, el tiempo de entrada, o salida, se reduce a la mitad.

(Figura 1.11)
Tres buscadores típicos con indicadores de progreso en memoria intermedia.

- Cuarto, además de los búfers, se desarrolló una forma temprana de spooling al mover fuera de línea las operaciones de lectura, impresión y "perforación" de tarjetas. Aunque "spool" se usa hoy como sustantivo, es un acrónimo que significa Simultaneous Peripheral Operation On-Line (operación periférica simultánea en línea). Por ejemplo, los trabajos de entrada deben transferirse de conjuntos de tarjetas a carretes de cinta magnética fuera de línea. Después es posible leerlas en la CPU desde la cinta a una velocidad mucho más alta que la de una lectora de tarjetas. Un spooler funciona de la misma manera que un búfer pero, en este ejemplo, se trata de un dispositivo fuera de línea por separado, mientras que un búfer forma parte del hardware de la computadora principal.

También durante la segunda generación se desarrollaron técnicas para administrar bibliotecas de programas, crear y mantener archivos e índices de datos, mantener direcciones aleatorias de acceso directo, crear y verificar etiquetas de archivos. Se desarrollaron interruptores de tiempo para permitir la compartición de trabajos y proteger la CPU de ciclos infinitos en programas a los que erróneamente se instruía para ejecutar una sola serie de comandos por siempre. Una cantidad fija de tiempo de ejecución se asignaba a cada programa cuando éste entraba en el sistema y era supervisado por el sistema operativo. Programas que seguían funcionando cuando expiraba el tiempo.

Durante la segunda generación los programas seguían asignándose al procesador uno a la vez. El paso siguiente hacia una mejor utilización de los recursos del sistema consistió en desplazarse hacia el procesamiento compartido.

Década de 1960

La tercera generación de computadoras data de mediados de la década de 1960. Fueron diseñadas con CPU más rápidas, pero su velocidad continuó causando problemas cuando interactuaban con impresoras y otros dispositivos de E/S relativamente lentos. La solución fue la **multiprogramación**, que introdujo el concepto de cargar muchos programas a la vez para compartir la atención de una sola CPU.

Los primeros sistemas de multiprogramación permitían que cada programa fuese atendido por turno, uno después de otro. El mecanismo más común para implementar la multiprogramación consistía en la introducción del concepto de interrupción, que es cuando se notifica a la CPU de eventos que requieren los servicios del sistema operativo. Por ejemplo, cuando un programa emite un comando de E/S, genera una interrupción que solicita los servicios del procesador de E/S y la CPU es liberada para empezar la ejecución del siguiente trabajo. Esto se denominaba *multiprogramación pasiva* porque el sistema operativo no controlaba las interrupciones, sino que esperaba que cada trabajo terminara una secuencia

de ejecución. Era menos que ideal porque si un trabajo estaba limitado por la CPU (lo cual significaba que efectuaba una gran cantidad de procesamiento sin detención antes de emitir una interrupción) podía atar a la CPU durante periodos largos mientras otros trabajos debían esperar.

Para contrarrestar este efecto, al sistema operativo pronto se le asignó un rol más activo con la aparición de la *multiprogramación activa*, que permitía que cada programa utilizase sólo una rebanada preestablecida de tiempo de CPU, lo que se analiza en el Capítulo 4. Una vez que expiraba el tiempo, el trabajo se interrumpía y se permitía que otro trabajo empezara su ejecución. El trabajo interrumpido debía esperar su turno hasta que se le permitía reanudar su ejecución más tarde. La idea de las rebanadas de tiempo pronto se hizo común en muchos sistemas de tiempo compartido.

La planificación de programas, que empezó con los sistemas de segunda generación, continúa hasta la fecha, aunque se complicó por el hecho de que la memoria principal estaba ocupada por muchos trabajos. Para resolver este problema, se ordenaron los trabajos por grupos y luego cargados en la memoria según una rotación prestablecida. Con frecuencia, este ordenamiento era determinado según su prioridad o sus requerimientos de memoria: aquello que asegurara el uso más eficiente de los recursos. Además de la planificación de trabajos, el manejo de interrupciones y asignación de la memoria, los sistemas operativos debían resolver conflictos cuando dos trabajos solicitaban el mismo dispositivo al mismo tiempo, exploraremos algunos en el Capítulo 5.

Siempre que ha habido progreso en la administración de procesadores, se han registrado unos cuantos avances en la administración de datos.

Década de 1970

Después de la tercera generación, a finales de la década de 1970, las CPU de las computadoras eran más rápidas, lo cual originó una disparidad aún mayor entre la alta velocidad de procesamiento y el lento tiempo de acceso de E/S. La primera supercomputadora Cray, que se muestra en la figura 1.9, fue lanzada en 1976. Los esquemas de multiprogramación para incrementar el uso de la CPU estaban limitados por la capacidad física de la memoria principal, que era un recurso limitado y muy costoso.

Una solución a esta limitación física fue el desarrollo de la memoria virtual, que aprovechó el hecho de que la CPU podía procesar sólo una instrucción a la vez. Con la memoria virtual no era necesario que todo el programa residiera en la memoria antes de poder empezar su ejecución. Un sistema con memoria virtual debía dividir los programas en partes y mantenerlas en almacenamiento secundario, llevando cada parte a la memoria sólo en la medida en que era necesaria. (Los programadores de computadoras de segunda generación habían usado este concepto en el método de programación roll in/roll out, hoy denominado superposición, para ejecutar programas que exceden la memoria física de estas computadoras.)

En esta ocasión también había una atención creciente respecto a la necesidad de conservación de los recursos de datos. El software para la administración de datos comenzó a ser una herramienta popular debido a sus datos organizados de manera integrada, a la redundancia minimizada y a una actualización y acceso simplificados de los datos. Se introdujeron varios sistemas de consulta que permitían incluso a los usuarios novatos recuperar porciones específicas de la base de datos. Estas consultas solían hacerse por medio de una terminal que, a su vez, administraba un crecimiento en el software de atención a terminales y comunicación de datos.

Pronto, los programadores se fueron alejando cada vez más de las complicaciones de la computadora, y los programas de aplicación empezaron a usar palabras coloquiales, estructuras modulares y operaciones normales. Esta tendencia hacia el uso de normas mejoró la administración de programas porque el mantenimiento del programa se hizo cada vez más rápido y fácil.

Década de 1980

El desarrollo en esta década mejoró drásticamente la relación costo/rendimiento de los componentes de las computadoras. El hardware era más flexible, con funciones lógicas integradas en tarjetas de circuitos de fácil sustitución. Y puesto que era menos costoso crear estos circuitos impresos, muchas funciones del sistema operativo se integraron al hardware en sí, dando paso a un nuevo concepto: **firmware**, palabra usada para indicar que un programa se mantiene en forma permanente en la ROM (memoria de sólo lectura), en oposición a su mantenimiento en almacenamiento secundario. La tarea del programador, según se había definido en años previos, cambió drásticamente porque muchas funciones de programación las efectuaba el software del sistema, haciendo así que la tarea del programador fuese más sencilla y menos dependiente del hardware.

Eventualmente, la industria se desplazó al **multiprocesamiento** (contar con más de un procesador), y se diseñaron lenguajes más complicados para coordinar las actividades de los múltiples procesadores que atendían un solo trabajo. Como resultado, se hizo posible ejecutar programas en paralelo, y al final se esperaba de manera rutinaria que los sistemas operativos de computadoras de todos tamaños fuesen capaces de realizar multiprocesamiento.

La evolución de las computadoras personales y las comunicaciones a gran velocidad desencadenaron el desplazamiento hacia sistemas de redes y procesamiento distribuido, lo cual permitió que los usuarios en ubicaciones remotas compartieran recursos de hardware y software. Estos sistemas requerían un nuevo tipo de sistema operativo, uno capaz de administrar múltiples conjuntos de administradores de subsistemas, como hardware que pudiera encontrarse a la mitad del mundo de distancia.

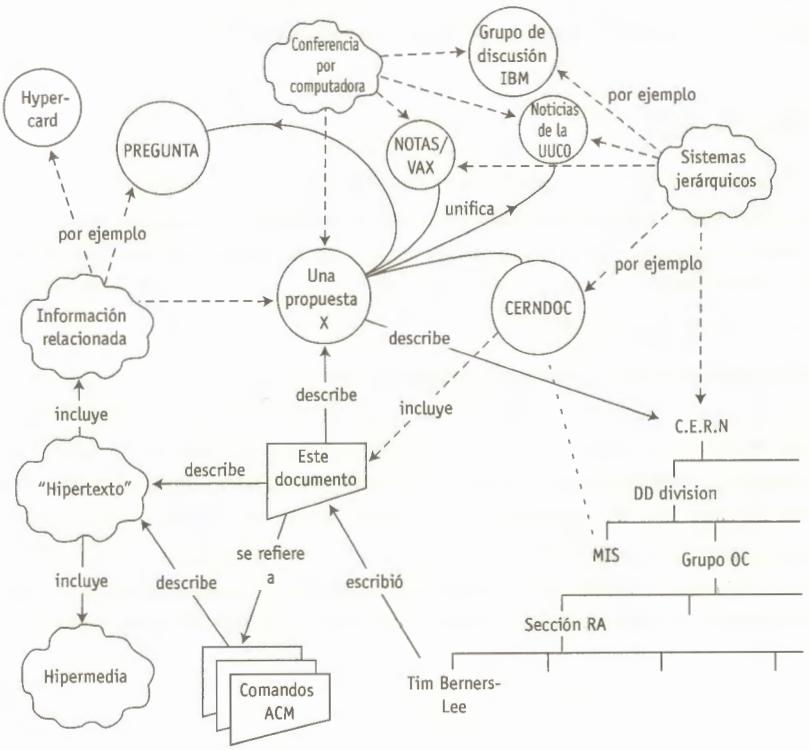
Con los sistemas operativos de red, los usuarios en general se enteraron de la existencia de muchos recursos en red, que podían registrarse con ubicaciones remotas y que podían manipular archivos en computadoras de red distribuidas en una amplia zona geográfica. Los sistemas operativos de red eran semejantes a los sistemas operativos de procesador único en el sentido de que cada máquina ejecutaba su propio sistema operativo local y tenía sus

propios usuarios. La diferencia consistía en la adición de un controlador de la interfaz de red con software de bajo nivel para controlar el sistema operativo local, así como programas para permitir el registro remoto y el acceso remoto a archivos. Aún así, incluso con estas adiciones, la estructura básica del sistema operativo de red se aproximaba bastante a la de un sistema independiente.

Por otra parte, con sistemas operativos distribuidos, los usuarios podían pensar que estaban trabajando con un sistema uniprocesador típico, cuando de hecho estaban conectados a un grupo de muchos procesadores trabajando juntos estrechamente. Con estos sistemas, los usuarios no necesitaban saber cuál procesador estaba ejecutando sus aplicaciones o cuáles dispositivos estaban almacenando sus archivos. Todos estos detalles eran manejados de manera transparente por el sistema operativo, algo que requería más que simplemente agregar unas cuantas líneas de código a un sistema operativo uniprocesador. La desventaja de este sistema operativo tan complicado era el requerimiento de más algoritmos de planificación del procesador complejos. Además, los retrasos en la comunicación dentro de la red algunas veces significaban que los algoritmos de planificación tenían que operar con información incompleta u obsoleta.

Década de 1990

La abrumadora demanda por capacidad para internet a mediados de 1990 desencadenó la proliferación de la capacidad de red. La Red Global Mundial o World Wide Web, ideada por Tim Berners-Lee en un papel, que se muestra en la figura 1.12, hizo posible el acceso a inter-



(Figura 1.12)

Ilustración de la primera página de la propuesta de 1989 de Tim Berners-Lee que describe su revolucionario "sistema de información relacionado". Con base en esta investigación, Berners-Lee diseñó el primer navegador y servidor de la Red Global Mundial, poniéndolo a disposición del público en 1991.

net para todos los usuarios de computadora en el mundo entero, no sólo para los investigadores que se habían vuelto dependientes de ésta para comunicación global. La accesibilidad a la red y el correo electrónico se convirtieron en características esenciales de casi todos los sistemas operativos. Sin embargo, el incremento de redes también ocasionó un aumento en la demanda de seguridad más estrecha para proteger el hardware y software.

En esta década también se introdujo una proliferación de aplicaciones multimedia que demandaban potencia y flexibilidad adicionales, así como compatibilidad de dispositivos para casi todos los sistemas operativos. Una computadora multimedia típica aloja dispositivos para efectuar creación y edición de audio, video y gráficos. Esto puede requerir muchos dispositivos especializados como un micrófono, un piano digital, una interfaz digital para instrumentos musicales (MIDI), una cámara digital, una unidad de disco de video digital (DVD), unidades de disco compacto (CD), altavoces, monitores adicionales, dispositivos para proyección, impresoras a color y conexiones de alta velocidad a internet. Estas computadoras también requieren software y hardware especiales (como controladores, tarjetas, buses) para hacer que funcionen correctamente.

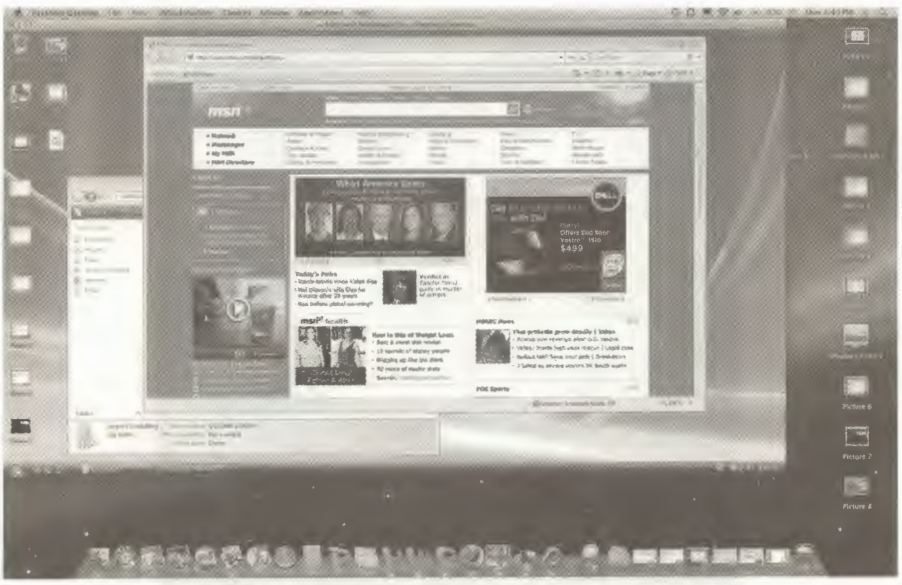
Las aplicaciones multimedia requieren grandes cantidades de capacidad de almacenamiento que el sistema operativo debe manejar con elegancia. Por ejemplo, cada segundo de un video a pantalla completa de 30 cuadros por minuto requiere 27 MB de almacenamiento, a menos que los datos estén comprimidos de alguna manera. A fin de satisfacer la demanda de video comprimido, las compañías de hardware han desarrollado chips y tarjetas de video de propósito especial.

¿Cuál es el efecto de estos avances tecnológicos sobre el sistema operativo? Cada avance requiere un avance paralelo en las capacidades de administración en el software.

Década de 2000

El nuevo siglo recalcó la necesidad de que los sistemas operativos ofrecieran mejor flexibilidad, confiabilidad y velocidad. Para satisfacer la necesidad de computadoras capaces de dar cabida a múltiples sistemas operativos en ejecución al mismo tiempo y compartiendo recursos, se desarrolló el concepto de máquinas virtuales, mostrado en la figura 1.13 y que se volvió viable comercialmente.

La **virtualización** es la creación de particiones en un servidor único, con cada partición atendiendo un sistema operativo diferente. En otras palabras, convierte un solo servidor físico en múltiples servidores virtuales, a menudo con múltiples sistemas operativos. La virtualización requiere que el sistema operativo cuente con un administrador intermedio para supervisar el acceso de cada sistema operativo a los recursos físicos del servidor. Por ejemplo, con la virtualización, un solo procesador puede ejecutar 64 sistemas operativos independientes en estaciones de trabajo usando un procesador capaz de permitir la ejecución simultánea de 64 hilos (secuencias de instrucciones) por separado para ejecutarse al mismo tiempo.



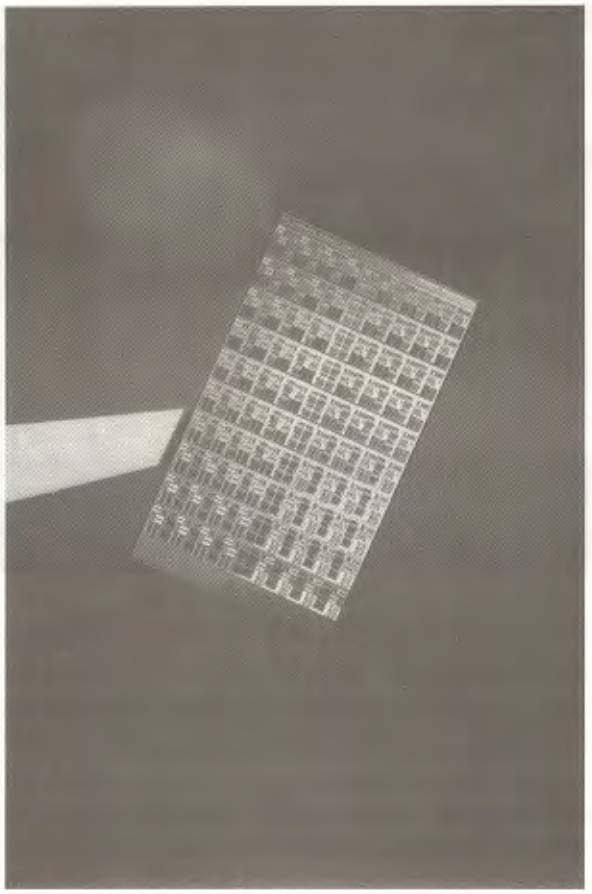
(Figura 1.13)

Con la virtualización, en una sola computadora es posible ejecutar sistemas operativos diferentes.

Cortesía de Parallels, Inc.

La velocidad de procesamiento ha experimentado un avance semejante con el desarrollo de procesadores de núcleo múltiple, mostrados en la figura 1.14. Hasta hace pocos años, las obleas de silicio que constituyen la base de los circuitos del chip de computadora alojaban un solo CPU. Sin embargo, con la introducción de procesadores de doble núcleo, un solo chip puede contener núcleos de procesadores múltiples. Así, un chip de doble núcleo permite la ejecución de dos conjuntos de cálculos al mismo tiempo, lo cual algunas veces conduce a que el trabajo se complete más rápido. Es como si el usuario tuviera dos computadoras por separado, y dos procesadores, cooperando en un solo trabajo. Al momento de escribir esto, los diseñadores han creado chips que contienen 80 núcleos simples.

Esta innovación, ¿afecta al software del sistema operativo? Por supuesto, ya que ahora debe administrar el trabajo de estos procesadores múltiples y ser capaz de planificar y administrar el procesamiento de sus múltiples tareas. En el capítulo 6 exploraremos algunas de las complejidades de este tema.



(Figura 1.14)

Una sola pieza de silicio es capaz de contener 80 núcleos, que (para plantearlo en términos más sencillos) pueden realizar 80 cálculos a la vez.

Cortesía de Intel Corporation

Hilos

La tecnología multinúcleo ayuda a que el sistema operativo maneje hilos y acciones múltiples que pueden ejecutarse al mismo tiempo. Primero, una explicación: el administrador de procesos es responsable de procesar cada trabajo presentado por un usuario. Los trabajos constan de procesos (algunas veces denominados "tareas" en otros libros de texto), y los procesos constan de hilos múltiples.

Un proceso posee dos características:

- Necesita espacio en la memoria principal donde permanece durante su ejecución, aunque de vez en cuando requiere otros recursos como archivos de datos o dispositivos de E/S.
- Pasa por varios estados (como ***en ejecución***, ***en espera*** y ***listo***) desde su arribo inicial al sistema de cómputo hasta su terminación.

La multiprogramación y la memoria virtual determinan que los procesos deben intercambiarse entre la memoria principal y el almacenamiento secundario durante su ejecución. Con los procesos convencionales (también denominados procesos pesados), este intercambio re-

sulta en una gran cantidad de procesamiento extra. Lo anterior se debe a que cada vez que ocurre un intercambio es necesario guardar toda la información del proceso para preservar la integridad del proceso.

Un **hilo** (o proceso ligero) puede definirse como una unidad más pequeña que un proceso, que puede planificarse y ejecutarse. Al usar esta técnica, el proceso pesado, que es propietario de los recursos, se vuelve un elemento más pasivo, mientras un hilo se convierte en el elemento que usa la CPU y es planificado para su ejecución. La manipulación de hilos consume menos tiempo que la manipulación de procesos, que son más complicados. Algunos sistemas operativos atienden procesos múltiples en un solo hilo, mientras otros lo hacen con multihilos.

Los navegadores de red usan de manera rutinaria multihilos para permitir que los usuarios exploren al mismo tiempo múltiples áreas de interés en internet.

Los multihilos posibilitan que las aplicaciones manejen un proceso por separado con varios hilos de control. Los navegadores de red usan multihilos de manera rutinaria. Por ejemplo, un hilo es capaz de recuperar imágenes mientras otro envía y recupera correo electrónico. Los multihilos también se usan para incrementar la capacidad de respuesta en un sistema de tiempo compartido, para incrementar la compartición de recursos y disminuir la sobrecarga.

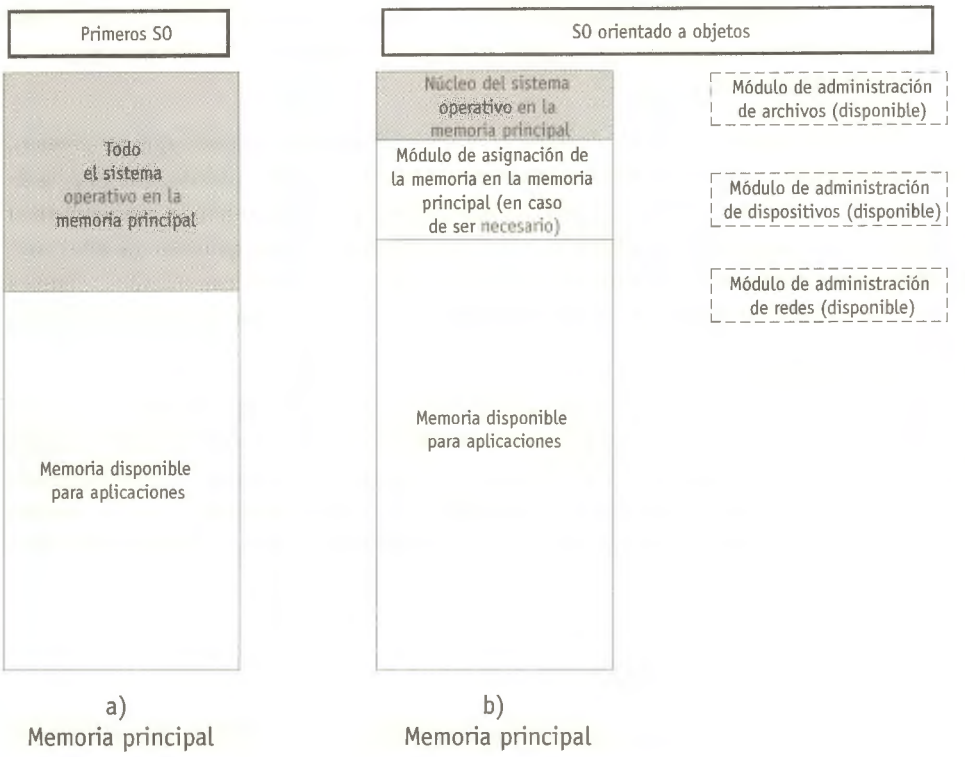
Diseño orientado a objetos

Un área importante de investigación que dio por resultado gran eficiencia sustancial era la de la arquitectura del sistema de los sistemas operativos; la forma en que sus componentes se programan y organizan, específicamente el uso del diseño orientado a objetos y la reorganización del núcleo del sistema operativo. El núcleo es la parte del sistema operativo que se encuentra todo el tiempo en la memoria, efectúa las tareas más importantes del sistema operativo y está protegido por el hardware contra la manipulación de parte del usuario.

Los primeros sistemas operativos fueron diseñados como una sola unidad extensa, como se muestra en la figura 1.15 *a*). Almacenaban todos los elementos requeridos del sistema operativo en la memoria, como asignación de la memoria, planificación de procesos, asignación de recursos y administración de archivos. Este tipo de arquitectura hizo engorroso y consumidor de tiempo para los programadores añadir nuevas componentes al sistema operativo o modificar componentes existentes.

Más recientemente, la parte del sistema operativo que reside en la memoria estaba limitada a unas cuantas funciones esenciales, como planificación de procesos y asignación de memoria, mientras todas las otras funciones, como asignación de dispositivos, son realizadas por módulos especiales, que son tratados como aplicaciones regulares, como se muestra en la figura 1.15 *b*). Este enfoque hace más fácil agregar nuevas componentes o modificar componentes existentes.

El diseño orientado a objetos fue la fuerza motriz detrás de esta nueva organización. Los objetos son módulos independientes (unidades de software) que proporcionan modelos del mundo real y pueden ser reutilizados en aplicaciones diferentes. Al trabajar sobre objetos, los programadores pueden modificar y personalizar partes de un sistema operativo sin en-



(Figura 1.15)

Los primeros sistemas operativos a) se cargan en su totalidad en la memoria principal. Los sistemas modernos orientados a objetos b) sólo cargan los elementos críticos en la memoria principal y llaman a otros objetos según sea necesario.

torpecer la integridad del resto del sistema. Además, el uso de un enfoque modular orientado a objetos puede hacer que los grupos de desarrollo de software sean más productivos de lo que era posible con la programación estructurada de procedimiento.

Conclusión

En este capítulo consideramos la función global de los sistemas operativos y cómo han evolucionado para administrar sistemas de cómputo cada vez más complicados, pero como ocurre en cualquier otro tema difícil, hay muchos más detalles por explorar. Como veremos en el resto de este texto, hay muchas formas para efectuar cada tarea y corresponde al diseñador del sistema operativo elegir las políticas idóneas para el entorno del sistema.

En los siguientes capítulos analizaremos con detalle la manera en que trabaja cada parte del sistema operativo, así como sus características, funciones, beneficios y costos. Empezaremos con la parte del sistema operativo que constituye el corazón de cualquier computadora: el módulo que administra la memoria principal.

Términos clave

Administrador de archivos: Sección del sistema operativo responsable del control del uso de los archivos.

Administrador de dispositivos: Sección del sistema operativo responsable del control del uso de los dispositivos. Supervisa cada dispositivo, canal y unidad de control, y escoge la manera más eficiente para asignar a todos los dispositivos del sistema.

Administrador de la memoria: Sección del sistema operativo responsable de controlar el uso de la memoria. Verifica la validez de cada solicitud por espacio de memoria y, en caso de ser una solicitud legal, asigna la cantidad necesaria para ejecutar el trabajo.

Administrador de redes: Sección del sistema operativo responsable de controlar el acceso y uso de los recursos en red.

Administrador del procesador: Composición de dos subadministradores: el planificador de trabajos y el planificador de procesos, que decide la manera de asignar la CPU.

Almacenamiento: Sitio donde se almacenan los datos en el sistema de cómputo. El almacenamiento primario es la memoria principal. El almacenamiento secundario lo constituyen medios no volátiles, como discos y memoria flash.

Computadora central: Nombre histórico dado a un sistema de cómputo grande caracterizado por su gran tamaño, alto costo y elevado rendimiento.

Estación de trabajo: Computadora de escritorio conectada a una red de área local que sirve como punto de acceso a esa red.

Firmware: Instrucciones o datos de software que están almacenados de manera fija o "en firme", que suelen implementarse en la memoria de sólo lectura (ROM).

Hardware: La máquina física y sus componentes, incluyendo la memoria principal, dispositivos de E/S, canales de E/S, dispositivos de almacenamiento de acceso directo y la unidad central de procesamiento.

Hilo: Porción de un programa que puede funcionar independientemente de otras partes. Los programas de aplicación multihilos pueden tener varios hilos en ejecución al mismo tiempo con la misma prioridad o con prioridades distintas.

Kernel: Parte esencial del sistema operativo que permanece en la memoria de acceso aleatorio (RAM) y es cargada con la ejecución de las tareas más importantes del sistema, como la administración de la memoria principal y el acceso a discos.

Memoria principal: Unidad de la memoria que trabaja directamente con la CPU y donde deben residir los datos e instrucciones para su procesamiento. También se denomina almacenamiento primario o memoria interna.

Microcomputadora: Pequeña computadora equipada con todo el hardware y software necesarios para realizar una o más tareas.

Minicomputadora: Sistema de cómputo de tamaño pequeño a mediano, también denominado computadora de rango intermedio.

Multiprocesamiento: Cuando dos o más CPU comparten la misma memoria principal, la mayor parte de los dispositivos de E/S y las mismas rutinas del programa de control. Las CPU atienden el mismo flujo de trabajo y ejecutan programas de procesamiento distintos de manera concurrente.

Multiprogramación: Técnica que permite que un solo procesador procese varios programas que residen simultáneamente en la memoria principal y abandone su ejecución mediante la superposición de solicitudes de E/S con solicitudes de la CPU.

Núcleo: Parte de procesamiento de un chip del CPU formado por la unidad de control y la unidad lógica aritmética.

Orientado a objetos: Filosofía de programación donde los programas constan de módulos independientes reutilizables denominados objetos, cada uno de los cuales realiza una función específica, aunque están clasificados en clases de objetos que comparten la misma función.

Red: Conjunto de sistemas de cómputo y dispositivos periféricos interconectados que intercambian información entre sí.

Servidor: Nodo que proporciona a los clientes varios servicios de red, como recuperación de archivos, impresión o servicios de acceso a bases de datos.

Sistema en tiempo real: Sistema de cómputo usado en entornos críticos que requieren tiempos de respuesta garantizados, como sistemas de navegación, sistemas de transporte rápido y sistemas de control industrial.

Sistema híbrido: Sistema de cómputo que atiende procesos por lotes e interactivos.

Sistema inmerso: Sistema de cómputo dedicado, a menudo pequeño y rápido, que reside en un sistema físico más grande, como un avión de propulsión a chorro o un barco.

Sistema interactivo: Sistema que permite que cada usuario interactúe directamente con el sistema operativo mediante comandos introducidos desde un teclado.

Sistema operativo: Software que administra todos los recursos de un sistema de cómputo.

Sistema por lotes: Un tipo de sistema desarrollado para las primeras computadoras, que usaba tarjetas perforadas o cinta como entrada, que se introducían en un "lote".

Software: Colección de programas usados para realizar ciertas tareas. El software se clasifica en tres grandes categorías: programas del sistema operativo, compiladores y ensambladores y programas de aplicación.

Supercomputadora: Las primeras computadoras más avanzadas y rápidas que se produjeron, usadas para realizar cálculos complicados.

Unidad central de procesamiento (CPU): Componente con los circuitos, "chips", para controlar la interpretación y ejecución de las instrucciones.

Rendimiento: Medida compuesta de la eficiencia del sistema, que cuenta el número de trabajos realizados en una determinada unidad de tiempo.

Virtualización: Creación de una versión virtual de hardware o software. La virtualización del sistema operativo permite que una sola CPU ejecute imágenes de varios sistemas operativos al mismo tiempo.

Búsquedas de interés

Para mayor respaldo en algunos de los temas analizados en este capítulo, comience una búsqueda con los siguientes términos:

- Museo de la historia de la computación.
- NASA: Computadoras a bordo del transbordador espacial.
- IBM: Archivo de la historia de la computación.
- Historia del sistema operativo UNIX.
- Historia de los productos Microsoft.

Ejercicios

Temas de investigación

Siempre que investigue sobre tecnología informática, asegúrese de que sus recursos son oportunos. Observe la fecha en que fue publicada la investigación. Evite cualquier fuente que pueda ser cuestionable, como blogs y fuentes en línea editadas públicamente.

A. Escriba una revisión de una página de un artículo sobre el tema de sistemas operativos de reciente aparición en una revista especializada o en un periódico académico. Asegúrese de citar su fuente. Elabore un resumen del artículo, incluyendo el tema principal, la información presentada y la conclusión del autor. Proporcione su evaluación personal del artículo, incluyendo un comentario sobre el estilo de redacción del autor, el uso inapropiado del lenguaje técnico, los temas que hicieron interesante el artículo y su relevancia para sus propias experiencias.

B. Investigue en Internet sobre la literatura actual para identificar un sistema operativo que ejecute un teléfono celular o una computadora palm. (Estos dispositivos suelen denominarse sistemas operativos móviles.) Enumere las características principales del sistema operativo y del hardware que están diseñados para ejecutar. Cite sus fuentes.

Ejercicios

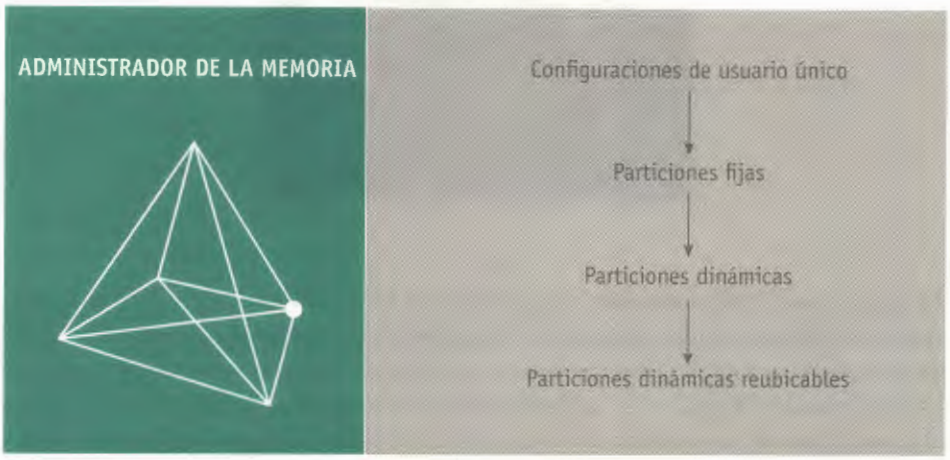
1. Mencione cinco sistemas operativos actuales (no mencionados en este capítulo) y las computadoras o configuraciones en que opera cada uno.
2. Mencione los cinco componentes fundamentales de un sistema operativo que considere necesario que los usuarios novatos conozcan y comprendan.
3. Explique el impacto de la evolución del hardware de las computadoras y la evolución correspondiente del software de los sistemas operativos.

4. En su opinión, ¿la ley de Moore ha sido un simple pronosticador del diseño de chips o un aliciente para los diseñadores de chips? Explique su respuesta.
5. Explique las diferencias fundamentales entre sistemas interactivos, por lotes, en tiempo real e inmersos.
6. Enumere tres situaciones que pudieran demandar un sistema operativo en tiempo real y explique por qué.
7. Proporcione un ejemplo de una organización que podría encontrar útil el procesamiento en modo de lotes y explique por qué.
8. Enumere tres recursos físicos (tangibles) de almacenamiento de datos de un sistema de cómputo típico. Explique las ventajas y desventajas de cada uno.
9. Compare brevemente la multiprogramación activa y pasiva.
10. Proporcione por lo menos dos razones por las que un banco multiestatal podría decidir comprar seis servidores de computadoras en lugar de una computadora más poderosa. Explique su respuesta.
11. Seleccione uno de los siguientes profesionales: un ajustador de seguros, una persona de servicio de entregas para un servicio de correos, un reportero de un periódico, un médico (general) o un gerente de un supermercado. Sugiera por lo menos dos formas en que esta persona podría utilizar una computadora manual para llevar a cabo sus funciones de manera más eficiente.

Ejercicios avanzados

12. Compare el diseño de objetivos y la evolución de dos sistemas operativos descritos en los capítulos 13 a 16 de este libro.
13. Elabore un diagrama de flujo que ilustre los pasos realizados por un sistema operativo en la medida en que ejecuta la instrucción para respaldar un disco en un sistema de cómputo de usuario único. Empiece cuando el usuario teclea el comando o hace clic en el mouse y concluya con la exhibición del resultado en el monitor.
14. Identifique las frecuencias de reloj de los procesadores que usan (o usaban) 8 bits, 16 bits, 32 bits y 64 bits. Analice varias implicaciones al planificar la CPU en un sistema multiprocesamiento usando estos procesadores.
15. En un entorno multiprogramación y de tiempo compartido, varios usuarios comparten el sistema de manera simultánea. Esta situación puede resultar en varios problemas de seguridad. Mencione dos de estos problemas. ¿Puede asegurarse el mismo grado de seguridad en una máquina de tiempo compartido que en una máquina dedicada? Explique sus respuestas.
16. Proporcione un ejemplo de aplicación donde el rendimiento proporcionado por multihilos sea superior al proporcionado por un solo hilo.
17. Si un proceso termina, ¿sus hilos también terminan o continúan su ejecución? Explique su respuesta.
18. Si se suspende un proceso (si una interrupción lo coloca en estado de espera), ¿sus hilos también se suspenden? Explique su respuesta y proporcione un ejemplo.

Capítulo 2 Administración de la memoria: primeros sistemas



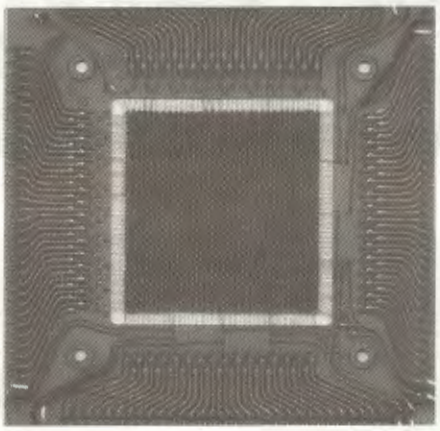
"La memoria constituye el poder primario y fundamental, sin la cual no puede haber ninguna otra operación intelectual."

Samuel Johnson *(1709–1784)*

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- La funcionalidad básica de los tres esquemas de asignación de la memoria presentados en este capítulo: particiones fijas, dinámicas y dinámicas reubicables.
- El esquema de asignación de la memoria del mejor ajuste, así como el del primer ajuste.
- Cómo una lista de memoria sigue la pista de la memoria disponible.
- La importancia de desasignación (o liberación) de la memoria en un sistema de particiones dinámicas.
- La importancia del registro de límites en esquemas de asignación de la memoria.
- El rol de la compactación y cómo ésta mejora la eficiencia de asignación de la memoria.



(Figura 2.1)

Circuito de memoria principal de 1961 (antes de que fueran tan pequeños para verse sin una amplificación).

Cortesía de technikum29

La administración de la **memoria principal** es crucial. De hecho, desde una perspectiva histórica, el rendimiento de *todo* el sistema ha dependido directamente de dos cosas: la cantidad de memoria disponible y la manera en que se optimiza mientras se procesan los trabajos. La fotografía de la figura 2.1 es un circuito de memoria principal de 1961. Desde entonces, el tamaño físico de las unidades de memoria ha cambiado y disminuido sus dimensiones y ahora están disponibles en pequeñas tabletas.

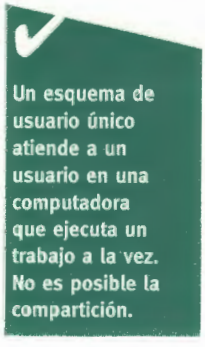
Este capítulo introduce el administrador de la memoria (también conocido como memoria de acceso aleatorio o RAM, núcleo de memoria o almacenamiento primario) y cuatro tipos de esquemas de asignación de la memoria: sistemas de usuario único, particiones fijas, dinámicas y dinámicas reubicables.

Estos primeros esquemas se usan rara vez en sistemas operativos actuales, pero su estudio es importante porque cada uno introdujo conceptos fundamentales que ayudaron a la evolución de la administración de la memoria, como se muestra en el capítulo 3, "Administración de la memoria: memoria virtual", donde se analizan estrategias de asignación de la memoria para Linux. En las secciones sobre administración de la memoria en la parte dos del texto se presenta información sobre la forma en que los sistemas operativos administran la memoria.

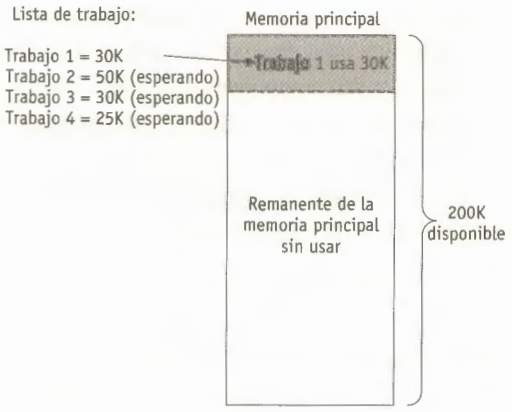
Empecemos con el esquema de administración de la memoria más simple: el que se usaba en las primeras generaciones de sistemas de cómputo.

Esquema contiguo de usuario único

El primer esquema de asignación de la memoria funcionaba como sigue: cada programa que iba a ser procesado se cargaba en su totalidad en la memoria y se le asignaba tanto espacio contiguo en la memoria como era necesario, según se muestra en la figura 2.2. Las palabras clave son *totalidad* y *contiguo*. Si el programa era demasiado grande y no cabía en el espacio disponible de la memoria, no podía ejecutarse. Y, aunque las primeras computadoras eran físicamente grandes, contaban con muy poca memoria.



Un esquema de usuario único atiende a un usuario en una computadora que ejecuta un trabajo a la vez. No es posible la compartición.



(Figura 2.2)

Un programa se instala en la memoria de una vez. El remanente de memoria permanece sin usarse.

Esto muestra un factor limitante significativo de todas las computadoras: sólo tienen una cantidad finita de memoria y si un programa no cabe, entonces debe aumentarse el tamaño de la memoria principal o debe modificarse el programa. El programa suele modificarse haciéndolo más pequeño o usando métodos que permiten la superposición de segmentos del programa (particiones hechas al programa). (La superposición consiste en transferir segmentos de un programa desde el almacenamiento secundario hacia la memoria principal para su ejecución, de modo que dos o más segmentos ocupan a su vez las mismas ubicaciones en la memoria.)

Los sistemas de usuario único en un entorno no de red funcionan de la misma manera. A cada usuario se le concede acceso a toda la memoria disponible para cada trabajo y los trabajos se procesan secuencialmente, uno después de otro. Para asignar la memoria, el sistema operativo usa un algoritmo simple (procedimiento paso a paso para resolver un problema):

Algoritmo para cargar un trabajo en un sistema de usuario único

1 Almacenar la primera ubicación en la memoria del programa en un registro base (para protección de la memoria).
2 Poner el contador del programa (éste sigue la pista del espacio de memoria usado por el programa) igual a la dirección de la primera ubicación en la memoria.
3 Leer la primera instrucción del programa.
4 Incrementar el contador de programa con el número de bytes en la instrucción.
5 ¿Ya se llegó a la última instrucción?
   En caso afirmativo, entonces detener el cargado del programa.
   En caso negativo, entonces continuar con el paso 6.
6 ¿El contador del programa es mayor que el tamaño de la memoria?
   En caso afirmativo, entonces detener al cargado del programa.
   En caso negativo, entonces continuar con el paso 7.
7 Cargar la instrucción en la memoria.
8 Leer la siguiente instrucción del programa.
9 Ir al paso 4.

Observe que la cantidad de trabajo realizado por el administrador de la memoria del sistema operativo es mínima, que el código para efectuar las funciones es directo y que la lógica es bastante simple. Sólo se requieren dos componentes de hardware: un registro para almacenar la dirección base y un acumulador para seguir la pista del tamaño del programa en la medida en que se lee en la memoria. Una vez que el programa se ha cargado por completo en la memoria, permanece ahí hasta que se completa la ejecución, ya sea mediante la terminación normal o por la intervención del sistema operativo.

Un problema fundamental con este esquema de asignación de la memoria es que no atiende de multiprogramación o redes (ambos temas se analizarán más tarde en este texto); sólo puede manejar un trabajo a la vez. Cuando estas configuraciones de usuario único estuvieron comercialmente disponibles por primera vez a fines de la década de 1940 y principios de la década de 1950, se usaron en instituciones de investigación, aunque demostraron ser inaceptables para la comunidad de negocios; no era económicamente eficaz gastar casi $200 000 por un equipo que sólo podía usar una persona a la vez. En consecuencia, a fines de la década de 1950 y principios de la década de 1960 se requería otro esquema para administrar la memoria, que usaba particiones para aprovechar los recursos del sistema de cómputo al superponer operaciones independientes.

Particiones fijas

El primer intento por permitir multiprogramación usaba **particiones fijas** (también denominadas **particiones estáticas**) dentro de la memoria principal: una partición para cada trabajo. Debido a que el tamaño de cada partición se diseñaba al encender la computadora, cada partición sólo podía reconfigurarse una vez que se apagaba el sistema de cómputo, se reconfiguraba y se volvía a iniciar. Así, una vez que el sistema estaba en operación, los tamaños de las particiones permanecían estáticos.

Cada partición podía usarse sólo por un programa. El tamaño de cada partición lo fijaba de antemano el operador de la computadora, de modo que no era posible modificar los tamaños sin volver a iniciar el sistema.

Con este esquema se introdujo un factor crítico: la protección del espacio de memoria del trabajo. Una vez que una partición se asignaba a un trabajo, no se permitía que ningún otro trabajo penetrara en sus límites, ya fuera de manera accidental o intencional. Este problema de intrusión en la partición no existía en esquemas de asignación contigua de usuario único porque en la memoria principal sólo había un trabajo en cualquier instante dado, de modo que sólo debía protegerse la porción del sistema operativo residente en la memoria principal. Sin embargo, para los esquemas de asignación de particiones fijas, la protección era obligatoria para todas las particiones presentes en la memoria principal. Normalmente esta era la responsabilidad conjunta del hardware de la computadora y el sistema operativo.

El algoritmo usado para almacenar trabajos en la memoria requiere unos cuantos pasos más que el usado para un sistema de usuario único porque el tamaño del trabajo debe coincidir con el tamaño de la partición para tener la certeza de que cabe por completo. Después, cuando se ubica un bloque de tamaño suficiente, es necesario verificar el estado de la partición para ver si está disponible.

Algoritmo para cargar un trabajo en una partición fija

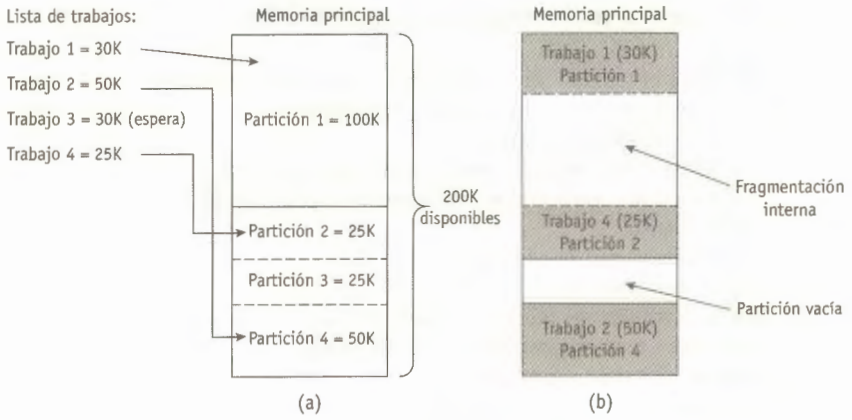
```
1 Determinar el tamaño de la memoria solicitada por el trabajo.
2 Si job_size > tamaño de la partición más grande
    Entonces rechazar el trabajo
      imprimir el mensaje idóneo para el operador
      ir al paso 1 para manejar el siguiente trabajo en línea
    En otro caso
      continuar con el paso 3
3 Poner el contador en 1
4 Mientras contador <= número de particiones en la memoria
  Si job_size > memory_partition_size(contador)
  Entonces contador = contador + 1
  En otro caso
    Si memory_partition_size(contador) = "libre"
      Entonces cargar el trabajo en memory_partition(contador)
        cambiar memory_partition_status(contador) a "ocupado"
        ir al paso 1 para manejar el siguiente trabajo en línea
    En otro caso
      contador = contador + 1
    Fin del ciclo
5 En este momento no hay disponible ninguna partición; poner el trabajo
  en cola de espera
6 Ir al paso 1 para manejar el siguiente trabajo en línea
```

Este esquema de particiones es más flexible que el de usuario único porque permite que varios programas estén en la memoria al mismo tiempo. Sin embargo, sigue requiriendo que *todo* el programa se almacene *contiguamente* y *en la memoria* desde el principio hasta el final de su ejecución. A fin de asignar espacios de memoria a trabajos, el administrador de la memoria del sistema operativo debe mantener una tabla, como la tabla 2.1, que muestra el tamaño de cada partición de la memoria, su dirección, sus restricciones de acceso y su estado actual (libre u ocupado) para el sistema ilustrado en la figura 2.3. (En la tabla 2.1 y en las otras tablas en este capítulo, K significa kilobytes, que es igual a 1024 bytes. En el capítulo 8, "Administración de archivos" se presenta un análisis más profundo sobre las tablas de mapeo de la memoria.)

(Tabla 2.1) *Tabla de memoria de partición fija simplificada, donde la partición libre está sombreada.*

Tamaño de la partición	Dirección de la memoria	Acceso	Estado de la partición
100K	200K	Trabajo 1	Ocupado
25K	300K	Trabajo 4	Ocupado
25K	325K		Libre
50K	350K	Trabajo 2	Ocupado

A medida que termina cada trabajo, el estado de su partición de la memoria cambia de ocupado a libre, de modo que un trabajo de entrada puede asignarse a esa partición.



(Figura 2.3)

Uso de la memoria principal durante la asignación de particiones fijas de la tabla 2.1. El trabajo 3 debe esperar incluso cuando en la partición 1 hay 70K de espacio libre disponibles, donde el trabajo 1 sólo ocupa 30K de los 100K disponibles. A los trabajos se asigna espacio con base en la "primera partición disponible del tamaño requerido".

El esquema de particiones fijas funciona bien si todos los trabajos que se ejecutan en el sistema son del mismo tamaño o si de antemano se conocen todos los tamaños y éstos no varían entre reconfiguraciones. Idealmente, lo anterior requeriría un conocimiento preciso de antemano de todos los trabajos a ejecutar en el sistema en las próximas horas, días o semanas. No obstante, a menos que el operador pueda predecir el futuro con precisión, el tamaño de las particiones se determina de manera arbitraria y pueden ser demasiado pequeñas o demasiado grandes para los trabajos que llegan.

Hay consecuencias importantes si los tamaños de las particiones son demasiado pequeños; los trabajos más grandes son rechazados si son demasiado extensos para caber en las particiones más grandes, o deben esperar si las particiones grandes están ocupadas. Como resultado, los trabajos más grandes pueden tener un tiempo de respuesta más largo a medida que esperan por particiones libres del tamaño suficiente e incluso es posible que no se ejecuten jamás.

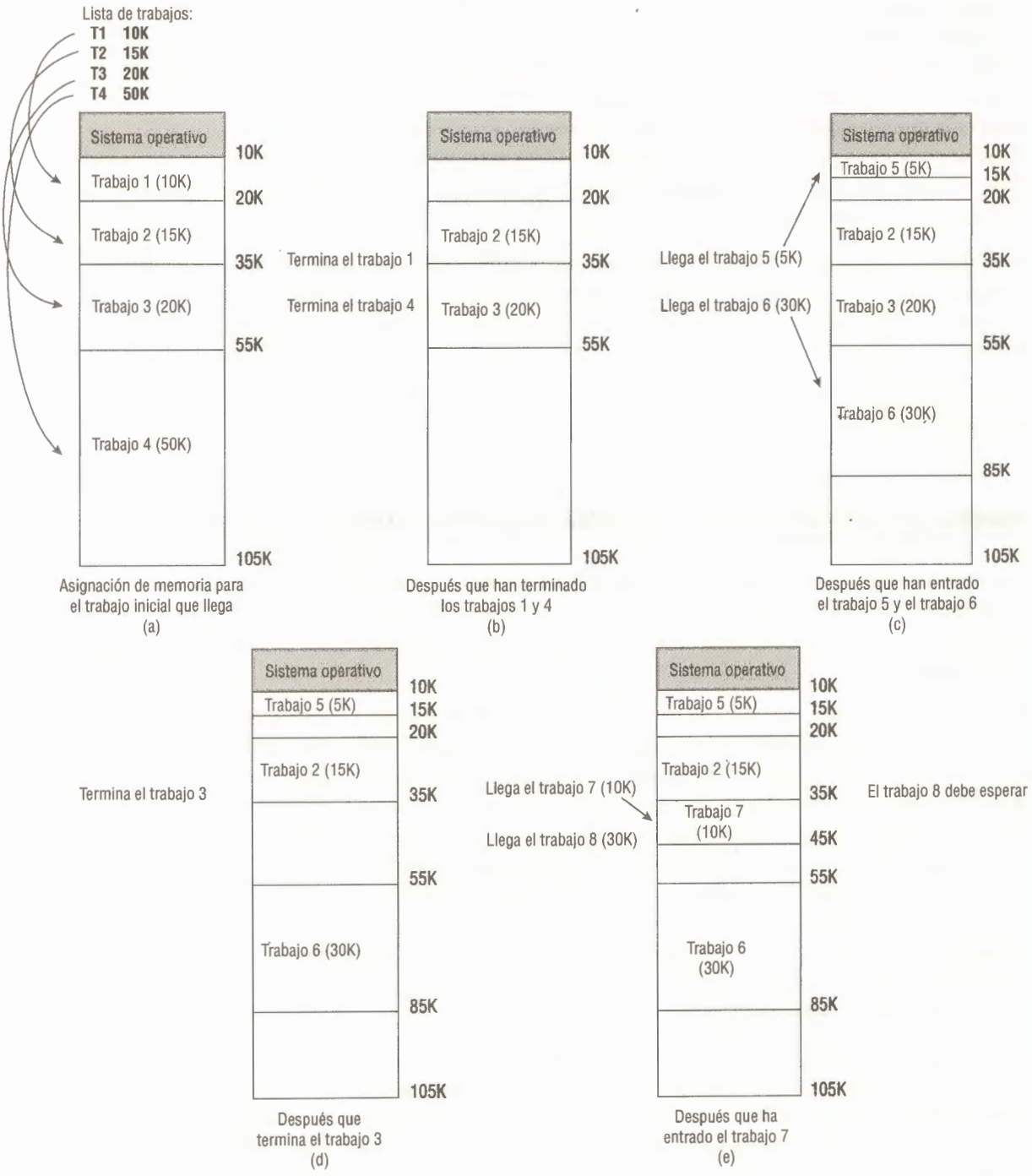
Por otra parte, si los tamaños de las particiones son demasiado grandes, se desperdicia memoria. Si un trabajo no ocupa toda la partición, la memoria sin usar de la partición permanece ociosa; no es posible darla a otro trabajo porque cada partición se asigna a un solo trabajo a la vez. Es una unidad indivisible. La figura 2.3 muestra una circunstancia así.

Hay dos tipos de fragmentación: interna y externa. El tipo depende de la ubicación del espacio desperdiciado.

Este fenómeno de uso parcial de particiones fijas y la creación coincidente de espacios sin usar dentro de la partición se denomina **fragmentación interna**, y es una desventaja fundamental del esquema de asignación de memoria por particiones fijas.

Particiones dinámicas

Con las **particiones dinámicas**, la memoria contigua sigue manteniéndose en bloques contiguos pero a los trabajos sólo se les proporciona tanta memoria como lo solicitan cuando se cargan para su procesamiento. Aunque esta mejora es importante con respecto a las particiones fijas porque dentro de las particiones no se desperdicia memoria, no elimina por completo el problema.



(Figura 2.4)

Uso de la memoria principal durante la asignación por particiones dinámicas. Cinco primeros incisos (a-e) de memoria principal cuando ocho trabajos se presentan para su procesamiento y se les asigna espacio con base en el "primer llegado, primer servido". El trabajo 8 debe esperar a e) aún cuando entre las particiones hay suficiente memoria libre para acomodarlo.

Como se muestra en la figura 2.4, un esquema de particiones dinámicas usa por completo la memoria cuando se cargan los primeros trabajos. Pero a medida que al sistema entran nuevos trabajos que no son del mismo tamaño que aquellos que acaban de abandonar la memoria, se colocan en los espacios disponibles siguiendo un criterio de prioridad. La figura 2.2 muestra la prioridad **primero en llegar, primero en servirse**. En consecuencia, la asignación subsecuente de memoria crea fragmentos de memoria libre entre bloques de memoria asignada. Este problema se denomina **fragmentación externa** y, como la fragmentación interna, deja que se desperdicie memoria.

En el inciso *e* de la figura 2.4 hay tres particiones de 5K, 10K y 20K (35K en total), suficiente para acomodar el trabajo 8, que sólo requiere 30K. Sin embargo, no son contiguos y, debido a que los trabajos se cargan de manera contigua, este esquema obliga a esperar al trabajo 8.

Antes de pasar al siguiente esquema de asignación, analizaremos la manera en que el sistema operativo sigue la pista de las secciones libres de la memoria.

Asignación del mejor ajuste contra el primer ajuste

Para los dos esquemas de asignación de memoria: fijo y dinámico, el sistema operativo debe mantener listas de cada ubicación de la memoria observando cuáles están libres y cuáles ocupadas. Luego, a medida que llegan nuevos trabajos al sistema es posible asignar las particiones libres.

Estas particiones pueden asignarse con base en la **asignación de memoria del primer ajuste** (primera partición que cumple con los requerimientos) o en la **asignación de memoria del mejor ajuste** (el menor espacio desperdiciado, la menor partición que cumple con los requisitos). Para ambos esquemas, el administrador de la memoria organiza las listas de memoria de las particiones libres y usadas (libre/ocupada), ya sea por tamaño o ubicación. El método de asignación del mejor ajuste mantiene las listas libre/ocupada en orden de tamaño, de la menor a la mayor. El método del primer ajuste mantiene las listas libre/ocupada organizadas según las ubicaciones en la memoria, de la de orden bajo a la de orden alto. Cada forma de asignación de memoria tiene sus ventajas, dependiendo de las necesidades del esquema de asignación particular: el mejor ajuste suele hacer mejor uso del espacio de la memoria; el del primer ajuste es más rápido para hacer la asignación.

Para comprender lo anterior, imagine que usted ha decidido que su colección de libros funcione como biblioteca pública con servicio de préstamo. Suponga que tiene libros de todas las formas y tamaños, e imagine también que hay un flujo continuo de personas que toman libros y luego los devuelven: siempre hay alguien esperando. Resulta evidente que usted siempre estará ocupado, lo cual es bueno, aunque nunca tenga tiempo para reorganizar los libreros.

Usted necesita un sistema. Sus libreros tienen posiciones fijas, con algunos espacios altos para colocar los libros de mayor tamaño, varios espacios para libros de bolsillo y mucho espacio para libros de texto. Usted requiere tener control de cuáles espacios en los libreros están llenos y dónde hay espacio para colocar más libros. Para efectos de nuestro ejemplo,

Si usted optimiza la velocidad, tal vez desperdicie espacio. Pero si optimiza el espacio, puede ser más lento.

consideraremos dos listas: una lista libre que muestra todos los espacios disponibles y una lista ocupada que muestra todos los espacios ocupados. En cada lista se incluye el tamaño y la ubicación de cada espacio.

Entonces, cuando cada libro se retira de su sitio, usted actualiza ambas listas quitando el espacio de la lista ocupada y poniéndolo en la lista libre. Después, a medida que los libros se devuelven y colocan de vuelta en un librero, vuelve a actualizar las dos listas.

Hay dos formas para organizar las listas: por tamaño o por ubicación. Si están organizadas por tamaño, los espacios para los libros más pequeños están en la parte superior de la lista y los espacios para los libros más grandes están en la parte inferior. Cuando están organizadas por ubicación, los espacios más próximos a su escritorio de préstamos están en la parte superior de la lista y las áreas más alejadas están en la parte inferior. ¿Cuál opción es la mejor? Depende de lo que usted quiera optimizar: espacio o velocidad de asignación.

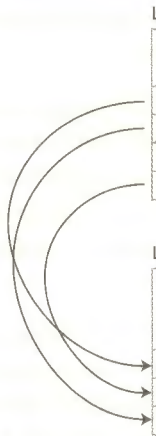
Si las listas están organizadas por tamaño, usted está optimizando el espacio en los libreros: a medida que llegan los libros, puede colocarlos de vuelta en los espacios en donde caben mejor. Este es un esquema del mejor ajuste. Si se devuelve un libro de bolsillo, usted lo coloca en un librero con los otros libros de bolsillo, o por lo menos con otros libros pequeños. En forma semejante, los libros de mayor tamaño se colocan con otros libros grandes. Sus listas facilitan la localización del espacio libre más pequeño disponible en donde puede caber el libro. La desventaja de este sistema consiste en que usted pierde tiempo buscando el mejor espacio. Las otras personas tienen que esperar a que usted coloque cada libro en su sitio, de modo que usted no puede atender a ninguna otra persona como podía hacerlo con el otro tipo de lista.

En el segundo caso, en una lista organizada por ubicación en el librero, usted optimiza el tiempo necesario para colocar los libros de vuelta en el librero. Este es un esquema del primer ajuste. Este sistema ignora el tamaño del libro que usted está intentando tomar de su librero. Si llega el mismo libro de bolsillo, puede encontrarle rápidamente un espacio vacío. De hecho, cualquier espacio vacío cercano es suficiente si es bastante grande: incluso puede usar un librero para enciclopedias si está cerca de su escritorio porque usted está optimizando el tiempo necesario para volver a colocar los libros en su librero.

Por supuesto, este es un método rápido para volver a poner los libros en su librero, y si la velocidad es importante, entonces ésta es la mejor de las dos alternativas. Sin embargo, no es una buena opción si el espacio en los libreros está limitado o si se han devuelto muchos libros grandes, ya que éstos deben esperar que haya espacios grandes. Si todos los espacios grandes están ocupados por libros pequeños, las personas que devuelven libros grandes deben esperar hasta que haya disponible un espacio idóneo. (Al final, usted necesitará tiempo para reorganizar los libros y compactar su colección.)

La figura 2.5 muestra cómo un trabajo grande puede tener problemas con una lista de asignación de memoria del primer ajuste. Los trabajos 1, 2 y 4 pueden entrar al sistema y empezar su ejecución; el trabajo 3 tiene que esperar aún cuando, si todos los fragmentos de la memoria se sumaran entre sí, hubiera espacio más que suficiente para acomodarlo. El primer ajuste ofrece una asignación rápida, aunque no siempre es eficiente.

Por otra parte, la misma lista de trabajos usando un esquema del mejor ajuste podía usar la memoria de manera más eficiente, como se muestra en la figura 2.6. En este caso particular, un esquema del mejor ajuste produciría una utilización superior de la memoria.



Lista de trabajos:

Número de trabajo	Memoria solicitada
J1	10K
J2	20K
J3	30K*
J4	10K

Lista de memoria:

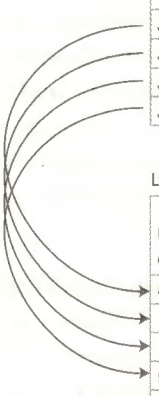
Ubicación en la memoria	Tamaño del bloque de memoria	Número de trabajo	Tamaño del trabajo	Estado	Fragmentación interna
10240	30K	J1	10K	Ocupada	20K
40960	15K	J4	10K	Ocupada	5K
56320	50K	J2	20K	Ocupada	30K
107520	20K			Libre	
Total disponible:	115K	Total usado:	40K		

(Figura 2.5)

Al usar un esquema del primer ajuste, el trabajo 1 reclama el primer espacio disponible. Después, el trabajo 2 reclama la primera partición suficientemente grande para acomodarlo, pero al hacer esto se necesita que el último bloque suficientemente grande acomode al trabajo 3. En consecuencia, el trabajo 3 (indicado por el asterisco) debe esperar hasta que haya disponible un bloque grande, aún cuando hay 75K de espacio de memoria sin usar (fragmentación interna). Observe que la lista de memoria está ordenada según la asignación de la memoria.

Se ha incrementado el uso de la memoria pero el proceso de asignación de la memoria requiere más tiempo. Aún más, aunque se ha disminuido la fragmentación interna, no se ha eliminado por completo.

El algoritmo del primer ajuste supone que el administrador de la memoria mantiene dos listas: una para los bloques de memoria libre y una para los bloques de memoria ocupada. La operación consta de un solo ciclo que compara el tamaño de cada trabajo con el tamaño de



Lista de trabajos:

Número de trabajo	Memoria solicitada
J1	10K
J2	20K
J3	30K
J4	10K

Lista de memoria

Ubicación en la memoria	Tamaño del bloque de memoria	Número de trabajo	Tamaño del trabajo	Estado	Fragmentación interna
40960	15K	J1	10K	Ocupada	5K
107520	20K	J2	20K	Ocupada	Ninguna
10240	30K	J3	30K	Ocupada	Ninguna
56320	50K	J4	10K	Ocupada	40K
Total disponible:	115K	Total usado:	70K		

(Figura 2.6)

Esquema del mejor ajuste. El trabajo 1 se asigna a la partición libre más próxima en que cabe, como los trabajos 2 y 3. El trabajo 4 se asigna a la única partición disponible, aunque no es a lo que se ajusto mejor. En este esquema, los cuatro trabajos se atienden sin que esperen. Observe que lo lista de memoria está ordenada según el tamaño de memoria. Este esquema usa de manera más eficiente la memoria, pero su implementación es más lenta.

cada bloque de memoria hasta que encuentra un bloque suficientemente grande para acomodar el trabajo. Luego, el trabajo se almacena en ese bloque de memoria y el administrador de la memoria mueve el ciclo para buscar el siguiente trabajo en la cola de entrada. Si se busca en toda la lista infructuosamente, entonces el trabajo se coloca en una cola de espera. A continuación, el administrador de la memoria busca el siguiente trabajo y repite el proceso.

Los algoritmos del mejor ajuste y del primer ajuste son muy diferentes. A continuación se muestra la manera de implementar el algoritmo del primer ajuste:

Algoritmo del primer ajuste

```
1 Poner el contador en 1
2 Mientras contador <= número de bloques en la memoria
    Si job_size > memory_size(contador)
       Entonces contador = contador + 1
    En otro caso
       cargar el trabajo en memory_size(contador)
       ajustar las listas de memoria libre/ocupada
       ir al paso 4
  del ciclo
3 Poner el trabajo en la cola de espera
4 Ir a buscar el siguiente trabajo
```

En la tabla 2.2, al administrador, de la memoria acaba de dársele una solicitud por un bloque con 200 espacios. (Los espacios pueden ser palabras, bytes o cualquier otra unidad que maneje el sistema.) Al usar el algoritmo del primer ajuste y comenzar desde la parte superior de la lista, el administrador de la memoria localiza el primer bloque de memoria suficientemente grande para acomodar el trabajo, que se encuentra en la ubicación 6785. Después, el

(Tabla 2.2)

En esta tabla se muestra el estado de cada bloque de memoria antes y después que se hace una solicitud usando el algoritmo del primer ajuste. (Nota: todos los volores están en notación decimal, a menos que se indique lo contrario.)

Antes de la solicitud		Después de la solicitud	
Dirección inicial	**Tamaño del bloque de memoria**	**Dirección inicial**	**Tamaño del bloque de memoria**
4075	105	4075	105
5225	5	5225	5
6785	600	*6985	400
7560	20	7560	20
7600	205	7600	205
10250	4050	10250	4050
15125	230	15125	230
24500	1000	24500	1000

trabajo se carga, empezando en la ubicación 6785 y ocupando los 200 espacios siguientes. El siguiente paso consiste en ajustar la lista libre para indicar que el bloque de memoria libre ahora empieza en la ubicación 6985 (no en la ubicación 6785 como antes) y que contiene sólo 400 espacios (no 600 como antes).

El algoritmo del mejor ajuste es ligeramente más complicado porque el objetivo consiste en encontrar el bloque de memoria más pequeño en que quepa el trabajo:

Algoritmo del mejor ajuste

```
1 Iniciar memory_block(0) = 99999
2 Calcular inicial_memory_waste = memory_block(0) – job_size
3 Iniciar subíndice = 0
4 Poner el contador en 1
5 Mientras contador <= número de bloques en la memoria
    Si job_size > memory_size(contador)
        Entonces contador = contador + 1
    En otro caso
        memory_waste = memory_size(contador) – job_size
  Si initial_memory_waste > memory_waste
        Entonces subíndice = contador
        initial_memory_waste = memory_waste
        contador = contador + 1
  Del ciclo
6 Si subíndice = 0
  Entonces poner el trabajo en una cola de espera
  En otro caso
        cargar el trabajo en memory_size(subíndice)
        ajustar las listas de la memoria libre/ocupada
7 Ir a buscar el siguiente trabajo
```

Uno de los problemas con el algoritmo del mejor ajuste es que es necesario buscar en toda la tabla antes que sea posible hacer una asignación, porque los bloques de memoria se almacenan físicamente en secuencia según su ubicación en la memoria (y no por tamaño de los bloques de memoria, como se muestra en la figura 2.6). El sistema puede ejecutar un algoritmo a fin de volver a ordenar de manera continua la lista en orden ascendente por tamaño de los bloques de memoria, aunque esto agrega más procesamiento extra y puede no ser un uso eficiente del tiempo de procesamiento a largo plazo.

El algoritmo del mejor ajuste se ilustra mostrando sólo la lista de los bloques de memoria libre. La tabla 2.3 muestra la lista libre antes y después que el bloque del mejor ajuste se ha asignado a la misma solicitud presentada en la tabla 2.2.

(Tabla 2.3)

En esta tabla se muestra el estado de cada bloque de memoria antes y después que se hace una solicitud usando el algoritmo del mejor ajuste.

Antes de la solicitud		Después de la solicitud	
Dirección inicial	**Tamaño del bloque de memoria**	**Dirección inicial**	**Tamaño del bloque de memoria**
4075	105	4075	105
5225	5	5225	5
6785	600	6785	600
7560	20	7560	20
7600	205	*7800	5
10250	4050	10250	4050
15125	230	15125	230
24500	1000	24500	1000

En la tabla 2.3, al administrador, de la memoria acaba de suministrársele una solicitud por un bloque de 200 espacios. Al usar el algoritmo del mejor ajuste y empezando en la parte superior de la lista, el administrador de la memoria busca en toda la lista y ubica un bloque de memoria empezando en la ubicación 7600, que es el menor bloque suficientemente grande para acomodar el trabajo. La elección de este bloque minimiza el espacio desperdiciado (sólo se desaprovechan 5 espacios, lo cual es menor que en los cuatro bloques alternativos. Después se almacena el trabajo, empezando en la ubicación 7600 y ocupando los 200 espacios siguientes. Ahora la lista libre debe ajustarse para mostrar que el bloque de memoria libre empieza en la ubicación 7800 (no en 7600 como antes) y que sólo contiene 5 espacios (no 205 como antes).

¿Qué es mejor, el primer ajuste o el mejor ajuste? Durante muchos años no hubo respuesta a esta pregunta general porque el rendimiento depende de la mezcla de trabajos. Observe que mientras el mejor ajuste dio por resultado un mejor ajuste, también dio por resultado (aunque no es el caso general) un menor espacio libre (5 espacios), lo cual se denomina astilla.

En los ejercicios al final del capítulo se exploran otros dos esquemas de asignación hipotéticos: el siguiente ajuste, que empieza por buscar desde el último bloque asignado el siguiente bloque disponible cuando llega un nuevo trabajo; y el peor ajuste, que asigna el bloque disponible más grande al nuevo trabajo. El peor ajuste es lo opuesto al mejor ajuste. Aunque constituye una manera aceptable para explorar la teoría de asignación de la memoria, puede no ser la mejor elección para un sistema real.

En años recientes los tiempos de acceso se han vuelto tan rápidos que el esquema que ahorra el recurso más valioso, el espacio de memoria, puede ser el mejor en algunos casos. La investigación continúa enfocándose en la determinación del esquema de asignación óptimo. Esto incluye el tamaño de página óptimo: un esquema de asignación fijo que abordaremos en el siguiente capítulo, que constituye la clave para mejorar el rendimiento del esquema de asignación del mejor ajuste.

Desasignación

Hasta ahora sólo hemos considerado el problema de cómo se asignan los bloques de memoria, pero finalmente llega el momento en que es necesario liberar, o **desasignar**, el espacio de la memoria.

Siempre que se libera memoria, se crea una oportunidad para fragmentación externa.

Para un sistema de particiones fijas, el proceso es bastante directo. Una vez que se completa el trabajo, el administrador de la memoria restablece el estado del bloque de memoria donde estaba almacenado el trabajo para "liberarlo". Cualquier código (por ejemplo, valores binarios con 0 para indicar libre y 1 para indicar ocupado) puede usarse de modo que la tarea mecánica de liberación de un bloque de memoria es relativamente simple.

Un sistema de particiones dinámicas usa un algoritmo más complicado porque éste intenta combinar áreas libres de memoria siempre que sea posible. En consecuencia, el sistema debe estar preparado para tres situaciones alternas:

- Caso 1. Cuando el bloque a liberar es adyacente a otro bloque libre.
- Caso 2. Cuando el bloque a liberar está entre dos bloques libres.
- Caso 3. Cuando el bloque a liberar está aislado de otros bloques libres.

El algoritmo de desasignación debe prepararse para los tres casos con un conjunto de condiciones anidadas. El siguiente algoritmo se basa en el hecho de que las ubicaciones en la memoria se enlistan usando un esquema de direcciones de la más baja a la más alta. Puede ser necesario modificar el algoritmo para acomodar una organización diferente de ubicaciones en la memoria. En este algoritmo, *job_size* es la cantidad de memoria que se libera al terminar el trabajo en cuestión, y *beginning_address* es la ubicación de la primera instrucción para el trabajo.

Algoritmo para desasignar bloques de memoria

```
Si job_location es adyacente a uno o más bloques libres
  Entonces
  Si job_location está entre dos bloques libres
      Entonces fusionar los tres bloques en uno
      memory_size(contador–1) = memory_size(contador–1) + job_size
      + memory_size(contador+1)
      poner el estado de memory_size(contador+1) en la entrada nula
  En otro caso
    fusionar ambos bloques en uno
    memory_size(contador)+1 = memory_size(contador)+1 + job_size
En otro caso
  buscar la entrada nula en la lista de memoria libre
  introducir job_size y beginning_address en la ranura de entrada
  poner su estado en "libre"
```

Caso 1: Unión de dos bloques libres

La tabla 2.4 muestra cómo ocurre la liberación en un sistema de asignación dinámica de memoria cuando el trabajo por liberar es el siguiente en un bloque de memoria libre.

(Tabla 2.4)
Esta es la lista libre original antes de la desasignación del caso 1. El asterisco indica el bloque de memoria libre que es adyacente al bloque de memoria próximo a ser liberado.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
7560	20	Libre
(7600)	(200)	(Ocupado)[1]
*7800	5	Libre
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

[1]Aunque los números entre paréntesis no aparecen en la lista libre, se han insertado aquí para efectos de claridad. El tamaño del trabajo es 200 y su ubicación inicial es 7600.

Después de la desasignación, la lista libre se ve como la que se muestra en la tabla 2.5.

(Tabla 2.5)
Caso 1. Esta es la lista libre después de la desasignación. El asterisco indica la ubicación donde se realizaron cambios para liberar bloques de memoria.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
7560	20	Libre
*7600	205	Libre
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

Al usar el algoritmo de desasignación, el sistema observa que la memoria por liberar está próxima a un bloque de memoria libre, que empieza en la ubicación 7800. En consecuencia, es necesario modificar la lista para reflejar la dirección inicial del nuevo bloque libre, 7600, que era la dirección de la primera instrucción del trabajo que acaba de liberar este bloque. Además, el tamaño del bloque de memoria para este nuevo espacio debe modificarse para mostrar su nuevo tamaño, que es el total combinado de las dos particiones libres (200 + 5).

Caso 2: Unión de tres bloques libres

Cuando el espacio de memoria liberado está entre dos bloques de memoria libre, el proceso es semejante, como se muestra en la tabla 2.6.

Al usar el algoritmo de desasignación, el sistema se percata que la memoria por liberar está entre dos bloques de memoria libre. En consecuencia, los tamaños de las tres particiones libres (20 + 20 + 205) deben combinarse y el total almacenarse con la dirección inicial más pequeña, 7560.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
*7560	20	Libre
(7580)	(20)	(Ocupado)[1]
*7600	205	Libre
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

[1]Aunque los números entre paréntesis no aparecen en la lista libre, se han insertado aquí para efectos de claridad.

(Tabla 2.6)

Caso 2. Esta es la lista libre original antes de la desasignación. Los asteriscos indican los dos bloques de memoria libre que son adyacentes al bloque de memoria próximo a ser liberado.

Debido a que la entrada en la ubicación 7600 se ha combinado con la entrada previa, es necesario vaciar ésta. Esto se efectúa al modificar el estado a **entrada nula**, sin dirección inicial ni tamaño de bloque de memoria, como se indica con el asterisco en la tabla 2.7. Esto refuta la necesidad de reordenar la lista a expensas de la memoria.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
7560	245	Libre
*		(entrada nula)
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

(Tabla 2.7)

Caso 2. La lista libre original después que un trabajo ha liberado memoria.

Caso 3: Desasignación de un bloque aislado

La tercera alternativa es cuando el espacio por liberar está aislado de todas las demás áreas libres.

Para este ejemplo es necesario conocer más sobre la forma en que está configurada la lista de memoria. Para simplificar las cosas, observe la lista ocupada para el área de memoria entre las ubicaciones 7560 y 10250. Recuerde que, empezando en 7560, hay un bloque de memoria libre de 245, de modo que el área ocupada incluye todo desde la ubicación 7805 (7560 + 245) hasta 10250, que es la dirección del siguiente bloque libre. La lista libre y la lista ocupada se muestran en la tabla 2.8 y en la tabla 2.9.

(Tabla 2.8)

Caso 3. Lista libre original antes de la desasignación. El bloque de memoria próximo a ser liberado no es adyacente a ningún bloque que ya está libre.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
7560	245	Libre
		(entrada nula)
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

(Tabla 2.9)

Caso 3. Lista de memoria ocupada antes de la desasignación. El trabajo a liberar es de tamaño 445 y empieza en la ubicación 8805. El asterisco indica el bloque de memoria próximo a ser liberado.

Dirección inicial	Tamaño del bloque de memoria	Estado
7805	1000	Ocupado
*8805	445	Ocupado
9250	1000	Ocupado

Al usar el algoritmo de desasignación, el sistema se da cuenta que el bloque de memoria por ser liberado no es adyacente a ningún bloque de memoria libre; en lugar de eso, está entre dos áreas ocupadas. En consecuencia, el sistema debe buscar en la tabla una entrada nula.

El esquema presentado en este ejemplo crea entradas nulas tanto en la lista ocupada como en la lista libre durante el proceso de asignación o desasignación de memoria. Un ejemplo de una entrada nula que ocurre como resultado de la liberación se presentó en el caso 2. Una entrada nula en la lista ocupada ocurre cuando un bloque de memoria entre otros dos bloques de memoria ocupados se regresa a la lista libre, como se muestra en la tabla 2.10. Este mecanismo asegura que todos los bloques se introduzcan en las listas según la dirección inicial de su ubicación en la memoria, desde la más pequeña hasta la más grande.

Dirección inicial	Tamaño del bloque de memoria	Estado
7805	1000	Ocupado
*		(entrada nula)
9250	1000	Ocupado

(Tabla 2.10)
Caso 3. Esta es la lista ocupada después que el trabajo ha liberado su memoria. El asterisco indica la nueva entrada nula en la lista ocupada.

Una vez que se encuentra la lista ocupada, la ubicación de la memoria inicial del trabajo que termina se introduce en la columna de la dirección inicial, el tamaño del trabajo se introduce en la columna del tamaño del bloque de memoria y el estado se cambia de una entrada nula a libre para indicar que un nuevo bloque de memoria está disponible, como se muestra en la tabla 2.11.

Dirección inicial	Tamaño del bloque de memoria	Estado
4075	105	Libre
5225	5	Libre
6785	600	Libre
7560	245	Libre
*8805	445	Libre
10250	4050	Libre
15125	230	Libre
24500	1000	Libre

(Tabla 2.11)
Caso 3. Esta es la lista libre después que el trabajo ha liberado su memoria. El asterisco indica la entrada del nuevo bloque libre que sustituye a la entrada nula.

Particiones dinámicas reubicables

Los dos esquemas de asignación de memoria –el fijo y el dinámico– descritos hasta el momento compartían algunas características de fragmentación inaceptables que era necesario resolver antes que el número de trabajos en espera de ser aceptados fuera aparatoso. Además, había una necesidad creciente por usar las astillas de la memoria que quedan a menudo.

La solución de ambos problemas consistía en el desarrollo de particiones **dinámicas reubicables**. Con este esquema de asignación de memoria, el administrador de la memoria reubica programas para reunir todos los bloques de memoria y compactarlos a fin de obtener un bloque de memoria suficientemente grande para acomodar algunos o todos los trabajos que están esperando entrar.

La **compactación** de la memoria, algunas veces denominada recolección de basura o desfragmentación, la lleva a cabo el sistema operativo para solicitar secciones fragmentadas del espacio de la memoria. ¿Recuerda el primer ejemplo de la biblioteca improvisada con servicio de préstamo de libros? Si usted deja de prestar libros durante unos momentos y reorganiza los libros en el orden más conveniente, entonces está compactando su colección.

Cuando se usa una utilidad de desfragmentación, la memoria se compacta y se reubican segmentos de archivo de modo que es posible recuperarlos más rápido.

Pero esto demuestra su desventaja: se trata de un proceso de procesamiento extra, de modo que mientras se lleva a cabo la compactación todo lo demás debe esperar.

La compactación no es una tarea fácil. Primero, todos los programas en la memoria deben reubicarse para hacerlos contiguos, después cada dirección y toda referencia a una dirección dentro de cada programa debe ajustarse para tomar en cuenta la nueva ubicación del programa en la memoria. Sin embargo, todos los otros valores dentro del programa (como los datos) deben dejarse tal cual. En otras palabras, el sistema operativo debe distinguir entre direcciones y valores de datos, y las distinciones no son evidentes una vez que el programa se ha cargado en la memoria.

Para apreciar la complejidad de la **reubicación**, considere un programa típico. Recuerde que todos los números se almacenan en la memoria como valores binarios, y en cualquier instrucción dada del programa no es extraño encontrar direcciones, así como valores de datos. Por ejemplo, un programa en lenguaje ensamblador puede incluir la instrucción "sumar el entero 1 a I". La instrucción del código fuente es como sigue:

```
ADDI I, 1
```

Sin embargo, después que se ha traducido a código real, podría verse como sigue (para efectos de legibilidad, los valores están representados aquí en código octal, no en código binario):

```
000007 271 01 0 00 000001
```

No resulta evidente de inmediato cuáles elementos son direcciones y cuáles son códigos de instrucciones o valores de datos. De hecho, la dirección es el número a la izquierda (000007). El código de instrucción está a continuación (271) y el valor del dato está a la derecha (000001).

El sistema operativo puede decir la función de cada grupo de dígitos por su ubicación en la línea y el código de operación. No obstante, si el programa debe moverse a otro sitio en la memoria, es necesario identificar, o marcar, cada dirección. Así, más tarde la cantidad de ubicaciones en la memoria por las que se ha desplazado el programa debe sumarse a (o restarse de) las direcciones originales en el programa.

Esto se vuelve especialmente importante cuando el programa incluye secuencias de ciclos, de decisiones y de ramificaciones; así como referencias a valores de datos. Si, por casualidad, cada dirección no se ajustó por el mismo valor, el programa podría bifurcarse, o podría referirse a datos equivocados.

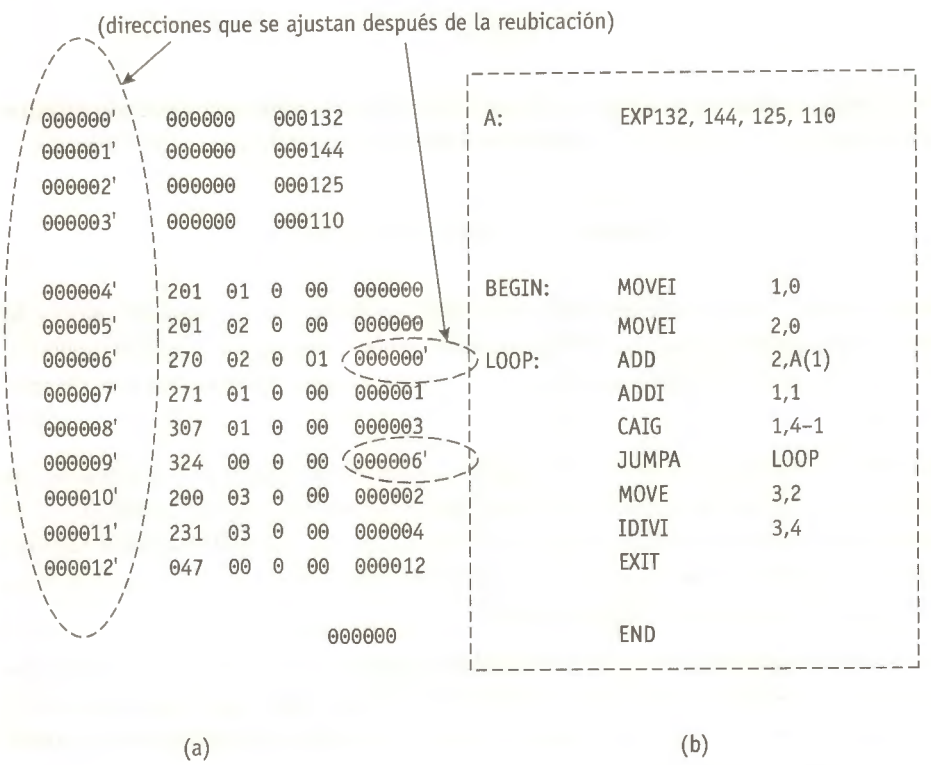
El programa en la figura 2.7 y en la figura 2.8 muestra la manera en que el sistema operativo marca las direcciones de modo que puedan ajustarse siempre y cuando un programa sea reubicado.

Internamente, las direcciones se marcan con un símbolo especial (indicado en la figura 2.8 por apóstrofos), de modo que el administrador de memoria sabrá ajustarlos por el valor

```
A       EXP 132, 144, 125, 110      ;the data values
BEGIN:  MOVEI           1,0         ;initialize register 1
        MOVEI           2,0         ;initialize register 2
LOOP:   ADD             2,A(1)      ;add (A + reg 1) to reg 2
        ADDI            1,1         ;add 1 to reg 1
        CAIG            1,4-1       ;is register 1 > 4-1?
        JUMPA           LOOP        ;if not, go to Loop
        MOVE            3,2         ;if so, move reg 2 to reg 3
        IDIVI           3,4         ;divide reg 3 by 4,
                                    ;remainder to register 4
        EXIT                        ;end
        END
```

(Figura 2.7)

Un programa en lenguaje ensamblador que efectúa una sola operación progresiva. Esto es lo que los programadores presentan al ensamblador. Los comandos se muestran a la izquierda y los comentarios que explican cada comando se muestran a la derecha después de los puntos y coma.

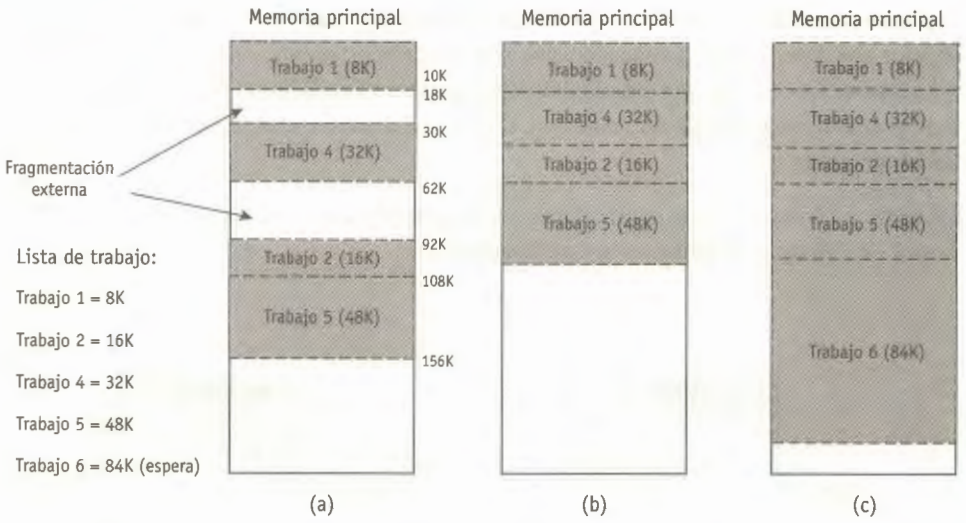


(Figura 2.8)

El programa original en lenguaje ensamblador después de ser procesado por el ensamblador mostrado a la derecha en (a). Para ejecutar el programa, el ensamblador lo traduce a código de máquina legible (que se muestra a la izquierda) con todas las direcciones marcadas con un símbolo especial (mostrado aquí con un apóstrofo) para distinguir las direcciones de los valores de datos. Todas las direcciones (y no los valores) deben ajustarse después de la reubicación.

almacenado en el registro de reubicación. Ningún otro valor (valores de datos) está marcado y no se modifica después de la reubicación. Otros números en el programa, los que indican instrucciones, registros o constantes usadas en la instrucción, también se dejan solos.

La figura 2.9 ilustra lo que ocurre a un programa en la memoria durante la compactación y reubicación.



(Figura 2.9)

Tres representaciones de la memoria antes y después de la compactación con el sistema operativo ocupando los primeros 10K de memoria. Cuando el trabajo 6 llega requiriendo 84K, la disposición inicial de la memoria en (a) muestra fragmentación externa totalizando 96K de espacio. Inmediatamente después de la compactación (b); la fragmentación externa ha sido eliminada, dejando espacio para el trabajo 6, el que después de descargado se muestra en (c).

Este análisis sobre compactación plantea tres preguntas:

1. ¿Qué ocurre detrás del escenario cuando tienen lugar la reubicación y la compactación?
2. ¿Qué sigue la pista de cuán lejos se ha movido cada trabajo desde su área de almacenamiento original?
3. ¿Qué listas es necesario actualizar?

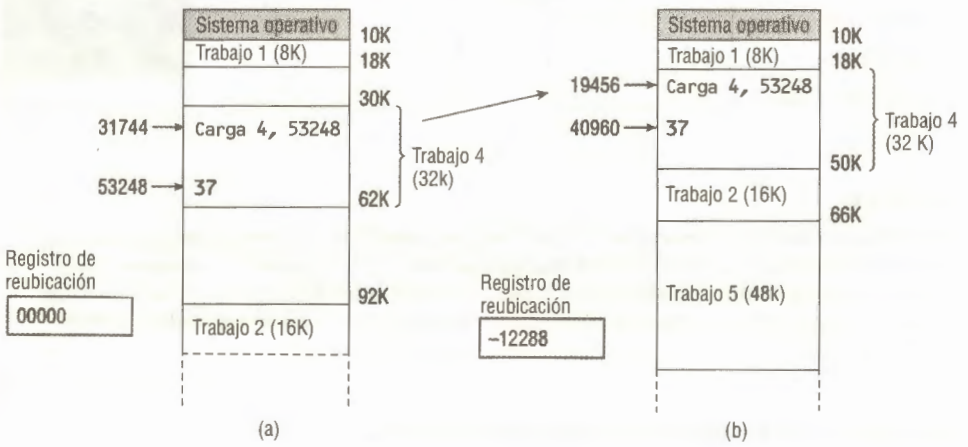
La última pregunta es la más fácil de contestar. Después de la reubicación y la compactación, se actualizan la lista libre y la lista ocupada. La lista libre se cambia a fin de mostrar la partición para el nuevo bloque de memoria libre: el formado como resultado de la compactación que será ubicado en la memoria empezando después de la última ubicación usada por el último trabajo. La lista ocupada se modifica a fin de mostrar las nuevas ubicaciones para todos los trabajos que ya están en proceso y que fueron reubicados. Cada trabajo cuenta con una nueva dirección, excepto aquellos que ya residían en las ubicaciones más bajas de la memoria.

Para responder las otras dos preguntas es necesario aprender algo más sobre los componentes del hardware de una computadora, especialmente los registros. Para ayudar con la reubicación se usan registros de propósito especial. En algunas computadoras, para llevar a cabo este objetivo se colocan juntos dos registros: el de límites y el de reubicación.

El **registro de límites** se usa para almacenar la ubicación más alta (o más baja, dependiendo del sistema específico) en la memoria accesible por todos los programas. Esto asegura que durante la ejecución, un programa no intente acceder a ubicaciones de la memoria que no

le pertenecen; es decir, las que están fuera de los límites. El **registro de reubicación** contiene el valor que debe sumarse a cada dirección referida en el programa, de modo que el sistema sea capaz de tener acceso a la dirección de la memoria correcta después de la reubicación. En caso de que el programa no sea reubicado, el valor almacenado en el registro de reubicación del programa es cero.

La figura 2.10 ilustra lo que ocurre durante el reacomodo usando el registro de reubicación (todos los valores se muestran en forma decimal).



(Figura 2.10)

Contenido del registro de reubicación y acercamiento del área de memoria del trabajo 4: a) antes de la reubicación y b) después de la reubicación y compactación.

Originalmente, el trabajo 4 se cargaba en la memoria empezando en la ubicación de la memoria 30K. (1K es igual a 1 024 bytes. En consecuencia, la dirección inicial exacta es 30 * 1 024 = 30 720). Requería un bloque de memoria de 32K (o 32 * 1 024 = 32 768) ubicaciones direccionables. En consecuencia, cuando el trabajo se cargaba originalmente, ocupaba el espacio de la ubicación de la memoria 30720 a la ubicación de la memoria 63488–1. Ahora suponga que en el programa, en la ubicación de la memoria 31744 hay una instrucción como ésta:

```
LOAD 4, ANSWER
```

Los comandos en lenguaje ensamblador piden que el dato conocido como ANSWER sea cargado en el registro 4 para su cálculo posterior. ANSWER, el valor 37, se almacena en la ubicación de la memoria 53248. (En este ejemplo el registro 4 es una anotación de trabajo/cálculo, que es distinto de ambos registros, el de reubicación y el de límites.)

Después de ser reubicado, el trabajo 4 se ha desplazado hacia una nueva dirección inicial de la memoria de 18K (en realidad, 18 * 1 024 = 18 432). Por supuesto, el trabajo sigue

teniendo 32K ubicaciones direccionables, de modo que ahora ocupa memoria desde la ubicación 18 432 hasta la ubicación 51 200 – 1 y, gracias al registro de reubicación, todas las direcciones se ajustan en consecuencia.

¿Qué contiene el registro de reubicación? En este ejemplo contiene el valor –12 288. Según se calculó antes, 12 288 es el tamaño del bloque libre que se ha movido hacia el extremo direccionable alto de la memoria. El signo es negativo porque el trabajo 4 se ha movido hacia atrás, más cerca del extremo direccionable bajo de la memoria, como se muestra en la parte superior de la figura 2.10*b*.

Sin embargo, la instrucción del programa (LOAD 4, ANSWER) no ha cambiado. La dirección original 53248, donde se ha almacenado ANSWER permanece igual en el programa, sin importar cuántas veces se reubique. Antes que se ejecute la instrucción, no obstante, es necesario calcular la dirección verdadera al sumar el valor almacenado en el registro de reubicación a la dirección encontrada en esa instrucción. Si las direcciones no se ajustan por el valor almacenado en el registro de reubicación, entonces aunque la ubicación 31 744 de la memoria sigue formando parte del conjunto de ubicaciones de la memoria del trabajo, no contiene el comando LOAD. No sólo eso, sino que ahora la ubicación 53 248 está fuera del límite. La instrucción que originalmente estaba en este programa se ha movido de regreso por 12K (12*1 024 = 12 288), que es el tamaño del bloque libre. En consecuencia, la ubicación 53 246 se ha desplazado por –12 288 y ANSWER, el dato con valor 37, ahora se encuentra en la dirección 40 960.

En efecto, al compactar y reubicar, el administrador de memoria optimiza el uso de la memoria, mejorando así su desempeño, la cual es una de las medidas del rendimiento del sistema. Un desafortunado efecto colateral es que se incurre en más procesamiento extra que con los dos esquemas de asignación de la memoria previos. El factor crucial aquí es la sincronización de la compactación; cuándo y cuán a menudo debe hacerse. Hay tres opciones.

Un enfoque es efectuarla cuando cierto porcentaje de la memoria se vuelve ocupado, por ejemplo 75%. La desventaja de este enfoque es que el sistema podría incurrir en procesamiento extra innecesario en caso que no haya trabajos esperando usar el 25% restante.

Un segundo enfoque consiste en compactar la memoria sólo cuando hay trabajos esperando entrar. Esto implicaría verificar constantemente la cola de entrada, lo cual podría resultar en procesamiento extra innecesario y en una disminución del procesamiento de los trabajos que ya están en el sistema.

Un tercer enfoque es efectuarla después que ha transcurrido un lapso prescrito. Si la cantidad de tiempo elegida es pequeña, entonces el sistema dedica más tiempo a la compactación que al procesamiento. Si es demasiado larga, entonces demasiados trabajos se congregan en la cola de espera y se pierden las ventajas de la compactación.

Como puede observar, cada opción tiene ventajas y desventajas. La mejor opción para cualquier sistema la decide el diseñador del sistema operativo quien, con base en la mezcla de trabajos y otros factores, intenta optimizar tanto el tiempo de procesamiento como el uso de la memoria mientras mantiene la sobrecarga, lo más baja posible.

Conclusión

En este capítulo se presentaron cuatro técnicas de administración de la memoria: sistemas de usuario único, particiones fijas, particiones dinámicas y particiones dinámicas reubicables. Presentan tres cosas en común: todas requieren que todo el programa: 1) sea cargado en la memoria; 2) se almacene de manera contigua, y 3) que permanezca en la memoria hasta que se complete el trabajo.

En consecuencia, cada una impone serias restricciones sobre el tamaño de los trabajos porque éstos sólo pueden ser tan grandes como las particiones más grandes en la memoria.

Estos esquemas fueron suficientes para las tres primeras generaciones de computadoras, que procesaban trabajos en el modo de lotes. El tiempo de respuesta se medía en horas, o algunas veces en días, pero se trataba de un periodo cuando los usuarios esperaban tales retrasos entre la presentación de sus trabajos y la obtención del resultado. Como veremos en el siguiente capítulo, una nueva tendencia surgió durante la tercera generación de computadoras a fines de la década de 1960 y principios de la de 1970. Los usuarios podían conectarse directamente con la unidad central de procesamiento a través de estaciones de ingreso de trabajos, al cargar sus trabajos desde terminales en línea que podían interactuar de manera más directa con el sistema. Para acomodarlos se requerían nuevos métodos de administración de la memoria.

Veremos que los esquemas de asignación de la memoria que surgieron después presentaban dos cosas nuevas en común: Primero, ya no era necesario almacenar los programas en ubicaciones contiguas de la memoria; podían dividirse en segmentos de tamaños variables o en páginas del mismo tamaño. Cada página, o segmento, podía almacenarse siempre que hubiera un bloque vacío suficientemente grande para darle cabida. Segundo, no todas las páginas, o segmentos, debían residir en la memoria durante la ejecución del trabajo. Estos avances fueron significativos para los diseñadores del sistema, los operadores y también para los usuarios.

Términos clave

Asignación de la memoria del mejor ajuste: Esquema de asignación de la memoria que considera todos los bloques libres y selecciona para asignación el que dé por resultado la menor cantidad de espacio desperdiciado.

Asignación de memoria del primer ajuste: Esquema de asignación de la memoria que busca desde el principio de la lista de bloques libres y selecciona para asignación el primer bloque de memoria suficientemente grande para satisfacer la solicitud.

Compactación: Proceso de recolección de fragmentos de espacio de memoria disponible en bloques contiguos al mover los programas y los datos en la memoria de una computadora o un disco. También se denomina *recolección de basura*.

Desasignación o liberación: Proceso de liberar un recurso asignado, ya sea espacio de memoria, un dispositivo, un archivo o CPU.

Dirección: Número que designa una ubicación particular de la memoria.

Entrada nula: Entrada vacía en una lista.

Fragmentación externa: Situación en que la asignación dinámica de memoria crea fragmentos no utilizables de memoria libre entre bloques de memoria ocupada o asignada.

Fragmentación interna: Situación en la que una partición fija sólo es usada parcialmente por el programa; el espacio restante dentro de la partición no está disponible para ningún otro trabajo, por lo que se desperdicia.

Kilobyte (K): Unidad de espacio de memoria o almacenamiento igual a 1 024 bytes o 2^{10} bytes.

Memoria principal: Unidad que trabaja directamente con la CPU y en la cual los datos y las instrucciones deben permanecer en el orden para ser procesadas. También se le conoce como *memoria de acceso aleatorio (RAM), almacenamiento primario o memoria interna.*

Particiones dinámicas: Esquema de asignación de memoria donde a los trabajos se les otorga tanta memoria como solicitan cuando se cargan para su procesamiento, creando así sus propias particiones en la memoria principal.

Particiones dinámicas reubicables: Esquema de asignación de la memoria donde el sistema reubica programas en la memoria para reunir todos los bloques vacíos y compactarlos a fin de obtener un bloque de memoria suficientemente grande para acomodar algunos o todos los trabajos que están en espera de memoria.

Particiones estáticas: Sinónimo de *particiones fijas*.

Particiones fijas: Esquema de asignación de memoria donde la memoria principal se divide en secciones, con porciones asignadas a cada trabajo.

Primero en llegar, primero en ser servido (FCFS): Política de planificación de procesos no preferencial que maneja trabajos según su momento de llegada; el primer trabajo en la lista READY es el primero que se procesa.

Registro de límites: Es el usado para almacenar la ubicación más alta en la memoria legalmente accesible por cada programa.

Registro de reubicación: Es aquél que contiene el valor que debe sumarse a cada dirección referida en un programa, de modo que pueda acceder a las direcciones correctas en la memoria después de la reubicación.

Reubicación: 1) Proceso de mover un programa de un área de la memoria a otra, o 2) Proceso de ajustar referencias a direcciones en un programa, ya sea por medio del software o hardware, para permitir que el programa se ejecute correctamente cuando se cargue en diferentes secciones de la memoria.

Búsquedas de interés

- Tecnología de núcleo de memoria
- technikum29 Museo de computación y tecnología de la comunicación
- Cómo trabaja una memoria RAM
- Algoritmo primero en llegar primero en ser servido
- Particiones estáticas contra particiones dinámicas
- Fragmentación interna contra fragmentación externa

Ejercicios

Temas de investigación

A. En informática se usan tres sistemas numéricos diferentes (además del conocido sistema decimal). Elabore una columna y numere los renglones con los enteros del 1 al 30. En las tres columnas siguientes, muestre cómo cada valor se representa usando los sistemas de numeración binario, octal y hexadecimal. Identifique cuándo y por qué se usa cada uno de los tres sistemas de numeración. Cite sus fuentes.

B. Para una plataforma de su elección, investigue el crecimiento en tamaño de la memoria principal (RAM) desde la época en que se desarrolló la plataforma hasta el presente. Elabore una gráfica que muestre hitos en el crecimiento de la memoria y la fecha aproximada. Escoja de entre microcomputadoras, computadoras de rango intermedio media y computadoras centrales. Asegúrese de mencionar la organización que realizó la investigación y desarrollo de la RAM y cite sus fuentes.

Ejercicios

1. Explique las diferencias fundamentales entre la fragmentación interna y la fragmentación externa. Para cada uno de los cuatro sistemas de administración de memoria explicados en este capítulo (usuario único, fijo, dinámico y dinámico reubicable) identifique una de las causas de cada tipo de fragmentación.
2. ¿Qué tipo de fragmentación es reducida por compactación? Explique su respuesta.
3. ¿Con qué frecuencia puede ejecutarse una reubicación? Explique su respuesta.
4. Imagine un sistema operativo que no ejecute una desasignación de memoria. Nombre al menos tres desafortunadas consecuencias que pueden resultar de esta situación y explique su respuesta.
5. Compare y contraste un sistema de partición fija y un sistema de partición dinámica.
6. Compare y contraste un sistema de partición dinámica y un sistema de partición dinámica reubicable.

7. Dada la siguiente información:

   Lista de trabajo:

Número de trabajo	Memoria solicitada	Bloque de memoria	Tamaño
T1	740K	Bloque 1	610K (memoria de orden bajo)
T2	500K	Bloque 2	850K
T3	700K	Bloque 3	700K (memoria de orden alto)

   a. Use el algoritmo del mejor ajuste para indicar qué bloques de memoria son asignados a los tres trabajos que llegan.

   b. Use el algoritmo del primer ajuste para indicar qué bloques de memoria son asignados a los tres trabajos que llegan.

8. Dada la siguiente información:

   Lista de trabajo:

Número de trabajo	Memoria solicitada	Bloque de memoria	Tamaño
T1	700K	Bloque 1	610K (memoria de orden bajo)
T2	500K	Bloque 2	850K
T3	740K	Bloque 3	700K (memoria de orden alto)

   a. Use el algoritmo del mejor ajuste para indicar qué bloques de memoria son asignados a los tres trabajos que llegan.

   b. Use el algoritmo del primer ajuste para indicar qué bloques de memoria son asignados a los tres trabajos que llegan.

9. El algoritmo del siguiente ajuste es un algoritmo de asignación que sigue la pista de la última partición asignada y comienza a buscar a partir de ese punto cuando llega un nuevo trabajo.

   a. ¿Existen algunas ventajas de este algoritmo? Si es así, ¿cuáles son?

   b. ¿Cómo se compararía con el mejor ajuste y el primer ajuste para las condiciones dadas en el ejercicio 7?

   c. ¿Cómo se compararía con el mejor ajuste y el primer ajuste para las condiciones dadas en el ejercicio 8?

10. El algoritmo del peor ajuste es un algoritmo de asignación que asigna el bloque libre más grande a un nuevo trabajo y es opuesto al algoritmo del mejor ajuste.

    a. ¿Existen ventajas en el algoritmo de peor ajuste? Si es así, ¿cuáles son?

    b. ¿Cómo compararía el mejor ajuste y el primer ajuste para las condiciones dadas en el ejercicio 7?

    c. ¿Cómo compararía el mejor ajuste y el primer ajuste para las condiciones dadas en el ejercicio 8?

Ejercicios avanzados

11. El ejemplo de reubicación presentado en este capítulo implica que la compactación se lleva a cabo por completo en la memoria, sin almacenamiento secundario. ¿Es posible fusionar todas las secciones libres de la memoria en un bloque contiguo usando este enfoque? ¿Por qué sí?¿Por qué no?

12. Para compactar la memoria en algunos sistemas, hay quien sugiere copiar todos los trabajos existentes en un dispositivo de almacenamiento secundario y luego volver a cargarlos (y reubicarlos) de manera contigua en la memoria principal, creando así un bloque libre una vez que se han copiado todos los trabajos en la memoria. ¿Es viable esto? ¿Podría usted idear una mejor forma para compactar la memoria? Escriba su algoritmo y explique por qué es mejor.

13. Dada la configuración de la memoria en la figura 2.11, conteste las siguientes preguntas. En este momento llega el trabajo 4 solicitando un bloque de 100K.

    a. ¿Es posible acomodar el trabajo 4? ¿Por qué sí?¿Por qué no?

    b. Si se usa reubicación, ¿cuál es el contenido de los registros de reubicación para los trabajos 1, 2 y 3 después de la compactación?

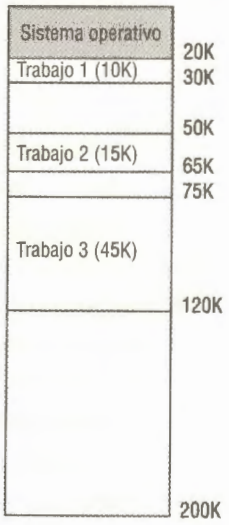
    c. ¿Cuál es el contenido del registro de reubicación para el trabajo 4 después que se ha cargado en la memoria?

    d. Una instrucción que es parte del trabajo 1, originalmente estaba cargada en la ubicación de la memoria 22K. ¿Cuál es la nueva ubicación después de la compactación?

    e. Una instrucción que es parte del trabajo 2, originalmente estaba cargada en la ubicación de la memoria 55K. ¿Cuál es la nueva ubicación después de la compactación?

    f. Una instrucción que es parte del trabajo 3, originalmente estaba cargada en la ubicación de la memoria 80K. ¿Cuál es la nueva ubicación después de la compactación?

    g. Si una instrucción originalmente estaba cargada en la ubicación de la memoria 110K, ¿cuál es su nueva ubicación después de la compactación?



(Figura 2.11) *Configuración de la memoria para el ejercicio 13.*

Ejercicios de programación

14. A continuación se muestra un proyecto de programación a largo plazo. Use la siguiente información para completar este ejercicio.

Lista de trabajo

Número del flujo de trabajo	Tiempo	Tamaño
1	5	5760
2	4	4190
3	8	3290
4	2	2030
5	2	2550
6	6	6990
7	8	8940
8	10	740
9	7	3930
10	6	6890
11	5	6580
12	8	3820
13	9	9140
14	10	420
15	10	220
16	7	7540
17	3	3210
18	1	1380
19	9	9850
20	3	3610
21	7	7540
22	2	2710
23	8	8390
24	5	5950
25	10	760

Lista de memoria

Bloque de memoria	Tamaño
1	9500
2	7000
3	4500
4	8500
5	3000
6	9000
7	1000
8	5500
9	1500
10	500

Durante la instalación de una gran computadora que procesa programas por lotes, la administración desea decidir qué estrategia de ubicación producirá el mejor rendimiento posible. La instalación ejecuta gran almacenamiento real en una computadora (en oposición al almacenamiento "virtual", que se cubrirá en el siguiente capítulo) bajo multiprogramación con particiones fijas. Cada programa de usuario se ejecuta en un solo grupo de ubicaciones de almacenamiento contiguo. Los usuarios plantean sus requisitos de almacenamiento y unidades de tiempo para usar la CPU en su tarjeta de control de trabajos (que solía –y aún lo hace– trabajar de esta manera, aunque actualmente puede que ya no se usen tarjetas). El sistema operativo asigna a cada usuario la partición idónea e inicia el trabajo del usuario. El trabajo permanece en la memoria hasta su terminación. Se dispone de un total de 50 000 ubicaciones de memoria, divididas en bloques como se indica en la tabla de la página anterior.

a. Escriba (o calcule) una simulación dirigida por eventos para poder decidir qué estrategia de almacenamiento usar en esta instalación. Su programa debe usar el flujo de trabajo y la manera de partición de la memoria según se indicó previamente. Ejecute el programa hasta que se hayan ejecutado todos los trabajos con la memoria así (en orden de dirección). Esto proporciona el rendimiento del tipo del primer ajuste.

b. Ordene las particiones de la memoria por tamaño y ejecute el programa por segunda ocasión; así obtendrá los resultados del rendimiento del mejor ajuste. Para ambos incisos, *a* y *b*, usted está investigando el rendimiento del sistema usando un flujo de trabajo típico al medir:

    1. El desempeño (cuántos trabajos se procesan por unidad de tiempo dada).
    2. La utilización del almacenamiento (porcentaje de particiones que no se usan nunca, porcentaje de particiones que se usan bastante, etcétera).
    3. Longitud de la cola de espera.
    4. Tiempo de espera en la cola.
    5. Fragmentación interna.

    Dado que los trabajos son atendidos según el esquema de primero en llegar, primero en servirse:

c. Explique la manera en que el sistema maneja los conflictos entre trabajos cuando éstos se colocan en una cola de espera y al sistema siguen entrando trabajos: ¿qué va primero?

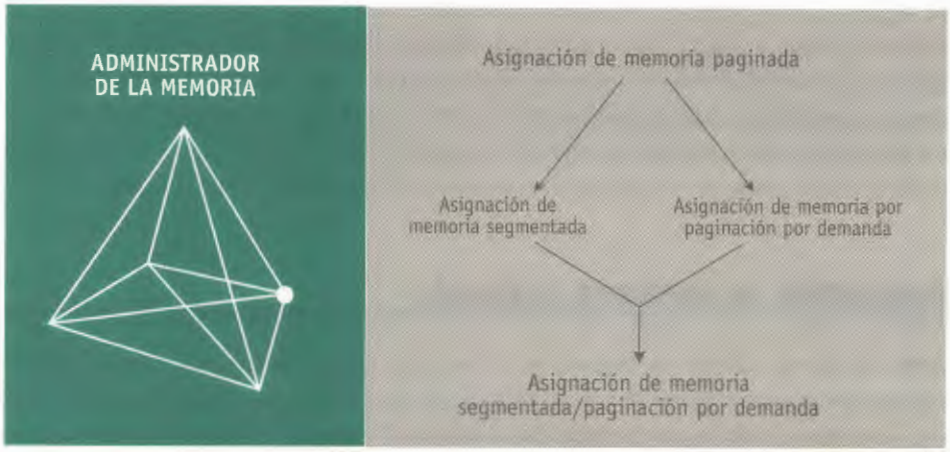
d. Explique la manera en que el sistema maneja los "relojes de trabajo", que siguen la pista de la cantidad de tiempo que se ha ejecutado cada trabajo, y los "relojes de espera", que siguen la pista de cuánto tiempo debe esperar cada trabajo en la cola de espera.

e. Puesto que se trata de un sistema dirigido por eventos, explique cómo definiría "evento" y lo que pasa en su sistema cuando éste ocurre.

f. Observe los resultados de la ejecución del mejor ajuste y compárelos con los resultados del primer ajuste. Explique lo que indican los resultados respecto al desempeño del sistema para esta mezcla de trabajos y organizaciones de la memoria. Un método de partición de la memoria, ¿es mejor que el otro? ¿Por qué? ¿Podría recomendar un método en lugar del otro dada la ejecución de su muestra? ¿Esto se cumple en todos los casos? Escriba algunas conclusiones y recomendaciones.

15. Suponga que su sistema (según se explicó en el ejercicio 14) ahora tiene un "spooler" (área de almacenamiento para mantener temporalmente el trabajo) y el planificador de trabajos puede escoger cuál de ellos de entre 25 trabajos residentes será atendido en primer lugar. También suponga que la política de primero en llegar, primero en ser servido es sustituida por la política "el trabajo más rápido, el primero en ser servido". Esto requeriría la realización de un ordenamiento por tiempo en la lista de trabajos antes de ejecutar el programa. ¿Esto modifica los resultados? ¿Modifica el análisis? ¿Modifica sus conclusiones y recomendaciones? El programa debe ejecutarse dos veces para probar esta nueva política con el mejor ajuste y con el primer ajuste.
16. Suponga que su spooler (según se describió en el ejercicio 14) sustituye la política previa con una de "el trabajo más pequeño, el primero en ser servido". Esto requeriría la realización de un ordenamiento por tamaño del trabajo en la lista de trabajos antes de ejecutar el programa. ¿Cómo se comparan sus resultados con los dos conjuntos de resultados previos? ¿Se modifica su análisis? ¿Se modifican sus conclusiones? El programa debe ejecutarse dos veces para probar esta nueva política con el mejor ajuste y con el primer ajuste.

Capítulo 3 Administración de la memoria: memoria virtual



"Nada se fortalece más con la práctica, o debilita por el descuido, que la memoria."

Quintiliano (*35-100 d.C.*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- La funcionalidad básica de los métodos de asignación de la memoria cubiertos en este capítulo: paginada, paginación por demanda, asignación de memoria segmentada y segmentada/paginación por demanda.
- La influencia que estos métodos de asignación de la memoria han tenido sobre la memoria virtual.
- La diferencia entre una política de remplazo de páginas primero en entrar, primero en salir, una política de remplazo de páginas la menos usada recientemente y una política de remplazo de páginas de reloj.
- La mecánica de la paginación y la manera en que el esquema de asignación de la memoria determina cuáles páginas deben intercambiarse de la memoria.
- El concepto de conjunto de trabajo y cómo se usa en esquemas de asignación de la memoria.
- El impacto que la memoria virtual tiene sobre la multiprogramación.
- La memoria caché y su rol para mejorar el tiempo de respuesta del sistema.

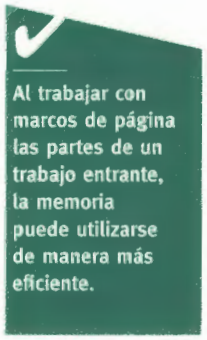
En el capítulo previo consideramos esquemas de asignación de la memoria simples. Cada uno requería que el administrador de la memoria almacenara todo el programa en la memoria principal en ubicaciones contiguas, y mencionamos que cada esquema resolvía algunos problemas aunque creaba otros, como fragmentación o el procesamiento extra de la reubicación.

En este capítulo continuamos con la evolución de la memoria virtual con cuatro esquemas de asignación de la memoria que primero eliminan la restricción de almacenar el programa de manera contigua, y luego eliminan el requisito de que todo el programa resida en la memoria durante su ejecución. Estos esquemas son asignación por paginación, por paginación por demanda, segmentada y segmentada/paginación por demanda, que constituyen los fundamentos de nuestros cuatro métodos de memoria virtual actuales. Nuestro análisis de la memoria caché mostrará cómo su uso mejora el rendimiento del administrador de la memoria.

Asignación de memoria paginada

Antes de que un trabajo sea cargado en la memoria, éste es dividido en partes llamadas páginas que pueden cargarse dentro de ubicaciones de memoria llamadas páginas con marcos. La **asignación de memoria paginada** se basa en el concepto de dividir cada trabajo de entrada en **páginas** del mismo tamaño. Algunos sistemas operativos escogen un tamaño de página igual al tamaño del bloque de memoria que también es igual a las secciones del disco en que se almacena el trabajo.

Las secciones de un disco se denominan **sectores** (o algunas veces, bloques), y las secciones de la memoria principal se denominan **marcos de página**. El esquema funciona de manera bastante eficiente cuando las páginas, los sectores y los marcos de página son todos del mismo tamaño. El tamaño exacto (el número de bytes que es posible almacenar en cada uno de estos) suele determinarse según el tamaño del sector del disco. En consecuencia, un sector contiene una página de instrucciones del trabajo y cabe en un marco de página de la memoria.



Antes de la ejecución de un programa, el administrador de la memoria la prepara al realizar lo siguiente:

1. Determina el número de páginas en el programa;
2. Localiza suficientes marcos de página en la memoria principal;
3. Carga todas las páginas del programa en éste.

Cuando el programa se prepara inicialmente para ser cargado, sus páginas están en secuencia lógica: las iniciales contienen las primeras instrucciones del programa y las finales contienen las últimas instrucciones del programa. Para que nuestra explicación resulte lo más clara posible, nos referiremos a las instrucciones del programa como bytes o palabras.

El proceso de carga es diferente de los esquemas que hemos estudiado en el capítulo 2 porque no es necesario cargar las páginas en bloques de memoria adyacentes. De hecho, cada página puede almacenarse en cualquier página disponible desde cualquier sitio en la memoria principal.

La ventaja principal de almacenar los programas en ubicaciones no contiguas es que la memoria principal puede usarse de manera más eficiente porque el marco de una página vacía puede ser usado por cualquier página de cualquier trabajo. Además, el esquema de compactación usado para particiones reubicables se elimina porque no hay fragmentación externa entre marcos de página (y tampoco fragmentación interna en la mayor parte de las páginas).

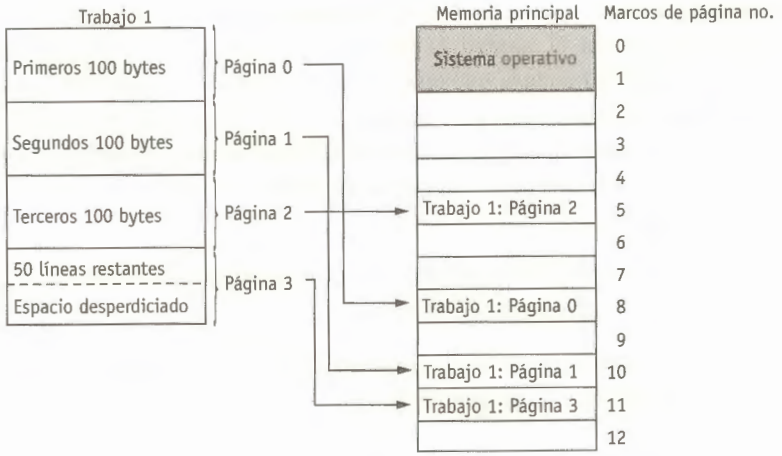
Sin embargo, con toda nueva solución viene asociado un problema. Debido a que las páginas de los trabajos no pueden ubicarse en cualquier parte en la memoria principal, el administrador de la memoria requiere un mecanismo para seguirles la pista, lo cual significa agrandar el tamaño y la complejidad del software del sistema operativo, lo cual incrementa el procesamiento extra.

En nuestros ejemplos, la primera página es la página 0 y la segunda es la 1, etc. Los marcos de página se numeran de la misma forma.

El ejemplo simplificador de la figura 3.1 muestra cómo el administrador de memoria sigue la pista de un programa con un tamaño de cuatro páginas. Para simplificar la aritmética, se ha iniciado arbitrariamente el tamaño de la pagina a 100 lineas (o bytes). El trabajo 1 tiene 350 líneas (o bytes) de tamaño y está siendo preparado para ejecutarse.

Observe en la figura 3.1 que la última página (página 3) no está utilizada por completo porque el trabajo es menor a 400 bytes: la última página usa sólo 50 de los 100 bytes disponibles. De hecho, muy pocos trabajos ocupan completamente todo el espacio de las páginas, de modo que la fragmentación interna sigue siendo un problema (aunque sólo en la última página de un trabajo).

En la figura 3.1 (con siete marcos de página), el sistema operativo puede acomodar trabajos que varían de tamaño desde 1 hasta 700 bytes porque pueden almacenarse en los siete marcos de página vacíos. Pero un trabajo mayor a 700 bytes no puede acomodarse sino hasta que el trabajo 1 termina su ejecución y libera las cuatro páginas que ocupa. Y un trabajo superior a 1100 bytes jamás cabe en la memoria de este sistema pequeño. En consecuencia,



(Figura 3.1) *Los programas demasiado grandes que no caben en una sola página se dividen en páginas del mismo tamaño que pueden almacenarse en páginas con marcos libres. En este ejemplo cada marco de página puede contener 100 bytes. El trabajo 1 mide 350 bytes de longitud y se divide entre cuatro marcos de página, dejando la fragmentación interna en el último marco de página. (La tabla del mapa de páginas para este trabajo se muestra en la tabla 3.2.)*

aunque la asignación de memoria paginada ofrece la ventaja de almacenamiento no contiguo, sigue requiriendo que todo el trabajo se almacene en la memoria durante su ejecución.

La figura 3.1 usa flechas y líneas para mostrar la manera en que las páginas de un trabajo caben en los marcos de página en la memoria, pero el administrador de la memoria usa tablas para seguirles la pista. Esencialmente hay tres tablas que llevan a cabo esta función: la de trabajo, la de mapa de páginas (PMT) y la de mapa de la memoria. Aunque en diferentes sistemas operativos estas tablas pueden tener nombres distintos, las tablas proporcionan el mismo servicio sin importar cómo se llamen. Las tres tablas residen en la parte de la memoria que está reservada para el sistema operativo.

Como se muestra en la tabla 3.1, la **tabla de trabajo** (JT) contiene dos valores por cada trabajo activo: el tamaño del trabajo (mostrado a la izquierda) y la ubicación en la memoria donde está almacenada la tabla del mapa de páginas (a la derecha). Por ejemplo, el primer trabajo tiene un tamaño de trabajo de 400 ubicado en 3096 en la memoria. La tabla de trabajo es una lista dinámica que crece a medida que los trabajos se cargan en el sistema y disminuye, como se muestra en el inciso *b*) de la tabla 3.1, a medida que se completan más tarde.

Tabla de trabajo

Tamaño del trabajo	Ubicación en la PMT
400	3096
200	3100
500	3150

(a)

Tabla de trabajo

Tamaño del trabajo	Ubicación en la PMT
400	3096
500	3150

(b)

Tabla de trabajo

Tamaño del trabajo	Ubicación en la PMT
400	3096
700	3100
500	3150

(c)

(Tabla 3.1)
Esta sección de la tabla de trabajo a) inicialmente tiene tres entradas, una para cada trabajo en proceso. Cuando termina el segundo trabajo b), su entrada en la tabla es liberado y se sustituye c) por información sobre el siguiente trabajo que será procesado.

Cada trabajo activo tiene su propia **tabla de mapa de páginas** (PMT), que contiene la información vital para cada página: el número de página y su dirección de memoria del marco de página. En realidad, la PMT incluye sólo una entrada por página. Los números de página son secuenciales (Página 0, Página 1, Página 2, hasta la última página), de modo que no es necesario enumerar el número de cada página en la PMT. La primera entrada en la PMT enlista la dirección de memoria del marco de página de la Página 0, la segunda entrada es la dirección de la Página 1, etcétera.

La **tabla de mapa de la memoria** (MEMORIAT) tiene una entrada por cada marco de página, que enlista su ubicación y el estado libre/ocupado.

En el momento de la compilación, cada trabajo se divide en páginas. Es posible ver cómo funciona esto al usar el trabajo 1 de la figura 3.1:

- La Página 0 contiene los primeros cien bytes.
- La Página 1 contiene los segundos cien bytes.

- La Página 2 contiene los terceros cien bytes.
- La Página 3 contiene los últimos cincuenta bytes.

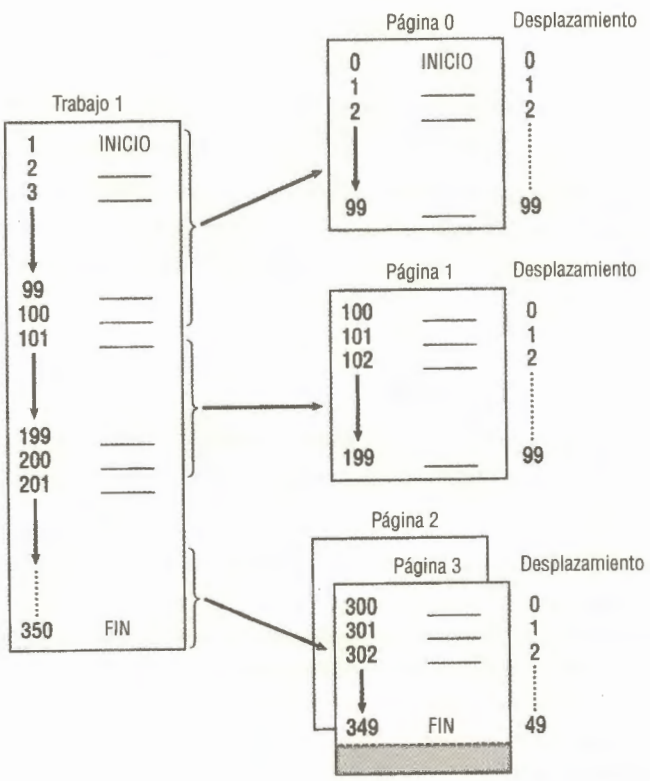
Como puede verse, el programa tiene 350 bytes, pero cuando se almacenan, los números del sistema van desde 0 hasta 349, de modo que el sistema se refiere a ellos como los bytes desde la 0 hasta la 349.

El **desplazamiento**, u **offset**, de un byte (es decir, cuán lejos está un byte del inicio de su página) es el factor que se usa para ubicar ese byte dentro de su página con marcos. Se trata de un factor relativo.

En el ejemplo simplificado mostrado en la figura 3.2, los bytes 0, 100, 200 y 300 son los primeros bytes para las páginas 0, 1, 2 y 3 respectivamente, así cada una está desplazada de cero. Igualmente, si el sistema operativo necesita tener acceso al byte 214, puede ir primero a la página 2 y luego ir al byte 14 (la línea número 15).

El primer byte de cada página tiene un desplazamiento de cero; la segunda, de uno y así sucesivamente hasta el último byte, cuyo desplazamiento es de 99. Así, una vez que el sistema operativo encuentra la página correcta, puede tener acceso a una línea usando la posición relativa de la línea dentro de su página.

(Figura 3.2)
El trabajo 1 mide 350 bytes y se divide en cuatro páginas, cada una con 100 bytes.



En este ejemplo resulta fácil ver que todos los números de línea menores que 100 están en la página 0, que todos los números de línea mayores o iguales que 100 pero menores que 200 están en la página 1 y así sucesivamente. (Esta es la ventaja de escoger un tamaño de página fijo, como 100 bytes.) El sistema operativo usa un algoritmo para calcular la página y el desplazamiento; se trata de un simple cálculo aritmético.

Para encontrar la dirección de un byte de programa dada, el número de bytes se divide entre el tamaño de la página, manteniendo el residuo como un entero. El cociente resultante es el número de página, y el residuo es el desplazamiento dentro de esa página. Cuando lo anterior se plantea como un problema de división larga, se ve como sigue:

```
                    número de la página
tamaño de la página)número del byte por encontrar
                   xxx
                    xxx
                   xxx
                     desplazamiento
```

Por ejemplo, si como tamaño de la página usamos 100 bytes, el número de la página y el desplazamiento (la ubicación dentro de esa página) del byte 214 pueden calcularse como sigue:

```
          2
100)214
        200
         14
```

El cociente (2) es el número de la página y el residuo (14) es el desplazamiento. Por tanto, el byte esta ubicado en la Página 2, a 15 líneas (línea 14) de la parte superior de la página.

A continuación se presenta otro ejemplo con un tamaño de página de 256 bytes que es el más común. Digamos que estamos buscando la ubicación del byte 384. Cuando dividimos 384 entre 256, el resultado es 1.5. Por tanto, el byte está ubicado en el punto medio de la segunda página (Página 1)

```
          1.5
256)384
```

Para encontrar la línea exacta de la ubicación, multiplique el tamaño de la página (256) por el decimal (0.5) para descubrir que la línea que estamos buscando está ubicada en la Línea 129 de la Página 1.

Use los conceptos recién presentados, y los mismos parámetros del primer ejemplo, para contestar las preguntas siguientes:

1. ¿Puede el sistema operativo (o el sistema de hardware) obtener un número de página mayor que 3, si el programa realizó la búsqueda para el byte 214?
2. En caso afirmativo, ¿qué debe hacer el sistema operativo?
3. ¿Puede el sistema operativo obtener un residuo superior a 99?
4. ¿Cuál es el menor residuo posible?

Estas son las respuestas:

1. No; no si el programa de aplicación fue escrito correctamente. (Para la excepción, consulte el ejercicio 14 al final de este capítulo.)
2. Enviar un mensaje de error y detener el procesamiento del programa (porque la página está fuera de los límites).
3. No; no si divide correctamente.
4. Cero.

El hardware de la computadora realiza la división, pero el sistema operativo es responsable de mantener las tablas que siguen la pista de la ubicación y la liberación de almacenamiento.

Este procedimiento proporciona la ubicación de una instrucción con respecto a las páginas del trabajo. Sin embargo, estas páginas sólo son relativas; cada página actualmente está almacenada en un marco de página que puede ser localizado en cualquier parte disponible de la memoria principal. Por tanto, el algoritmo necesita expandirse para localizar la ubicación exacta del byte en la memoria principal. Para hacer esto, necesitamos correlacionar cada uno de las páginas del trabajo con este número de marco de página utilizando la Tabla del mapa de páginas.

Por ejemplo, si vemos la PMT para el trabajo 1 en la figura 3.1, observamos que luce igual que los datos en la tabla 3.2.

(Tabla 3.2)
Tabla de mapa de páginas para el trabajo 1 en la figura 3.1.

Número de la tabla de trabajo	Número del marco de la página
0	8
1	10
2	5
3	11

En el primer ejemplo de división estábamos buscando una instrucción con un desplazamiento de 14 en la página 2. Para encontrar la ubicación exacta en la memoria, el sistema operativo (o el hardware) debe llevar a cabo los siguientes cuatro pasos. (En realidad, el sistema operativo identifica las líneas, o valores de los datos o instrucciones, como direcciones [bytes o palabras]. Aquí nos referiremos a ellas como líneas para facilitar la explicación.)

PASO 1 Efectuar el cálculo aritmético del algoritmo descrito previamente para determinar el número de página y el desplazamiento de la línea.

- Número de página = el cociente entero de la división de la dirección del espacio de trabajo entre el tamaño de página.
- Desplazamiento = el residuo de la división del número de página.

En este ejemplo, el cálculo muestra que el número de página es 2 y que el desplazamiento es 14.

PASO 2 Referirse a la PMT de este trabajo (mostrada en la tabla 3.2) y encontrar qué página con marcos contiene la página 2. La página 2 está ubicada en el marco de página 5.

PASO 3 Obtener la dirección del inicio del marco de página al multiplicar el número del marco de página (5) por el tamaño del marco de página (100).

```
ADDR_PAGE_FRAME = PAGE_FRAME_NUM * PAGE_SIZE
ADDR_PAGE_FRAME = 5(100)
```

PASO 4 Ahora sumar el desplazamiento (calculado en el paso 1) a la dirección inicial del marco de página para calcular la ubicación precisa en la memoria de la instrucción:

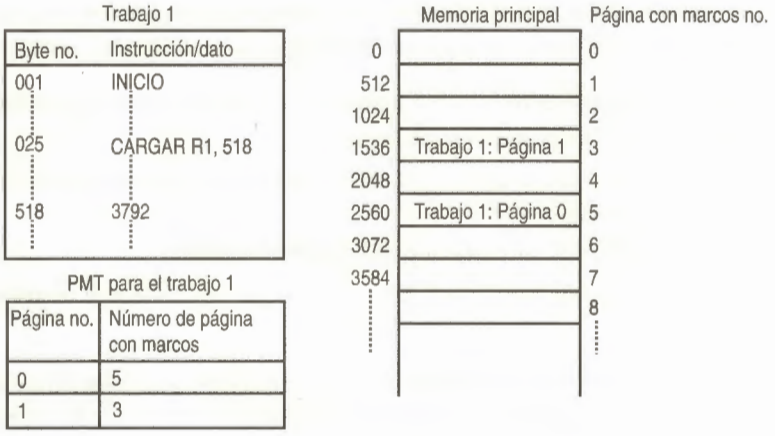
```
INSTR_ADDR_IN_MEM = ADDR_PAGE_FRAME + DISPL
INSTR_ADDR_IN_MEM = 500 + 14
```

El resultado de esta maniobra indica exactamente dónde se encuentra el byte 14 en la memoria principal.

La figura 3.3 muestra otro ejemplo y sigue el hardware (y el sistema operativo) según ejecuta un programa en lenguaje ensamblador que instruye al programa para cargar en el registro 1 el valor encontrado en el byte 518.

En la figura 3.3 los tamaños del marco de página en la memoria principal se establecen en 512 bytes cada uno y el tamaño de página es de 512 bytes para este sistema. Con base en la PMT puede observarse que este trabajo se ha dividido en dos páginas. Para encontrar la ubicación exacta del byte 518 (donde el sistema encuentra el valor a cargar en el registro 1), el sistema hace lo siguiente:

1. Calcula el número de página y el desplazamiento: el número de página es 1 y el desplazamiento, 6.
2. Va a la tabla de mapa de páginas y recupera el número idóneo del marco de página para la página 1. Se trata del marco de página 3.
3. Calcula la dirección inicial del marco de página al multiplicar el número del marco de página por el tamaño del marco de página: (3 * 512 = 1536).
4. Calcula la dirección exacta de la instrucción en la memoria principal al sumar el desplazamiento a la dirección inicial: (1536 + 6 = 1542). En consecuencia, la dirección de memoria 1542 contiene el valor que debe cargarse en el registro 1.



(Figura 3.3)
El trabajo 1 con su tabla de mapa de páginas. Esta figura de la memoria principal muestra la asignación del marco de página al trabajo 1.

Como puede verse, esta operación es larga. Cada que se ejecuta una instrucción o se usa un valor de un dato, el sistema operativo (o el hardware) debe traducir la dirección del espacio de trabajo, que es relativa, en su dirección física, que es absoluta. Esto se denomina resolver la dirección, **resolución de la dirección** o traducción de la dirección. Por supuesto, todo este procesamiento representa procesamiento extra, que ocupa la capacidad de procesamiento de los trabajos que están en espera de su terminación. Sin embargo, en la mayor parte de los sistemas, el hardware realiza la paginación, aunque el sistema operativo está implicado en la paginación dinámica, que será cubierta después.

La ventaja de un esquema de paginación es que permite que los trabajos sean asignados en ubicaciones no contiguas de la memoria, de modo que ésta se usa de manera más eficiente y más trabajos pueden caber en la memoria principal (lo cual es sinónimo). Sin embargo, hay desventajas: el procesamiento extra se incrementan y la fragmentación interna sigue siendo un problema, aunque sólo en la última página de cada trabajo. La clave del éxito de este esquema es el tamaño de la página. Un tamaño de página demasiado pequeño genera tablas de mapa de páginas muy largas, mientras un tamaño de página demasiado grande resulta en excesiva fragmentación interna. La determinación del mejor tamaño de página constituye una decisión de política importante; no hay reglas fáciles y rápidas que garanticen el uso óptimo de recursos, y se trata de un problema que veremos de nuevo al analizar otras alternativas de paginación. El mejor tamaño depende del entorno de trabajo real, de la naturaleza de los trabajos que se están procesando y de las restricciones impuestas sobre el sistema.

Paginación por demanda

La **paginación por demanda** introdujo el concepto de cargar sólo una parte del programa en la memoria para su procesamiento. Fue el primer esquema usado ampliamente que eliminó la restricción de tener todo el trabajo en la memoria, desde el principio hasta el final de su procesamiento. Con la paginación por demanda, los trabajos siguen dividiéndose en páginas del mismo tamaño que inicialmente residen en almacenamiento secundario. Cuando empieza la ejecución del programa, sus páginas se llevan a la memoria sólo según sean necesarias.

Con la paginación por demanda las páginas son cargadas conforme se requieren. Esto demanda acceso de alta velocidad a las páginas.

Con paginación por demanda, las páginas se cargan a medida que se solicita cada una. Para esto se requiere acceso a gran velocidad a las páginas. La paginación por demanda aprovecha el hecho de que los programas están escritos secuencialmente, de modo que mientras se está procesando una sección o módulo, todos los demás módulos están ociosos. No se accede a todas las páginas al mismo tiempo, o incluso de manera secuencial. Por ejemplo:

- Los módulos para manejar errores escritos por el usuario sólo se procesan cuando durante la ejecución se detecta un error específico. (Por ejemplo, pueden usarse para indicar al operador que los datos de entrada son incorrectos, o que el cálculo resultó en una respuesta inválida.) En caso de que no ocurra ningún error, y se espera que éste sea el caso en general, estas instrucciones no se procesan jamás y nunca es necesario cargarlas en la memoria.

- Muchos módulos son mutuamente excluyentes. Por ejemplo, si el módulo de entrada está activo (como cuando se empieza a cargar una hoja de cálculo), entonces el módulo de procesamiento está inactivo. En forma semejante, si el módulo de procesamiento está activo, entonces el módulo de salida (como la impresora) está ocioso.
- Ciertas opciones de programas son mutuamente excluyentes o no siempre son accesibles. Esto es fácil de observar en programas controlados por menús. Por ejemplo, un programa de aplicación puede ofrecer al usuario varias opciones de menús, como se muestra en la figura 3.4. El sistema permite que el operador sólo escoja una opción a la vez. Si el usuario selecciona la primera opción, entonces el módulo con las instrucciones del programa para mover registros al archivo es el único que se utiliza, de modo que es el único módulo que requiere estar en la memoria en esta ocasión. Todos los demás módulos permanecen en almacenamiento secundario hasta que son llamados desde el menú.
- A muchas tablas se les asigna una cantidad fija de espacio de direcciones aunque en realidad sólo se use una fracción de la tabla. Por ejemplo, una tabla de símbolos para un ensamblador puede prepararse para manejar 100 símbolos. Si sólo se usan 10 símbolos, entonces 90% de la tabla permanece sin usar.

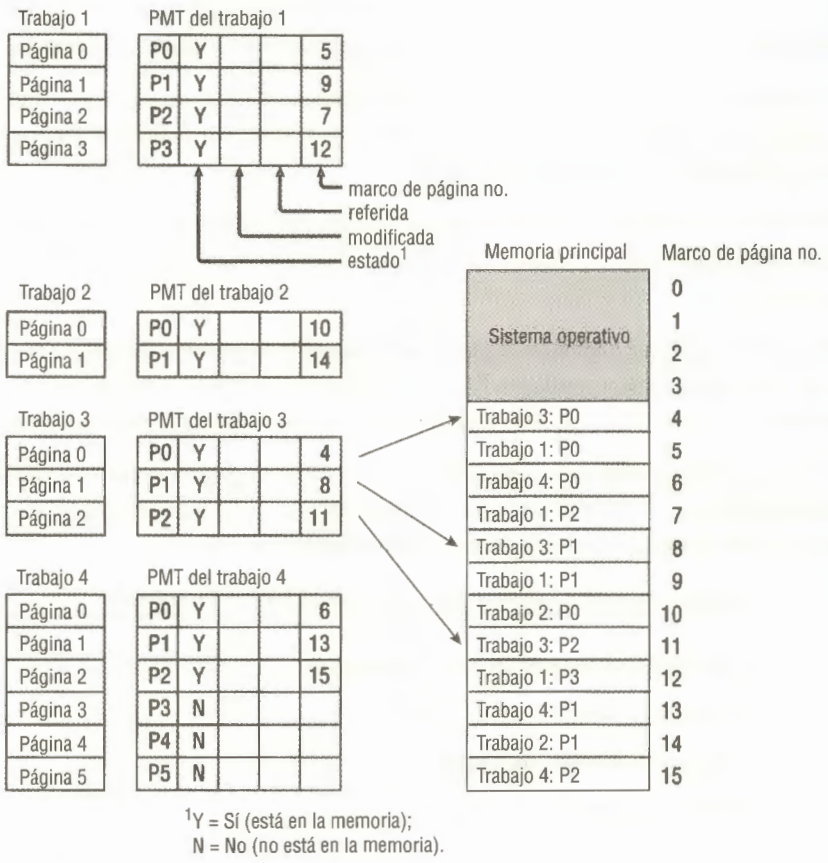
(Figura 3.4)

Cuando se elige una opción del menú de un programa de aplicación como este, no es necesario mover de inmediato hacia la memoria los otros módulos (tal como Ayuda) que no se requieren actualmente.

Una de las innovaciones más importantes de la paginación por demanda fue que hizo factible la memoria virtual (que se analizará más tarde en este capítulo). El esquema de paginación por demanda permite que el usuario ejecute trabajos con menos memoria principal que la necesaria si el sistema operativo utiliza el esquema de asignación de memoria descrito antes. De hecho, un esquema de paginación por demanda puede dar la impresión de una cantidad casi infinita o no finita de memoria física cuando, en realidad, la memoria física es significativamente menor que la infinita.

La clave para una implementación exitosa de este esquema es el uso de un dispositivo de almacenamiento de acceso directo de alta velocidad (como un disco duro o memoria flash) que puede trabajar directamente con la CPU. Esto es crucial porque las páginas deben pasarse rápidamente de almacenamiento secundario a la memoria principal y luego hacerlo de regreso.

Cómo y cuándo se pasan las páginas (también denominado intercambio) depende de políticas predefinidas que determinan cuándo hacer espacio para páginas necesarias y cómo hacerlo. El sistema operativo depende de tablas (como la de trabajo, la del mapa de páginas y del mapa de la memoria) para implementar el algoritmo. Estas tablas son básicamente las mismas que para asignación de memoria paginada pero con la adición de tres nuevos campos para cada página en la PMT: uno para determinar si la página solicitada ya está en



(Figura 3.5)

La paginación por demanda requiere que la tabla de mopa de páginas para cada trabajo siga la pista de cada página en la medida en que se carga o elimina de la memoria principal. Cada PMT sigue la pista del estado de la página, si ha sido modificada, referida recientemente y el número de marco de página para cada página que actualmente está en la memoria principal. (Nota: Para esta ilustración se han simplificado las tablas de mapa de páginas. Consulte la tabla 3.3 para más detalles.)

la memoria; otro para determinar si se ha modificado el contenido de la página, y un tercero para determinar si a la página se ha hecho referencia recientemente, como se muestra en la parte superior de la figura 3.5.

El primer campo indica al sistema dónde encontrar cada página. Si la página ya está en la memoria, el sistema se ahorra el tiempo necesario para traerla del almacenamiento secundario. Para el sistema operativo es más rápido explorar una tabla localizada en la memoria principal que recuperar una página desde el disco.

El segundo campo, al notar si se ha modificado la página, se usa para ahorrar tiempo cuando las páginas se eliminan de la memoria principal y regresan al almacenamiento secundario. En caso que no se haya modificado el contenido de la página, entonces no es necesario volver a escribir cada página en el almacenamiento secundario. La original, que ya está ahí, es correcta.

El tercer campo, que indica cualquier actividad reciente, se usa para determinar cuáles páginas muestran la mayor actividad de procesamiento, y cuáles están relativamente inactivas. Esta información es usada por varios esquemas de política de intercambio de páginas para

determinar cuáles páginas deben permanecer en la memoria principal y cuáles deben intercambiarse cuando el sistema requiere hacer espacio para otras páginas solicitadas.

Por ejemplo, en la figura 3.5, el número total de páginas de trabajo es 15, y el número total de marcos de páginas disponibles es 12. (El sistema operativo ocupa los cuatro primeros marcos de páginas en la memoria principal.)

Si se supone el estado de procesamiento que se ilustra en la figura 3.5, ¿qué ocurre cuando el trabajo 4 solicita que la página 3 sea llevada a la memoria si no hay marcos de páginas disponibles?

Un intercambio requiere de una interacción cercana entre los componentes del hardware, los algoritmos del software y las políticas de combinación.

Para mover una nueva página, es necesario intercambiar de regreso una página residente hacia el almacenamiento secundario. Específicamente, esto incluye copiar la página residente en el disco (en caso de haber sido modificada), y escribir la nueva en la página vacía.

Los componentes del hardware generan la dirección de la página requerida, encuentran el número de página y determinan si ya está en la memoria. El algoritmo que se presenta a continuación lleva a cabo el ciclo de procesamiento del hardware.

Algoritmo para el procesamiento de instrucciones del hardware

```
1 Iniciar el procesamiento de la instrucción
2 Generar la dirección de los datos
3 Calcular el número de página
4 Si la página está en la memoria
        Entonces
        obtener los datos y terminar la instrucción
        avanzar a la siguiente instrucción
        volver al paso 1
    En otro caso
        generar una interrupción de página
        llamar al manejador de fallo de página
    Terminar el if
```

Cuando se busca una instrucción se sigue el mismo proceso.

Cuando falla la prueba (lo cual significa que la página está en almacenamiento secundario pero no en la memoria), el software del sistema operativo asume el control. La sección del sistema operativo que resuelve el problema se denomina **manejador de fallo de página**. Éste determina si en la memoria hay marcos de páginas vacíos, de modo que la página solicitada puede copiarse de inmediato desde el almacenamiento secundario.

Si todos los marcos de página están ocupados, el manejador de fallo de página debe decidir cuál página intercambiar. (Esta decisión depende directamente de la política predefinida para eliminación de páginas.) Luego se efectúa el intercambio.

Algoritmo para el manejador de fallo de página

1 Si no hay página con marcos libres

    Entonces
    seleccionar la página a intercambiar, usando el algoritmo para eliminación de páginas
    actualizar la tabla de mapa de páginas del trabajo
    Si ha cambiado el contenido de la página entonces
        escribir la página en el disco
    Terminar el if

  Terminar el if

2 Usar el número de la página del paso 3 del algoritmo para el procesamiento de instrucciones del hardware para obtener la dirección del disco donde está almacenada la página solicitada (el administrador de archivos, que se analizará en el capítulo 8, usa el número de página para obtener la dirección del disco)

3 Leer la página en la memoria

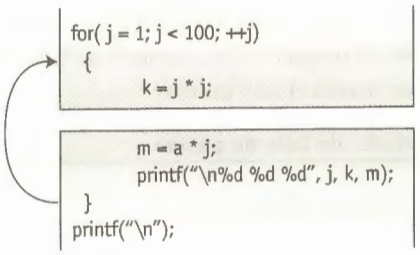
4 Actualizar la tabla de mapa de páginas del trabajo

5 Actualizar la tabla de mapa de la memoria

6 Reiniciar la instrucción interrumpida

Antes de proseguir es necesario actualizar tres tablas: las tablas de mapa de páginas para ambos trabajos (la PMT con la página que fue intercambiada hacia fuera y la PMT con la página que fue intercambiada hacia dentro) y la tabla de mapa de la memoria. Por último, se reanuda la instrucción que fue interrumpida y continúa el procesamiento.

Aunque la paginación por demanda es una solución para la utilización ineficiente de la memoria, no está exenta de problemas. Cuando hay una cantidad excesiva de **intercambio** de páginas entre la memoria principal y el almacenamiento secundario, la operación se vuelve ineficiente. Este fenómeno se denomina **thrashing**. Consume bastante trabajo de la computadora aunque se logra muy poco, y es originada cuando una página se elimina de la memoria pero en poco tiempo es llamada nuevamente. Puede ocurrir thrashing a través de los trabajos, cuando un gran número de ellos está compitiendo por un número relativamente bajo de páginas libres (la razón de páginas de trabajo a marcos de página de memoria libre es elevada), o puede ocurrir dentro de un trabajo; por ejemplo, en ciclos que cruzan los límites de páginas. Esto puede demostrarse con un ejemplo simple. Suponga que el inicio de un ciclos cae en la parte inferior de la página y que se completa en la parte superior de la página siguiente, como en el programa en C en la figura 3.6.



(Figura 3.6)

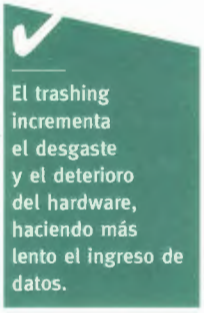
Un ejemplo de paginación por demanda que ocasiona un intercambio de página cada que se ejecuta el ciclo y resulta en thrashing. Si sólo está disponible un marco de página con, este programa tendrá un fallo de página cada que se ejecute el ciclo.

La situación en la figura 3.6 supone que sólo hay disponible un marco de página vacío. La primera página se carga en la memoria y empieza la ejecución, pero después de la ejecución del último comando en la página 0, la página se intercambia hacia fuera a fin de dejar espacio para la página 1. Ahora la ejecución puede continuar con la ejecución del primer comando en la página 1, pero en el símbolo "}" es necesario intercambiar la página 1, de modo que sea posible llevar de regreso la página 0 para continuar con el ciclo. Antes que se complete este programa, el intercambio ocurre 100 veces (a menos que otro marco de página se vuelva libre, de modo que las páginas puedan residir en la memoria al mismo tiempo). Un fallo en encontrar una página en la memoria suele denominarse **fallo de página** y este ejemplo generaría 100 fallos (e intercambios) de página.

En estos casos extremos, la tasa de cálculo útil puede degradarse por un factor de 100. Idealmente, un esquema de paginación por demanda es más eficaz cuando los programadores saben cuál es el tamaño de página que usa su sistema operativo y son cuidadosos al diseñar sus programas para mantener en un mínimo los fallos de página, aunque en realidad esto suele no ser factible.

Políticas y conceptos de remplazo de páginas

Según acabamos de aprender, la política que selecciona la página a eliminar, la **política de remplazo de páginas,** es crucial para la eficiencia del sistema, por lo cual el algoritmo que la lleva a cabo debe seleccionarse con cuidado.



Existen varios de estos algoritmos, y es un tema que cuenta con una gran cantidad de atención e investigación teórica. Dos de los algoritmos más conocidos son el primero en entrar, primero en salir y la menos usada recientemente. La política **primero en entrar, primero en salir (FIFO)** se basa en la teoría de que la mejor página a eliminar es la que ha permanecido más tiempo en la memoria. La política de la **menos usada recientemente (LRU)** se escoge para intercambiar la página a la que se ha tenido el menor acceso recientemente.

Para ilustrar la diferencia entre FIFO y LRU, imagine que usted tiene una cómoda llena con sus suéteres favoritos. La cómoda está llena, pero esto no impide que usted compre un suéter nuevo. Ahora usted debe guardarlo. Resulta evidente que no cabe en la cómoda con suéteres, a menos que saque alguno pero, ¿cuál suéter debe retirar de la cómoda y ponerlo en el armario? Su decisión se basará en una política para retirar suéteres.

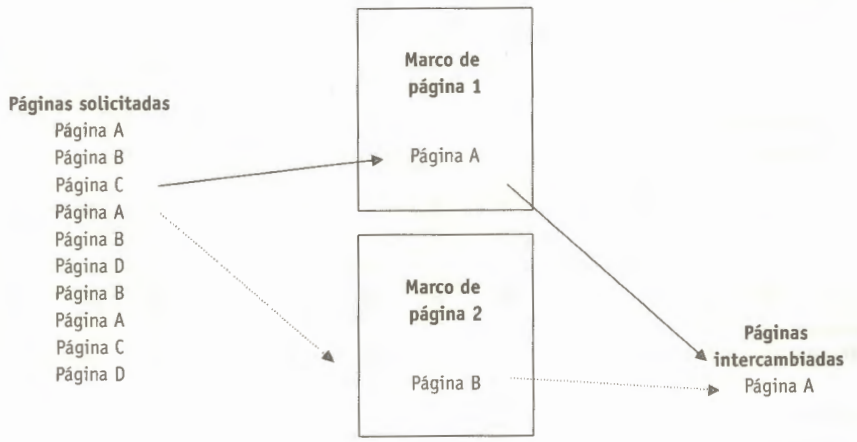
Usted puede retirar su suéter más viejo (el primero que llegó al cajón), si supone que tal vez ya no lo va a usar de nuevo, con la esperanza de no descubrir en los días siguientes que se trata de su posesión más usada y más apreciada. O bien, puede retirar el suéter que no ha usado recientemente y que no ha usado durante mucho tiempo (el que ha usado menos recientemente). Resulta fácil identificar este suéter porque es el que está en el fondo de la cómoda. Pero simplemente porque no ha sido usado recientemente no significa que una ocasión al año no demande su aparición dentro de poco.

¿Qué garantía tiene usted de que una vez que haya tomado su decisión no buscará en el armario para recuperar el suéter que guardó ayer? Puede convertirse en una víctima del thrashing.

¿Cuál es la mejor política? Depende del clima, de quién lo use y del guardarropa. Por supuesto, una opción es comprar otra cómoda. Para un sistema operativo (o una computadora), esto equivale a agregar más memoria accesible, y esta opción se explora después de descubrir cómo usar de manera más eficiente la memoria que ya se tiene.

Primero en entrar, primero en salir

La política de remplazo de páginas primero en entrar, primero en salir (FIFO) retira las páginas que han permanecido el mayor tiempo en la memoria. El proceso de intercambio de páginas se ilustra en la figura 3.7.



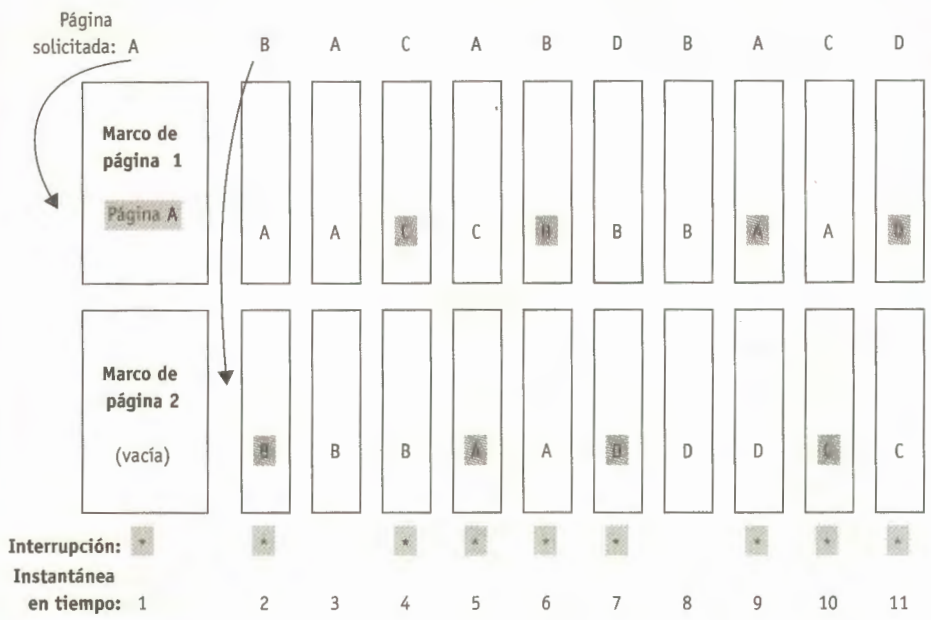
(Figura 3.7)

La política FIFO en acción con sólo dos páginas con marcos disponibles. Cuando el programa llama la página C, la página A debe sacarse del primer marco de página a fin de dejar espacio para la página C, como se muestra con las líneas continuas. Cuando la página A se necesita de nuevo, ésta remplaza a la página B en el segundo marco de página, como se muestra con las líneas discontinuas. Toda la secuencia se muestra en la figura 3.8.

La figura 3.8 muestra cómo funciona el algoritmo FIFO al seguir un trabajo con cuatro páginas (A, B, C, D) según es procesado por un sistema que sólo cuenta con dos marcos de página disponibles. En la figura 3.8 se muestra la manera en que cada página se intercambia hacia dentro y hacia fuera de la memoria y cada interrupción está indicada con un asterisco. El trabajo a procesar necesita sus páginas en el siguiente orden: A, B, A, C, A, B, D, B, A, C, D.

Cuando los dos marcos de páginas están ocupados, cada página nueva que se lleva a la memoria provoca que una de las páginas existentes se intercambie fuera del almacenamiento secundario. Cuando una nueva página debe cargarse en la memoria, sin importar que una página se intercambie hacia fuera de la memoria o no, se genera una interrupción de página, que puede identificarse con un asterisco (*).

La eficiencia de esta configuración es fatal: de 11 solicitudes de página hay 9 interrupciones de página debido al número limitado de marcos de páginas disponibles y a la necesidad de páginas nuevas. Para calcular la tasa de fracaso, el número de interrupciones se divide entre el número de solicitudes de página. La tasa de fracaso de este sistema es 9/11, igual a 82%. Planteado de otra forma, la tasa de éxito es 2/11, o 18%. Una tasa de fracaso tan alta como ésta suele ser inaceptable.



(Figura 3.8)

Al usar una política FIFO, esta página de análisis indica cómo cada página solicitada se intercambia hacia los dos marcos de página disponibles. Una vez que el programa está listo para su procesamiento, todas las cuatro páginas están en almacenamiento secundario. Cuando el programa llama una página que aún no está en la memoria, se emite una interrupción de página, como se muestra con los recuadros grises y los asteriscos. Ese programa resultó en nueve interrupciones de página.

En la figura 3.8, usando FIFO, la página A se intercambia hacia fuera cuando llega una nueva página, aun cuando a menudo sea la más usada.

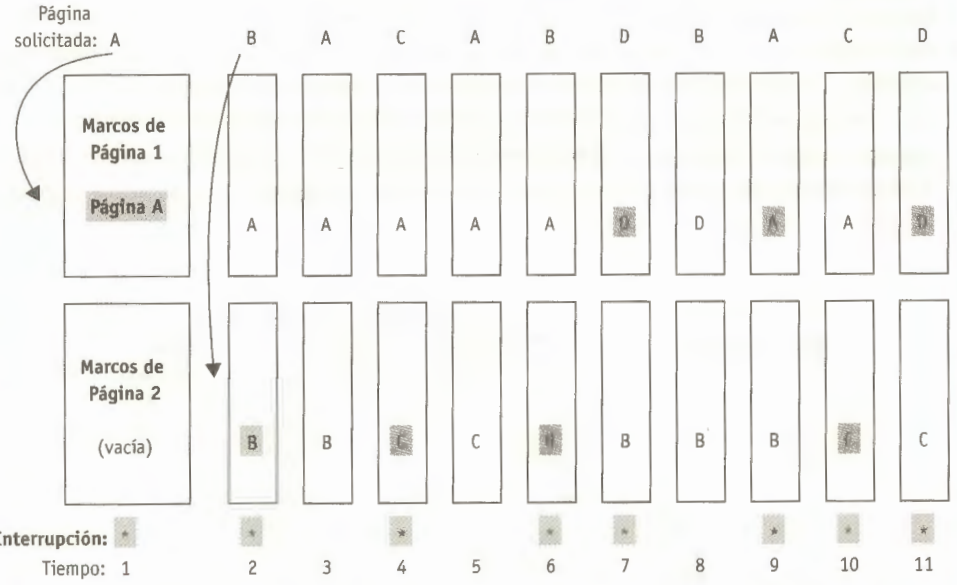
No estamos diciendo que FIFO está mal. Escogimos este ejemplo para mostrar la manera en que funciona FIFO, no para disminuir su interés como política de intercambio. La elevada tasa de fracaso aquí es provocada tanto por la cantidad limitada de memoria disponible como por el orden en que las páginas son solicitadas por el programa. El sistema no puede modificar el orden de las páginas, aunque es posible cambiar el tamaño de la memoria principal; pero comprar más memoria no siempre es la mejor solución, especialmente cuando se tienen muchos usuarios y cada uno quiere una cantidad ilimitada de memoria. No hay ninguna garantía de que comprar más memoria siempre dé por resultado un mejor rendimiento; esto se denomina **anomalía FIFO** o **anomalía Belady**, que se explicará más tarde en este capítulo.

Menos usada recientemente

La política de remplazo de páginas denominada menos usada recientemente (LRU) intercambia hacia fuera las páginas que muestran la menor cantidad de actividad reciente, suponiendo que estas páginas son las que tienen menor probabilidad de ser usadas nuevamente en el futuro próximo. Al revés, si se usa una página, es probable que vuelva a ser usada rápido nuevamente; este hecho se basa en la teoría de la ubicación, que se explicará más tarde en este capítulo.

Para ver cómo funciona lo anterior, seguiremos el mismo trabajo en la figura 3.8, pero usando la política LRU. Los resultados se muestran en la figura 3.9. Para implementar esta política, se mantiene una cola de las solicitudes en orden FIFO, se guarda una imagen de cuándo el trabajo entró al sistema, o periódicamente se hace una marca en la PMT del trabajo.

Al usar LRU en la figura 3.9, la página A permanece más tiempo en la memoria porque se usa más a menudo.



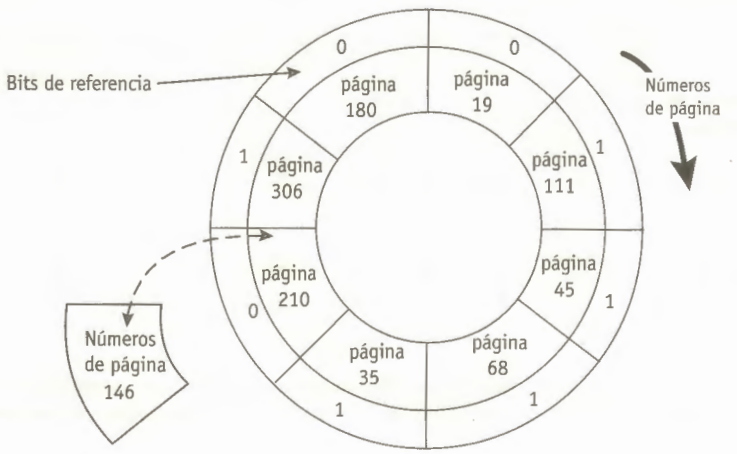
(Figura 3.9)

Administración de la memoria usando una política de eliminación de páginas LRU para el programa que se muestra en la figura 3.8. A lo largo de todo el programa se emiten 11 solicitudes de páginas, aunque sólo provocan 8 interrupciones de página.

La eficiencia de esta configuración es sólo ligeramente mejor que con FIFO. Aquí hay 8 interrupciones de página de 11 solicitudes de página, de modo que la tasa de fracaso es 8/11, o 73%. En este ejemplo, un incremento en la memoria principal de un marco de página aumentaría la tasa de éxito tanto de FIFO como de LRU (usted tendrá la oportunidad de calcular el incremento exacto en los ejercicios 6 y 7 al final del capítulo). No obstante, con base en un solo ejemplo no podemos concluir que una política es mejor que las otras. De hecho, LRU es una política de algoritmo de eliminación de pilas, lo que significa que un incremento en la memoria nunca origina un aumento en el número de interrupciones de página.

Por otra parte, se ha demostrado (Belady, Nelson, & Shelder, 1969) que en ciertas circunstancias el hecho de agregar más memoria puede, en casos excepcionales, provocar realmente un incremento en las interrupciones de página cuando se usa una política FIFO. Como ya se observó, esto se denomina anomalía FIFO. Pero aunque se trata de una ocurrencia inusual, el hecho de que exista asociada con el hecho de que las páginas se eliminan sin importar su actividad (como era el caso en la figura 3.8) ha alejado a FIFO de la posición de política más favorecida que mantenía en algunos casos.

Una variante del algoritmo de remplazo de páginas LRU se denomina **política de remplazo de páginas de reloj** porque se implementa con una cola circular y usa un apuntador para avanzar a través de los bits de referencia de las páginas activas, simulando el movimiento de un reloj analógico. El ritmo del algoritmo sigue el **ciclo de reloj**, que es el lapso entre dos tics en su reloj del sistema. El algoritmo selecciona una página para remplazarla si ésta tiene un bit de referencia igual a cero. Sin embargo, si su bit de referencia es uno, entonces a la página se le otorga una segunda oportunidad, su bit de referencia se iguala a cero y se inspecciona el siguiente bit de referencia de la página. Este proceso continúa hasta que se encuentra un bit de referencia cero. Si todos los bits de referencia se igualan a uno, entonces el apuntador debe realizar un ciclo a través de toda la cola circular de nuevo dando a cada página una segunda, y quizás una tercera o cuarta oportunidad. La figura 3.10 muestra la cola circular que contiene los bits de referencia para páginas que actualmente están en la memoria. El apuntador indica la página que debe considerarse en seguida para su eliminación. La figura 3.10 muestra lo que ocurre con los bits de referencia de las páginas a las que se ha otorgado



(Figura 3.10)
Una cola circular, que contiene el número de página y su bit de referencia. El apuntador busca el siguiente candidato a eliminar y remplaza la página 210 por una nueva página: 146.

una segunda oportunidad cuando una página nueva, la 146, debe asignarse a un marco de página, el espacio ocupado previamente por la página 210 que tenía un bit de referencia igual a cero.

Una segunda variante de la LRU usa un byte de referencia de 8 bits y una técnica de desplazamiento de bits para seguir la pista del uso de cada página que se encuentra actualmente en la memoria. Cuando la página se copia por primera vez en la memoria, el bit de su referencia a la extrema izquierda se iguala a 1; y todos los bits a la derecha del uno se igualan a cero, como se muestra en la figura 3.11. A intervalos de tiempo específicos del ciclo de reloj, el administrador de la memoria desplaza todos los bytes de referencia de la página a la derecha por un bit, abandonando el bit de la extrema derecha. Mientras tanto, cada vez que se hace referencia a una página, el bit de su referencia a la extrema izquierda se iguala a 1.

El proceso de desplazamiento de bits a la derecha e igualar el de la extrema izquierda a 1 cuando se está haciendo referencia a una página proporciona la historia del uso de cada página. Por ejemplo, una página que no se ha usado durante los ocho últimos tics del reloj tendría un byte de referencia de 00000000, mientras una a la que se haya hecho referencia una vez cada tic del reloj tiene un byte de referencia de 11111111.

Cuando ocurre un fallo de página, la política LRU escoge la página con el menor valor en su byte de referencia porque esta sería la menos usada recientemente. La figura 3.11 muestra la manera en que los bytes de referencia activan el cambio de página durante cuatro instantáneas para su uso. En *a*, las seis páginas se han iniciado; esto indica que a todas ellas se ha hecho referencia una vez. En *b*, a las páginas 1, 3, 5 y 6 se ha hecho referencia de nuevo (lo cual se ha marcado con 1), pero no a las páginas 2 y 4 (ahora marcadas con 0 en la posición de la extrema izquierda). En *c*, se ha hecho referencia a las páginas 1, 2 y 4. En *d*, se ha hecho referencia a las páginas 1, 2, 4 y 6. En *e*, se ha hecho referencia a las páginas 1 y 4.

Como se muestra en la figura 3.11, los valores almacenados en los bytes de referencia no son únicos; las páginas 3 y 5 tienen el mismo valor. En este caso la política LRU puede optar por intercambiar hacia fuera todas las páginas con el menor valor o puede elegir una de entre ellas con base en otros criterios como FIFO, prioridad o si se ha modificado el contenido de la página.

En los ejercicios al final de este capítulo se analizan otros algoritmos para la eliminación de páginas: MRU (más recientemente usada) y LFU (menos usada frecuentemente).

(Figura 3.11)

Observe cómo el bit de referencia para cada página se actualiza con cada tic de tiempo. Las flechas desde a) hasta e) muestran la manera en que se desplaza el bit de referencia con cada tic del reloj.

Número de página	Instantánea de tiempo 0	Instantánea de tiempo 1	Instantánea de tiempo 2	Instantánea de tiempo 3	Instantánea de tiempo 4
1	10000000	11000000	11100000	11110000	11111000
2	10000000	01000000	10100000	11010000	01101000
3	10000000	11000000	01100000	00110000	00011000
4	10000000	01000000	10100000	11010000	11101000
5	10000000	11000000	01100000	00110000	00011000
6	10000000	11000000	01100000	10110000	01011000

(a) (b) (c) (d) (e)

Mecánica de la paginación

Antes que el administrador de la memoria pueda determinar cuáles páginas serán intercambiadas, necesita información específica sobre cada página en la memoria; información incluida en las tablas de mapa de páginas.

Por ejemplo, en la figura 3.5, la tabla de mapa de páginas para el trabajo 1 incluía tres bits: el bit de estado, el bit referido y el bit modificado (estas eran las columnas de en medio; las dos columnas vacías y la columna Y/N que representa "en la memoria"). Pero la representación de la tabla que se muestra en la figura 3.5 estaba simplificada para efectos de ilustración. En realidad se parece a una tabla como la que se muestra en la tabla 3.3.

Cada PMT debe seguir la pista del estado, las modificaciones y referencias de cada página. Esto lo lleva a cabo con tres bits, cada uno de los cuales puede ser 0 o 1.

Página	Bit de estado	Bit de referencia	Bit modificado	Página con marcos
0	1	1	1	5
1	1	0	0	9
2	1	0	0	7
3	1	1	0	12

(Tabla 3.3)
Tabla de mapa de páginas para el trabajo 1 que se muestra en la figura 3.5.

Como ya mencionamos, el bit de estado indica si la página está actualmente en la memoria. El bit de referencia indica si la página ha sido llamada (referida) recientemente. Este bit es importante porque el algoritmo LRU lo usa para determinar cuáles páginas deben intercambiarse.

El bit modificado indica si el contenido de la página ha sido alterado, si es así, la página debe volver a escribirse en almacenamiento secundario cuando se intercambie antes que su marco de página sea liberado. (En un marco de página cuyo contenido no ha sido modificado es posible sobrescribir directamente, con lo cual se ahorra un paso.) Esto se debe a que cuando una página se intercambia hacia la memoria, no se elimina del almacenamiento secundario. En consecuencia, si la página no se modifica cuando está en la memoria principal (en cuyo caso el bit modificado permanece sin cambio, igual a cero), entonces no es necesario volver a copiar la página en almacenamiento secundario cuando se intercambie fuera de la memoria: la página que está ahí ya es correcta. Sin embargo, si se hicieran modificaciones a la página, la nueva versión de ésta debe volver a escribirse sobre la versión antigua, lo cual requiere tiempo.

Cada bit puede ser 0 o 1, como se muestra en la tabla 3.4.

(Tabla 3.4)
Significado de los bits usados en la tabla de mapa de la memoria.

Bit de estado		Bit modificado		Bit de referencia	
Valor	Significado	Valor	Significado	Valor	Significado
0	No está en la memoria	0	No se ha modificado	0	No se ha llamado
1	Está en la memoria	1	Fue modificado	1	Fue llamado

El bit de estado para todas las páginas que están en la memoria es igual a 1. Una página debe estar en la memoria antes que pueda intercambiarse, de modo que todos los candidatos para el intercambio tienen un 1 en esta columna. Los otros dos bits son 0 o 1, de modo que hay cuatro combinaciones posibles de los bits de referencia y modificado, como se muestra en la tabla 3.5.

(Tabla 3.5)
Cuatro combinaciones posibles de los bits de referencia y modificado, así como el significado de cada uno.

	Modificado	De referencia	Significado
Caso 1	0	0	No está modificado Y no se ha referido
Caso 2	0	1	No está modificado PERO se ha referido
Caso 3	1	0	Fue modificado PERO no se ha referido (¿imposible?)
Caso 4	1	1	Fue modificado Y fue referido

El algoritmo FIFO sólo usa los bits modificado y de estado cuando se intercambian páginas, pero el LRU busca a los tres bits antes de decidir cuáles páginas intercambiar.

¿Qué página escoge la política LRU primero para intercambiar? De los cuatro casos descritos en la tabla 3.5, escoge las páginas en el caso 1 como los candidatos ideales para su eliminación, porque no han sido modificadas ni referidas. Esto significa que no es necesario volver a escribirlas en almacenamiento secundario, y que no han sido referidas recientemente. Así, las páginas con ceros para estos dos bits serían las primeras en ser intercambiadas.

¿Cuál es el candidato más probable? La política LRU escogería a continuación el caso 3 porque los otros dos, el caso 2 y el caso 4, fueron referidos recientemente. Las malas noticias es que las páginas del caso 3 han sido modificadas, de modo que se requiere más tiempo para intercambiarlas. Por eliminación, entonces puede afirmarse que el caso 2 es la tercera opción y que el caso 4 serían las páginas con menor probabilidad de ser eliminadas.

Tal vez usted ya observó que el caso 3 presenta una situación interesante: aparentemente estas páginas han sido modificadas sin haber sido referidas. ¿Cómo es posible? La clave está en la manera en que el sistema operativo manipula los bits de referencia. Cuando las páginas se llevan a la memoria, suelen referirse por lo menos una vez y eso significa que todas las páginas pronto tendrán un bit de referencia igual a 1. Por supuesto, al algoritmo LRU sería derrotado si cada página indicara que ha sido referida. En consecuencia, para tener la certeza de que el bit de referencia en realidad indica *recientemente* referida, el sistema operativo periódicamente lo restablece en 0. Luego, a medida que todas las páginas son referidas durante el procesamiento, el bit se cambia de 0 a 1 y la política LRU es capaz de identificar cuáles páginas en realidad son referidas frecuentemente. Como puede imaginar, hay un breve instante, justo después que se han restablecido los bits, en que todas las páginas (incluso las páginas activas) tienen bits de referencia iguales a 0 y son vulnerables. Pero a medida que continúa el procesamiento, las páginas que son más referidas pronto tienen su bit restablecido en 1, de modo que se minimiza el riesgo.

El conjunto de trabajo

Una innovación que mejoró el rendimiento de los esquemas de paginación por demanda fue el concepto de **conjunto de trabajo**. El conjunto de trabajo es el conjunto de páginas residentes en la memoria a las que es posible tener acceso directamente sin incurrir en un fallo de página.

Cuando un usuario solicita la ejecución de un programa, la primera página se carga en la memoria y la ejecución continúa a medida que se cargan más páginas: las que contienen declaraciones variables, otras instrucciones, otros datos, etcétera. Luego de un rato, la mayor parte de los programas alcanzan un estado bastante estable y el procesamiento prosigue de manera continua con muy pocos fallos de página adicionales. En este momento el conjunto del trabajo está en la memoria y el programa no genera muchos fallos de página a menos que llegue a otra fase que requiere un número diferente de páginas para hacer el trabajo: un conjunto de trabajo diferente.

Por supuesto, es posible que un programa estructurado de manera deficiente requiera que todas sus páginas en la memoria estén en la memoria antes de poder iniciar el procesamiento.

Por fortuna, la mayor parte de los programas están estructurados, lo cual conduce a una **localidad de referencia** durante la ejecución del programa, lo que significa que durante cualquier fase de su ejecución, el programa se refiere sólo a una pequeña fracción de sus páginas. Por ejemplo, si un trabajo está ejecutando un ciclo cuando las instrucciones en el ciclo son referidas ampliamente mientras las que están fuera del ciclo no se usan en absoluto sino hasta que se completa todo el ciclo, eso es localidad de referencia. Lo mismo es válido para instrucciones secuenciales, llamadas a subrutinas (dentro de la subrutina), implementaciones de pilas, acceso a variables que actúan como contadores o sumas, variables multidimensionales como arreglos y tablas (sólo se requieren unas cuantas páginas para manejar las referencias).

Sería conveniente si todas las páginas en el conjunto de trabajo se cargaran en la memoria al mismo tiempo para minimizar el número de fallos de página y acelerar la velocidad de

procesamiento, aunque resulta más fácil decirlo que hacerlo. Para hacer lo anterior, el sistema necesita respuestas definitivas a algunas preguntas difíciles: ¿Cuántas páginas integran el conjunto de trabajo? ¿Cuál es el número máximo de páginas que el sistema operativo permite para un conjunto de trabajo?

La segunda pregunta es particularmente importante en oficinas en red o sistemas de tiempo compartido, que regularmente intercambian trabajos (o páginas de trabajos) en la memoria y de vuelta al almacenamiento secundario para acomodarse a las necesidades de muchos usuarios. El problema es este: cada que un trabajo vuelve a cargarse de vuelta en la memoria (o tiene páginas intercambiadas) debe generar varios fallos de página hasta que su conjunto de trabajo está de regreso en la memoria y el procesamiento puede continuar. Se trata de una tarea consumidora de tiempo para la CPU, que no puede procesar trabajos durante el tiempo que requiere para procesar cada fallo de página, como se muestra en la figura 3.12.

Una solución adoptada por muchos sistemas de paginación es empezar por identificar cada conjunto de trabajo y luego cargarlo en su totalidad en la memoria antes de permitir el inicio de la ejecución. Esto es difícil de hacer antes que un trabajo sea ejecutado, pero puede ser identificado a medida que procede su ejecución.

En un sistema de tiempo compartido o de oficinas en red, esto significa que el sistema operativo debe seguir la pista del tamaño e identidad de todos los conjuntos de trabajo, asegurándose que los trabajos destinados para procesamiento en cualquier instante no excedan la memoria disponible. Algunos sistemas operativos usan un tamaño variable de conjunto de trabajo y lo incrementan según sea necesario (el trabajo requiere más procesamiento) o lo disminuyen según sea necesario. Esto puede significar que es necesario reducir el número de trabajos en la memoria si se necesita, al hacerlo, el sistema puede asegurar la terminación de cada trabajo y la liberación subsecuente de su espacio de memoria.

Hemos considerado varios ejemplos de esquemas de asignación de memoria para paginación por demanda. La paginación por demanda presentaba dos ventajas. Fue el primer esquema en que un trabajo no estaba restringido por el tamaño de la memoria física e introdujo el concepto de memoria virtual. La segunda ventaja era que utilizaba la memoria de manera

(Figura 3.12)

Línea de tiempo que muestra la cantidad de tiempo necesaria para procesar fallos de página para un solo programa. El programa en este ejemplo requiere 120 milisegundos (ms) para su ejecución, pero 900 ms adicionales para cargar las páginas necesarias en la memoria. En consecuencia, el tiempo de espera por trabajo es de 1020 ms.

más eficiente que los esquemas previos porque las secciones de los trabajos que se usaban en raras ocasiones o no se usaban en absoluto (como las rutinas de error) no se cargaban en la memoria a menos que se solicitara específicamente. Su desventaja era el procesamiento extra incrementado por las tablas y las interrupciones de página. El siguiente esquema de asignación contará con las ventajas de paginación y particiones dinámicas.

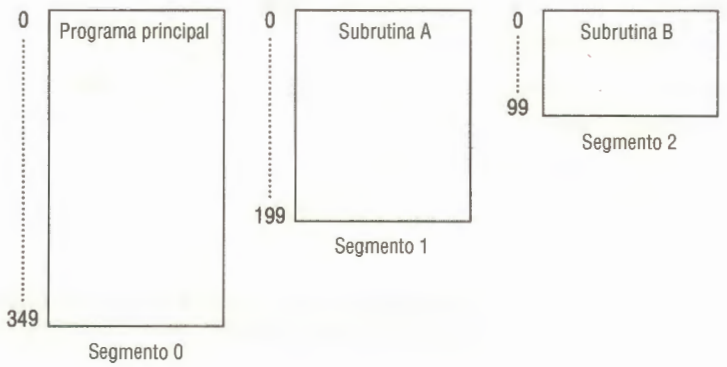
Asignación de memoria segmentada

El concepto de segmentación se basa en la práctica común de los programadores para estructurar sus programas en módulos: agrupaciones lógicas de código. Con la **asignación de memoria segmentada**, cada trabajo se divide en varios **segmentos** de diferentes tamaños, uno para cada módulo que contiene piezas que realizan funciones relacionadas. La asignación de memoria segmentada fue diseñada para reducir los fallos de página que resultaban de tener un ciclo de un segmento dividido en dos o más páginas. Una **subrutina** es un ejemplo de uno de estos grupos lógicos. Es fundamentalmente diferente de un esquema de paginación, que divide el trabajo en varias páginas, todas del mismo tamaño, donde cada una contiene a menudo piezas de más de un módulo del programa.

Una segunda diferencia importante es que la memoria principal ya no está dividida en marcos de páginas porque el tamaño de cada segmento es diferente: algunos son grandes y algunos son pequeños. En consecuencia, como ocurre con las particiones dinámicas analizadas en el capítulo 2, la memoria es asignada en una manera dinámica.

Cuando un programa se compila o ensambla, los segmentos se establecen según los módulos estructurales del programa. Cada segmento se numera y para cada trabajo se genera una **tabla de mapa de segmentos** (**SMT**) que contiene los números de los segmentos, sus longitudes, derechos de acceso y (cuando cada trabajo se carga en la memoria), su ubicación en la memoria. Las figuras 3.13 y 3.14 muestran el mismo trabajo, el trabajo 1, compuesto de un programa principal y dos subrutinas, junto con su tabla de mapa de segmentos y la asignación verdadera en la memoria principal.

La tabla de mapa de segmentos funciona igual que la tabla de mapa de páginas, aunque gestiona segmentos en lugar de páginas.



(Figura 3.13)
Asignación de memoria segmentada. El trabajo 1 incluye un programa principal, la subrutina A y la subrutina B. Se trata de un trabajo dividido en tres segmentos.

Así como ocurre con la paginación por demanda, los bits de referencia, modificado y de estado se usan en la segmentación y aparecen en la SMT pero no se muestran en las figuras 3.13 y 3.14.

El administrador de la memoria requiere seguir la pista de los segmentos en la memoria. Esto se lleva a cabo con tres tablas que combinan aspectos tanto de particiones dinámicas como de administración de la memoria por paginación por demanda:

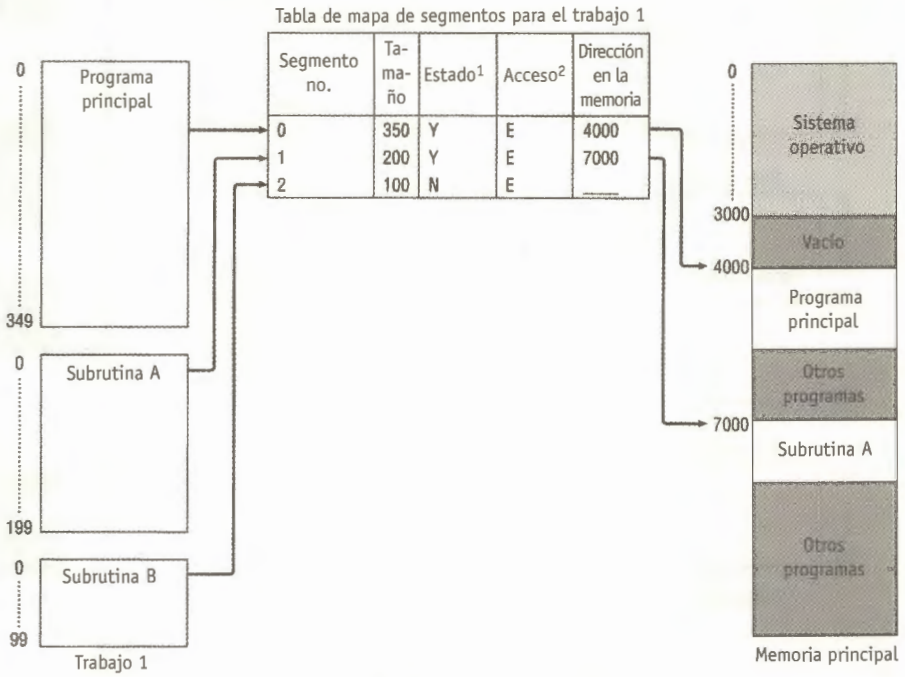
- La tabla de trabajo enumera todos los trabajos en proceso (una para todo el sistema).
- La tabla de mapa de segmentos enumera detalles sobre cada segmento (una para cada trabajo).
- La tabla de mapa de la memoria supervisa la asignación de la memoria principal (una para todo el sistema).

Como en la paginación por demanda, las instrucciones dentro de cada segmento están ordenados secuencialmente, aunque no es necesario almacenarlos continuamente en la memoria. Sólo se necesita saber dónde está almacenado cada segmento. Los contenidos de los segmentos en sí son contiguos en este esquema.

Para tener acceso a una ubicación específica dentro de un segmento es posible efectuar una operación semejante a la que se usó para la adminstración de la memoria paginada. La única diferencia es que se trabaja con segmentos, en lugar de páginas. El esquema de direc-

(Figura 3.14)

La tabla del mapa de segmentos sigue la pista de cada segmento para el trabajo 1.



[1] Y = está en la memoria;
N = no está en la memoria.

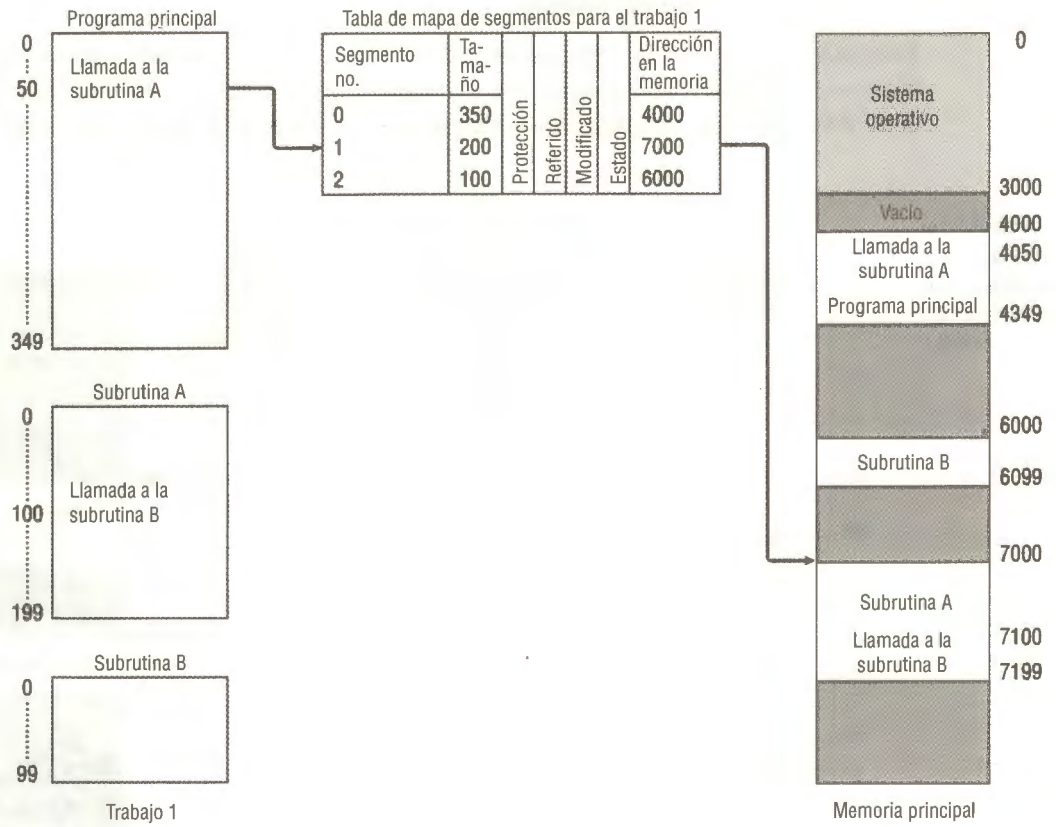
[2] E = sólo ejecutar.

cionamiento requiere el número de segmento y el desplazamiento dentro de ese segmento, y debido a que éstos son de tamaños diferentes, es necesario verificar el desplazamiento para tener la certeza de que no está fuera del intervalo del segmento.

En la figura 3.15, el segmento 1 incluye a toda la subrutina A, de modo que el sistema encuentra la dirección inicial del segmento 1, la dirección 7000, y empieza ahí.

Si la instrucción solicitaba que el procesamiento empezara en la línea 100 de la subrutina A (lo que es posible con lenguajes que atienden entradas múltiples en las subrutinas), entonces, para ubicar esa línea en la memoria, el administrador de la memoria puede requerir sumar 100 (el desplazamiento) a 7000 (la dirección inicial del segmento 1). Su código se vería como sigue:

```
ACTUAL_MEM_LOC = BEGIN_MEM_LOC_OF_SEG + DISPLACEMENT
```



(Figura 3.15)

Durante la ejecución, el programa principal llama a la rutina A, que activa la SMT para buscar su ubicación en la memoria.

¿Es posible que el desplazamiento sea más grande que el tamaño del segmento? No, no si el programa está codificado correctamente; sin embargo, los accidentes ocurren y el administrador de la memoria siempre debe protegerse contra esta posibilidad al verificar el desplazamiento contra el tamaño del segmento, comprobando que no se encuentra fuera de los límites.

Para tener acceso a una ubicación cuando se usa administración de memoria paginada o segmentada, la dirección está compuesta por dos valores: el número de página o de segmento y el desplazamiento. En consecuencia, se trata de un esquema de direccionamiento bidireccional:

```
SEGMENT_NUMBER & DISPLACEMENT
```

La desventaja de cualquier esquema de asignación en donde la memoria se parte dinámicamente es el regreso de la fragmentación externa. Por tanto, de vez en cuando es necesario volver a compactar la memoria disponible (en caso de usar ese esquema).

Como puede observar, hay muchas semejanzas entre paginación y segmentación, por lo que a menudo se confunden. La diferencia más importante es conceptual: las páginas son unidades físicas que son indivisibles para el programa del usuario y constan de tamaños fijos; los segmentos son unidades lógicas que son visibles para el programa del usuario y constan de tamaños variables.

Asignación de memoria segmentada/ paginación por demanda

El esquema de **asignación de memoria segmentada/paginación por demanda** evolucionó a partir de los dos que hemos analizado. Es una combinación de segmentación y paginación por demanda, y ofrece los beneficios lógicos de la segmentación, así como los beneficios físicos de la paginación. La lógica no es nueva. Los algoritmos usados por los esquemas de paginación por demanda y administración segmentada de la memoria se aplican aquí con modificaciones menores.

Este esquema de asignación no mantiene cada segmento como una sola unidad contigua, sino que lo subdivide en páginas del mismo tamaño, menores que la mayor parte, y más fácilmente manejables que todos los segmentos. En consecuencia, muchos de los problemas de la segmentación (compactación, fragmentación externa y manejo del almacenamiento secundario) se eliminan porque las páginas son de longitud fija.

Este esquema, ilustrado en la figura 3.16, requiere cuatro tablas:

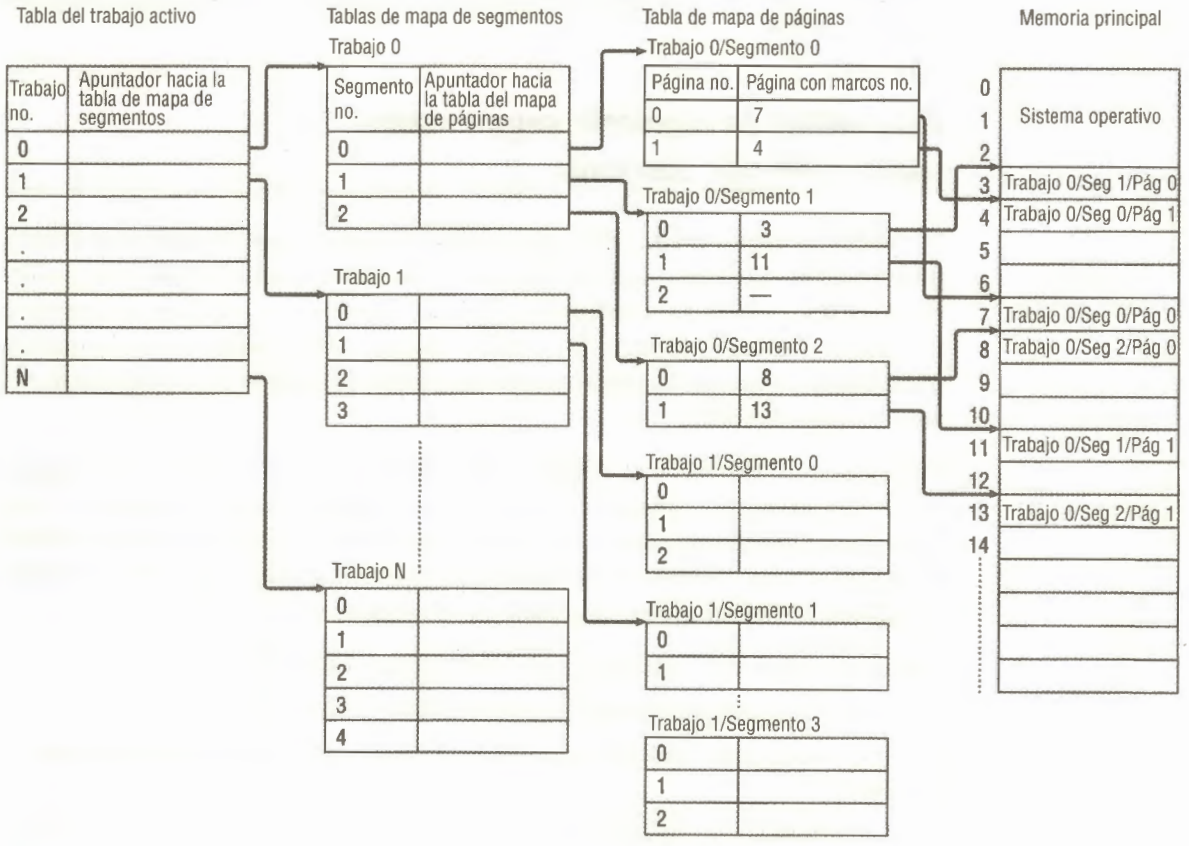
- La tabla de trabajo enumera todo trabajo en proceso (una para todo el sistema).
- La tabla de mapa de segmentos enumera detalles sobre cada segmento (una para cada trabajo).
- La tabla de mapa de páginas enumera detalles sobre cada página (una para cada segmento).
- La tabla de mapa de la memoria supervisa la asignación de los marcos de páginas en la memoria principal (una para todo el sistema).

Observe que las tablas en la figura 3.16 se han simplificado. La SMT en realidad incluye información adicional concerniente a la protección (como la autoridad para leer, escribir, ejecutar y borrar partes del archivo), así como cuáles usuarios tienen acceso a ese segmento (sólo el usuario, sólo un grupo, o todo mundo: algunos sistemas denominan a estas categorías de acceso propietario, grupo y mundo, respectivamente). Además, la PMT incluye los bits de estado, modificado y de referencia.

Para tener acceso a una ubicación en la memoria, el sistema debe localizar la dirección, que está compuesta por tres entradas: número de segmento, número de página dentro de ese segmento y desplazamiento dentro de esa página. Se trata de un esquema de direccionamiento tridimensional:

```
SEGMENT_NUMBER & PAGE_NUMBER & DISPLACEMENT
```

Las desventajas más importantes de este esquema de asignación de memoria son el procesamiento extra requeridos por las tablas extras y el tiempo necesario para referirse a la tabla



(Figura 3.16)

Manera en que interactúan la tabla de trabajo, la tabla del mapa de segmentos, la tabla del mapa de páginas y la memoria principal en un esquema segmento/paginación.

de segmentos y de páginas. Para minimizar el número de referencias, muchos sistemas usan memoria asociativa a fin de acelerar el proceso.

La **memoria asociativa** es una denominación dada a varios registros que están asignados a cada trabajo que esté activo. Su tarea consiste en asociar varios segmentos y números de página pertenecientes al trabajo que se está procesando con sus direcciones en la memoria principal. Estos registros asociativos residen en la memoria principal, y el número exacto de registros varía de un sistema al otro.

Para apreciar el rol de la memoria asociativa, es importante comprender la manera en que funciona el sistema con segmentos y páginas. En general, cuando un trabajo se asigna a la CPU, su tabla de mapa de segmentos se carga en la memoria principal, mientras las tablas de mapa de páginas se cargan sólo cuando es necesario. A medida que las páginas se intercambian entre la memoria principal y el almacenamiento secundario, todas las tablas se actualizan.

A continuación se presenta un procedimiento típico: cuando una página se solicita por primera vez, la SMT del trabajo se busca para ubicar su PMT; luego, la PMT se carga y busca para determinar la ubicación de la página en la memoria. Si la página no está en la memoria, entonces se emite una interrupción de página, la página se lleva a la memoria y la tabla se actualiza. (Como indica el ejemplo, el hecho de cargar la PMT también puede ocasionar una interrupción de página, o fallo de página.) Este proceso es tan tedioso como se oye, aunque es más fácil. Puesto que la PMT de este segmento (o parte de éste) ahora reside en la memoria, cualquier otra solicitud por páginas dentro de este segmento puede acomodarse rápidamente porque no es necesario llevar la PMT a la memoria. Sin embargo, tener acceso a estas tablas (SMT y PMT) consume tiempo.

Las dos búsquedas a través de la memoria asociativa y los segmentos en las tablas de mapa de páginas se llevan a cabo al mismo tiempo.

Este es el problema relacionado con la memoria asociativa, el cual almacena la información relacionada a las páginas usadas más recientemente. Entonces cuando una petición de página es suministrada, dos busquedas comienzan —una a través de las tablas de segmentos página y otra a través del contenido de los registros asociativos.

Si la búsqueda de los registros asociativos tiene éxito, entonces la búsqueda a través de las tablas se detiene (o elimina) y la traducción de la dirección se lleva a cabo usando la información en los registros asociativos. Sin embargo, si la búsqueda de la memoria asociativa fracasa, no se pierde tiempo porque la búsqueda a través de la SMT y la PMT ya ha empezado (en este esquema). Cuando esta búsqueda es exitosa y se ha determinado la dirección de la memoria principal de la PMT, la dirección se usa para continuar la ejecución del programa y la referencia también se almacena en uno de los registros asociativos. Si todos los registros asociativos están llenos, entonces se usa un algoritmo LRU (u otro) y el registro asociativo menos recientemente referido se usa para mantener la información en esta página solicitada.

Por ejemplo, un sistema con ocho registros asociativos por trabajo los usa para almacenar la SMT y la PMT para las ocho últimas páginas referidas por ese trabajo. Cuando es necesario traducir una dirección de números de segmento y página a una ubicación en la memoria, el sistema busca primero en los ocho registros asociativos. Si se encuentra una coincidencia, la

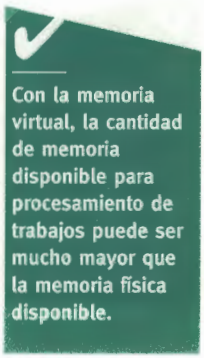
ubicación de la memoria se toma del registro asociativo; si no hay ninguna coincidencia, entonces continúa la búsqueda en las SMT y las PMT y como resultado, la nueva información se almacena en uno de los ocho registros.

Si un trabajo se intercambia hacia el almacenamiento secundario durante su ejecución, entonces toda la información almacenada en sus registros asociativos se guarda, así como las SMT y PMT actuales, de modo que el trabajo desplazado puede reanudarse rápidamente cuando vuelve a asignársele la CPU. La ventaja principal de una gran memoria asociativa es el incremento en velocidad. La desventaja consiste en el elevado costo del hardware complicado necesario para efectuar las búsquedas paralelas. En algunos sistemas, las búsquedas no se realizan en paralelo, sino que la búsqueda de la SMT y la PMT sigue la búsqueda de los registros asociativos.

Memoria virtual

La paginación por demanda hizo posible que un programa se ejecute aun cuando sólo una parte del programa esté cargado en la memoria principal. En efecto, la memoria virtual eliminaba la restricción impuesta sobre el tamaño máximo del programa. La capacidad de mover páginas a voluntad entre la memoria principal y el almacenamiento secundario proporcionó un nuevo concepto denominado de manera idónea **memoria virtual**. Aunque sólo una porción de cada programa está almacenada en la memoria, proporcionó a los usuarios la impresión de que sus programas estaban cargados por completo en la memoria principal durante todo el tiempo de procesamiento: una hazaña que requería una cantidad increíble de memoria principal.

Hasta la implementación de la memoria virtual, el problema de hacer que los programas entraran en la memoria disponible se dejaba a los usuarios. En los primeros días, los programadores tenían que limitar el tamaño de sus programas para asegurarse que entraran en la memoria principal; pero a veces esto no era posible porque la cantidad de memoria que se les asignaba era demasiado pequeña para realizar el trabajo. Programadores inteligentes resolvieron el problema al escribir programas condensados siempre que era posible. Lo más importante era el tamaño del programa, y las instrucciones para estos programas condensados eran casi imposibles de comprender para cualquier persona, excepto para sus autores. La vida útil de un programa estaba limitada al empleo de su programador.



Durante la segunda generación, los programadores empezaron a dividir sus programas en secciones que parecían conjuntos de trabajo, segmentos verdaderos, originalmente denominados roll in/roll out y ahora denominados **superposición**. El programa podía iniciarse con sólo la primera superposición cargada en la memoria. A medida que la primera sección se aproximaba a su término, podía instruir al sistema que colocara la segunda sección de código sobre la primera sección ya en la memoria. Luego era posible procesar la segunda sección. Cuando terminaba esa sección, podía llamar a que la tercera sección se sobrepusiera y así sucesivamente hasta que terminaba el programa. Algunos programas tenían múltiples superposiciones en la memoria al mismo tiempo.

Aunque el intercambio de superposiciones entre la memoria principal y el almacenamiento secundario lo llevaba a cabo el sistema, la tediosa tarea de dividir el programa en secciones la realizaba el programador. Fue el concepto de superposiciones lo que sugirió la paginación y la segmentación y condujo a la memoria virtual, que después se implementó a través de la paginación por demanda y esquemas de segmentación. Estos esquemas se comparan en la tabla 3.6.

(Tabla 3.6)

Comparación de las ventajas y desventajas de la memoria virtual con la paginación y la segmentación.

Memoria virtual con paginación	Memoria virtual con segmentación
Permite fragmentación interna dentro de los marcos de página	No permite fragmentación interna
No permite fragmentación interna	Permite fragmentación externa
Los programas se dividen en páginas del mismo tamaño	Los programas se dividen en segmentos de distinto tamaño que contienen agrupamientos lógicos de código
La dirección absoluta se calcula usando el número de página y desplazamiento	La dirección absoluta se calcula usando el número de segmento y desplazamiento
Requiere la PMT	Requiere la SMT

La segmentación permitía compartir el código entre los usuarios (consulte el ejercicio 12 para ver un ejemplo). Esto significa que el segmento compartido contiene: 1) un área donde se almacena código no intercambiable (denominado **código rentrante**) y 2) varias áreas de datos, una para cada usuario. En este esquema los usuarios comparten el código, que no es posible cambiar, y pueden modificar la información almacenada en sus propias áreas de datos según sea necesario sin afectar los datos almacenados en las áreas de datos de otros usuarios.

Antes de la memoria virtual, compartir significaba que las copias de los archivos estaban almacenadas en la cuenta de cada usuario. Esto les permitía cargar su propia copia y trabajar en ésta en cualquier momento. Este tipo de compartición creó mucho costo innecesario del sistema: el procesamiento extra de E/S al cargar las copias y el almacenamiento secundario extra necesario. Con la memoria virtual, estos costos se redujeron sustancialmente porque los programas y las subrutinas compartidos se cargan por demanda, reduciendo de manera satisfactoria los requerimientos de almacenamiento de la memoria principal (aunque esto se logra a expensas de la tabla de mapa de la memoria).

El uso de la memoria virtual requiere cooperación entre el administrador de la memoria (que sigue la pista de cada página o segmento) y el hardware del procesador (que emite una interrupción y resuelve la dirección virtual). Por ejemplo, cuando se necesita una página que aún no está en la memoria, se emite un fallo de página y el administrador de la memoria escoge un marco de página, carga la página y actualiza entradas en la tabla de mapa de la memoria y en las tablas de mapa de páginas.

La memoria virtual funciona bien en un entorno multiprogramación porque la mayor parte de los programas dedican bastante tiempo a esperar: esperan que se realicen las ope-

raciones de E/S; esperan que las páginas se intercambien hacia dentro o hacia fuera; y en un entorno de tiempo compartido, esperan el momento en que se alcanza su rebanada de tiempo (su turno para usar el procesador ha expirado). En un entorno multiprogramación, el tiempo de espera no se pierde y la CPU simplemente se mueve hacia otro trabajo; esta era la ventaja de las particiones.

La memoria virtual ha incrementado el uso de varias técnicas de programación. Por ejemplo, ayuda al progreso de grandes sistemas de software porque las piezas individuales pueden desarrollarse de manera independiente y vincularse más tarde.

La administración de la memoria virtual tiene varias ventajas:

- El tamaño de un trabajo ya no está restringido al tamaño de la memoria principal (o al espacio libre dentro de la memoria principal).
- La memoria se usa de manera más eficiente porque las únicas secciones de un trabajo almacenadas en la memoria son aquellas que se requieren de inmediato, mientras las que no se necesitan permanecen en el almacenamiento secundario.
- Permite una cantidad ilimitada de multiprogramación, que puede aplicar a muchos trabajos, como con las particiones dinámicas y estáticas, o muchos usuarios en un entorno de tiempo compartido.
- Elimina la fragmentación externa y minimiza la fragmentación interna al combinar segmentación y paginación (ocurre fragmentación interna en el programa).
- Permite compartir código y datos.
- Facilita la vinculación dinámica de segmentos de programa.

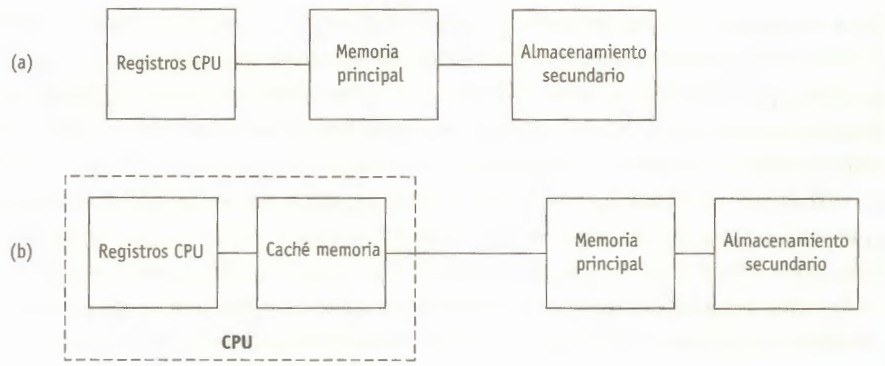
Las ventajas superan en exceso a las desventajas:

- Costos incrementados del hardware del procesador.
- Procesamiento extra incrementado para el manejo de interrupciones de página.
- Complejidad incrementada del software para impedir el thrashing.

Memoria caché

El almacenamiento en caché se basa en la idea de que el sistema puede utilizar una pequeña y costosa cantidad de memoria de alta velocidad para hacer que una gran cantidad de memoria más lenta y menos costosa trabaje más rápido que la memoria principal.

Debido a que la memoria caché es pequeña en tamaño (comparada con la memoria principal), puede utilizar más rápido los costosos chips de memoria y puede ser de 5 a 10 veces más rápida que la memoria principal y ajustarse a la velocidad de la CPU. Por tanto, cuando usamos frecuentemente datos o instrucciones almacenadas en la memoria caché, el tiempo de acceso a la memoria puede reducirse significativamente y la CPU puede ejecutar rápidamente las instrucciones, alcanzando un alto desempeño total del sistema de cómputo.



(Figura 3.17)
Comparación de (a) la ruta tradicional entre la memoria principal y la CPU utilizada por las primeras computadoras y (b) la ruta utilizada por las computadoras modernas para conectar la memoria principal y la CPU vía la memoria caché.

Como se muestra en la figura 3.17*a*, la arquitectura original de una computadora era tal que los datos y las instrucciones se transferían desde el almacenamiento secundario hacia la memoria principal y luego a registros de propósito especial para procesar, incrementando la cantidad de tiempo necesaria para completar un programa. Sin embargo, debido a que las mismas instrucciones se usaban de manera repetida en la mayor parte de los programas, los diseñadores de sistemas de cómputo consideraban que sería más eficiente si el sistema no usara un ciclo de memoria completo cada vez que se requería una instrucción o valor de datos. Los diseñadores encontraron que esto era posible si colocaban repetidamente datos usados en registros de propósito especial en lugar de hacerlo en la memoria principal, aunque descubrieron que esta técnica suponía trabajo extra para el programador. Además, desde el punto de vista del sistema, este uso de registros de propósito especial no era la solución óptima porque estos registros a menudo se requerían para almacenar temporalmente resultados de otros cálculos, y porque la cantidad de instrucciones usadas repetidamente a menudo superaba la capacidad de los registros de propósito especial.

Para resolver este problema, los sistemas de cómputo almacenan automáticamente datos en una unidad de memoria intermedia denominada **memoria caché**. Esto agrega una capa intermedia a la jerarquía original. La memoria caché puede entenderse como un intermediario entre la memoria principal y los registros de propósito especial, que pertenecen al dominio de la CPU, como se muestra en la figura 3.17*b*.

Un microprocesador típico cuenta con dos niveles de cachés: nivel 1 (L1) y nivel 2 (L2). La información entra al procesador a través de la unidad de interfaz de bus, que de inmediato envía una copia a la caché L2, que forma parte integral del microprocesador y está conectada directamente a la CPU. Una segunda copia se envía a un par de cachés L1, que están integradas directamente en la CPU. Una de estas cachés L1 está diseñada para almacenar instrucciones, mientras que la otra almacena datos que serán usadas por las instrucciones.Si una instrucción requiere más datos, se coloca en espera mientras el procesador los busca primero en la caché L1, y luego en la caché L2 más grande antes de buscar en la memoria principal.

Debido a que la caché L2 forma parte integral del microprocesador, los datos se mueven de 2 a 4 veces más rápido entre la CPU y la L2 que entre la CPU y la memoria principal.

Para comprender la relación entre la memoria principal y la memoria caché, considere la relación entre el tamaño de la Web y el tamaño de su archivo marcador privado. Si la memoria principal es la Web y la memoria caché es su archivo marcador privado, donde reúne las direcciones de las que usa más a menudo. Su marcador privado es pequeño y puede contener sólo 0.00001% de todas las direcciones en la red, pero la posibilidad de que usted visite pronto un sitio en la red que esté en su archivo marcador privado es alta. En consecuencia, el propósito de su archivo marcador es mantener sus direcciones a las que ha tenido acceso más recientemente a fin de que pueda acceder rápidamente a ellas, justo como el propósito de la memoria caché es mantener a la mano a los datos e instrucciones a los que se ha accedido más recientemente de modo que la CPU pueda tener acceso a ellos de manera repetida sin desperdiciar tiempo.

El movimiento de datos o instrucciones desde la memoria principal hasta la memoria caché usa un método semejante al que usan los algoritmos de paginación. Primero, la memoria caché se divide en bloques del mismo tamaño denominados ranuras. Luego, cuando la CPU solicita por primera vez una instrucción o datos desde una ubicación en la memoria principal, la instrucción solicitada y varias otras a su alrededor se transfieren desde la memoria principal hasta la memoria caché, donde se almacenan en una de las ranuras libres. El hecho de mover un bloque a la vez se basa en el principio de localidad, que establece que es muy probable que la siguiente solicitud de la CPU esté físicamente próxima a la que acaba de solicitarse. Además del bloque de datos transferidos, la ranura también contiene una etiqueta que indica la dirección de la memoria principal desde la que se copió el bloque. Cuando la CPU solicita información adicional desde esa ubicación en la memoria principal, primero se tiene acceso a la memoria caché, y si el contenido de una de las etiquetas en una ranura coincide con la dirección solicitada, entonces no es necesario tener acceso a la memoria principal.

El algoritmo para ejecutar una de estas transferencias "desde la memoria principal" es simple de implementar como sigue:

Algoritmo de transferencia de la memoria principal

1 La CPU coloca la dirección de una ubicación en la memoria en el registro de dirección de la memoria y solicita que desde esa dirección se recupere algún dato o una instrucción.

2 Se lleva a cabo una prueba para determinar si el bloque que contiene esta dirección ya está en una ranura caché:

Si SÍ, transferir la información al registro de la CPU: TERMINADO

Si NO, acceder a la memoria principal para el bloque que contiene la dirección solicitada

Asignar una ranura de caché libre al bloque

Realizar esto en paralelo: Transferir la información a la CPU
Cargar el bloque en la ranura

TERMINADO

Este algoritmo se vuelve más complicado si no hay ninguna ranura libre, lo que puede ocurrir porque el tamaño de la memoria caché es menor que el de la memoria principal, lo cual significa que ranuras individuales no pueden asignarse de manera permanente a bloques. Para tratar con esta contingencia, el sistema requiere una política para remplazo de bloques, que puede ser una semejante a las que se usan en el remplazo de páginas.

Al diseñar la memoria caché es necesario tomar en cuenta los cuatro factores siguientes:

- *Tamaño de la caché*. Estudios han demostrado que contar con cualquier caché, incluso uno pequeño, puede mejorar sustancialmente el rendimiento del sistema de cómputo.
- *Tamaño de bloque*. Debido al principio de localidad de referencia, a medida que crece el tamaño del bloque, la razón del número de referencias encontradas en la caché al número total de referencias es alta.
- *Algoritmo de remplazo de bloques*. Cuando todas las ranuras están ocupadas y es necesario llevar el caché un nuevo bloque, para remplazamiento debe seleccionarse el bloque con la menor probabilidad de ser usado en el futuro inmediato. Sin embargo, como se vio en la paginación, esto es casi imposible de pronosticar. Un curso de acción razonable es seleccionar un bloque que no se ha usado durante largo tiempo. En consecuencia, el algoritmo que suele aplicarse para el remplazo de bloque es LRU, que requiere un mecanismo de hardware para especificar la ranura menos usada recientemente.
- *Política de reescritura*. Cuando se modifica el contenido de un bloque que reside en la memoria, es necesario escribirlo nuevamente en la memoria principal antes de remplazarlo por otro bloque. Es necesario contar con una política de reescritura para determinar cuándo se lleva a cabo esta operación de escritura. Por otra parte, puede efectuarse cada que ocurre un cambio, lo cual puede incrementar el número de operaciones de escritura en la memoria, aumentando la sobrecarga. Por otra parte, puede realizarse sólo cuando el bloque se remplaza o cuando el proceso ha terminado, con lo cual pueden minimizarse la sobrecarga aunque el bloque en la memoria se dejaría en un estado inconsistente. Esto crearía problemas en entornos de multiprocesamiento y en el caso en que los módulos de E/S puedan tener acceso directo a la memoria principal.

La selección óptima del tamaño de la caché y del algoritmo de remplazo puede resultar en que del 80 al 90% de todas las solicitudes estén en la caché, constituyendo un sistema de memoria muy eficiente. Para medir la eficiencia, denominada tasa de aciertos de la caché (h), se usa para determinar el rendimiento de la memoria caché y representa el porcentaje del total de solicitudes por memoria que se encuentran en la caché:

$$\text{Tasa de aciertos} = \frac{\text{Número de solicitudes que hay en la caché}}{\text{Número total de solicitudes}} * 100$$

Por ejemplo, si el número total de solicitudes es 10 y 6 de estas se encuentran en la memoria caché, entonces la tasa de aciertos es 60 por ciento.

```
Tasa de aciertos = (6 / 10) * 100 = 60%
```

Por otra parte, si el número total de solicitudes es 100 y 9 de éstas se encuentran en la memoria caché, entonces la tasa de aciertos es sólo 9 por ciento.

```
Tasa de aciertos = (9 / 100) * 100 = 9%
```

Otra forma de medir la eficiencia de un sistema con memoria caché, en el supuesto de que el sistema siempre verifica primero la caché, consiste en calcular el tiempo promedio de acceso a la memoria con la fórmula siguiente:

```
AvgMemAccessTime = AvgCacheAccessTime + (1 - h) * AvgMainMemAccTime
```

Por ejemplo, si se sabe que el tiempo promedio de acceso a la caché es de 200 nanosegundos (ns) y que el tiempo promedio de acceso a la memoria principal es de 1000 ns, entonces un sistema con una tasa de aciertos de 60 por ciento tiene un tiempo promedio de acceso a la memoria de 600 ns:

```
AvgMemAccessTime = 200 + (1 - 0.60) * 1000 = 600 nsec
```

Un sistema con una tasa de aciertos de 9% tiene un tiempo promedio de acceso a la memoria de 1 110 ns:

```
AvgMemAccessTime = 200 + (1 - 0.09) * 1000 = 1110 nsec
```

El tamaño de la caché contribuye de manera importante a una respuesta global y es un elemento importante en el diseño del sistema.

Conclusión

El administrador de la memoria tiene la tarea de asignar memoria a cada trabajo por ejecutar, y reclamarlo una vez que se complete la ejecución.

Cada esquema analizado en los capítulos 2 y 3 fue diseñado para tratar un conjunto diferente de problemas críticos pero, como hemos visto, cuando se resolvieron algunos problemas, surgieron otros. La tabla 3.7 muestra la manera en que se comparan los esquemas de asignación de la memoria.

(Tabla 3.7)

Comparación de esquemas de asignación de la memoria analizados en los capítulos 2 y 3.

Esquema	Problema resuelto	Problema creado	Cambios en el software
Usuario único contiguo		El tamaño del trabajo esta limitado por el tamaño fisico de la memoria: la CPU esta a menudo detenida	Ninguno
Particiones fijas	Tiempo de CPU ocioso	Fragmentación interna; tamaño del trabajo limitado al tamaño de la partición	Se agregó el planificador del procesador; se agregó el administrador de protección
Particiones dinámicas	Fragmentación interna	Fragmentación externa	Ninguno
Particiones dinámicas reubicables	Fragmentación interna	Procesamiento extra por la compactación; tamaño del trabajo limitado al tamaño físico de la memoria	Algoritmo de compactación
Paginado	Necesidad de compactación	Memoria necesaria por las tablas; tamaño del trabajo limitado al tamaño físico de la memoria; regreso de la fragmentación interna	Algoritmos para manejar tablas del mapa de páginas
Paginación por demanda	Tamaño del trabajo ya no limitado por el tamaño de la memoria; más eficiencia en el uso de la memoria; permite multiprogramación a gran escala y compartición de tiempo	Mayor número de tablas; posibilidad de thrashing; procesamiento extra requerido por interrupciones de página; hardware de paginación necesario	Algoritmo de remplazo de páginas; algoritmo de búsqueda de páginas en el almacenamiento secundario
Segmentado	Fragmentación interna	Dificultad para administrar segmentos de longitud variable en el almacenamiento secundario; fragmentación externa	Paquete de vinculación dinámica; esquema de direccionamiento bidimensional
Segmentado/ paginación por demanda	Memoria virtual grande; carga de segmentos por demanda	Procesamiento extra por manejo de tabla; necesidad de memoria para tablas de páginas y de segmentos	Esquema de direccionamiento tridimensional

El administrador de la memoria es sólo uno de muchos administradores que integran el sistema operativo. Una vez que los trabajos se cargan en la memoria por medio de un esquema de asignación de la memoria, el administrador del procesador debe asignar el procesador para que ejecute cada trabajo de la manera más eficiente posible. En el capítulo siguiente veremos cómo se logra esto.

Términos clave

Anomalía Belady: Vea Anomalía FIFO.

Anomalía FIFO: Circunstancia inusual donde agregar más marcos de páginas provoca un incremento en las interrupciones de página cuando se usa una política de remplazo de páginas FIFO. También se denomina anomalía Belady.

Asignación de memoria segmentada: Esquema de asignación de la memoria basado en el concepto de dividir un trabajo de un usuario en grupos lógicos de código a fin de permitir almacenamiento no contiguo de programas durante la ejecución.

Asignación de memoria segmentada/ paginación por demanda: Esquema de asignación de la memoria basado en el concepto de dividir un trabajo de un usuario en agrupamientos lógicos de código y cargarlos en la memoria según sea necesario para minimizar fragmentación.

Ciclo de reloj: Lapso entre dos tics del reloj del sistema de cómputo.

Código reentrante: Código que pueden usar dos o más procesos al mismo tiempo; cada uno comparte la misma copia del código ejecutable pero tiene áreas de datos por separado.

Conjunto de trabajo: Colección de páginas que deben mantenerse en la memoria principal para cada proceso activo en un entorno de memoria virtual.

Desplazamiento: En un entorno de asignación de la memoria paginado o segmentado, la diferencia entre una dirección relativa de una página y la dirección verdadera en lenguaje de máquina. También se denomina *offset*.

Fallo de página: Tipo de interrupción del hardware provocada por una referencia a una página que no está en la memoria. El efecto consiste en sacar de la memoria una página y llevarla al almacenamiento secundario, de modo que sea posible llevar otra página a la memoria.

Intercambio de páginas: Proceso de sacar una página de la memoria principal y llevarla al almacenamiento secundario, de modo que en su lugar sea posible introducir otra página en la memoria.

Localidad de referencia: Comportamiento observado en muchos programas de ejecución donde las ubicaciones de la memoria referidas recientemente, y las que están próximas, tienen la probabilidad de ser referidas en el futuro próximo.

Manejador de fallo de página: Parte del administrador de la memoria que determina si en la memoria hay marcos de páginas vacíos de modo que la página solicitada puede copiarse de inmediato desde el almacenamiento secundario, o determina cuál página debe intercambiarse si todos los marcos de páginas están ocupados. También se denomina manejador de interrupción de página.

Marco de página: Sección individual de la memoria principal, de tamaño uniforme en la que es posible cargar una sola página sin provocar fragmentación externa.

Memoria asociativa: Denominación dada a varios registros, asignados a cada proceso activo, cuyo contenido asocia varios de los segmentos del proceso y números de página con sus direcciones en la memoria principal.

Memoria caché: Pequeña memoria rápida usada para mantener datos selectos y proporcionar acceso más rápido que sería imposible en otro caso.

Memoria virtual: Técnica que permite la ejecución de programas aún cuando no estén almacenados completamente en la memoria.

Offset: Vea desplazamiento.

Página: Sección de tamaño fijo de un trabajo de un usuario que corresponde a marcos de página en la memoria principal.

Paginación por demanda: Esquema de asignación de la memoria que carga una página del programa en la memoria cuando es necesaria para procesamiento.

Política de remplazo de páginas: Algoritmo usado por sistemas con memoria virtual para decidir cuál página o segmento eliminar de la memoria principal cuando se necesita un marco página y la memoria está llena.

Política de remplazo de páginas de reloj: Variante de la política LRU que elimina de la memoria principal las páginas que muestran la menor cantidad de actividad durante ciclos de reloj recientes.

Política menos usada recientemente (LRU): Política de remplazo de páginas que elimina de la memoria principal las páginas que muestran la menor cantidad de actividad reciente.

Política primero en entrar, primero en salir (FIFO): Política de remplazo de páginas que elimina de la memoria principal las páginas que llegaron primero.

Resolución de la dirección: Proceso de cambiar la dirección de una instrucción o un dato a la dirección en la memoria principal en que debe cargarse o reubicarse.

Sector: División en una pista de un disco, algunas veces denominada "bloque". Las pistas se dividen en sectores durante el proceso de formateo.

Segmento: Sección de tamaño variable de un trabajo de un usuario que contiene un agrupamiento lógico de código.

Subrutina: También se denomina "subprograma"; segmento de un programa capaz de efectuar una función específica. Algunas subrutinas pueden reducir el tiempo de programación cuando una función específica se requiere en más de un punto en un programa.

Tabla de trabajo (JT): Tabla en la memoria principal que contiene dos valores para cada trabajo activo: el tamaño del trabajo y la ubicación en la memoria donde está almacenada la tabla de mapa.

Tabla de mapa de la memoria: Tabla en la memoria principal que contiene tantas entradas como el número de marcos de página que hay, y enumera la ubicación y el estado libre/ocupado para cada una.

Tabla de mapa de página (PMT): Tabla en la memoria principal con la información vital para cada página, incluyendo el número de página y la dirección en la memoria del marco de página con correspondiente.

Tabla de mapa de segmentos: Tabla en la memoria principal con la información vital para cada segmento, incluyendo el número de segmento y la dirección correspondiente en la memoria.

Thrashing: Fenómeno en un sistema con memoria virtual donde una cantidad excesiva de intercambio de páginas de un lado a otro entre la memoria principal y el almacenamiento secundario resulta en procesamiento extra más elevado y poco trabajo útil.

Búsquedas de interés

- Proveedores de tarjetas de memoria
- Memoria virtual
- Conjunto de trabajo
- Memoria caché
- Thrashing

Ejercicios

Temas de investigación

A. Los tamaños de las páginas y de las páginas con marcos, a menudo son idénticos. Busque fuentes académicas para descubrir los tamaños típicos de página, los factores que son considerados por los desarrolladores de sistemas operativos cuando establecen estos tamaños y si las consideraciones de hardware son importantes o no. Cite sus fuentes.

B. El núcleo de memoria consiste de la CPU y de la unidad aritmética lógica pero no contiene la memoria caché. En internet o usando fuentes académicas, investigue el diseño de memoria de núcleos múltiples e identifique los papeles desempeñados por la memoria caché de Nivel 1 y Nivel 2. ¿La implementación de memoria caché en chips de núcleos múltiples varía de un fabricante a otro? Explique y cite sus fuentes.

Ejercicios

1. Explique las circunstancias en las que preferiría una sobre la otra.
2. Describa cómo la función de la tabla del mapa de página difiere en el paginado contra la asignación de memoria segmentada/paginación por demanda.
3. Una vez que el trashing es detectado, qué puede hacer el sistema operativo para detenerlo.

4. Dado que la memoria principal está compuesta por tres marcos de página para uso público y que un programa de siete páginas (con páginas a, b, c, d, e, f, g) solicita páginas en el siguiente orden:

   a, b, a, c, d, a, e, f, g, c, b, g

   a. Use el algoritmo FIFO para eliminar páginas y elabore un análisis de trazo de página indicando fallos de página con asteriscos (*). Luego, calcule las relaciones de fallos y de aciertos.

   b. Incremente el tamaño de la memoria de modo que contenga cuatro marcos de páginas para uso público. Use las mismas solicitudes de páginas que antes y el esquema FIFO para hacer otro análisis de trazo de página y calcule las relaciones de fallos y de aciertos.

   c. ¿El resultado coincide con su intuición? Explique.

5. Dado que la memoria principal está compuesta por tres marcos de página para uso público y que un programa solicita páginas en el siguiente orden:

   a, d, b, a, f, b, e, c, g, f, b, g

   a. Use el algoritmo FIFO para eliminar páginas y elabore un análisis de trazo de página indicando fallos de página con asteriscos (*). Luego, calcule las relaciones de fallos y de aciertos.

   b. Use el algoritmo LRU para eliminar páginas y elabore un análisis de trazo de página indicando fallos de página con asteriscos (*). Luego, calcule las relaciones de fallos y de aciertos.

   c. ¿Cuál es mejor? ¿Por qué considera que es mejor? ¿Puede hacer un planteamiento general a partir de este ejemplo? ¿Por qué sí? o ¿por qué no?

6. Definamos "más recientemente usada" (MRU) como un algoritmo para eliminar páginas que elimina de la memoria la página más recientemente usada. Haga un análisis de traza de página usando tres marcos de página y las solicitudes de páginas del ejercicio anterior. Calcule las relaciones de fallos y de aciertos y explique por qué piensa que la MRU es o no un sistema de asignación de memoria viable.

7. Examinando los bits de referencia para las seis páginas mostradas en la figura 3.11, identifique cuál de las seis páginas fue referenciada con más frecuencia como la instantánea de tiempo [mostrada en (e)]. ¿Qué página fue referenciada con menos frecuencia? Explique su respuesta.

8. Para implementar LRU, cada página requiere un bit de referencia. Si se desea implementar un algoritmo para eliminar páginas menos frecuentemente usadas (LFU), donde la página que se usó menos debe eliminarse de la memoria, ¿qué se requiere agregar a las tablas? ¿Qué modificaciones deben hacerse al software para atender este nuevo algoritmo?

9. Calcule la relación de aciertos usando la fórmula presentada al final de este capítulo suponiendo que el número total de solicitudes es 2056 y 1029 de aquellas que se encuentran en la caché.

10. Suponiendo una relación de aciertos de 67%, calcule el tiempo promedio de acceso a la memoria usando la fórmula presentada en este capítulo si el tiempo promedio

de acceso a la caché es 200 ns y el tiempo promedio de acceso a la memoria principal es 500 ns.

11. Suponiendo una relación de aciertos de 31%, calcule el tiempo promedio de acceso a la memoria usando la fórmula presentada en este capítulo si el tiempo promedio de acceso a la caché es 125 ns y el tiempo promedio de acceso a la memoria principal es 300 ns.
12. Usando un sistema paginado de asignación de memoria con un tamaño de página de 2048 bytes y una página con marco de igual tamaño y suponiendo que la entrada de un archivo de datos es 25 600, calcule cuántas páginas deben ser creadas por el archivo. Calcule el tamaño de cualquier fragmentación resultante. Explique si esta situación resulta en una fragmentación interna, en una fragmentación externa o ambas.

Ejercicios avanzados

13. Dado que la memoria principal está compuesta por cuatro marcos de página para uso público, use la tabla siguiente para contestar todas las partes de este problema:

Marcos de página	Tiempo en que se cargó	Tiempo en que se refirió por última vez	Bit de referencia	Bit modificado
0	126	279	0	0
1	230	280	1	0
2	120	282	1	1
3	160	290	1	1

    a. ¿Cuál es el marco de página cuyo contenido debe intercambiarse aplicando FIFO?
    b. ¿Cuál es el marco de página cuyo contenido debe intercambiarse aplicando LRU?
    c. ¿Cuál es el marco de página cuyo contenido debe intercambiarse aplicando MRU?
    d. ¿Cuál es el marco de página cuyo contenido debe intercambiarse aplicando LFU?
14. Dadas tres subrutinas con 700, 200 y 500 palabras cada una, si se usa segmentación, entonces la memoria total requerida es la suma de los tres tamaños (en caso que se carguen las tres subrutinas). No obstante, si se usa paginación, entonces se pierde algo de espacio de almacenamiento porque las subrutinas rara vez llenan por completo la última página, lo cual resulta en fragmentación interna. Determine la cantidad total de memoria desperdiciada debido a fragmentación interna cuando las tres subrutinas se cargan en la memoria usando cada uno de los siguientes tamaños de página:
    a. 100 palabras.
    b. 600 palabras.
    c. 700 palabras.
    d. 900 palabras.

15. Dada la siguiente tabla de mapa de segmentos para dos trabajos:

SMT para el trabajo 1 Número de segmento	Ubicación en la memoria
0	4096
1	6144
2	9216
3	2048
4	7168

SMT para el trabajo 2 Número de segmento	Ubicación en la memoria
0	2048
1	6144
2	9216

a. ¿Cuáles segmentos, en caso de haber alguno, se comparten entre dos trabajos?

b. Si el segmento ahora ubicado en 7168 se intercambia y luego se vuelve a cargar en 8192, y el segmento que ahora está en 2048 se intercambia y vuelve a cargarse en 1024, ¿cómo se verán las nuevas tablas de segmentos?

Ejercicios de programación

16. Este problema estudia el efecto de cambiar el tamaño de las páginas en un sistema de demanda de paginación.

    La siguiente secuencia de solicitudes por palabras del programa se tomó de un programa de 460 palabras: 10, 11, 104, 170, 309, 185, 245, 246, 434, 458, 364. La memoria principal puede contener un total de 200 palabras para este programa y el tamaño del marco de página coincide con el tamaño de las páginas en las que se ha dividido el programa.

    Calcule los números de página según el tamaño de página, divídalo entre el tamaño de página y el cociente proporciona el número de página. El número del marco de página en la memoria es el número total, 200, dividido entre el tamaño de página. Por ejemplo, en el problema (a) el tamaño de página es 100, lo cual significa que las solicitudes 10 y 11 están en la página 0 y que las solicitudes 104 y 170 están en la página 1. El número del marco de página es dos.

    a. Encuentre la frecuencia de aciertos para la lista de solicitudes usando un algoritmo de remplazo FIFO y un tamaño de página de 100 palabras (hay dos páginas con marcos).

b. Encuentre la frecuencia de aciertos para la lista de solicitudes usando un algoritmo de remplazo FIFO y un tamaño de página de 20 palabras (10 páginas; de la 0 a la 9).

c. Encuentre la frecuencia de aciertos para la lista de solicitudes usando un algoritmo de remplazo FIFO y un tamaño de página de 200 palabras.

d. ¿Qué indican sus resultados? ¿Puede hacer un planteamiento general sobre lo que ocurre cuando los tamaños de página se reducen a la mitad o se duplican?

e. ¿Hay alguna otra ventaja de sobreescritura al usar páginas más pequeñas? ¿Cuáles son los factores de offset? Recuerde que la transferencia de 200 palabras de información requiere menos tiempo que el doble por la transferencia de 100 palabras debido a la forma en que operan los dispositivos del almacenamiento secundario (la tasa de transferencia es más alta que la tasa de acceso [buscar/encontrar]).

f. Repita los incisos *a* a *c* anteriores, usando una memoria principal de 400 palabras. El tamaño de cada marco de página coincide nuevamente con el tamaño de la página.

g. ¿Qué ocurrió cuando al programa se proporcionó más memoria? ¿Puede hacer algún planteamiento general sobre este hecho? ¿Qué cambios esperaría ver si la lista de solicitudes fuera mucho más larga, como sería en la vida real?

h. ¿Esta lista de solicitudes, podría ocurrir durante la ejecución de un programa real?

i. ¿Esperaría que la relación de aciertos de un programa real en condiciones semejantes sea más alta o más baja que la de este problema?

17. Dada la siguiente información para un programa en lenguaje ensamblador:

Tamaño del trabajo: 3126 bytes
Tamaño de página: 1042 bytes

instrucción en la ubicación de la memoria 532:	`Cargar 1,2098`
instrucción en la ubicación de la memoria 1156:	`Sumar 1,2087`
instrucción en la ubicación de la memoria 2086:	`Restar 1,1052`
dato en la ubicación de la memoria 1052:	`015672`
dato en la ubicación de la memoria 2098:	`114321`
dato en la ubicación de la memoria 2087:	`077435`

a. ¿Cuántas páginas se requieren para almacenar todo el trabajo?

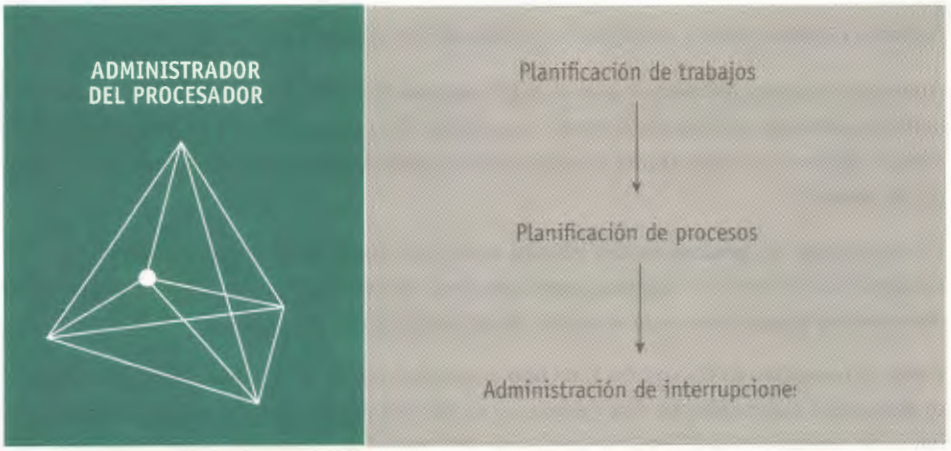
b. Calcule el número de página y el desplazamiento para cada una de las direcciones de bytes donde se almacenan los datos. (Recuerde que la numeración de las páginas inicia en cero.)

c. Determine si el número de página y/o el desplazamiento son legales para este trabajo.

d. Explique por qué el número de página y/o el desplazamiento pueden no ser legales para este trabajo.

e. Indique cuál acción podría emprender el sistema operativo cuando un número de página o un desplazamiento no es legal.

Capítulo 4 Administración del procesador



"La naturaleza actúa por avances. . . Va y regresa; después avanza más, luego avanza el doble de lo que retrocede y luego avanza más hacia delante que nunca."

Blaise Pascal (*1623-1662*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- La diferencia entre planificación de trabajos y planificación de procesos, y cómo se relacionan.
- Las ventajas y las desventajas de los algoritmos de planificación de procesos expropiativos contra los no expropiativos.
- Los objetivos de las políticas de planificación de procesos en CPU de núcleo único.
- Hasta seis algoritmos de planificación de procesos diferentes.
- El rol de las interrupciones internas y de las tareas efectuadas por el manejador de interrupciones.

El administrador del procesador es responsable de la asignación del procesador para ejecutar los trabajos de entrada, y las tareas para estos trabajos. En este capítulo veremos cómo un administrador del procesador administra una CPU única para lograr lo anterior.

Visión general

En un sistema simple, uno con un usuario único y un procesador, este sólo está ocupado cuando la CPU ejecuta un trabajo. Sin embargo, cuando en el sistema hay muchos usuarios con muchos trabajos (lo cual se denomina entorno de multiprogramación), el procesador debe asignarse a cada trabajo de manera justa y eficiente, lo que puede ser una tarea complicada, como veremos en este capítulo dedicado a los sistemas de un procesador simple. Aquellos de procesadores múltiples son analizados en el capítulo 6.

En muchos sistemas operativos, el tiempo ocioso entre trabajos especificados por el usuario se usa para procesar tareas de rutina de segundo plano. Así, incluso si un usuario no está ejecutando aplicaciones, la CPU puede estar ocupada ejecutando otras tareas.

Antes de comenzar, definiremos con claridad algunos términos. Un **programa** es una unidad inactiva, como un archivo almacenado en un disco. Un programa no es un proceso. Para un sistema operativo, un programa o trabajo es una unidad de trabajo que ha sido presentado por el usuario.

Por otra parte, un proceso es una entidad activa que requiere un conjunto de recursos, incluyendo un procesador y registros especiales para efectuar su función. Un proceso, también denominado **tarea**, es una sola instancia de un programa en ejecución.

Como se mencionó en el capítulo 1, un **hilo** es una porción de un proceso que puede ejecutarse de manera independiente. Por ejemplo, si su sistema permite procesos que tienen un hilo único de control y usted desea ver una serie de fotografías de amigos en un sitio en la red, puede instruir al navegador para que establezca una conexión entre los dos sitios y descargue una fotografía a la vez. No obstante, si su sistema permite que los procesos tengan multihilos de control, entonces usted puede solicitar varias fotografías al mismo tiempo y el navegador establecerá múltiples conexiones y descargará varias fotografías al mismo tiempo.

El **procesador**, también conocido como CPU (unidad central de procesamiento), es la parte de la computadora que realiza los cálculos y ejecuta los programas.

La **multiprogramación** requiere que el procesador sea asignado a cada trabajo o a cada proceso durante un periodo y que sea liberado en un momento idóneo. Si el procesador se libera durante la ejecución de un programa, esto debe hacerse de modo que pueda reiniciarse más tarde lo más fácilmente posible. Se trata de un procedimiento delicado. Para demostrarlo, consideraremos un ejemplo de la vida diaria.

He aquí al lector, confiado en que puede hacer funcionar un juguete a pesar de la advertencia de que se requiere ensamblarlo. Armado con las instrucciones y toneladas de paciencia, usted se embarca en esta tarea: leer las instrucciones, reunir las herramientas necesarias, seguir cada paso según se indica y terminar con el producto final.

El primer paso consiste en unir la parte A con la parte B con un tornillo de 2 pulgadas, y cuando termina esta tarea verifica el paso 1. Inspirado por su éxito, prosigue con el paso 2 y luego con el paso 3. Apenas acaba de completar el paso 3 cuando un vecino se lastima al trabajar con una herramienta eléctrica y grita pidiendo ayuda.

Usted verifica rápidamente las instrucciones del paso 3, de modo que sabe dónde se quedó; luego, deja sus herramientas y corre en auxilio del vecino. Después de todo, las necesidades inmediatas de alguien son más importantes que su éxito momentáneo con el juguete. Ahora

se encuentra implicado en una tarea muy diferente: seguir las instrucciones de un botiquín de primeros auxilios para usar vendas y antisépticos.

Una vez que la herida ha sido atendida exitosamente, usted vuelve a su trabajo previo. A medida que recoge sus herramientas, se refiere a las instrucciones y observa que debió empezar con el paso 4. Luego, prosigue con este proyecto hasta que lo ha terminado por completo.

En terminología de sistemas operativos, usted desempeñó el papel de la CPU o *procesador*. Había dos *programas* o *tareas*: una era la misión de ensamblar el juguete y la segunda, atender la herida del vecino. Cuando estaba ensamblando el juguete (trabajo A), cada paso seguido era un *proceso*. La llamada por ayuda fue una *interrupción* y cuando dejó el juguete para atender a su amigo herido, lo dejó por un *programa con prioridad superior*. Cuando usted fue interrumpido, realizó un *cambio de contexto* cuando marcó el paso 3 como la última instrucción completada y dejó sus herramientas. Atender la herida del vecino se volvió el trabajo B. Mientras usted ejecutaba las instrucciones de primeros auxilios, cada uno de los pasos que ejecutó de nuevo era un *proceso*. Y, por supuesto, cuando cada trabajo se completaba, el trabajo estaba *acabado* o terminado.

El administrador del procesador identifica la serie de eventos como sigue:

`obtener la entrada para el trabajo A`	(encuentra las instrucciones en la caja)
`identificar los recursos`	(reúne las herramientas necesarias)
`ejecutar el proceso`	(sigue cada paso según se indica)
`interrumpir`	(llamada del vecino)
`cambio de contexto a trabajo B`	(señala el sitio en que se quedó en las instrucciones)
`obtener la entrada para el trabajo B`	(encuentra su libro de primeros auxilios)
`identificar los recursos`	(reúne los medicamentos necesarios)
`ejecutar el proceso`	(sigue cada paso indicado en el botiquín)
`termina trabajo B`	(regresa a casa)
`cambio de contexto a trabajo A`	(se prepara para reanudar el ensamblado del juguete)
`reanudar la ejecución del proceso interrumpido`	(sigue los pasos restantes según se indica)
`termina trabajo A`	(apaga el juguete terminado)

Como hemos mostrado, un solo procesador puede compartirse entre varios trabajos o varios procesos, pero, y sólo si, el sistema operativo cuenta con una política de planificación, así como con un algoritmo de planificación, para determinar cuándo detener el trabajo con una actividad y proceder con el otro.

En este ejemplo, el algoritmo de planificación estaba basado en prioridades: usted trabajaba en los procesos pertenecientes al trabajo A hasta que se presentó un trabajo con prioridad superior. Aunque en este caso se trataba de un buen algoritmo, un algoritmo de planificación basado en prioridades no siempre es el mejor, como veremos más tarde en este capítulo.

Acerca de las tecnologías multinúcleo

Una CPU de doble núcleo, cuádruple núcleo, u otra CPU multinúcleo tiene más de un procesador (también denominado núcleo) en el chip de la computadora. La ingeniería multinúcleo fue impulsada debido a los problemas provocados por nanotransistores y su colocación ultracerrada en un chip de computadora. Aunque chips con millones de transistores ubicados bastante próximos entre sí ayudaron a incrementar drásticamente el rendimiento del sistema, la estrecha proximidad de estos transistores también aumentó la fuga de corriente y la cantidad de calor generada por el chip.

Una solución consistía en crear un chip único (una pieza de silicio) con dos o más núcleos del procesador. En otras palabras, sustituyeron a un gran procesador único con dos procesadores de la mitad de tamaño, o por cuatro procesadores, cada uno con un tamaño igual a la cuarta parte del tamaño del procesador grande. Este diseño permitió que el chip del mismo tamaño produjera menos calor y ofreciera la oportunidad de permitir la realización de múltiples cálculos al mismo tiempo.

Para el administrador del procesador, los núcleos múltiples son más difíciles de gestionar que un solo núcleo. En el capítulo 6 analizaremos el procesamiento con núcleos múltiples.

Planificación de trabajos contra planificación de procesos

El administrador de procesador es una composición de dos administradores: uno a cargo de la planificación de trabajos y otro a cargo de la planificación de procesos. Se denominan **planificador de trabajos** y **planificador de procesos**.

Normalmente, un usuario considera un trabajo como una serie de pasos globales de un trabajo: compilación, cargado y ejecución; o como un paso que abarca todo: ejecución. Sin embargo, la planificación de trabajos en realidad es manejada en dos niveles por la mayor parte de los sistemas operativos. Si volvemos al ejemplo presentado antes, vemos que existe una jerarquía entre el planificador de trabajos y el planificador de procesos.

La planificación de los dos trabajos, ensamblar el juguete y vendar la herida estaba basada en una prioridad de primero en llegar, primero en servirse. Cada trabajo es iniciado por el planificador de trabajos con base en ciertos criterios. Una vez que se ha elegido un trabajo para su ejecución, el planificador de procesos determina cuándo debe ejecutarse cada paso, o conjunto de pasos: una decisión que también está basada en ciertos criterios. Cuando usted empezó a ensamblar el juguete, el planificador de procesos pudo escoger cada paso en las instrucciones de ensamblado para ejecutarlo.

En consecuencia, cada trabajo (o programa) pasa a través de una jerarquía de administradores. Puesto que el primero que encuentra es el planificador de trabajos, éste también se denomina **planificador de alto nivel**. Sólo le concierne seleccionar trabajos de una cola de trabajos de entrada y colocarlos en la cola de procesos, ya sea por lotes o interactiva, con base en las características de cada trabajo. El objetivo del planificador de trabajos es poner los trabajos en una secuencia que usará todos los recursos del sistema de la manera más completa posible.

Esta función es importante. Por ejemplo, si el planificador de trabajos escoge varios trabajos para ejecutarlos consecutivamente y hubiera bastantes operaciones de E/S, entonces los dispositivos de E/S estarían muy ocupados. La CPU podría estar ocupada manejando las operaciones de E/S (en caso de no usarse un controlador de E/S), de modo que podría realizarse muy poco de computación. Por otra parte, si el planificador de trabajos escoge varios trabajos consecutivos que requieren mucho trabajo de computación, entonces la CPU podría estar muy ocupada haciendo esto. Los dispositivos de E/S estarían ociosos esperando solicitudes de E/S. En consecuencia, el planificador de trabajos se esfuerza por contar con una mezcla equilibrada de trabajos que requieren grandes cantidades de interacción de E/S y trabajos que requieren grandes cantidades de computación. Su objetivo es hacer que la mayor parte de los componentes del sistema de cómputo estén ocupados la mayor parte del tiempo.

Planificador de procesos

Casi todo este capítulo está dedicado al planificador de procesos porque una vez que el planificador de trabajos coloca un trabajo en la cola LISTOS, el planificador de procesos se hace cargo. Determina cuáles trabajos obtienen la CPU, cuándo y durante cuánto tiempo. También decide cuándo es necesario interrumpir el procesamiento, determina cuáles colas debe mover el trabajo durante su ejecución, y reconoce cuándo ha concluido un trabajo y debe terminarse.

El planificador de procesos es el **planificador de bajo nivel** que asigna la CPU para ejecutar los procesos de aquellos trabajos que el planificador de trabajos ha colocado en la cola LISTOS. Esta función se vuelve crucial cuando debe orquestarse el procesamiento de varios trabajos: justo como cuando usted tuvo que dejar de lado el ensamblado del juguete para ayudar a su vecino.

Para planificar la CPU, el planificador de procesos aprovecha un rasgo común entre la mayor parte de los programas de computadora: que se alternan entre ciclos de la CPU y ciclos de E/S. Observe que el siguiente trabajo tiene un ciclo de CPU relativamente largo y dos ciclos de E/S muy breves:

```
{
printf(""\nIntroducir el primer entero: ");
scanf("%d", &a);
printf(""\nIntroducir el segundo entero: );
scanf("%d", &b);
```

} Ciclo de E/S

```
c = a+b
d = (a*b)-c
e = a-b
f = d/e
```

} Ciclo de CPU

```
printf("\n a+b= %d", c);
printf("\n (a*b)-c = %d", d);
printf("\n a-b = %d", e);
printf("\n d/e = %d", f);
}
```

} Ciclo de E/S

La entrada de datos (el primer ciclo de E/S) y la impresión (el último ciclo de E/S) son breves en comparación con el tiempo necesario para realizar los cálculos (el ciclo de CPU).

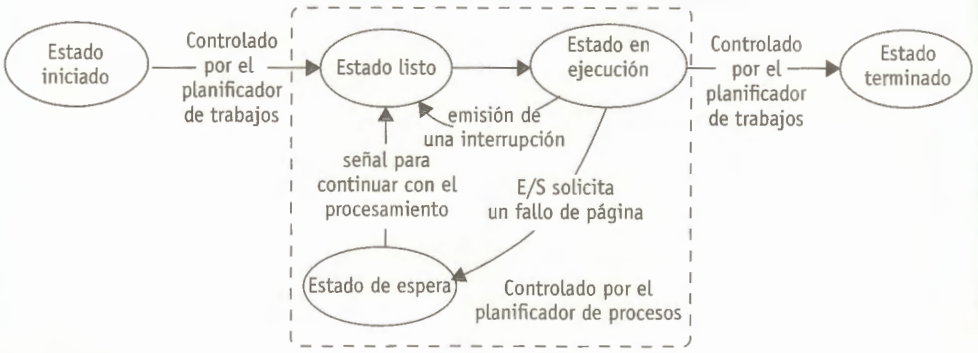
(Figura 4.1)

Distribución de la duración de los ciclos de la CPU. Esta distribución muestra que un mayor número de trabajos solicitan ciclos cortos de la CPU (los picos de frecuencia cerca del extremo inferior del eje del ciclo de la CPU), y menos trabajos requieren de largos ciclos de CPU

Aunque la duración y frecuencia del ciclo de CPU varían de un programa a otro, hay algunas tendencias generales que pueden aprovecharse cuando se selecciona un algoritmo de planificación. Por ejemplo, los **trabajos limitados por E/S** (como la impresión de una serie de documentos) tienen ciclos de CPU muy breves y largos ciclos de E/S, mientras los trabajos **limitados por CPU** (como encontrar los primeros 300 números primos) tienen ciclos de CPU largos y ciclos de E/S más cortos. El efecto total de todos los ciclos de CPU, de trabajos limitados por E/S y trabajos limitados por CPU, se aproxima a una curva de distribución de Poisson como se muestra en la figura 4.1.

En un entorno altamente interactivo hay una tercera capa del administrador de procesos denominada **planificador de nivel medio**. En algunos casos, especialmente cuando el sistema está sobrecargado, el planificador de nivel medio encuentra ventajoso eliminar trabajos activos de la memoria para reducir el grado de multiprogramación, lo cual permite que los trabajos se completen más rápido. Los trabajos que se intercambian hacia fuera y que terminan intercambiándose de vuelta son administrados por el planificador de nivel medio.

En un entorno de usuario único no hay distinción entre la planificación de trabajos y de procesos porque sólo un trabajo es activo en el sistema en cualquier instante. Así, la CPU y todos los otros recursos están dedicados a ese trabajo, y a cada uno de los procesos a su vez, hasta que se completa el trabajo.



(Figura 4.2)

Un trabajo (o proceso) típico cambia de estado a medida que se mueve a través del sistema desde INICIADO hasta TERMINADO.

Estado del trabajo y del proceso

A medida que un trabajo se mueve a través del sistema siempre se encuentra en uno de cinco estados (o por lo menos en tres) cuando cambia de iniciado a LISTO a en EJECUCIÓN a ESPERA y finalmente a TERMINADO como se muestra en la figura 4.2. Lo anterior se denomina **estados del trabajo** o **estados del proceso**.

A continuación se describe la manera en que el trabajo cambia de estado cuando un usuario presenta un trabajo, ya sea en modo por lotes o interactivo. Una vez que el sistema acepta el trabajo, éste se coloca en INICIADO y se pone en una cola. En algunos sistemas un *spoiler* de trabajos (o controlador de disco) crea una tabla con las características de cada trabajo en la cola y anota las características importantes del trabajo, como un estimado del tiempo de CPU, la prioridad, dispositivos de E/S especiales requeridos y memoria máxima requerida. Esta tabla la usa el planificador de trabajos para decidir cuál trabajo es el siguiente a ejecutar.

A partir de INICIADO, el trabajo se mueve a LISTO una vez que se encuentra listo para su ejecución pero está en espera de la CPU. En algunos sistemas, el trabajo (o proceso) puede colocarse en el estado LISTO directamente. En EJECUCIÓN, por supuesto, significa que el trabajo está en procesamiento. En un sistema de procesador único, este es un "trabajo" o proceso. En ESPERA significa que el trabajo no puede continuar sino hasta que se asigne un recurso específico o haya terminado una operación de E/S. Una vez completado, el trabajo está TERMINADO y se devuelve al usuario.

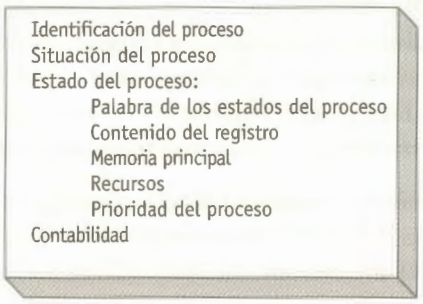
En un sistema multiprogramación, la CPU debe asignarse a muchos trabajos, cada uno con numerosos procesos, haciendo aún más complicada la administración del procesador. (La multiprogramación se analizará en el capítulo 6.)

La transición del estado de un trabajo o proceso a otro estado es iniciada por el planificador de trabajos o el planificador de procesos:

- La transición de INICIADO a LISTO la inicia el planificador de trabajos según alguna política predefinida. En este punto se verifican la disponibilidad de suficiente memoria principal y cualquier dispositivo solicitado.
- La transición de LISTO a en EJECUCIÓN la maneja el planificador de procesos según algún algoritmo predefinido (es decir, FCFS, SJN, planificación de prioridad, SRT, o round robin, todos los cuales se analizarán en breve).
- La transición de en EJECUCIÓN a LISTO la maneja el planificador de procesos según algún tiempo límite o algún otro criterio; por ejemplo, una interrupción de prioridades.
- La transición de en EJECUCIÓN a en ESPERA la maneja el planificador de procesos y es iniciada por una instrucción en el trabajo, como un comando para READ, WRITE o alguna otra solicitud de E/S, o una que requiera la búsqueda de una página.
- La transición de en ESPERA a LISTO la maneja el planificador de procesos y es iniciada por una señal desde el administrador de dispositivos de E/S de que la solicitud de E/S ha sido satisfecha y el trabajo puede continuar. En el caso de una búsqueda de una página, el manejador de fallo de página indica que la página se encuentra ahora en la memoria y el procesador puede colocarse en la cola de LISTO.
- Por último, la transición de en EJECUCIÓN a TERMINADO es iniciada por el planificador de procesos o por el planificador de trabajos cuando: 1) el trabajo ha sido completado exitosamente y termina su ejecución o 2) el sistema operativo indica que ha ocurrido un error y que el trabajo está terminando de manera prematura.

Bloques de control de proceso

Cada proceso en el sistema está representado por una estructura de datos denominada **bloque de control de proceso** (PCB) que realiza la misma función que el pasaporte de un viajero. El PCB (ilustrado en la figura 4.3) contiene la información básica sobre el trabajo, incluyendo qué es, a dónde va, cuánto de su procesamiento se ha completado, dónde está almacenado y cuánto ha dedicado a la utilización de recursos.



(Figura 4.3)
Contenido del bloque de control de procesos de cada trabajo.

Identificación del proceso

Cada trabajo está identificado de manera única por la identificación del usuario y un apuntador que lo conecta con su descriptor (proporcionado por el planificador de trabajos cuando el trabajo entra por primera vez en el sistema y se coloca en INICIADO).

Situación del proceso

Esto indica el estado actual del trabajo: INICIADO, LISTO, en EJECUCIÓN o en ESPERA, así como los recursos responsables de ese estado.

Estado del procesador

Esto contiene toda la información necesaria para indicar el estado actual del trabajo como:

- *Palabra del estado del proceso*: Contenido del contador de instrucciones y del registro cuando el trabajo no estaba en ejecución, sino que estaba en INICIADO o está LISTO o en ESPERA. Si el proceso está en EJECUCIÓN, esta información se deja indefinida.
- *Contenido de los registros*: El contenido de los registros si el trabajo ha sido interrumpido y está esperando para reanudar el procesamiento.
- *Memoria principal*: Información pertinente, incluyendo la dirección dónde está almacenado el trabajo y, en caso de memoria virtual, la asignación entre ubicaciones de la memoria virtual y la memoria física.

- *Recursos*: Información sobre todos los recursos asignados a este trabajo. Cada recurso tiene un campo de identificación que indica su tipo y un campo que describe los detalles de su asignación, como la dirección del sector en un disco. Estos recursos pueden ser unidades de hardware (unidades de disco o impresoras, por ejemplo) o archivos.
- *Prioridad del proceso*: Usada por sistemas que usan un algoritmo de planificación de prioridad para elegir cuál trabajo será ejecutado a continuación.

Contabilidad

Esto contiene información usada principalmente para propósitos de contabilización y medición del rendimiento. Indica qué tipo de recursos usó el trabajo y durante cuánto tiempo. Algunos datos típicos son los siguientes:

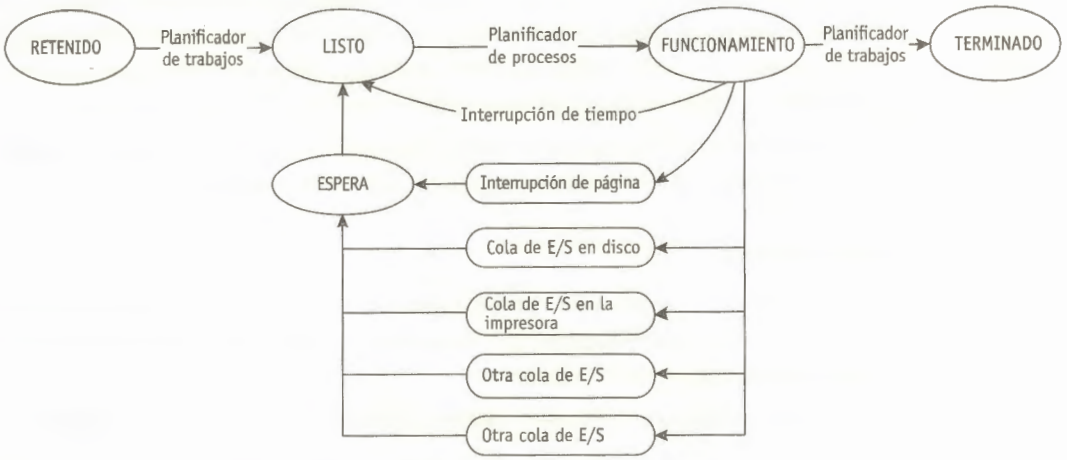
- Cantidad de tiempo de CPU usado desde el principio hasta el final de su ejecución.
- Tiempo total que el trabajo estuvo en el sistema hasta que salió.
- Ocupación del almacenamiento secundario: cuánto tiempo permaneció el trabajo en la memoria hasta que terminó su ejecución. Esto suele ser una combinación de tiempo y espacio usados; por ejemplo, en un sistema paginado puede registrarse en unidades de páginas-segundos.
- Almacenamiento secundario usado durante la ejecución. Esto, también, se registra como una combinación de tiempo y espacio usados.
- Programas del sistema usados, como compiladores, editores o utilidades.
- Número y tipo de operaciones de E/S, incluyendo el tiempo de trasmisión de E/S, que incluye la utilización de canales, unidades de control y dispositivos.
- Tiempo dedicado a esperar el completamiento de E/S.
- Número de registros de entrada leídos (específicamente, los introducidos en línea o provenientes de exploradores ópticos, lectoras de tarjetas u otros dispositivos de entrada), y número de registros de salida escritos.

Bloques de control de proceso y colas

Un bloque de control de proceso de un trabajo se crea cuando el planificador de trabajos acepta el trabajo y se actualiza a medida que los trabajos avanzan desde el inicio hasta el final de su ejecución.

Las colas usan bloques de control de proceso para seguir la pista de trabajos de la misma forma en que las autoridades migratorias usan los pasaportes para seguir la pista de visitantes internacionales. El bloque de control de proceso contiene todos los datos acerca del trabajo que el sistema operativo necesita para administrar el procesamiento de un trabajo. A medida que el trabajo se mueve a través del sistema, su avance se anota en el PCB.

Los bloques de control de proceso, no los trabajos, se vinculan para formar colas, como se muestra en la figura 4.4. Aunque ningún PCB se ha dibujado en detalle, el lector debe imaginar cada cola como una lista vinculada de bloques de control de proceso. Los PCB para



(Figura 4.4)

Rutas de cola desde RETENIDO hasta TERMINADO. Los planificadores de trabajos y de procesos liberan los recursos cuando el trabajo deja el estado en FUNCIONAMIENTO.

cada trabajo listo están vinculados en la cola LISTO, y todos los PCB para los trabajos que están entrando al sistema están vinculados en la cola INICIADO. Los trabajos que están en ESPERA, no obstante, están vinculados entre sí por "razones de espera", de modo que los PCB para los trabajos en esta categoría están vinculados en varias colas. Por ejemplo, los PCB para los trabajos que están esperando E/S en una unidad de disco específica están vinculados entre sí, mientras los que están esperando la impresora están vinculados en una cola diferente. Es necesario que la administración de las colas se lleve a cabo de manera ordenada, lo cual es determinado por las políticas y los algoritmos de planificación de procesos.

Políticas de planificación de procesos

En un entorno multiprogramación suele haber más trabajos a ejecutar de los que posiblemente podrían ejecutarse al mismo tiempo. Antes que el sistema operativo pueda planificarlos, necesita resolver tres limitaciones del sistema: 1) hay un número finito de recursos (como unidades de disco, impresoras y unidades de cinta); 2) algunos recursos, una vez asignados, no pueden compartirse con otro trabajo (por ejemplo, impresoras); y 3) algunos recursos requieren la intervención del operador; es decir, no es posible reasignarlos automáticamente de un trabajo a otro (como las unidades de disco).

¿Qué es una buena política de planificación de procesos? Varios criterios vienen a la mente, aunque observe en la siguiente lista que algunos se contradicen con otros:

- *Maximizar el resultado*. Ejecutar tantos trabajos como sea posible en una cantidad de tiempo dada. Esto puede lograrse fácilmente al ejecutar sólo trabajos cortos o al ejecutar trabajos sin interrupciones.

- *Minimizar el tiempo de respuesta.* Dar pronta respuesta a solicitudes interactivas. Esto puede hacerse al ejecutar sólo trabajos interactivos y hacer que los trabajos por lotes esperen hasta que termine la carga interactiva.
- *Minimizar el tiempo de retorno.* Introducir y sacar rápidamente trabajos enteros del sistema. Esto puede hacerse al ejecutar primero todos los trabajos por lotes (ya que éstos pueden agruparse para que funcionen de manera más eficiente que los trabajos interactivos).
- *Minimizar el tiempo de espera.* Mover todos los trabajos de la cola LISTO lo más rápido posible. Esto puede hacerse al reducir el número de usuarios permitidos en el sistema, de modo que la CPU esté disponible de inmediato siempre que un trabajo entra a la cola LISTO.
- *Maximizar la eficiencia de la CPU.* Mantener ocupada la CPU 100% del tiempo. Esto puede hacerse al ejecutar sólo trabajos limitados por la CPU (y no trabajos limitados por E/S).
- *Asegurar lo justo para todos los trabajos.* Proporcionar a cada uno una cantidad igual de tiempo de CPU y de E/S. Esto puede hacerse no otorgando un tratamiento especial a ningún trabajo, sin importar sus características o prioridad de procesamiento.

Como puede verse a partir de la lista anterior, si el sistema favorece algún tipo de usuario, es en detrimento de otro y no utiliza de manera eficiente sus recursos. La decisión final pertenece al diseñador del sistema, quien debe determinar cuáles criterios son los más importantes para ese sistema específico. Por ejemplo, usted podría decidir "maximizar la utilización de la CPU a la vez que se minimiza el tiempo de respuesta y equilibra el uso de todos los componentes del sistema mediante una mezcla de trabajos limitados por el CPU y trabajos limitados por E/S". Así usted puede seleccionar la política de programación que más satisfaga su criterio.

Aunque el planificador de trabajos selecciona trabajos a fin de asegurar el equilibrio de las colas LISTO y de E/S, hay instancias en que un trabajo reclama la CPU durante largo tiempo antes de emitir una solicitud de E/S. Si las solicitudes de E/S están siendo atendidas (lo cual se lleva a cabo por medio de un controlador de E/S que se analizará más tarde), este extenso uso de la CPU se integra a la cola LISTO mientras se vacían las colas de E/S, lo cual crea un desequilibrio inaceptable en el sistema.

Para resolver este problema, el planificador de procesos a menudo usa un mecanismo de sincronización e interrumpe de manera periódica los procesos en ejecución cuando ha expirado una rebanada de tiempo predeterminada. Cuando esto ocurre, el planificador suspende todas las actividades del trabajo en EJECUCIÓN actual y vuelve a planificarlo en la cola LISTO; más tarde continuará su ejecución. Ahora, la CPU se asigna a otro trabajo que funciona hasta que ocurre una de las tres situaciones siguientes: el temporizador llega a cero, el trabajo emite un comando de E/S, o el trabajo se completa. Luego, el trabajo se mueve a la cola LISTO, a la cola en ESPERA o a la cola TERMINADO, respectivamente. Una solicitud de E/S se denomina **espera natural** en entornos multiprogramación (permite la asignación del procesador a otro trabajo).

Una estrategia de planificación que interrumpe el procesamiento de un trabajo y transfiere la CPU a otro trabajo se denomina **política de planificación expropiativas**; se usa ampliamente en entornos de tiempo compartido. La alternativa, por supuesto, es una **política de planificación no expropiativas**, que funciona sin interrupciones externas (interrupciones externas al trabajo).

En consecuencia, una vez que un trabajo captura al procesador y empieza su ejecución, permanece en el estado en EJECUCIÓN de manera ininterrumpida hasta que emite una solicitud de E/S (espera natural) o hasta que ha terminado (con algunas excepciones hechas para ciclos infinitos, que se interrumpen tanto por políticas expropiativas como no expropiativas).

Algoritmos de planificación de procesos

El planificador de procesos depende de un **algoritmo de planificación de procesos**, basado en una política específica, para asignar la CPU y mover los trabajos a través del sistema. Los primeros sistemas operativos usaban políticas de planificación no expropiativas diseñadas para mover trabajos por lotes a través del sistema de la manera más eficiente posible. La mayor parte de los sistemas modernos, con su énfasis en uso interactivo y **tiempo de respuesta**, usan un algoritmo que se ocupa de las solicitudes inmediatas de usuarios interactivos.

A continuación se presentan los seis algoritmos de planificación que se han usado ampliamente.

Primero en llegar, primero en servirse

Primero en llegar, primero en servirse (FCFS), es un algoritmo de planificación no expropiativo que maneja trabajos según su tiempo de llegada, donde se atiende a los primeros en hacerlo. La implementación de este algoritmo es muy simple porque usa una cola FIFO. Este algoritmo funciona bien para la mayor parte de los sistemas por lotes, aunque es inaceptable para sistemas interactivos porque los usuarios interactivos esperan tiempos de respuesta rápidos.

Con FCFS, a medida que un nuevo trabajo entra al sistema, su PCB se vincula con el final de la cola LISTO y se elimina del frente de la cola cuando el procesador se vuelve disponible; es decir, después que ha procesado todos los trabajos que lo preceden en la cola.

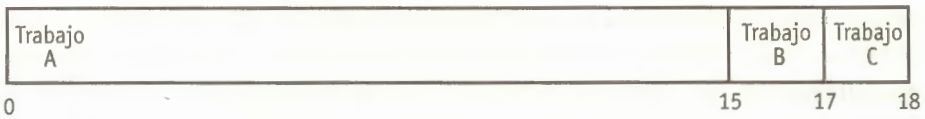
En un sistema FCFS estricto no hay listas en ESPERA (cada trabajo se ejecuta hasta su terminación), aunque puede haber sistemas donde el control (contexto) se intercambia sobre una espera natural (solicitud de E/S) y luego el trabajo se reanuda después de la terminación de la E/S.

Los siguientes ejemplos presumen un entorno estrictamente FCFS (sin multiprogramación). Con la política FCFS es impredecible el **tiempo de respuesta**; considere los tres trabajos siguientes:

- El trabajo A tiene un ciclo de CPU de 15 milisegundos.
- El trabajo B tiene un ciclo de CPU de 2 milisegundos.
- El trabajo C tiene un ciclo de CPU de 1 milisegundo.

Para cada trabajo, el ciclo de CPU contiene el uso actual de la CPU y las solicitudes de E/S. Es decir, es el tiempo de respuesta total. Al usar un algoritmo FCFS con una secuencia de llegada de A, B, C, la línea de tiempo (diagrama de Gantt) se muestra en la figura 4.5.

(Figura 4.5)
Línea de tiempo para la secuencia de los trabajos A, B, C usando el algoritmo FCFS.



Si los tres trabajos llegan casi simultáneamente, es posible calcular que el tiempo de respuesta para el trabajo A es 15; para el trabajo B, 17 y para el trabajo C, 18. Entonces el promedio de tiempo de respuesta es:

$$\frac{15 + 17 + 18}{3} = 16.67$$

Sin embargo, si los trabajos llegan en otro orden; por ejemplo C, B, A, entonces los resultados al usar el mismo algoritmo FCFS serían como se muestra en la figura 4.6.

(Figura 4.6)
Línea de tiempo para la secuencia de los trabajos C, B , A, usando el algoritmo FCFS.

En este ejemplo el tiempo de respuesta para el trabajo A es 18; para el trabajo B, 3; y para el trabajo C, 1 y el tiempo de respuesta promedio es:

$$\frac{18 + 3 + 1}{3} = 7.3$$

Estos datos representan una mejora importante respecto a la primera secuencia de llegada. Desafortunadamente, estos dos ejemplos ilustran la principal desventaja de usar el concepto FCFS: los tiempos de respuesta medios varían ampliamente y raras veces se minimizan. De hecho, cuando en la cola LISTO hay tres trabajos, el sistema sólo tiene una posibilidad en 6 de ejecutar los trabajos en la secuencia más ventajosa (C, B, A). Con cuatro trabajos, esta cifra cae a 1 de 24 y así sucesivamente.

FCFS es el único algoritmo analizado en este capítulo que incluye un elemento de azar. Los otros no lo ofrecen.

Si un trabajo monopoliza el sistema, la cantidad de su efecto global sobre el rendimiento del sistema depende de la política de planificación y si el trabajo está limitado por CPU o es trabajo limitado por E/S. Mientras un trabajo con un ciclo de CPU más largo (como en este ejemplo el trabajo A) está usando la CPU, los otros trabajos en el sistema están en espera de ser procesados o terminando sus solicitudes de E/S (en caso que se use un controlador de E/S) y se unen a la cola LISTO a fin de esperar su turno para usar el procesador. Si las solicitudes de E/S no están siendo atendidas, las colas de E/S permanecen estables mientras la cola LISTO crece (con nuevas llegadas). En casos extremos, la cola LISTO puede llenarse mientras las listas de E/S están vacías, o estables, y entonces es necesario dejar ociosos los dispositivos de E/S.

Por otra parte, si el trabajo está procesando un ciclo de E/S lento, las colas de E/S se acumulan rápidamente, provocan desbordamiento y la CPU permanece ociosa (en caso que se use un controlador de E/S). Esta situación termina por resolverse cuando el trabajo limitado por E/S termina su ciclo de E/S, las colas comienzan a moverse de nuevo y el sistema puede recuperarse del cuello de botella.

En un algoritmo FCFS estricto no ocurre ninguna de estas situaciones. Sin embargo, el tiempo de respuesta es variable (impredecible). Es por ello que el algoritmo FCFS es menos atractivo que uno que atiende primero el siguiente trabajo más corto, como lo hace el próximo algoritmo de planificación, incluso en un entorno multiprogramación.

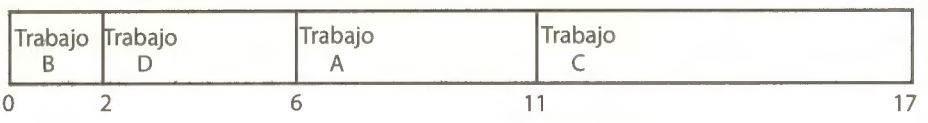
Siguiente trabajo más corto primero

Siguiente trabajo más corto primero (SJN), es un algoritmo de planificación no expropiativo (también denominado primero el trabajo más corto: SJF) que maneja trabajos con base en la longitud de su ciclo de CPU. Es más fácil implementarlo en entornos por lotes donde el tiempo de CPU estimado necesario para ejecutar los trabajos lo proporciona de antemano cada usuario al inicio de cada trabajo. Sin embargo, no funciona en sistemas interactivos porque en éstos los usuarios no estiman de antemano el tiempo de CPU necesario para ejecutar sus trabajos.

Por ejemplo, hay cuatro trabajos por lotes, todos en la cola LISTO, para los cuales el ciclo de CPU, o tiempo de ejecución, se calcula como sigue:

Trabajo:	A	B	C	D
Ciclo de CPU:	5	2	6	4

El algoritmo SJN revisa los cuatro trabajos y los planifica para su procesamiento en este orden: B, D, A, C. En la figura 4.7 se muestra la línea de tiempo.



(Figura 4.7)
Línea de tiempo para la secuencia de los trabajos B, D, A, C usando el algoritmo SJN.

El tiempo de respuesta promedio es:

$$\frac{2+6+11+17}{4} = 9.0$$

Dedicaremos unos minutos para ver por qué es posible demostrar que este algoritmo es óptimo y por qué proporciona de manera consistente el tiempo de respuesta promedio mínimo. Usaremos el ejemplo previo para obtener la fórmula general.

Si observamos la figura 4.7, podemos ver que el trabajo B termina en su tiempo dado (2), que el trabajo D termina en su tiempo dado más el tiempo que ha esperado para que B se ejecute (4 + 2), que el trabajo A termina en su tiempo dado más el tiempo de D más el tiempo de B (5 + 4 + 2), y que el trabajo C termina en su tiempo dado más los tiempos previos (6 + 5 + 4 + 2). Así, cuando se calcula el promedio se obtiene:

$$\frac{[(2) + (4 + 2) + (5 + 4 + 2) + (6 + 5 + 4 + 2)]}{4} = 9.0$$

Como puede verse, el tiempo para el primer trabajo aparece cuatro veces en la ecuación: una vez por cada trabajo. En forma semejante, el tiempo para el segundo trabajo aparece tres veces (número de trabajos menos uno). El tiempo para el tercer trabajo aparece dos veces (número de trabajos menos 2) y el tiempo para el cuarto trabajo aparece sólo una vez (número de trabajos menos 3).

Así, la ecuación anterior puede volver a escribirse como:

$$\frac{4 * 2 + 3 * 4 + 2 * 5 + 1 * 6}{4} = 9.0$$

Debido a que el tiempo para el primer trabajo aparece cuatro veces en la ecuación, tiene cuatro veces el efecto sobre el promedio que la longitud del cuarto trabajo, que sólo aparece una vez. En consecuencia, si el primer trabajo requiere el menor tiempo de cálculo, seguido a su vez por los otros trabajos, ordenados del más corto al más largo, entonces el resultado es el menor promedio posible. La fórmula para el promedio es la siguiente:

$$\frac{t_1(n) + t_2(n - 1) + t_3(n - 2) + \ ... + t_n(n(1))}{n}$$

Donde n es el número de trabajos en la cola y $t_j(j = 1, 2, 3,\ldots,n)$ es la longitud del ciclo de CPU para cada uno de los trabajos.

Sin embargo, el algoritmo SJN es óptimo sólo cuando todos los trabajos están disponibles al mismo tiempo y la CPU estima que están disponibles y son seguros.

Planificación de prioridad

La **planificación de prioridad** es un algoritmo no expropiativo y uno de los algoritmos de planificación más comunes en sistemas por lotes, aun cuando puede proporcionar un tiempo de respuesta más lento para algunos usuarios. Este algoritmo otorga tratamiento preferencial a trabajos importantes. Permite el procesamiento primero de los programas con prioridad más alta, y que no sean interrumpidos sino hasta que se completan sus ciclos de CPU (tiempos de ejecución) o que ocurra una espera natural. Si dos o más trabajos con la misma prioridad están en la cola LISTO, el procesador se asigna al que llegó primero (prioridad primero en llegar, primero en servirse).

Las prioridades puede asignarlas un administrador del sistema usando características extrínsecas a los trabajos. Por ejemplo, es posible asignarlas con base en la posición del usuario (primero los investigadores, los estudiantes al final) o, en entornos comerciales, pueden ser compradas por los usuarios que paguen más por una prioridad más alta a fin de garantizar que sus trabajos reciban el procesamiento más rápido posible. Con un algoritmo basado en prioridades, el planificador de trabajos suele vincular los trabajos a una de varias colas LISTO en vez de a una sola. Los detalles sobre colas múltiples se presentan más tarde en este capítulo.

Las prioridades también pueden determinarse por el administrador del procesador con base en las características intrínsecas a los trabajos, como:

- *Requerimientos de memoria.* A los trabajos que requieren grandes cantidades de memoria pueden asignarse prioridades menores que a los que solicitan pequeñas cantidades de memoria, o viceversa.
- *Número y tipo de dispositivos periféricos.* A los trabajos que requieren muchos dispositivos periféricos pueden asignarse prioridades menores que a los que requieren menos dispositivos.
- *Tiempo total de CPU.* A los trabajos que tienen un gran ciclo de CPU o un tiempo de ejecución grande pueden asignarse prioridades menores que a los que tienen un tiempo de ejecución breve.
- *Cantidad de tiempo de permanencia en el sistema.* Esta es la cantidad total de tiempo transcurrido desde que el trabajo fue aceptado para su procesamiento. Algunos sistemas incrementan la prioridad de los trabajos que han permanecido en el sistema durante un tiempo inusualmente largo a fin de apresurar su salida. Esto se denomina **envejecimiento**.

Estos criterios se usan para determinar prioridades predeterminadas en muchos sistemas. Las prioridades predeterminadas pueden ser omitidas por prioridades específicas nombradas por los usuarios.

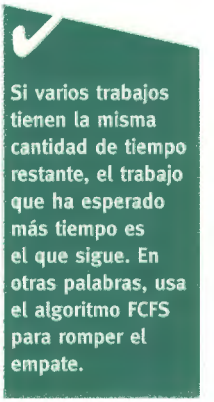
También hay esquemas de prioridad expropiativos, que se analizarán más tarde en este capítulo, en la sección sobre colas múltiples.

Tiempo más corto restante

El **tiempo más corto restante (SRT)** es la versión expropiativa del algoritmo SJN. El procesador se asigna al trabajo más próximo a completar, pero incluso este trabajo puede expropiarse si un nuevo trabajo en la cola LISTO tiene un tiempo de terminación más corto.

Este algoritmo no puede implementarse en un sistema interactivo porque es necesario conocer de antemano el tiempo de CPU requerido para terminar cada trabajo. Se usa a menudo en entornos por lotes, cuando es aconsejable otorgar preferencia a trabajos más cortos, aun cuando SRT implica más sobrecarga que SJN porque el sistema operativo tiene que controlar a menudo el tiempo de CPU para todos los trabajos en la cola LISTO y debe realizar cambio de contexto para los trabajos que se intercambian en el momento de la anticipación (no necesariamente cambiados hacia fuera del disco, aunque esto también puede ocurrir).

El ejemplo en la figura 4.8 muestra la forma en que el algoritmo SRT funciona con cuatro trabajos que llegaron en sucesión rápida (con un ciclo de separación entre cada uno).



Si varios trabajos tienen la misma cantidad de tiempo restante, el trabajo que ha esperado más tiempo es el que sigue. En otras palabras, usa el algoritmo FCFS para romper el empate.

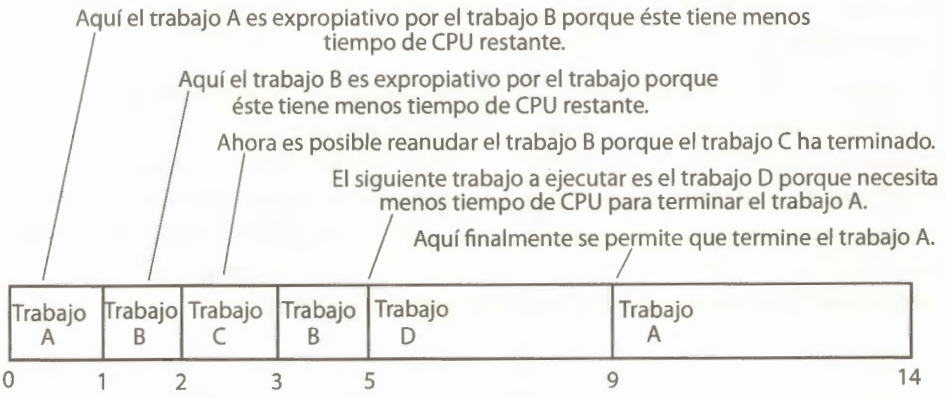
Tiempo de llegada:	0	1	2	3
Trabajo:	A	B	C	D
Ciclo de CPU:	6	3	1	4

En este caso, el tiempo de respuesta es el tiempo de completamiento de cada trabajo menos su tiempo de llegada:

Trabajo:	A	B	C	D
Tiempo de respuesta:	14	4	1	6

De modo que su tiempo de respuesta promedio es:

$$\frac{14 + 4 + 1 + 6}{4} = 6.25$$



(Figura 4.8) *Línea de tiempo para la secuencia de los trabajos A, B, C, D usando el algoritmo expropiativo SRT. Cada trabajo se interrumpe después de un ciclo de CPU si otro trabajo está esperando con menos tiempo de CPU restante.*

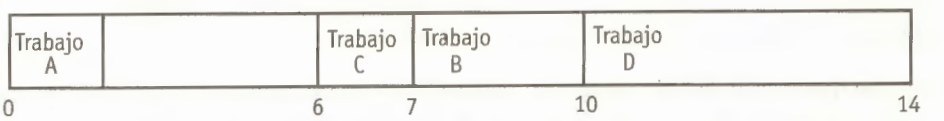
¿Cómo se compara lo anterior con el mismo problema usando la política no expropiativa SJN? La figura 4.9 muestra la misma situación usando SJN.

En este caso el tiempo de respuesta es:

Trabajo:	A	B	C	D
Tiempo de respuestas:	6	9	5	11

De modo que el tiempo de respuesta promedio es:

$$\frac{6 + 9 + 5 + 11}{4} = 7.75$$



(Figura 4.9) *Línea de tiempo para la misma secuencia de los trabajos A, B, C, D usando el algoritmo no expropiativo SJN.*

Observe en la figura 4.9 que inicialmente el trabajo A es el único que está en la cola LISTO, de modo que es el primero que se ejecuta y así continúa hasta que termina porque SJN es un algoritmo no expropiativo. El siguiente trabajo que se ejecuta es el trabajo C porque cuando termina el trabajo A (en el momento 6), todos los otros trabajos (B, C y D) han llegado. De estos tres, C tiene el menor ciclo de CPU, de modo que es el siguiente que se ejecuta; luego se ejecuta B y finalmente, D.

En consecuencia, con este ejemplo, SRT en 6.25 es más rápido que SJN en 7.75. Sin embargo, omitimos incluir el tiempo requerido por el algoritmo SRT para hacer el siguiente cambio de contexto. El **cambio de contexto** es necesario para todos los algoritmos expropiativos. Cuando el trabajo A se anticipa, toda su información de procesamiento debe guardarse en su PCB para después, cuando continúa la ejecución del trabajo A, y el contenido del PCB del trabajo B se carga en los registros idóneos de modo que sea posible poder empezar su ejecución después; es un cambio de contexto. Más tarde, cuando el trabajo A se asigna de nuevo al procesador, se efectúa otro cambio de contexto. Esta vez la información del trabajo anticipado se almacena en su PCB, y el contenido del PCB del trabajo A se carga en los registros idóneos.

La manera en que se realiza realmente el cambio de contexto depende de la arquitectura de la CPU; en muchos sistemas hay instrucciones especiales para que el almacenamiento y la restitución de la información sean rápidos. El cambio está diseñado para realizarse de manera eficiente pero, no importa cuán rápido sea, sigue requiriendo valioso tiempo de CPU. Así, aunque parece que el SRT es más rápido, en un entorno operativo real sus ventajas disminuyen por el tiempo gastado en cambios de contexto. Una comparación precisa de SRT y SJN debe incluir el tiempo requerido para efectuar cambio de contexto.

Round robin

Round robin es un algoritmo de planificación de procesos expropiativo que se usa ampliamente en sistemas interactivos. Es fácil de implementar y no se basa en las características del trabajo, sino en una rebanada de tiempo predeterminada que se proporciona a cada trabajo para asegurar que la CPU se comparta de manera equitativa entre todos los procesos activos y ningún trabajo la monopolice.

La rebanada de tiempo se denomina **cuanto de tiempo** y su tamaño es crucial para el rendimiento del sistema. Suele variar desde 100 milisegundos hasta 1 o 2 segundos.

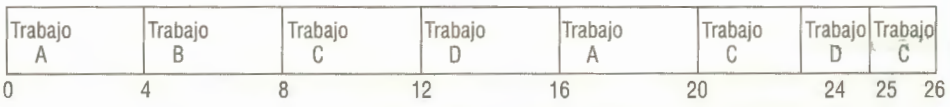
Los trabajos se colocan en la cola LISTO usando un esquema de primero en llegar, primero en servirse y el planificador de procesos selecciona el primer trabajo desde el frente de la cola, establece el temporizador en el cuanto de tiempo y asigna la CPU a este trabajo. Si el procesamiento no ha terminado cuando expira el tiempo, el trabajo es expropiado, se coloca al final de la cola y su información se guarda en su PCB.

En caso que el ciclo de CPU del trabajo sea más corto que el cuanto de tiempo, ocurre una de dos acciones: 1) Si este es el último ciclo de CPU del trabajo y el trabajo ha terminado, entonces se liberan todos los recursos que se le han asignado y el trabajo completado se

devuelve al usuario; 2) si el ciclo de CPU ha sido interrumpido por una solicitud de E/S, entonces la información sobre el trabajo se guarda en su PCB y se vincula al final de la cola de E/S idónea. Después, una vez que se ha satisfecho la solicitud de E/S, se regresa al final de la cola LISTO para esperar la asignación de la CPU.

El ejemplo en la figura 4.10 ilustra un algoritmo round robin con una rebanada de tiempo 4 milisegundos (se ignoran las respuestas E/S)

Tiempo de llegada:	0	1	2	3
Trabajo:	A	B	C	D
Ciclo de CPU:	8	4	9	5



(Figura 4.10) *Línea de tiempo para la secuencia de los trabajos A, B, C, D usando el algoritmo expropiativo round robin con rebanadas de tiempo de 4 ms.*

El tiempo de respuesta es el tiempo de completamiento menos el tiempo de llegada:

Trabajo:	A	B	C	D
Tiempo de respuesta:	20	7	24	22

De modo que el tiempo de respuesta promedio es:

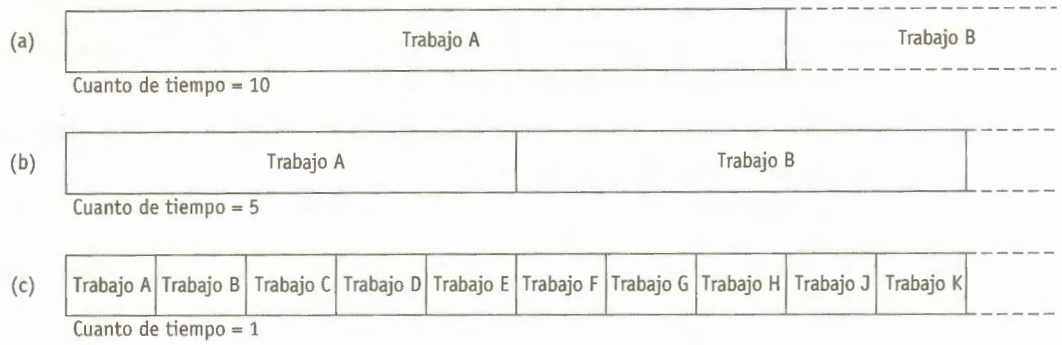
$$\frac{20 + 7 + 24 + 22}{4} = 18.25$$

Observe que en la figura 4.10 el trabajo A ha sido expropiado una vez porque requería 8 milisegundos para completar su ciclo de CPU, mientras el trabajo B terminó en un cuanto de tiempo. El trabajo C fue expropiado dos veces porque requería 9 milisegundos para completar su ciclo de CPU, y el trabajo D fue expropiado una vez porque requería 5 milisegundos. En su última ejecución o intercambio hacia la memoria, ambos trabajos D y C, usaron la CPU sólo durante 1 milisegundo y terminaron antes que expirara su último cuanto de tiempo, liberando más rápido a la CPU.

Con round robin y una cola con numerosos procesos, cada uno de éstos tiene acceso al procesador antes que el primero obtenga acceso por segunda ocasión.

La eficiencia del round robin depende del tamaño del cuanto de tiempo en relación con el ciclo de CPU promedio. Si el cuanto es demasiado largo; es decir, si es mayor que la mayor parte de los ciclos de CPU, entonces el algoritmo se reduce al esquema FCFS. Si el cuanto es demasiado pequeño, entonces la cantidad de cambio de contexto aminora la ejecución de los trabajos y la cantidad de procesamiento extra aumenta dramáticamente, como se observa en los tres ejemplos de la figura 4.11. El trabajo A tiene un ciclo de CPU de 8 milisegundos. La cantidad de cambio de contexto aumenta a medida que disminuye el tamaño del cuanto de tamaño.

En la figura 4.11, el primer caso (*a*) tiene un cuanto de tiempo de 10 milisegundos y no hay cambio de contexto (tampoco procesamiento extra). El ciclo de CPU termina poco antes



(Figura 4.11)

Cambios de contexto para tres diferentes cuantos de tiempo. En (a) el trabajo A (el cual requiere de sólo 8 ciclos para ejecutarse por completo) termina antes que expire el cuanto de tiempo 10. En (b y c), el cuanto de tiempo expira primero, interrumpiendo el trabajo.

que expire el cuanto de tiempo y el trabajo termine su ejecución. Para este trabajo con este cuanto de tiempo, no hay diferencia entre el algoritmo round robin y el algoritmo FCFS.

En el segundo caso (*b*), con un cuanto de tiempo de 5 milisegundos, sólo hay un cambio de contexto. El trabajo es expropiado una vez cuando expira el cuanto de tiempo, de modo que hay algo de procesamiento extra para el cambio de contexto y debe haber un tiempo de respuesta retrasado con base en el número de los otros trabajos en el sistema.

En el tercer caso (*c*), con un cuanto de tiempo de 1 milisegundo, hay siete cambios de contexto porque el trabajo es expropiado cada que expira el cuanto de tiempo; el procesamiento extra se vuelve oneroso y el tiempo de respuesta cambia en concordancia.

¿Cuál es el mejor tamaño del cuanto de tiempo? Ya debe ser posible pronosticar la respuesta: depende del sistema. Si es un entorno interactivo, se espera que el sistema responda rápidamente a todos los usuarios, especialmente cuando hacen solicitudes simples. Si se trata de un sistema por lotes, el tiempo de respuesta no es un factor (el tiempo de espera sí lo es) y el procesamiento extra se vuelve muy importante.

A continuación se presentan dos reglas empíricas para seleccionar el cuanto de tiempo idóneo: 1) debe ser suficientemente largo para permitir que 80% de los ciclos de CPU terminen su ejecución, y 2) debe ser por lo menos 100 veces más largo que el tiempo requerido para efectuar un cambio de contexto. Estas reglas se usan en algunos sistemas, pero no son inflexibles.

Colas de multinivel

Colas de multinivel, estas colas en realidad no constituyen un algoritmo de planificación por separado, sino que funcionan de manera conjunta con varios de los esquemas ya analizados y se encuentran en sistemas con trabajos que pueden agruparse según una característica común. Ya hemos presentado por lo menos un tipo de cola de multinivel: la de un sistema basado en prioridades con colas diferentes para cada nivel de prioridad.

Otro tipo de sistema podría reunir todos los trabajos limitados por CPU en una cola y todos los trabajos limitados por E/S en otra. Luego, el planificador de procesos puede escoger de manera alterna trabajos de cada cola a fin de mantener equilibrado el sistema.

Un tercer ejemplo común es uno usado en un entorno híbrido que atiende trabajos tanto por lotes como interactivos. Los trabajos por lotes se colocan juntos en una cola denominada cola de segundo plano, mientras los trabajos interactivos se colocan juntos en una cola de primer plano y son tratados de manera más favorable que los de la cola de segundo plano.

Todos estos ejemplos poseen una cosa en común: La política de planificación está basada en algún esquema predeterminado que asigna un tratamiento especial a los trabajos en cada cola. Dentro de cada cola, los trabajos son atendidos de manera FCFS.

Las colas de multinivel permiten usar diferentes algoritmos en colas diferentes, posibilitando combinar las ventajas de varios algoritmos.

Las colas de multinivel plantean algunas preguntas interesantes:

- ¿El procesador se asigna a los trabajos en la primera cola hasta que esté vacía antes de moverlo a la cola siguiente o se desplaza de cola en cola hasta que se ha atendido el último trabajo en la última y después regresa para servir al primer trabajo en la primera cola, o algo en medio?
- ¿Esto es justo para quienes han ganado, o pagado por, una prioridad más alta?
- ¿Esto es justo para quienes están en una cola de prioridad baja?
- Si el procesador se asigna a los trabajos en la primera cola y ésta nunca se vacía, ¿cuándo son atendidos los trabajos en la última cola?
- ¿Pueden los trabajos en la última cola obtener "un ascenso por buen comportamiento" y terminar moviéndose a mejores colas?

Las respuestas dependen de la política usada por el sistema para atender las colas. Hay cuatro métodos primarios para el movimiento: no permitir el movimiento entre colas, mover los trabajos de cola en cola, mover los trabajos de cola en cola y aumentar los cuantos de tiempo para colas más bajas, y otorgar un tratamiento especial a los trabajos que han estado en el sistema durante largo tiempo (envejecimiento).

Caso 1: Sin movimiento entre colas

Sin movimiento entre colas es una política muy simple que recompensa a quienes tienen trabajos con prioridad alta. El procesador se asigna de una manera FCFS a los trabajos que están en la cola de prioridad alta y se asigna a los trabajos en colas de prioridad más baja sólo cuando las colas de prioridad alta están vacías. Esta política puede justificarse si hay relativamente pocos usuarios con trabajos de prioridad alta, de modo que las colas superiores se vacían rápidamente, permitiendo que el procesador dedique una cantidad justa de tiempo a la ejecución de los trabajos de prioridad baja.

Caso 2: Movimiento entre colas

Movimiento entre colas es una política que ajusta las prioridades asignadas a cada trabajo: los trabajos con prioridad alta son tratados como todos los otros una vez que están en el sistema. (Su prioridad inicial puede ser favorable.) Cuando ocurre una interrupción de un cuanto de tiempo, el trabajo es expropiado y movido hacia el extremo de la siguiente cola más baja. También es posible incrementar la prioridad de un trabajo; por ejemplo, cuando emite una solicitud de E/S antes que expire su cuanto de tiempo.

Esta política es más justa en un sistema en que los trabajos son manejados según sus características de ciclo de CPU: trabajos limitados por CPU o trabajos limitados por E/S. Esto supone que un trabajo que excede su cuanto de tiempo está limitado por CPU y que requiere más asignación de CPU que uno que solicita E/S antes que expire el cuanto de tiempo. En consecuencia, los trabajos limitados por CPU se colocan al final de la siguiente cola de nivel más bajo cuando son anticipados debido a la expiración del cuanto de tiempo, mientras que los trabajos limitados por E/S se regresan al final de la siguiente cola de nivel más alto una vez que ha terminado su solicitud de E/S. Esto facilita los trabajos limitados por E/S y es bueno en sistemas interactivos.

Caso 3: Cuanto de tiempo variable por cola

Esta es una variante de la política de movimiento entre colas, y permite un tiempo de respuesta más rápido de trabajos limitados por CPU.

En este esquema, a cada una de las colas se le proporciona un cuanto de tiempo del doble de largo que a la cola previa. La cola más alta puede tener un cuanto de tiempo de 100 milisegundos. Así, la segunda cola más alta puede tener un cuanto de tiempo de 200 milisegundos, la tercera, uno de 400 milisegundos, etcétera. Si hay suficientes colas, la más baja puede tener un cuanto de tiempo relativamente largo de 3 segundos o más.

Si un trabajo no termina su ciclo de CPU en el primer cuanto de tiempo, se mueve hacia el final de la siguiente cola de nivel más bajo y cuando el procesador se le asigna en breve, el trabajo se ejecuta durante el doble de tiempo que antes.

Con este esquema, un trabajo limitado por CPU puede ejecutarse cada vez durante más tiempo, mejorando así las posibilidades de terminar más rápido.

Caso 4: Envejecimiento

El envejecimiento se usa para asegurar que los trabajos en las colas de nivel bajo terminen por completo su ejecución. El sistema operativo sigue la pista del tiempo de espera de cada trabajo y cuando un trabajo se vuelve demasiado viejo; es decir, cuando llega a cierto límite de tiempo, el sistema lo mueve hacia la siguiente cola más alta, y así sucesivamente hasta que llega a la cola superior. Una política de envejecimiento más drástica es una que mueve el trabajo viejo directamente desde la cola más baja hasta el final de la cola en la parte superior. Sin importar su implementación verdadera, una política de envejecimiento se protege contra el aplazamiento indefinido de trabajos indeseados. Como es de esperar, **aplazamiento indefinido** significa que la ejecución de un trabajo se aplaza durante una cantidad indefinida de tiempo porque es expropiado repetidamente, de modo que es posible procesar otros trabajos. (Todos conocemos ejemplos de una tarea desagradable que se ha aplazado de manera indefinida a fin de ganar tiempo para realizar otra actividad más atractiva.) Por último, la situación puede conducir a la vieja inanición del trabajo. El aplazamiento indefinido constituye un problema primordial cuando se asignan recursos y se analizará con mayor detalle en el capítulo 5.

Comentario sobre interrupciones

Encontramos las **interrupciones** por primera vez en el capítulo 3 cuando el administrador de la memoria emitía interrupciones de página para acomodar solicitudes de trabajo. En este capítulo analizamos otro tipo de interrupción que ocurre cuando expira el cuanto de tiempo y el procesador se libera del trabajo en ejecución y se asigna a otro.

Hay otras interrupciones provocadas por eventos externos al proceso. Las interrupciones de E/S se emiten cuando se lanza un comando READ o WRITE. (En el capítulo 7 explicaremos estos con detalle.) Interrupciones internas, o interrupciones síncronas, también ocurren como un resultado directo de la operación aritmética o instrucción de trabajo que se está procesando en ese momento.

Operaciones aritméticas ilegales, como las siguientes, pueden generar interrupciones:

- Intentos por dividir entre cero.
- Operaciones de punto flotante que generan desbordamiento o subdesbordamiento.
- Suma o resta de punto fijo que provoquen desbordamiento aritmético.

Instrucciones ilegales de trabajo, como las siguientes, también pueden ocasionar interrupciones:

- Intentos por acceder a ubicaciones de almacenamiento protegidas o inexistentes.
- Intentos por usar un código de operación indefinido.
- Operar sobre datos inválidos.
- Intentos por hacer cambios al sistema, como tratar de modificar el tamaño del cuanto de tiempo.

El programa de control que maneja la secuencia de interrupciones de eventos se denomina **manejador de interrupciones**. Cuando el sistema operativo detecta un error no recuperable, normalmente el manejador de interrupciones sigue la siguiente secuencia:

1. Se describe y almacena el tipo de interrupción, a fin de pasarlo al usuario como un mensaje de error.
2. Se guarda el estado del proceso interrumpido, incluyendo el valor del contador de programa, la especificación del modo y el contenido de todos los registros.
3. Se procesa la interrupción: el mensaje de error y el estado del proceso interrumpido se envían al usuario; se detiene la ejecución del programa; se liberan todos los recursos asignados al trabajo y el trabajo sale del sistema.
4. El procesador reanuda la operación normal.

Si sólo se está tratando con interrupciones internas, que son irrecuperables, el trabajo termina en el paso 3. Sin embargo, cuando el manejador de interrupciones está trabajando con una interrupción de E/S, con un cuanto de tiempo o con otra interrupción irrecuperable, el paso 3 simplemente detiene el trabajo y lo mueve a la cola de dispositivos de E/S idónea, o a la cola LISTO (en tiempo de descanso). Después, una vez que termina la solicitud de E/S, el trabajo se devuelve a la cola LISTO. Si se trataba de un descanso (interrupción de un cuanto), el trabajo (o el proceso) ya se encuentra en la cola LISTO.

Conclusión

El administrador del procesador debe asignar la CPU entre todos los usuarios del sistema. En este capítulo establecimos la distinción entre planificación de trabajos, la selección de los trabajos de entrada con base en sus características, y la planificación de procesos, la asignación de la CPU instante a instante. También describimos la forma en que se generan las interrupciones y cómo las resuelve el manejador de interrupciones.

Cada algoritmo de planificación presentado en este capítulo posee características, objetivos y aplicaciones únicas. Un diseñador del sistema puede escoger la mejor política y el mejor algoritmo sólo después de evaluar cuidadosamente sus fortalezas y debilidades. La tabla 4.1 muestra cómo se comparan los algoritmos presentados en este capítulo.

En el siguiente capítulo analizaremos las demandas planteadas al administrador del procesador cuando éste intenta sincronizar la ejecución de todos los trabajos en el sistema.

Algoritmo	Tipo de política	Es mejor para	Desventajas	Ventajas
FCFS	No expropiativo	Por lotes	Tiempos de respuesta impredecibles	Fácil de implementar
SJN	No expropiativo	Por lotes	Aplazamiento indefinido de algunos trabajos	Minimiza el tiempo medio de espera
Planificación de prioridad	No expropiativo	Por lotes	Aplazamiento indefinido de algunos trabajos	Asegura la terminación rápida de trabajos importantes
SRT	Expropiativo	Por lotes	Se incurre en procesamiento extra debido a cambios de contexto	Asegura la terminación rápida de trabajos importantes
Round robin	Expropiativo	Interactivo	Requiere la selección de un buen cuanto de tiempo	Proporciona tiempos de respuesta razonables para usuarios interactivos; proporciona asignación justa de CPU.
Colas de multinivel	Expropiativo/ No expropiativo	Por lotes/ Interactivo	Se incurre en procesamiento extra debido a monitores de las colas	Esquema flexible, contrarresta el aplazamiento indefinido con el envejecimiento o el movimiento de alguna otra cola; otorga un tratamiento justo a trabajos limitados por CPU al incrementar cuantos de tiempo en colas de prioridad más baja o el movimiento de alguna otra cola

(Tabla 4.1)

Comparación de los algoritmos de planificación analizados en este capítulo.

Términos clave

Algoritmo de planificación de procesos: Algoritmo usado por el planificador de trabajos para asignar la CPU y mover los trabajos a través del sistema.

Aplazamiento indefinido: Significa que la ejecución de un trabajo se aplaza indefinidamente porque es expropiado de manera repetida, de modo que otros trabajos pueden procesarse.

Bloque de control de proceso (PCB): Estructura de datos que contiene información sobre el estado y las características actuales de un proceso.

Cambio de contexto: Actos para guardar la información del procesamiento de un trabajo en su bloque de control de proceso de modo que el trabajo pueda intercambiarse hacia fuera de la memoria y cargar la información del procesamiento desde el PCB de otro trabajo en los registros idóneos, de modo que la CPU pueda procesarlo. El cambio de contexto ocurre en todas las políticas expropiativas.

Cola: Lista vinculada de bloques de control de proceso que indica el orden en que los trabajos o procesos serán atendidos.

Colas de multinivel: Esquema de planificación de procesos (usado con otros algoritmos de planificación) que agrupa los trabajos según una característica común.

Cuanto de tiempo: Periodo asignado a un proceso para la ejecución antes de ser expropiado.

Envejecimiento: Política usada para asegurar que los trabajos que han estado en el sistema durante bastante tiempo en las colas de nivel inferior terminen por completar su ejecución.

Espera natural: Término común usado para identificar una solicitud de E/S de un programa en un entorno multiprogramación, que origina que un proceso espere "naturalmente" antes de reanudar su ejecución.

Estado del proceso: Información almacenada en el PCB del trabajo que indica la posición actual del trabajo y los recursos responsables de ese estado.

Estados del trabajo: Condición de un trabajo a medida que éste se mueve a través del sistema desde el principio hasta el fin de su ejecución.

Hilo: Porción de un programa que puede ejecutarse de manera independiente de otras porciones. Los programas de aplicaciones multihilos pueden tener varios hilos en ejecución a la vez con la misma prioridad o con prioridades distintas.

Interrupción: Señal del hardware que suspende la ejecución de un programa y activa la ejecución de un programa especial denominado manejador de interrupciones.

Manejador de interrupciones: Programa que controla cuál acción debe emprender el sistema operativo cuando se interrumpe una secuencia de eventos.

Multiprogramación: Técnica que permite que un solo procesador procese varios programas residentes simultáneamente en la memoria principal y abandonando su ejecución al superponer solicitudes de E/S con solicitudes de CPU.

Planificación de prioridad: Política de planificación (o algoritmo) no preferencial que permite la ejecución de trabajos con prioridad alta antes que la de trabajos de prioridad baja.

Planificador de alto nivel: Sinónimo de planificador de trabajos.

Planificador de bajo nivel: Sinónimo del planificador de procesos.

Planificador de nivel medio: Planificador usado por el administrador del procesador cuando el sistema se sobrecarga al remover procesos activos de la memoria. Este planificador cambia estos procesos regresándolos a la memoria cuando el sistema sobrecargado está libre.

Planificador de procesos: Planificador de bajo nivel del administrador del procesador que establece el orden en que los procesos en la cola LISTO serán atendidos por la CPU.

Planificador de trabajos: Es el planificador de alto nivel del administrador del procesador que selecciona trabajos de una cola de trabajos de entrada con base en las características de cada trabajo.

Política de planificación de procesos: Cualquier política usada por el administrador del procesador para seleccionar el orden en que serán ejecutados los trabajos de entrada.

Política de planificación no expropiativa: Estrategia de planificación de trabajos que funciona sin interrupciones externas, de modo que una vez que un trabajo captura al procesador y empieza su ejecución, permanece en el estado en EJECUCIÓN sin interrupción hasta que emite una solicitud de E/S o ha terminado.

Política de planificación expropiativa: Cualquier estrategia de planificación de procesos que, basada en políticas predeterminadas, interrumpe el procesamiento de un trabajo y transfiere la CPU a otro trabajo. Se usa ampliamente en entornos de tiempo compartido.

Primero en llegar, primero en servirse (FCFS): Política de planificación (o algoritmo) no expropiativa que maneja trabajos según su tiempo de llegada.

Procesador: 1) Sinónimo de la CPU, o 2) cualquier componente en un sistema de cómputo capaz de realizar una secuencia de actividades.

Proceso: Instancia de la ejecución de un programa que es identificable y controlable por el sistema operativo.

Programa: Unidad interactiva, como un archivo almacenado en un disco.

Round robin: Política de planificación (o algoritmo) expropiativo que asigna a cada trabajo una unidad de tiempo de procesamiento por turno para asegurar que la CPU sea compartida equitativamente entre todos los procesos activos y ninguno la monopolice.

Siguiente trabajo más corto primero (SJN): Política de planificación (o algoritmo) no expropiativo que selecciona el trabajo en espera con el tiempo de ciclo de CPU más corto.

Tarea: 1) Término usado para describir un proceso, o 2) unidad básica de lenguajes de programación concurrentes que define una secuencia de instrucciones que pueden ejecutarse en paralelo con otras unidades semejantes.

Tiempo de respuesta: Medida de la eficiencia de un sistema que sigue la pista del tiempo requerido para ejecutar un trabajo y regresar el resultado al usuario.

Tiempo más corto restante (SRT): Política de planificación (o algoritmo) expropiativa semejante al algoritmo SJN que asigna el procesador al trabajo más próximo a completarse.

Trabajo limitado por E/S: Trabajo que requiere un gran número de operaciones de entrada/salida, resultando en mucho más tiempo libre para la CPU.

Trabajo limitado por CPU: Trabajo que realiza una gran cantidad de procesamiento sin detenerse antes de emitir una interrupción.

Búsquedas de interés

- Ciclo de tiempo de la CPU
- Bloque de control de tareas (TCB)
- Cuello de botella del procesador
- Longitud de la cola del procesador
- Interrupciones de E/S

Ejercicios

Temas de investigación

A. La tecnología de núcleos múltiples puede con frecuencia, pero no necesariamente siempre, hacer más rápida la ejecución de aplicaciones. Investigue algunos entornos informáticos de la vida real de los que se esperan beneficios de chips de núcleos múltiples y explique brevemente por qué. Cite sus fuentes académicas.

B. Compare dos procesadores de uso actual usados en computadoras personales. Use benchmarks industriales normales para su comparación y enumere brevemente las ventajas y las desventajas de cada uno. Puede comparar procesadores diferentes del mismo fabricante (como dos procesadores Intel) o procesadores diferentes de distintos fabricantes (como Intel y AMD).

Ejercicios

1. La figura 4.12 es un modelo de proceso simplificado de usted, donde sólo hay dos estados: dormido y vigilia. Usted realiza la transición de vigilia a dormido cuando está cansado, y de dormido a vigilia cuando se apaga la alarma del reloj.

   a. Agregue más estados al diagrama (por ejemplo, uno podría ser comiendo).

   b. Escriba todas las transiciones posibles entre los cinco estados.

(Figura 4.12)

Modelo de proceso de dos estados para el ejercicio 5.

2. Describa el cambio de contexto en los términos establecidos e identifique qué información sobre un proceso es necesario guardar, cambiar o actualizar cuando se lleva a cabo un cambio de contexto.
3. Cinco trabajos (A, B, C, D, E) están en la cola LISTO en espera de ser procesados. Sus ciclos de CPU estimados son como sigue: 2, 10, 15, 6 y 8. Use SJN para encontrar en qué orden deben procesarse a fin de minimizar el tiempo de espera promedio.
4. Un trabajo que funciona en un sistema, con cuantos de tiempo variables por cola, requiere 30 milisegundos para su ejecución hasta su completamiento. Si la primera cola tiene un cuanto de tiempo de 5 milisegundos y cada cola a partir de ahí tiene un cuanto de tiempo igual al doble del cuanto de tiempo previo, ¿cuántas veces será interrumpido el trabajo y en qué cola terminará su ejecución?
5. Describa las ventajas de tener una cola separada para Imprimir E/S y para un Disco E/S como se ilustra en la figura 4.4.
6. Dada la siguiente información:

Trabajo	Tiempo de llegada	Ciclo de CPU
A	0	2
B	1	12
C	2	4
D	4	1
E	5	8
F	7	5
G	8	3

Usando SJN, dibuje una línea de tiempo que muestre el tiempo en el que llega cada trabajo y el orden en el que cada uno es procesado. Calcule el tiempo para terminar cada trabajo.

7. Dada la siguiente información:

Trabajo	Tiempo de llegada	Ciclo de CPU
A	0	10
B	2	12
C	3	3
D	6	1
E	9	15

Dibuje una línea de tiempo para cada uno de los siguientes algoritmos de planificación. (Puede resultar útil calcular un tiempo de inicio y un tiempo de terminación para cada trabajo.)

a. FCFS

b. SJN

c. SRT

d. Round robin (usando un cuanto de tiempo de 5, ignore cambio de contexto y espera natural).

8. Use la misma información dada para el ejercicio 7 para calcular cuáles trabajos pueden tener una llegada lista para ser procesada por el tiempo en el que el primer trabajo es terminado o interrumpido, empleando cada uno de los siguientes algoritmos de planificación.

a. FCFS

b. SJN

c. SRT

d. Round robin (usando un cuanto de tiempo de 5, ignore el cambio de contexto y la espera natural).

9. Use la misma información dada para el ejercicio 7 para calcular el tiempo promedio de espera y el tiempo promedio de respuesta para cualquier trabajo para cada uno de los siguientes algoritmos de planificación (ignore la sobrecarga por cambio de contexto).

a. FCFS

b. SJN

c. SRT

d. Round robin (usando un cuanto de tiempo de 2).

Ejercicios avanzados

10. Considere una variante del round robin donde un proceso que ha usado su cuanto de tiempo completo es devuelto al final de la cola LISTO, mientras otro que ha usado la mitad de su cuanto de tiempo se devuelve a la parte media de la cola y otro

que ha usado la cuarta parte de su cuanto de tiempo se regresa a un sitio ubicado a la cuarta parte de la distancia desde el inicio de la cola.

a. ¿Cuál es el objetivo de esta política de planificación?

b. Analice las ventajas y las desventajas de su implementación.

11. En un sistema dedicado de usuario único, como una computadora personal, es fácil para el usuario determinar cuándo un trabajo está atrapado en un ciclo infinito. La solución típica a este problema es que el usuario intervenga manualmente y termine el trabajo. ¿Qué mecanismo implementaría usted en el planificador de procesos para terminar automáticamente un trabajo que está en un bucle infinito? Tome en cuenta trabajos que de manera legítima usan grandes cantidades de tiempo de CPU; por ejemplo, "encontrar los 10 000 primeros números primos".

12. En este capítulo se proporcionaron algunas directrices para seleccionar el cuanto de tiempo idóneo. Como diseñador del sistema, ¿cómo podría saber cuándo ha escogido el mejor cuanto de tiempo? ¿Qué factores hacen de este cuanto de tiempo el mejor desde el punto de vista del usuario? ¿Qué factores hacen de este cuanto de tiempo el mejor desde el punto de vista del sistema?

13. Use los diagramas de estado de proceso de la figura 4.2 para explicar por qué no hay transición:

    a. Del estado LISTO al estado de ESPERA.

    b. Del estado de ESPERA al estado en EJECUCIÓN.

Ejercicios de programación

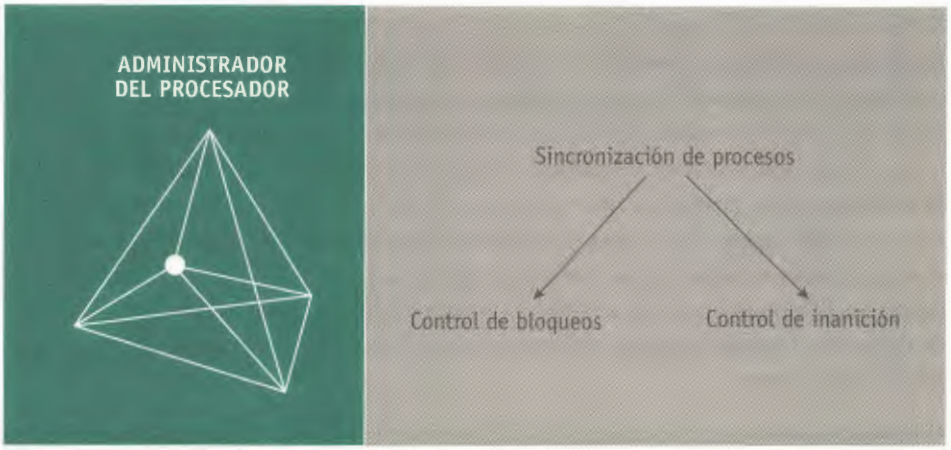
14. Escriba un programa que simule los algoritmos de planificación FCFS, SJN, SRT y round robin. Para cada algoritmo, el programa debe calcular el tiempo de espera y el tiempo de respuesta de cada trabajo, así como el tiempo promedio de espera y el tiempo promedio de respuesta. Los valores promedio deben tabularse para efectuar comparaciones fáciles. Puede usar los datos siguientes para probar su programa. El cuanto de tiempo para round robin es 4 milisegundos y el tiempo de cambio de contexto es 0.

Tiempo de llegada	Ciclo de CPU (en milisegundos)
0	6
3	2
5	1
9	7
10	5
12	3

14	4
16	5
17	7
19	2

15. Use su programa del ejercicio 14 para cambiar el tiempo de cambio de contexto a 0.4 milisegundos. Compare sus resultados de ambas ejecuciones y analice cuál es la mejor política. Describa cualquier cambio drástico encontrado o la falta de cambios y por qué.

Capítulo 5 | Administración de procesos



"Todos hemos escuchado la historia del animal que duda entre dos pilas de heno y muere de inanición."

Abraham Lincoln (*1809-1865*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Varias causas del bloqueo mutuo y bloqueo activo del sistema.
- La diferencia entre prevenir y evitar bloqueos mutuos.
- Cómo detectar bloqueos mutuos y recuperarse de ellos.
- El concepto de inanición de un proceso, cómo detectarlo y recuperarse de él.
- El concepto de competencia y cómo prevenirlo.
- La diferencia entre bloqueo mutuo, inanición y competencia.

Ya hemos abordado dos aspectos de la compartición de recursos desde dos perspectivas, la compartición de la memoria y compartición del procesador, pero el procesador compartido descrito está muy lejos del escenario del mejor caso, libre de conflictos y complicaciones. En este capítulo analizamos los problemas ocasionados cuando muchos procesos compiten por relativamente pocos recursos y el sistema es incapaz de atender todos los procesos que hay en el sistema.

Ya vimos cómo la falta de sincronización de procesos puede resultar en dos situaciones extremas: bloqueo mutuo o inanición.

En los primeros sistemas operativos, el bloqueo mutuo era conocido por la frase más descriptiva "**abrazo de la muerte**" y esto es lo que sucede exactamente cuando el sistema se congela. Es un gran enredo de solicitudes de recursos en el sistema que empieza cuando dos o más trabajos se colocan en estado de retención, en espera que algún recurso vital esté disponible. El problema surge cuando los recursos necesarios para ese trabajo son los mismos en posesión de otros trabajos que están esperando su ejecución pero no es posible porque están esperando otros recursos que no están disponibles. Los trabajos llegan a una situación de detención. El bloqueo mutuo se completa si el resto del sistema también llega a una situación de detención. Cuando el sistema operativo no puede resolver esta situación, se requiere la intervención externa.

Un **bloqueo mutuo** puede describirse más fácilmente con un ejemplo: una escalera estrecha en un edificio (volveremos a este ejemplo a lo largo de todo este capítulo). La escalera fue construida como ruta de escape en caso de incendio, pero quienes trabajan en el edificio a menudo la usan en lugar de esperar los lentos ascensores. La circulación en la escalera es fluida a menos que dos personas, en direcciones opuestas, se encuentren frente a frente; sólo hay espacio para una persona. En este ejemplo, la escalera es el sistema y los escalones y descansillos son los recursos. Entre cada piso hay un descansillo suficientemente ancho para que lo compartan las personas, pero la escalera no lo es, de modo que sólo es posible que la use una persona a la vez. Hay problemas cuando alguien que sube por la escalera encuentra a alguien que baja y ninguna acepta retirarse a un descansillo para ceder el paso a la otra. Esto ocasiona un bloqueo mutuo, que constituye el tema principal de nuestro análisis sobre sincronización de procesos.

Del mismo modo, si dos personas al bajar por una escalera intentan pasarse una a otra, no pueden hacerlo porque uno de sus pasos va a la derecha y el otro va a la izquierda y viceversa, entonces los escaladores continuarán moviéndose, pero nunca avanzarán. Esto se denomina **livelock** o bloqueo activo.

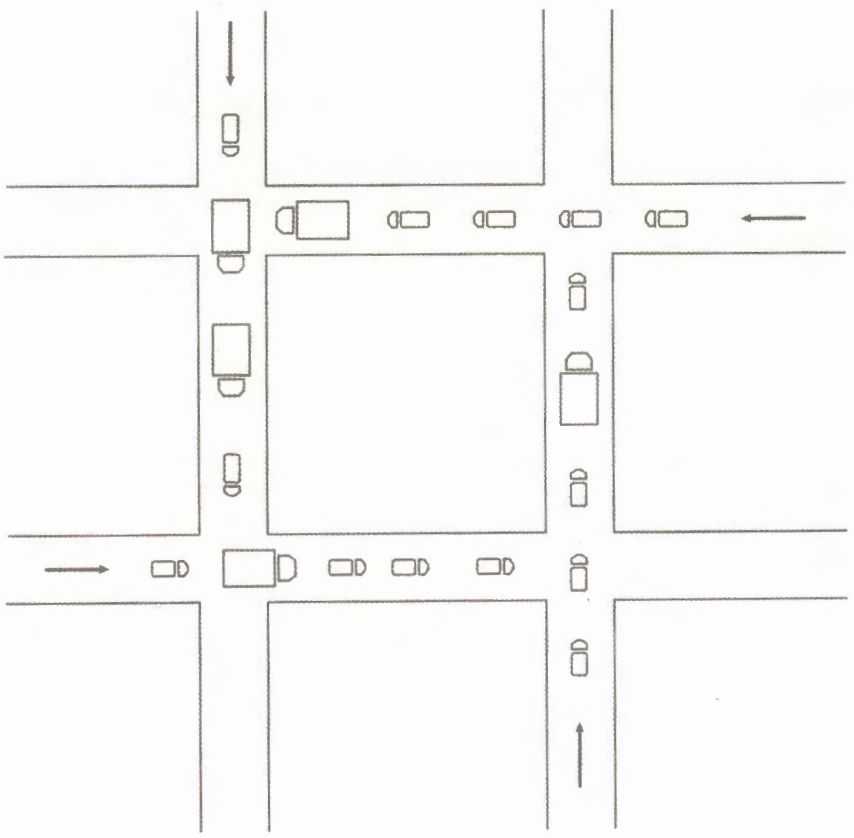
Por otra parte, si algunas personas poco pacientes esperan en el descansillo a que el tráfico opuesto fluya más y eso no ocurre nunca, entonces pueden esperar por siempre. Esto resulta en **inanición**, un caso extremo de aplazamiento indefinido, que se analiza más tarde en este capítulo.

Bloqueo mutuo

El bloqueo mutuo es más grave que el aplazamiento indefinido o la inanición porque afecta a más de un trabajo. Debido a que los recursos se están atando, todo el sistema (no sólo unos cuantos programas) se ve afectado. El ejemplo que se presenta más a menudo para ilustrar el bloqueo mutuo es un embotellamiento de tráfico.

Como se muestra en la figura 5.1, para un bloqueo mutuo no hay ninguna solución simple e inmediata; nadie puede moverse hacia delante hasta que alguien se quite del camino, pero nadie puede hacerlo hasta que alguien avance o la parte posterior de una línea retroceda. Resulta evidente que para esto se requiere intervención externa para quitar uno de los cuatro vehículos en una intersección o para hacer retroceder una línea. Sólo entonces es posible resolver el bloqueo mutuo.

Los bloqueos mutuos se volvieron más frecuentes con la introducción de los sistemas interactivos, los que en general mejoran el uso de recursos a través de una compartición dinámica de recursos, aunque esta capacidad para compartir recursos también incrementa la posibilidad de bloqueos mutuos.



(Figura 5.1)

Un caso clásico de bloquea mutuo de tráfico en cuatro calles de un solo sentido. Se trata de un "embotellamiento", donde ningún vehículo puede moverse hacia delante para liberar el embotellamiento.

En algunos sistemas de cómputo, los bloqueos mutuos se consideran como simples inconvenientes que pueden ocasionar retrasos. Pero para los sistemas en tiempo real, los bloqueos mutuos pronto se vuelven situaciones críticas. Por ejemplo, en un hospital un bloqueo mutuo en el sistema de apoyo vital, o en el sistema de orientación a bordo de una nave espacial, puede poner en peligro vidas. Sin importar el entorno, el sistema operativo debe prevenir bloqueos mutuos o resolverlos cuando sucedan. En el capítulo 12, veremos cómo calcular la confiabilidad y disponibilidad del sistema, las cuales pueden ser afectadas por conflictos en el procesador.

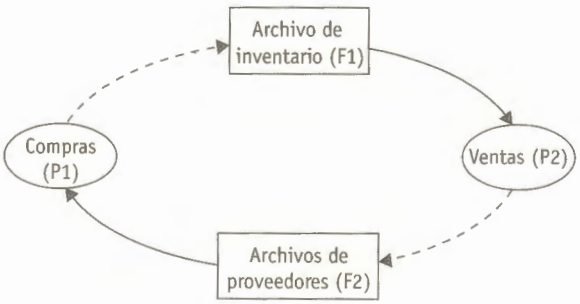
Siete casos de bloqueo mutuo

Un bloqueo mutuo suele ocurrir cuando recursos no compartibles y no expropiativos, como archivos, impresoras o escáneres, se asignan a trabajos que finalmente requieren otros recursos compartibles no expropiativos: recursos que han sido bloqueados por otros trabajos. Sin embargo, los bloqueos mutuos no están restringidos a archivos, impresoras y escáneres. También pueden ocurrir en recursos compartibles, como discos y bases de datos.

Las gráficas representan directamente de manera visual los recursos y procesos del sistema, y muestran cómo se bloquean mutuamente. Utilizando series de cuadrados (para los recursos), círculos (para los procesos) y conectores con flechas (para las solicitudes), las gráficas dirigidas pueden manipularse para entender cómo ocurre un bloqueo mutuo.

Caso 1: Bloqueos mutuos en solicitudes de archivos

Si a los trabajos se les permite solicitar y retener archivos durante la duración de su ejecución, puede ocurrir un bloqueo, como lo muestra de forma simplificada la gráfica dirigida en la figura 5.2.



(Figura 5.2)

Caso 1. Estos dos procesos se muestran como círculos, cada uno está esperando por un archivo, mostrado como rectángulos, que ya ha sido asignado al otro proceso, creando así un bloqueo mutuo en el sistema.

Por ejemplo, considere el caso de una empresa de construcción de casas con dos programas de aplicación., compras (P1) y ventas (P2), que están activos al mismo tiempo. Ambos requieren tener acceso a dos archivos: inventario (F1) y proveedores (F2) para leer y registrar sus transacciones. Un día, el sistema se bloquea cuando ocurre la siguiente secuencia de eventos:

1. Compras (P1) accede al archivo del proveedor (F2) para colocar una orden por más madera.
2. Ventas (P2) accede al archivo de inventario (F1) a fin de reservar las partes que serán requeridas para construir la casa ordenada ese día.
3. Compras (P1) no libera el archivo del proveedor (F2), sino que solicita el archivo de inventario (F1) para verificar la cantidad de madera a la mano antes de colocar su orden por más madera, pero P1 está bloqueado porque P2 está reteniendo a F1.
4. Mientras tanto, ventas (P2) no libera el archivo de inventario (F1), sino que solicita que el archivo del proveedor (F2) compruebe el calendario de un subcontratista. En este punto P2 también está bloqueado porque P1 está reteniendo a F2.

Cualquier otro proceso que requiera a F1 o F2 se pone en estado de espera mientras dura esta situación. Este bloqueo permanece hasta que uno de los dos procesos se cierra o es retirado por la fuerza y su archivo se libera. Sólo entonces es posible que los otros procesos continúen y el sistema vuelva a la situación normal.

Caso 2: Bloqueos mutuos en bases de datos

Un bloqueo mutuo también puede ocurrir si dos procesos tienen acceso a una base de datos y bloquean registros.

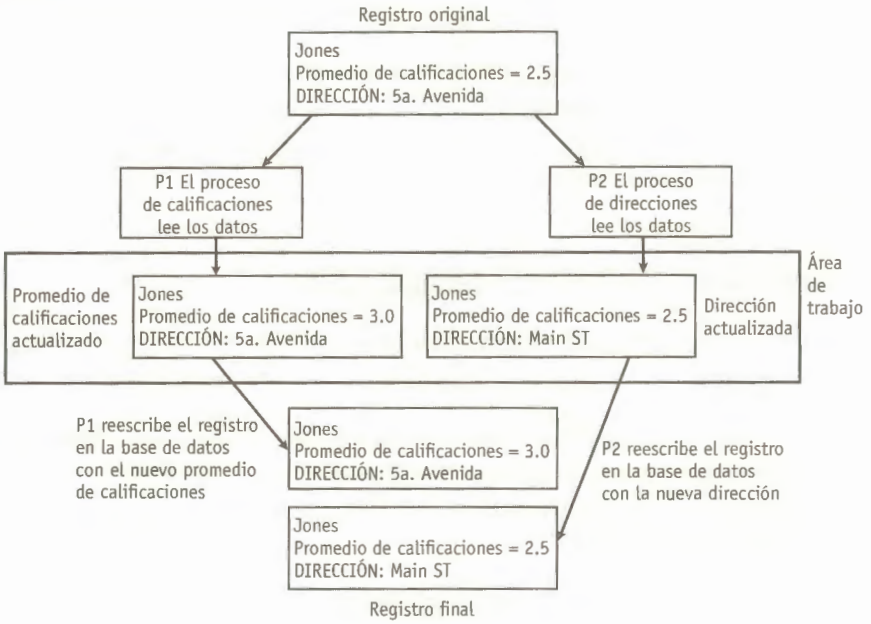
Para apreciar el siguiente escenario es necesario recordar que las consultas y transacciones de bases de datos a menudo son procesos relativamente breves que buscan o modifican partes de una base de datos. Las solicitudes suelen llegar en forma aleatoria y pueden intercalarse de manera arbitraria.

Bloquear es una técnica usada para garantizar la integridad de los datos, por medio de la cual el usuario bloquea a todos los otros usuarios mientras está trabajando con esa base de datos. Bloquear puede hacerse en tres niveles diferentes: puede bloquearse toda la base de datos mientras dura la solicitud; puede bloquearse una subsección de la base de datos; o puede bloquearse sólo el archivo individual hasta que se completa el proceso. Bloquear toda la base de datos (la solución más extrema y exitosa) previene la ocurrencia de un bloqueo mutuo pero restringe el acceso a la base de datos a un usuario a la vez y, en un entorno multiusuario, los tiempos de respuesta aminoran significativamente; esta solución es normalmente inaceptable. Cuando el bloqueo se lleva a cabo en sólo una parte de la base de datos, el tiempo de acceso mejora pero la posibilidad de un bloqueo mutuo aumenta porque procesos diferentes a veces necesitan trabajar con varias partes de la base de datos al mismo tiempo.

A continuación se presenta un sistema que bloquea cada registro cuando se tiene acceso a éste hasta que se completa el proceso. Hay dos procesos (P1 y P2), cada uno de los cuales necesita actualizar dos registros (R1 y R2), y la siguiente secuencia conduce a un bloqueo mutuo:

1. P1 accede a R1 y lo bloquea.
2. P2 accede a R2 y lo bloquea.
3. P1 solicita R2, que está bloqueado por P2.
4. P2 solicita R1, que está bloqueado por P1.

Una alternativa, por supuesto, es no usar bloqueos, aunque esto lleva a otras dificultades. Si no se usan bloqueos para preservar la integridad de los registros actualizados, éstos en la base de datos podrían incluir sólo algunos de los datos, y su contenido dependería del orden en que cada proceso termina su ejecución. Esto se denomina **competencia** entre procesos y se ilustra en el siguiente ejemplo y en la figura 5.3.



(Figura 5.3)

Caso 2. P1 termina primero y gana la competencia pero su versión del registro pronto será sobrescrita por P2. Sin importar cuál proceso gane la competencia, la versión final de los datos es incorrecta.

Supongamos que usted es un estudiante de una universidad que mantiene la mayor parte de sus archivos en una base de datos a la que varios programas pueden tener acceso, incluyendo uno para calificaciones y otro para la lista de direcciones de los domicilios. Usted acaba de enviar a la universidad una notificación de cambio de domicilio a partir del final del semestre de verano, poco después que se han entregado las calificaciones. Y un día fatal ambos programas compiten por tener acceso a su registro personal en la base de datos:

1. El proceso de calificaciones (P1) es el primero en acceder a su registro (R1) y copia el registro en su área de trabajo.
2. El proceso de direcciones (P2) accede a su registro (R1) y lo copia en su área de trabajo.

Una competencia introduce el elemento de azar, que es totalmente inaceptable en administración de bases de datos. Es necesario mantener la integridad de la base de datos.

3. P1 cambia su registro de estudiante (R1) al introducir sus calificaciones del semestre de verano y calcular su nuevo promedio de calificaciones.
4. P2 cambia su registro (R1) al actualizar el campo de dirección.
5. P1 termina su trabajo primero y vuelve a escribir su versión de su registro de vuelta en la base de datos. Sus calificaciones se han actualizado, pero no así su dirección.
6. P2 termina y vuelve a escribir su versión de su registro actualizado de nuevo en la base de datos. Su dirección ha sido modificada, pero no así sus calificaciones. Según la base de datos, usted no asistió a la escuela este semestre.

Si invertimos el orden y decimos que P2 ganó la carrera, sus calificaciones estarán actualizadas, pero no así su dirección. Dependiendo de su éxito en el aula, tal vez usted prefiera una desgracia después de la otra, pero desde el punto de vista del sistema operativo cualquier alternativa es inaceptable porque se ha permitido que datos incorrectos contaminen la base de datos. El sistema no puede permitir que la integridad de la base de datos dependa de una secuencia de eventos aleatorios.

Caso 3: Bloqueos mutuos en asignación de dispositivos dedicados

El uso de un grupo de dispositivos dedicados, como un grupo de unidades de lectura/escritura de DVD, también puede bloquear el sistema.

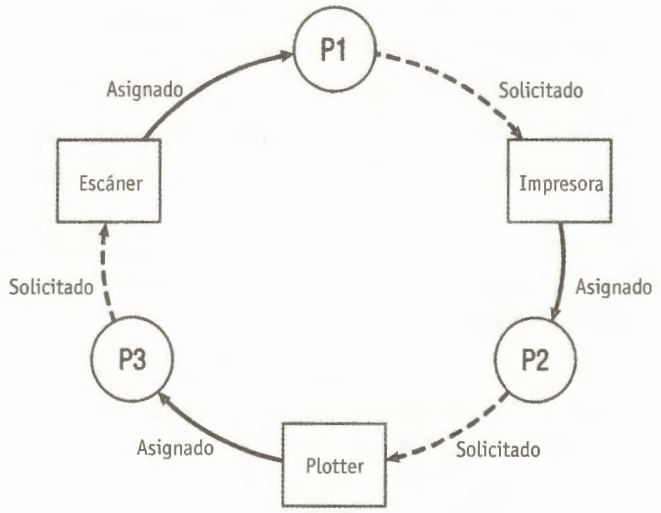
Considere que dos usuarios de la junta local de educación están ejecutando, cada uno, un programa (P1 y P2) y que ambos procesos van a requerir dos unidades de DVD para copiar archivos de un disco a otro. El sistema es pequeño, no obstante, y cuando se inician dos procesos, sólo se dispone de dos unidades de DVD-R, que son asignadas aplicando el criterio "según sea necesaria". Pronto se tiene la siguiente secuencia:

1. P1 solicita el lector 1 y lo obtiene.
2. P2 solicita el lector 2 y lo obtiene.
3. P1 solicita el lector 2 pero está bloqueado.
4. P2 solicita el lector 1 pero está bloqueado.

Ningún trabajo puede continuar porque cada uno está esperando que el otro termine y libere su unidad de cinta: un evento que jamás ocurre. Una serie de eventos semejante puede bloquear cualquier grupo de dispositivos dedicados.

Caso 4: Bloqueos mutuos en asignación de dispositivos múltiples

Los bloqueos mutuos no están restringidos a procesos que compiten por el mismo tipo de dispositivo; pueden ocurrir cuando varios procesos solicitan, y retienen, varios dispositivos dedicados mientras otros procesos actúan de manera semejante como se muestra en la figura 5.4.



(Figura 5.4)

Caso 4. Cada uno de los tres procesos, mostrados con círculos, están esperando un dispositivo que ya ha sido asignado a otro proceso, creando así un bloqueo mutuo.

Considere el caso de una empresa de diseño industrial con tres procesos (P1, P2 y P3) y tres dispositivos dedicados: escáner, impresora y plotter. La siguiente secuencia de eventos da por resultado un bloqueo mutuo:

1. P1 solicita y obtiene el escáner.
2. P2 solicita y obtiene la impresora.
3. P3 solicita y obtiene el plotter.
4. P1 solicita la impresora pero está bloqueado.
5. P2 solicita el plotter pero está bloqueado.
6. P3 solicita el escáner pero está bloqueado.

Así como ocurre en los primeros ejemplos, ninguno de los trabajos puede continuar porque cada uno está esperando un recurso que tienen los otros.

Caso 5: Bloqueos en *spooling*

Aunque en el ejemplo previo la impresora era un dispositivo dedicado, la mayor parte de los dispositivos han convertido la impresora en un dispositivo compartible (también denominada "dispositivo virtual") que usa un dispositivo de almacenamiento de alta velocidad, para transferir datos entre ésta y la CPU. El sistema o *spooler* acepta resultados de varios usuarios y actúa como un área de almacenamiento temporal para todos los resultados hasta que la impresora está lista para aceptarlos. Este procesamiento se denomina ***spooling***. Si la impresora necesita todos los resultados de un trabajo antes de empezar a imprimir, pero el sistema de *spooling* llena el espacio disponible con un resultado parcialmente completo, entonces puede ocurrir un bloqueo mutuo. Esto ocurre como se describe a continuación.

Considere que falta una hora para entregar el gran proyecto en un curso de computación. Veintiséis programadores desesperados están ocupados escribiendo los últimos detalles y, con sólo unos minutos disponibles, todos emiten comandos de impresión. El *spooler* recibe las páginas una por una de cada uno de los estudiantes pero las recibe por separado; varias

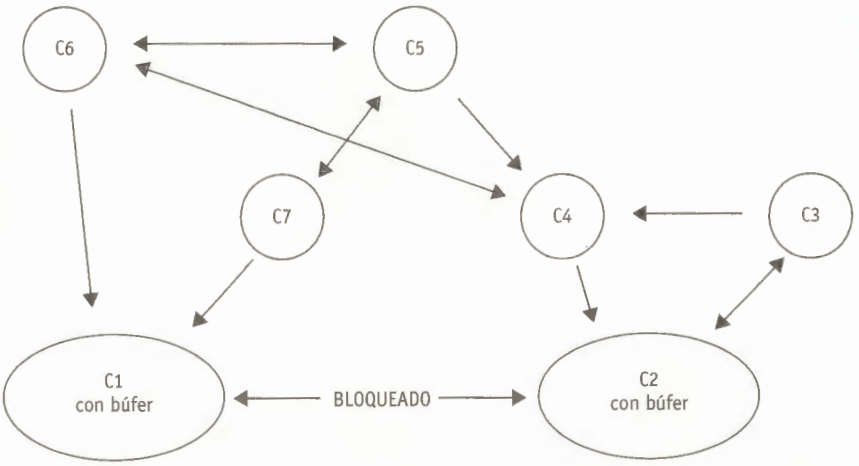
páginas uno, páginas dos, etc. La impresora está lista para imprimir el primer documento completo que obtiene, pero a medida que el *spooler* consulta sus archivos, tiene la primera página pero no la última de ninguno de ellos. ¡Lástima!, el *spooler* está lleno de resultados parcialmente completados, de modo que ya no es posible aceptar otras páginas pero ninguno de los trabajos puede imprimirse (con lo cual se liberaría el espacio de su disco) porque la impresora sólo acepta archivos de salida completos. Un lamentable estado de cosas.

Este escenario no está limitado a las impresoras. Cualquier parte del sistema que depende del *spooling*, como la que maneja las partes de entrada o transfiere archivos sobre la red, es vulnerable a tal bloqueo mutuo.

Caso 6: Bloqueos en una red

Una red congestionada o que ha llenado un gran porcentaje de su espacio de búfer de E/S puede llegar a bloquearse si no cuenta con protocolos para controlar el flujo de mensajes a través de la red como se muestra en la figura 5.5.

(Figura 5.5)
Flujo bloqueado en una red. Observe que sólo dos nodos, C1 y C2, tienen búfer. Cada círculo representa un nodo y cada línea representa una ruta de comunicación. Las flechas indican la dirección del flujo de datos.



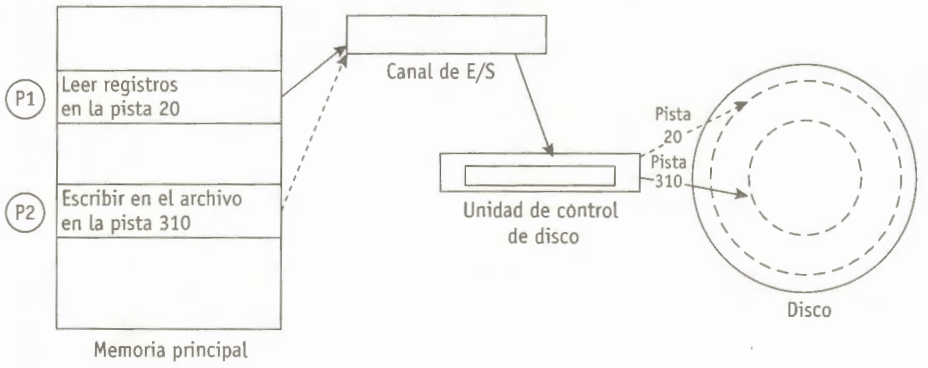
Por ejemplo, un centro de procesamiento de palabras de tamaño medio tiene siete computadoras en una red, cada una en nodos diferentes. C1 recibe mensajes de los nodos C2, C6 y C7 y envía mensajes sólo a uno: C2. C2 recibe mensajes de los nodos C1, C3 y C4 y envía mensajes sólo a C1 y C3. La dirección de las flechas en la figura 5.5 indica el flujo de los mensajes.

Los mensajes recibidos por C1 provenientes de C6 y C7 y cuyo destino es C2 son enviados a almacenamiento intermedio en una cola de salida. Los mensajes recibidos por C2 provenientes de C3 y C4 y cuyo destino es C1 son enviados a almacenamiento intermedio en una cola de salida. A medida que se incrementa el tráfico, la longitud de cada cola de salida aumenta hasta que se ha llenado todo el espacio de almacenamiento intermedio. En este momento C1 ya no puede aceptar ningún otro mensaje (de C2 o de cualquier otra computadora) porque ya no hay más espacio de búfer disponible para almacenarlos. Por la misma razón, C2 ya no puede aceptar más mensajes de C1 o de cualquier otra computadora, ni siquiera una solicitud a enviar. La ruta de comunicación entre C1 y C2 se bloquea y debido a que C1 no puede

enviar mensajes a ninguna otra computadora excepto C2 y sólo puede recibir mensajes de C6 y C7, estas rutas también se bloquean. C1 no puede enviar ninguna palabra a C2 sobre el problema y así el bloqueo no puede resolverse sin intervención externa.

Caso 7: Bloqueos en compartición de discos

Los discos están diseñados para ser compartidos, de modo que no es inusual que dos procesos que tengan acceso a áreas diferentes del mismo disco. Esta capacidad para compartir crea un tipo activo de bloqueo, denominado **bloqueo activo**. Los procesos usan una forma de espera activa que es diferente de una espera natural. En este caso está esperando compartir un recurso pero en realidad nunca obtiene el control de éste. En la figura 5.6, dos procesos competidores envían comandos contradictorios, ocasionando un bloqueo activo. Observe que ninguno de los procesos es bloqueado, lo que puede causar un bloqueo mutuo. En cambio, cada uno está activo pero nunca alcanza a completarse.



(Figura 5.6)

Caso 7. Cada uno de los dos procesos está esperando satisfacer una solicitud de E/S: una en la pista 20 y otra en la pista 310. Pero por el tiempo que tarda el brazo de lectura/escritura en alcanzar una pista, un comando competidor para la otra pista está siendo emitido, así que ningún comando es satisfecho y ocurre un bloqueo mutuo.

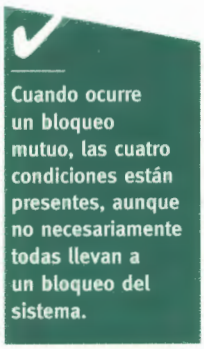
Por ejemplo, en una compañía de seguros, el sistema realiza muchas transacciones diarias. Un día, la siguiente serie de eventos ata al sistema:

1. Servicio al cliente (P1) desea mostrar un pago, de modo que emite un comando para leer el balance, que se almacena en la pista 20 de un paquete de discos.
2. Mientras la unidad de control mueve el brazo a la pista 20, P1 se coloca en estado retenido y el canal de E/S está libre para procesar la siguiente solicitud de E/S.
3. Mientras el brazo se está moviendo a su posición, Cuentas por pagar (P2) obtiene el control del canal de E/S y emite un comando para escribir el pago de otra persona en un registro almacenado en la pista 310. Si el comando no se "bloquea", P2 se coloca en estado de retenido mientras la unidad de control mueve el brazo a la pista 310.
4. Debido a que P2 está "retenido" mientras el brazo se mueve, el canal puede ser capturado de nuevo por P1, que reconfirma su comando de "leer desde la pista 20".
5. Debido a que el último comando de P2 ha forzado el mecanismo del brazo a la pista 310, la unidad de control de disco empieza a reacomodar el brazo en la pista 20 para satisfacer a P1. El canal de E/S puede liberarse porque P1 de nuevo se coloca en estado retenido, de modo que puede ser capturado por P2, que emite un comando WRITE sólo para descubrir que es necesario reacomodar el mecanismo del brazo.

Como resultado, el brazo está en un estado de movimiento continuo, moviéndose de un lado a otro entre la pista 20 y la pista 310 a medida que responde a los dos comandos competidores, aunque no satisface a ninguno.

Condiciones para bloqueos

En cada uno de los siete casos, el bloqueo mutuo (o bloqueo activo) implicó la interacción de varios procesos y recursos, pero cada bloqueo mutuo estaba precedido por la ocurrencia simultánea de cuatro condiciones que el sistema operativo (u otros sistemas) puede reconocer: exclusión mutua, retención de recursos, inexistencia de expropiabilidad y espera circular. Es importante recordar que cada una de estas cuatro condiciones es necesaria para que el sistema operativo funcione sin contratiempos. Ninguna de ellas puede removerse fácilmente sin ocasionar problemas al funcionamiento global del sistema. Por consiguiente, el sistema necesita reconocer la combinación de condiciones antes que ocurran y amenacen con provocar un bloqueo del sistema.



Para ilustrar estas cuatro condiciones, revisaremos el ejemplo de la escalera presentado al inicio de este capítulo e identificaremos las cuatro condiciones requeridas para un bloqueo mutuo.

Cuando dos personas se encuentran entre descansillos no pueden pasar porque la escalera es demasiado estrecha y sólo cabe una persona a la vez. La **exclusión mutua**, el acto de permitir que sólo una persona tenga acceso a un escalón (un recurso dedicado), es la primera condición para un bloqueo mutuo.

Cuando dos personas se encuentran en las escaleras y cada una permanece parada en espera que la otra se retire constituye un ejemplo de **retención de recursos** (en oposición a la compartición de recursos), la segunda condición para un bloqueo mutuo.

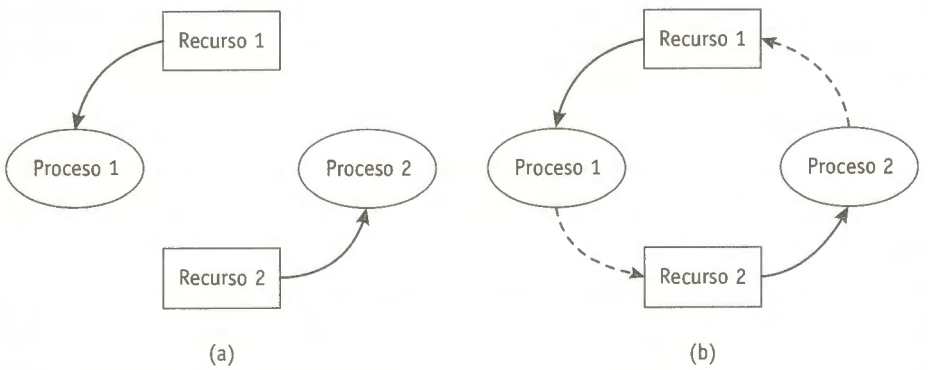
En este ejemplo, cada escalón está dedicado para la persona que sube (o baja); se dedica a quien se encuentre ahí durante el tiempo que lo requiera. Esto se denomina **sin expropiabilidad**, que es la falta de reasignación temporal de recursos, y es la tercera condición para bloqueo mutuo.

Estas tres condiciones conducen a la cuarta condición de **espera circular** donde cada persona (o proceso) implicada en este callejón sin salida espera que la otra libere voluntariamente el escalón (o el recurso), de modo que por lo menos una pueda continuar y llegar finalmente a su destino.

Para que ocurra bloqueo mutuo se requieren las cuatro condiciones y mientras todas están presentes, el bloqueo continúa; pero si es posible remover una condición, entonces se resuelve el bloqueo mutuo. De hecho, si es posible prevenir que las cuatro condiciones ocurran al mismo tiempo, es posible prevenir el bloqueo mutuo. Aunque este concepto es evidente, no es fácil de implementar.

Modelado de bloqueos mutuos

Holt demostró la manera en que las cuatro condiciones pueden modelarse usando **gráficas dirigidas**. (Aquí usamos gráficas modificadas en la figura 5.2 y en la figura 5.4.) En estas gráficas se usan dos tipos de símbolos: procesos, representados por círculos y recursos, representados por cuadrados. Una flecha continua de un recurso a un proceso, mostrada en la figura 5.7*a*, significa que el proceso está reteniendo ese recurso. Una línea discontinua con una flecha de un proceso a un recurso, mostrada en la figura 5.7*b*, significa que el proceso está esperando ese recurso. La dirección de la flecha indica el flujo. Si en la gráfica hay un círculo, entonces hay un bloqueo mutuo implicando los procesos y los recursos en el ciclo, como se muestra en la figura 5.7*c*.



(Figura 5.7)
En (a), El recurso R1 es retenido por el proceso P1 y el recurso R2 es retenido por el proceso P2 en un sistema que no es bloqueado mutuamente. En (b), el proceso P1 solicita al recurso R2 pero no puede liberar al recurso R1 y el proceso P2 hace lo mismo creando un bloqueo mutuo. (Si uno de los procesos liberados es un recurso, el bloqueo mutuo se resuelve.)

El siguiente sistema tiene tres procesos: P1, P2 y P3, y tres recursos: R1, R2 y R3, cada uno de un tipo diferente: impresora, unidad de disco y plotter. Debido a que no hay un orden específico para manejar las tres solicitudes, analizaremos los tres escenarios posibles usando gráficas para detectar bloqueos mutuos.

Escenario 1

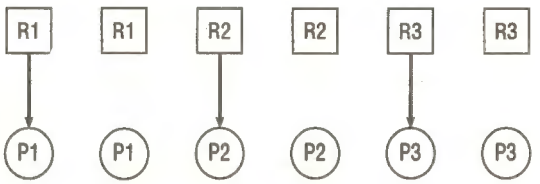
La secuencia de eventos del escenario 1 se muestra en la tabla 5.1. La gráfica dirigida se muestra en la figura 5.8.

Evento	Acción
1	P1 solicita y se le asigna la impresora R1.
2	P1 libera la impresora R1.
3	P2 solicita y se le asigna la unidad de disco R2.
4	P2 libera el disco R2.
5	P3 solicita y se le asigna el plotter R3.
6	P3 libera el plotter R3.

(Tabla 5.1)
La secuencia de eventos del escenario 1 se muestra en la gráfica dirigida en la figura 5.8.

Observe en la gráfica dirigida que no hay ciclos. En consecuencia, puede concluirse con seguridad que un bloqueo mutuo no puede ocurrir incluso si cada proceso solicita cualquier recurso *si* los recursos se liberan antes que los solicite el siguiente proceso.

(Figura 5.8)
Primer escenario. El sistema permanece libre de bloqueos mutuos si todos los recursos se liberan antes que los solicite el siguiente proceso.



Escenario 2

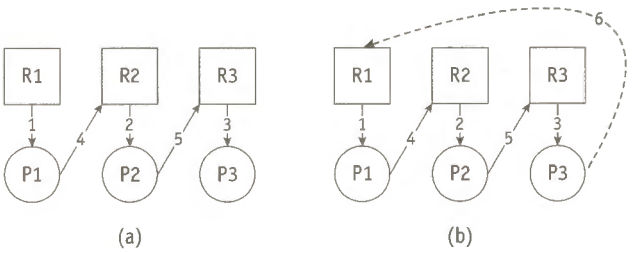
Ahora, consideraremos una secuencia de eventos del segundo escenario mostrada en la tabla 5.2.

(Tabla 5.2)
La secuencia de eventos del escenario 2 se muestra en la gráfica dirigida en la figura 5.9.

Evento	Acción
1	P1 solicita y se le asigna la impresora R1.
2	P2 solicita y se le asigna la impresora R2.
3	P3 solicita y se le asigna R3.
4	P1 solicita R2.
5	P2 solicita R3.
6	P3 solicita R1.

El progreso de la gráfica dirigida se muestra en la figura 5.9. Ocurre un bloqueo mutuo porque todos los procesos están esperando un recurso retenido por otro proceso, pero ninguno es liberado sin la intervención del operador.

(Figura 5.9)
Segundo escenario. El sistema (a) se bloquea mutuamente (b) cuando P3 solicita R1. Observe la espera circular.



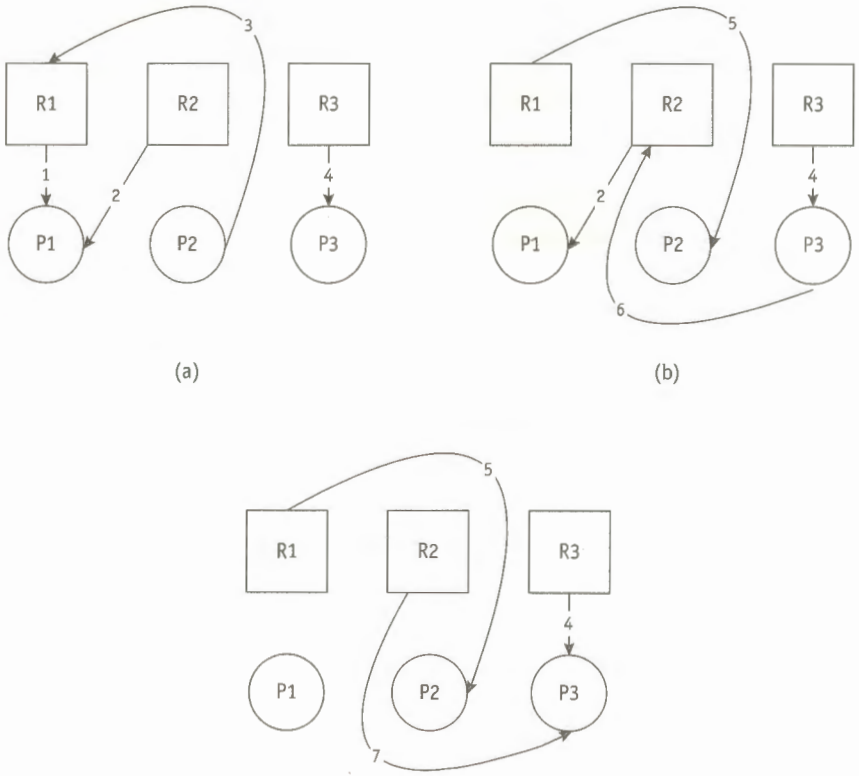
Escenario 3

El tercer escenario se muestra en la tabla 5.3. Como se muestra en la figura 5.10, los recursos se liberan antes que ocurra un bloqueo mutuo.

Evento	Acción
1	P1 solicita y se le asigna la impresora R1.
2	P1 solicita y se le asigna la impresora R2.
3	P2 solicita R1.
4	P3 solicita y se le asigna R3.
5	P1 libera R1, que se asigna a P2.
6	P3 solicita R2.
7	P1 libera R2, que se asigna a P3.

(Tabla 5.3)

La secuencia de eventos del tercer escenario se muestra en la gráfica dirigida en la figura 5.10.



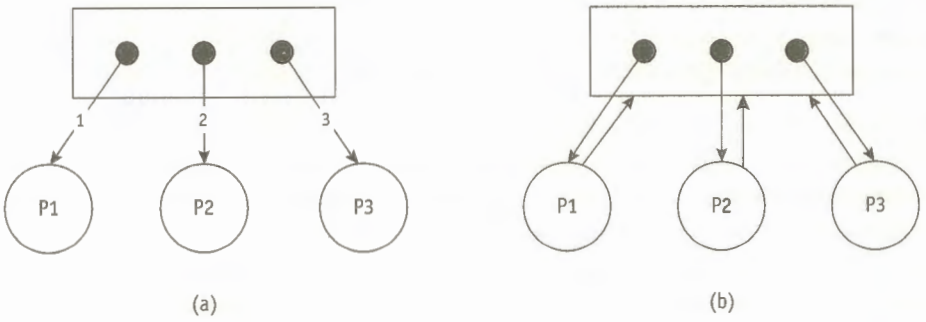
(Figura 5.10)

El tercer escenario. Después del evento 4, la gráfica dirigida es como (a) y P_2 está bloqueado porque P_1 está retenido sobre R_1. Sin embargo, el evento 5 rompe el bloqueo y pronto la gráfica se ve como (b). De nuevo hay un proceso bloqueado, P_3, que debe esperar la liberación de R_2 en el evento 7 cuando la gráfica es como (c).

Otro ejemplo

En los ejemplos presentados hasta el momento se han analizado casos en los que uno o más recursos de diferentes tipos se han asignado a un proceso. Sin embargo, las gráficas pueden expandirse para incluir varios recursos del mismo tipo, como unidades de cinta, que pueden asignarse individualmente o en grupos al mismo proceso. Estas gráficas agrupan los dispositivos del mismo tipo en una entidad, mostrada en la figura 5.11 como un rectángulo, y las flechas muestran los vínculos entre el recurso único y los procesos que lo están usando.

La figura 5.11 constituye el ejemplo de un grupo con tres recursos del mismo tipo, como tres unidades de disco, cada una asignada a un proceso diferente. Aunque la figura 5.11*a* parece estable (que no puede ocurrir ningún bloqueo), éste no es el caso porque si los tres procesos solicitan un recurso más sin liberar el que están usando, entonces ocurre un bloqueo mutuo como se muestra en la figura 5.11*b*.



(Figura 5.11) *Un grupo de recursos totalmente asignados. Hay tontos recursos como líneas que salen de cada rectángulo. El estado (a) es incierto, ya que una solicitud por otra unidad de parte de los tres procesos origina un bloqueo mutuo como se muestra en (b).*

Estrategias para manejo de bloqueos

Como muestran estos ejemplos, las solicitudes y las liberaciones se reciben en orden impredecible, lo cual hace bastante difícil diseñar una política a prueba de errores. En general, los sistemas operativos usan una de tres estrategias para ocuparse de los bloqueos mutuos:

- Prevenir la ocurrencia de una de las cuatro condiciones (**prevención**).
- Evitar el bloqueo mutuo si se vuelve probable (**evasión**).
- Detectar el bloqueo mutuo cuando ocurre y recuperarse de éste con elegancia (**detección**).

Prevención

Para prevenir un bloqueo mutuo, el sistema operativo debe eliminar una de las cuatro condiciones necesarias, una tarea complicada por el hecho de que la misma condición no puede eliminarse en cada recurso.

La exclusión mutua es necesaria en cualquier sistema de cómputo porque algunos recursos como memoria, CPU o dispositivos dedicados deben asignarse de manera exclusiva a un usuario a la vez. En el caso de los dispositivos de E/S, como impresoras, la exclusión mutua puede evitarse por medio de *spooling*, que permite que el resultado de muchos trabajos se almacene en archivos *spool* temporales al mismo tiempo, y luego cada archivo de salida completo se selecciona para impresión cuando el dispositivo está listo. Sin embargo, es posible negociar un tipo de bloqueo mutuo (caso 3): Bloqueos mutuos en asignación de dispositivos dedicados) por otro (caso 5: Bloqueos en *Spooling*).

La retención de recursos, donde un trabajo detiene un recurso mientras espera por otro que aún no está disponible, puede soslayarse forzando a cada trabajo a solicitar, en el momento de su creación, todos los recursos que necesita para su ejecución hasta el final. Por ejemplo, si a todos los trabajos en un sistema por lotes se les proporciona tanta memoria como requieren, entonces el número de trabajos activos es determinado por el número de ellos que pueden caber en la memoria: una política que reduce significativamente el grado de multiprogramación. Además, los dispositivos periféricos permanecerían ociosos porque se asignarían a un trabajo aunque no sean usados todo el tiempo. Como ya mencionamos, esto se usaba exitosamente en entornos por lotes aunque reducía el uso efectivo de recursos y restringía la cantidad de multiprogramación. Pero no funciona tan bien en sistemas interactivos.

Es posible evitar la no expropiación al permitir que el sistema operativo libere recursos de trabajos. Esto puede lograrse si es posible guardar y restituir fácilmente el estado del trabajo, como cuando el trabajo es expropiado en un entorno round robin o una página se intercambia a almacenamiento secundario en un sistema con memoria virtual. Por otra parte, la preferencia de un dispositivo de E/S dedicado (impresora, plotter, unidad de cinta, etcétera), o de archivos durante el proceso de modificación, puede requerir algunas tareas de recuperación desagradables.

La espera circular puede evitarse si el sistema operativo previene la formación de un círculo. Una de estas soluciones fue propuesta por Havender y se basa en un sistema de numeración de los recursos como: impresora = 1, disco = 2, cinta = 3, plotter = 4 y así sucesivamente. El sistema obliga a cada trabajo a solicitar sus recursos en orden ascendente: cualquier dispositivo "número uno" solicitado por el trabajo debe solicitarse primero; cualquier dispositivo "número dos" debe solicitarse a continuación; etcétera. Entonces, si un trabajo necesita una impresora y después un plotter, debe solicitarlos en ese orden: primero la impresora (#1) y luego el plotter (#4). Si el trabajo requiere primero el plotter y luego la impresora, debe solicitar primero la impresora (que es un #1) aun cuando no haya sido usada. Un trabajo puede solicitar una impresora (#1) y luego un disco (#2) y luego una cinta (#3), pero si más tarde necesita otra impresora (#1) en el procesamiento, debe seguir anticipándose a esa necesidad cuando solicita la primera impresora, y hacerlo antes de solicitar el disco.

Este esquema por "orden jerárquico" elimina la posibilidad de una espera circular y garantiza así la remoción de bloqueos mutuos. No requiere que los trabajos planteen sus necesidades de antemano, pero sí que anticipen el orden en que solicitarán recursos. Desde el

punto de vista de un diseñador del sistema, una de las dificultades de este esquema consiste en descubrir el mejor orden para los recursos, de modo que se cumpla con las necesidades de la mayoría de los usuarios. Otra dificultad es la asignación de una clasificación a recursos no físicos como archivos o registros de bases de datos donde no existe ninguna base para asignar un mayor número a uno que a otro.

Evasión

Aun si el sistema operativo no puede remover una de las condiciones para bloqueo mutuo, puede evitar una si el sistema conoce de antemano la secuencia de solicitudes asociadas con cada uno de los procesos activos. Como se ilustró en las gráficas presentadas en las figuras de la 5.7 a la 5.11, hay por lo menos una secuencia de asignación de recursos que permite continuar al trabajo sin que éste llegue a bloquearse.

Uno de tales algoritmos fue propuesto por Dijkstra en 1965 para regular la asignación a fin de evitar bloqueos mutuos. El algoritmo del banquero se basa en un banco con una cantidad de capital fija que opera según los siguientes principios:

- A ningún cliente se le otorga un préstamo que exceda el capital total del banco.
- A todos los clientes se les otorga un límite de crédito máximo cuando abran una cuenta.
- A ningún cliente se le permite un crédito superior al límite.
- La suma de todos los préstamos no debe exceder el capital total del banco.

Para permanecer en un estado seguro, el banco debe contar con fondos suficientes para satisfacer las necesidades de por lo menos un cliente.

Bajo estas condiciones, el banco no es requerido para tener a la mano el total de todas las cuotas de préstamo máximo antes que pueda realizar negocios (supondremos que el banco siempre tiene el mismo total fijo y no consideraremos el interés aplicado a los préstamos). Para nuestro ejemplo, el banco tiene un fondo de capital total de $10 000 y tres clientes, C1, C2 y C3, cuyos límites de crédito máximo son $4 000, $5 000 y $8 000, respectivamente. La tabla 5.4 ilustra el estado de los negocios del banco luego que ha otorgado algunos préstamos a C2 y C3. Éste se denomina **estado seguro** porque el banco aún cuenta con dinero suficiente para satisfacer las solicitudes máximas de C1, C2 o C3.

(Tabla 5.4)

El banco empezó con $10 000 y tiene un capital restante de $4 000 después de estos préstamos. Por tanto, se encuentra en un "estado seguro".

Cliente	Cantidad prestada	Crédito máximo	Crédito restante
C1	0	4 000	4 000
C2	2 000	5 000	3 000
C3	4 000	8 000	4 000
Total prestado: $6 000			
Fondo de capital total: $10 000			

Unas semanas después que se han otorgado algunos préstamos más, y algunos ya se han pagado, el banco se encuentra en el estado inseguro representado en la tabla 5.5.

Cliente	Cantidad prestada	Crédito máximo	Crédito restante
C1	2 000	4 000	2 000
C2	3 000	5 000	2 000
C3	4 000	8 000	4 000
Total prestado: $9 000			
Fondo de capital total: $10 000			

(Tabla 5.5)
Después de estos préstamos, el capital restante del banco es de sólo $1 000, de modo que se encuentra en un "estado inseguro".

Este es un estado inseguro porque con sólo $1 000 que le quedan, el banco no puede satisfacer la solicitud máxima de ningún cliente, y si el banco presta los $1 000 a alguien, entonces se bloquearía (no puede hacer un préstamo). Un estado inseguro no necesariamente conduce a un bloqueo, pero indica que el banco es un excelente candidato para bloquearse. Después de todo, a ningún cliente se le requiere que solicite el máximo, pero el banco no conoce la cantidad exacta que será solicitada al final, y en tanto el capital del banco sea menos que la máxima cantidad disponible para préstamos individuales, no puede garantizar que será capaz de satisfacer todas las solicitudes de préstamo.

Si sustituimos trabajos por clientes y dispositivos dedicados por dólares, podemos aplicar los mismos principios bancarios a un sistema operativo. En este caso, el sistema tiene 10 dispositivos.

La tabla 5.6 muestra el sistema en un estado seguro y la tabla 5.7 representa el mismo sistema en un estado inseguro. Como antes, un estado seguro es uno en que por lo menos puede terminarse un trabajo porque hay suficientes recursos disponibles para satisfacer sus necesidades máximas. Luego, usando los recursos liberados por el trabajo terminado, es posible satisfacer las necesidades de otro trabajo y es posible terminar ese trabajo, y así hasta que se terminan todos los trabajos.

Trabajo no.	Dispositivos asignados	Máximo requerido	Necesidades restantes
1	0	4	4
2	2	5	3
3	4	8	4
Número total de dispositivos asignados: 6			
Número total de dispositivos en el sistema: 10			

(Tabla 5.6)
Asignación de recursos después de las asignaciones iniciales. Estado seguro: se han asignado seis dispositivos y cuatro unidades siguen disponibles.

(Tabla 5.7)

Asignación de recursos después de asignaciones posteriores. Estado inseguro: sólo una unidad está disponible, pero cada trabajo requiere por lo menos dos para completar su ejecución.

Trabajo no.	Dispositivos asignados	Máximo requerido	Necesidades restantes
1	2	4	2
2	3	5	2
3	4	8	4
Número total de dispositivos asignados: 9			
Número total de dispositivos en el sistema: 10			

El sistema operativo debe estar seguro de nunca satisfacer una demanda que lo mueva de un estado seguro a uno inseguro. En consecuencia, a medida que se cumplen las solicitudes de un usuario, el sistema operativo debe identificar el trabajo con el menor número de recursos restantes y asegurarse que el número de recursos restantes siempre sea igual, o mayor que, al número necesario para que este trabajo se ejecute hasta su terminación. El sistema operativo debe bloquear todas las solicitudes que pueden colocar el estado seguro en peligro hasta que puedan acomodarse con seguridad.

Si el sistema siempre se mantiene en un estado seguro, todas las solicitudes terminan por cumplirse, evitando así un bloqueo mutuo.

Si esta solución elegante se expande para funcionar con varias clases de recursos, el sistema establece una "tabla de administración de recursos" para cada tipo de recurso y sigue la pista de cada tabla a fin de mantener al sistema en un estado seguro.

Aunque el algoritmo del banquero se ha usado para evitar bloqueos mutuos en sistemas con pocos recursos, no siempre es práctico para la mayor parte de los sistemas por las siguientes razones:

- A medida que los trabajos entran al sistema, deben pronosticar el número máximo de recursos necesarios. Como ya se mencionó, esto no es práctico con sistemas interactivos.
- El número de recursos totales para cada clase debe permanecer constante. Si un dispositivo se descompone y de repente queda indisponible, el algoritmo no funciona (el sistema ya puede estar en un estado inseguro).
- El número de trabajos debe permanecer fijo, algo imposible en sistemas interactivos donde el número de trabajos activos cambia constantemente.
- El procesamiento extra en que se incurre por la ejecución del algoritmo de evasión pueden ser bastante altos cuando hay muchos trabajos activos y muchos dispositivos porque debe ser invocado por cada una de las solicitudes.
- Los recursos no se utilizan bien porque el algoritmo supone el peor caso y, como resultado, mantiene indisponibles recursos vitales a fin de protegerse contra estados inseguros.
- Como resultado de la utilización deficiente, la planificación sufre y los trabajos se mantienen en espera de asignación de recursos. Un flujo estable de trabajos solicitando nuevos recursos puede ocasionar el aplazamiento indefinido de un trabajo más complicado que requiere muchos recursos.

Detección

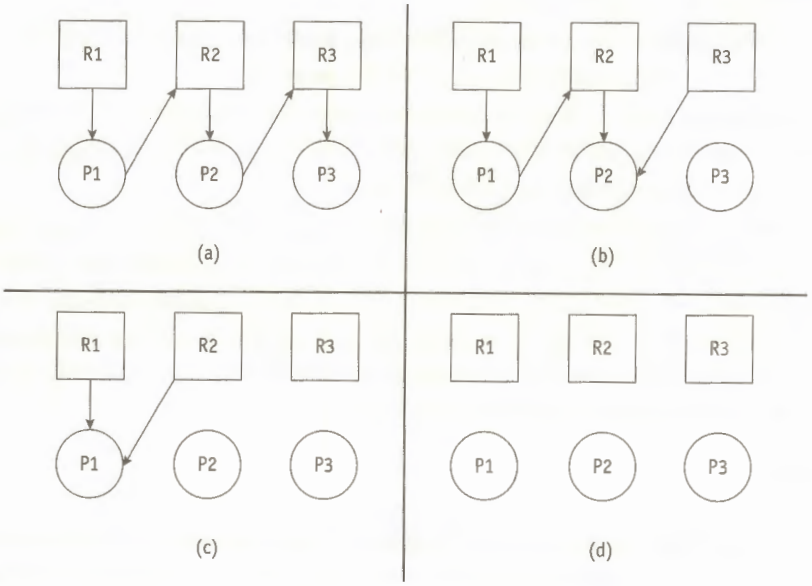
Las gráficas dirigidas presentadas antes en este capítulo mostraron cómo la existencia de una espera circular indicaba un bloqueo mutuo, por lo que resulta razonable concluir que los

bloqueos mutuos de recursos pueden detectarse al construir gráficas dirigidas de recursos y buscar ciclos. A diferencia del algoritmo de evasión, que debe ejecutarse cada que hay una solicitud, el algoritmo usado para detectar circularidad puede ejecutarse siempre que sea conveniente; cada hora, una vez al día, sólo cuando el operador observa que se ha deteriorado la salida, o cuando un usuario enojado se queja.

El algoritmo de detección puede explicarse usando gráficas de recursos dirigidas y "reduciéndolas". Se empieza con un sistema en uso, como se muestra en la figura 5.12*a*. Los pasos para reducir la gráfica son los siguientes:

1. Encontrar un proceso que esté usando un recurso en ese momento y no esté esperando por ninguno. Este proceso puede removerse de la gráfica (al desconectar el vínculo que va del recurso al proceso, como P3 en la figura 5.12*b*) y el recurso puede devolverse a la "lista disponible". Esto es posible porque el proceso terminará finalmente y regresará el recurso.
2. Encontrar un proceso que esté esperando sólo por clases de recursos que no hayan sido asignadas por completo (como P2 en la figura 5.12*c*). Este proceso no contribuye a ningún bloqueo mutuo porque termina por obtener el recurso que está esperando, termina su trabajo y regresa el recurso a la "lista disponible", como se muestra en la figura 5.12*c*.
3. Regresar al paso 1 y continuar con los pasos 1 y 2 hasta que se hayan eliminado todas las líneas que conectan los recursos con los procesos, llegando finalmente a la etapa que se muestra en la figura 5.12*d*.

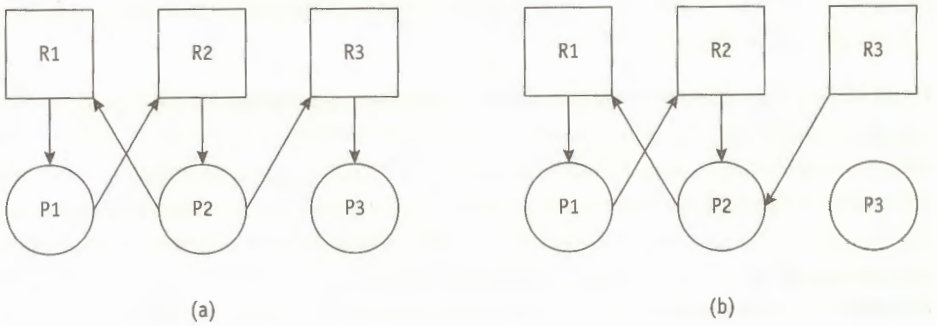
Si quedan algunas líneas, esto indica que la solicitud del proceso en cuestión no puede cumplirse y que existe un bloqueo mutuo. La figura 5.12 ilustra un sistema en que hay tres procesos: P1, P2 y P3, y tres recursos: R1, R2 y R3 que no están bloqueados mutuamente.



(Figura 5.12)

Este sistema está libre de bloqueos mutuos porque la gráfica puede reducirse completamente, como se muestra en (d).

La figura 5.12 muestra las etapas de la reducción de una gráfica desde *a*), el estado original. En *b*), el vínculo entre P3 y R3 puede eliminarse porque P3 no está esperando que termine ninguno de los otros recursos, de modo que R3 se libera y asigna a P2 (paso 1). En *c*) es posible eliminar los vínculos entre P2 y R3 y entre P2 y R2 porque P2 tiene todos sus recursos asignados y puede continuar su ejecución hasta terminar, y luego R2 puede asignarse a P1. Finalmente, en *d*), los vínculos entre P1 y R2 y entre P1 y R1 pueden eliminarse debido a que P1 tiene todos sus recursos solicitados y puede terminar exitosamente. En consecuencia, la gráfica se ha resuelto completamente. Sin embargo, la figura 5.13 muestra una situación muy semejante que está bloqueada debido a una diferencia fundamental: P2 está vinculado con R1.



(Figura 5.13)
Incluso después de que esta gráfica (a) se reduce tanto como es posible (por la remoción de la solicitud de P3), ésta permanece en un bloqueo mutuo(b).

El sistema bloqueado mutuamente en la figura 5.13 no puede reducirse. En *a*), el vínculo entre P3 y R3 puede eliminarse porque R3 no está esperando ningún otro recurso, de modo que R3 se libera y asigna a P2. Pero en *b*), P2 sólo tiene dos de los tres recursos que necesita para terminar y está esperando a R1. Pero R1 no puede ser liberado por P1 porque P1 está esperando por R2, que es retenido por P2; además, P1 no puede terminar porque está esperando que P2 termine (y libere a R2), y P2 no puede terminar porque está esperando a R1. Esta es una espera circular.

Recuperación

Una vez que se ha detectado un bloqueo mutuo, es necesario desentrañarlo y el sistema devuelto a la normalidad lo más rápido posible. Hay varios algoritmos de **recuperación**, aunque todos tienen un rasgo en común: todos requieren por lo menos una **víctima**, un trabajo prescindible que, al ser eliminado del bloqueo mutuo, libera el sistema. Desafortunadamente para la víctima, la eliminación suele requerir que el trabajo se reinicie desde el principio o desde un punto medio conveniente.

El primer y más simple método de recuperación, así como el más drástico, consiste en terminar todos los trabajos activos en el sistema y reiniciarlos desde el principio.

El segundo método consiste en terminar sólo los trabajos implicados en el bloqueo mutuo y pedir a sus usuarios que los vuelvan a presentar.

El tercer método consiste en identificar cuáles trabajos están implicados en el bloqueo mutuo y terminarlos uno por uno, comprobando si el bloqueo mutuo se ha eliminado después de cada eliminación, hasta que se haya resuelto el bloqueo mutuo. Una vez que se ha liberado el sistema, se permite que los trabajos restantes completen su procesamiento y más tarde los trabajos detenidos vuelven a iniciarse desde el principio.

El cuarto método sólo puede ponerse en efecto si el trabajo mantiene un registro, una instantánea, de su avance de modo que sea posible interrumpirlo y luego continuar sin iniciar de nuevo desde el principio de su ejecución. La instantánea es como el rellano en el ejemplo de la escalera: en lugar de obligar a las personas bloqueadas en la escalera a que regresen a la parte inferior de la escalera, es necesario retirarlos sólo hasta el descansillo más próximo o que esperen hasta que han pasado los demás. Después es posible reanudar la circulación por la escalera. En general, este método es el favorito para trabajos a largo plazo para ayudarlos a tener una pronta recuperación.

Hasta ahora se han dado soluciones que involucran trabajos atrapados en un bloqueo mutuo. Los dos métodos siguientes se concentran en trabajos sin bloqueo mutuo y en los recursos que mantienen Uno de ellos, el quinto metodo de la lista, selecciona un trabajo sin bloqueo mutuo, expropiando los recursos que mantiene, y asignandolos a un proceso con bloqueo mutuo de manera que éste pueda reanudar su ejecución, rompiéndose el bloqueo mutuo. El sexto método, impide la entrada de nuevos trabajos al sistema, lo cual permite que los trabajos no bloqueados terminen su ejecución, de modo que liberan sus recursos. Por último, con menos trabajos en el sistema, la competencia por recursos está restringida, de modo que los procesos bloqueados obtienen los recursos que necesitan para completar su ejecución. Este método es el único presentado aquí que no depende de una víctima, y no se garantiza que trabaje a menos que el número de recursos disponibles exceda los necesarios para que por lo menos tenga uno de los trabajos bloqueados para su ejecución (esto es posible con recursos múltiples).

Es necesario considerar varios factores para escoger a la víctima que tenga el efecto menos negativo sobre el sistema. Los más comunes son:

- La prioridad del trabajo en consideración: los trabajos con alta prioridad suelen ser intocables.
- Tiempo de CPU usado por el trabajo: los trabajos próximos a su completamiento suelen dejarse solos.
- El número de otros trabajos que pueden resultar afectados si este trabajo se escoge como la víctima.

Además, los trabajos que funcionan con bases de datos también merecen un tratamiento especial porque una base de datos que está actualizada sólo en parte es sólo parcialmente correcta. En consecuencia, los trabajos que están modificando datos no deben escogerse para su terminación porque podría ponerse en peligro la consistencia y validez de los datos. Afortunadamente, los diseñadores de muchos sistemas de bases de datos han incluido sofisticados mecanismos de recuperación, de modo que se minimice el daño a la base de datos si una transacción se interrumpe o termina antes de completarse.

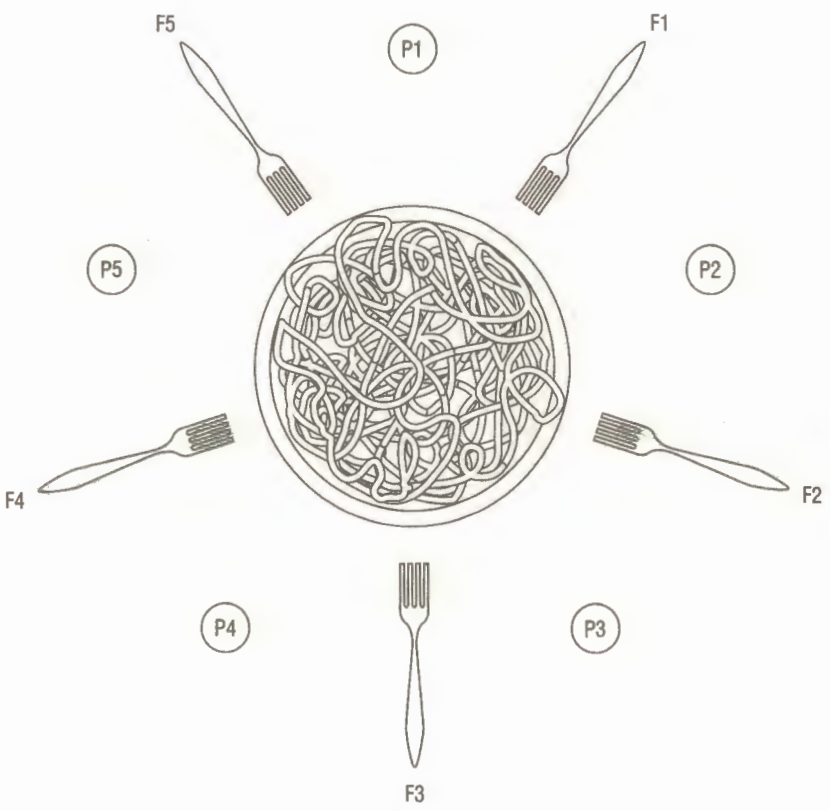
Mientras el bloqueo mutuo afecta el rendimiento de todo el sistema, la inanición afecta trabajos o procesos individuales. Para encontrar los trabajos privados, el sistema supervisa los tiempos de espera para bloques de control de procesos en las colas en ESPERA.

Inanición

Hasta el momento nos hemos centrado en bloqueos mutuos, el resultado de la asignación liberal de recursos. En el extremo opuesto de la inanición, el resultado de la asignación conservadora de recursos donde a un solo trabajo se le impide su ejecución porque se mantiene en espera de recursos que nunca se vuelven disponibles. Para ilustrar esto, Dijkstra introdujo el caso de "los filósofos cenando" en 1968.

Cinco filósofos están sentados alrededor de una mesa redonda, cada uno en profunda reflexión, y en el centro de la mesa hay un tazón con espagueti accesible para todos. Sobre la mesa hay tenedores: uno entre cada par de filósofos, como se ilustra en la figura 5.14. Las reglas locales dictan que cada filósofo debe usar dos tenedores: los que están a cada lado del plato, para comer el espagueti, pero sólo hay cinco tenedores: no los diez requeridos para que todos los filósofos coman al mismo tiempo, lo cual es desafortunado para el filósofo 2.

Cuando se sientan a cenar, el filósofo 1 (P1) es el primero en tomar los dos tenedores (F1 y F5) a cada lado del plato y empieza a comer. Inspirado por su colega, el filósofo 3 (P3) hace

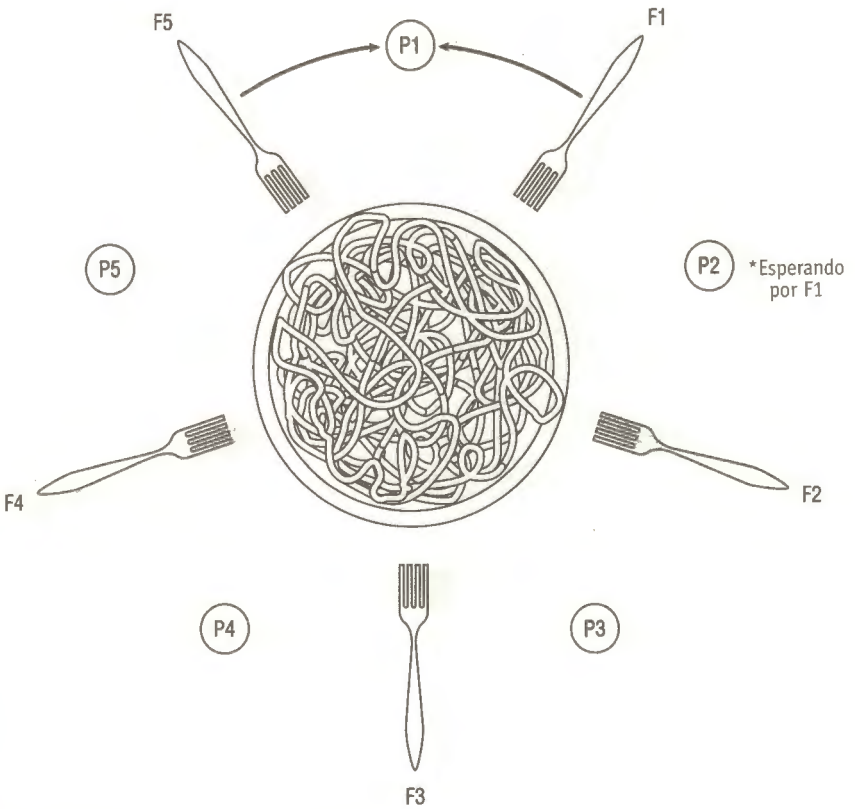


(Figura 5.14)
Mesa de los filósofos cenando antes que comience la cena.

lo mismo, usando F2 y F3. Ahora, el filósofo 2 (P2) decide empezar a cenar pero no puede hacerlo porque no hay tenedores disponibles: F1 ha sido asignado a P1 y F2 ha sido asignado a P3, y el único tenedor que queda sólo pueden usarlo P4 o P5. Así (P2), debe esperar.

Pronto, P3 termina de cenar, coloca sus dos tenedores sobre la mesa y reanuda su reflexión. El tenedor que está a su lado (F2), que ahora está libre, ¿debe asignarse al filósofo hambriento (P2)? Aunque es tentador, hacerlo sentaría un mal precedente porque si a los filósofos se les permite atar recursos con la única esperanza de que el otro recurso requerido esté disponible, la cena podría llegar fácilmente a un estado inseguro; sólo sería cuestión de tiempo antes que cada filósofo cogiera un simple tenedor y nadie podría cenar. Así, los recursos se asignan a los filósofos sólo cuando ambos tenedores estén disponibles al mismo tiempo. El estado del "sistema" se ilustra en la figura 5.15.

P4 y P5 están pensando serenamente y P1 sigue comiendo cuando P3 (quien debe estar lleno) decide comer algo más y como los recursos son libres, puede tomar F2 y F3 de nuevo. Pronto a partir de ese momento, P1 termina y libera F1 y F5, pero P2 aún no puede comer porque F2 no ha sido asignado. Este escenario puede continuar para siempre, y en tanto P1



(Figura 5.15)

Cada filósofo debe tener ambos tenedores para empezar a cenar; el de la derecha y el de la izquierda. A menos que los recursos, los tenedores, estén asignados de manera justa, algunos filósofos sufrirán inanición.

y P3 alternan el uso de los recursos disponibles, P2 debe esperar. P1 y P3 pueden cenar en cualquier momento que decidan mientras P2 desfallece de hambre; a sólo unos centímetros de la comida.

En un entorno informático, los recursos son como tenedores y los procesos en competencia son como los filósofos cenando. Si el administrador de recursos no atiende a procesos y trabajos privados de recursos, y planifica su completamiento final, pueden permanecer en el sistema para siempre esperando la combinación idónea de recursos.

Para tratar este problema, es posible implementar un algoritmo diseñado para detectar trabajos privados de recursos que sigue la pista de cuánto tiempo ha esperado recursos un trabajo (esto es lo mismo que el envejecimiento, descrito en el capítulo 4). Una vez que se ha detectado inanición, el sistema puede bloquear nuevos trabajos hasta que se hayan satisfecho. Este algoritmo debe supervisarse estrechamente: si se aplica muy a menudo, entonces nuevos trabajos serán bloqueados con demasiada frecuencia y entonces disminuye el resultado. Si no se aplica a menudo, entonces suficientes trabajos en inanición permanecen en el sistema durante un periodo inaceptable.

Conclusión

Todo sistema operativo debe asignar dinámicamente un número limitado de recursos, a la vez que evitar los dos extremos de bloqueo mutuo e inanición.

En este capítulo analizamos varios métodos para tratar con los bloqueos mutuos: prevención, evasión, detección y recuperación. Los bloqueos mutuos pueden prevenirse al no permitir que las cuatro condiciones de un bloqueo ocurran en un sistema al mismo tiempo. Al eliminar por lo menos una de las cuatro condiciones (exclusión mutua, retención de recursos, ausencia de expropiación y espera circular), el sistema puede permanecer libre de bloqueos mutuos. Como hemos visto, la desventaja de una política preventiva es que cada una de estas condiciones es vital para diferentes partes del sistema por lo menos parte del tiempo, de modo que los algoritmos de prevención son complicados y rutinariamente su ejecución implica procesamiento extra elevado.

Los bloqueos mutuos pueden evitarse al identificar claramente estados seguros y estados inseguros, y requiriendo al sistema que mantenga suficientes recursos en reserva para garantizar que todos los trabajos activos en el sistema puedan ejecutarse hasta su terminación. La desventaja de una política de evasión es que los recursos del sistema no se asignan a todo su potencial.

Si un sistema no atiende prevención o evasión, entonces debe prepararse para detectar y recuperarse de los bloqueos que puedan ocurrir. Desafortunadamente, esta opción suele depender de la selección de por lo menos una "víctima": un trabajo que debe terminarse antes de completar su ejecución y reiniciarse desde el principio.

En el siguiente capítulo analizaremos problemas relacionados con la sincronización de procesos en un entorno multiprocesamiento.

Términos clave

Bloquear: Técnica usada para garantizar la integridad de los datos en una base de datos a través de la cual el usuario bloquea a todos los otros usuarios mientras trabaja con la base de datos.

Bloqueo mutuo: Problema que ocurre cuando los recursos necesarios por algunos trabajos para terminar su ejecución están retenidos por otros trabajos que, a su vez, esperan que otros recursos estén disponibles. También se denomina *abrazo de la muerte*.

Bloqueo activo: Sistema bloqueado donde dos (o más) procesos bloquean continuamente el avance del orden sin hacer ningún avance en sí. Es semejante a un bloqueo mutuo excepto que ninguno de los procesos está bloqueado o en espera; ambos se encuentran en un estado de cambio continuo.

Competencia: Problema de sincronización entre dos procesos que compiten por el mismo recurso

Detección: Proceso de examinar un estado de un sistema operativo para determinar si existe un bloqueo.

Espera circular: Una de cuatro condiciones para que ocurran bloqueos, a través de la cual cada proceso implicado está esperando por un recurso retenido por otro proceso; cada proceso está bloqueado y no puede continuar, resultando en un bloqueo mutuo.

Estado inseguro: Situación en que el sistema tiene muy pocos recursos disponibles para garantizar el completamiento de por lo menos un trabajo en ejecución en el sistema. Puede conducir a bloqueo mutuo.

Estado seguro: Situación en que el sistema cuenta con suficientes recursos para garantizar la terminación de por lo menos un trabajo en ejecución en el sistema.

Evasión: Estrategia dinámica para evitar bloqueos mutuos que intenta asegurar que los recursos nunca se asignen de manera que pongan un sistema en un estado inseguro.

Exclusión mutua: Una de cuatro condiciones para bloqueo mutuo donde sólo a un proceso se le permite tener acceso a un recurso.

Gráfica dirigida: Modelo gráfico que representa varios estados de asignación de recursos.

Inanición: Resultado de la asignación conservadora de recursos donde se previene la ejecución de un solo trabajo porque se mantiene esperando recursos que nunca están disponibles.

Prevención: Estrategia de diseño para un sistema operativo donde los recursos son controlados de modo que algunas de las condiciones necesarias para el bloqueo no se cumplan.

Recuperación: Pasos que deben emprenderse, cuando se detecta un bloqueo mutuo, al romper el círculo de procesos en espera.

Retención de recursos: Una de cuatro condiciones para bloqueo mutuo donde cada proceso rehúsa liberar los recursos que posee hasta que se completa su ejecución, aun cuando no los esté usando porque está esperando otros recursos.

Sin expropiación: Una de cuatro condiciones para bloqueo mutuo donde a un proceso se le permite retener los recursos mientras está esperando otros recursos para completar su ejecución.

Sincronización de procesos: 1) La necesidad de algoritmos para resolver conflictos entre procesadores en un entorno multiprocesamiento; o 2) la necesidad de asegurar que los eventos ocurran en el orden idóneo si han sido efectuados por varios procesos.

Spooling: Técnica desarrollada para acelerar la E/S al recolectar en un archivo de disco ya sea entrada recibida desde dispositivos de entrada lentos o salida dirigida a dispositivos de salida lentos, como impresoras.

Víctima: Trabajo prescindible que se escoge en un sistema bloqueado a fin de proveer más recursos a los trabajos en espera y resolver el bloqueo mutuo.

Búsquedas de interés

- Detección de un falso bloqueo mutuo
- Inanición y detección de un bloqueo activo
- Detección de un bloqueo mutuo distribuido
- Algoritmos para la solución de bloqueos mutuos
- Sistema operativo congelado

Ejercicios

Temas de investigación

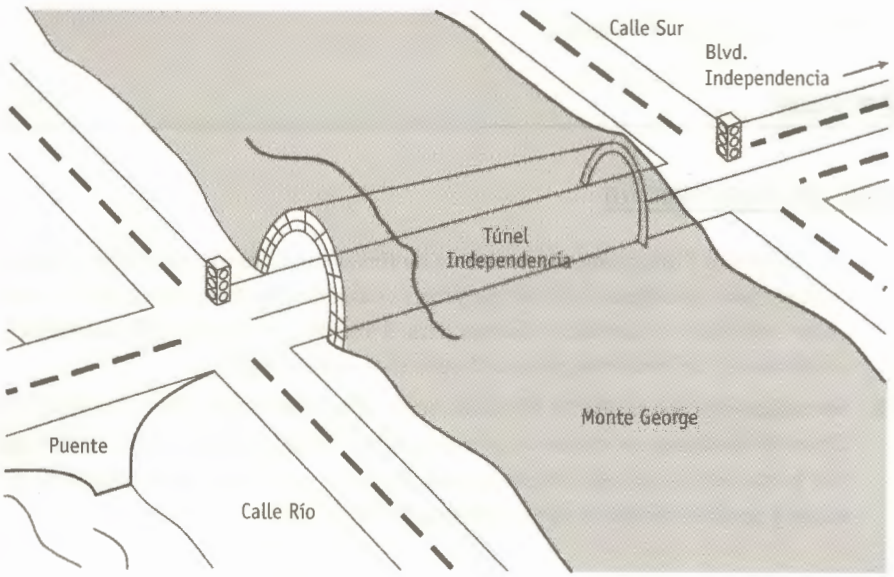
A. En el capítulo 3 analizamos el problema de *thrashing*. Consulte literatura actual en internet para investigar el rol del bloqueo y cualquier *thrashing* resultante. Analice cómo empezaría a cuantificar el costo para el sistema (en términos de resultados y rendimiento) del *thrashing* provocado por bloqueos mutuos.

B. Investigue el problema de los bloqueos vivos en un entorno de redes. Describa cómo difiere de los bloqueos mutuos y proporcione un ejemplo del problema. Identifique por lo menos dos métodos diferentes que el sistema operativo podría usar para detectar y resolver bloqueos vivos. Cite sus fuentes.

Ejercicios

1. Proporcione un ejemplo de sistema computarizado (diferente de los descritos en este capítulo) de una competencia que debe producir un resultado diferente dependiendo del orden de procesamiento.
2. Proporcione al menos dos ejemplos de la "vida real" (no relacionados con entornos informáticos) de cada uno de estos conceptos: bloqueos mutuos, inanición y competencia. Describa cómo pueden resolverse los bloqueos mutuos.

3. Seleccione un ejemplo de bloqueo mutuo del ejercicio 2 e identifique qué elementos de éste representan las cuatro condiciones necesarias para todos los bloqueos mutuos.
4. Describa el destino de la "víctima" en la solución de un bloqueo mutuo. Describa las acciones requeridas para completar las tareas de las víctimas.
5. Utilizando el ejemplo de la escalera estrecha del inicio de este capítulo, genere una lista de acciones o tareas que permitan a la gente usar la escalera sin causar bloqueos mutuos o inanición.
6. La figura 5.16 muestra un túnel que atraviesa una montaña y dos calles paralelas entre sí: una en cada entrada/salida del túnel. En el final del túnel hay semáforos para controlar el tráfico transversal en cada intersección. Con base en la figura, conteste las siguientes preguntas:
   a. ¿Puede ocurrir bloqueo mutuo? ¿Cómo y en qué circunstancias?
   b. ¿Cómo puede ocurrir un bloqueo mutuo y bajo qué circunstancias?
   c. Proporcione una solución para prevenir un bloqueo mutuo e inanición.



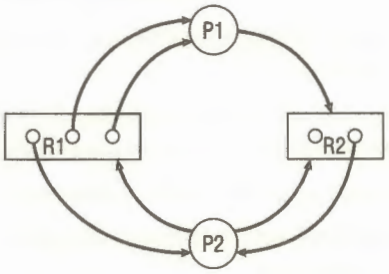
(Figura 5.16)
Diagrama de flujo del tráfico para el ejercicio 6.

7. Considere la gráfica dirigida de los recursos que se muestra en la figura 5.17 y conteste las siguientes preguntas:
   a. ¿Hay algún proceso bloqueado?
   b. El sistema, ¿está bloqueado?
   c. ¿Cuál es la gráfica resultante después de la reducción de P1?
   d. ¿Cuál es la gráfica resultante después de la reducción de P2?

e. Si P1 y P2 han solicitado a R2, responda estas preguntas:

1. ¿Cuál es el estado del sistema si primero se concede la solicitud de P2, antes que la de P1?
2. ¿Cuál es el estado del sistema si primero se concede la solicitud de P1, antes que la de P2?

(Figura 5.17)
Gráfica dirigida de los recursos para el ejercicio 7.



8. Considere la gráfica dirigida de los recursos que se muestra en la figura 5.18 y conteste las siguientes preguntas:

a. Identifique todos los procesos bloqueados mutuamente.

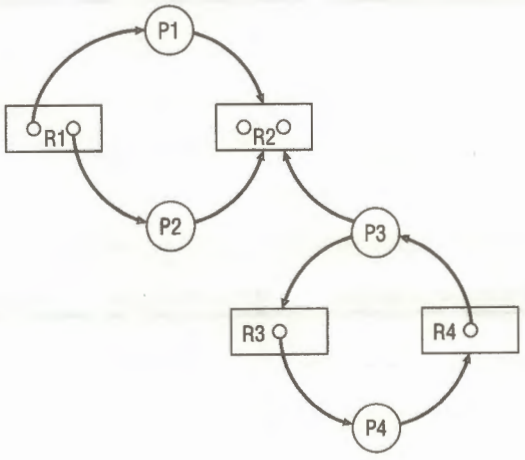
b. ¿Es posible reducir parcial o totalmente la gráfica dirigida?

c. ¿Es posible resolver el bloqueo sin escoger una víctima?

d. ¿Cuáles solicitudes de los tres procesos de recursos de R2 satisfaría usted para minimizar el número de procesos implicados en el bloqueo mutuo?

e. En caso contrario ¿Cuáles solicitudes satisfaría el lector para maximizar el número de procesos implicados en el bloqueo mutuo?

(Figura 5.18)
Gráfica dirigida de los recursos para el ejercicio 8.



9. Considere un sistema de archivo con 13 dispositivos dedicados. Para completar su ejecución todos los trabajos funcionando en este sistema requieren un máximo de 5 controladores, pero cada uno se ejecuta durante largos periodos con sólo 4 controladores y solicita el quinto sólo al final de la ejecución. El flujo de trabajo es interminable.
   a. Suponga que su sistema operativo ejerce una política de asignación de dispositivos muy conservadora, tal que ningún trabajo empieza a menos que todos los controladores necesarios le han sido asignados durante el tiempo de su ejecución:
      1. ¿Cuál es el número máximo de trabajos que pueden estar activos a la vez? Explique su respuesta.
      2. ¿Cuáles son los números mínimo y máximo de unidades de cinta que pueden permanecer ociosas como resultado de esa política? Explique su respuesta.
   b. Suponga que su sistema operativo aplica el algoritmo del banquero:
      1. ¿Cuál es el número máximo de trabajos que pueden estar en ejecución a la vez? Explique su respuesta.
      2. ¿Cuáles son los números mínimo y máximo de unidades de cinta que pueden permanecer ociosas como resultado de esa política? Explique su respuesta.

10-12. Para los tres sistemas descritos a continuación, dado que todos los dispositivos son del mismo tipo, y usando las definiciones presentadas en el análisis del algoritmo del banquero, conteste estas preguntas:
   a. Determine las "necesidades restantes" para cada trabajo en el sistema.
   b. Determine si cada sistema es seguro o inseguro.
   c. Si el sistema se encuentra en un estado seguro, enumere la secuencia de solicitudes y liberaciones que hacen posible que todos los procesos completen su ejecución.
   d. Si el sistema se encuentra en un estado inseguro, ¿cómo es posible que ocurra bloqueo mutuo?

10. El sistema A tiene 12 dispositivos; sólo 1 está disponible.

Trabajo no.	Dispositivos asignados	Máximo requerido	Necesidades restantes
1	5	6	
2	4	7	
3	2	6	
4	0	2	

11. El sistema B tiene 14 dispositivos; sólo 2 están disponibles.

Trabajo no.	Dispositivos asignados	Máximo requerido	Necesidades restantes
1	5	8	
2	3	9	
3	4	8	

12. El sistema C tiene 12 dispositivos; sólo 2 están disponibles.

Trabajo no.	Dispositivos asignados	Máximo requerido	Necesidades restantes
1	5	8	
2	4	6	
3	1	4	

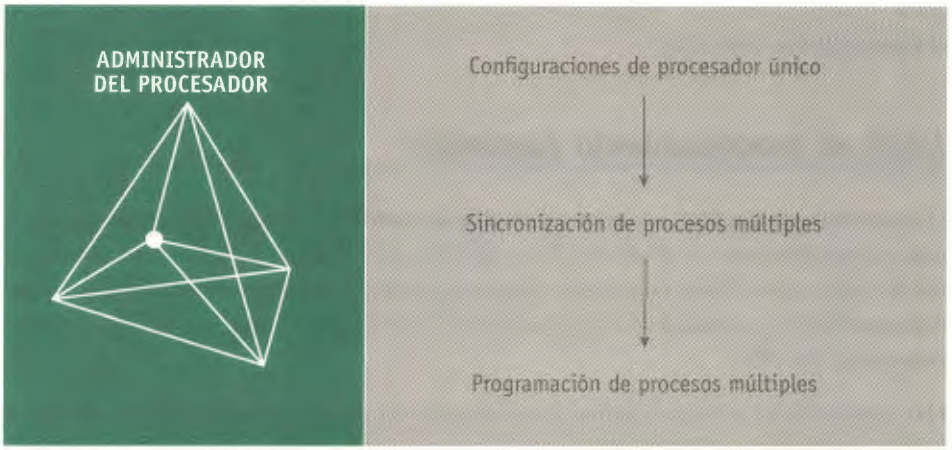
Ejercicios avanzados

13. Suponga que usted es un diseñador de sistemas operativos y el administrador le ha solicitado su ayuda para resolver el problema de bloqueo mutuo recurrente en el sistema de *spooling* de su instalación. ¿Qué características debe incorporar en el sistema operativo de modo que sea posible resolver los bloqueos mutuos en el sistema *spooling* sin pérdida del trabajo realizado (el procesamiento del sistema) por los procesos implicados?
14. Como lo analizamos en este capítulo, un sistema en estado inseguro no necesariamente está bloqueado. Explique por qué esto es verdadero. Proporcione un ejemplo de un sistema en estado inseguro y muestre cómo es posible completar todos los procesos sin que ocurra bloqueo mutuo.
15. Explique cómo diseñaría e implementaría usted un mecanismo para permitir que el sistema operativo detecte cuál, en caso de haber alguno, de los procesos padece inanición.
16. Dados los cuatro tipos de recursos primarios: CPU, memoria, dispositivos de almacenamiento y archivos, para cada uno escoja la técnica más adecuada descrita en este capítulo para combatir los bloqueos y explique brevemente por qué ésta es su elección.
17. Establezca las limitaciones impuestas a los programas (y a los sistemas) que deben seguir un orden jerárquico de recursos; por ejemplo, discos, impresoras y archivos.
18. Considere un sistema bancario con 10 cuentas. La transferencia de fondos entre dos de estas cuentas puede ocurrir si se siguen los pasos siguientes:

```
bloquear A(i);          bloquear A(j);
actualizar A(i);        actualizar A(j);
desbloquear A(i);       desbloquear A(j);
```

    a. Este sistema ¿puede bloquearse? En caso afirmativo, muestre cómo. En caso contrario, explique por qué no.
    b. La política de numeración de solicitudes (presentada en este capítulo en el análisis sobre detección), ¿puede implementarse para prevenir bloqueos si el número de cuentas es dinámico? Explique por qué.

Capítulo 6 Procesos concurrentes



"La medida del poder es superar obstáculos."

Oliver Wendell Holmes, Jr. (*1841-1935*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- La diferencia fundamental entre procesos y procesadores, y su relación.
- La diferencia entre varias configuraciones de sistemas multiprocesamiento.
- Cómo detectar bloqueos mutuos y recuperarse de ellos.
- Los conceptos básicos de tecnología de procesador multinúcleo.
- La importancia de una región crítica en sincronización de procesos.
- La necesidad de cooperación de procesos cuando varios procesadores trabajan juntos.
- Las semejanzas y diferencias entre procesos e hilos.
- La forma en que cooperan los procesos cuando ejecutan un trabajo, proceso o hilo.
- La importancia de los lenguajes de programación concurrentes y sus aplicaciones.

En los capítulos 4 y 5 describimos sistemas multiprogramación que usan una sola CPU, un procesador, que se comparte entre varios trabajos o procesos. A esto se le denomina multiprocesamiento. En este capítulo abordamos otra situación común, los sistemas multiprocesamiento, los que tienen varios procesadores que trabajan juntos en una variedad de configuraciones.

Los sistemas de multiprocesamiento incluyen computadoras únicas con núcleos múltiples así como también sistemas de computadoras con sólo un procesador cada una, compartiendo el procesamiento entre ellas.

¿Qué es procesamiento paralelo?

El **procesamiento paralelo**, una forma de **multiprocesamiento**, es una situación en que dos o más procesadores operan al unísono. Esto significa que dos o más CPU ejecutan instrucciones de manera simultánea. En sistemas multiprocesamiento, el administrador del procesador debe coordinar la actividad de cada procesador, así como sincronizar la interacción cooperativa entre las CPU.

Hay dos beneficios primarios para el procesamiento en paralelo: incremento en la confiabilidad y procesamiento más rápido.

La confiabilidad surge de la disponibilidad de más de una CPU: si falla un procesador, entonces los otros pueden continuar operando y absorber la carga. Esto no es fácil de hacer; el sistema debe diseñarse cuidadosamente de modo que, primero, el procesador con falla pueda informar a los otros procesadores que asuman la situación y, segundo, el sistema operativo pueda reestructurar sus estrategias de asignación de recursos de modo que los procesadores restantes no se sobrecarguen.

La velocidad de procesamiento incrementada a menudo se logra porque algunas veces las instrucciones pueden procesarse en paralelo, dos o más a la vez, en una de varias formas. Algunos sistemas asignan una CPU para cada programa o trabajo. Otros asignan una CPU a cada conjunto de trabajo o a partes de éste. Y hay algunos que subdividen las instrucciones individuales de modo que cada subdivisión pueda procesarse de manera simultánea (esto se denomina programación concurrente).

La flexibilidad aumentada conlleva una mayor complejidad, no obstante, y persisten dos desafíos fundamentales: cómo conectar los procesadores en configuraciones y cómo orquestar su interacción, que también se aplica a múltiples procesos que interactúan. (Podría ser de ayuda si usted imagina que cada proceso se ejecuta en un procesador por separado.)

Las complejidades de la tarea del administrador del procesador cuando trata con múltiples procesadores pueden ilustrarse fácilmente con un ejemplo: usted llega atrasado a una cita por la tarde y está en peligro de perder la comida, de modo que se comunica con la línea de autoservicio de la tienda local de comida rápida. Cuando coloca su orden, el empleado confirma su solicitud, le indica el precio y le pide conducir hasta otra ventanilla donde puede pagar y recoger su pedido. Todo marcha bien y nuevamente ya está usted en su camino a

casa, conduciendo y radiante. Usted acaba de presenciar un sistema multiprocesamiento bien sincronizado. Aunque usted entró en contacto sólo con dos procesadores: el empleado que recibió la orden y el cajero, por lo menos había otros dos procesadores detrás de las escenas, quienes cooperaron para que funcionara el sistema: el cocinero y el empacador.

Un punto de venta de comida rápida es semejante al siguiente sistema de recuperación de información de seis pasos. Se describe en una forma distinta en la tabla 6.1.

a) El procesador 1 (el empleado que recibió la orden) acepta la solicitud, verifica errores y pasa la solicitud al procesador 2 (el empacador).

b) El procesador 2 (el empacador) busca en la base de datos la información requerida (la hamburguesa).

c) El procesador 3 (el cocinero) recupera la información de la base de datos (la carne para cocinar la hamburguesa) si se mantiene fuera de línea en almacenamiento secundario.

d) Una vez que se han reunido los datos (la hamburguesa ya está cocinada), se coloca donde el procesador 2 puede obtenerla (en el estante para las hamburguesas).

e) El procesador 2 (el empacador) pasa lo anterior al procesador 4 (el cajero).

f) El procesador 4 (el cajero) encamina la respuesta (la orden) de regreso al originador de la solicitud: usted.

(Tabla 6.1)

Los seis pasos de la parada para almorzar comida rápida

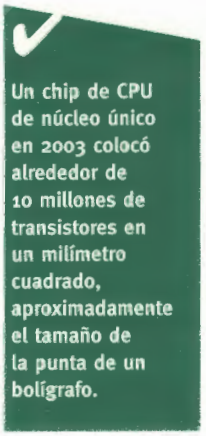
Emisor	Acción	Receptor
Procesador 1 (el empleado que recibió la orden)	Acepta la solicitud, verifica errores y pasa la solicitud hacia =>	Procesador 2 (el empacador)
Procesador 2 (el empacador)	Busca en la base de datos la información requerida (la hamburguesa)	
Procesador 3 (el cocinero)	Recupera los datos de la base de datos (la carne para cocinar la hamburguesa) si está fuera de línea en almacenamiento secundario	
Procesador 3 (el cocinero)	Una vez que se reúnen los datos (la hamburguesa está cocinada), es colocada donde el receptor => puede obtenerla (en el estante para las hamburguesas)	Procesador 2 (el empacador)
Procesador 2 (el empacador)	La pasa hacia =>	Procesador 4 (el cajero)
Procesador 4 (el cajero)	Encamina la respuesta (su orden) de vuelta al originador de la solicitud =>	Usted

La clave del éxito del sistema es la sincronización, porque muchas cosas pueden ir mal en un sistema multiprocesamiento. Por ejemplo, ¿qué ocurre si el sistema de comunicación deja de funcionar y no es posible hablar con el empleado que recibió la orden? ¿Qué ocurre si el cocinero produce hamburguesas a toda velocidad durante todo el día, incluso durante periodos de calma? ¿Qué ocurriría con las hamburguesas extra? ¿Qué ocurriría si el horno se descompone y ya no es posible cocinar más? ¿Qué haría el empacador si ya no hubiera más

hamburguesas? ¿Y si el cajero decide aceptar su dinero pero no entrega la hamburguesa? Resulta evidente que el sistema no puede funcionar apropiadamente a menos que cada procesador se comunique y coopere con todos y cada uno de los otros procesadores.

Evolución de los multiprocesadores

El multiprocesamiento puede tener lugar en varios niveles diferentes, cada uno requiere una frecuencia diferente de sincronización, como se muestra en la tabla 6.2. Observe que en el nivel de proceso, el multiprocesamiento es bastante benigno. Es como si cada proceso se ejecuta en su propia estación de trabajo con recursos del sistema compartido. Por otra parte, cuando el multiprocesamiento tiene lugar en el nivel de subproceso, se requiere un alto grado de sincronización para desensamblar cada proceso, realizar las instrucciones del subproceso y, a continuación, volver a montar el proceso correctamente. Esto puede requerir un trabajo adicional por parte del programador, como veremos más adelante en este capítulo.



Nivel de paralelismo	Asignaciones del proceso	Sincronización necesaria
Nivel de trabajo	Cada trabajo tiene su propio procesador y todos los procesos e hilos son ejecutados por el mismo procesador	No se requiere sincronización explícita
Nivel del proceso	Procesos no relacionados, sin importar el trabajo, son asignados a cualquier procesador disponible	Se requiere una cantidad moderada de sincronización para rastrear procesos
Nivel del hilo	Los hilos se asignan a procesadores disponibles	Se requiere un alto grado de sincronización, y a menudo se requieren instrucciones explícitas del programador

(Tabla 6.2)

Niveles de paralelismo y la sincronización necesaria entre procesadores.

Introducción a los procesadores de núcleo múltiple

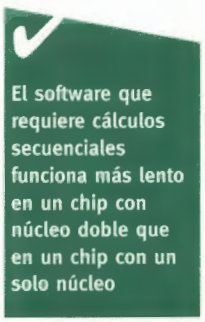
Los **procesadores de núcleo múltiple** tienen varios procesadores en un solo chip. Como los procesadores redujeron su tamaño (como pronostica la ley de Moore) y aumentó la velocidad de procesamiento, los diseñadores de CPU comenzaron a usar transistores del tamaño de nanómetros. Cada transistor conmuta entre dos posiciones: cero y uno, a medida que la computadora lleva a cabo su aritmética binaria a velocidades cada vez más altas. Sin embargo, a medida que los transistores alcanzaban dimensiones del orden de los nanómetros y el espacio entre ellos se volvía cada vez más pequeño, la física cuántica de los electrones ya estaba en camino.

En breve, este es el problema. Cuando los transistores se colocan extremadamente próximos unos de otros, los electrones tienen la habilidad de crear espontáneamente túneles, al azar, de un transistor a otro, causando que una cantidad pequeña, pero mensurable de corriente de fuga. Mientras más pequeños son los transistores, más importante es la fuga. (Cuando

un electrón hace este efecto de túnel, da la impresión de que desaparece espontáneamente de un transistor y aparece en otro transistor próximo. Es como si un viajero de *Star Trek* solicitara al electrón ser "enviado en un haz" al segundo transistor.)

Un segundo problema consistía en el calor generado por el chip. A medida que los procesadores se volvieron más rápidos, el calor también ascendía y comenzaba a ser cada vez más difícil de dispersar. Estas cuestiones del efecto túnel y de calentamiento amenazaron con limitar la capacidad de los diseñadores de chips para hacer aún más pequeños a los procesadores.

Una solución fue crear un chip único (una pieza de silicio) con dos "núcleos procesadores" en la misma cantidad de espacio. Con esta disposición, era posible llevar a cabo dos conjuntos de cálculos al mismo tiempo. Los dos núcleos en el chip generan menos calor que uno solo del mismo tamaño y se reduce el efecto túnel, sin embargo, cada uno de los dos núcleos es más lento que el chip con un solo núcleo. En consecuencia, para obtener rendimiento mejorado de un chip con núcleo dual, el software debe estructurarse a fin de aprovechar la doble capacidad de cálculo del nuevo diseño del chip. basados en el éxito con chips de núcleo doble, los diseñadores han creado procesadores de núcleo múltiple con predicciones, como la de este escrito, que ochenta o más núcleos serán colocados en un solo chip, como se muestra en el capítulo 1.



El software que requiere cálculos secuenciales funciona más lento en un chip con núcleo doble que en un chip con un solo núcleo

¿Esta innovación del hardware afecta al sistema operativo? Sí, porque debe administrar múltiples procesadores, múltiples RAM y el procesamiento de muchas tareas a la vez. Sin embargo, un chip con núcleo doble no siempre es más rápido que un chip con un solo núcleo. Depende de las tareas que se realicen y de si son multihilos o secuenciales.

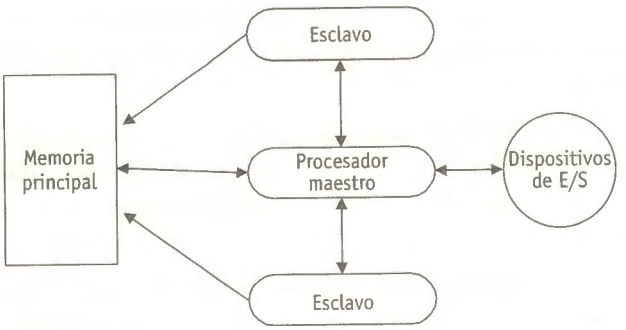
Configuraciones típicas de multiprocesamiento

Mucho depende de la forma en que los procesadores múltiples están configurados con el sistema. Tres configuraciones típicas son: maestro/esclavo, débilmente acoplada y simétrica.

Configuración maestro/esclavo

La configuración **maestro/esclavo** es un sistema de multiprocesamiento asimétrico. Considérelo como un sistema con procesador único con procesadores esclavos adicionales, cada uno de los cuales es administrado por el procesador maestro primario como se muestra en la figura 6.1.

El procesador maestro es responsable de administrar todo el sistema: todos los archivos, dispositivos, memoria y procesadores. En consecuencia, mantiene el estado de todos los procesos en el sistema, desempeña actividades de administración de almacenamiento, planifica el trabajo para otros procesadores y ejecuta todos los programas de control. Esta configuración se adapta bien para entornos informáticos donde el tiempo de procesamiento está dividido entre procesadores de base de datos y procesadores de comunicaciones; en estos casos el procesador de comunicaciones se ocupa de los usuarios interactivos y los trabajos rápidos, y el procesador de base de datos se ocupa de los trabajos largos usando modo por lotes.



(Figura 6.1)

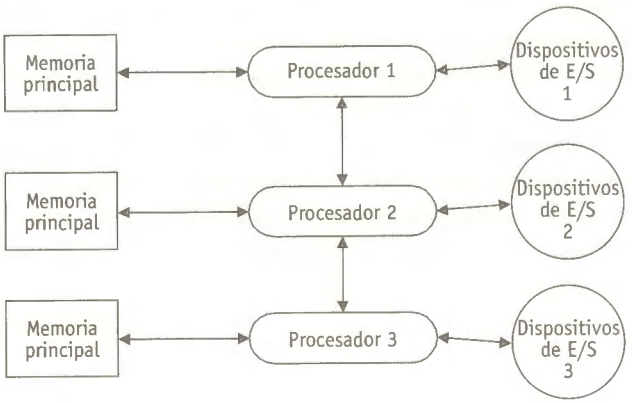
En una configuración de multiprocesamiento maestro/esclavo, los procesadores esclavos pueden acceder directamente a la memoria principal, pero deben enviar todas las solicitudes de E/S a través del procesador maestro.

La ventaja más importante de esta configuración es su simpleza. Sin embargo, presenta tres desventajas serias:

- Su confiabilidad no es mayor que la de un sistema con procesador único porque si falla el procesador maestro, falla todo el sistema.
- Puede llevar a un uso deficiente de los recursos porque si un procesador esclavo se libera mientras el procesador maestro está ocupado, el esclavo debe esperar hasta que el maestro sea libre y le pueda asignar más trabajo.
- Incrementa el número de interrupciones porque todos los procesadores esclavos deben interrumpir al procesador maestro cada que requieren la intervención del sistema operativo, como para solicitudes de E/S. Esto crea largas colas a nivel del procesador maestro cuando hay muchos procesadores y muchas interrupciones.

Configuración débilmente acoplada

La **configuración débilmente** acoplada presenta muchos sistemas de cómputo completos, cada uno con su propia memoria, dispositivos de E/S, CPU y sistema operativo, como se muestra en la figura 6.2. Esta configuración se denomina débilmente acoplada porque cada procesador controla sus propios recursos: sus propios archivos, accesos a la memoria y sus propios dispositivos de E/S, lo cual significa que cada procesador mantiene sus propios comandos y tablas de administración de E/S. La única diferencia entre un sistema multipro-



(Figura 6.2)

En una configuración débilmente acoplada, cada procesador tiene sus propios dispositivos dedicados.

cesamiento débilmente acoplado y una colección de sistemas multiprocesamiento independientes es que cada procesador puede comunicarse y cooperar con los otros.

Cuando un trabajo llega por primera vez, se asigna a un procesador. Una vez asignado, el trabajo permanece con el mismo procesador hasta que es terminado. En consecuencia, cada procesador debe tener tablas globales que indiquen a cuál procesador se ha asignado el trabajo.

Para mantener bien equilibrado el sistema y asegurar el mejor uso de los recursos, la planificación de trabajos está basada en varios requerimientos y políticas. Por ejemplo, nuevos trabajos pueden asignarse al procesador con la menor carga o la mejor combinación de dispositivos de salida disponibles.

Este sistema no es propenso a fallas catastróficas del sistema porque incluso cuando falla un solo procesador, los otros pueden continuar trabajando de manera independiente. Sin embargo, puede ser difícil detectar cuándo ha fallado un procesador.

Configuración simétrica

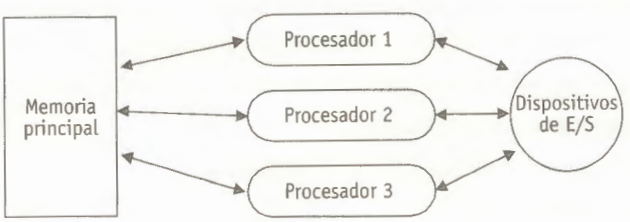
La **configuración simétrica** (también denominada fuertemente acoplada) presenta cuatro ventajas sobre la configuración débilmente acoplada:

- Es más confiable.
- Usa los recursos de manera efectiva.
- Puede equilibrar bien las cargas.
- Puede degradarse con elegancia en caso de fallo.

Esta configuración simétrica se implementa mejor si todos los procesadores son del mismo tipo.

Sin embargo, es la configuración más difícil de implementar porque los procesos deben estar bien sincronizados para evitar los problemas de competencia o bloqueos analizados en el capítulo 5.

En una configuración simétrica (como se muestra en la figura 6.3), el planificador del procesador está descentralizado. Una simple copia del sistema operativo y una tabla global que enumera cada proceso y su estado se almacenan en un área común de la memoria de modo que todos los procesadores tengan acceso a éstos. Cada procesador usa el mismo algoritmo de planificación para escoger el proceso que se ejecutará a continuación.



(Figura 6.3)
Configuración simétrica de multiprocesamiento con procesadores homogéneos. Los procesos deben sincronizarse cuidadosamente para evitar bloqueos e inanición.

Cada que se interrumpe un proceso, ya sea debido a una interrupción de E/S u otro tipo, su procesador actualiza la entrada correspondiente en la lista de procesos y encuentra otro proceso a ejecutar. Esto significa que los procesadores se mantienen bastante ocupados. Pero también significa que cualquier trabajo o tarea dados puede ser ejecutado por muchos procesadores diferentes durante su tiempo de ejecución. Y debido a que cada procesador tiene acceso a todos los dispositivos de E/S y puede referirse a cualquier unidad de almacenamiento, hay más conflictos a medida que varios procesadores intentan tener acceso al mismo recurso al mismo tiempo.

Esta situación presenta la necesidad evidente de algoritmos para resolver conflictos entre procesadores, lo cual se denomina **sincronización de procesos.**

Software de sincronización de procesos

El éxito de la sincronización de procesos depende de la capacidad del sistema operativo en hacer indisponible un recurso para otros procesos mientras uno de ellos lo está usando. Estos "recursos" pueden incluir impresoras y otros dispositivos de E/S, una ubicación en almacenamiento o un archivo de datos. En esencia, el recurso usado debe bloquearse de otros procesos hasta que es liberado. Sólo cuando es liberado se permite que otro proceso en espera lo utilice. Es aquí donde es crítica la sincronización. Un error puede dejar un trabajo en espera indefinida (inanición) o, si se trata de un recurso clave, ocasionar un bloqueo.

Es lo mismo que puede ocurrir en una tienda abarrotada de gente. Los clientes pueden tomar un turno para que sean atendidos. El turno indicado en la pared lo cambian los empleados para atender al siguiente cliente. Pero ¿qué ocurre cuando no hay sincronización entre atender a los clientes y cambiar el turno correspondiente? Caos. Éste es el caso de omitir a un cliente en espera.

Suponga que su turno es el 75. El empleado 1 está esperando al cliente 73 y el empleado 2 está esperando al cliente 74. El turno indicado en la pared dice: "atendiendo ahora al cliente #74" y usted ya está listo para ir con su orden. El empleado 2 termina con el cliente 74 y cambia el turno en la pared de modo que ahora éste indica "atendiendo ahora al cliente #75". Pero justo en ese momento el empleado recibe una llamada telefónica y abandona el edificio para no volver (una interrupción). Mientras tanto, el empleado 1 cambia el turno en la pared y procede a esperar al cliente #76", de modo que usted ha perdido su turno. Si usted reacciona de inmediato, puede corregir el error con elegancia, pero cuando esto ocurre en un sistema de cómputo, no es tan fácil de resolver.

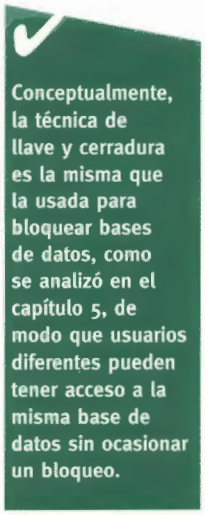
Considere el escenario en que el procesador 1 y el procesador 2 terminan al mismo tiempo los trabajos que están realizando. Para ejecutar el siguiente trabajo, cada procesador debe:

1. Consultar la lista de trabajos para ver cuál debe ejecutarse a continuación.
2. Recuperar el trabajo para ejecutarlo.
3. Incrementar la lista LISTO para el siguiente trabajo.
4. Ejecutar el trabajo.

Ambos se dirigen a la lista LISTO para escoger un trabajo. El procesador 1 observa que el trabajo 74 es el siguiente a ejecutar y se dirige a recuperarlo. Un momento después, el procesador 2 también escoge al trabajo 74 y va a recuperarlo. Poco después, el procesador 1, una vez que ha recuperado el trabajo 74, regresa a la lista LISTO y la incrementa, moviendo el trabajo 75 a la parte superior. Después regresa el procesador 2; también éste ha recuperado el trabajo 74 y está listo para procesarlo, de modo que incrementa la lista LISTO y ahora el trabajo 76 se mueve a la parte superior, volviéndose el siguiente trabajo en línea a procesar. El trabajo 75 se convierte en el cliente omitido y jamás será procesado, mientras el trabajo 74 es procesado dos veces: un estado inaceptable de cosas.

Hay varios otros sitios donde puede ocurrir este problema: en la memoria y en las tablas de asignación de páginas, en las tablas de E/S, en las bases de datos de aplicaciones, y en cualquier recurso compartido.

Resulta evidente que esta situación requiere sincronización. Hay disponibles varios mecanismos de sincronización para proporcionar cooperación y comunicación entre procesos. El elemento común en todos los esquemas de sincronización es permitir que un proceso termine su trabajo en una parte crítica del programa antes que otro proceso tenga acceso a éste. Lo anterior es aplicable tanto a multiprocesadores como a dos o más procesos en un sistema multiprocesamiento de procesador único (de tiempo compartido). Se denomina **región crítica** porque es una sección crítica y su ejecución debe manejarse como una unidad. Como hemos visto, los procesos dentro de una región crítica no pueden intercalarse sin amenazar la integridad de la operación.



Conceptualmente, la técnica de llave y cerradura es la misma que la usada para bloquear bases de datos, como se analizó en el capítulo 5, de modo que usuarios diferentes pueden tener acceso a la misma base de datos sin ocasionar un bloqueo.

Algunas veces, la sincronización se implementa como una disposición de llave y cerradura: antes que un proceso pueda trabajar en una región crítica, debe tener la llave. Y una vez que la tiene, todos los otros procesos están bloqueados hasta que termina, desbloquea la entrada a la región crítica y devuelve la llave a otro proceso para que empiece a trabajar. Esta secuencia consta de dos acciones: 1) primero, el proceso debe verificar si la llave está disponible y 2) en caso de estar disponible, el proceso debe tomarla y ponerla en la cerradura a fin de inhabilitarla para otros procesos. Para que este esquema funcione, ambas acciones deben realizarse en un solo ciclo de máquina; en caso contrario es concebible que mientras el primer proceso está listo para tomar la llave, otro podría encontrar disponible la llave y prepararse para tomarla, y cada uno podría bloquear al otro e impedir que prosiga más allá.

Se han desarrollado varios mecanismos de cerradura, incluyendo prueba e inicio; WAIT, SIGNAL y semáforos.

Prueba e inicio

Prueba e inicio es una instrucción de máquina única e indivisible conocida simplemente como TS y fue introducida por IBM para las computadoras de su sistema multiprocesamiento 360/370. En un solo ciclo de máquina, esta instrucción prueba si está disponible la llave y, en caso de estarlo, la establece como indisponible.

La llave real es un bit único en una ubicación del almacenamiento que contiene un cero (si está libre) o un uno (ocupada). Puede considerarse que TS es una función de subprograma

que tiene un parámetro (la ubicación del almacenamiento) y regresa un valor (el código de condición: ocupada/libre) con la excepción de que sólo requiere un ciclo de máquina.

En consecuencia, un proceso (proceso 1) prueba el código de condición usando la instrucción TS antes de entrar a una región crítica. Si en la región crítica no hay ningún otro proceso, entonces se permite que el proceso 1 continúe y el código de condición cambia de cero a uno. Después, cuando el proceso 1 sale de la región crítica, el código de condición se restablece en cero para que pueda entrar otro proceso. Por otra parte, si el proceso 1 encuentra un código de condición ocupado, entonces es colocado en un ciclo de espera donde continúa probando el código de condición y espera hasta que esté libre.

Aunque es fácil implementar este procedimiento y funciona bien para un pequeño número de procesos, presenta dos desventajas fundamentales. Primero, cuando muchos procesos están esperando entrar a una región crítica, puede ocurrir inanición porque los procesos obtienen acceso de manera arbitraria. A menos que se establezca una política de primero en llegar, primero en servirse, es posible que algunos procesos sean favorecidos sobre otros. Una segunda desventaja es que los procesos en espera permanecen en ciclos improductivos que consumen recursos, requiriendo cambio de contexto. Esto se denomina **espera activa**, que no sólo consume valioso tiempo de procesamiento, sino que también depende de que los procesos en competencia prueben la llave, algo que es mejor manejado por el sistema operativo o el hardware.

WAIT y SIGNAL

WAIT y SIGNAL es una modificación de prueba e inicio, diseñada para eliminar la espera activa. Dos nuevas operaciones, que son mutuamente excluyentes y se vuelven parte del conjunto de operaciones del planificador de procesos, son WAIT y SIGNAL.

WAIT se activa cuando el proceso encuentra un código de condición ocupado. WAIT establece el bloque de control de proceso (PCB) en el estado bloqueado y lo vincula con la cola de los procesos que están esperando entrar en esta región crítica particular. Luego, el planificador de procesos escoge otro proceso para su ejecución. SIGNAL se activa cuando en la región crítica hay un proceso y el código de condición se establece en "libre". Verifica la cola de procesos que están esperando entrar en esta región crítica y escoge uno, estableciéndolo en el estado LISTO. Finalmente, el planificador de procesos escoge este proceso para su ejecución. La adición de las operaciones WAIT y SIGNAL libera los procesos del dilema de espera activa y regresa el control al sistema operativo, que entonces puede ejecutar otros trabajos mientras los procesos en espera están ociosos (WAIT).

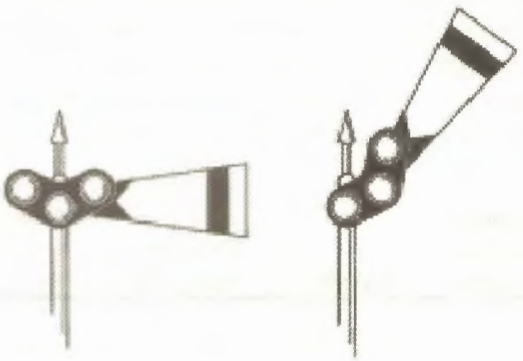
Semáforos

Un **semáforo** es una variable entera no negativa que se usa como señal binaria: una bandera. Uno de los semáforos más conocidos era el dispositivo de señalización, mostrado en la figura 6.4, usado en las carreteras para indicar si una sección de la carretera estaba libre. Cuando se

alza el brazo del semáforo, significa que la carretera está libre y se permitía proceder al tren. Cuando el brazo se baja, la carretera está ocupada y el tren debía esperar hasta que se bajara el brazo. Sólo tenía dos posiciones: arriba o abajo (encendido o apagado).

(Figura 6.4)

El semáforo usado por los ferrocarriles indica cuando puede avanzar el tren. Cuando está abajo (a), otro tren está aproximándose y es necesario que usted detenga su tren para dejarlo pasar. Si está alzado (b), su tren puede continuar.



(a) Alto (b) Todo libre

En un sistema operativo, un semáforo realiza una función semejante: indica sí y cuándo un recurso está libre y puede usarlo el proceso. Dijkstra (1965) introdujo dos operaciones para superar el problema de sincronización de procesos que acabamos de analizar. Dijkstra las denominó P y V y es como se conocen actualmente. La P y la V representan, respectivamente, las palabras en holandés *proberen* (probar) y *verhogen* (incrementar). Las operaciones P y V hacen justo eso: prueban e incrementan.

A continuación se describe cómo funcionan. Si se considera que s es una variable semáforo, entonces la operación V sobre s consiste simplemente en incrementar s por 1. La acción puede plantearse como sigue:

$$V(s): s := s + 1$$

A su vez, esto requiere una búsqueda, incrementar, y una secuencia de almacenamiento. Como la operación prueba e inicio, la operación incrementar debe realizarse como una acción indivisible a fin de evitar bloqueos. Y esto significa que ningún otro proceso puede tener acceso a s durante su operación.

La operación P sobre s consiste en probar el valor de s y, si es diferente de cero, disminuirlo por uno. La acción puede plantearse como sigue:

$$P(s): \text{Si } s > 0 \text{ entonces } s := s - 1$$

Esto requiere una prueba, una búsqueda, una disminución y una secuencia de almacenamiento. De nuevo, esta secuencia debe efectuarse como una acción indivisible en un solo ciclo de máquina o arreglarse de modo que el proceso no pueda llevarse a cabo sino hasta que haya terminado la operación (probar o incrementar).

Las operaciones para probar o incrementar son ejecutadas por el sistema operativo en respuesta a llamadas emitidas por cualquier proceso que mencione un semáforo como parámetro (esto alivia que el proceso tenga el control). Si $s = 0$, significa que la región crítica está

ocupada y que el proceso que llama a la operación probar debe esperar hasta que sea posible ejecutarla, lo cual no es posible sino hasta que $s > 0$.

Como se muestra en la tabla 6.3, P3 se coloca en el estado WAIT (para el semáforo) en el estado 4. En la tabla 6.3 también se muestra que para los estados 6 y 8, cuando un proceso sale de la región crítica, el valor de *s* se restablece en 1, indicando que la región crítica está libre. Esto, a su vez, hace que uno de los procesos bloqueados despierte, ingrese en la región crítica y *s* se restablezca en 0. En el estado 7, P1 y P2 no están intentando efectuar procesamiento en esa región crítica y P4 sigue bloqueado.

Número de estado	Acciones que llaman al proceso	Operación	Ejecución en la región crítica	Resultados bloqueados sobre *s*	Valor de *s*
0					1
1	P1	prueba(s)	P1		0
2	P1	incremento(s)			1
3	P2	prueba(s)	P2		0
4	P3	prueba(s)	P2	P3	0
5	P4	prueba(s)	P2	P3, P4	0
6	P2	incremento(s)	P3	P4	0
7			P3	P4	0
8	P3	incremento(s)	P4		0
9	P4	incremento(s)			1

(Tabla 6.3)
Secuencia de los estados de cuatro procesos que llaman a las operaciones a probar e incrementar (P y V) sobre el semáforo binario. (Nota: el valor del semáforo antes de la operación se muestra en la línea que precede la operación. El valor actual también está en la misma línea.)

Después del estado 5 en la tabla 6.3, el proceso de espera más largo fue escogido para entrar en la región crítica, aunque éste no necesariamente es el caso a menos que el sistema esté usando una política de primero en entrar, primero en salir. De hecho, la elección de cuál trabajo debe procesarse a continuación depende del algoritmo usado por esta porción del planificador de procesos.

Como puede verse a partir de la tabla 6.3, las operaciones probar e incrementar sobre el semáforo *s* refuerza el concepto de exclusión mutua, que es necesario para evitar que haya dos operaciones que intenten ejecutarse al mismo tiempo. En la literatura, el nombre tradicionalmente otorgado a este semáforo es **mutex**, que significa MUTual EXclusion (exclusión mutua). Así, las operaciones se vuelven:

```
test(mutex): if mutex > 0 then mutex: = mutex - 1
increment(mutex): mutex: = mutex + 1
```

En el capítulo 5 hablamos sobre el requerimiento de la exclusión mutua cuando varios trabajos estaban intentando tener acceso a los mismos recursos físicos. El concepto es el mismo aquí, pero se tienen varios procesos intentando tener acceso a la misma región crítica

compartida. Es posible generalizar el procedimiento a semáforos que tienen valores mayores que cero y uno.

Hasta el momento hemos considerado el problema de la exclusión mutua presentado por procesos paralelos interactivos usando los mismos datos compartidos a diferentes ritmos de ejecución. Esto puede aplicarse a varios procesos en más de un procesador, o procesos interactivos (codependientes) en un solo procesador. En este caso el concepto de región crítica se vuelve necesario porque asegura que los procesos paralelos modifican datos compartidos sólo mientras están en la región crítica.

En cálculos secuenciales, la exclusión mutua se obtiene automáticamente porque cada operación se maneja en orden, una a la vez. Sin embargo, en cálculos paralelos puede cambiar el orden de ejecución, de modo que la exclusión mutua debe plantearse y mantenerse de manera explícita. De hecho, toda la premisa de procesos paralelos depende del requerimiento que todas las operaciones sobre variables comunes se excluyen una a otra de manera consistente en el tiempo.

Cooperación de procesos

Hay ocasiones en que varios procesos trabajan directamente juntos para completar una tarea común. Dos ejemplos famosos son el problema de los productores y consumidores y el de los lectores y los escritores. Cada caso requiere tanto exclusión mutua como sincronización, y cada uno se implementa usando semáforos.

Productores y consumidores

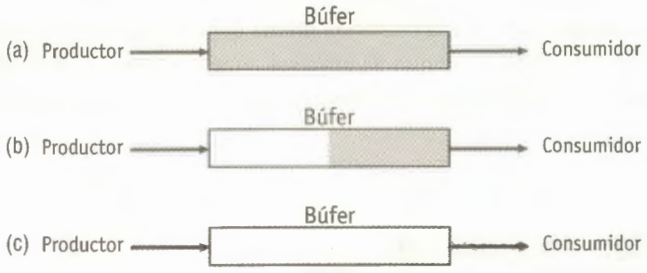
El problema clásico de los **productores y consumidores** consiste en que un proceso produce algunos datos que otro proceso consume después. Aunque describiremos el caso con un productor y un consumidor, es posible ampliarlo a varios pares de productores y consumidores.

Por un momento regresaremos al marco de referencia de la comida rápida presentado al inicio de este capítulo porque la sincronización entre dos de los procesadores (el cocinero y el empacador) constituye un problema importante en los sistemas operativos. El cocinero *produce* hamburguesas que se envían al empacador (*consumidor*). Ambos procesadores tienen acceso a un área común, el estante para las hamburguesas, que sólo puede contener un número finito de hamburguesas (esta se denomina área de búfer). El estante es necesario en el área de búfer porque la rapidez a la que se producen las hamburguesas es independiente de la velocidad a la que se consumen.

En dos extremos surgen problemas: cuando el productor intenta agregar una hamburguesa a un estante que ya está lleno (como cuando el cocinero trata de colocar una hamburguesa a un estante lleno) y cuando el consumidor acepta retirar una hamburguesa de un estante vacío (como cuando el cocinero trata de tomar una hamburguesa que aún no se ha cocinado). En la

vida real, las personas ven el estante y si está vacío o demasiado lleno, el problema se identifica y resuelve de inmediato. Sin embargo, en un sistema de cómputo esta solución no es fácil.

Considere el caso de la CPU prolífica. La CPU puede generar datos de salida mucho más rápido que una impresora. En consecuencia, puesto que esto implica un productor y un consumidor de dos velocidades distintas, es necesario un búfer donde el productor puede almacenar temporalmente datos que puede recuperar el consumidor a una velocidad más apropiada. La figura 6.5 muestra tres estados de búfer típicos.



(Figura 6.5)

El búfer puede estar en cualquiera de estos tres estados: a) búfer completo, b) búfer parcialmente vacío, o c) búfer vacío.

Debido a que el búfer sólo puede contener una cantidad finita de datos, el proceso de sincronización debe retrasar que el productor genere más datos una vez que el búfer está lleno. También debe estar preparado para retrasar que el consumidor recupere datos cuando el búfer está vacío. Esta tarea puede implementarse mediante dos semáforos de conteo: uno para indicar el número de posiciones completas en el búfer y el otro para indicar el número de posiciones vacías en el búfer.

Un tercer semáforo, mutex, asegura exclusión mutua entre procesos.

Productor	Consumidor
Produce datos	P (lleno)
P (vacío)	P (mutex)
P (mutex)	Lee datos de un búfer
Escribe datos en un búfer	V (mutex)
V (mutex)	V (vacío)
V (lleno)	Consume datos

(Tabla 6.4)

Definiciones de los procesos de productores y consumidores.

(Tabla 6.5)
Definiciones de los elementos en el algoritmo de productores y consumidores.

Variables, funciones	Definiciones
`full`	Definido como un semáforo
`empty`	Definido como un semáforo
`mutex`	Definido como un semáforo
`n`	Número máximo de posiciones en el búfer
`V (x)`	x: = x + 1 (x es cualquier variable definida como un semáforo)
`P (x)`	Si x > 0, entonces x: = x – 1
`mutex = 1`	Significa que al proceso se le permite entrar en la región crítica
`COBEGIN`	Delimitador que indica el inicio de un proceso concurrente
`COEND`	Delimitador que indica el final de un proceso concurrente

Dadas las definiciones en la tabla 6.4 y en la tabla 6.5, el algoritmo de productores y consumidores que se muestra a continuación sincroniza la interacción entre el productor y el consumidor.

Algoritmo de productores y consumidores

```
empty:          = n
full:           = 0
mutex:          = 1
COBEGIN
  repeat until no more data PRODUCER
  repeat until buffer is empty CONSUMER
COEND
```

Luego se ejecutan los procesos (`PRODUCTORES` y `CONSUMIDORES`) como se describe a continuación. Intente el código con $n = 3$ o intente un orden alterno de ejecución para ver cómo funciona en realidad.

El concepto de productores y consumidores puede extenderse a búfers que contienen registros u otros datos, así como a otras situaciones en las que se requiere la comunicación directa de proceso a proceso.

Lectores y escritores

El problema de los lectores y escritores fue planteado por primera vez por Courtois, Heymans y Parnas (1971) y se presenta cuando dos tipos de procesos necesitan tener acceso a un recurso compartido como un archivo o una base de datos. Ellos denominaron estos procesos como lectores y escritores.

El sistema de reservaciones de una aerolínea constituye un buen ejemplo. Los lectores son quienes desean información de vuelos. Se denominan lectores porque sólo leen los datos existentes; no los modifican. Y como ninguno modifica la base de datos, el sistema puede

permitir que muchos lectores estén activos al mismo tiempo: no es necesario reforzar la exclusión mutua entre ellos.

Los escritores son quienes están haciendo reservaciones para un vuelo particular. Los escritores deben ser acomodados cuidadosamente porque están modificando los datos existentes en la base de datos. El sistema no puede permitir que nadie escriba mientras alguien está leyendo (o escribiendo). En consecuencia, debe reforzar la exclusión mutua si en el sistema hay grupos de lectores y un escritor, o si hay varios escritores. Por supuesto, el sistema debe ser justo cuando refuerce su política para evitar el aplazamiento indefinido de lectores y escritores.

En el artículo original, Courtois, Heymans y Parnas ofrecieron dos soluciones usando las operaciones P y V. La primera otorga prioridad a los lectores sobre los escritores, de modo que los lectores se mantienen esperando sólo si un escritor en realidad está modificando los datos. Sin embargo, esta política resulta en inanición de los escritores si hay un flujo continuo de lectores. La segunda política otorga prioridad a los escritores. En este caso, tan pronto como llega un escritor, a cualesquiera lectores que ya estén activos se les permite terminar el procesamiento, pero todos los lectores adicionales se colocan en estado retenido hasta que haya terminado el escritor. Resulta evidente que esta política resulta en inanición del lector si está presente un flujo continuo de escritores. Este escenario es inaceptable.

Para prevenir cualquier tipo de inanición, Hoare (1974) propuso la siguiente política de combinación de prioridades. Cuando un escritor termina, a todos y cada uno de los lectores que están esperando, o en estado de retención, se permite leer. Luego, cuando ese grupo de lectores ha terminado, el escritor que está en estado de retención puede empezar, y a cualquier lector *nuevo* que llega entretanto no se le permite empezar sino hasta que termina el escritor.

El estado del sistema puede resumirse mediante cuatro contadores que inician en cero:

- Número de lectores que han *solicitado* un recurso y aún no lo liberan (R1 = 0).
- Número de lectores que están *usando* un recurso y aún no lo liberan (R2 = 0).
- Número de escritores que han *solicitado* un recurso y aún no lo liberan (W1 = 0).
- Número de escritores que están *usando* un recurso y aún no lo liberan (W2 = 0).

Esto puede implementarse usando dos semáforos para asegurar exclusión mutua entre lectores y escritores. Un recurso puede proporcionarse a todos los lectores, en el supuesto de que ningún escritor está procesando (W2 = 0). Un recurso puede proporcionarse a un escritor, en el supuesto de que ningún lector está leyendo (R2 = 0) y que ningún escritor está escribiendo (W2 = 0).

Los lectores siempre deben llamar a dos procedimientos: el primero verifica si los recursos pueden otorgarse de inmediato para lectura; y después, una vez que se ha liberado el recurso, el segundo verifica si hay algún otro escritor esperando. Lo mismo es válido para los escritores. El primer procedimiento debe determinar si el recurso puede otorgarse de inmediato para escritura y luego, una vez que se ha liberado el recurso, el segundo procedimiento encuentra si hay algún lector esperando.

Programación concurrente

Hasta el momento hemos analizado el multiprocesamiento de varios trabajos que se ejecutan al mismo tiempo en un solo procesador (que interactúa con procesadores de E/S, por ejemplo), o en multiprocesadores. El multiprocesamiento también puede referirse a un trabajo que usa varios procesadores para ejecutar conjuntos de instrucciones en paralelo. El concepto no es nuevo, pero requiere un lenguaje de programación y un sistema de cómputo que puedan atender este tipo de construcción. Este tipo de sistema se denomina sistema de procesamiento concurrente.

Aplicaciones de programación concurrente

Por naturaleza muchos lenguajes de programación son en serie: las instrucciones se ejecutan una a la vez. En consecuencia, para resolver una expresión aritmética, cada operación se realiza en secuencia siguiendo el orden prescrito por el programador y el compilador. La tabla 6.6 muestra los pasos para calcular la siguiente expresión:

$$A = 3 * B * C + 4 / (D + E) ** (F - G)$$

(Tabla 6.6)

El cálculo secuencial de la expresión requiere varios pasos. (En este ejemplo hay siete pasos, aunque cada paso puede implicar más de una operación de máquina.)

Paso no.	Operación	Resultado
1	(F – G)	Almacena la diferencia en T1
2	(D + E)	Almacena la suma en T2
3	(T2) ** (T1)	Almacena la potencia en T1
4	4 / (T1)	Almacena el cociente en T2
5	3 * B	Almacena el producto en T1
6	(T1) * C	Almacena el producto en T1
7	(T1) + (T2)	Almacena la suma en A

El orden de las operaciones es un convencionalismo matemático, un acuerdo universal que dicta la secuencia de cálculos para resolver cualquier ecuación.

Todas las ecuaciones siguen un orden de las operaciones, el cual indica que para resolver una ecuación usted primero tiene que realizar todos los cálculos que están entre paréntesis. En segundo lugar, tiene que calcular todos los exponentes. En tercero, tiene que realizar todas las multiplicaciones y divisiones. En cuarto, realice las sumas y restas. Para cada paso hay que ir de izquierda a derecha. Si usted quisiera realizar el cálculo de alguna otra forma, correría el riesgo de encontrar la respuesta incorrecta.

Para muchos propósitos de cálculo, el procesamiento en serie es suficiente; resulta fácil implementarlo y es suficientemente rápido para la mayoría de los usuarios.

Sin embargo, las expresiones aritméticas pueden procesarse de manera diferente si se usa un lenguaje que permite procesamiento concurrente. A continuación revisitaremos dos términos: COBEGIN y COEND, que indican al compilador cuáles instrucciones pueden procesar-

se de manera concurrente. Luego volveremos a escribir nuestra expresión para aprovechar un compilador de procesamiento concurrente.

```
COBEGIN
     T1 = 3 * B
     T2 = D + E
     T3 = F - G
COEND
COBEGIN
     T4 = T1 * C
     T5 = T2 ** T3
COEND
     A = T4 + 4 / T5
```

Como se muestra en la tabla 6.7, para resolver $A = 3 * B * C + 4 / (D + E) ** (F - G)$, las tres primeras operaciones pueden hacerse al mismo tiempo si el sistema de cómputo cuenta por lo menos con tres procesadores. Las dos siguientes operaciones se hacen al mismo tiempo, y la última expresión se efectúa con los resultados de los dos primeros pasos.

Paso no.	Procesador	Operación	Resultado
1	1	3 * B	Almacena el producto en T1
	2	(D + E)	Almacena la suma en T2
	3	(F – G)	Almacena la diferencia en T3
2	1	(T1) * C	Almacena el producto en T4
	2	(T2) ** (T3)	Almacena la potencia en T5
3	1	4 / (T5)	Almacena el cociente en T1
4	1	(T4) + (T1)	Almacena la suma en A

(Tabla 6.7)

Con procesamiento concurrente, el procedimiento de siete pasos puede procesarse en sólo cuatro, lo cual reduce el tiempo de ejecución.

Con este sistema hemos incrementado la velocidad de cálculo, pero también hemos aumentado la complejidad del lenguaje de programación y el hardware (tanto la maquinaria y la comunicación entre máquinas). De hecho, también hemos impuesto una gran carga sobre el programador: plantear explícitamente cuáles instrucciones pueden ejecutarse en paralelo. Esto se denomina **paralelismo explícito**.

La solución: detección automática por el *compilador* de instrucciones que pueden realizarse en paralelo. Esto se denomina **paralelismo implícito**.

Con un verdadero sistema de procesamiento concurrente, el ejemplo presentado en la tabla 6.6 y en la tabla 6.7 está codificado como una sola expresión. Es el compilador el que traduce la expresión algebraica en instrucciones por separado y decide cuáles pasos pueden efectuarse en paralelo y cuáles en el modo en serie.

Por ejemplo, el compilador puede reordenar la ecuación $Y = A + B * C + D$ como $A + D + B * C$, de modo que las operaciones $A + D$ y $B * C$ puedan hacerse en paralelo, dejando para el último el cálculo de la suma final.

Como se muestra en los cuatro casos que siguen, el procesamiento concurrente también puede reducir drásticamente la complejidad de trabajar con operaciones de matriz con ciclos, o realizar multiplicación matricial, o conducir búsquedas en paralelo y de ordenamiento de archivos fusionados. Algunos de estos sistemas usan procesadores en paralelo que ejecutan el mismo tipo de tareas.

Caso 1: Operaciones con matrices

Para efectuar una operación con matrices dentro de un bucle en tres pasos, la instrucción podría decir:

```
for(j = 1; j <= 3; j++)
a(j) = b(j) + c(j);
```

Si se usan tres procesos, la instrucción puede realizarse en un solo paso como sigue:

Procesador #1:	Procesador #2:	Procesador #3:
a(1) = b(1) + c(1)	a(2) = b(2) + c(2)	a(3) = b(3) + c(3)

Caso 2: Multiplicación matricial

La multiplicación de matrices requiere de un gran número de operaciones de multiplicación y suma que pueden presentarse de manera concurrente, tal como la ecuación: *Matriz C = Matriz 1 * Matriz 2*.

$$\underset{\text{Matriz C}}{\begin{bmatrix} z & y & x \\ w & v & u \\ t & s & r \end{bmatrix}} = \underset{\text{Matriz 1}}{\begin{bmatrix} A & B & C \\ D & E & F \end{bmatrix}} * \underset{\text{Matriz 2}}{\begin{bmatrix} K & L \\ M & N \\ O & P \end{bmatrix}}$$

Para encontrar z en la matriz C, debe multiplicar los elementos en la primera *columna* de la Matriz 1 por elementos correspondientes del primer renglón de la Matriz 2 y luego sumar los productos. Así, un cálculo de esto es: z = (A * K) + (D * L). Asimismo x = (C * K) + (F * L) y R = (C * O) + (F * P).

Usando un procesador, la respuesta a esta ecuación puede calcularse en 27 pasos. Al multiplicar algunos elementos del primer renglón de la Matriz 1 por sus elementos correspondientes en la Matriz 2, tres procesadores podrían cooperar para resolver esta ecuación en unos cuantos pasos. El número actual de productos que pueden ser calculados al mismo tiempo dependerá del número de procesadores disponible. Con dos procesadores, esto sólo

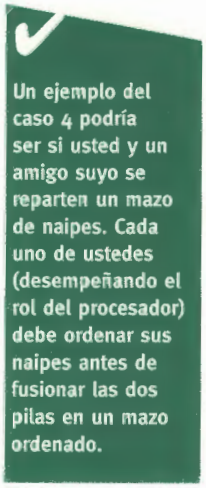
toma 18 pasos para realizar las operaciones en paralelo. Con tres, se requerirán aún menos pasos. Observe que el procesamiento concurrente no necesariamente corta la actividad de procesamiento en proporción directa al número de procesadores disponible. En este ejemplo, al duplicar el número de procesadores de uno a dos, el número de cálculos se reduce en un tercio y no en un medio.

Caso 3: Búsqueda de bases de datos

La búsqueda de base de datos es una aplicación no matemática común en los sistemas de procesamiento concurrente. Por ejemplo, si una palabra es buscada en el diccionario de una base de datos o un número de parte de una lista de inventario, el archivo entero puede ser separado en secciones discretas con un procesador asignado a cada sección. Esto da como resultado una reducción significativa en el tiempo de búsqueda. Una vez que el artículo es encontrado, todos los procesadores pueden ser desasignados y agrupados para trabajar en la siguiente tarea. Aun si el artículo buscado está en el último archivo de la base de datos, una búsqueda concurrente es más rápida que si un solo procesador fuera asignado para la búsqueda en la base de datos.

Caso 4: Ordenamiento de archivos fusionados

Al dividir en secciones un gran archivo, cada una con su propio procesador, todas las secciones pueden ordenarse al mismo tiempo. Luego, pares de secciones pueden fusionarse entre sí hasta que todo el documento esté nuevamente completo, y ordenado.



Hilos y programación concurrente

Hasta el momento hemos considerado la cooperación y sincronización de procesos tradicionales, que también se denominan procesos pesados y poseen las siguientes características:

- Pasan a través de varios estados desde su entrada inicial al sistema de cómputo hasta su terminación: listo, en ejecución, en espera, retrasado y bloqueado.
- Requieren espacio en la memoria principal, donde residen durante su ejecución.
- De vez en cuando requieren otros recursos, como datos.

Como vimos en los capítulos 3 y 4, estos procesos a menudo se intercambian entre la memoria principal y el almacenamiento secundario durante su ejecución a fin de acomodar la multiprogramación y aprovechar la memoria virtual. Cada que ocurre un intercambio, aumenta el procesamiento extra porque es necesario guardar toda la información.

Para minimizar el procesamiento extra de tiempo, la mayor parte de los sistemas operativos atienden la implementación de hilos, o procesos ligeros, que se han vuelto parte de numerosos paquetes de aplicación. Los hilos son atendidos a nivel tanto de núcleo como de usuario y es posible administrarlos ya sea por el sistema operativo o por la aplicación que los ha creado.

Un hilo, presentado en el capítulo 1, es una unidad más pequeña dentro de un proceso, que puede planificarse y ejecutarse. Los hilos comparten los mismos recursos que el proceso que los creó y ahora se vuelven un elemento más pasivo porque el hilo es la unidad que usa la CPU y es planificado para su ejecución. Los procesos pueden tener desde uno hasta varios hilos activos, que pueden crearse, planificarse y sincronizarse de manera más eficiente porque la cantidad de información necesaria se reduce. Cuando un proceso se ejecuta con múltiples hilos en un sistema de cómputo con una sola CPU, el procesador conmuta muy rápidamente de un hilo a otro, dando la impresión de que los hilos se ejecutan en paralelo. Sin embargo, es sólo en sistemas con múltiples CPU que los hilos múltiples en un proceso en realidad se ejecutan en paralelo.

Cada hilo activo en un proceso tiene sus propios registros del procesador, contador de programa, pila y estado, aunque comparte el área de datos y los recursos asignados a su proceso. Cada hilo tiene su propio contador de programa y pila, que se usa para almacenar variables creadas dinámicamente por un proceso. Por ejemplo, llamadas a funciones en C podrían crear variables que son locales para la función. Estas variables se almacenan en la pila cuando se invoca la función y se destruyen cuando la función sale. Puesto que los hilos dentro de un proceso comparten el mismo espacio y los recursos, pueden comunicarse de manera más eficiente, incrementando el rendimiento del procesamiento.

Considere cómo el servidor web puede mejorar el rendimiento y la interactividad usando hilos. Cuando un servidor web recibe solicitudes por imágenes o páginas, atiende cada solicitud con un hilo diferente. Por ejemplo, el proceso que recibe todas las solicitudes puede tener un hilo que acepta estas solicitudes y crea un nuevo hilo por separado para cada solicitud recibida. Este nuevo hilo recupera la información requerida y la envía al cliente remoto. Mientras este hilo realiza su tarea, el hilo original es libre para aceptar más solicitudes. Los servidores web son sistemas multiprocesamiento que permiten el completamiento concurrente de varias solicitudes, mejorando así el resultado y el tiempo de respuesta. En lugar de crear y destruir hilos para atender solicitudes de entrada, lo cual incrementaría el procesamiento extra, los servidores web mantienen un grupo de hilos que puede asignarse a tales solicitudes. Después que un hilo ha completado su tarea, en vez de ser destruido, se regresa al grupo para ser asignado a otra solicitud (Deitel, 2004).

Estados de un hilo

A medida que un hilo se desplaza a través del sistema, se encuentra en uno de sus cinco estados, sin contar sus estados de creación y terminado, como se muestra en la figura 6.6. Cuando una aplicación crea un hilo, lo habilita al asignarle los recursos necesarios y colocarlo en la cola LISTO. El hilo cambia de los estados LISTO a en EJECUCIÓN cuando el planificador de hilos, cuya función es semejante a la del panificador de procesos, lo asigna a un procesador.

Un hilo cambia de en EJECUCIÓN a en ESPERA cuando debe esperar que ocurra un evento fuera de su control. Por ejemplo, un clic en el ratón puede ser el evento activador para que un hilo cambie de estado, ocasionando una transición de en ESPERA a LISTO. En forma alterna, otro hilo, que ya ha completado su tarea, puede enviar una señal que indique que el

hilo en espera puede continuar para su ejecución. Esto es semejante al algoritmo de sincronización de procesos WAIT y SIGNAL.

(Figura 6.6)
Un hilo típico cambia de estado de LISTO a TERMINADO a medida que se mueve a través del sistema.

Cuando una aplicación como un procesador de palabras cuenta con la capacidad de retrasar el procesamiento de un hilo por una cantidad específica de tiempo, hace que el hilo cambie de en EJECUCIÓN a RETRASADO. Una vez que transcurre el lapso indicado, el hilo cambia de RETRASADO a LISTO. Por ejemplo, el hilo que periódicamente respalda un documento actual en el disco se retrasa durante un periodo una vez que ha completado el respaldo. Cuando termina el tiempo, realiza el siguiente respaldo y es obligado a entrar en un ciclo que prueba de manera continua si es el momento de efectuar un respaldo, desperdiciando tiempo de procesamiento y reduciendo el rendimiento del sistema. El hecho de habilitar un estado de retrasado evita los problemas del algoritmo de sincronización de procesos prueba e inicio.

Un hilo cambia de en EJECUCIÓN a BLOQUEADO cuando una aplicación emite una solicitud de E/S. Después que se completa la E/S, el hilo regresa al estado LISTO. Cuando un hilo cambia de en EJECUCIÓN a TERMINADO, todos sus recursos se liberan y puede salir del sistema.

Como puede ver, las mismas operaciones se realizan tanto en procesos tradicionales como en hilos. En consecuencia, el sistema operativo debe poder atender lo siguiente:

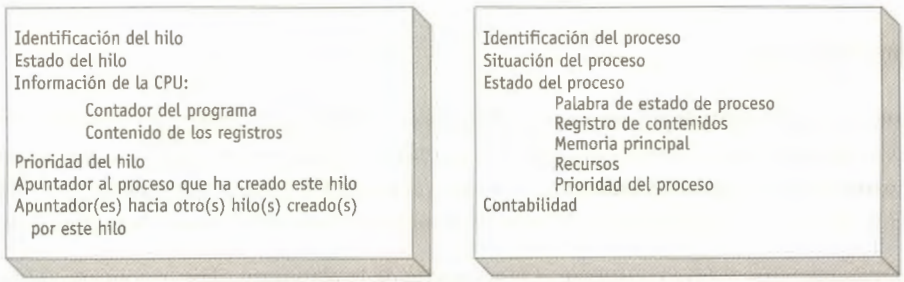
- Crear nuevos hilos.
- Habilitar un hilo de modo que esté listo para su ejecución.
- Retrasar, o colocar en hibernación, hilos durante una cantidad de tiempo específica.
- Bloquear, o suspender, hilos que están esperando el completamiento de E/S.
- Habilitar hilos en un estado de ESPERA hasta que ocurra un evento específico.
- Planificar hilos para su ejecución.
- Sincronizar la ejecución de hilos usando semáforos, eventos o variables condicionales.
- Terminar un hilo y liberar sus recursos.

Para hacer lo anterior, el sistema operativo necesita seguir la pista de la información crítica de cada hilo.

Bloques de control de hilo

Justo como los procesos están representados por bloques de control de proceso (PCB), los hilos están representados por **bloques de control de hilo** (TCB), que contienen información básica sobre un hilo, como su identificación, estado y apuntador hacia el proceso que ha creado. La figura 6.7 muestra el contenido de un TCB típico.

- La identificación de un hilo, un identificador único asignado por el sistema operativo cuando se crea el hilo.
- El estado del hilo, que cambia a medida que el hilo avanza a través de su ejecución. Los cambios de estado, así como otras entradas en el TCB, se aplican individualmente a cada hilo.
- Información de la CPU, que contiene todo lo que el sistema operativo necesita saber sobre cuál es el avance de la ejecución del hilo, qué instrucción se está realizando en ese momento y cuáles datos se están usando.
- Prioridad del hilo, usada para indicar el peso de este hilo con respecto a los otros hilos, y usado por el planificador de hilos al determinar el hilo que debe escogerse de la cola LISTO.
- Un **apuntador** al proceso que ha creado al hilo.
- Apuntadores hacia otros hilos creados por este hilo.



(Figura 6.7)
Comparación de un bloque de control de hilos (TCB) típico contra un bloque de control de procesos (PCB) del capítulo 4.

Lenguajes de programación concurrente

Los primeros lenguajes de programación no atendían la creación de hilos o la existencia de procesos concurrentes. Normalmente, proporcionaban la posibilidad a los programadores de crear un solo proceso o hilo de control. El lenguaje de programación Ada, desarrollado a fines de la década de 1970, fue uno de los primeros lenguajes en proporcionar comandos de concurrencia específicos. Java, desarrollado por Sun Microsystems, Inc., fue diseñado como una plataforma de software universal para aplicaciones de internet y ha sido ampliamente adoptado en entornos comerciales y educativos.

Java

Java fue desarrollado en 1995 como la primera plataforma de software que permitía a los programadores codificar una aplicación con la capacidad de poder ejecutarse en cualquier computadora. Este tipo de plataforma de software universal era un intento por resolver varias cuestiones: primero, el alto costo de desarrollar aplicaciones de software para cada una de las muchas arquitecturas de computadora incompatibles disponibles en el mundo; segundo, la necesidad de entornos cliente-servidor; y tercero, el crecimiento de la internet y la web, que añadieron más complejidad al desarrollo de los programas.

Mientras otros lenguajes de programación de alto nivel son compilados o interpretados, pero no ambas cosas, Java usa tanto un compilador como un intérprete. El código fuente de los procesos Java se completa primero en un lenguaje intermedio denominado Java bytecodes, que es independiente de la plataforma. Esto significa que es posible compilar un programa Java en cualquier computadora que cuente con un compilador Java, y luego los bytecodes pueden ejecutarse en cualquier computadora que tenga un intérprete Java.

El intérprete está diseñado para cumplir con las especificaciones del hardware y del sistema operativo de la computadora que ejecutará los bytecodes Java. Su función es analizar sintácticamente y ejecutar cada instrucción en bytecode en esa computadora.

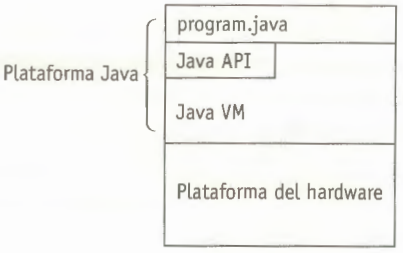
Esta combinación de compilador e intérprete facilita distribuir las aplicaciones Java porque no es necesario volver a escribirlas a fin de acomodar las características de cada sistema de cómputo. Una vez que se ha compilado un programa, puede portarse y ejecutarse en cualquier sistema que cuente con un intérprete Java.

Plataforma Java

Normalmente, la plataforma de una computadora contiene el hardware y el software donde se ejecuta un proceso. La plataforma Java es una plataforma de sólo software que funciona en la parte superior de otras plataformas basadas en el hardware. Tiene dos componentes: la Máquina Virtual Java (Java VM) y la Interfaz de Programación de Aplicaciones Java (Java API).

La máquina virtual Java constituye el fundamento de la plataforma Java y contiene el intérprete Java, que ejecuta los bytecodes proporcionados por el compilador. La máquina virtual Java se encuentra en la parte superior de muchas plataformas diferentes basadas en hardware, como se muestra en la figura 6.8.

Uno de los grandes puntos fuertes de Java es que es un programa que puede ejecutarse en diferentes plataformas desde computadoras hasta teléfonos, sin necesidad de personalizarlo para cada una.



(Figura 6.8)
Proceso usado por la plataforma Java para proteger un programa Java del hardware de una computadora.

La Java API es una colección de módulos de software que los programadores pueden usar en sus aplicaciones. La Java API está agrupada en bibliotecas de clases e interfaces relacionadas. Estas bibliotecas, también conocidas como paquetes, proporcionan objetos bien probados que varían de tipos de datos básicos hasta funciones de E/S, desde interfaces a redes hasta kits de interfaces gráficas de usuario.

Entorno del lenguaje Java

Java fue diseñado para facilitar el aprendizaje a programadores con experiencia. Su sintaxis es conocida porque se ve y se parece a C++. Está orientada a objetos, lo que significa que aprovecha metodologías modernas de desarrollo de software y se ajusta bien a aplicaciones distribuidas cliente-servidor.

Una de las características de Java es que la asignación de la memoria se hace en el tiempo de ejecución, a diferencia de C y C++, donde ésta se realiza en tiempo de compilación. El código compilado de Java se refiere a la memoria a través de "manipuladores" simbólicos que hacen que sea el intérprete Java que traduzca a direcciones de la memoria real en tiempo de ejecución. Esto significa que la asignación de la memoria y los modelos de referencia no son visibles para los programadores, sino que están controlados por completo por la plataforma subyacente en tiempo de ejecución.

Debido a que las aplicaciones Java se ejecutan en entornos distribuidos, la seguridad constituye una característica integrada muy importante del lenguaje y el sistema en tiempo de ejecución. Proporciona verificación en tiempo de compilación y verificación en tiempo de ejecución, lo cual ayuda a los programadores a crear software muy confiable. Por ejemplo, mientras se ejecuta un programa Java, puede solicitar que clases particulares se carguen desde cualquier parte en la red. En este caso, todo el código de entrada lo comprueba el verificador de bytecode, que asegura que el código es correcto y puede funcionar con seguridad sin poner en riesgo al intérprete Java. Algunas de las comprobaciones realizadas por el verificador de bytecode son para tener la certeza de que ningún operando se desborde o subdesborde, o que los tipos de los parámetros son correctos.

Con sus capacidades de sincronización sofisticadas, Java atiende multihilos al nivel de lenguaje. La biblioteca del lenguaje proporciona la clase *thread*, y el sistema en tiempo de ejecución proporciona primitivas de supervisión y condición de bloqueo. La clase *thread* es una colección de métodos usados para iniciar, ejecutar, detener y comprobar el estado de un hilo. Los hilos Java son expropiativos y, dependiendo de la plataforma en que se ejecute el intérprete Java, también puede ser por rebanadas de tiempo.

Cuando un programador declara que algunos métodos dentro de una clase deben sincronizarse, no se ejecutan de manera concurrente. Estos métodos sincronizados están bajo el control de monitores que aseguran que las variables permanezcan en un estado consistente. Cuando un método sincronizado comienza a ejecutarse a éste se le da un monitor para el objeto actual, lo que no permite la ejecución de ningún otro método sincronizado en ese objeto. El monitor es liberado cuando sale de un método sincronizado, permitiendo la ejecución de otros métodos sincronizados dentro del mismo objeto.

La tecnología Java continúa siendo popular entre los programadores por varias razones:

- Ofrece la capacidad de escribir un programa una vez y poder ejecutarlo en varias plataformas sin necesidad de realizar ningún cambio.
- Ofrece un conjunto de características robusto, como asignación de la memoria en tiempo de ejecución, seguridad y multihilos, que son características importantes en aplicaciones multimedia y distribuidas.
- Se usa para muchas aplicaciones de la web e internet, y se integra bien con navegadores como HotJava, que pueden ejecutar applets Java con audio, video y animación directamente en una página web.

Conclusión

El multiprocesamiento puede ocurrir en varias configuraciones: en un sistema con procesador único donde procesos interactivos obtienen el control de la única CPU en momentos diferentes o en sistemas con múltiples procesadores donde el trabajo de cada procesador se comunica y coopera con los otros y es sincronizado por el administrador del procesador. Tres sistemas de multiprocesamiento se describen en este capítulo: master/slave, estrechamente acoplada y simétrica. Cada una puede ser configurada en una variedad de formas.

El éxito de cualquier sistema de multiprocesamiento depende de la capacidad el sistema para sincronizar los procesos con otros recursos del sistema. El concepto de exclusión mutua es de ayuda para evitar que los procesos con los recursos asignados se bloqueen mutuamente. La exclusión mutua es mantenida por una serie de técnicas, incluyendo prueba y orden, WAIT y SIGNAL y semáforos: probar (P), incrementar (V) y mutex.

Para sincronizar los diversos procesos se usan mecanismos de hardware y software, pero debe tenerse cuidado para evitar los problemas típicos de sincronización: omisión de clientes en espera, la sincronización de productores y consumidores y la exclusión mutua de lectores y escritores.

Continuas innovaciones en procesamiento concurrente, incluyendo hilos y procesadores de núcleo múltiple, están requiriendo cambios fundamentales a los sistemas operativos, de modo que éstos puedan aprovechar las nuevas tecnologías. Los viejos días de una sola CPU, trabajando en un proceso o trabajo a la vez, ya se han ido. Pero estas innovaciones requieren volver a equipar con herramientas a las muchas aplicaciones que se ejecutan en éstas, así como al software del sistema operativo.

En el siguiente capítulo abordaremos el módulo del sistema operativo que administra impresoras, unidades de disco, unidades de cinta y terminales: el administrador de dispositivos.

Términos clave

Apuntador (o **puntero**): Una dirección u otro indicador de ubicación.

Bloque de control de hilo (TCB): Estructura de datos que contiene información sobre el estado y las características actuales de un hilo.

COBEGIN: Comando usado con COEND para indicarle a un compilador multiprocesamiento el inicio de una sección donde las instrucciones pueden procesarse de manera concurrente.

COEND: Comando usado con COBEGIN para indicarle a un compilador multiprocesamiento el final de una sección donde las instrucciones pueden procesarse de manera concurrente.

Configuración débilmente acoplada: Configuración de multiprocesamiento donde cada procesador tiene una copia del sistema operativo y controla sus propios recursos.

Configuración simétrica: Configuración multiprocesamiento donde la planificación del procesador está descentralizada y cada procesador es del mismo tipo.

Espera activa: Método por el cual los procesos, esperando la ocurrencia de un evento, prueban de manera continua si la condición ha cambiado y permanece en ciclos improductivos consumidores de recursos.

Java: Lenguaje de programación multiplataforma, desarrollado por Sun Microsystems, que semeja estrechamente a C++ y funciona en cualquier computadora capaz de ejecutar el intérprete Java.

Lectores y escritores: Problema que surge cuando dos tipos de procesos necesitan tener acceso a un recurso compartido como un archivo o una base de datos.

Maestro/esclavo: Configuración de multiprocesamiento asimétrica que consta de un sistema con procesador único conectado a procesadores "esclavos", cada uno de los cuales es administrado por el procesador "maestro" primario, que proporciona las funciones y los trabajos de planificación.

Multiprocesamiento: Cuando dos o más procesadores comparten recursos del sistema incluyendo algunos o la totalidad de los siguientes: la misma memoria principal, los dispositivos de E/S y las rutinas del programa de control.

Mutex: Condición que especifica que sólo es un proceso que puede actualizar (modificar) un recurso compartido a la vez para asegurar operación y resultados correctos.

Orden de las operaciones: Convencionalismo algebraico que dicta el orden en que se calculan los elementos de una fórmula. También se denomina *precedencia de las operaciones* o *reglas de precedencia*.

Paralelismo explícito: Tipo de programación concurrente que requiere que el programador establezca de manera explícita cuáles instrucciones pueden ejecutarse en paralelo.

Paralelismo implícito: Tipo de programación concurrente donde el compilador detecta automáticamente cuáles instrucciones pueden ejecutarse en paralelo.

Procesador de núcleo múltiple: Chip de computadora que contiene más de una unidad central de procesamiento (CPU).

Procesamiento concurrente: Ejecución por un solo procesador de un conjunto de procesos, de modo que dan la impresión de estar ocurriendo al mismo tiempo.

Procesamiento paralelo: El proceso de operar dos o más CPU en paralelo, donde más de una CPU ejecuta instrucciones de manera simultánea.

Productores y consumidores: Problema clásico en que un proceso produce datos que serán consumidos, o usados, por otro proceso.

Programación concurrente: Técnica de programación que permite que un solo procesador ejecute simultáneamente múltiples conjuntos de instrucciones.

Prueba e inicio (TS): Instrucción indivisible de máquina, que se ejecuta en un solo ciclo de máquina para determinar si el procesador está disponible.

Región crítica: Parte de un programa que debe completar su ejecución antes que otros procesos tengan acceso a los recursos que se están usando.

Semáforo: Tipo de tratamiento de datos compartidos que pueden contener valores binarios o enteros no negativos y que se usan para proporcionar exclusión mutua.

Sincronización de procesos: 1) Necesidad de que algoritmos resuelvan conflictos entre procesadores en un entorno multiprocesamiento; o 2) la necesidad de asegurar la ocurrencia de eventos en el orden idóneo si cada uno es llevado a cabo por varios procesos.

WAIT y SIGNAL: Modificación del mecanismo de sincronización prueba e inicio, diseñada para eliminar la espera activa.

Búsquedas de interés

- Lenguaje de programación ADA
- Velocidad de CPU de núcleo múltiple
- Procesamiento multihilos
- Procesamiento en tiempo real
- Cálculo por grupos

Ejercicios

Temas de investigación

A. Investigue en la literatura actual o en internet lo que se necesita para identificar un modelo informático con capacidad extraordinaria de procesamiento paralelo. Identifique el fabricante, el número máximo de procesadores que usa la computadora, cuán rápido puede realizar cálculos la computadora y aplicaciones típicas de ella. Cite sus fuentes. Si su respuesta incluye términos no usados en este capítulo, asegúrese de identificarlos.

B. Investigue en la literatura actual o en internet para identificar un proyecto reciente que haya combinado la capacidad de procesamiento de numerosas computadoras pequeñas para tratar un problema. Identifique el sistema operativo usado para coordinar los procesadores para este proyecto y analice los obstáculos a superar para hacer que el sistema trabaje. Si su respuesta incluye términos no utilizados en este capítulo, asegúrese de definirlos. Cite sus fuentes.

Ejercicios

1. Compare los procesadores de acceso para impresoras y otros dispositivos de E/S para las configuraciones master/slave y multiprocesamiento simétrico. Proporcione un ejemplo real en el que la configuración master/slave sea la preferida.
2. Compare los procesadores de acceso para la memoria principal para las configuraciones débilmente acoplada y de multiprocesamiento simétrico. Proporcione un ejemplo real en donde la configuración de multiprocesamiento simétrico sea la preferida.
3. Describa el papel del programador cuando se implementa el paralelismo explícito.
4. Describa el papel del programador cuando se implementa el paralelismo implícito.
5. ¿Qué pasos ejecuta un sistema de multiprocesamiento bien diseñado cuando detecta que un procesador falla? ¿Cuál es el objetivo principal de la mayor parte de los sistemas de multiprocesamiento?
6. ¿Cuál es el significado del término espera activa? Proporcione un ejemplo real.
7. En el capítulo anterior analizamos los bloqueos mutuos. Describa con sus propias palabras por qué es necesaria la exclusión mutua en los sistemas de multiprogramación.
8. Compare y contraste el multiprocesamiento con el procesamiento concurrente. Describa el papel de la sincronización para ambos sistemas.
9. Describa el propósito de un búfer y proporcione un ejemplo de su propia experiencia donde use con claridad los beneficios de la respuesta del sistema.
10. Considere esta fórmula:

$$G = (A + C^2) * (E-1)^3 / D + B$$

    a. Muestre el orden que seguiría un procesador para calcular G. Para hacerlo, descomponga la ecuación en el orden correcto de operaciones con un cálculo por paso. Muestre la fórmula para cada paso.
    b. Después, encuentre el valor de G: si A = 5, B = 10, C = 3, D = 8 y E = 5.
11. Considere esta fórmula:

$$G = D + (A + C^2) * E / (D + B)^3$$

    a. Muestre el orden que seguiría un procesador para calcular G. Para hacerlo, descomponga la ecuación en el orden correcto de operaciones con un cálculo por paso. Muestre la fórmula para cada paso.
    b. Después, encuentre el valor de G: si A = 5, B = 10, C = 3, D = 8 y E = 5.

12. Vuelva a escribir cada una de las siguientes expresiones aritméticas para aprovechar el modelo de procesamiento concurrente y luego codifique cada una de ellas. Use los términos `COBEGIN` y `COEND` para delimitar las secciones de código concurrente.

    a. $A+B*R*Z - N*M+C^2$

    b. `(X*(Y*Z*W*R)+M+N+P)`

    c. `((J+K*L*M*N)*I)`

13. Vuelva a escribir cada una de las siguientes expresiones aritméticas para aprovechar el modelo de procesamiento concurrente y luego codifique cada una de ellas. Use los términos `COBEGIN` y `COEND` para delimitar las secciones de código concurrente. Identifique cuáles expresiones, si hay alguna, NO se ejecutan rápidamente en un ambiente de procesamiento concurrente.

    a. $H^2*(O*(N+T))$

    b. `X*(Y*Z*W*R)`

    c. `M*T* R`

Ejercicios avanzados

14. Use el semáforo probar e incrementar (P y V) para simular el flujo de tráfico en el cruce de dos calles de sentido único. Deben cumplirse las siguientes reglas:
    - Sólo puede cruzar un automóvil a la vez.
    - Debe permitirse el cruce de un automóvil sólo si no hay ningún automóvil que venga en la dirección opuesta.
    - Cuando hay automóviles circulando en ambas calles, debe turnarse para prevenir aplazamiento indefinido en cualquiera de las calles.
15. Compare y contraste la región crítica y el grupo de trabajo.
16. Considere los siguientes segmentos de programa para dos procesos distintos (P1, P2) que se ejecutan concurrentemente y donde A y B no son variables compartidas, aunque *x* empieza en cero y es una variable compartida.

Procesador #1:

```
for(a = 1; a <= 3; a++)
x = x + 1;
```

Procesador #2:

```
for(b = 1; b <= 3; b++)
x = x + 1;
```

    Si los procesos P1 y P2 se ejecutan una sola vez a cualquier velocidad, ¿cuáles son los posibles valores resultantes de *x*? Explique sus respuestas.

17. Examine cada uno de los programas que ha escrito recientemente e indique cuáles operaciones pueden ejecutarse concurrentemente. ¿Cuánto tiempo le lleva hacer lo anterior? ¿Cuándo podría ser una buena idea escribir su programa de modo que las operaciones puedan ejecutarse concurrentemente?

18. Considere el siguiente segmento de programa tomado de un programa en C:

```
for(j = 1; j <= 12; j++)
{
   printf("\nIntroducir un valor entero:");
   scanf("%d", &x);
   if(x == 0)
     y(j)=0;
   if(x != 0)
     y(j)=10;
}
```

   a. Vuelva a codificarlo de modo que funcione de manera más efectiva en un sistema de procesador único.

   b. Dado que se dispone de un entorno multiprocesamiento con cuatro procesadores simétricos, vuelva a codificar el segmento como un programa concurrente eficiente que realiza la misma función que el programa original en C.

   c. Dado que las capacidades de todos los procesadores son idénticas, compare las velocidades de ejecución del segmento original en C con las velocidades de ejecución de los segmentos que escribió para los incisos *a*) y *b*).

Ejercicios de programación

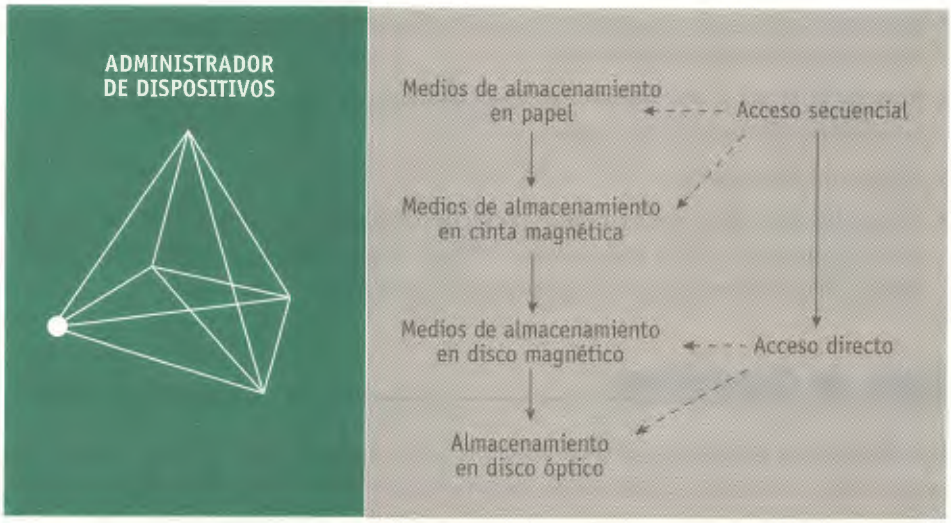
19. Dijkstra introdujo el problema del barbero que duerme (Dijkstra, 1965): Una barbería está dividida en dos salas. La sala de espera tiene n sillas y la sala de trabajo sólo tiene la silla del barbero. Cuando la sala de espera está vacía, el barbero se retira para tomar una siesta en la silla del barbero. Si llega un cliente y el barbero está durmiendo, el cliente sabe que es su turno para cortarse el cabello. Así, el cliente despierta al barbero y espera su turno en la silla del barbero. Pero si la sala de espera no está vacía, el cliente debe sentarse en la sala de espera y esperar su turno. Escriba un programa que coordine al barbero con sus clientes.

20. Patil introdujo el problema del fumador de cigarrillos (Patil, 1971): Este problema lo integran tres fumadores y un proveedor. Cada fumador desea rolar un cigarrillo y fumarlo de inmediato. Sin embargo, para fumar un cigarrillo, el fumador requiere tres ingredientes: papel, tabaco y un cerillo. Para la mayor incomodidad de todo mundo implicado, cada fumador sólo tiene uno de los ingredientes: el fumador 1 tiene bastante papel, el fumador 2 tiene bastante tabaco y el fumador 3 tiene los cerillos. Y, por supuesto, las reglas del grupo no permiten el acaparamiento, el intercambio o la compartición. Los tres ingredientes son suministrados por el proveedor, quien no fuma y posee cantidades ilimitadas de los tres productos. Pero sólo los suministra uno a la vez, y sólo cuando nadie está fumando. He aquí cómo marcha la situación: el proveedor selecciona al azar y coloca dos artículos en la mesa (a la que tienen acceso todos los fumadores), y el fumador con el ingrediente restante de

inmediato los toma, enrolla y fuma su cigarrillo. Una vez que ha terminado de fumar, lo indica al proveedor, quien entonces pone sobre la mesa otros dos artículos seleccionados al azar, y así sucesivamente.

Escriba un programa que sincronice el proveedor con los fumadores. Siga la pista de cuántos cigarrillos consume cada fumador. ¿Es justo el proveedor? ¿Por qué?

Capítulo 7 | Administración de dispositivos



"Nada permanece, todo cambia."

Heráclito de Efeso (*c. 540 d.C. - 480 d.C.*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Las características de dispositivos dedicados, compartidos y virtuales.
- Las diferencias entre medios con acceso secuencial y acceso directo.
- Los conceptos de bloqueo y almacenamiento temporal y la manera en que mejoran el rendimiento de E/S.
- Los roles del tiempo de búsqueda, tiempo de acceso y tiempo de transferencia al calcular el tiempo de acceso.
- Las diferencias en los tiempos de acceso en varios tipos de dispositivos.
- Los componentes críticos del subsistema de entrada/salida y cómo interactúan.
- Los puntos fuertes y los puntos débiles de estrategias de búsqueda común, incluyendo FCFS, SSTF, SCAN/LOOK, C-SCAN/C-LOOK y cómo se comparan.
- Los diversos niveles de RAID y las diferencias entre éstos.

A pesar de la multitud de dispositivos que aparecen (y desaparecen) en el mercado y la veloz tasa de cambio en la tecnología de los dispositivos, el administrador de dispositivos debe administrar cada uno de los dispositivos periféricos del sistema. Para lograrlo, debe mantener un equilibrio delicado de oferta y demanda: equilibrar la oferta finita de dispositivos del sistema con la casi infinita demanda de éstos por parte de los usuarios.

La administración de dispositivos implica cuatro funciones básicas:

- Supervisar el estado de cada dispositivo, como controladores de almacenamiento, impresoras y otros dispositivos periféricos.
- Reforzar políticas establecidas de antemano para determinar qué proceso obtiene un dispositivo y durante cuánto tiempo.
- Asignar los dispositivos.
- Liberar los dispositivos a dos niveles: a nivel del procesador (o tarea) cuando se ha ejecutado un comando de E/S y el dispositivo se libera temporalmente, y después al nivel del trabajo cuando se termina y el dispositivo se libera de manera permanente.

Tipos de dispositivos

Los dispositivos periféricos del sistema suelen estar en una de tres categorías: dedicados, compartidos y virtuales. Las diferencias son una función de las características de los dispositivos, así como de la manera en que los administra el administrador de dispositivos.

Los **dispositivos dedicados** sólo se asignan a un trabajo a la vez: sirven a ese trabajo durante todo el tiempo que está activo o hasta que los libera. Algunos dispositivos, como unidades de cinta, impresoras y plotters, demandan este tipo de esquema de asignación, porque sería difícil dejar que muchos usuarios los compartan. Un plotter compartido podría producir la mitad de la gráfica de un usuario y la mitad de la gráfica de otro. La desventaja de los dispositivos dedicados es que deben asignarse a un usuario único durante la ejecución de un trabajo, lo cual puede ser muy ineficiente, especialmente cuando el dispositivo no se usa 100% del tiempo. Algunos dispositivos pueden ser compartidos o virtuales.

Los **dispositivos compartidos** pueden asignarse a varios procesos. Por ejemplo, un disco, o cualquier otro dispositivo de **almacenamiento** con acceso directo (lo cual suele abreviarse como DASD: *direct access storage device*) puede ser compartido por varios procesos al mismo tiempo al intercalar sus solicitudes, aunque el administrador de dispositivos debe controlar cuidadosamente este intercalamiento. Todos los conflictos, como cuando el proceso A y el proceso B necesitan –cada uno– leer desde el mismo disco, deben resolverse con base en políticas predeterminadas para decidir qué solicitud debe manejarse primero. Más tarde en este capítulo analizaremos algunas de tales políticas.

Los **dispositivos virtuales** son una combinación de los dos primeros: son dispositivos dedicados que han sido transformados en dispositivos compartidos. Por ejemplo, las impresoras (que son dispositivos dedicados) se convierten en dispositivos compartibles a través de un programa de *spooling* que reencamina todas las solicitudes de impresión a un disco. Sólo cuando la salida de todos los trabajos está completa, y la impresora está lista para imprimir

todo el documento, es que la salida se envía a la impresora para su impresión. (Este procedimiento debe controlarse con sumo cuidado para evitar bloqueos mutuos, como explicamos en el capítulo 5.) Debido a que los discos son dispositivos compartibles, esta técnica puede convertir una impresora en varias impresoras virtuales, mejorando así tanto su rendimiento como su uso. El *spooling* es una técnica usada a menudo para acelerar el funcionamiento de dispositivos dedicados de E/S lentos.

Por ejemplo, el **controlador del bus universal en serie (USB)** actúa como una interfaz entre el sistema operativo, los controladores de dispositivos, aplicaciones y los dispositivos que están conectados vía el USB huésped. Un USB huésped (asistido por concentradores USB) puede acomodar hasta 127 dispositivos diferentes, incluyendo memoria flash, cámaras, escáners, teclados musicales, etc. Cada dispositivo está identificado por el controlador del USB huésped con un número de identificación único, que permite que varios dispositivos intercambien datos con la computadora usando la misma conexión USB.

Los controladores del USB asignan ancho de banda a cada dispositivo, dependiendo de su prioridad:

- La prioridad más alta se asigna a intercambios en tiempo real donde no se permite ninguna interrupción del flujo de datos, como datos de video y sonido.
- La prioridad media se asigna a dispositivos que pueden permitir interrupciones ocasionales sin poner en peligro el uso del dispositivo, como un teclado o una palanca de control.
- La prioridad más baja se asigna a transferencias o intercambios de volumen que pueden acomodar flujo de datos más lento, como impresoras o escáners.

Sin importar los atributos específicos del dispositivo, las diferencias más importantes entre ellos son la velocidad y el grado de compatibilidad.

Los medios de almacenamiento se dividen en dos grupos: **medios con acceso secuencial**, que almacenan registros de manera secuencial, y **dispositivos de almacenamiento con acceso directo (DASD)**, que pueden almacenar archivos secuenciales o con acceso directo. Hay grandes diferencias en su velocidad y compatibilidad.

El primer medio de almacenamiento fue el papel en forma de impresiones, tarjetas perforadas y papel para cinta.

Acceso secuencial de medios de almacenamiento

La cinta magnética fue desarrollada para los primeros sistemas de cómputo para almacenamiento secundario de rutina y se caracteriza por almacenar los registros en serie, uno después de otro.

La longitud de estos registros suele determinarla el programa de aplicación y cada registro debe identificarse por su posición en la cinta. En consecuencia, para tener acceso a un solo registro, la cinta debe montarse y adelantarse desde su inicio hasta que se encuentra la ubicación deseada. Este proceso consume tiempo porque pueden necesitarse varios minutos para leer toda la cinta.

Para ver cuánto tiempo se requiere, consideremos un sistema de cómputo hipotético que usa un carrete de cinta de 1/2 pulg de ancho y 2400 pies de largo (véase la figura 7.1). Los

(Figura 7.1)

Cinta magnética de nueve pistas con tres caracteres grabados usando paridad impar. Un carrete de cinta de 1/2 pulg, que suele usarse para respaldar una computadora central, puede almacenar miles de caracteres, o bytes, por pulgada.

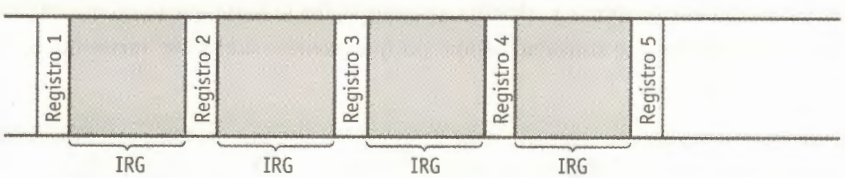
datos se registran en ocho de las pistas paralelas que recorren la cinta a lo largo. (Las nueve pistas, mostradas en la parte superior, contienen un bit de paridad que se usa para verificar errores de rutina.)

El número de caracteres que es posible registrar por pulgada está determinado por la densidad de la cinta, como 1 600 o 6 250 bytes por pulgada (bpi). Por ejemplo, si se tienen registros de 160 caracteres cada uno, y se almacenaran en una cinta con densidad de 1 600 bpi, entonces teóricamente dependería de cómo se decidiría almacenar los registros: individualmente o agrupados en bloques. Si los registros estuvieran almacenados individualmente, cada registro necesitaría estar separado por un espacio a fin de indicar sus sitios inicial y final. Si los registros estuvieran almacenados por bloques, entonces todo el bloque estaría precedido por un espacio y seguido por un espacio, pero los registros individuales están almacenados secuencialmente dentro del bloque.

Para apreciar la diferencia entre almacenar individualmente o en bloques, considere la mecánica de leer y escribir en cinta magnética. La cinta magnética se mueve bajo la cabeza de lectura/escritura sólo cuando es necesario tener acceso a un registro; el resto del tiempo está detenida. Así, la cinta se mueve a saltos: se detiene, lee y se mueve a alta velocidad o se detiene, graba y comienza de nuevo y así sucesivamente. Los registros se escriben de la misma manera.

La cinta necesita tiempo y espacio para detenerse, de modo que entre cada registro se introduce una brecha. Esta **brecha entre registros** (**IRG**) mide alrededor de 1/2 pulg de largo sin importar los tamaños de los registros que separa. En consecuencia, si 10 registros se almacenan individualmente, entonces entre cada registro hay nueve brechas de 1/2 pulg. (En este ejemplo se supone que cada registro mide 1/10 pulg.)

En la figura 7.2, para almacenar 1 pulg de datos se requieren 5.5 pulg de cinta, lo cual no es una manera muy eficiente de usar el medio de almacenamiento.



(Figura 7.2)

Brechas entre registros en cinta magnética. Cada registro sólo requiere 1/10 pulg de cinta, para un total de 1 pulg. Cuando diez registros se almacenan individualmente en cinta magnética, están separados por brechas entre registros, que suman 4.5 pulg de cinta. Esto hace un total de 5.5 pulg de cinta.

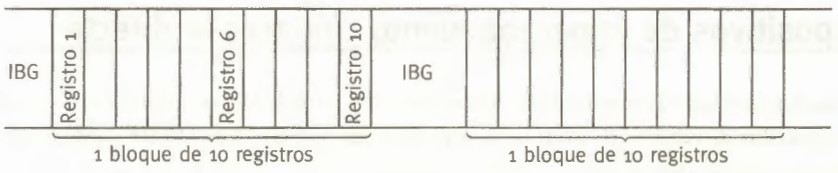
Una alternativa consiste en agrupar los registros en bloques antes de grabarlos en cinta. Esto se denomina **bloqueo** y se lleva a cabo cuando se crea el archivo. (Después, cuando se recuperan, es necesario asegurarse de desbloquearlos con seguridad.)

El número de registros en un bloque suele determinarlo el programa de aplicación, y a menudo se habilita para aprovechar la **tasa de transferencia**, que es la densidad de la cinta (medida en bpi) multiplicada por la velocidad de la unidad de cinta, o velocidad de transporte, que se mide en pulgadas por segundo (ips):

tasa de transferencia (ips) = densidad * velocidad de transporte

Digamos que en nuestro sistema hipotético, la velocidad de transporte es 200 ips. En consecuencia, a 1 600 bpi, en un segundo puede transferirse un total de 320 000 bytes, de modo que teóricamente el tamaño óptimo para un bloque es 320 000 bytes. Pero hay una trampa: esta técnica requiere que *todo* el bloque se lea en un *búfer*, de modo que éste debe ser por lo menos tan grande como el bloque.

Observe en la figura 7.3 que la brecha (ahora denominada brecha interbloques o IBG) sigue siendo de 0.5 pulg de largo, pero los datos de cada bloque de 10 registros ahora se almacenan en sólo 1 pulg de cinta, de modo que sólo se han usado 1.5 pulg de cinta (en lugar de las 5.5 pulg usadas en la figura 7.2), y sólo se han desperdiciado 0.5 pulg de cinta (en lugar de 4.5 pulg).



(Figura 7.3)
Dos bloques de registros almacenados en cinta magnética, cada uno precedido por un IBG de 1/2 pulg. Cado bloque contiene 10 registros, cada uno de los cuales sigue siendo de 1/10 pulg. El bloque, no obstante, mide 1 pulg, para un total de 1.5 pulg.

El bloqueo presenta dos ventajas:

- Se necesitan menos operaciones de E/S porque un solo comando READ puede mover todo un bloque, el registro físico que incluye varios registros lógicos, hacia la memoria principal.
- Se desperdicia menos cinta porque el tamaño del registro físico excede el tamaño de la brecha.

Las dos desventajas del bloqueo parecen ligeras por comparación:

- Para bloqueo, desbloqueo y mantenimiento de registros se requieren sobrecarga y rutinas de software.
- Es posible desperdiciar espacio de *búfer* si sólo se requiere un registro lógico, aunque para obtenerlo es necesario leer un bloque completo.

¿Cuánto tiempo es necesario para tener acceso a un bloque o un registro en cinta magnética? Depende de la ubicación del registro, aunque es posible hacer algunos cálculos generales. Por ejemplo, la cinta de carrete de 2 400 pies de longitud con una velocidad de transporte de 200 ips puede leerse sin detención en aproximadamente 2.5 minutos. En consecuencia, se necesitarían 2.5 minutos para tener acceso al último registro en la cinta. En promedio, entonces, para tener acceso a un registro se requieren 1.25 minutos. Y para tener acceso a un registro y después a otro secuencialmente se requiere tanto como para iniciar y detener una cinta, lo cual es igual a 0.003 segundo, o 3 milisegundos (ms).

Como puede verse en la tabla 7.1, los tiempos de acceso pueden variar bastante. Esto hace de la cinta magnética un medio deficiente para almacenamiento secundario de rutina, excepto para archivos con actividad secuencial muy alta; es decir, para los que en una aplicación es necesario que del 90 al 100% de los registros sean ingresados de manera secuencial durante una aplicación.

Benchmarks	Tiempo de acceso
Acceso máximo	2.5 minutos
Acceso promedio	1.25 minutos
Acceso secuencial	3 milisegundos

(Tabla 7.1)
Tiempos de acceso para una cinta magnética de 2 400 pies con una velocidad de transporte de 200 ips.

Dispositivos de almacenamiento con acceso directo

Los dispositivos de almacenamiento con acceso directo (DASD) son dispositivos que pueden leer o escribir directamente en un sitio específico en el disco. Los DASD pueden agruparse en tres categorías: discos magnéticos, discos ópticos y memoria flash. Aunque la variación en los tiempos de acceso en los DASD no son tan amplios como con la cinta magnética, la ubicación del registro específico sigue teniendo un efecto directo sobre la cantidad de tiempo necesario para tener acceso al registro.

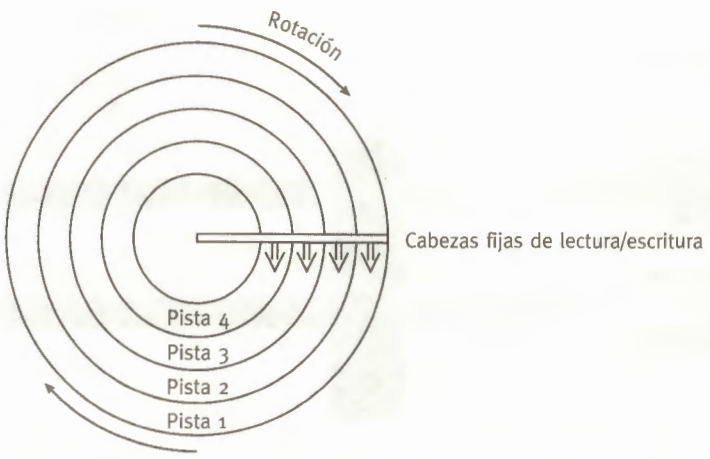
Almacenamiento en disco magnético de cabeza fija

Un disco magnético de cabeza fija semeja un gran CD o DVD cubierto con película magnética que ha sido formateada, normalmente en ambos lados, en círculos concéntricos. Cada círculo es una **pista**. Los datos se graban en serie en cada pista mediante la cabeza fija de lectura/escritura colocada arriba de la pista en cuestión.

Un disco de cabeza fija, mostrado en la figura 7.4, también es muy rápido; más rápido que los discos de cabeza móvil de los que hablaremos en la siguiente sección. Sin embargo, sus desventajas más importantes son su alto costo y reducido espacio de almacenamiento en comparación con un disco de cabeza móvil (porque las pistas deben colocarse más lejos para acomodar el ancho de las cabezas de lectura/escritura). En otra época, estos dispositivos se usaban cuando la velocidad era de importancia capital, como vuelos espaciales o aplicaciones de aeronáutica.

(Figura 7.4)

Disco de cabeza fija con cuatro cabezas de lectura/escritura, una por pista.



Almacenamiento en disco magnético de cabeza móvil

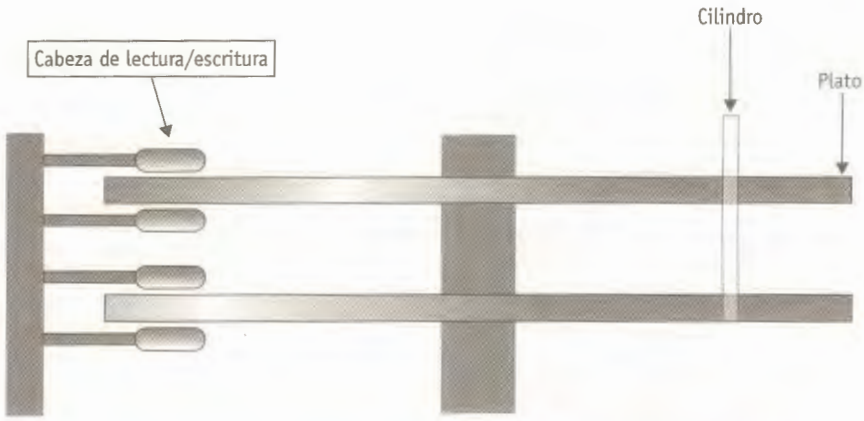
Los discos magnéticos de cabeza móvil, como la unidad de disco de una computadora, tienen una cabeza de lectura/escritura que flota sobre cada superficie de cada disco. Los discos pueden ser un solo plato, o parte de un paquete de discos, que es una pila de platos magnéticos. La figura 7.5 muestra un paquete típico de discos: varios platos están apilados sobre un eje central común, separados por espacio suficiente a fin de permitir que las cabezas de lectura/escritura se muevan entre cada par de discos.

Como se muestra en la figura 7.5, cada plato (excepto los que están en la parte superior y en la parte inferior de la pila) tiene dos superficies para grabar, y cada superficie está formateada con un número específico de pistas concéntricas donde se graban los datos. El número de pistas cambia de un fabricante a otro, aunque normalmente varían desde unas cuantas pistas –alrededor de 100 en un disquete– hasta mil o más en un disco de mayor capacidad. Cada pista en cada superficie está numerada: la pista 0 identifica el círculo concéntrico más alejado en cada superficie; la pista con el número más alto está en el centro.

El brazo, mostrado en la figura 7.6, mueve dos cabezas de lectura/escritura entre cada par de superficies: una para la superficie de arriba y una para la superficie de abajo. El brazo mueve todas las cabezas al unísono, de modo que si una cabeza está en la pista 36, entonces todas las cabezas están en la pista 36; en otras palabras, todas están posicionadas en la misma pista, pero en sus superficies respectivas creando un cilindro virtual.

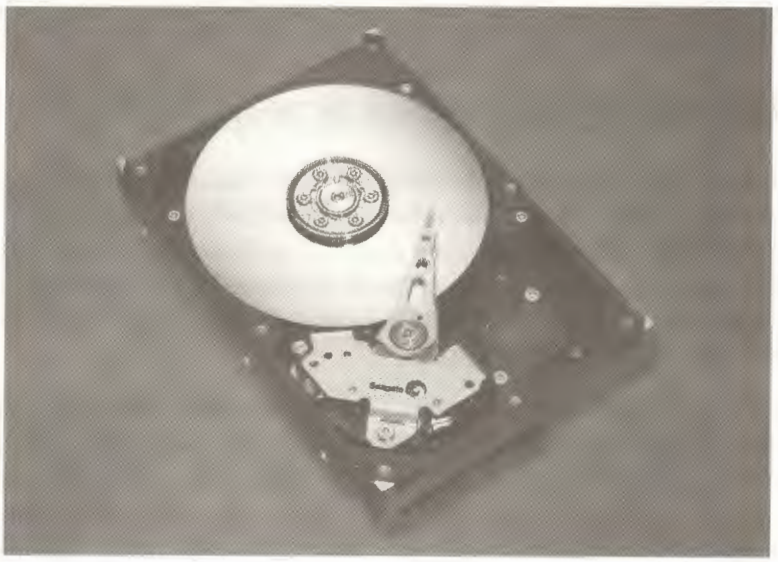
Lo anterior plantea algunas preguntas interesantes: ¿Es más eficiente escribir una serie de registros en la superficie uno y después, cuando esa superficie está llena, continuar escribiendo en la superficie dos, y después en la superficie tres y así sucesivamente? ¿O es mejor llenar todas las pistas exteriores de cada superficie antes de mover las cabezas hacia dentro a la siguiente posición de pista para continuar escribiendo?

Es más lento llenar un paquete de discos superficie por superficie que hacerlo pista por pista, lo cual conduce a un concepto valioso. Si se llena la pista 0 de todas las superficies, se obtie-



(Figura 7.5)

El paquete de discos es una pila de platos magnéticos. Las cabezas de lectura/escritura se mueven entre cada par de superficies, y el brazo mueve todas las cabezas al unísono.



(Figura 7.6)

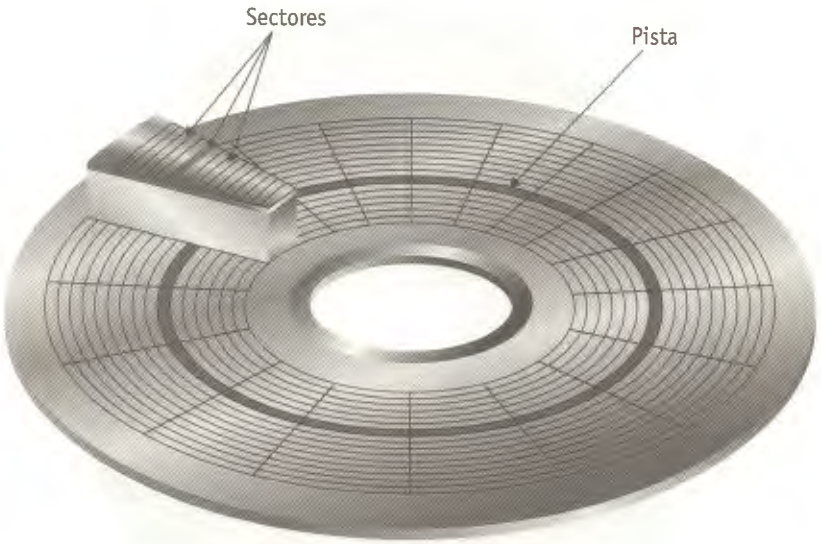
Unidad de disco típica de una PC que muestra el brazo que flota sobre la superficie de ese disco.

© Cortesía de Seagate Technology

ne un **cilindro** virtual de datos; este es el concepto de cilindro ilustrado en la figura 5.6 del capítulo 5. Hay tantos cilindros como pistas, y los cilindros son tan altos como el paquete de discos. Es posible visualizar los cilindros como una serie de latas de sopa cada vez más pequeñas, cada una anidada en las más grandes.

Para tener acceso a cualquier registro dado, el sistema requiere tres cosas: su número de cilindro, de modo que el brazo pueda mover hacia él las cabezas de lectura/escritura; su número de superficie, de modo que se active la cabeza de lectura/escritura idónea; y su número de sector, como se muestra en la figura 7.7, de modo que la cabeza de lectura/escritura sepa en qué instante debe comenzar a leer o escribir.

Una aclaración: en este análisis hemos usado el término *superficie* porque facilita la comprensión de los conceptos. Sin embargo, la literatura convencional suele usar el término *pista* para identificar tanto la superficie como la pista concéntrica. En consecuencia, nuestro uso de *superficie/pista* coincide con el término *pista* o *cabeza* usados en algunos otros textos.



(Figura 7.7)
En un disco magnético, los sectores son de tamaños diferentes, más grandes en el borde y más pequeños en el centro. El disco gira a velocidad angular constante (VAC) para compensar esta diferencia. Algunos discos ópticos pueden leer y escribir en múltiples pistas, mejorando en gran medida su capacidad de almacenamiento.

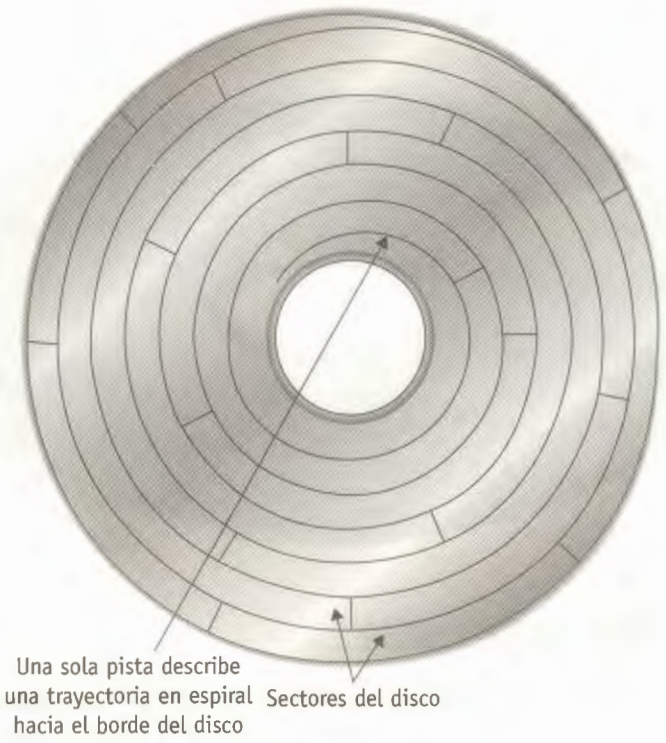
Almacenamiento en disco óptico

La aparición del almacenamiento en discos ópticos fue posible debido al desarrollo de tecnología láser. De las muchas diferencias que hay entre un disco óptico y uno magnético se encuentra el diseño de la pista y los sectores del disco.

El convenio de ortografía para un dispositivo óptico de almacenamiento es *CD*, para diferenciarlo de un almacenamiento de *disco magnético*.

Un disco magnético consta de pistas concéntricas de sectores y gira a velocidad constante, lo cual se denomina velocidad angular constante (VAC). Debido a que los sectores en la parte externa del disco pasan más rápido por la cabeza de lectura/escritura que los sectores internos, los sectores externos son mucho más grandes que los sectores ubicados cerca del centro del disco. Este formato desperdicia espacio de almacenamiento pero maximiza la velocidad con que es posible recuperar los datos.

Por otra parte, un disco óptico consta de una sola pista en espiral de sectores del mismo tamaño que va del centro a la orilla del disco, como se muestra en la figura 7.8. La pista única también consta de sectores, pero todos los sectores son del mismo tamaño, sin importar su ubicación en el disco. Ese diseño permite que muchos más sectores, y muchos más datos, quepan en un disco óptico en comparación con uno magnético del mismo tamaño. La unidad de disco ajusta la velocidad de giro del disco para compensar la ubicación del sector en el disco, lo cual se denomina velocidad lineal constante (VLC). En consecuencia, el disco gira más rápido para leer los sectores localizados en el centro del disco, y lo hace más lento para



(Figura 7.8)

En un disco óptico, todos los sectores son del mismo tamaño en todo el disco. La unidad de disco cambia la velocidad para compensar esto, aunque gira a velocidad lineal constante (VLC).

leer los sectores próximos al borde externo (White, 2006). Si usted escucha una unidad de disco en acción, puede oír sus cambios de velocidad a medida que efectúa estos ajustes.

Dos de las medidas más importantes del rendimiento de una unidad de disco óptico están sustentadas por la tasa de transferencia de datos y el tiempo de acceso medio. La tasa de transferencia de datos se mide en megabytes por segundo y se refiere a la velocidad a la que es posible leer cantidades masivas de datos desde el disco. Este factor es crucial para aplicaciones que requieren acceso secuencial, como reproducción de audio y video. Por ejemplo, un DVD con una tasa de transferencia de datos rápida omite menos marcos cuando se reproduce un segmento de video grabado que en una unidad con tasa de transferencia de datos más lenta.

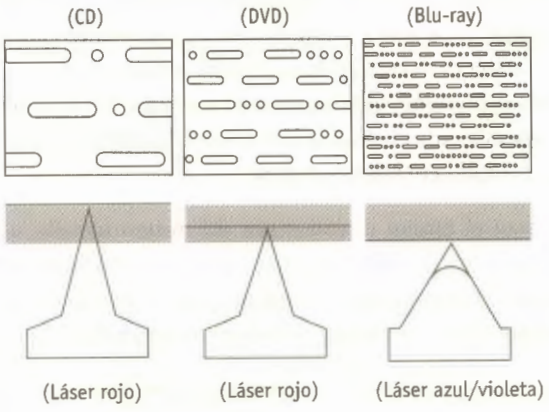
Sin embargo, la recuperación de datos que *no* se almacenaron secuencialmente, el tiempo de acceso a la unidad puede ser más importante. El tiempo de acceso, que indica el tiempo medio necesario para mover la cabeza hacia un sitio específico en el disco, se expresa en milisegundos (ms). Las unidades más rápidas tienen el menor tiempo de acceso medio, lo que es fundamental cuando la información se busca aleatoriamente, como en una base de datos. En consecuencia, una tasa de transferencia de datos rápida es más importante para acceso secuencial al disco, mientras que un tiempo de acceso rápido es más crucial cuando se recuperan datos que están ampliamente dispersos en el disco.

Una tercera característica importante de las unidades de discos ópticos es el tamaño del caché. Aunque no es una medida de velocidad, el tamaño del caché tiene un impacto

sustancial en el rendimiento percibido. Un hardware que actúa como *búfer* al transferir bloques de datos desde el disco, anticipando que el usuario pudiera desear volver a leer alguna información recientemente recuperada, lo cual puede hacerse rápidamente si la información permanece en el caché. En algunos casos, el caché también puede actuar como *búfer* de lectura anticipada, buscando el siguiente bloque de información en el disco. Los cachés de lectura anticipada pueden parecer más útiles para reproducción de multimedia, donde fluye una cantidad continua de datos. Sin embargo, debido a que se llenan pronto, los cachés de lectura anticipada se vuelven más útiles cuando se realiza paginación a través de una base de datos o un libro electrónico. En estos casos, el caché tiene tiempo para recuperarse mientras el usuario lee la pieza actual de información.

Hay varios tipos de sistemas de discos ópticos, dependiendo del medio y la capacidad de los discos: CD, DVD y Blu-ray que se muestran en la figura 7.9.

Para colocar datos en un disco óptico, un rayo láser de alta densidad hace marcas o depresiones en el disco llamadas pistas. Estas **pistas**, que representan 0's, contrastan con las áreas sin marcar llamadas **salientes**, que representan 1's. Los primeros sectores se localizan en el centro del disco y el láser se mueve hacia fuera, leyendo cada sector en turno. Si el disco tiene múltiples capas, el cursor del láser es regresado para leer la segunda capa con el brazo moviéndose desde el borde exterior hacia el centro del disco.



(Figura 7.9)
La tecnología de lectura de CD (izquierda) utiliza un láser rojo para escribir en el fondo del sustrato del disco. El DVD (centro) utiliza un pequeño láser rojo para escribir en el sustrato del disco algunas pistas que pueden ser guardadas una encima de otra. El Blu-ray (derecha), utiliza un fino láser azul para escribir en múltiples capas o pistas mejor empaquetadas (de blu-raydisc.com).

Tecnología CD-DVD

En un reproductor de CD o DVD, los datos se leen de regreso al enfocar sobre éstos un láser de baja potencia, que ilumina la capa protectora del disco sobre la pista (o pistas del DVD) en que están grabados los datos. La luz que incide en una depresión es reflejada hacia un foto-detector, mientras la luz que incide en una saliente es dispersada y absorbida. Luego, el foto-detector convierte la intensidad de la luz en una señal digital de 1's y 0's.

Las unidades CD-R y DVD requieren un controlador de disco más costoso que las unidades de disco de sólo lectura porque necesitan incorporar mecanismos de escritura específicos para cada medio. Por ejemplo, un CD está compuesto por varias capas, incluyendo una capa reflectora de oro y una capa de colorante, que se usa para grabar los datos. La cabeza de escritura usa un haz láser de alta potencia para grabar los datos. En el colorante se hace una marca permanente cuando la energía del láser es absorbida sobre ésta y no puede borrarse después de haber sido grabado. Cuando éste es leído, la existencia de una marca en la capa causa que el haz se disperse y la luz no regrese a la cabeza de lectura. Sin embargo, cuando no hay marcas, la capa de oro refleja la luz de regreso a la cabeza de lectura. Esto es semejante al proceso de leer salientes y depresiones. El software usado para crear un disco CD-R usa un formato estándar, como ISO 9096, que automáticamente verifica la existencia de errores y crea una tabla de contenido que se usa para seguir la pista de la ubicación de cada archivo.

De manera similar, los CDs regrabables y reescribibles (CD-RWs) usan un proceso denominado tecnología de cambio de fase para escribir, modificar y borrar datos. La capa de grabación del disco usa una aleación de plata, indio, antimonio y telurio. La capa de grabación tiene dos estados de fase diferentes: amorfo y cristalino. En el estado amorfo, la luz no se refleja tan bien como en el estado cristalino.

Para grabar datos, un haz láser calienta el disco y cambia el estado de cristalino a amorfo. Cuando los datos se leen por medio de un láser de baja potencia, el estado amorfo dispersa la luz que no es recogida por la cabeza de lectura. Esto es semejante a lo que ocurre cuando se leen salientes y se interpretan como un cero. Por otra parte, cuando la luz incide en las áreas cristalinas, la luz se refleja de regreso a la cabeza de lectura, lo cual es semejante a lo que ocurre cuando se leen depresiones y se interpretan como un 1. Para borrar datos, la unidad del CD-RW usa un láser de baja potencia para calentar las salientes lo suficiente para perder la aleación y volver al estado cristalino original.

Aunque los DVDs usan el mismo diseño y son del mismo tamaño que los CDs, pueden almacenar muchos más datos. Un DVD de capa doble y un solo lado puede contener el equivalente de 13 CDs; este láser rojo, con una longitud de onda más corta que el láser rojo de los CDs, hace pequeñas pistas y permite una espiral más estrecha.

Cuando las ventajas de la tecnología de compresión (que se analizan en el siguiente capítulo) se añaden a la alta capacidad de un DVD, como compresión de video MPEG, un DVD con un solo lado y doble capa puede contener 8.6 GB, espacio más que suficiente para que quepa una película de dos horas con audio mejorado.

Tecnología Blu-ray

Un disco Blu-ray es físicamente del mismo tamaño que un DVD o un CD, pero la tecnología láser utilizada para leer y escribir datos es muy diferente. Como se mostró en la figura 7.9, las pistas (cada una representa un 1) en un disco Blu-ray son mucho más pequeñas y las pistas están marcadas más estrechamente que en un DVD o en un CD. Aunque los productos Blu-ray pueden ser compatibles hacia atrás y acomodar los viejos CDs y DVDs, el láser azul/

violeta del Blu-ray (405 nm) tiene una longitud de onda más corta que la del láser rojo de los CDs/DVDs (650 nm). Esto permite que los datos sean empaquetados más estrechamente y almacenados en menos espacio.

En suma, el láser azul/violeta puede escribir en una capa mucho más delgada en el disco, permitiendo que múltiples capas sean escritas una sobre la otra incrementando de manera vasta la cantidad de datos que pueden ser almacenados en un disco sencillo. El formato del disco fue creado para promover las perspectivas comerciales del video de alta definición y para almacenar grandes cantidades de datos, particularmente para juegos y aplicaciones interactivas por medio del lenguaje de programación Java. Los reproductores de Blu-ray ejecutan programas en Java para menús e interacción con el usuario.

Como hemos escrito, cada disco Blu-ray puede contener muchos más datos (50 GB para un disco de doble capa) que un DVD similar (8.5 GB para un disco de doble capa). (Pionner Electronics reportó que sus nuevos discos de 20 capas pueden contener 500 GB.) La velocidad de lectura también es mucho más rápida con los reproductores Blu-ray, 432 Mbps (comparada con la de los reproductores de DVD que alcanzan 168.75 Mbps). Al igual que los CDs y los DVDs, los discos Blu-ray están disponibles en diversos formatos: sólo lectura (BD-ROM), regrabable (BD-R) y reescribible (BD-RE).

Almacenamiento en la memoria flash

La memoria **flash** es un tipo de memoria programable y eléctricamente borrable de sólo lectura (EEP-ROM). Se trata de un medio removible no volátil que emula la memoria de acceso aleatorio, pero a diferencia de ésta, almacena datos con seguridad incluso cuando se remueve de su fuente de energía. Históricamente, las memorias flash se usaban principalmente para almacenar información de inicio (de arranque) para computadoras, aunque ahora se usan para almacenar datos para teléfonos celulares, dispositivos móviles, reproductores de música, cámaras y más.

La memoria flash usa un fenómeno (denominado efecto túnel Fowler-Nordheim) para enviar electrones a través de un transistor de compuerta flotante donde permanecen incluso después que se ha cortado la energía eléctrica. La memoria flash permite a los usuarios almacenar datos (documentos, fotografías, música, etc.) en una tarjeta de microchip o "llave" y moverlos de un dispositivo a otro, como lo harían medios de almacenamiento magnético u óptico. La memoria flash se vende en una gran variedad de configuraciones, incluyendo *compact flash*, tarjetas inteligentes y *memory sticks*, y a menudo se conectan a la computadora mediante el puerto USB.

Los dispositivos removibles de memoria flash poseen varios nombres: unidades flash, tarjetas de memoria, tarjetas inteligentes, *thumb drives*, tarjetas de cámara, *memory sticks* y más.

La memoria flash obtiene su nombre de la técnica usada para borrar sus datos. Para escribir datos, una carga eléctrica se envía a través de un transistor, denominado compuerta flotante, luego a través de una capa de óxido metálico y hacia un segundo transistor denominado compuerta de control donde la carga se almacena en una celda hasta que es borrada. Para habilitar todos los valores, un intenso campo eléctrico, denominado flash, se aplica a toda la tarjeta. Sin embargo, la memoria flash no es indestructible. Presenta dos limitaciones: los bits

se borran sólo al aplicar el flash a un gran bloque de memoria y, con cada borrado flash, el bloque se vuelve menos estable. Con el tiempo (después de 10 000 a 1 000 000 de usos), un dispositivo de memoria flash deja de almacenar datos de manera confiable.

Tiempos de acceso DASD

Dependiendo de si un DASD tiene cabezas móviles o fijas, puede haber hasta tres factores que contribuyen al tiempo requerido para tener acceso a un archivo: tiempo de acceso, de búsqueda y de transferencia.

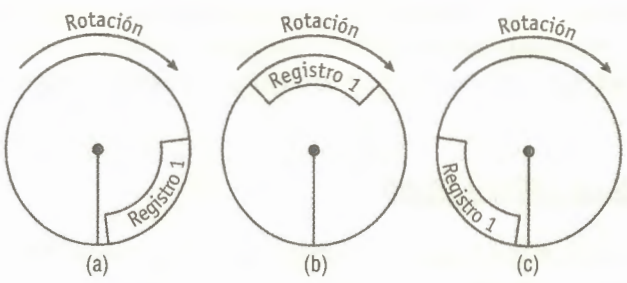
Hasta la fecha, el **tiempo de acceso** ha sido el más lento de los tres factores. Se trata del tiempo requerido para posicionar la cabeza de lectura/escritura en la pista idónea. Resulta evidente que el tiempo de acceso no es válido para dispositivos que cuentan con cabezas fijas de lectura/escritura porque cada pista tiene su propia cabeza de lectura/escritura. El **tiempo de búsqueda**, también denominado demora de rotación, es el tiempo necesario para rotar el DASD hasta que el registro solicitado se mueve bajo la cabeza de lectura/escritura. El **tiempo de transferencia** es el más rápido de los tres; es cuando los datos se transfieren realmente del almacenamiento secundario a la memoria principal.

Dispositivos de cabeza fija

Estos dispositivos pueden tener acceso a un registro al conocer su número de pista y su número de registro. La cantidad total de tiempo necesario para tener acceso a los datos depende de la velocidad de rotación, que varía de un dispositivo a otro aunque es constante en cada dispositivo, y con la posición del disco en relación a la posición de la cabeza de lectura/escritura. En consecuencia, el tiempo de acceso total es la suma del tiempo de búsqueda y de transferencia.

tiempo de búsqueda (demora de rotación)
\+ tiempo de transferencia (transferencia de datos)
tiempo de acceso

Debido a que los DASD rotan de manera continua, para el registro solicitado hay tres posiciones básicas respecto a la posición de la cabeza de lectura/escritura. La figura 7.10*a* muestra la mejor situación posible porque el registro está próximo a la cabeza de lectura/escritura cuando se ejecuta el comando de E/S; esto proporciona una demora de rotación de cero. La figura 7.10*b* muestra la situación promedio porque el registro está directamente opuesto a la cabeza de lectura/escritura cuando se ejecuta el comando de E/S, lo cual proporciona una demora de rotación de $t/2$, donde t (tiempo) es una rotación completa. La figura 7.10*c* muestra la peor situación porque el registro justo acaba de rotar por la cabeza de lectura/escritura cuando se ejecuta el comando de E/S; esto proporciona una demora de rotación de t porque se requiere una rotación completa para que el registro vuelva a posicionarse bajo la cabeza de lectura/escritura.



(Figura 7.10)
A medida que gira el disco, el registro 1 puede estar cerca de la cabeza de lectura/escritura y listo para ser escaneado, como se ve en (a); en la posición más alejada justo después de la cabeza (c); o en algún sitio en medio, como es el caso promedio (b).

¿En cuánto tiempo se tiene acceso a un registro? Si una revolución completa se hace en 16.8 ms, entonces la demora de rotación, como se muestra en la figura 7.10*b*, es de 8.4 ms. El tiempo de transferencia de datos varía de un dispositivo a otro, pero un valor típico es 0.00094 ms por byte; el tamaño del registro determina este valor. Por ejemplo, para transferir un registro con 100 bytes se requieren 0.094 ms (casi 0.1 ms). En consecuencia, usando estas cifras, los tiempos de acceso resultantes se muestran en la tabla 7.2.

(Tabla 7.2)
Tiempos de acceso para un DASD de cabeza fija a 16.8 ms/revolución.

Benchmarks	Tiempo de acceso
Acceso máximo	16.8 ms + 0.00094 ms/byte
Acceso promedio	8.4 ms + 0.00094 ms/byte
Acceso secuencial	Depende de la longitud del registro; en general es menor que 1 ms (denominado tasa de transferencia)

Los datos registrados en DASD pueden o no estar bloqueados a criterio del programador de la aplicación. Con los DASD, el bloqueo no se usa para ahorrar espacio porque entre registros no hay brecha. En lugar de ello, para ahorrar tiempo se usa bloqueo.

Para ilustrar las ventajas de bloquear los registros, se usarán los mismos valores mostrados en la tabla 7.2 para un registro que contiene 100 bytes y bloques que contienen 10 registros. Si se desea leer 10 registros de manera individual, es necesario multiplicar por diez el tiempo de acceso para un solo registro:

tiempo de acceso = 8.4 + 0.094 = 8.494 ms para un registro
tiempo de acceso total = 10(8.4 + 0.094) = 84.940 ms para diez registros

Por otra parte, para leer un bloque de 10 registros es necesario efectuar un solo acceso, de modo que calcularemos el tiempo de acceso una sola vez, multiplicando por 10 la tasa de transferencia:

tiempo de acceso = 8.4 + (0.094 * 10)
= 8.4 + 0.94
= 9.34 ms para diez registros en un bloque

Una vez que el bloque está en la memoria, se hace cargo el software que manipula el bloqueo y el desbloqueo. Pero para que esta acción sea productiva, la cantidad de tiempo usada para el desbloqueo debe ser menor de lo que se ha ahorrado en tiempo de acceso (75.6 ms).

Dispositivos de cabeza móvil

Los DASD de cabeza móvil agregan el tercer elemento al cálculo del tiempo de acceso. El tiempo de acceso es el tiempo necesario para mover el brazo a la posición sobre la pista idónea. Así, ahora la fórmula para el tiempo de búsqueda es:

```
    tiempo de posicionamiento (movimiento del brazo)
    tiempo de búsqueda (demora de rotación)
+   tiempo de transferencia (transferencia de datos)
    tiempo de acceso
```

De las tres componentes del tiempo de acceso en esta ecuación, el tiempo de **posicionamiento** es el más largo. En un momento analizaremos varias estrategias de posicionamiento.

Los cálculos para encontrar el tiempo de búsqueda (demora de rotación) y el tiempo de transferencia son los mismos que los presentados para DASD de cabeza fija. El máximo tiempo de acceso, que es el tiempo máximo necesario para mover el brazo, suele ser de 50 ms. La tabla 7.3 compara tiempos de acceso típicos para DASD de cabeza móvil.

Benchmarks	Tiempo de acceso
Acceso máximo	50 ms + 16.8 ms + 0.00094 ms/byte
Acceso promedio	25 ms + 8.4 ms + 0.00094 ms/byte
Acceso secuencial	Depende de la longitud del registro; en general es menor que 1 ms

(Tabla 7.3)
Tiempos de acceso típicos para un DASD, como la unidad de disco duro mostrada en la figura 7.6.

La variación en el tiempo de acceso ha aumentado en comparación con el del DASD de cabeza fija, aunque es relativamente pequeña; especialmente cuando se compara con el acceso a cinta, que varía desde milisegundos hasta minutos. De nuevo, el bloqueo constituye una manera aceptable para minimizar el tiempo de acceso. Si usamos el mismo ejemplo que para discos de cabeza fija y consideramos el caso promedio con 10 accesos seguidas por 10 búsquedas, obtenemos:

```
tiempo de acceso        = 25 + 8.4 + 0.094 = 33.494 ms para un registro
tiempo de acceso total  = 10 * 33.494
                        = 334.94 ms para 10 registros
                          (aproximadamente ⅓ s)
```

Pero cuando colocamos 10 registros en un bloque, el tiempo de acceso disminuye significativamente:

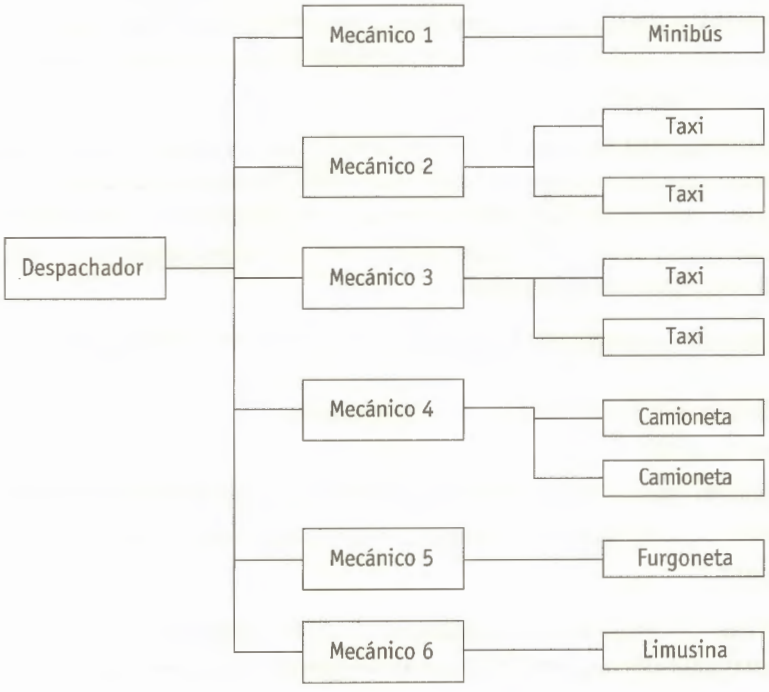
tiempo de acceso total = 25 + 8.4 + (0.094 * 10)
= 33.4 + 0.94
= 34.34 ms para 10 registros
(aproximadamente 1/29 s)

Destacamos que estas cifras no son válidas en un entorno operativo real. Por ejemplo, no hemos tomado en consideración qué más ocurre en el sistema mientras se está llevando a cabo E/S. En consecuencia, aunque es posible demostrar el rendimiento comparable de estos componentes del sistema, no estamos viendo la película completa.

Componentes del subsistema de E/S

Todas las partes del subsistema de E/S deben funcionar en armonía, y funcionan de manera semejante a la mítica "Flynn Taxicab Company" mostrada en la figura 7.11.

Desde todo el país llegan muchas solicitudes al despachador de la compañía de taxis. El trabajo del despachador consiste en atender lo más rápido posible las llamadas que llegan y encontrar quién requiere transporte, dónde está, a dónde va y cuándo. Después, el despachador organiza las llamadas en un orden que usará los recursos de la compañía de la manera más eficiente posible. Esto no es fácil, ya que la compañía de taxis cuenta con una variedad de vehículos a su disposición; taxis normales, furgonetas, camionetas, limusinas y minibuses.



(Figura 7.11) *La compañía de taxis funciona en forma semejante a un subsistema de E/S. Un mecánico puede dar servicio a varios automóviles, justo como una unidad de control de E/S puede operor varios dispositivos, como se muestra en la figura 7.12.*

El mantenimiento de estos vehículos es proporcionado por mecánicos especializados. Un mecánico atiende un tipo de vehículo, que queda a disposición de muchos conductores. Una vez que se establece el orden, el despachador llama al mecánico, quien –idealmente– tiene listo el vehículo para el conductor, quien aborda el vehículo idóneo, recoge al pasajero que está esperando y rápidamente lo lleva a su destino respectivo.

Esta situación es la ideal: pero a veces ocurren problemas; días lluviosos significan demasiadas llamadas por teléfono para cumplir cada solicitud, los vehículos no están mecánicamente del todo bien, y algunas veces la limusina ya está ocupada.

Las componentes del **subsistema** de E/S realizan funciones semejantes. El canal desempeña la parte del despachador en este ejemplo. Su trabajo consiste en ocuparse de las solicitudes de E/S provenientes de la CPU y pasarlas por la línea a la unidad de control idónea. Las unidades de control juegan el papel del mecánico. Los dispositivos de E/S desempeñan el papel de los vehículos.

Los **canales de E/S** son unidades programables ubicadas entre la CPU y las unidades de control. Su trabajo consiste en sincronizar la gran velocidad de la CPU con la lenta velocidad del dispositivo de E/S, y hacen posible la superposición de las operaciones de E/S con las operaciones del procesador, de modo que la CPU y la E/S puedan procesar de manera concurrente. Los canales usan **programas de canal** de E/S, que pueden variar de tamaño desde una hasta varias instrucciones. Cada programa de canal especifica las acciones que deben realizar los dispositivos y controla la transmisión de los datos entre la memoria principal y las unidades de control.

El canal envía una señal para cada función, y la **unidad de control de E/S** interpreta la señal, que podría decir "ir a la parte superior de la página" si el dispositivo es una impresora, o "rebobinar" si el dispositivo es una unidad de cinta. Aunque una unidad de control a veces forma parte del dispositivo, en la mayor parte de los sistemas una sola unidad de control está conectada a varios dispositivos semejantes, de modo que puede distinguirse entre la unidad de control y el dispositivo.

Algunos sistemas también tienen un controlador de disco, o interfaz de disco duro, que es un dispositivo de propósito especial usado para enlazar las unidades de disco con el bus del sistema. Las interfaces de disco duro controlan la transferencia de información entre las unidades de disco y el resto del sistema de cómputo. El sistema operativo normalmente se ocupa del controlador, no del dispositivo.

Al principio de un comando de E/S, el paso de la información de la CPU al canal es como sigue:

- Comando de E/S (READ, WRITE, RWIND, etcétera).
- Número de canal.
- Dirección del registro físico a transferir (desde o hacia almacenamiento secundario).
- Dirección inicial de un *búfer* de memoria desde el cual o hacia el cual ha de transferirse un registro.

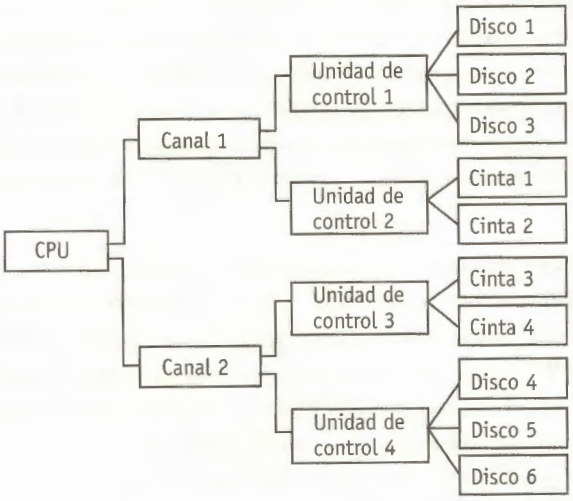
Debido a que los canales son tan rápidos como la CPU con que trabajan, cada canal puede dirigir varias unidades de control al intercalar comandos (justo como cuando se tenían mu-

chos mecánicos dirigidos por un solo despachador). Además, cada unidad de control puede dirigir varios dispositivos (justo como un mecánico podía reparar varios vehículos al mismo tiempo). Una configuración típica podría tener un canal y hasta ocho unidades de control, cada una que comunica hasta ocho dispositivos de E/S. Los canales a menudo son compartidos porque se trata de los artículos más costosos en todo el subsistema de E/S.

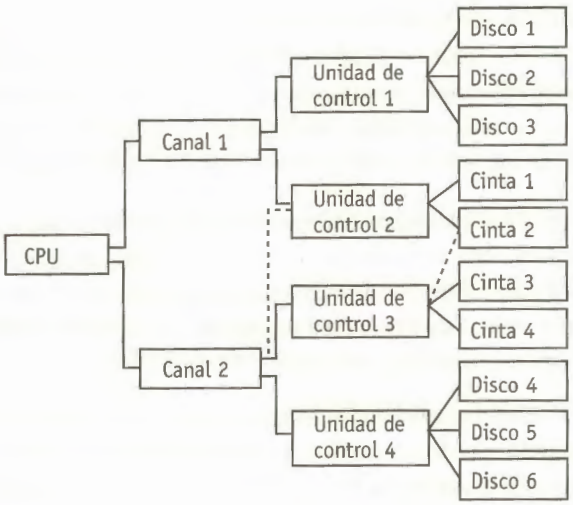
El sistema que se muestra en la figura 7.12 requiere que toda la ruta esté disponible cuando se inicia un comando de E/S. Sin embargo, en el sistema se ha integrado cierta flexibilidad porque cada unidad puede terminar de manera independiente de las otras, como explicaremos en la siguiente sección. Esta figura también muestra la naturaleza jerárquica de la interconexión y la correspondencia uno a uno entre cada dispositivo y su ruta de transmisión.

Es posible agregar flexibilidad adicional al sistema conectando más de un canal a una unidad de control o conectando más de una unidad de control a un solo dispositivo. Esto es lo mismo que si el mecánico de la compañía de taxis Flynn Taxicab puede hacer reparaciones para la compañía de taxis ABC Taxicab o si los vehículos pudieran ser usados por conductores de ABC (o si los conductores en nuestra empresa compartieran vehículos).

Estas rutas múltiples incrementan la confiabilidad del subsistema de E/S al mantener líneas de comunicación abiertas incluso si un componente funciona mal. La figura 7.13 muestra el mismo sistema presentado en la figura 7.12, pero con una unidad de control conectada a dos canales y un dispositivo conectado a dos unidades de control.



(Figura 7.12)
Configuración típica de un subsistema de E/S.



(Figura 7.13)

Configuración de un subsistema de E/S con rutas múltiples, lo cual incrementa tanto la flexibilidad como la confiabilidad. Con dos rutas adicionales, mostradas con líneas discontinuas, si la unidad de control 2 funciona mal, entonces la cinta 2 sigue siendo accesible por la unidad de control 3.

Comunicación entre dispositivos

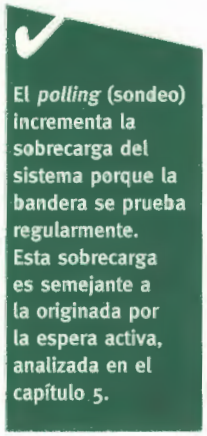
El administrador de dispositivos depende de varias características auxiliares para continuar funcionando de manera eficiente bajo las condiciones de demanda de un sistema de cómputo ocupado, y hay problemas que deben resolverse:

- Necesita saber cuáles componentes están ocupados y cuáles están libres.
- Debe poder acomodar las solicitudes que llegan durante el tráfico pesado de E/S.
- Debe poder acomodar la disparidad de velocidades entre la CPU y los dispositivos de E/S.

El primero es resuelto al estructurar la interacción entre unidades. Los dos últimos problemas se manipulan mediante registros de almacenamiento temporal y solicitudes de colas.

Como ya mencionamos, cada unidad en el subsistema de E/S puede terminar su operación de manera independiente de las otras. Por ejemplo, una vez que un dispositivo ha empezado a escribir un registro y antes que haya terminado la tarea, la conexión entre el dispositivo y su unidad de control puede cortarse de modo que la unidad de control pueda iniciar otra tarea de E/S con otro dispositivo. Mientras tanto, en el otro extremo del sistema, la CPU es libre de procesar datos mientras se lleva a cabo la E/S, lo cual permite procesamiento y E/S concurrentes.

El éxito de la operación depende de la capacidad del sistema para conocer cuándo un dispositivo ha terminado una operación. Esto se lleva a cabo con una bandera de hardware que debe probar la CPU. Esta bandera está compuesta por tres bits y reside en la **palabra de estado del canal** (CSW), que es una ubicación predefinida en la memoria principal y contiene información que indica el estado del canal. Cada bit representa uno de los componentes del subsistema de E/S, uno para el canal, otro para la unidad de control y otro para un dispositivo. Cada bit cambia de cero a uno para indicar que la unidad ha cambiado de libre a ocupada. Cada componente tiene acceso a la bandera, que puede probarse antes de proceder



con la siguiente operación de E/S para asegurar que toda la ruta está libre y viceversa. Hay dos formas comunes para llevar a cabo la prueba: *polling* y usando interrupciones.

El *polling* usa una instrucción de máquina especial para probar la bandera. Por ejemplo, la CPU prueba de manera periódica el bit de estado del canal (en la CSW). Si el canal sigue ocupado, la CPU efectúa alguna otra tarea de procesamiento hasta que la prueba muestra que el canal está libre; luego, el canal realiza la operación de E/S. La ventaja principal con este esquema es la determinación de cuán a menudo debe sondearse la bandera. Si el sondeo se hace con demasiada frecuencia, la CPU desperdicia tiempo probando la bandera justo para encontrar que el canal sigue ocupado. Por otra parte, si el sondeo se hace muy poco a menudo, el canal puede permanecer ocioso durante largos periodos.

El uso de **interrupciones** es más eficiente para probar la bandera. En vez de hacer que la CPU pruebe la bandera, un mecanismo del hardware efectúa la prueba como parte de toda instrucción de máquina ejecutada por la CPU. Si el canal está ocupado, la bandera se habilita de modo que la ejecución de la secuencia de instrucciones actuales se interrumpe automáticamente y el control se transfiere al manejador de interrupciones, que forma parte del sistema operativo y reside en una ubicación predefinida de la memoria.

El trabajo del manejador de interrupciones consiste en determinar el mejor curso de acción con base en la situación actual porque no es inusual que más de una unidad haya ocasionado la interrupción de E/S. Así, el manejador de interrupciones debe encontrar la unidad que ha enviado la señal, analiza su estado, se reinicia cuando es idóneo con la siguiente operación y finalmente devuelve el control al proceso interrumpido.

Algunos sistemas sofisticados están equipados con hardware capaz de distinguir entre varios tipos de interrupciones. Estas interrupciones están ordenadas por prioridad, y cada uno puede transferir el control a la ubicación correspondiente en la memoria. Las ubicaciones en la memoria están clasificadas en orden según las mismas prioridades. Así, en caso que la CPU esté ejecutando la rutina del manejador de interrupciones asociada con una prioridad dada, el hardware intercepta automáticamente todas las interrupciones que tengan prioridades menores o iguales. Este sistema de interrupciones múltiples ayuda a mejorar la utilización de recursos debido a que cada recurso es manejado según su importancia relativa.

El acceso directo a la memoria (DMA) es una técnica que permite que una unidad de control tenga acceso directamente a la memoria. Esto significa que una vez que ha empezado la lectura o la escritura, el resto de los datos pueden transferirse hacia y desde la memoria sin intervención de la CPU. Sin embargo, es posible que la unidad de control DMA y la CPU compitan por el bus del sistema si ocurre que lo requieren al mismo tiempo. Para activar este proceso, la CPU envía suficiente información; como el tipo de operación (lectura o escritura), el número de unidad del dispositivo de E/S requerido, la ubicación en la memoria desde donde deben leerse los datos o hacia donde deben escribirse y la cantidad de datos (bytes o palabras) a transferir hacia la unidad de control DMA para iniciar la transferencia de datos. Luego, la CPU puede dirigirse hacia otra tarea mientras la unidad de control termina la transferencia de manera independiente. El controlador DMA envía

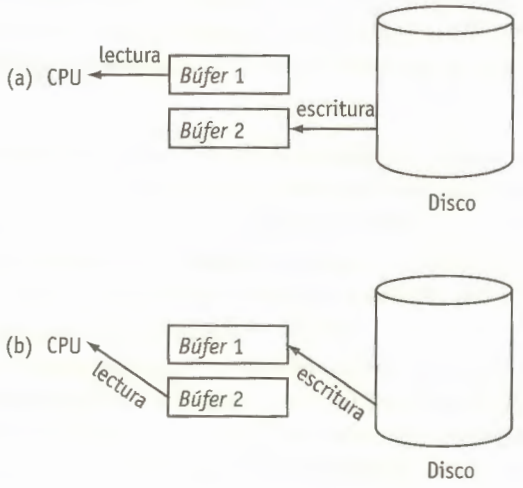
una interrupción a la CPU para indicar que la operación se ha completado. Este modo de transferencia de datos se usa para dispositivos de alta velocidad, como discos.

Sin DMA, la CPU es responsable del movimiento físico de los datos entre la memoria principal y el dispositivo: una tarea que consume tiempo y resulta en sobrecarga importante y en una disminución del uso de la CPU.

Los *búfers* se usan ampliamente para sincronizar mejor el movimiento de datos entre dispositivos de E/S relativamente lentos y la muy rápida CPU. Los *búfers* son áreas de almacenamiento temporal que residen en tres ubicaciones convenientes en todo el sistema; la memoria principal, canales y unidades de control. Se usan para almacenar datos que se leen desde un dispositivo de entrada antes que sea necesario por el procesador y para almacenar datos que serán escritos hacia un dispositivo de salida. Un uso típico de los *búfers* (ya mencionado en este capítulo) ocurre cuando registros bloqueados se leen desde, o escriben hacia, un dispositivo de E/S. En este caso, un registro físico contiene varios registros lógicos y debe residir en la memoria mientras el procesamiento de cada registro individual se lleva a cabo. Por ejemplo, si un bloque contiene cinco registros, entonces una READ física ocurre con cada seis comandos READ; todas las demás solicitudes READ se dirigen para recuperar información del *búfer* (este *búfer* puede ser habilitado por el programa de aplicación).

Otro concepto de almacenamiento temporal es el de *búfer* circular, que constituye el tema de investigación al final de este capítulo.

Para minimizar el tiempo de ocio para los dispositivos y, lo que es más importante, maximizar su resultado, se usa la técnica de doble almacenamiento temporal, como se muestra en la figura 7.14. En este sistema hay dos *búfers* en la memoria principal, en los canales y unidades de control. El objetivo consiste en tener un registro listo para ser transferido hacia o desde la memoria en cualquier instante a fin de evitar cualquier demora posible que pudiera ser ocasionada por la espera para que un *búfer* se llene de datos. Así, mientras un registro es procesado por la CPU, otro puede leerse o escribirse por el canal.



(Figura 7.14)
Ejemplo de doble almacenamiento temporal: a) La CPU está leyendo desde el búfer 1 mientras el búfer 2 se está llenando; b) Una vez que el búfer 2 está lleno, la CPU puede leerlo rápidamente mientras el búfer 1 se llena de nuevo.

Cuando se usan registros bloqueados, después de recibir el comando "READ el último registro lógico", el canal puede comenzar a leer el siguiente registro físico, lo cual resulta en superposición de E/S y procesamiento. Cuando se recibe el primer comando READ, dos registros se transfieren desde el dispositivo para llenar de inmediato ambos *búfers*. Luego, a medida que los datos de un *búfer* se han procesado, el segundo *búfer* está listo. A medida que se lee el segundo, el primer *búfer* se está llenando con datos de un tercer registro y así sucesivamente.

Administración de solicitudes de E/S

Aunque la mayoría de los usuarios creen que una solicitud de E/S es una acción elemental de máquina, el administrador de dispositivos en realidad divide la tarea en tres partes, donde cada una es administrada por un componente específico de software del subsistema de E/S. El controlador de tráfico de E/S observa el estado de todos los dispositivos, unidades de control y canales. El planificador de E/S implementa las políticas que determinan la ubicación de, y el acceso a, los dispositivos, unidades de control y canales. El manejador de dispositivos de E/S realiza la transferencia de datos real y procesa las interrupciones del dispositivo. A continuación analizaremos esto con mayor detalle.

El controlador de tráfico de E/S supervisa el estado de cada dispositivo, unidad de control y canal. Es una tarea que se vuelve más complicada a medida que aumenta el número de unidades en el subsistema de E/S y a medida que se incrementa el número de rutas entre estas unidades. El controlador de tráfico tiene tres tareas importantes a efectuar: 1) debe determinar si hay por lo menos una ruta disponible; 2) en caso de haber más de una ruta disponible, debe determinar cuál escoger; y 3) si todas las rutas están ocupadas, debe determinar cuándo habrá una disponible.

Para hacer todo lo anterior, el controlador de tráfico mantiene una base de datos que contiene el estado y las conexiones para cada unidad en el subsistema de E/S, agrupados en bloques de control de canal, de unidades de control y de dispositivos, como se muestra en la tabla 7.4.

(Tabla 7.4)
Cada bloque de control contiene la información que necesita para administrar su parte del subsistema de E/S.

Bloque de control de canal	Bloque de control de la unidad de control	Bloque de control de dispositivos
• Identificación del canal • Estado • Lista de las unidades de control conectadas con éste • Lista de procesos en espera de éste	• Identificación de la unidad de control • Estado • Lista de canales conectados con éste • Lista de dispositivos conectados con éste • Lista de procesos en espera de éste	• Identificación del dispositivo • Estado • Lista de unidades de control conectadas con éste • Lista de procesos en espera de éste

Para escoger una ruta libre que cumpla una solicitud de E/S, el controlador de tráfico rastrea hacia atrás desde el bloque de control del dispositivo solicitado a través de las unidades de control hacia los canales. En caso que no haya ninguna ruta disponible, lo cual es común en condiciones de carga pesada, el proceso (en realidad su bloque de control de proceso, o

PCB, como se describió en el capítulo 4) se enlaza con las colas mantenidas en los bloques de control con una cola por ruta. Después, cuando hay una ruta disponible, el controlador de tráfico escoge rápidamente el primer PCB de la cola para esa ruta.

El **planificador de E/S** realiza el mismo trabajo que el planificador de procesos descrito en el capítulo 4 sobre administración del procesador; es decir, asigna los dispositivos, las unidades de control y los canales. En situaciones de carga pesada, cuando el número de solicitudes es mayor que el número de rutas disponibles, el planificador de E/S debe decidir cuál solicitud satisfacer primero. Muchos de los criterios y objetivos analizados en el capítulo 4 también son válidos aquí. En muchos sistemas, la diferencia principal entre planificación de E/S y planificación de procesos es que las solicitudes de E/S no se anticipan. Una vez que ha empezado el programa del canal, se permite que prosiga hasta su completamiento incluso si solicitudes de E/S con prioridades más altas hayan ingresado a la cola. Esto es factible porque los programas del canal son relativamente cortos, de 50 a 100 ms. Otros sistemas dividen una solicitud de E/S en varias etapas y permiten preferencia de la solicitud de E/S en cualquiera de esas etapas.

Algunos sistemas permiten que el planificador de E/S otorgue un tratamiento preferencial a solicitudes de E/S provenientes de programas con alta prioridad. En ese caso, si un proceso tiene alta prioridad, entonces sus solicitudes de E/S también tienen alta prioridad y deben satisfacerse antes que otras solicitudes de E/S con prioridades más bajas. El planificador de E/S debe sincronizar su trabajo con el controlador de tráfico para asegurarse que una ruta esté disponible a fin de satisfacer solicitudes de E/S selectas.

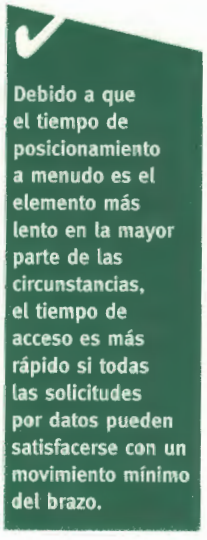
El **manejador de dispositivos de E/S** procesa las interrupciones de E/S, maneja las condiciones de error y proporciona detallados algoritmos de planificación, que dependen en grado extremo del dispositivo. Cada tipo de dispositivo de E/S tiene su propio algoritmo manejador de dispositivos.

Estrategias de posicionamiento del manejador de dispositivos

Una **estrategia de posicionamiento** para el manejador de dispositivos es la política predeterminada que el manejador de dispositivos usa para permitir acceso al dispositivo entre los muchos procesos que pueden estar esperándolo. Determina el orden en que los procesos obtienen el dispositivo, y el objetivo es mantener en un mínimo el tiempo de **posicionamiento**. Consideraremos algunas de las estrategias de **posicionamiento** de mayor uso: primero en llegar, primero en servirse (FCFS); primero el tiempo de **posicionamiento** más corto (SSTF); y SCAN con sus variantes LOOK, N-step SCAN, C-SCAN y C-LOOK.

Todo algoritmo de planificación debe hacer lo siguiente:

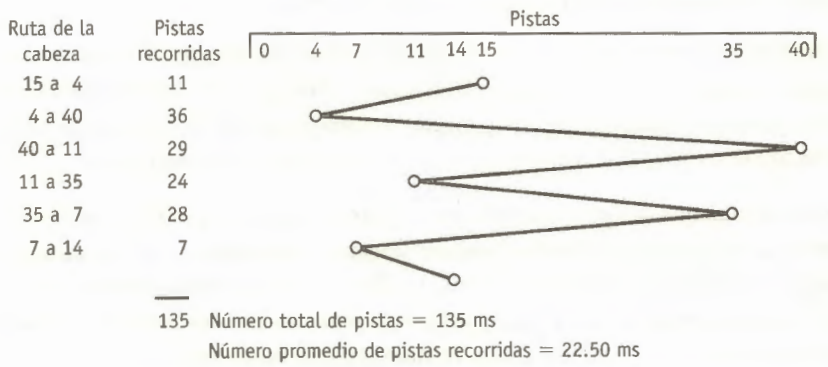
- Minimizar el movimiento del brazo.
- Minimizar el tiempo de respuesta medio.
- Minimizar la variación en el tiempo de respuesta.



Estos objetivos son sólo una guía. En sistemas reales, el diseñador debe escoger la estrategia que haga al sistema lo más justo posible para la población de usuarios generales a la vez que los recursos del sistema se usan de la manera más eficiente posible.

El algoritmo de planificación de dispositivos más simple es **primero en llegar, primero en servirse** (FCFS); es fácil de programar y esencialmente justo para los usuarios. Sin embargo, en promedio, no cumple ninguno de los objetivos de una estrategia de búsqueda. Para ilustrar este hecho, considere un disco con un solo lado con una superficie grabable donde las pistas están numeradas de 0 a 49. Se requiere 1 ms para ir de una pista a la siguiente adyacente. Para este ejemplo, suponga que mientras se recuperan datos desde la pista 15, ha llegado la siguiente lista de solicitudes: pistas 4, 40, 11, 35, 7 y 14. También se supondrá que una vez que se ha llegado a una pista solicitada, toda la pista se lee hacia la memoria principal. La ruta de la cabeza de lectura/escritura se ve como la gráfica que se muestra en la figura 7.15.

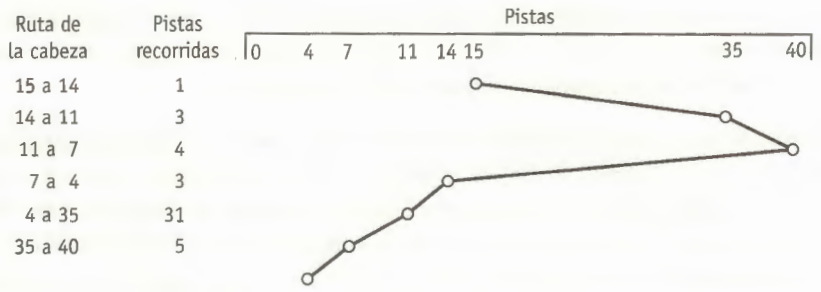
(Figura 7.15)
El brazo efectúa muchos movimientos que consumen tiempo a medida que se desplaza de una pista a otra para cumplir con todas las solicitudes en orden FCFS.



En la figura 7.15, se requiere mucho tiempo, 135 ms, para satisfacer toda la serie de solicitudes; y esto es antes de considerar el trabajo a realizar cuando el brazo se encuentra finalmente en su sitio; tiempo de búsqueda y transferencia de datos.

FCFS presenta una desventaja evidente de movimiento extremo del brazo: de 15 a 4, luego a 40, después a 11, luego a 35, después a 7 y por último, a 14. Recuerde que de las tres funciones realizadas aquí el tiempo de **posicionamiento** es la que más consume tiempo, de modo que cualquier algoritmo que pueda minimizarlo es preferible a FCFS.

El algoritmo **primero el tiempo de búsqueda más** corto (SSTF), usa la misma filosofía subyacente que el siguiente trabajo más corto o trabajo más corto primero (descrito en el capítulo 4) donde los trabajos más cortos se procesan primero y los más largos deben esperar su turno. Con SSTF, la solicitud con la pista más próxima a la que está siendo atendida (es decir, aquella con la menor distancia por recorrer) es la siguiente a cumplir, minimizando así el tiempo de acceso global. La figura 7.16 muestra lo que ocurre a las solicitudes de la misma pista que para ser atendidas requirieron 135 ms usando FCFS; en este ejemplo, todas las solicitudes de la pista están presentes y en la cola de espera.



(Figura 7.16)

Usando el algoritmo SSTF, con todas las solicitudes de las pistas en la cola de espera, el movimiento del brazo se reduce por casi un tercio mientras se satisfacen las mismas solicitudes mostradas en la figura 7.14 (usando el algoritmo FCFS).

De nuevo, sin considerar el tiempo de búsqueda y el tiempo para la transferencia de datos, fueron necesarios 47 ms para satisfacer todas las solicitudes: lo que es alrededor de un tercio del tiempo requerido por FCFS. Esta es una mejora sustancial.

Pero SSTF tiene sus desventajas. Recuerde que el algoritmo de planificación siguiente trabajo más corto o trabajo más corto primero (SJN), tiende a favorecer los trabajos cortos y aplazar los poco manejables trabajos largos. Lo mismo es cierto para SSTF. Favorece solicitudes fáciles de alcanzar y aplaza el recorrido hacia aquéllas que están fuera del camino.

Por ejemplo, suponga que en el ejemplo previo el brazo está en la pista 11 y que se está preparando para ir a la pista 7 cuando al sistema repentinamente llega un diluvio de solicitudes, incluyendo solicitudes por las pistas 22, 13, 16, 29, 1 y 21. Con SSTF, el sistema nota que la pista 13 está más próxima a la posición actual del brazo (a sólo dos pistas de distancia) que la solicitud por la pista 7 (a cinco pistas de distancia), de modo que la pista 13 es atendida primero. De las solicitudes en espera ahora, la siguiente más próxima es la pista 16, de modo que se aleja cada vez más de las pistas 7 y 1. De hecho, durante periodos de cargas pesadas, el brazo permanece en el centro del disco, donde puede satisfacer la mayor parte de las solicitudes e ignora (o aplaza indefinidamente) las que están en los bordes externos del disco. En consecuencia, este algoritmo cumple el primer objetivo de estrategias de **posicionamiento** pero falla en el cumplimiento de los otros dos.

¿Qué ocurre si dos o más trabajos o procesos están empatados? Depende de la política, pero la forma más común para deshacer empates es aplicar la política FCFS. Al más antiguo se le permite ser el siguiente.

SCAN usa un bit de dirección para indicar si el brazo se está moviendo hacia el centro del disco o se está alejando de éste. El algoritmo mueve de manera metódica el brazo desde la pista externa hacia la pista interna, atendiendo cada solicitud en su ruta. Cuando llega a la pista más interna, invierte su dirección y se mueve hacia las pistas externas, atendiendo de nuevo todas las solicitudes que están en su ruta. La variante más común de SCAN es **LOOK**, algunas veces denominado algoritmo del elevador, donde el brazo no necesariamente recorre todo el camino hacia cualquier borde, a menos que ahí haya solicitudes. En efecto, "mira" hacia delante por una solicitud antes de ir a atenderla. En la figura 7.17 se supone que el brazo se mueve primero hacia las pistas internas (con alta numeración) antes de invertir la dirección. En este ejemplo, todas las solicitudes de pistas están presentes y en la cola de espera.

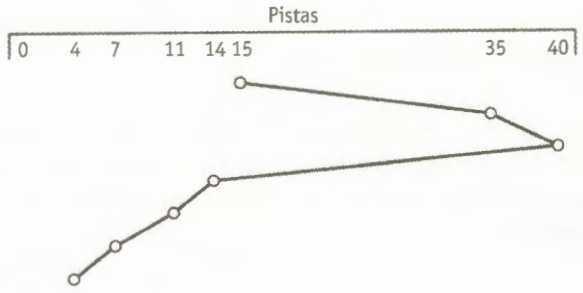
De nuevo, sin agregar tiempo de búsqueda y tiempo de transferencia de datos, fueron necesarios 61 ms para satisfacer todas las solicitudes, 14 ms más que con SSTF. Este hecho, ¿hace de SCAN un algoritmo menos atractivo que SSTF? Para este ejemplo particular, la respuesta es sí. Pero para el sistema global, la respuesta es no porque elimina la posibilidad de aplazamiento indefinido de solicitudes en sitios fuera del camino, en cualquier borde del disco.

También, a medida que llegan las solicitudes, cada una es incorporada en su sitio idóneo en la cola y es atendida cuando el brazo llega a la pista. En consecuencia, si la pista 11 está siendo atendida cuando llega la solicitud por la pista 13, el brazo continúa en su camino hacia la pista 7 y luego hacia la pista 1. La pista 13 debe esperar hasta que el brazo inicie su camino de regreso, así como debe esperar la solicitud por la pista 16. Esto elimina una gran cantidad de movimiento del brazo y ahorra tiempo al final. De hecho, SCAN satisface los tres objetivos de las estrategias de búsqueda.

Otras variantes de SCAN, además de LOOK, son N-step-SCAN, C-SCAN y C-LOOK.

(Figura 7.17) *El algoritmo LOOK hace que el brazo se mueva sistemáticamente desde la primera pista solicitada en un borde del disco a la última pista solicitada en el otro borde. En este ejemplo, todas las solicitudes por pistas están en la cola de espera.*

Ruta de la cabeza	Pistas recorridas
15 a 35	20
35 a 40	5
40 a 14	26
14 a 11	3
11 a 7	4
7 a 4	3
	61



Número total de pistas = 61 ms
Número promedio de pistas recorridas = 10.16 ms

N-step SCAN retiene todas las solicitudes nuevas hasta que el brazo empieza su recorrido de regreso. Cualquier solicitud que llegue mientras el brazo está en movimiento se agrupa para el siguiente barrido del brazo.

Con C-SCAN (abreviatura de Circular SCAN), el brazo recoge las solicitudes en su ruta durante el barrido hacia dentro. Una vez que llega a la pista más interna, de inmediato regresa a la pista más externa y empieza a atender las solicitudes que llegaron durante su último barrido hacia dentro. Con esta modificación, el sistema puede proporcionar un servicio más rápido a las solicitudes que se han acumulado en las pistas de baja numeración mientras el brazo se encontraba en su recorrido hacia dentro. La teoría aquí es que para el tiempo en que el brazo llega a las pistas con numeración más alta, inmediatamente detrás de él hay pocas solicitudes. Sin embargo, al final del disco hay tantas solicitudes como en el extremo lejano del disco y éstas han estado en espera durante el mayor tiempo. En consecuencia, C-SCAN está diseñado para proporcionar un tiempo de espera más uniforme.

C-LOOK es una optimización de C-SCAN, justo como LOOK es una optimización de SCAN. En este algoritmo, el barrido hacia dentro se detiene en la última solicitud con numeración alta, de modo que el brazo no recorre todo el camino hasta la última pista a menos que así se le pida. Además, el brazo no necesariamente regresa a la pista con numeración más baja; sólo regresa a la pista con numeración más baja que se solicita.

¿Cuál es la mejor estrategia? Corresponde al diseñador del sistema escoger el mejor algoritmo para cada entorno. Se trata de un trabajo complicado porque el rendimiento día a día de cualquier algoritmo de planificación depende de la carga que debe manejar; sin embargo, con base en estudios de simulación es posible hacer algunas amplias generalizaciones:

- FCFS trabaja bien con cargas ligeras, pero en cuanto aumenta la carga, el tiempo de servicio se vuelve inaceptablemente largo.
- SSTF es bastante conocido e intuitivamente atractivo. Funciona bien con cargas moderadas pero tiene el problema de localización bajo cargas pesadas.
- SCAN funciona bien con cargas de ligeras a moderadas y elimina el problema de aplazamiento indefinido. SCAN es semejante a SSTF en resultados y tiempos de servicio medios.
- C-SCAN funciona bien con cargas de ligeras a moderadas y presenta una variación muy pequeña en los tiempos de servicio.

El mejor algoritmo de planificación para un sistema de cómputo específico puede ser una combinación de más de un esquema. Por ejemplo, puede ser una combinación de dos esquemas: SCAN o LOOK durante cargas ligeras a moderadas, y de C-SCAN o C-LOOK durante cargas pesadas.

Estrategias de búsqueda: ordenamiento de rotación

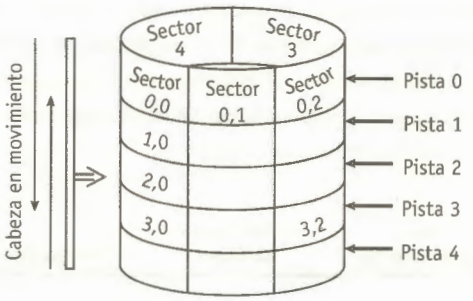
Hasta el momento sólo hemos intentado optimizar los tiempos de posicionamiento. Para completar la película, a continuación abordaremos una manera para optimizar los tiempos de búsqueda al ordenar las solicitudes una vez que se han posicionado las cabezas de lectura/escritura. Esta estrategia de búsqueda se denomina **ordenamiento de rotación**.

Para ayudar a ilustrar el concepto abstracto de ordenamiento de rotación, consideraremos un cilindro virtual con una cabeza de lectura/escritura móvil.

La figura 7.18 ilustra la lista de solicitudes que llegan a este cilindro para sectores diferentes en pistas diferentes. Para este ejemplo se supone que el cilindro sólo tiene cinco pistas numeradas del 0 al 4, y que cada pista contiene cinco sectores, numerados del 0 al 4. Las solicitudes se toman en el orden en que llegan.

(Figura 7.18)

El cilindro de cabeza móvil requiere 5 ms para mover la cabeza de lectura/escritura de una pista a la siguiente. La cabeza de lectura/escritura está posicionada inicialmente en la pista 0, sector 0. Requiere 5 ms para girar el cilindro del sector 0 al sector 4 y un milisegundo para transferir un sector desde el cilindro hasta la memoria principal.



Lista de solicitud

Pista	Sector
0	1
1	4
1	3
2	0
2	3
2	4
3	2
3	0

Cada solicitud se satisface según llega. Los resultados se muestran en la tabla 7.5.

(Tabla 7.5)

Se requieren 36 ms para cumplir las acho solicitudes en el cilindro de cabezas móviles mostrado en la figura 7.18.

Solicitud (pista, sector)		Tiempo de posicionamiento		Tiempo de búsqueda		Transferencia de datos		Tiempo total
1.	0,1	0		1		1		2
2.	1,4	5		2		1		8
3.	1,3	0		3		1		4
4.	2,0	5		1		1		7
5.	2,3	0		2		1		3
6.	2,4	0		0		1		1
7.	3,2	5		2		1		8
8.	3,0	0		2		1		3
	TOTALES	15 ms	+	13 ms	+	8 ms	=	36 ms

Aunque nada puede hacerse para mejorar el tiempo dedicado al movimiento de la cabeza de lectura/escritura porque esto depende del hardware, la cantidad de tiempo desperdiciado debido a la demora de rotación puede reducirse. Si las solicitudes están ordenadas dentro de cada pista de modo que el primer sector solicitado en la segunda pista es el siguiente número más alto que el que acaba de ser atendido, la demora de rotación se minimiza, como se muestra en la tabla 7.6.

Para implementar correctamente este algoritmo, el controlador de dispositivos debe contar con detección de rotación, de modo que el dispositivo controlador pueda ver qué sector se encuentra actualmente bajo la cabeza de lectura/escritura. En condiciones de carga pesada de E/S, este tipo de ordenamiento puede aumentar significativamente el resultado, especialmente si el dispositivo cuenta con cabezas de lectura/escritura en lugar de cabezas móviles.

Los cilindros de paquetes de discos son una extensión del ejemplo previo. Una vez que las cabezas están posicionadas sobre un cilindro, cada superficie tiene su propia cabeza de lectura/escritura móvil como se muestra en la figura 7.5. Así, el ordenamiento de rotación puede lograrse según una base de superficie por superficie, y las cabezas de lectura/escritura pueden activarse a su vez sin que sea necesario ningún movimiento adicional.

Solicitud	(pista, sector)	Tiempo de posicionamiento		Tiempo de búsqueda		Transferencia de datos		Tiempo total
1.	0,1	0		1		1		2
2.	1,3	5		1		1		7
3.	1,4	0		0		1		1
4.	2,0	5		0		1		6
5.	2,3	0		2		1		3
6.	2,4	0		0		1		1
7.	3,0	5		0		1		6
8.	3,2	0		1		1		2
	TOTALES	15 ms	+	5 ms	+	8 ms	=	28 ms

(Tabla 7.6)

Se requieren 28 ms para cumplir las mismas ocho solicitudes mostradas en la tabla 7.5 después que las solicitudes se ordenan para minimizar el tiempo de búsqueda, reduciéndolo de 13 ms a 5 ms.

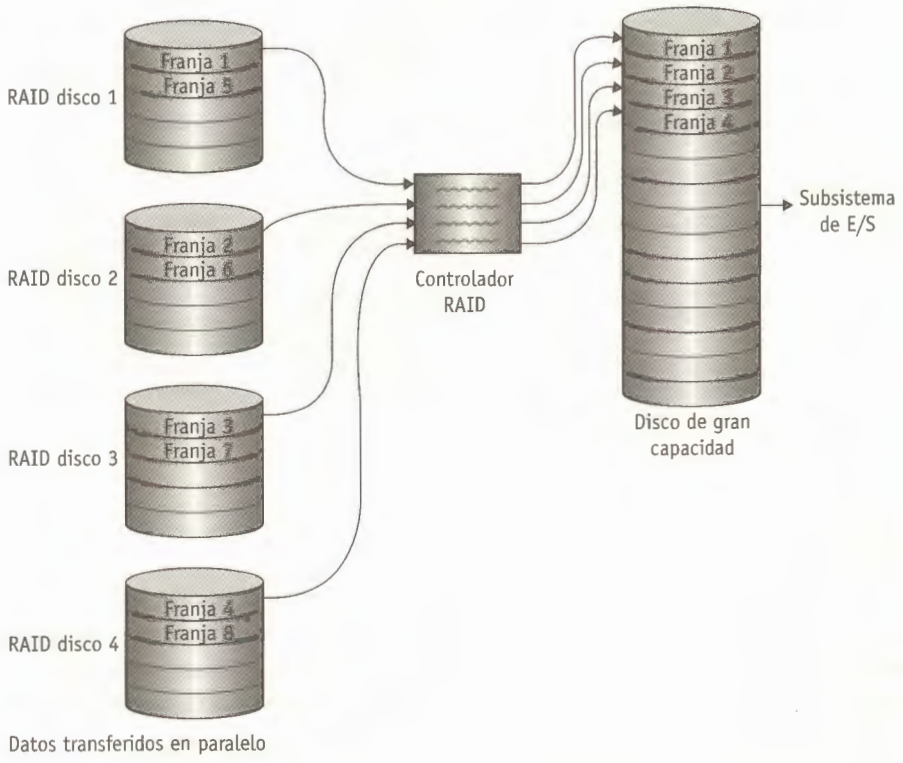
Sólo una cabeza de lectura/escritura puede estar activa al mismo tiempo, de modo que el controlador debe estar listo para manejar solicitudes mutuamente excluyentes como la solicitud 2 y la solicitud 5 en la tabla 7.6. Son mutuamente excluyentes porque ambas están solicitando el sector 3, una en la pista 1 y otra en la pista 2, pero sólo una de las dos cabezas de lectura/escritura puede transmitir en cualquier instante dado. Así, la política podría establecer que las pistas sean procesadas desde las de baja numeración hasta las de alta numeración y luego desde las de alta numeración hasta las de baja numeración en un movimiento de barrido como el usado en SCAN. En consecuencia, para manejar solicitudes en un paquete de discos, habría dos ordenamientos de las solicitudes: uno para manejar la posición de las cabezas de lectura/escritura que integran el cilindro y la otra para manejar el procesamiento de cada cilindro.

RAID

RAID es un conjunto de unidades de disco físicas que el sistema operativo considera como una sola unidad lógica. Fue introducido para cerrar la brecha cada vez más amplia entre procesadores cada vez más rápidos y unidades de disco más lentas. RAID supone que varias unidades de disco de pequeña capacidad son preferibles a unas cuantas unidades de disco de gran capacidad porque, al distribuir los datos entre discos más pequeños, el sistema puede tener acceso de manera simultánea a los datos solicitados desde las múltiples unidades, lo cual resulta en un rendimiento de E/S mejorado y recuperación de datos mejorada en caso de fallo de algún disco.

Una configuración típica de discos puede tener cinco unidades de disco conectadas a un controlador especializado, que aloja al software que coordina la transferencia de datos desde los discos en el arreglo hasta un disco de gran capacidad conectado al subsistema de E/S, como se muestra en la figura 7.19. Esta configuración es considerada por el sistema operativo como un disco de gran capacidad, por lo que no se requiere ningún cambio de software.

(Figura 7.19)
Los datos transferidos en paralelo de una configuración RAID de nivel 0 a un disco de gran capacidad. El software en el controlador asegura que las franjas se almacenen en el orden correcto.



Aunque RAID introduce el concepto bastante necesario de redundancia para ayudar a que los sistemas se recuperen de fallo en el hardware, la mayor parte de las configuraciones RAID requieren más unidades de disco, lo que incrementa los costos del hardware.

Los datos están divididos en segmentos denominados franjas, que están distribuidos a través de los discos en el arreglo. Un conjunto de franjas consecutivas a través de los discos se denomina **división** y todo el proceso se denomina por divisiones. La figura 7.19 muestra cómo en un arreglo de cuatro discos están distribuidas franjas de datos.

La tecnología RAID fue propuesta originalmente por investigadores en la universidad de California en Berkeley, quienes crearon el acrónimo que significa *Redundant Array of Inexpensive Disks* (conjunto redundante de discos baratos). Desde entonces, la industria ha modificado el acrónimo, de modo que ahora representa *Redundant Array of Independent Disks* (conjunto redundante de discos independientes) a fin de recalcar el rendimiento y la confiabilidad mejoradas del esquema.

Hay siete niveles primarios de RAID, numerados de 0 a 6. Debe observarse que los niveles no indican ninguna jerarquía, sino que denotan diferentes tipos de configuraciones y capa-

cidades de corrección de errores. La tabla 7.7 constituye un resumen de los siete niveles, incluyendo la forma en que se implementa la corrección de errores y la calidad percibida del rendimiento.

Nivel RAID	Método para corrección de errores	Tasa de solicitudes de E/S	Tasa de transferencia de datos
0	Ninguno	Excelente	Excelente
1	De espejo	Lectura: Buena Escritura: Aceptable	Lectura: Aceptable Escritura: Aceptable
2	Código Hamming	Deficiente	Excelente
3	Palabra de paridad	Deficiente	Excelente
4	Franja de paridad	Lectura: Excelente Escritura: Aceptable	Lectura: Aceptable Escritura: Deficiente
5	Franja de paridad distribuida	Lectura: Excelente Escritura: Aceptable	Lectura: Aceptable Escritura: Deficiente
6	Franja de paridad distribuida y verificación de datos independiente	Lectura: Excelente Escritura: Deficiente	Lectura: Aceptable Escritura: Deficiente

(Tabla 7.7)
Los siete niveles normales de RAID proporcionan varios grados de corrección de errores. Cuando se escoge un sistema, algunos factores importantes a considerar son costo, velocidad y aplicaciones del sistema.

Nivel cero

RAID nivel 0 usa las franjas de datos sin paridad, sin corrección de errores. Es el único nivel que no proporciona corrección de errores, o redundancia, de modo que no es considerado una forma verdadera de RAID porque no puede recuperarse de fallos de hardware. Sin embargo, ofrece el mismo beneficio importante de todos los sistemas RAID; que para el sistema operativo este grupo de dispositivos parece como una sola unidad lógica.

RAID nivel 0 está bien adaptado para transferir grandes cantidades de datos no críticos. En la configuración que se muestra en la figura 7.20, cuando el sistema operativo emite un comando de lectura para las cuatro primeras franjas, éstas pueden transferirse en paralelo, mejorando el rendimiento del sistema. El nivel 0 trabaja bien en combinación con otras configuraciones, como veremos en nuestro análisis sobre niveles READ anidados.

Nivel uno

RAID nivel 1 también usa divisiones, y se denomina configuración reflejo porque proporciona redundancia al tener un conjunto duplicado de todos los datos en un arreglo especular de discos, que actúa como sistema de respaldo en caso de fallo en el hardware. Si una unidad de disco falla, el sistema recupera de inmediato los datos de su disco de respaldo, lo cual hace muy confiable a este sistema. La figura 7.21 muestra una configuración RAID nivel 1 con tres discos en cada arreglo idéntico: principal y reflejo.

Al usar el nivel 1, cualquier disco que contenga los datos solicitados puede satisfacer las solicitudes de lectura. La elección del disco puede minimizar el tiempo de posicionamiento y la demora de rotación. La desventaja consiste en que las solicitudes de escritura requieren el doble de *esfuerzo* porque los datos deben escribirse dos veces, una vez en cada conjunto de discos. Sin embargo, esto no requiere el doble de la cantidad de *tiempo* porque ambas escrituras pueden hacerse en paralelo.

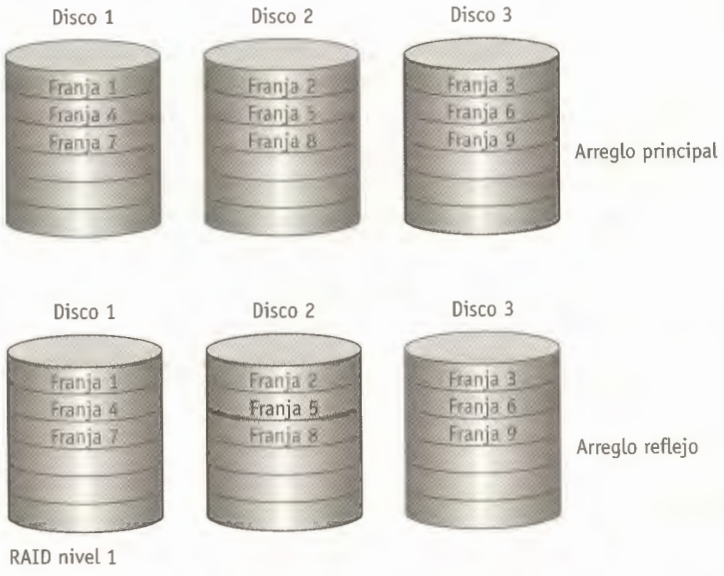
El nivel 1 es un sistema costoso de construir porque requiere por lo menos el doble de capacidad de disco que un sistema de nivel 0, pero su ventaja de confiabilidad mejorada los hace ideal para sistemas de datos críticos en tiempo real.

(Figura 7.20)

RAID nivel 0 con cuatro discos en el arreglo. Las franjas 1, 2, 3 y 4 constituyen una división. Las franjas 5, 6, 7 y 8 constituyen otra división, etcétera.

(Figura 7.21)

RAID nivel 1 con tres discos en el arreglo principal y tres correspondientes en el arreglo de respaldo, el arreglo reflejo.



Nivel dos

RAID nivel 2 usa franjas muy pequeñas (a menudo del tamaño de una palabra o byte) y usa un código Hamming para proporcionar detección y corrección de errores, o redundancia. El **código Hamming** es un algoritmo que añade bits redundantes extra a los datos, por lo que es capaz de corregir errores de un solo bit y detectar errores de bits dobles.

El nivel 2 es una configuración costosa y complicada de implementar porque el número de discos en los arreglos depende del tamaño de la franja y todas las unidades deben sincronizarse con bastante precisión tanto en el movimiento rotacional como en el posicionamiento del brazo. Por ejemplo, si cada franja es de 4 bits, entonces el código Hamming añade tres bits de paridad en las posiciones 1, 2 y 4 del elemento de reciente creación de datos de 7 bits. En el ejemplo de RAID nivel 2 mostrado en la figura 7.22, el arreglo consta de siete discos, uno para cada bit. Su ventaja es que, si un disco funciona mal sólo se vería afectado un bit, de modo que los datos podrían corregirse rápidamente.

RAID nivel 2: Código Hamming

(Figura 7.22)
RAID nivel 2. Para almacenar un artículo de datos con 7 bits se requieren siete discos en el arreglo, uno para cada bit y tres para los bits de paridad. Cada disco almacena un bit o un bit de paridad, según el código Hamming usado para redundancia.

Nivel tres

RAID nivel 3 es una modificación del nivel 2 que sólo requiere un disco para redundancia. Para cada franja sólo se calcula un bit de paridad, que se almacena en el disco de redundancia designado. La figura 7.23 muestra un arreglo de cinco discos que puede almacenar franjas de 4 bits y su bit de paridad. Si un disco funciona mal, el controlador de RAID considera que todos bits provenientes de ese disco son 0 y anota la ubicación del bit dañado. En consecuencia, si un artículo de datos que se está leyendo tiene un error de paridad, entonces el controlador sabe que el bit del disco con fallo debe haber sido 1 y lo corrige. Si los datos están escritos en un arreglo que tiene un disco que funciona mal, el controlador mantiene la paridad consistente, de modo que los datos pueden regenerarse una vez que se restituye el arreglo. El sistema vuelve a la normalidad cuando el disco con fallo se sustituye y su contenido se regenera sobre el nuevo disco.

Nivel cuatro

RAID nivel 4 usa el mismo esquema de franjas encontrado en los niveles 0 y 1, pero calcula un bit de paridad para cada franja y luego almacena estas paridades en la franja correspondiente en el disco de paridad designado. La figura 7.24 muestra un arreglo de discos de nivel 4

(Figura 7.23)

RAID nivel 3. Un artículo de datos con 4 bits se almacena en los cuatro primeros discos del arreglo. El quinto disco se usa para almacenar el bit de paridad del artículo de datos almacenado.

(Figura 7.24)

RAID nivel 4. El arreglo contiene cuatro discos: los tres primeros se usan para almacenar franjas de datos; el cuarto se usa para almacenar la paridad de esas franjas.

con 4 discos; los tres primeros contienen datos, mientras el cuarto almacena las paridades de las franjas correspondientes en los tres primeros discos. La ventaja del nivel 4 es que si cualquiera de los discos funciona mal, los datos pueden recuperarse usando los bits en el disco de paridad. La paridad se calcula cada que se ejecuta un comando de escritura. No obstante, si los datos se reescriben, entonces el controlador RAID debe actualizar los datos y las franjas de paridad. En consecuencia, al disco de paridad se tiene acceso con cada operación de escritura o reescritura, lo que algunas veces ocasiona un embotellamiento en el sistema.

Nivel cinco

RAID nivel 5 es una modificación del nivel 4. En lugar de designar un disco para almacenar paridades, distribuye las franjas de paridad en los discos, lo cual evita el embotellamiento creado en el nivel 4. Su desventaja es que la regeneración de los datos de un disco con fallo es más complicada. La figura 7.25 muestra un arreglo de discos de nivel 5 con cuatro discos.

(Figura 7.25)

RAID nivel 5 con cuatro discos. Observe cómo las franjas de paridad están distribuidas entre los discos.

Nivel seis

RAID nivel 6 fue introducido por el equipo de investigación de Berkeley en un artículo que siguió su bosquejo original de los niveles RAID. Este sistema proporciona un nivel extra de detección y corrección de errores porque requiere dos cálculos de paridad diferentes. Un cálculo es el mismo que el usado en los niveles 4 y 5; el otro es un algoritmo de verificación de datos. Ambas paridades se distribuyen en discos por separado en el arreglo, y luego se almacenan en la franja que corresponde a las franjas de datos, como se muestra en la figura 7.26. La ventaja es que la doble paridad permite la restauración de los datos incluso si fallan dos discos.

Sin embargo, la redundancia incrementa el tiempo necesario para escribir datos porque cada escritura afecta dos franjas de paridad. Además, el nivel 6 requiere que dos discos se dediquen a franjas de paridad y no a datos, lo que reduce así el número de discos para datos en el arreglo.

RAID nivel 6

(Figura 7.26)

RAID nivel 6. Observe cómo las franjas de paridad y las franjas de verificación de paridad (CD) están distribuidas entre los discos.

Niveles RAID anidados

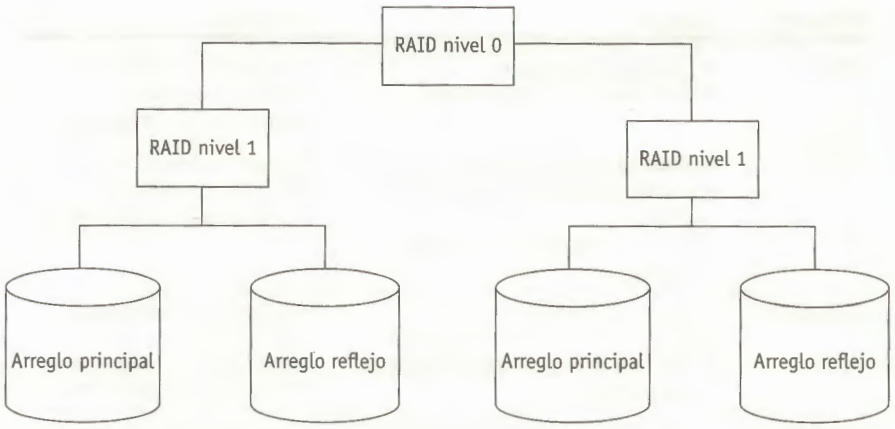
Es posible crear configuraciones complicadas adicionales de RAID al combinar múltiples niveles. Por ejemplo, un sistema RAID nivel 10 consta de un sistema nivel 1 reflejado en un segundo sistema nivel 1, ambos controlados por un único sistema nivel 1 como se muestra en la figura 7.27. En la tabla 7.8 se enumeran algunas combinaciones RAID.

Nivel RAID	Combinaciones
01 (0 0+1)	Un sistema nivel 1 que consta de múltiples sistemas nivel 0
10 (0 1+0)	Un sistema nivel 0 que consta de múltiples sistemas nivel 1
03 (0 0+3)	Un sistema nivel 3 que consta de múltiples sistemas nivel 0
30 (0 3+0)	Un sistema nivel 0 que consta de múltiples sistemas nivel 3
50 (0 5+0)	Un sistema nivel 0 que consta de múltiples sistemas nivel 5
60 (0 6+0)	Un sistema nivel 0 que consta de múltiples sistemas nivel 6

(Tabla 7.8)

Algunas configuraciones RAID anidadas. Importante: RAID 01 y RAID 03 no deben confundirse con los niveles RAID 1 y 3, respectivamente.

(Figura 7.27)
Sistema RAID nivel 10.



Los niveles RAID anidados aumentan la complejidad de la administración de discos, pero también agregan más protección contra la pérdida de datos, un beneficio que para algunos propietarios del sistema supera el aumento en la sobrecarga.

Conclusión

El trabajo del administrador de dispositivos consiste en administrar todos y cada uno de los dispositivos del sistema de la manera más eficiente posible a pesar de las características únicas de cada uno. Los dispositivos poseen velocidad y grados de compatibilidad variables; algunos pueden manejar acceso directo y algunos sólo acceso secuencial. Para medios magnéticos, pueden tener una o varias cabezas de lectura/escritura, y las cabezas pueden estar en una posición fija para velocidad óptima o poder moverse a través de la superficie para espacio de almacenamiento óptimo. Para medios ópticos, el administrador de dispositivos sigue la pista de las ubicaciones de almacenamiento y ajusta la velocidad del disco según los datos se graban y recuperan correctamente. Para memoria flash, el administrador de dispositivos sigue la pista de cada dispositivo USB y asegura que los datos se envíen y reciban correctamente.

Balancear la demanda por estos dispositivos es una tarea complicada que está dividida entre varios componentes del hardware: canales, unidades de control y los dispositivos en sí. El éxito del subsistema de E/S depende de las comunicaciones que vinculan estas partes.

En este capítulo revisamos varias estrategias de **posicionamiento**, cada una con ventajas y desventajas distintas, como se muestra en la tabla 7.9

Estrategia	Ventajas	Desventajas
FCFS	• Fácil de implementar • Suficiente para cargas ligeras	• No proporciona el mejor servicio promedio • No maximiza el resultado
SSTF	• Mejor resultado que FCFS • Tiende a minimizar el movimiento del brazo • Tiende a minimizar el tiempo de respuesta	• Puede ocasionar inanición de algunas solicitudes • Localiza bajo cargas fuertes
SCAN/LOOK	• Elimina la inanición • Resultado semejante al de SSTF • Funciona bien con cargas de ligeras a moderadas	• Requiere bit de dirección • Es el algoritmo más complicado de implementar • Sobrecarga incrementada
N-Step SCAN	• Más fácil de implementar que SCAN	• Las solicitudes más recientes esperan más que con SCAN
C-SCAN/C-LOOK	• Funciona bien con cargas de moderadas a pesadas • No hay bit de dirección • Variación pequeña en el tiempo de servicio • C-LOOK no se desplaza a pistas sin usar	• Puede no ser justo con solicitudes recientes para pistas con numeración alta • Algoritmo más complicado que N-step SCAN, lo que ocasiona más sobrecarga

(Tabla 7.9)
Comparación de las estrategias de búsqueda DASD analizadas en este capítulo.

Nuestro análisis de RAID incluyó una comparación de los puntos fuertes y los puntos débiles de cada nivel, y combinaciones de niveles, así como el arranque potencial para confiabilidad del sistema y corrección de errores que cada uno representa.

Hasta ahora en este texto hemos revisado tres de los administradores del sistema operativo: el administrador de la memoria, el administrador del procesador y el administrador de dispositivos. En el capítulo siguiente abordaremos el cuarto: el administrador de archivos, que es responsable de la salud y el bienestar de todos y cada uno de los archivos usados por el sistema, incluyendo los archivos del sistema, los que presentan los usuarios y los generados como resultado.

Términos clave

Acceso directo a la memoria (DMA): Técnica de E/S que permite que una unidad de control tenga acceso a la memoria principal directamente y transfiera datos sin intervención de la CPU.

Almacenamiento: Sitio del sistema de cómputo en que se almacenan los datos. El almacenamiento secundario es un medio no volátil, como los discos y la memoria flash. El almacenamiento primario es la memoria principal.

Bit de paridad: Bit extra agregado a un caracter, palabra o alguna otra unidad de datos que se usa para verificación de errores. Se habilita en 0 o 1 de modo que la suma de los bits uno en la unidad de datos siempre sea par, para paridad par, o impar para paridad impar, según la lógica del sistema.

Bloqueo: Técnica para ahorrar almacenamiento y E/S que agrupa registros individuales en un bloque que se almacena y recupera como una unidad.

Brecha entre registros (IRG): Espacio sin usar entre los registros de una cinta magnética. Facilita las operaciones de inicio/detención de la cinta.

***Búfers*:** Áreas de almacenamiento temporal que residen en la memoria principal, canales y unidades de control.

Búsqueda: Algoritmo usado para optimizar el tiempo de búsqueda en DASD.

Canal de E/S: Unidad especial programable ubicada entre la CPU y las unidades de control, que sincroniza la gran velocidad de la CPU con la lenta velocidad del dispositivo de E/S y viceversa, haciendo posible superponer operaciones de E/S con operaciones de la CPU.

Cilindro: Concepto que describe un tubo virtual que se forma cuando dos o más cabezas de lectura/escritura se posicionan sobre la misma pista, en la misma posición relativa, sobre sus superficies respectivas.

C-LOOK: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que es una optimización de C-SCAN.

Código Hamming: Código para detectar y corregir errores que mejora bastante la confiabilidad de los datos. Se llama así en honor del matemático Richard Hamming, su inventor.

Controlador de tráfico de E/S: Uno de los módulos del subsistema de E/S que supervisa el estado de cada dispositivo, unidad de control y canal.

Controlador del bus universal en serie (USB): Interfaz entre el sistema operativo, los controladores de dispositivos y las aplicaciones, que lee y escribe desde y hacia dispositivos conectados a la computadora mediante el puerto USB. Cada puerto USB puede alojar hasta 127 dispositivos diferentes.

C-SCAN: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de posicionamiento. Es una abreviatura de circular-SCAN.

Depresiones: Pequeñas depresiones sobre la capa reflectora de un disco óptico. Cada depresión se interpreta como un 0. Contrasta con las salientes, que se interpretan como unos.

Dispositivo compartido: Dispositivo que puede asignarse a varios procesos activos al mismo tiempo.

Dispositivo de almacenamiento con acceso directo (DASD): Cualquier dispositivo de almacenamiento secundario que puede leerse o escribirse directamente desde o hacia un sitio específico. Algunas veces se denomina dispositivo de almacenamiento con acceso aleatorio.

Dispositivo dedicado: Dispositivo que puede asignarse sólo a un trabajo a la vez; sirve a ese trabajo durante todo el tiempo que el trabajo está activo.

Dispositivo virtual: Dispositivo dedicado que ha sido transformado en un dispositivo compartido por medio de técnicas de *spooling*.

División: Conjunto de franjas consecutivas a través de discos; las divisiones contienen bits de datos y algunas veces bits de paridad, dependiendo del nivel RAID.

Estrategia de posicionamiento: Política predeterminada usada por el manejador de dispositivos de E/S para optimizar los tiempos de posicionamiento.

Interrupción: Señal del hardware que suspende la ejecución de un programa y activa la ejecución de un programa especial denominado manejador de interrupciones.

LOOK: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de posicionamiento. Algunas veces se denomina algoritmo del elevador.

Manejador de dispositivos de E/S: Módulo que procesa las interrupciones de E/S, maneja las condiciones de error y proporciona algoritmos de planificación detallados que son extremadamente dependientes del dispositivo.

Medio con acceso secuencial: Cualquier medio que almacene registros de manera secuencial, uno después de otro, como una cinta magnética.

Memoria flash: Tipo de memoria no volátil usada como dispositivo de almacenamiento secundario que puede borrarse y reprogramarse en bloques de datos.

N-step SCAN: Variante de la estrategia de planificación SCAN para dispositivos de almacenamiento con acceso directo que se usa para optimizar los tiempos de posicionamiento.

Ordenamiento de rotación: Algoritmo usado para reordenar solicitudes por registros dentro de pistas para optimizar el tiempo de búsqueda.

Palabra de estado del canal: Estructura de datos que contiene información que indica la condición del canal, incluyendo tres bits para los tres componentes del subsistema de E/S: uno para el canal, otro para la unidad de control y otro para el dispositivo.

Pista: Ruta en un medio de almacenamiento a lo largo de la cual se graban datos.

Planificador de E/S: Uno de los módulos del subsistema de E/S que asigna los dispositivos, las unidades de control y los canales.

Primero en llegar, primero en servirse (FCFS): El algoritmo de planificación más simple para dispositivos de almacenamiento con acceso directo que satisface las solicitudes por pistas en el orden en que se reciben.

Primero el tiempo de posicionamiento más corto (SSTF): Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de posicionamiento. Las solicitudes por pistas se ordenan de modo que la más próxima a la pista activa actual se satisface primero y las más alejadas se hacen esperar.

Programa del canal de E/S: Programa que controla los canales.

RAID: Acrónimo de conjunto redundante de discos independientes; grupo de discos duros controlados de modo que aceleran el acceso a dispositivos de almacenamiento secundario y ayudan a la recuperación de datos.

Salientes: Áreas de superficie planas sobre la capa reflejante de un disco óptico. Cada depresión se interpreta como un 1. Contrasta con las salientes, que se interpretan como un 0.

SCAN: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de **posicionamiento**. Las variantes más comunes son N-step SCAN y C-SCAN.

Subsistema de E/S: Colección de módulos dentro del sistema operativo que controla todas las solicitudes de E/S.

Tasa de transferencia: Tasa a la que se transfieren los datos desde medios con acceso secuencial.

Tiempo de posicionamiento: Tiempo necesario para posicionar la cabeza de lectura/escritura sobre la pista idónea a partir del momento en que se emite la solicitud de E/S.

Tiempo de acceso: Tiempo total requerido para tener acceso a datos en almacenamiento secundario.

Tiempo de búsqueda: Tiempo necesario para girar el disco desde el momento en que se emite un comando de E/S hasta que el registro solicitado se mueve bajo la cabeza de lectura/escritura. También se denomina demora de rotación.

Tiempo de transferencia: Tiempo necesario para la transferencia de datos entre almacenamiento secundario y la memoria principal.

Unidad de control de E/S: Unidad de hardware que contiene los componentes electrónicos comunes a un tipo de dispositivo de E/S, como una unidad de disco.

Unidad de disco magneto-óptico: Unidad de almacenamiento de datos que usa un haz láser para leer y/o escribir información grabada sobre discos magneto-ópticos.

Búsquedas de interés

- Unidad de disco duro de estado sólido
- Búfers circulares
- Memoria de acceso directo
- Nivel de desempeño RAID
- Medio de almacenamiento Blu-ray

Ejercicios

Temas de investigación

A. Lleve a cabo una investigación para descubrir dos de los dispositivos más rápidos disponibles actualmente para computadoras de escritorio o estaciones de trabajo que puedan usarse para copiar archivos importantes desde la computadora hacia un medio (papel, disco, memoria flash, cinta, etc.), y que puedan removerse y almacenarse en un lugar seguro fuera de sitio. Para ambos dispositivos, explique en un párrafo corto por qué considera que estos dos dispositivos son los mejores que hay. Cite sus fuentes y agregue la documentación de su investigación para sustentar su respuesta.

B. Consulte literatura académica actual para investigar el concepto de *búfer* circular. Defínalo y compárelo con un *búfer* único y con *búfers* dobles. Explique por qué el *búfer* circular puede ser ventajoso en aplicaciones multimedia. Cite sus fuentes y las fechas en que fueron publicadas.

Ejercicios

1. Mencione tres ejemplos de medios de almacenamiento secundario además de los discos duros.
2. Describa la deferencia entre almacenamiento primario y almacenamiento secundario.
3. Explique las diferencias entre almacenamiento temporal y bloqueo.
4. Dadas las siguientes características de un paquete de discos con 10 platos que proporcionan 18 superficies grabables:

   Velocidad de rotación = 10 ms

   Tasa de transferencia = 0.1 ms/pista

   Densidad por pista = 19 000 bytes

   Número de registros a almacenar = 200 000 registros

   Tamaño de cada registro = 160 bytes

   Tamaño del bloque = 10 registros lógicos

   Número de pistas por superficie = 500

   Encuentre lo siguiente:

   a. Número de bloques por pista.
   b. Desperdicio por pista.
   c. Número total de pistas necesarias para almacenar todo el archivo.
   d. Desperdicio total para almacenar todo el archivo.
   e. Tiempo para escribir todos los bloques (use velocidad de rotación; ignore el tiempo necesario para ir a la pista siguiente).
   f. Tiempo para escribir todos los registros si no están bloqueados (use velocidad de rotación; ignore el tiempo necesario para ir a la pista siguiente).
   g. El factor de bloqueo óptimo para minimizar el desperdicio.
   h. ¿Cuáles serían las respuestas al inciso *e*) si el tiempo para ir a la pista siguiente es de 5 ms?
   i. ¿Cuál sería la respuesta al inciso *f*) si el tiempo necesario para ir a la pista siguiente es de 5 ms?

5. Dadas las siguientes características de una cinta magnética:

   Densidad = 1 600 bpi

   Velocidad = 200 pulgadas/segundo

   Tamaño = 2 400 pies

   Tiempo de inicio/detención = 3 ms

Número de registros a almacenar = 200 000 registros

Tamaño de cada registro = 160 bytes

Tamaño del bloque = 10 registros lógicos

IBG = 0.5 pulg

Encuentre lo siguiente:

a. Número de bloques necesarios.

b. Tamaño de los bloques en bytes.

c. Tiempo necesario para leer un bloque.

d. Tiempo necesario para escribir todos los bloques.

e. Cantidad de cinta usada sólo para datos, en pulgadas.

f. Cantidad total de cinta usada (datos + IBGs), en pulgadas.

6. Dado que se requiere 1 ms para ir de una pista a la siguiente, y que el brazo está posicionado originalmente en la pista 15 moviéndose hacia las pistas con baja numeración y usted está usando una política de planificación LOOK, calcule cuánto tiempo se requiere para satisfacer las siguientes solicitudes: 4, 40, 35, 11, 14 y 7. Todas las solicitudes están en la cola de espera. (Ignore el tiempo de rotación y el tiempo de transferencia; considere sólo el tiempo de acceso.) ¿Cómo se comparan sus resultados con los de la figura 7.17? ¿Por qué cree que hay una diferencia entre ambos?

7. Los sistemas interactivos deben responder con prontitud a los usuarios. Un objetivo importante es minimizar la variación del tiempo de respuesta, aunque no siempre previene que un usuario ocasional sufra aplazamiento indefinido. ¿Qué mecanismos incorporaría usted en una política de planificación de discos para contrarrestar este problema a la vez que proporciona un tiempo de respuesta razonable a la población de usuarios como un todo?

Ejercicios avanzados

8. ¿Cuál es la diferencia entre almacenamiento y almacenamiento temporal?

9. En condiciones de carga ligeras, cada una de las políticas de planificación de discos analizada en este capítulo tiende a comportarse como una de las políticas analizadas. ¿Cuál es ésta y por qué?

10. Suponga que usted tiene un archivo de 10 registros (identificados como A, B, . . ., J) que debe almacenar en un disco que contiene 10 registros por pista. Una vez que ha almacenado el archivo, el acceso a los registros es secuencial: A, B, . . ., J. Para procesar cada registro se requieren 2 ms una vez que ha sido transferido hacia la memoria. Se requieren 10 ms para que el disco complete una rotación, y 1 ms para transferir el registro desde el disco hasta la memoria principal. Suponga que almacena los registros en el orden dado: A, B, . . ., J. Calcule cuánto tiempo se requiere para procesar todos los 10 registros. Descomponga su cálculo en: 1) tiempo para transferir un registro; 2) tiempo para procesar un registro, y 3) tiempo para tener acceso al siguiente registro.

11. Dada la misma situación que en el ejercicio previo:
    a. Organice los registros de modo que estén almacenados en desorden alfabético (no A, B, C,. . ., J) a fin de reducir el tiempo necesario para procesarlos secuencialmente en orden alfabético.
    b. Calcule cuánto tiempo se requiere para procesar todos los 10 registros usando este nuevo orden. Descomponga su cálculo en: 1) tiempo para transferir un registro; 2) tiempo para procesar un registro, y 3) tiempo para tener acceso al siguiente registro.
12. Las solicitudes por pistas no suelen estar distribuidas de manera equitativa o uniforme. Por ejemplo, el acceso a las pistas donde reside el directorio del disco ocurre más a menudo que donde se encuentran los archivos del usuario. Suponga que usted sabe que 50% de las solicitudes fueron por un número pequeño y fijo de cilindros.
    a. ¿Cuál de las políticas de planificación presentadas en este capítulo sería la mejor a utilizar en estas condiciones?
    b. ¿Puede diseñar una que pudiera ser mejor?
13. Complete la siguiente tabla para tres dispositivos de almacenamiento óptico. Cite sus fuentes.

Tipo	Velocidad (revoluciones por minuto)	Capacidad de almacenamiento	Tiempo de acceso promedio	Costo en dólares
CD-RW				
DVD-RW				
Blu-ray				

14. Proporcione un ejemplo de un ambiente o aplicación que coincidan mejor con las características de cada uno de los niveles RAID siguientes:
    a. Nivel 0
    b. Nivel 1
    c. Nivel 3
    d. Nivel 5
    e. Nivel 6

Ejercicio de programación

15. Escriba un programa que simule las estrategias de optimización de búsqueda. Suponga lo siguiente:
    a. La pista exterior del disco es la pista 0 y el disco contiene 200 pistas por superficie. Cada pista contiene ocho sectores numerados de 0 a 7.
    b. Una búsqueda requiere 10 + 0.1 * T ms, donde T es el número de pistas de movimiento desde una solicitud a otra, y 10 es una constante de tiempo de movimiento.
    c. Una rotación completa se hace en 8 ms.
    d. El tiempo de transferencia es 1 ms.

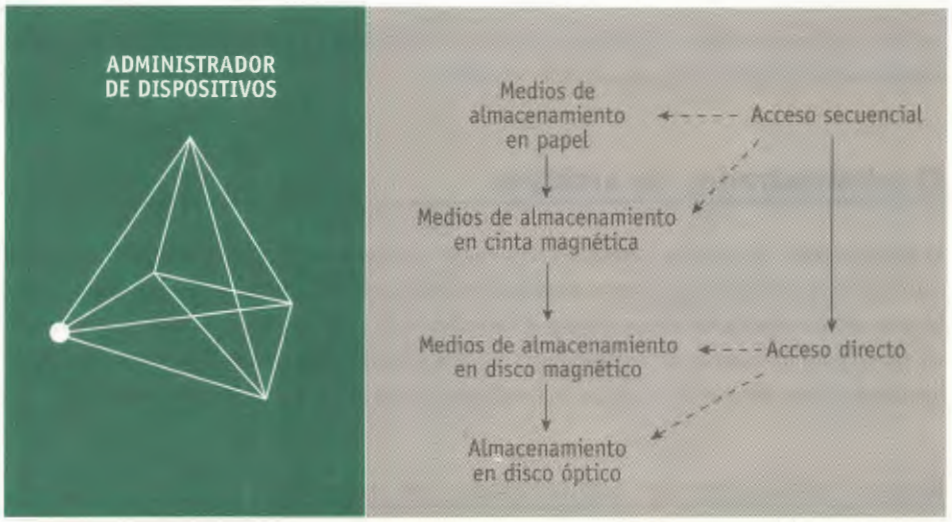
Utilice los siguientes datos para probar su programa:

Hora de llegada	Pista solicitada	Sector solicitado
0	45	0
23	132	6
25	20	2
29	23	1
35	198	7
45	170	5
57	180	3
83	78	4
88	73	5
95	150	7

Para efectos de comparación, calcular el promedio, variación y desviación estándar del tiempo necesario para acomodar toda solicitud de cada una de las estrategias. Consolidar sus resultados en una tabla.

Opcional: ejecutar su programa de nuevo con diferentes datos y comparar sus resultados. Recomendar la mejor política y explicar por qué.

Capítulo 8 Administración de archivos



"El conocimiento es de dos tipos. Sabemos sobre un tema o sabemos dónde encontrar información al respecto."

Samuel Johnson (*1709-1784*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Los fundamentos de la administración de archivos y la estructura del sistema de administración de archivos.
- Los convencionalismos para nombrar archivos, incluyendo el rol de las extensiones.
- La diferencia entre formato de registros de longitud fija y variable.
- Las ventajas y desventajas de las técnicas de almacenamiento contiguo, no contiguo e indexado de archivos.
- Comparaciones entre acceso secuencial y directo a archivos.
- Técnicas de control de acceso y cómo se comparan.
- El rol de la compresión de datos en el almacenamiento de archivos.

El administrador de archivos controla todos los archivos en el sistema. En este capítulo aprenderemos la forma en que los archivos están organizados lógicamente, cómo están almacenados físicamente, cómo se tiene acceso a ellos y a quién se permite tener acceso. También estudiaremos la interacción entre el administrador de archivos y el administrador de dispositivos.

La eficiencia del administrador de archivos depende de cómo están organizados los archivos del sistema (secuenciales, directos o indexados secuenciales); cómo están almacenados (contiguamente, no contiguamente, o indexados); cómo está estructurado el registro de cada archivo (de longitud fija o variable), y cómo se controla el acceso a estos archivos. En este capítulo abordaremos cada una de estas variables.

El administrador de archivos

El administrador de archivos (también denominado sistema de administración de archivos) es el software responsable de crear, borrar, modificar y controlar el acceso a los archivos, así como también de administrar los recursos usados por los archivos. El administrador de archivos proporciona apoyo para bibliotecas de los programas y datos para usuarios en línea, y para computación interactiva. Estas funciones las realiza en colaboración con el administrador de dispositivos.

Responsabilidades del administrador de archivos

El administrador de archivos tiene un trabajo complicado. Está a cargo de los componentes físicos del sistema, sus recursos de información y las políticas usadas para almacenar y distribuir los archivos. Para llevar a cabo sus responsabilidades, debe realizar estas cuatro funciones:

1. Seguir la pista del lugar en que está almacenado cada archivo.
2. Usar una política que determine dónde y cómo han de almacenarse los archivos, asegurándose de usar eficientemente el espacio de almacenamiento disponible y proporcionar acceso eficiente a los archivos.
3. Asignar cada archivo cuando a un usuario se le ha permitido el acceso al archivo y luego registrar su uso.
4. Liberar el archivo cuando debe devolverse a almacenamiento y comunicar su disponibilidad a otros que pueden estar esperándolo.

Por ejemplo, el sistema de archivos es como una biblioteca, donde el administrador de archivos desempeña el papel del bibliotecario que lleva a cabo las mismas cuatro tareas:

1. Un bibliotecario usa el catálogo para seguir la pista de cada artículo en la colección; cada entrada enumera el número de llamada y los detalles que ayudan a los lectores a encontrar el libro que buscan.
2. El bibliotecario depende de una política para almacenar todos los artículos en la colección, incluyendo los libros de gran tamaño, revistas, libros en cinta, DVD, mapas y videos. Y es necesario ordenarlos físicamente de modo que sea posible encontrarlos cuando se requieran.

3. Cuando se solicita, el libro se extrae de su librero y en los registros de préstamo se escribe el nombre del prestatario.
4. Cuando se devuelve el libro, el bibliotecario hace la anotación correspondiente en los registros de préstamo y vuelve a colocarlo en su sitio.

En un sistema de cómputo, el administrador de archivos sigue la pista de sus archivos con directorios que contienen el nombre del archivo, su ubicación física en almacenamiento secundario e información importante sobre cada archivo.

La política del administrador de archivos determina dónde se almacena cada archivo, cómo el sistema y sus usuarios; puede tener fácil acceso a los archivos: mediante comandos que son independientes de los detalles del dispositivo. Además, la política debe determinar quién tendrá acceso a qué material, lo cual implica dos factores: flexibilidad de acceso a la información y protección subsecuente. El administrador de archivos hace lo anterior al permitir acceso a archivos compartidos, proporcionando acceso distribuido y permitiendo que los usuarios naveguen a través de directorios públicos. Mientras tanto, el sistema operativo debe proteger sus archivos contra malfuncionamientos del sistema y proporcionar verificaciones de seguridad mediante números de cuenta, contraseñas y palabras de bloqueo para preservar la integridad de los datos y salvaguardarlos contra manipulaciones. Estas técnicas de protección se explicarán más tarde en este capítulo.

El sistema de cómputo *asigna* un archivo al activar el dispositivo idóneo de almacenamiento secundario y cargar el archivo en la memoria a la vez que actualiza sus registros de quién está usando qué archivo.

Finalmente, el administrador de archivos *desasigna* o libera un archivo al actualizar las tablas del archivo y volviendo a escribir el archivo (si se ha revisado) en el dispositivo de almacenamiento secundario. Cualquier proceso que esté esperando tener acceso a ese archivo es notificado entonces de su disponibilidad.

Definiciones

Antes de continuar, tomaremos un minuto para definir algunos elementos básicos de los archivos, ilustrados en la figura 8.1, que se relacionan con nuestro análisis del administrador de archivos.

(Figura 8.1)
Los archivos constan de registros. Los registros constan de campos.

Registro 19 →	Campo A	Campo B	Campo C	Campo D
Registro 20 →	Campo A	Campo B	Campo C	Campo D
Registro 21 →	Campo A	Campo B	Campo C	Campo D
Registro 22 →	Campo A	Campo B	Campo C	Campo D

Un **campo** es un grupo de bytes relacionados que el usuario puede identificar mediante un nombre, tipo y tamaño. Un **registro** es un grupo de campos relacionados.

Un **archivo** es un grupo de registros relacionados que contienen información a usar por programas de aplicación específicos para generar reportes. El tipo de archivo contiene datos y algunas veces se denomina archivo plano porque no tiene ninguna conexión con otros archivos; a diferencia de las bases de datos, carece de dimensionamiento.

Una **base de datos** aparece con el administrador de archivos como un tipo de archivo, pero las bases de datos son más complicadas, porque en realidad son grupos de archivos relacionados que están interconectados a varios niveles a fin de proporcionar a los usuarios flexibilidad de acceso a los datos almacenados. Si la base de datos del usuario requiere una estructura específica, el administrador de archivos debe ser capaz de proporcionarla.

Los **archivos del programa** contienen instrucciones y los archivos de datos contienen datos, pero en cuanto al almacenamiento concierne, el administrador de archivos los trata exactamente de la misma forma.

Los **directorios** son archivos especiales con listados de los nombres de los archivos y sus atributos. Los datos recolectados para supervisar el rendimiento del sistema y proporcionar su contabilidad, se reúnen en archivos. De hecho, todo programa y archivo de datos a los que tiene acceso el sistema de cómputo, así como toda pieza del software de la computadora, se tratan como un archivo.

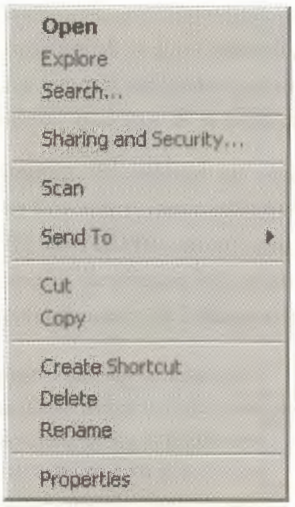
Interacción con el administrador de archivos

El usuario se comunica con el administrador de archivos, el cual responde a comandos específicos.

Algunos ejemplos de comandos interactivos, mostrados en la figura 8.2, son OPEN, DELETE, RENAME y COPY. En realidad, los archivos pueden crearse con otros términos específicos del sistema; por ejemplo, la primera vez que un usuario da un comando para guardar un archivo, en realidad se trata del comando CREATE. En otros sistemas operativos, el comando OPEN FILE dentro de un programa indica al administrador de archivos que es necesario crear un archivo.

Estos comandos y muchos más fueron diseñados para ser muy fáciles de usar, así que están desprovistos de las instrucciones detalladas para correr el dispositivo (información en el dispositivo) donde el archivo puede ser almacenado. Es decir, éstos son dispositivos independientes. Por tanto, para acceder a un archivo, el usuario no necesita conocer su ubicación física en el paquete de discos (el cilindro, superficie, y sector), el medio en el sean almacenadas (cinta de archivo, disco magnético, disco óptico, o memoria flash), o la red específicos. Eso está bien porque el acceso a archivos es un proceso complicado. Cada comando lógico se descompone en una secuencia de señales de bajo nivel que accionan las acciones paso a paso realizadas por el dispositivo y supervisan el avance de la operación al probar el estado

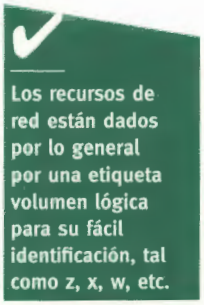
(Figura 8.2)
Menú típico de opciones de archivos.



del dispositivo. Por ejemplo, cuando el programa de un usuario emite un comando para leer el registro de un disco, la instrucción READ debe descomponerse en los pasos siguientes:

1. Mover las cabezas de lectura/escritura hacia el cilindro o pista donde debe estar el registro.
2. Esperar por la demora de rotación hasta que el sector que contiene el registro deseado pasa bajo la cabeza de lectura/escritura.
3. Activar la cabeza de lectura/escritura idónea y lee el registro.
4. Transferir el registro a la memoria principal.
5. Enviar una bandera para indicar que el dispositivo está libre para satisfacer otra solicitud.

Mientras se lleva a cabo todo lo anterior, el sistema debe verificar condiciones de posibles errores. El administrador de archivos efectúa todo esto, liberando a usuario de incluir en cada programa las instrucciones de nivel bajo para cada dispositivo a usar: la terminal, el teclado, la impresora, el CD, la unidad de disco, etc. Sin el administrador de archivos, cada programa requeriría incluir instrucciones para operar todos los diferentes tipos de dispositivos y todos los diferentes modelos dentro de cada tipo. Al considerar el rápido desarrollo y la sofisticación creciente de los dispositivos de E/S, sería impráctico, y ciertamente no muy amigable para el usuario, solicitar que cada programa incluya estos detalles operacionales mínimos. Esta es la ventaja de la independencia de los dispositivos.



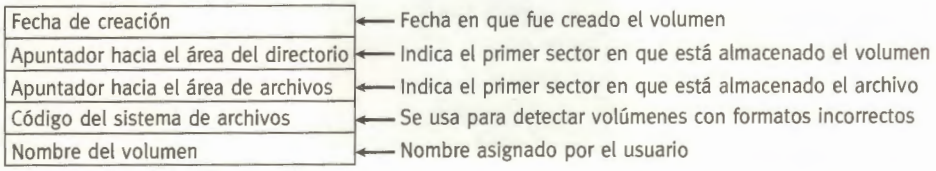
Los recursos de red están dados por lo general por una etiqueta volumen lógica para su fácil identificación, tal como z, x, w, etc.

Configuración de volumen típica

Normalmente, los archivos activos de una computadora residen en unidades de almacenamiento secundario. Algunos dispositivos acomodan unidades de almacenamiento removibles –como CD, DVD, disquetes, dispositivos USB y otros medios removibles– de modo que los archivos que no se usan a menudo pueden almacenarse fuera de línea y montarse sólo cuando el usuario los solicita específicamente. Otros dispositivos presentan unidades de almacenamiento integradas, como discos duros y paquetes de discos no removibles.

Cada unidad de almacenamiento, removible o no, se considera un **volumen**, y cada volumen puede contener varios archivos, de modo que se denominan "volúmenes multiarchivos". Sin embargo, algunos archivos son extremadamente largos y están contenidos en varios volúmenes; de manera no sorprendente, éstos se denominan "archivos multivolúmenes".

Cada volumen en el sistema tiene un nombre. El administrador de archivos escribe este nombre y otra información descriptiva, como se muestra en la figura 8.3, en un sitio de fácil acceso en cada unidad; la parte más interna del CD o del DVD, el principio de una cinta, o el primer sector de la pista más externa del paquete de discos. Una vez identificado, el sistema operativo puede interactuar con la unidad de almacenamiento.



(Figura 8.3)
El descriptor del volumen, almacenado al principio de cada volumen, incluye el nombre del volumen y otra información vital sobre la unidad de almacenamiento.

El **directorio maestro de archivos (MFD)** se almacena inmediatamente después del descriptor de volumen y enumera los nombres y las características de todos los archivos contenidos en ese volumen. Los nombres de los archivos en el MFD pueden referirse a archivos del programa, archivos de datos y/o archivos del sistema. Y si el administrador de archivos atiende subdirectorios, también aparecen enumerados en el MFD. El resto del volumen se usa para almacenamiento de archivos.

Los primeros sistemas operativos atendían un solo directorio por volumen. Este directorio era creado por el administrador de archivos y contenía los nombres de los archivos, usualmente organizados en orden alfabético, espacial o cronológico. Aunque resultaba fácil de implementar y mantener, este esquema presentaba algunas desventajas importantes.

- Se requería mucho tiempo para buscar un archivo individual, específicamente si el MFD estaba organizado en orden arbitrario.
- Si el usuario tenía más de 256 pequeños archivos almacenados en el volumen, el espacio del directorio (con límite de 256 archivos), podía llenarse antes que el espacio de almacenamiento del disco. Entonces, el usuario recibía un mensaje de "disco lleno" cuando sólo estaba lleno el directorio.
- Los usuarios sólo podían crear subdirectorios para agrupar los archivos que estaban relacionados.
- No era posible que usuarios múltiples resguardaran sus propios archivos de otros usuarios, porque todo el directorio estaba libremente a disposición de cualquier usuario en el grupo solicitado.
- Cada programa en todo el directorio requería un nombre único, incluso los directorios que daban servicio a muchos usuarios, de modo que sólo una persona que estuviera usando ese directorio podía tener un programa denominado PROG1.

Esto podía producir estragos en un curso introductorio de informática. Por ejemplo, ¿qué ocurría si la primera persona denominaba PROG1 a la primera asignación de programación? Entonces, el resto de la clase podía tener opciones interesantes: escribir un nuevo

programa y nombrarlo de otra forma; escribir un programa nuevo y llamarlo PROG1 (con lo cual se borraría la versión original); o simplemente abrir PROG1, modificarlo y luego guardarlo con los cambios realizados. Con la última opción, toda la clase podía terminar con un programa único, tal vez terrorífico.

Acerca de los subdirectorios

Administradores de archivos más recientes crearon un MFD para cada volumen que contiene entradas para archivos y para subdirectorios. Un **subdirectorio** se crea cuando un usuario abre una cuenta para tener acceso al sistema de cómputo. Aunque este directorio del usuario se trata como un archivo, su entrada en el MFD está señalada para indicar al administrador de archivos que este archivo es en realidad un subdirectorio y que posee propiedades únicas: de hecho, sus registros son nombres de archivos que apuntan a archivos.

Aunque ésta era una mejora con respecto al esquema de directorio único (hoy todos los estudiantes pueden nombrar PROG1 a su primer programa), no resuelve los problemas que encuentran usuarios prolíficos que desean agrupar sus archivos en orden lógico para mejorar la accesibilidad y eficiencia del sistema.

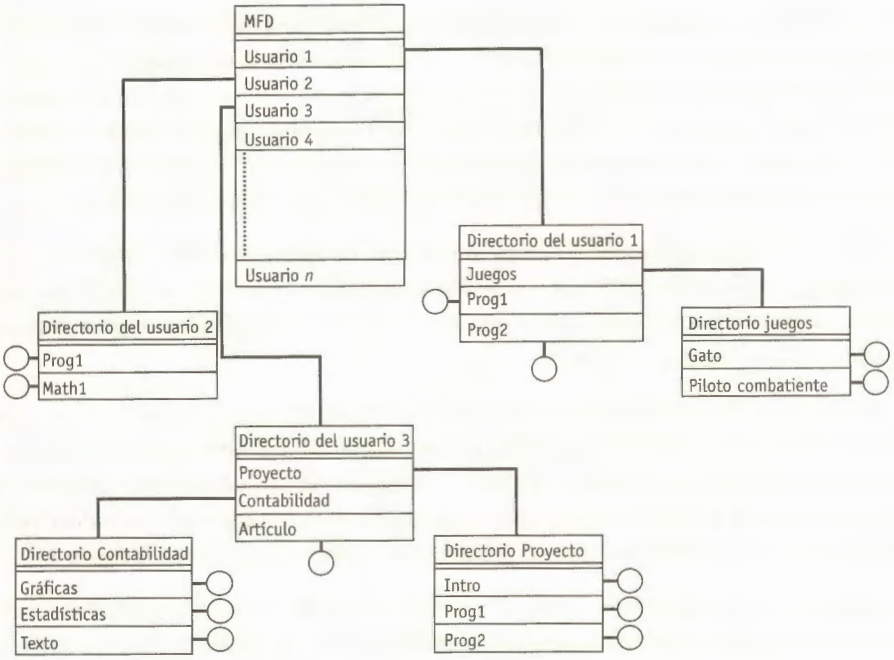
Los sofisticados administradores de archivos actuales permiten que el usuario elabore sus propios subdirectorios, de modo que los archivos relacionados pueden agruparse entre sí. Muchos usuarios de computadoras y algunos sistemas operativos denominan "carpetas" a estos subdirectorios. Esta estructura es una extensión de la organización previa de directorio de dos niveles, y se implementa como un árbol al revés, como se muestra en la figura 8.4.

Las estructuras de árbol permiten que el sistema busque de manera eficiente directorios individuales, ya que en cada directorio hay menos entradas. Sin embargo, la ruta hacia el archivo solicitado puede llevar a través de varios directorios. Para toda solicitud por algún archivo, el MFD es el punto de entrada. En realidad, el MFD suele ser transparente para el usuario; sólo es accesible para el sistema operativo. Cuando el usuario desea tener acceso a un archivo específico, el nombre del archivo se envía al administrador de archivos. Éste busca el directorio del usuario en el MFD y luego en el directorio del usuario y en todos los subdirectorios busca el archivo solicitado y su ubicación.

Sin importar la complejidad de la estructura del directorio, cada entrada de archivo en cualquier directorio contiene información que describe el archivo; se denomina **descriptor del archivo**. La información que suele incluirse en un descriptor del archivo incluye lo siguiente:

- Nombre del archivo: dentro de un directorio, los nombres de los archivos deben ser únicos. En algunos sistemas operativos, los nombres llevan mayúsculas y minúsculas.
- Tipo de archivo: la organización y el uso que son dependientes del sistema (por ejemplo, archivos y directorios).
- Tamaño del archivo: aunque es posible calcularlo a partir de otra información, el tamaño se guarda aquí por conveniencia.
- Ubicación del archivo: identificación del primer bloque físico (o de todos los bloques) donde está almacenado el archivo.

- Fecha y hora de creación.
- Propietario.
- Información de protección: restricciones de acceso, con base en a quién se permite tener acceso al archivo y el tipo de acceso permitido.
- Tamaño del archivo: su tamaño fijo o su tamaño máximo, dependiendo del tipo de registro.



(Figura 8.4)

Estructura de árbol del directorio de archivos. La "raíz" es el MFD que se muestra en la parte superior; cada nodo es un archivo del directorio y cada rama es una entrada del directorio que apunta a otro directorio o a un archivo real. Todos los programas y archivos de datos agregados más tarde al árbol son las hojas, representadas por círculos.

Convenciones de nomenclatura de archivos

El nombre de un archivo puede ser más largo de lo que cree el usuario. Dependiendo del nivel de sofisticación del administrador de archivos, el nombre puede tener desde dos hasta muchas componentes. Las dos componentes comunes a muchos nombres de archivos son el **nombre relativo del archivo** y una **extensión**.

Para evitar confusiones, en el siguiente análisis usaremos los términos "nombre completo del archivo" para identificar el **nombre absoluto del archivo** (es el nombre largo que incluye toda la información de ruta), y "nombre relativo" para indicar el nombre sin la información de ruta que aparece en listados de directorios y carpetas.

El nombre relativo del archivo es el nombre que lo diferencia de otros archivos en el mismo directorio. Algunos ejemplos pueden incluir DEPARTMENT ADDRESSES, TAXES_PHOTOG, o AUTOEXEC. En general, el nombre relativo puede variar de longitud desde uno hasta muchos caracteres y puede incluir letras, así como dígitos. Sin embargo, todo sistema operativo posee reglas específicas que afectan la longitud del nombre relativo y los tipos de caracteres permitidos. La mayor parte de los sistemas operativos permiten nombres con

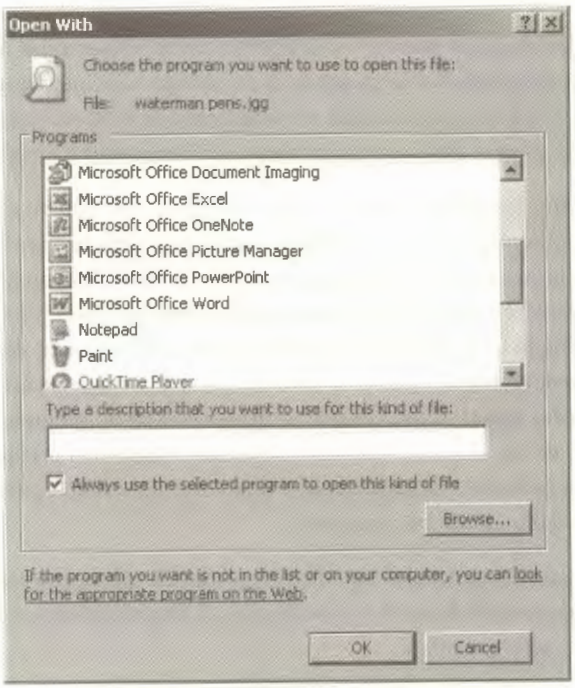
El límite de caracteres para el nombre de un archivo (256 para el caso de algunos sistemas operativos de Windows) puede aplicarse a la ruta entera y no sólo al nombre del archivo relativo.

docenas de caracteres incluyendo espacios, guiones, subrayados y algunos caracteres del teclado.

Algunos sistemas operativos requieren una extensión que se agrega al nombre relativo del archivo. Suele ser de dos o tres caracteres de largo y está separado del nombre del archivo por un punto, y su propósito es identificar el tipo de archivo o su contenido. Por ejemplo, en un sistema operativo Windows, un nombre relativo de archivo típico con extensión sería BASIA_TUNE.MP3. En forma semejante, TAKE OUT MENU.RTF y TAKE OUT MENU. DOC indican que pueden ser abiertos con una aplicación de procesador de palabras. ¿Qué ocurre si una extensión es incorrecta o desconocida? La mayor parte solicita orientación de parte del usuario, como se muestra en la figura 8.5.

(Figura 8.5)

Para abrir un archivo con una extensión no reconocida, Windows pide al usuario que escoja una aplicación asociada con ese tipo de archivo.



Algunas extensiones (como EXE, BAT, COB y FOR) están restringidas por algunos sistemas operativos, ya que constituyen un conjunto estándar. Es decir, funcionan como una señal para que el sistema use un compilador o programa especial para ejecutar estos archivos.

Dependiendo del sistema operativo, puede haber otros componentes requeridos para el nombre completo de un archivo. He aquí cómo tres sistemas operativos diferentes identifican un archivo denominado INVENTORY_COST.DOC:

1. Usando un sistema operativo Windows y una computadora personal con tres unidades de disco, el nombre completo del archivo está compuesto por su nombre relativo y una extensión, precedida por la etiqueta de la unidad de disco y el nombre del directorio:

```
C:\IMFST\FLYNN\INVENTORY_COST.DOC
```

Esto indica al sistema que el archivo INVENTORY_COST.DOC requiere un programa de aplicación de un procesador de palabras y que puede encontrarse en el directorio; IMFST; subdirectorio FLYNN en el volumen que reside en la unidad C.

2. Un sistema UNIX o Linux podría identificar este archivo como:

/usr/imfst/flynn/inventory_cost.doc

La primera entrada está representada por la diagonal hacia delante (/), que representa un directorio maestro especial denominado *raíz*. Luego está el nombre del primer subdirectorio, usr/imfst, seguido por un sub-subdirectorio, /Flynn, en este sistema de múltiples directorios. La entrada final es el nombre relativo del archivo, inventory_cost.doc. (Observe que en UNIX y en Linux los nombres de archivo son sensibles y a menudo se escriben con minúsculas.)

Como puede ver, los nombres tienden a crecer en longitud, en tanto los administradores de archivos crecen en flexibilidad. Las carpetas en un sistema con interfaz gráfica de usuario, como Windows o Macintosh, en realidad son directorios o subdirectorios. Cuando alguien crea una carpeta, el sistema crea un subdirectorio en el directorio o en la carpeta actuales.

¿Por qué los usuarios no pueden ver el nombre completo del archivo cuando ingresan en éste? Primero, el administrador de archivos escoge un directorio para el usuario cuando empieza la sesión interactiva, de modo que todas las operaciones con archivos solicitadas por ese usuario empiezan desde este directorio "principal" o "base". Segundo, a partir de este directorio principal, el usuario escoge un subdirectorio, que se denomina **directorio actual** o **directorio de trabajo**. A partir de entonces, se supone que los archivos están ubicados en este directorio actual. Cada vez que se tiene acceso a un archivo, el usuario escribe el nombre relativo de éste y el administrador de archivos agrega el prefijo idóneo. En tanto los usuarios hagan referencia a archivos en el directorio de trabajo, pueden tener acceso a sus archivos sin escribir el nombre completo.

Los sistemas operativos separan los elementos de los archivos con delimitadores. Por ejemplo, algunos usan una diagonal hacia delante (/) y otros, una hacia atrás (\).

El concepto de directorio actual está basado en la jerarquía subyacente de una estructura de árbol, como se muestra en la figura 8.4, y permite a los programadores recuperar un archivo al escribir sólo su nombre relativo...

```
INVENTORY_COST.DOC
```

...y no su nombre absoluto:

```
C:\IMFST\FLYNN\INVENTORY_COST.DOC
```

Organización de archivos

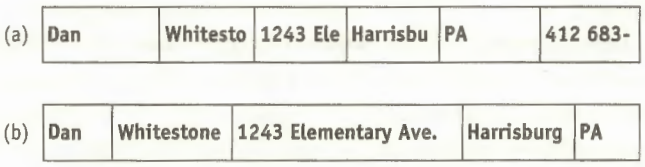
Cuando se analiza la organización de archivos se está hablando del arreglo de los registros en un archivo, ya que todos los archivos están compuestos por registros. Cuando un usuario emite un comando para modificar el contenido de un archivo, en realidad se trata de un comando para tener acceso a registros dentro del archivo.

Formato de los registros

Todos los archivos están compuestos por registros. Cuando un usuario emite un comando para modificar el contenido de un archivo, en realidad se trata de un comando para tener acceso a registros dentro del archivo. Dentro de cada archivo, se supone que todos los registros tienen el mismo formato; pueden ser de longitud fija o longitud variable, como se muestra en la figura 8.6. Y estos registros, sin importar su formato, pueden estar bloqueados o no.

(Figura 8.6)

Cuando las datos se almacenan en campos de longitud fija (a), los datos que rebasan el tamaño fijo son truncados. Cuando los datos se almacenan en un registro con formata de longitud variable (b), el tamaño se agranda a fin de dar cabida al contenido, aunque para tener acceso al archivo se requiere más tiempo.



Los **registros de longitud fija** son los más comunes, ya que son a los que es más fácil acceder directamente. Es por esto que son ideales para archivos de datos. La parte crítica de los registros de longitud fija es el tamaño del registro. Si es demasiado pequeño –menor que el número de caracteres a almacenar en el registro– los caracteres que no caben son truncados. Pero si el tamaño del registro es demasiado grande –más grande que el número de caracteres a almacenar– se desperdicia espacio de almacenamiento.

Los **registros de longitud variable** no dejan espacio de almacenamiento vacío y no truncan ningún caracter, eliminando así las dos desventajas de los registros de longitud fija. Pero aunque es fácil leerlos (uno después de otro), es difícil tener acceso directo a ellos, ya que es difícil calcular exactamente el sitio en que se encuentra ubicado el registro. Es por ello que se usan principalmente en archivos que probablemente sean accedidos secuencialmente, como archivos de texto y archivos de programa o archivos que usan un índice para tener acceso a sus registros. Al formato del registro, la forma en que está bloqueado y otra información relacionada se mantienen en el descriptor del archivo.

La cantidad de espacio que realmente se usa para almacenar la información complementaria suele variar de un sistema a otro y se ajusta a las limitaciones físicas del medio de almacenamiento, como veremos más tarde en este capítulo.

Organización física de los archivos

La organización física de un archivo tiene que ver con la forma en que están dispuestos los registros y con las características del medio que se usa para almacenarlos.

En discos magnéticos (discos duros), los archivos pueden organizarse en una de tres formas: secuencial, directa o indexada secuencial. Para escoger la mejor de estas organizaciones de archivos, el programador o analista suele considerar las siguientes características prácticas:

- Volatibilidad de los datos: la frecuencia con que se hacen adiciones y borrados.
- Actividad del archivo: porcentaje de registros procesados durante una ejecución dada.
- Tamaño del archivo.
- Tiempo de respuesta: cantidad de tiempo que el usuario está dispuesto a esperar antes que se complete la operación solicitada. (Esto es especialmente importante cuando se realizan investigaciones sensibles al tiempo.)

La **organización secuencial de los registros** es con mucho la más fácil de implementar, porque los registros se almacenan y recuperan en serie, uno después de otro. Para encontrar un registro específico, el archivo se busca desde su inicio hasta que se encuentra el registro solicitado.

Para acelerar el proceso es posible integrar algunas características al sistema. Una consiste en escoger del registro un campo clave y luego ordenar los registros por ese campo antes de almacenarlos. Luego, cuando un usuario solicite un registro específico, el sistema sólo busca en el campo clave de cada registro en el archivo. Esta búsqueda termina cuando se encuentra una coincidencia exacta o el campo clave del registro solicitado es menor que el valor del último registro comparado, en cuyo caso se envía el mensaje "registro no encontrado" al usuario y termina la búsqueda.

Aunque esta técnica ayuda al proceso de búsqueda, complica el mantenimiento de los archivos, ya que el orden original debe preservarse siempre que se agreguen o borren registros. Y para preservar el orden físico, el archivo debe volver a escribirse por completo o mantenerse ordenado dinámicamente cada vez que se actualiza.

Una **organización de registros directa** usa archivos de acceso directo, lo que, por supuesto, sólo puede implementarse en dispositivos de almacenamiento con acceso directo (analizados en el capítulo 7). Estos archivos proporcionan al usuario la flexibilidad de tener acceso a cualquier registro en cualquier orden sin tener que comenzar a buscar desde el inicio del archivo para hacerlo. También se denomina "organización aleatoria" y los archivos se denominan "archivos de acceso aleatorio".

Los registros se identifican por sus **direcciones relativas**: sus direcciones con respecto al principio del archivo. Estas **direcciones lógicas** se calculan cuando el registro se ha almacenado y luego otra vez cuando se recuperan los registros.

El método es bastante directo. El usuario identifica un campo (o una combinación de campos) en el formato del registro y lo designa como su **campo clave**, porque identifica de manera única cada registro. El programa usado para almacenar los datos sigue un conjunto de instrucciones, denominado **algoritmo hashing**, que transforma cada clave en un número: la dirección lógica del registro. Esto se proporciona al administrador de archivos, que lleva a cabo los pasos necesarios para transformar la dirección lógica en una dirección física (cilindro, superficie y números de registro) preservando la organización del archivo. Para recuperar un registro se usa el mismo procedimiento.

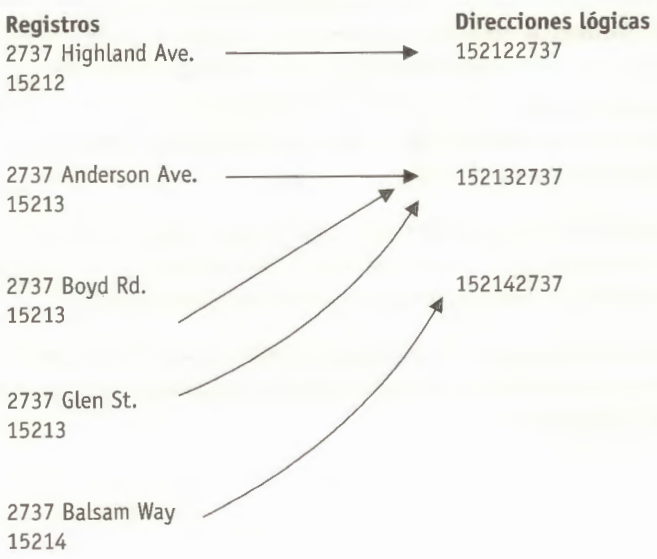
Las bases de datos usan una tabla hash para acelerar el acceso a los registros de datos. Al usar un algoritmo hash, cada registro se identifica de manera única, y la tabla hash contiene los apuntadores hacia cada registro.

Puede accederse de manera secuencial a un archivo de acceso directo, empezando en la primera dirección relativa y yendo a cada registro bajo la línea.

Los archivos de acceso directo pueden actualizarse más fácilmente que los ficheros secuenciales, porque los registros pueden ser reescritos rápidamente a sus direcciones originales después que se han introducido modificaciones. Y no hay necesidad de preservar el orden de los registros, para añadir o borrar éstos toma muy poco tiempo.

Por ejemplo, los datos para una empresa que recibe órdenes por correo telefónico deben ser ingresados rápidamente, así que la información de sus clientes debe ser recuperada con la misma rapidez. Suponga que usted está colocando una orden y que le piden el número de su código postal y el de su calle (por ejemplo, 15231 y 2737). El programa que recupera información del archivo de datos usa esta clave en un algoritmo hash para calcular la dirección lógica del sitio en que está almacenado su registro. Así, cuando el empleado que tomó la orden escribe 152132737, la pantalla muestra pronto una lista de clientes concurrentes cuyos números de cliente generaron la misma dirección lógica. Si usted está en la base de datos, el operador lo sabe al instante. En caso contrario, pronto lo estará.

El problema con los algoritmos hash es que varios registros con claves únicas (como los números de cliente) pueden generar la misma dirección lógica: entonces hay una colisión, como se muestra en la figura 8.7. Cuando esto ocurre, el programa debe generar otra dirección lógica antes de presentarla al administrador de archivos para su almacenamiento. Los registros que chocan se almacenan en un área de desbordamiento que se hizo a un lado cuando se creó el archivo. Aunque el programa hace todo el trabajo de vincular los registros desde el área de desbordamiento con su dirección lógica correspondiente, el administrador de archivos debe manejar la asignación física del espacio.



(Figura 8.7)

El algoritmo hashing provoca una colisión cuando el algoritmo, usando una combinación de dirección de calle y código postal, genera la misma dirección lógica (152132737) para tres registros diferentes.

El tamaño máximo del archivo se establece, cuando es creado, y finalmente cualquiera de los archivos se llena por completo o el número de registros almacenados en el área de desbordamiento puede hacerse tan grande que se pierde la eficiencia de recuperación. En este caso el archivo debe reorganizarse y volver a escribirse, lo cual requiere de la intervención del administrador de archivos.

La **organización secuencial indexada de registros** combina lo mejor del acceso directo y secuencial. Es creado y mantenido mediante un paquete de software denominado método de acceso secuencial indexado (ISAM, por sus siglas en inglés), que elimina la carga de manejar desbordamientos y preserva el orden de los registros en los hombros del programador.

Este tipo de organización no provoca colisiones, ya que no usa el resultado del algoritmo hash para generar la dirección de un registro. En lugar de eso, usa esta información para generar un archivo índice por medio del cual se recuperan los registros. Esta organización divide un archivo en orden secuencial en bloques del mismo tamaño. Su tamaño está determinado por el administrador de archivos, a fin de aprovechar los dispositivos de almacenamiento físico y optimizar las estrategias de recuperación. Cada entrada en el archivo índice contiene la clave del registro más alto y la ubicación física del bloque de datos donde están almacenados este registro y los registros con claves menores.

En consecuencia, para tener acceso a cualquier registro en el archivo, el sistema empieza por buscar en el archivo índice y luego se dirige a la ubicación física indicada en esa entrada. Puede decirse, entonces, que el archivo índice actúa como un apuntador hacia el archivo de datos. Un archivo secuencial indexado también tiene áreas de desbordamiento, aunque están dispersas en todo el archivo, tal vez muy pocos registros, de modo que puede ocurrir expansión de registros existentes, y nuevos registros pueden ubicarse en secuencia física próxima, así como en secuencia lógica. Otra área de desbordamiento está ubicada lejos del área de los datos principales, pero sólo se usa cuando las otras áreas de desbordamiento están completamente llenas. Este desbordamiento se denomina el último recurso.

Esta área de desbordamiento del último recurso puede almacenar registros que se han agregado durante el tiempo de duración del archivo. El paquete de software mantiene los registros en orden lógico sin mucho esfuerzo de parte del programador. Por supuesto, cuando aquí se han agregado demasiados registros, el proceso de recuperación se vuelve lento, ya que la búsqueda por un registro debe ir desde el índice hasta el área de datos principales y terminar en el área de desbordamiento.

Cuando el tiempo de recuperación se hace demasiado lento, es necesario reorganizar el archivo. Se trata de un trabajo que, aunque no es tan tedioso como reorganizar archivos de acceso directo, suele ser realizado por un programa de mantenimiento.

Para la mayor parte de los archivos dinámicos, el acceso secuencial indexado es la organización a elegir, ya que permite tanto acceso directo a unos cuantos registros solicitados como acceso secuencial a muchos.

Asignación de almacenamiento físico

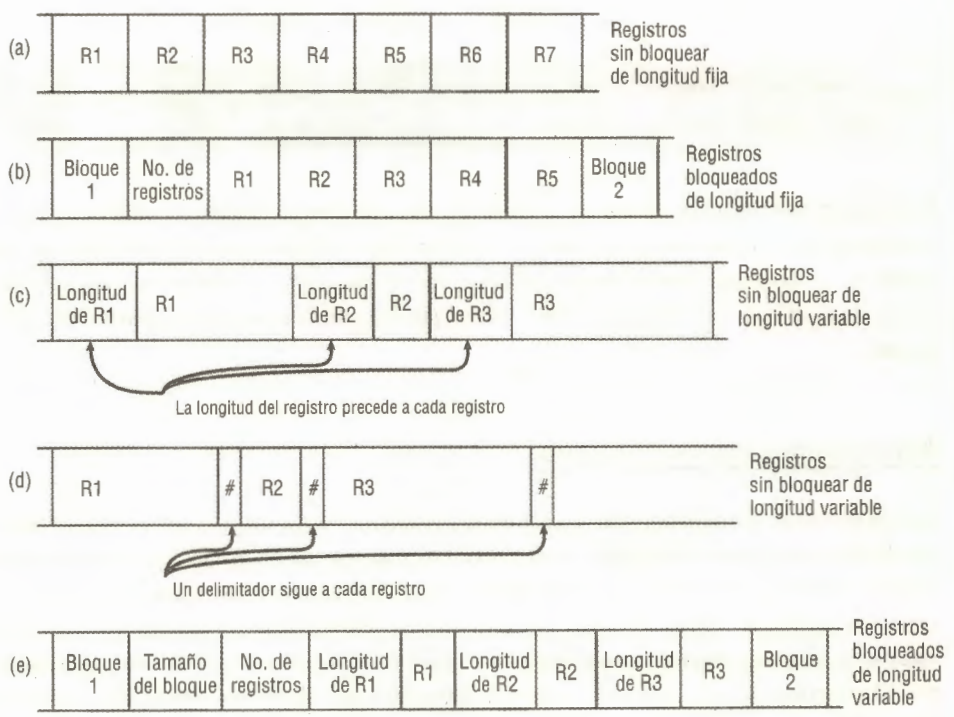
El administrador de archivos debe trabajar con archivos no sólo como unidades completas, sino también como unidades o registros lógicos. Los registros dentro de un archivo deben tener el mismo formato, aunque pueden variar en longitud, como se muestra en la figura 8.8.

A su vez, los registros se subdividen en campos. En la mayor parte de los casos, su estructura es administrada por programas de aplicación y no por el sistema operativo. Una excepción es la de los sistemas que están fuertemente orientados a aplicaciones de bases de datos, donde el administrador de archivos maneja la estructura de campo.

Así, cuando hablamos sobre almacenamiento de archivos, en realidad nos referimos a almacenamiento de registros. ¿Cómo se almacenan los registros dentro de un archivo? En este momento, el administrador de archivos y el administrador de dispositivos deben cooperar para asegurar el almacenamiento y la recuperación exitosas de registros. En el capítulo 7 sobre administración de dispositivos, presentamos el concepto de registros lógicos contra registros físicos, y este tema recurre aquí desde el punto de vista del administrador de archivos.

(Figura 8.8)

Todos los registros en un archivo deben tener el mismo formato, aunque sus tamaños pueden ser diferentes, como se muestra en estos cinco ejemplos de los formatos más comunes de registros. La información complementaria en (b, c, d y e) es proporcionada por el administrador de archivos, cuando se almacena el registro.

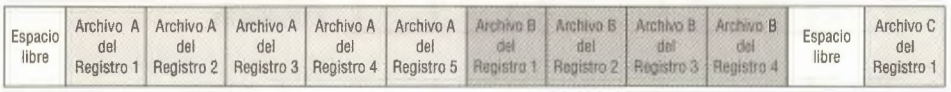


Almacenamiento contiguo

Cuando los registros usan **almacenamiento contiguo**, se almacenan unos después de otro. Este era el esquema usado en los primeros sistemas operativos. Es muy fácil de implementar y administrar. Cualquier registro puede encontrarse y leerse, una vez que se conocen su di-

rección inicial y tamaño, de modo que el directorio es muy alineado. Su segunda ventaja es la facilidad de acceso directo, porque cada una de las partes del archivo se almacena en la misma área compacta.

La desventaja más importante es que no es posible agrandar el archivo a menos que haya espacio vacío inmediatamente a continuación, como se muestra en la figura 8.9. En consecuencia, el espacio para la expansión debe tenerse en cuenta cuando se crea el archivo. En caso de que no haya suficiente espacio, todo el archivo debe volver a copiarse en una sección más grande del disco cada que se agregan registros. La segunda desventaja es la fragmentación (huecos de espacio no usado), lo cual puede resolverse al compactar y reordenar los archivos. Y, por supuesto, mientras se lleva a cabo la compactación no es posible tener acceso a los archivos.



(Figura 8.9)

Con un almacenamiento contiguo de archivos, el archivo A no puede agrandarse sin que vuelva a escribirse en un área de almacenamiento más grande. El archivo B puede agrandarse, pero sólo por un registro que sustituya el espacio libre que precede al archivo C.

El administrador de archivos sigue la pista de las áreas de almacenamiento vacías al tratarlas como archivos: se introducen en el directorio pero están indicadas para diferenciarlas de los archivos verdaderos. Normalmente, el directorio se mantiene en orden por número de sector, de modo que las áreas adyacentes vacías pueden combinarse en un espacio libre más grande.

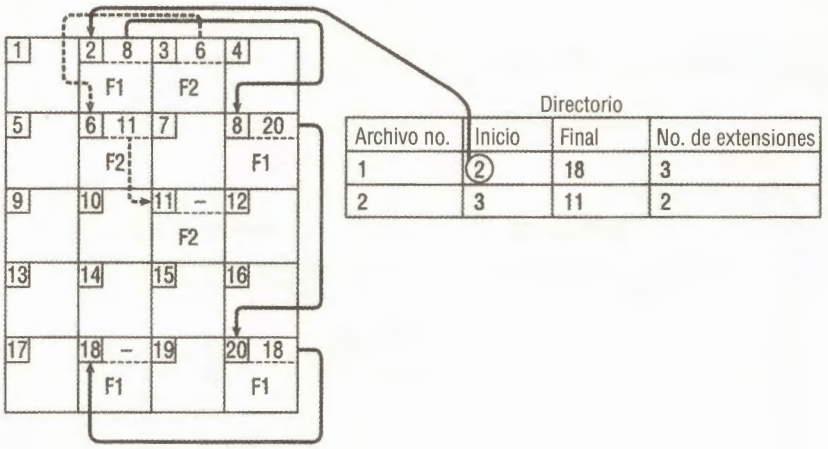
Almacenamiento no contiguo

La asignación de **almacenamiento no contiguo** permite que los archivos usen cualquier espacio de almacenamiento disponible en el disco. Los registros de un archivo se almacenan de manera contigua, sólo si hay suficiente espacio vacío. Cualquier registro restante y todas las demás adiciones al archivo se almacenan en otras secciones del disco. En algunos sistemas éstas se denominan **extensiones** del archivo y están vinculadas entre sí mediante apuntadores. El tamaño físico de cada extensión es determinada por el sistema operativo y suele ser 256; o alguna potencia de dos, bytes.

Las extensiones de los archivos suelen estar vinculadas de dos formas. La vinculación puede llevarse a cabo a nivel del almacenamiento, donde cada punto de la extensión apunta al siguiente en la secuencia, como se muestra en la figura 8.10. La entrada del directorio consta del nombre del archivo, la ubicación de almacenamiento de la primera extensión, la ubicación de la última extensión y el número total de extensiones sin contar la primera.

(Figura 8.10)

Almacenamiento no contiguo de archivos donde la vinculación se lleva a cabo a nivel del almacenamiento. El archivo 1 empieza en la dirección 2 y continúa en las direcciones 8, 20 y 18. El directorio enlista la dirección inicial del archivo, su dirección final y el número de extensiones que usa. Cada bloque de almacenamiento incluye su dirección y un apuntador hacia el siguiente bloque para el archivo, así como los datos en sí.



La alternativa es que la vinculación se lleve a cabo a nivel del directorio, como se muestra en la figura 8.11. Cada extensión se enumera con su dirección física, su tamaño y un apuntador hacia la siguiente extensión. Un apuntador nulo, mostrado aquí como un guión (-), indica que es el último.

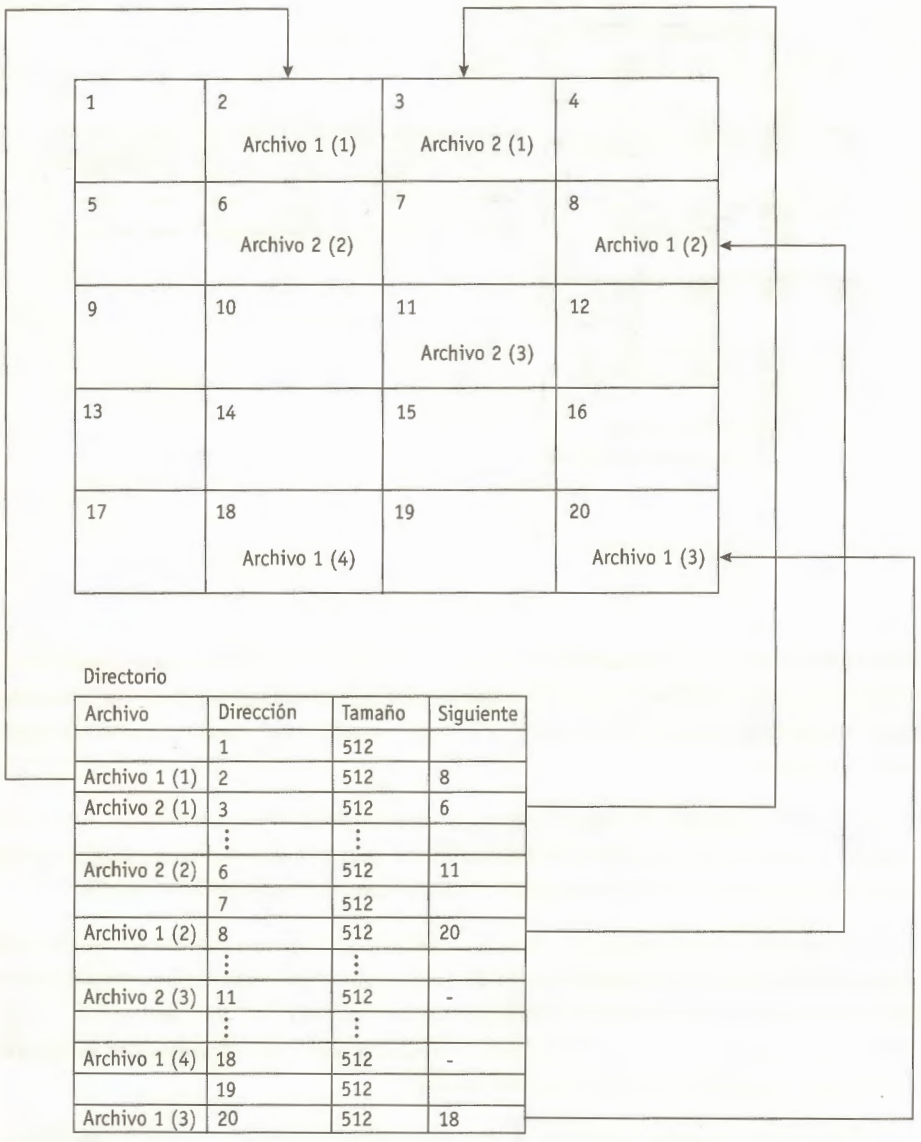
Aunque ambos esquemas de asignación no contiguo eliminan fragmentación externa de almacenamiento y la necesidad de compactación, no atienden acceso directo, ya que no hay ninguna forma sencilla para determinar la ubicación exacta de un registro específico.

Los archivos suelen declararse como de acceso secuencial o directo cuando son creados, de modo que el administrador de archivos puede escoger el método más eficiente de asignación de almacenamiento; contiguo para archivos con acceso directo y no contiguo para los archivos de acceso secuencial. Los sistemas operativos deben contar con la capacidad de atender ambos esquemas de asignación de almacenamiento.

Luego, los archivos pueden convertirse de un tipo a otro al crear un archivo del tipo deseado y copiando el contenido del archivo anterior en el archivo nuevo, usando un programa diseñado para este propósito especial.

Almacenamiento indexado

La asignación de almacenamiento indexado permite acceso directo a los registros, al reunir los apuntadores que vinculan todas las extensiones de ese archivo en un bloque de índice. Cada archivo tiene su propio bloque de índice, que consta de las direcciones de cada sector del disco que integran el archivo. El índice enumera cada entrada en el mismo orden en que



(Figura 8.11)
Asignación de almacenamiento no contiguo de archivos donde la vinculación se lleva a cabo a nivel del directorio para los archivos que se muestran en la figura 8.10.

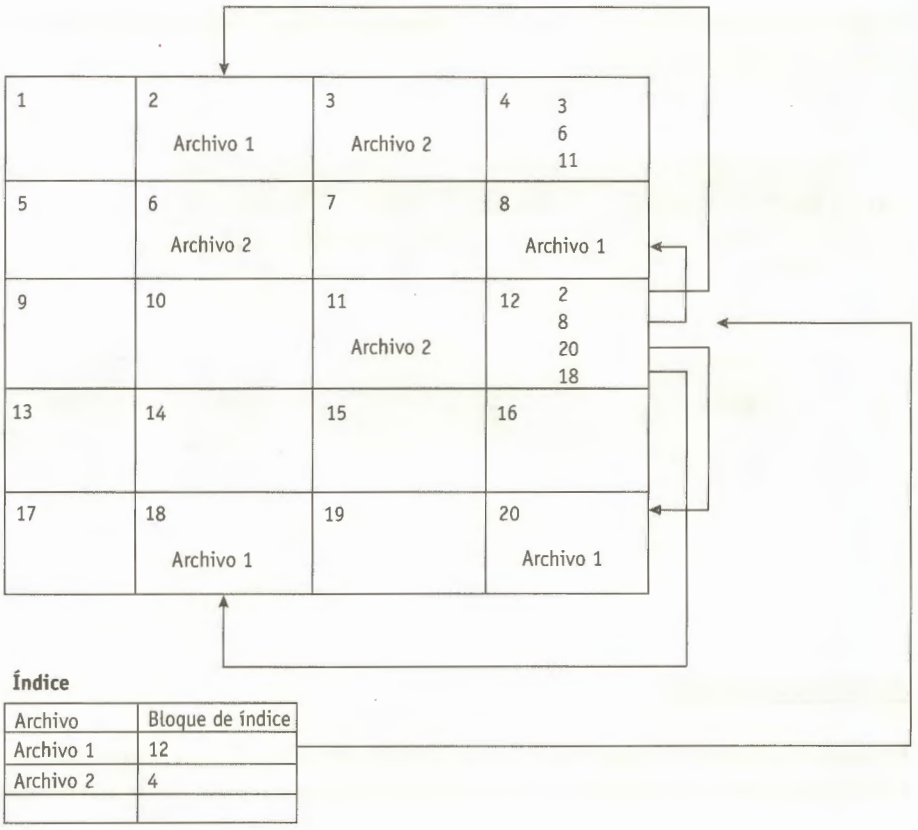
están vinculados los sectores, como se muestra en la figura 8.12. Por ejemplo, la tercera entrada en el bloque de índice corresponde al tercer sector que integra la lista.

Cuando se crea un archivo, todos los apuntadores en bloque de índice se habilitan en nulo. Luego, a medida que se llena cada sector, el apuntador se habilita en la dirección idónea del sector: para ser precisos, la dirección se elimina de la lista del espacio vacío y se copia en su posición en el bloque de índice.

Este esquema atiende acceso secuencial y directo, aunque no necesariamente mejora el uso del espacio de almacenamiento, ya que cada archivo debe tener un bloque de índice: en general del tamaño de un sector del disco. Para archivos más grandes con más entradas, es

(Figura 8.12)

Asignación de almacenamiento indexado con un índice de un nivel, permitiendo acceso directo a cada registro para los archivos que se muestran en las figuras 8.10 y 8.11.



Índice

Archivo	Bloque de índice
Archivo 1	12
Archivo 2	4

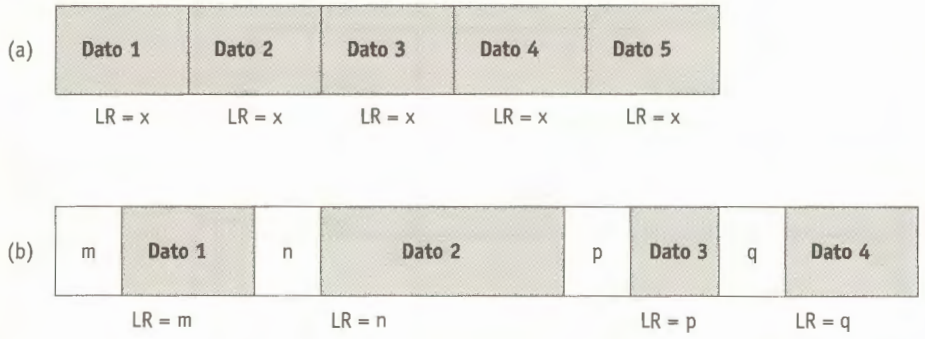
posible generar varios niveles de índices, en cuyo caso, para encontrar un registro deseado el administrador de archivos tiene acceso al primer índice (el nivel más alto), que apunta hacia un segundo índice (nivel más bajo), que apunta hacia un índice de nivel aún más bajo y finalmente hacia el registro de datos.

Métodos de acceso

Los métodos de acceso los determina una organización de archivos; se permite máxima flexibilidad con archivos secuenciales indexados y mínima, con secuenciales.

Un archivo que ha sido organizado de manera secuencial puede atender sólo acceso a sus registros, y estos registros pueden ser de longitud fija o variable, como se muestra en la figura 8.6. El administrador de archivos usa la dirección del último byte leído para tener acceso al siguiente archivo secuencial. En consecuencia, la **dirección del byte actual** (CBA) debe actualizarse cada vez que se tiene acceso a un registro, como cuando se ejecuta un comando READ.

La figura 8.13 muestra la diferencia entre el almacenamiento de registros de longitud fija y registros de longitud variable.



(Figura 8.13)
Archivos de longitud fija contra registros de longitud variable. a) Los registros de longitud fija tienen el mismo número de bytes, de modo que la longitud del registro (LR) es la constante x. b) Con registros de longitud variable, LR_k no es una constante. En consecuencia, se graba en los medios secuenciales, como cinta magnética, precediendo inmediatamente cada registro.

Acceso secuencial

Para *acceso secuencial de registros de longitud fija*, la dirección del byte actual (CBA) se actualiza simplemente al incrementarla por la longitud del registro (LR), que es una constante:

$$CBA = CBA + LR$$

Para *acceso secuencial de registros de longitud variable*, el administrador de archivos suma la longitud del registro (LR) más el número de bytes usados para contener la longitud del registro (N, el cual se mantiene constante, representado como m, n, p o q, en la figura 8.13.

$$CBA = CBA + N + LR_k$$

Acceso directo

Si un archivo está organizado de manera directa, es posible acceder fácilmente a él en orden directo o secuencial si los registros son de longitud fija. En el caso de *acceso directo con registros de longitud variable*, la dirección del byte actual (CBA) puede calcularse directamente a partir de la longitud del registro y del número de registro deseado NR (información proporcionada mediante el comando READ) menos uno:

$$CBA = (NR - 1) * LR$$

Por ejemplo, si se está buscando el comienzo del undécimo registro y la longitud fija de éste es de 25 bytes, la CBA sería:

$$(11 - 1) * 25 = 250$$

Sin embargo, si el archivo está organizado para *acceso directo con registros de longitud variable*, es virtualmente imposible tener acceso directo a un registro, ya que la dirección del registro deseado no puede calcularse fácilmente. En consecuencia, para tener acceso a un archivo, el

Al principio, el acceso directo se denominaba "acceso aleatorio", ya que permitía acceso a los datos almacenados en orden aleatorio. No requiere almacenamiento secuencial.

administrador de archivos debe efectuar una búsqueda secuencial en los registros. De hecho, se vuelve una lectura semisecuencial a través del archivo, porque el administrador de archivos puede guardar la dirección del último registro al que se tuvo acceso, y cuando llega la siguiente solicitud, puede buscar hacia delante a partir de la CBA: si la dirección del registro deseado estaba entre la CBA y el final del archivo. En caso contrario, la búsqueda podría empezar desde el principio del archivo. Podría decirse que esta búsqueda semisecuencial es sólo semiadecuada.

Una alternativa es que el administrador de archivos mantenga una tabla de números de registro y sus CBA. Después, para atender una solicitud, en esta tabla se busca la ubicación exacta de almacenamiento del registro deseado, de modo que el acceso directo se reduce a buscar en la tabla.

Para evitar tener que tratar con este problema, muchos sistemas obligan a los usuarios a tener organizados sus archivos para registros de longitud fija, si el acceso a los registros ha de ser directo.

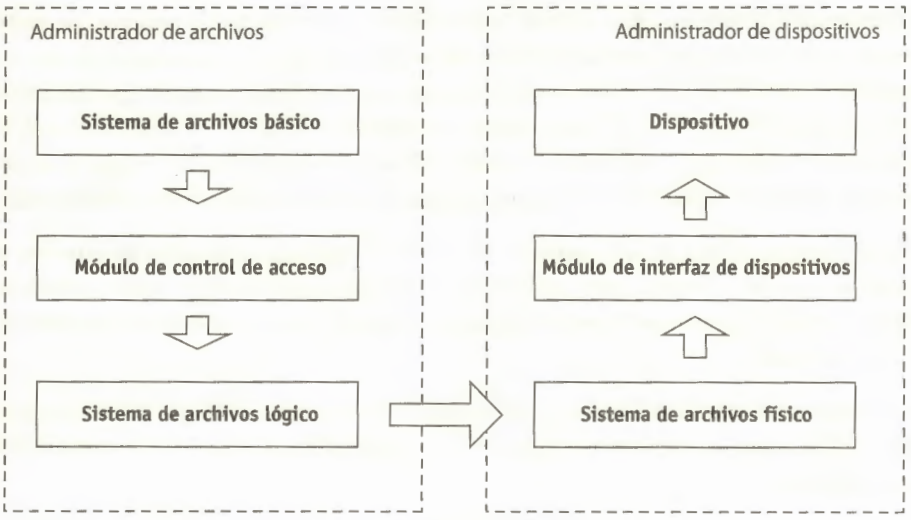
El acceso a los registros en un *archivo secuencial indexado* puede ser secuencial o directo, de modo que cualquiera de los procedimientos para calcular la CBA presentados en esta sección es válido, pero con un paso adicional: es necesario buscar en el archivo índice el apuntador hacia el bloque en que están almacenados los datos. Debido a que el archivo índice es más pequeño que el archivo de datos, puede mantenerse en la memoria principal y es posible realizar una búsqueda rápida para localizar el bloque donde se encuentra el registro deseado. Luego, el bloque puede recuperarse del almacenamiento secundario, y es posible calcular el byte inicial de la dirección del registro. En sistemas que atienden varios niveles de indexación para mejorar el acceso a archivos muy grandes, el índice en cada nivel debe buscarse antes que pueda efectuarse el cálculo de la dirección del byte actual. El punto de entrada a este tipo de datos suele ser mediante un archivo índice.

Como hemos visto, la organización de un archivo y los métodos usados para tener acceso a sus registros están estrechamente entrelazados, de modo que cuando se habla sobre un tipo de organización específica, ciertamente casi siempre se implica un tipo de acceso específico.

Niveles en un sistema de administración de archivos

La administración eficiente de archivos no puede separarse de la administración eficiente de los dispositivos que los alojan. Este capítulo y el capítulo previo sobre administración de dispositivos han presentado la amplia gama de funciones que es necesario organizar para que un sistema de E/S funcione de manera eficiente. En esta sección delinearemos una de las muchas jerarquías usadas para llevar a cabo estas funciones.

El módulo de nivel más alto se denomina "sistema de archivos básico" y pasa información a través del módulo de verificación y control de acceso al sistema de archivos lógico, que, a su vez, notifica al sistema de archivos físico, que trabaja con el administrador de archivos. La figura 8.14 muestra la jerarquía.



(Figura 8.14)
Módulos típicos de un sistema de administración de archivos que muestra cómo se pasa la información desde el administrador de archivos en la parte superior de la jerarquía hasta el administrador de dispositivos en la parte inferior.

Cada nivel del sistema de administración de archivos se implementa usando técnicas de programación estructurada y modular que también establecen una jerarquía; es decir, los módulos ubicados en las posiciones superiores pasan la información a los módulos ubicados en las posiciones inferiores y la comunicación continúa siguiendo la cadena hasta el módulo más bajo, que se comunica con el dispositivo físico e interactúa con el administrador de archivos. Sólo entonces el registro está disponible para el programa del usuario.

Cada uno de los módulos puede dividirse aún más en tareas más específicas, como puede verse cuando esta instrucción de E/S se sigue a través del sistema de administración de archivos:

```
READ RECORD NUMBER 7 FROM FILE CLASSES INTO STUDENT
```

CLASSES es el nombre de un archivo con acceso directo abierto previamente para datos de entrada, y STUDENT es un registro de datos definido previamente dentro del programa y que ocupa ubicaciones específicas de la memoria.

Debido a que ya se ha abierto el archivo, ya se ha buscado en el directorio de archivos para verificar la existencia de CLASSES, e información pertinente sobre el archivo ya se ha llevado a la tabla de archivos activos del sistema operativo. Esta información incluye su tamaño de registro, la dirección de su primer registro físico, su protección e información de control de acceso, como se muestra en el listado del directorio UNIX de la tabla 8.1.

Control de acceso	No. enlaces	Grupo	Propietario	No. de bytes	Fecha	Hora	Nombre del archivo
drwxrwxr-x	2	journal	comp	12820	10 ene	19:32	ArtWarehouse
drwxrwxr-x	2	journal	comp	12844	15 dic	09:59	bus_transport
-rwxr-xr-x	1	journal	comp	2705221	6 mar	11:38	CLASSES
-rwxr--r--	1	journal	comp	12556	20 feb	18:08	PAYroll
-rwx------	1	journal	comp	8721	17 ene	07:32	supplier

(Tabla 8.1)

Lista típica de los archivos almacenados en el directorio denominado Journal.

Esta información se usa en el sistema de archivos básico, que activa el módulo de verificación de control de acceso para verificar que ese usuario está autorizado para efectuar esta operación con este archivo. Si se permite el acceso, la información y el control pasan al sistema de archivos lógico. En caso contrario, al usuario se le envía un mensaje que dice "acceso denegado".

Al usar la información pasada por el sistema de archivos básico, el sistema de archivos lógico transforma el número de registro en su byte de dirección usando la fórmula conocida:

$$CBA = (NR - 1) * LR$$

Este resultado, junto con la dirección del primer registro físico y, en el caso donde los registros están bloqueados, el tamaño del bloque físico, se envían al sistema de archivos físico, que calcula la ubicación donde reside físicamente el registro deseado. Si en ese bloque hay más de un registro, calcula el desplazamiento del registro dentro de ese bloque usando estas fórmulas:

$$\text{número de bloque} = \text{enteros}\left[\frac{\text{byte de dirección}}{\text{tamaño del bloque físico}}\right] + \text{dirección del primer registro físico}$$

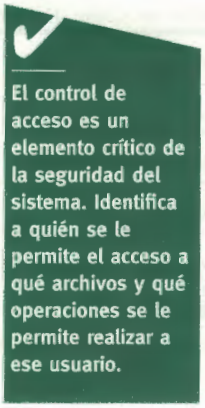
$$\text{desplazamiento} = \text{residuo}\left[\frac{\text{byte de dirección}}{\text{tamaño del bloque físico}}\right]$$

Esta información se pasa al módulo de interfaz de dispositivos, que, a su vez, transforma el número de bloque en la combinación real cilindro/superficie/registro necesaria para recuperar la información del dispositivo de almacenamiento secundario. Una vez recuperada, aquí es donde entran en escena los algoritmos de planificación de dispositivos, ya que la información es colocada en un búfer y el control regresa al sistema de archivos físico, que copia la información en la ubicación deseada de la memoria. Finalmente, cuando la operación está completa, a todos los otros módulos se envía el mensaje "todo bien".

Aunque para nuestro ejemplo usamos un comando READ, un comando WRITE se maneja exactamente igual hasta que el proceso llega al manejador de dispositivos. En ese momento, la porción del módulo de interfaz de dispositivos que maneja la asignación de espacio libre, el módulo de asignación, entra en acción, ya que es responsable de seguir la pista de las áreas sin usar en cada dispositivo de almacenamiento.

Debemos observar aquí que la verificación, el proceso de asegurarse que una solicitud es válida, ocurre a todos los niveles del sistema de administración de archivos. La primera verificación se lleva a cabo a nivel del directorio cuando el sistema comprueba la existencia del archivo solicitado. La segunda ocurre cuando el módulo de verificación de control de acceso determina si se permite el acceso. La tercera ocurre cuando el sistema de archivos lógico comprueba si el byte de dirección está dentro de los límites del archivo. Finalmente, el módulo de interfaz de dispositivos comprueba si existe el dispositivo de almacenamiento.

En consecuencia, la operación correcta de un simple comando de usuario requiere el esfuerzo coordinado del sistema de administración de archivos.



Módulo de verificación de control de acceso

Los primeros sistemas operativos no podían prestar el servicio de compartición de archivos entre los usuarios. Por ejemplo, los primeros sistemas necesitaban 10 copias de un compilador para atender a 10 usuarios. Los sistemas actuales sólo requieren una copia para atender a todos, sin importar el número de programas activos que haya en el sistema. De hecho, cualquier archivo puede compartirse; desde archivos de datos y archivos de programas propiedad del usuario hasta sistemas de archivos. Las ventajas de compartir archivos son numerosas. Además de ahorrar espacio, permite la sincronización de actualizaciones de datos, como cuando dos aplicaciones actualizan el mismo archivo de datos. También mejora la eficiencia de los recursos del sistema, ya que si se comparten los archivos en la memoria principal entonces hay una reducción de operaciones de E/S.

Sin embargo, como suele ocurrir, el progreso conlleva problemas. La desventaja de compartir archivos es que debe salvaguardarse la integridad de cada archivo; eso requiere el control sobre a quién se le permite el acceso al archivo y qué tipo de acceso se permite. Hay cinco acciones posibles que pueden realizarse sobre un archivo: la capacidad de sólo LECTURA, de sólo ESCRITURA, de sólo EJECUTAR, de sólo BORRAR o alguna combinación de las cuatro. Cada sistema de administración de archivos tiene su propio método para controlar el acceso a archivos.

Matriz de control de acceso

Intuitivamente, la **matriz de control de acceso** es atractiva y fácil de implementar, pero por su tamaño, sólo funciona bien para sistemas con pocos archivos y pocos usuarios. En la matriz, cada columna identifica un usuario y cada renglón identifica un archivo. La intersección de un renglón y una columna contiene los derechos de acceso para ese usuario a ese archivo, como se ilustra en la tabla 8.2.

(Tabla 8.2)

Matriz de control de acceso que muestra los derechos de acceso para cada usuario a cada archivo. Al usuario 1 se le permite acceso ilimitado al archivo 1 pero sólo para leer y ejecutar el archivo 4, y se le niega el acceso a los otros tres archivos. R = acceso para lectura, W = acceso para escritura, E = acceso para ejecutar, D = acceso para borrar, y un guión (-) = acceso no concedido.

	Usuario 1	Usuario 2	Usuario 3	Usuario 4	Usuario 5
Archivo 1	RWED	R-E-	----	RWE	---E-
Archivo 2	----	R-E-	R-E-	--E-	----
Archivo 3	----	RWED	----	--E-	----
Archivo 4	R-E-	----	----	----	RWED
Archivo 5	----	----	----	----	RWED

En la implementación real, las letras RWED están representadas por bits uno y cero: un uno indica que se ha concedido el acceso, y un cero, que se ha negado el acceso. En consecuencia, como se muestra en la tabla 8.3, el código para el usuario 2 para el archivo 1 se leería "1010" y no "R-E-".

(Tabla 8.3)

Los cinco códigos de acceso para el usuario 2 de la tabla 8.2. El código resultante para cada archivo es creado al asignar un 1 a cada arca de verificación y un 0 a cada espacio en blanco.

Acceso	R	W	E	D	Código resultante
R-E-	√		√		1010
R-E-	√		√		1010
RWED	√	√	√	√	1111
----					0000
----					0000

Como puede ver, la matriz de control de acceso es un método simple, pero a medida que aumentan el número de usuarios y archivos, la matriz se vuelve extremadamente grande; algunas veces demasiado grande para almacenarse en la memoria principal. Otra desventaja es que se desperdicia mucho espacio, ya que muchas de las entradas son nulas, como en la tabla 8.2, donde al usuario 3 no se permite acceder a la mayor parte de los archivos, y el archivo 5 está restringido a todos menos a un usuario. Un esquema que conservaba espacio sólo tendría una entrada para el usuario 3 o una para el archivo 5, pero esto es incompatible con el formato de la matriz.

Listas de control de acceso

La **lista de control de acceso** es una modificación de la matriz de control de acceso. Cada archivo se escribe en la lista y contiene los nombres de los usuarios a quienes se les ha permitido el acceso al archivo, así como el tipo de acceso que se permite a cada uno. Para acortar la lista, sólo se menciona a quienes pueden usar el archivo; a quienes se les ha negado el acceso se agrupan bajo un encabezado global como WORLD, como se muestra en la tabla 8.4.

Archivo	Acceso
Archivo 1	USUARIO1 (RWED), USUARIO2 (R-E-), USUARIO4 (RWE-), USUARIO5 (--E-), WORLD (----)
Archivo 2	USUARIO2 (R-E-), USUARIO3 (R-E-), USUARIO4 (--E-), WORLD (----)
Archivo 3	USUARIO2 (RWED), USUARIO4 (--E-), WORLD (----)
Archivo 4	USUARIO1 (R-E-), USUARIO5 (RWED), WORLD (----)
Archivo 5	USUARIO5 (RWED), WORLD (----)

(Tabla 8.4)
Lista de control de acceso que muestra a cuáles usuarios se les ha permitido el acceso a cada archivo. Este método requiere menos espacio de almacenamiento que una matriz de control de acceso.

Algunos sistemas acortan aún más la lista de control de acceso al colocar a cada usuario en una categoría: sistema, propietario, grupo y mundo. SYSTEM o ADMIN está diseñado para personal del sistema que tiene acceso ilimitado a todos los archivos en el sistema. El OWNER tiene control absoluto sobre todos los archivos creados en la cuenta del propietario. Un propietario puede crear un archivo GROUP de modo que todos los usuarios que pertenecen al grupo idóneo tienen acceso al archivo. WORLD está integrado por todos los otros usuarios en el sistema; es decir, los que no están en ninguna de las otras tres categorías. En este sistema, el administrador de archivos designa tipos de acceso predeterminados a todos los archivos en el momento de su creación, y es responsabilidad del propietario modificarlos según sea necesario.

Listas de capacidad

Una **lista de capacidad** muestra la información de control de acceso desde una perspectiva diferente. Enumera todos los usuarios y los archivos a los que cada uno tiene acceso, como se muestra en la tabla 8.5.

Usuario	Acceso
Usuario 1	Archivo 1 (RWED), Archivo 4 (R-E-)
Usuario 2	Archivo 1 (R-E-), Archivo 2 (R-E-), Archivo 3 (RWED)
Usuario 3	Archivo 2 (R-E-)
Usuario 4	Archivo 1 (RWE-), Archivo 2 (--E-), Archivo 3 (--E-)
Usuario 5	Archivo 1 (--E-), Archivo 4 (RWED), Archivo 5 (RWED)

(Tabla 8.5)
Una lista de capacidad que muestra los archivos para cada usuario requiere menos espacio de almacenamiento que una matriz de control de acceso y es más fácil de mantener que una lista de control de acceso cuando se agregan o eliminan usuarios del sistema.

De los tres esquemas descritos hasta el momento, el de uso más común es la lista de control de acceso. Sin embargo, las listas de capacidad están ganando popularidad, ya que en sistemas operativos como Linux o UNIX, controlan el acceso a dispositivos, así como también a archivos.

Aunque parece que ambos métodos son iguales, hay algunas diferencias sutiles que se explican mejor con una analogía. Una lista de capacidad puede igualarse a boletos de un concierto específico donde sólo se ponen a disposición de personas cuyo nombre aparece en la lista. Por otra parte, una lista de control de acceso puede igualarse a una lista de reservaciones en un restaurante con aforo limitado, donde cada sitio se asigna a una persona específica.

Compresión de datos

Los algoritmos de **compresión de datos** son de dos tipos: algoritmos fuertes que típicamente son utilizados para archivos de texto o aritméticos, los que retienen los datos en el archivo a lo largo del proceso de compresión-descompresión; y algoritmos débiles que generalmente son utilizados para archivos de datos y video y para borrar datos de forma permanente. A primera vista, no podemos pensar que una pérdida de datos puede ser tolerable, pero cuando los datos borrados son un ruido indeseable, tonos más allá de la capacidad del oído humano o espectros de luz que no podemos ver, eliminar estos datos puede ser indetectable y por tanto aceptable.

Compresión de texto

Para comprimir un texto en una base de datos, aquí se describen brevemente tres métodos: registros con caracteres repetidos, términos repetidos y compresión *front-end*.

Registros con caracteres repetidos: los datos en un campo de longitud fija pueden incluir un nombre corto seguido por muchos caracteres en blanco. Esto puede sustituirse por un campo de longitud variable y un código especial para indicar cuántos espacios en blanco fueron truncados.

Por ejemplo, suponga que la cadena original, ADAMS, se ve como sigue cuando se almacena sin comprimir en un campo de 15 caracteres de ancho (*b* representa un caracter en blanco):

```
ADAMSbbbbbbbbbb
```

Una vez codificada se ve como sigue:

```
ADAMSb10
```

De manera semejante, es posible acortar números con muchos ceros, también con un código para indicar cuántos ceros deben agregarse para recrear el número original (en este caso con el signo #). Por ejemplo, si la entrada original es este número:

300000000

la entrada codificada es ésta:

3#8

Términos repetidos pueden comprimirse usando símbolos para representar cada una de las palabras más usadas en la base de datos. Por ejemplo, en la base de datos de un estudiante universitario, cada una de las palabras comunes como *estudiante, curso, maestro, aula, calificación* y *departamento* pueden representarse con un solo caracter. Por supuesto, el sistema debe poder distinguir entre datos comprimidos y no comprimidos.

La *compresión front-end* trabaja con base en el elemento de datos previos. Por ejemplo, la base de datos del estudiante donde los nombres de los estudiantes se guardan en orden alfabético puede comprimirse, como se muestra en la tabla 8.6.

Lista original	Lista comprimida
Smith, Betty	Smith, Betty
Smith, Gino	7Gino
Smith, Donald	7Donald
Smithberger, John	5berger, John
Smithbren, Ali	6ren, Ali
Smithco, Rachel	5co, Rachel
Smither, Kevin	5er, Kevin
Smithers, Renny	7s, Renny
Snyder, Katherine	1nyder, Katherine

(Tabla 8.6)

Cada entrada asume un número dado de caracteres de la entrada previa que tienen en común y agrega los caracteres que lo hacen único. Así, "Smithbren, Ali" usa los seis primeros caracteres de "Smithbren, John" y agrega "ren". En consecuencia, la entrada es 6ren, Ali.

Hay una negociación: se gana espacio de almacenamiento, pero se pierde tiempo de procesamiento. Recuerde que para todos los esquemas de compresión de datos, el sistema debe poder distinguir entre datos comprimidos y no comprimidos.

Otros esquemas de compresión

La compresión débil permite una pérdida de datos del archivo original para admitir una compresión significativa. Esto significa que el proceso de compresión es irreversible y que el archivo original no puede ser reconstruido. Las especificaciones de los algoritmos de compresión dependen en gran medida del tipo de archivo que se va a comprimir, con JPEG como una opción po-

pular para conservar imágenes y MPEG para imágenes y video. Para archivos de video y música, la International Organization for Standarization (ISO, por sus siglas en inglés) ha emitido normas MPEG que "son normas internacionales relacionadas con la compresión, descompresión, procesamiento y representación codificada de fotografías móviles, audio y su combinación".

ISO es el desarrollador mundial líder de normas internacionales. Para más información sobre normas de compresión actuales y otras normas industriales, le pedimos al lector visitar la página *http://www.iso.org*.

Conclusión

El administrador de archivos controla todos los archivos en el sistema y procesa los comandos del usuario (leer, escribir, modificar, crear, borrar, etc.) para interactuar con cualquier archivo en el sistema. También administra los procedimientos de control de acceso para mantener la integridad y la seguridad de los archivos bajo su control.

Para alcanzar sus objetivos, el administrador de archivos debe poder acomodar una variedad de organizaciones de archivos, esquemas de asignación de almacenamiento físico, tipos de registros y métodos de acceso. Y, como hemos visto, esto requiere software de administración de archivos cada vez más complicado.

En este capítulo hemos analizado:

- La organización de archivos secuencial, directa y secuencial indexada.
- La asignación de almacenamiento contiguo, no contiguo e indexado de archivos.
- Registros de longitud fija contra registros de longitud variable.
- Los tres métodos de control de acceso.
- Técnicas de compresión de datos.

Para obtener el máximo de un administrador de archivos, es importante que los usuarios conozcan los puntos fuertes y débiles de sus segmentos: cuáles métodos de acceso son permitidos sobre cuáles dispositivos y con qué estructuras de archivos, así como las ventajas y desventajas de cada una en eficiencia total.

Términos clave

Algoritmo *hashing*: Conjunto de instrucciones usadas para realizar una transformación clave donde un campo clave del registro determina su ubicación.

Almacenamiento contiguo: Tipo de almacenamiento de archivos donde toda la información se guarda en ubicaciones adyacentes en un medio de almacenamiento.

Almacenamiento no contiguo: Tipo de almacenamiento donde la información se guarda en ubicaciones no adyacentes en un medio de almacenamiento.

Archivo: Grupo de registros relacionados que contiene información que los programas de aplicación específicos usan para generar reportes.

Archivo de datos: Un archivo que contiene sólo datos.

Archivo de programa: Archivo que contiene instrucciones para la computadora.

Base de datos: Grupo de archivos relacionados interconectados a varios niveles para proporcionar a los usuarios flexibilidad de acceso a los datos almacenados.

Campo: Grupo de bytes relacionados que el usuario puede identificar con un nombre, tipo y tamaño. Un registro está compuesto por campos.

Campo clave: 1) Campo único o combinación de campos única en un registro que identifica de manera única ese registro; o 2) es el campo que determina la posición de un registro en una secuencia ordenada.

Compresión de datos: Procedimiento usado para reducir la cantidad de espacio requerido para almacenar datos al reducir, codificar o abreviar términos o caracteres repetitivos.

Descriptor de archivos: Información que se mantiene en el directorio para describir un archivo o su extensión

Dirección del byte actual (CBA): Dirección del último byte leído. Es usada por el administrador de archivos para tener acceso a registros en almacenamiento secundario y es necesario actualizarlo cada que se accede a un registro.

Dirección lógica: El resultado de una transformación clave.

Dirección relativa: En un entorno de organización directa, indica la posición de un registro en relación con el inicio de un archivo.

Directorio: Área de almacenamiento en un volumen de almacenamiento secundario (disco, plato de disco, etc.), que contiene información sobre los archivos almacenados en ese volumen.

Directorio actual: el directorio o subdirectorio en que está trabajando el usuario.

Directorio de trabajo: El directorio o subdirectorio donde trabaja actualmente el usuario.

Directorio maestro de archivos (MFD): Es un archivo almacenado inmediatamente después del descriptor de volumen. Enumera los nombres y las características de cada campo contenido en ese volumen.

Extensión: En algunos sistemas operativos, es la parte del nombre del archivo que indica el compilador o paquete de software necesario para ejecutar los archivos. En UNIX y Linux es opcional y se denomina *sufijo*.

Extensiones: Cualquier registro restante y todas las otras adiciones al archivo que están almacenadas en otras secciones del disco.

Independiente del dispositivo: Programas desprovistos de instrucciones detalladas necesarias para interactuar con cualquier dispositivo de E/S presente en el sistema de cómputo.

Lista de capacidad: Método de control de acceso que enumera cada usuario, los archivos a los que tiene acceso cada uno y el tipo de acceso que se permite a estos archivos.

Lista de control de acceso: Método de control que enumera cada archivo, los nombres de los usuarios a quienes se permite tener acceso al archivo, y el tipo de acceso que se permite a cada uno.

Matriz de control de acceso: Método de control que usa una matriz con cada archivo, cada usuario y el tipo de acceso que se permite a cada usuario para cada archivo.

Nombre absoluto del archivo: Nombre de un archivo, otorgado por el usuario, precedido por el directorio (o directorios) donde se encuentra el archivo y, cuando es necesario, la etiqueta del dispositivo específico.

Nombre relativo del archivo: Nombre y extensión de un archivo que lo distingue de otros archivos en el mismo directorio.

Organización de archivos directos: Archivos guardados en un dispositivo de almacenamiento con acceso directo y organizados para proporcionar a los usuarios la flexibilidad de tener acceso aleatorio a cualquier registro, sin importar su posición en el archivo.

Organización indexada secuencial de registros: Forma para organizar datos en un dispositivo de almacenamiento con acceso directo. El objetivo de crear un índice es mostrar dónde se almacenan los registros de datos. Cualquier registro de datos puede recuperarse al consultar primero el índice.

Organización secuencial de registros: Organización de registros en una secuencia específica. Los registros en un archivo secuencial deben procesarse uno después de otro.

Palabra de bloqueo: Secuencia de letras y/o números que se proporcionan a los usuarios para evitar manipulación peligrosa con sus archivos.

Registro: Grupo de archivos relacionados tratados como una unidad. Un archivo es un grupo de registros relacionados.

Registro de longitud fija: Registro que siempre contiene el mismo número de caracteres.

Registro de longitud variable: Registro que no es de longitud uniforme, no deja espacio de almacenamiento vacío y no trunca ningún caracter, eliminando así las dos desventajas de los registros de longitud fija.

Subdirectorio: Directorio creado por el usuario dentro de los límites de un directorio existente. Algunos sistemas operativos lo denominan carpeta.

Volumen: Cualquier unidad de almacenamiento secundario, como discos duros, paquetes de discos, CD, DVD, discos removibles o cintas.

Búsquedas de interés

- Almacenamiento de archivos en internet (cloud computing)
- Políticas de respaldo de archivos
- Técnicas de compresión de archivos
- Auditoría de archivos de acceso
- Limitaciones del nombre de archivos
- Función Hash

Ejercicios

Temas de investigación

A. Investigue el tamaño del software del sistema operativo al encontrar la cantidad de espacio de almacenamiento secundario (disco) requerido por versiones diferentes del mismo sistema operativo o diferentes sistemas operativos. Si los tamaños son sustancialmente diferentes, explique por qué ocurre esto, como cuestiones de plataformas, características, etc. Cite sus fuentes.

B. Consulte literatura actual o internet para investigar convenciones de nomenclatura de archivos para cuatro sistemas operativos diferentes (sin incluir UNIX, MS-DOS, Windows o Linux). Observe el intervalo aceptable de caracteres, la longitud máxima la sensibilidad al tipo de letra, etc. Proporcione ejemplos de nombres de archivos aceptables e inaceptables. Para crédito extra, explique cómo el administrador de archivos de estos sistemas operativos acorta nombres de archivos largos (en caso de hacerlo) en sus listas internas para facilitar su manipulación. Cite sus fuentes.

Ejercicios

1. Con sus propias palabras, explique por qué la desubicación de archivos es importante y qué puede suceder si esto no ocurre en una base regular.
2. Describa cómo el administrador de archivos ubica un archivo para un usuario único. Enliste los pasos que usted piensa que deben seguirse y explique su razonamiento.
3. ¿La independencia del dispositivo es importante para el administrador de archivos? ¿Por qué si? ¿Por qué no? Describa las consecuencias si este no fuera el caso.
4. ¿Piensa que la recuperación de archivos es diferente en un sistema guiado por menús y en uno guiado por comandos? Explique su respuesta y describa las diferencias entre los dos. Proporcione un ejemplo de cuándo es preferible uno sobre el otro.
5. Imagine un ejemplo real para cada uno: un volumen de múltiarchivo y un archivo multivolumen. Incluya una descripción del medio utilizado para almacenamiento y una descripción general de los datos en el archivo.
6. Como se describió en este capítulo, los archivos pueden formatearse con campos de longitud fija o de longitud variable. En su opinión, ¿es posible combinar ambos formatos en un solo disco? Explique las razones de su respuesta.
7. Explique por qué es difícil soportar accesos directos a archivos con registros de longitud variable. Sugiera un método para manejar este tipo de archivo si se requiere un acceso directo.
8. Proporcione un ejemplo de los nombres de tres archivos de su propia computadora que no residan en el directorio maestro o raíz. Para cada archivo, liste su nombre relativo y su nombre completo.
9. En sus propias palabras, describa el propósito del directorio de trabajo y cómo puede acceder de forma rápida o lenta a los archivos. En su opinión, ¿puede existir más de un directorio de trabajo? Explique.

10. Para cada uno de los siguientes elementos en una matriz de control de acceso para User 2010, proporcione el tipo de acceso permitido para cada uno de los siguientes archivos:

    a. `Archivo1` `--E-`

    b. `Archivo2` `RWED`

    c. `Archivo3` `R-E-`

    d. `Archivo4` `R---`

11. Para cada uno de los elementos en una lista de control, proporcione el tipo de acceso permitido para cada usuario del archivo221 y describa quién está incluido en la categoría WORLD:

    a. `User2010` `R-E-`

    b. `User2014` `-E-`

    c. `User2017` `RWED`

    d. `WORLD` `R---`

12. Ideé una forma de comprimir la siguiente lista de nombres utilizando una técnica fuerte similar a la mostrada en la tabla 8.6. Describa su método y muestre la lista comprimida. Explique por qué su técnica es fuerte (y no débil).

    POWE

    POWELL

    POWER

    POWERS

    PORUN

Ejercicios avanzados

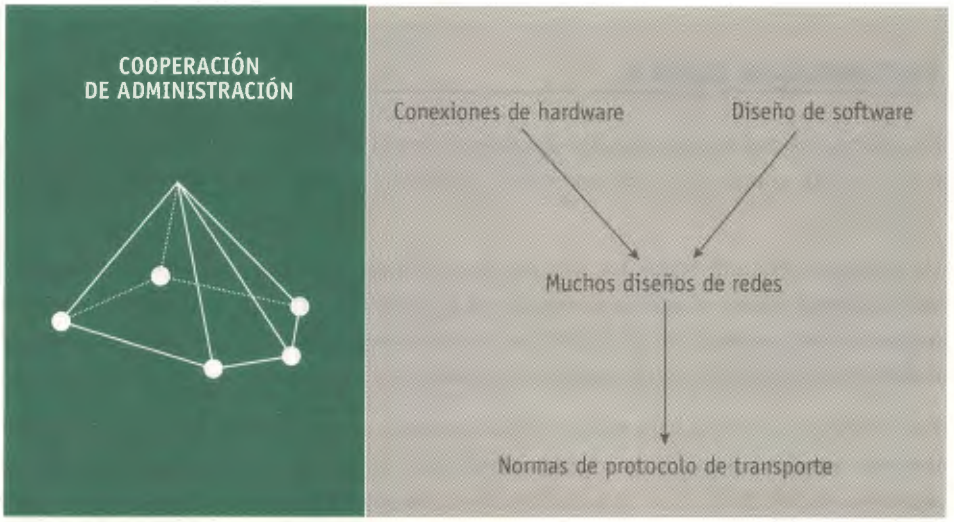
13. Si usted está diseñando el control de acceso a archivos para un entorno altamente seguro y pudiera escoger entre el establecimiento de muchas categorías de acceso y apenas unas cuantas, ¿qué escogería y por qué?

14. Compare y contraste asignación de memoria dinámica y la asignación de archivos en almacenamiento secundario.

15. ¿Cuándo la compactación del almacenamiento secundario es benéfica desde la perspectiva del administrador de archivos? Proporcione varios ejemplos. Enumere algunos problemas que podrían presentarse como resultado de la compactación y explique cómo podrían evitarse.

16. Aunque administradores de archivos sofisticados implementan la compartición de archivos al permitir que varios usuarios tengan acceso a una sola copia de un archivo al mismo tiempo, otros implementan la compartición de archivos al proporcionar una copia del archivo a cada usuario. Enumere las ventajas y desventajas de cada método.

Capítulo 9 Conceptos de organización de redes



"El conocimiento del universo es de alguna forma. . . defectuoso, sin ningún resultado práctico."

Cicerón (*106-43 a.C.*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Varias topologías de red diferentes, incluyendo la de estrella, de anillo e híbrida, y cómo conectan numerosos anfitriones a la red.
- Varios tipos de redes: LAN, MAN, WAN y LAN inalámbrica.
- La diferencia entre conmutación de circuitos y conmutación de paquetes, y ejemplos del uso cotidiano a favor de cada uno.
- Procedimientos de resolución de conflictos que permiten que una red comparta hardware y software de transmisión común de manera efectiva.
- Los dos modelos de protocolo de transporte (OSI y TCP/IP) y cómo se comparan las capas de cada uno.

Cuando las instalaciones informáticas de una computadora se conectan entre sí mediante componentes de comunicación de datos, constituyen una red de recursos automatizados en apoyo de muchas de las funciones de una organización. Las redes constituyen una infraestructura esencial para miembros de la comunidad basada en la información para procesar, manipular y distribuir datos e información entre sí. Este capítulo presenta la terminología y los conceptos básicos de redes.

Terminología básica

En general, una **red** es una colección de procesadores débilmente acoplados interconectados por enlaces de comunicación usando cables, tecnología inalámbrica, o una combinación de ambas cosas.

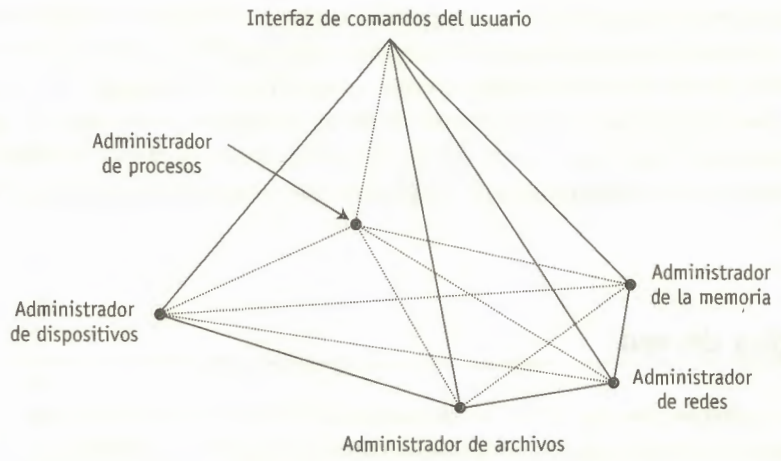
Un objetivo común de todos los sistemas de red es proporcionar una forma conveniente para compartir recursos mientras controlan el acceso de los usuarios a éstos. Estos recursos incluyen tanto hardware (como la CPU, la memoria, impresoras, puertos USB y unidades de disco) como software (como programas de aplicación y archivos de datos).

Para sistemas operativos para redes hay dos configuraciones generales. La más antigua añadió una capacidad de red a un sistema operativo de usuario único. Se denominaba **sistema operativo de red (NOS)**. Con esta configuración, los usuarios estaban enterados del surtido específico de computadoras y recursos en la red y podían tener acceso a ellos al registrarse en el anfitrión remoto idóneo o al transferir datos desde la computadora remota hacia la propia.

Con la segunda configuración, los usuarios no necesitan saber dónde y cómo cada máquina está conectada al sistema; podían tener acceso a recursos remotos como si se tratase de recursos locales. Un **sistema operativo distribuido (D/OS)** proporciona buen control para sistemas de cómputo distribuidos y permite tener acceso a sus recursos de forma unificada. Un sistema operativo distribuido representa una visión total a través de sistemas de cómputo múltiples para manejar y administrar recursos sin dependencias locales. La administración es un proceso cooperativo que abarca todos los recursos e implica todos los sitios.

Un sistema operativo distribuido está compuesto por los mismos cuatro administradores ya analizados, aunque con un alcance más amplio. Por lo menos, debe contar con las siguientes componentes: administración de procesos o de objetos, administración de la memoria, administración de archivos, administración de dispositivos y administración de redes, como se muestra en la figura 9.1. Un sistema operativo distribuido ofrece varias ventajas importantes sobre antiguos sistemas operativos y NOS, incluyendo compartición fácil y confiable de recursos, cómputos más rápidos, equilibrio de carga idóneo, buena confiabilidad y comunicaciones electrónicamente dependientes entre los usuarios de la red.

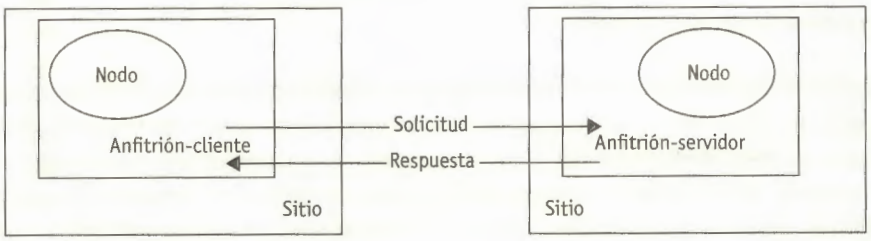
En un sistema operativo distribuido, cada procesador clasifica a los otros procesadores y sus recursos como **remotos** y considera sus propios recursos como **locales**. El tamaño, tipo e identificación de los procesadores varían. Los procesadores se denominan **sitios, anfitriones**



(Figura 9.1)
Esta pirámide de cinco lados ilustra gráficamente la forma en que los cinco administradores en un sistema de red trabajan juntos y atienden la interfaz de comandos del usuario.

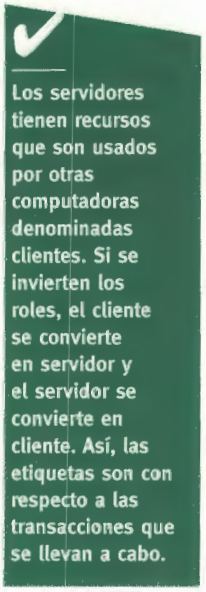
y **nodos**, dependiendo del contexto en que se mencionen. En general, el término "sitio" indica una ubicación específica en una red que contiene uno o más sistemas de cómputo, "anfitrión" indica un sistema de cómputo específico que se encuentra en un sitio cuyos servicios y recursos pueden usarse desde ubicaciones remotas, y "nodo" (o, más formalmente, "nombre de nodo") se refiere al nombre asignado a un sistema de cómputo conectado a una red a fin de identificarlo de otras computadoras en la red, lo cual se muestra en la figura 9.2.

En términos generales, un anfitrión en un sitio, denominado servidor, tiene recursos que un anfitrión en otro sitio, denominado cliente, desea utilizar. Pero las asignaciones no son estáticas. Los anfitriones pueden alternar entre clientes o servidores, dependiendo de sus requerimientos.



(Figura 9.2)
Los clientes solicitan datos o servicios del anfitrión servidor y esperan la respuesta. Si el anfitrión cliente tiene los recursos necesarios por el anfitrión servidor, los roles pueden invertirse.

En este análisis hemos simplificado los conceptos de cliente y servidor para representar el software de red estático que está ubicado en dos sitios diferentes. Es decir, hemos asumido que el anfitrión-cliente es un cliente todo el tiempo y que el anfitrión-servidor siempre es un servidor. Sin embargo, los roles verdaderos de cliente y servidor pueden alternar entre dos anfitriones de red, dependiendo de la aplicación y configuración de la red. En este texto hemos escogido usar la configuración más simple para que nuestra explicación sea lo más clara posible.



Topologías de red

Los sitios en cualquier sistema de red pueden conectarse física o lógicamente a otros en cierta topología, la disposición geométrica de las conexiones (cables, inalámbrica o ambas cosas) que vincula nodos. Las disposiciones geométricas más comunes son en estrella, en anillo, de bus, de árbol e híbrida. En cada topología hay negociaciones entre la necesidad por una comunicación rápida entre todos los sitios, la tolerancia de fallos en un sitio o enlace de comunicación, el costo de líneas de comunicación largas y la dificultad de conectar un sitio a un gran número de otros sitios. Es importante observar que la topología física de una red puede no reflejar su topología lógica. Por ejemplo, una red que está conectada en una configuración en estrella puede disponerse para operar lógicamente como si fuese un anillo. Es decir, puede hacerse que manipule un token en una manera semejante a un anillo aunque sus cables estén dispuestos en una topología en estrella. Para que nuestra explicación en este capítulo sea lo más simple posible, siempre que analicemos topologías supondremos que la estructura lógica de la red es idéntica a la estructura física.

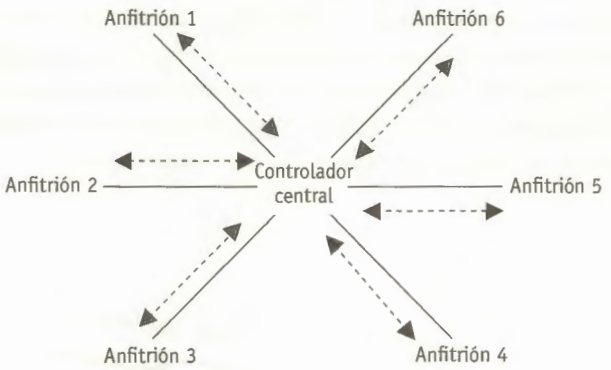
Para el diseñador de redes hay muchas alternativas disponibles, de las cuales quizá todas cumplen los requerimientos de los clientes. Al decidir cuál configuración usar, los diseñadores deben tomar en cuenta cuatro criterios:

- Costo básico: los gastos requeridos para enlazar los diversos sitios en el sistema.
- Costo de comunicaciones: el tiempo necesario para enviar un mensaje de un sitio a otro.
- Confiabilidad: la seguridad de que muchos sitios pueden seguir comunicándose entre sí incluso si fallan un enlace o el sistema.
- Entorno del usuario: los parámetros críticos que deben cumplir la red para que sea una inversión de negocios exitosa.

Es muy posible que para cada cliente haya varias soluciones probables. La mejor elección consiste en considerar los cuatro criterios. Por ejemplo, una compañía internacional de recuperación de datos podría considerar que las comunicaciones más rápidas y la configuración de hardware más flexible constituyen una inversión rentable. Pero un centro de procesamiento de palabras vecino podría recalcar el hecho de contar con una solución de red de bajo costo. Sobredimensionar la ingeniería del sistema vecino podría ser tan equivocado como subestimar la red de los clientes internacionales. La clave para escoger el mejor diseño consiste en comprender la tecnología disponible, así como los requerimientos y presupuesto de los negocios de los clientes.

En estrella

Una **topología en estrella**, algunas veces denominada concentrador o topología centralizada, es un enfoque tradicional para interconectar dispositivos donde los datos transmitidos deben pasar por un controlador central en su recorrido de un emisor a un receptor, como se muestra en la figura 9.3.



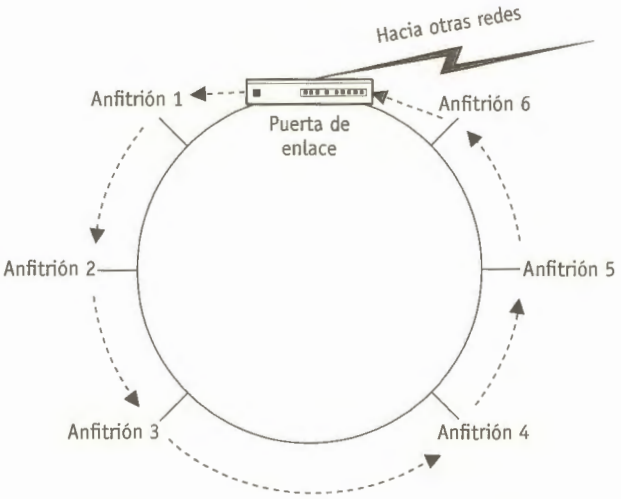
(Figura 9.3)

Topología en estrella. Los anfitriones se conectan entre sí mediante un controlador central, que asume toda la responsabilidad de encaminar los mensajes al anfitrión idóneo. El flujo de datos entre los anfitriones y el controlador central se representa con líneas punteadas. No se permite la comunicación anfitrión-anfitrión.

La topología en estrella permite el encaminamiento fácil, ya que la estación central conoce la ruta de todos los otros sitios, y porque hay un punto central, el acceso a la red puede controlarse fácilmente y es posible asignar un estado prioritario a los sitios escogidos. Sin embargo, esta centralización del control requiere que el sitio central sea extremadamente confiable y capaz de manejar todo el tráfico en la red, sin importar cuán pesado sea.

Topología de anillo

En la **topología de anillo** todos los sitios están conectados en un bucle cerrado, como se muestra en la figura 9.4, donde el primer sitio está conectado con el último.

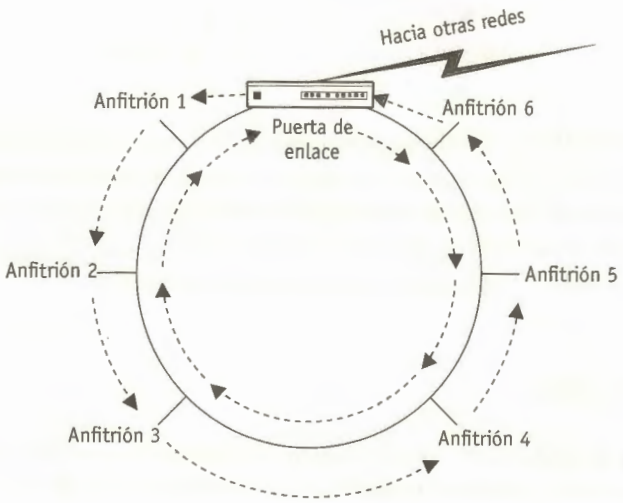


(Figura 9.4)

Topología de anillo. Los anfitriones están conectados entre sí de manera circular sólo en una dirección, la cual se muestra con líneas punteadas. La red puede conectarse con otra red mediante un puente o puerta de enlace.

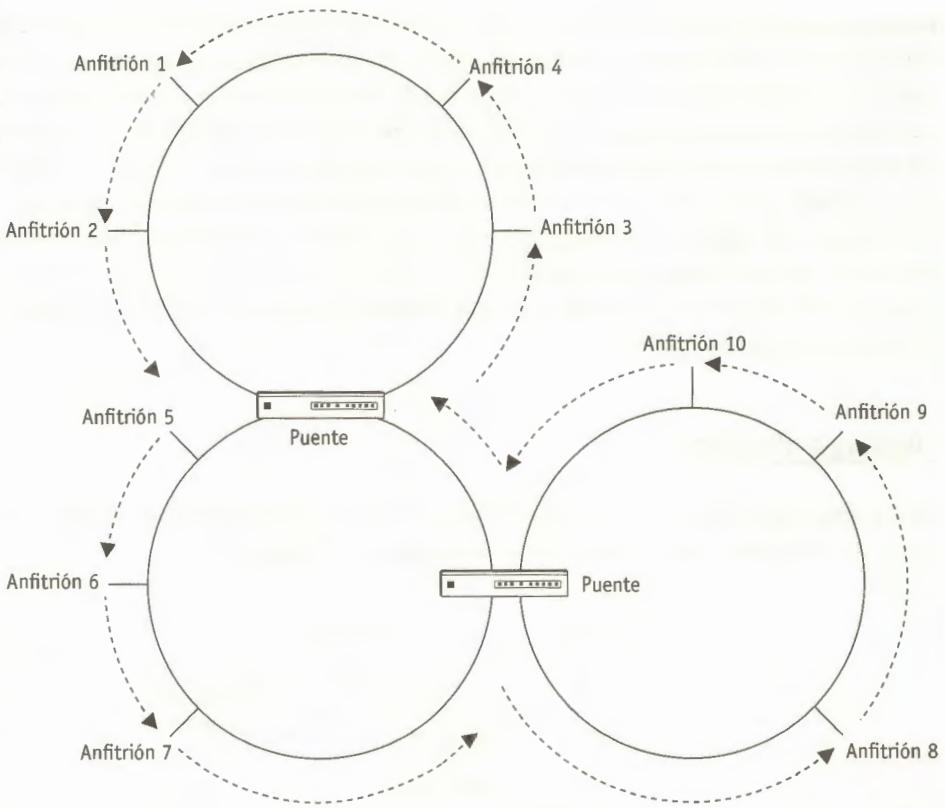
En un anillo, la red puede conectarse con otras redes mediante un puente o una puerta de enlace, dependiendo del **protocolo** usado por cada red. (El protocolo es el conjunto específico de reglas usadas para controlar el flujo de los mensajes a través de la red.) Si la otra red tiene el mismo protocolo, entonces para conectar las redes se usa un puente. Si la otra red tiene un protocolo diferente, entonces para conectar las redes se usa una puerta de enlace.

Los datos se transmiten en paquetes que también contienen campos fuente y destino. Cada paquete pasa de nodo en nodo sólo en una dirección, y la estación destino copia los datos en un *búfer* local. En general, el paquete continúa circulando hasta que regresa a la estación fuente, donde se elimina del anillo. Hay algunas variantes de esta topología básica, como la red de bucle doble que se muestra en la figura 9.5, y un conjunto de múltiples anillos puenteados entre sí, como se muestra en la figura 9.6, pero tienen su costo.



(Figura 9.5)

Red informática de bucle doble usando una topología de anillo. Los paquetes de datos fluyen en ambas direcciones.

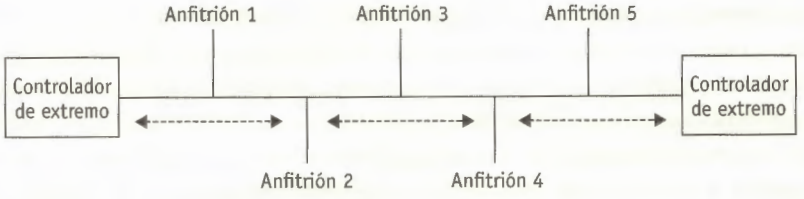


(Figura 9.6)
Múltiples anillos puenteados entre sí. Tres anillos conectados entre sí mediante dos puentes. Esta variante de la topología de anillo permite enlazar entre sí varias redes con el mismo protocolo.

Aunque las topologías de anillo comparten la desventaja de que cada nodo debe ser funcional para que una red funcione adecuadamente, los anillos pueden diseñarse de modo que sea posible evitar nodos con fallos, una consideración crítica para la estabilidad de la red.

Topología de bus

En la **topología de bus** todos los sitios están conectados a una sola línea de comunicación que recorre la red a lo largo, como se muestra en la figura 9.7.



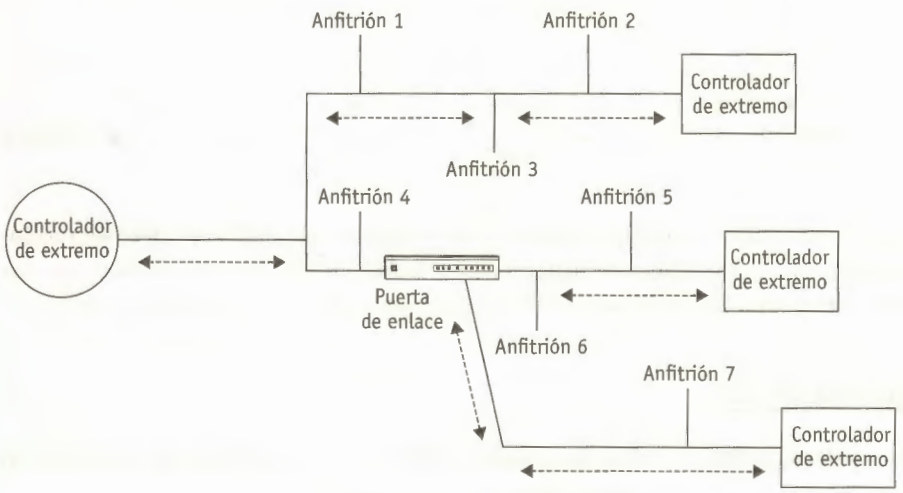
(Figura 9.7)
Topología de bus. Los anfitriones están conectados entre sí de manera lineal. Los datos fluyen en ambas direcciones de anfitrión a anfitrión y se invierten cuando llegan a un controlador de extremo.

Los dispositivos se conectan físicamente por medio de cables que corren entre ellos, pero sin pasar por un mecanismo de controlador centralizado. Los mensajes desde cualquier sitio circulan en ambas direcciones a través de toda la línea de comunicación y pueden ser recibidos por todos los otros sitios.

Debido a que todos los sitios comparten una línea de comunicación, sólo uno de ellos puede enviar mensajes exitosamente en cualquier momento. En este entorno, los datos pueden pasar directamente de un dispositivo a otro, o pueden encaminarse a un controlador de extremo que está al final de la línea. En un bus, si los datos llegan a un controlador de extremo sin haber sido aceptados por un anfitrión, el controlador de extremo los devuelve de modo que el mensaje pueda ser aceptado por el nodo idóneo en el camino hacia el otro controlador de extremo. Con algunos buses cada mensaje siempre debe ir al final de la línea antes de recorrer la línea de comunicación hacia el nodo al que ha sido direccionado. Sin embargo, otras redes de bus permiten el envío directo de mensajes hacia el nodo objetivo sin llegar a ningún controlador de extremo.

Topología de árbol

La **topología de árbol** es una colección de buses. La línea de comunicación es un cable que se ramifica sin ningún bucle cerrado, como se muestra en la figura 9.8.



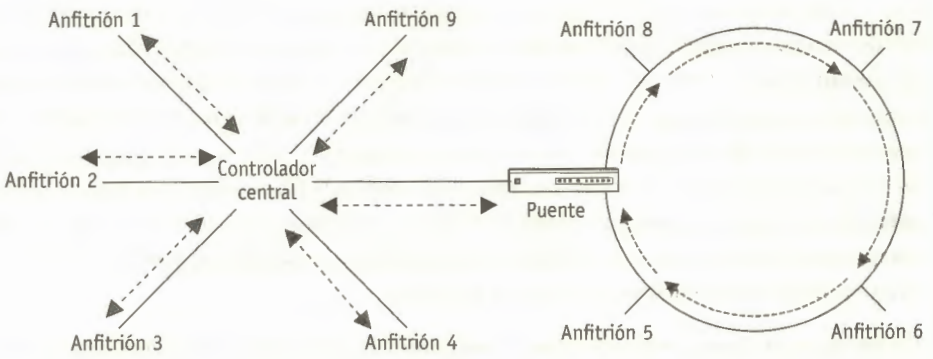
(Figura 9.8)

Topología de árbol. Los datos fluyen entre las ramas de los árboles y son absorbidos por controladores en los extremos. Las puertas de enlace ayudan a minimizar diferencias entre un protocolo utilizado en una parte de la red y el protocolo diferente utilizado en la rama con el anfitrión 7.

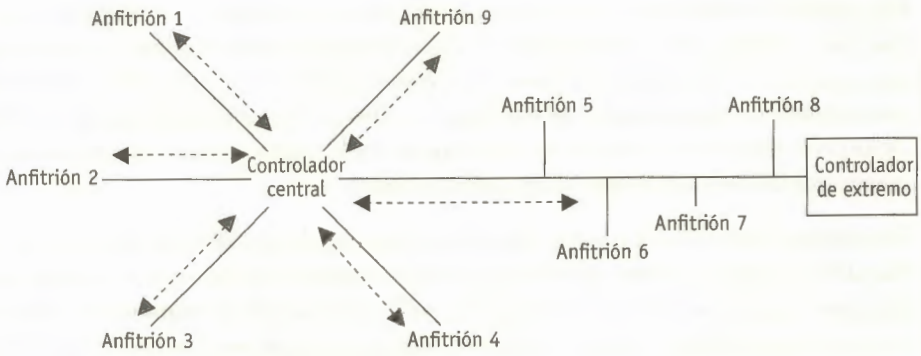
Las tres disposiciones empiezan en el extremo superior, donde comienzan uno o más cables. Cada cable puede tener ramificaciones que a su vez pueden tener ramificaciones adicionales, potencialmente resultando en disposiciones bastante complicadas. El uso de puentes como filtros especiales entre buses del mismo protocolo y como traductores hacia aquellos con protocolos distintos permite a los diseñadores crear redes que pueden operar a velocidades más sensibles a los anfitriones en la red. En una configuración de árbol el mensaje desde cualquier sitio circula a través de la línea de comunicación y puede recibirse en cualquiera de los otros sitios hasta que llega a los extremos. Si un mensaje llega a un controlador de extremo sin haber sido aceptado por ningún anfitrión, el controlador de extremo lo absorbe; se devuelve como cuando se usa una topología de bus. Una ventaja de las topologías de bus y de árbol es que incluso si falla un simple nodo, el tráfico del mensaje puede seguir fluyendo por la red.

Topología híbrida

Una **topología híbrida** es una combinación de cualquiera de las cuatro topologías analizadas aquí. Por ejemplo, una topología híbrida puede hacerse al sustituir un simple anfitrión en una configuración en estrella por un anillo, como se muestra en la figura 9.9. O una configuración en estrella puede tener una topología de bus como una de las líneas de comunicación que alimentan su concentrador, como se muestra en la figura 9.10.



(Figura 9.9)
Topología híbrida, versión 1. Esta red combina una estrella y un anillo conectados mediante un puente. Los anfitriones 5, 6, 7 y 8 están ubicados sobre el anillo.



(Figura 9.10)
Topología híbrida, versión 2. Esta red combina topologías en estrella y de bus. Los anfitriones 5, 6, 7 y 8 están ubicados sobre el bus.

El objetivo de una configuración híbrida es escoger entre los puntos fuertes de cada topología y combinarlos para satisfacer de la manera más efectiva los requerimientos de comunicaciones de ese sistema.

Tipos de redes

A menudo es de utilidad agrupar redes según las distancias físicas que cubren. Aunque las características que definen a cada grupo se están volviendo cada vez más difusas a medida que avanza la tecnología de la comunicación, las redes suelen dividirse en redes de área local, de área metropolitana y de área amplia. En años recientes, se ha vuelto cada vez más popular un tipo de red de área local: la red de área local inalámbrica.

Red de área local

Una **red de área local** (LAN) define una configuración encontrada dentro de un solo edificio de oficinas, almacén, hogar o entorno informático semejante. Este tipo de red suele ser propiedad, usada y operada por una sola organización y permite que las computadoras personales se comuniquen directamente a través de una línea de comunicación común. En general, se trata de un grupo de computadoras personales o estaciones de trabajo ubicadas en la misma área general. Aunque una LAN está confinada físicamente a un área local bien definida, sus comunicaciones no están limitadas a esa área porque la LAN puede ser una componente de una red de comunicación más grande que puede proporcionar fácil acceso al exterior mediante un puente o una puerta de enlace.

Un **puente** es un dispositivo, y el software para operarlo, que conecta dos o más redes de área local geográficamente distantes que usan los mismos protocolos. Por ejemplo, un solo puente puede usarse para conectar dos redes de área local que usan la tecnología de redes Ethernet, que se analizará más tarde en este capítulo.

Una **puerta de enlace**, por otra parte, es un dispositivo y software más complicados usados para conectar dos o más redes de área local o sistemas geográficamente distantes que usan protocolos diferentes. Una **puerta de enlace** traduce un protocolo de la red en otro, resolviendo incompatibilidades de hardware y software. Por ejemplo, la **puerta de enlace** de arquitectura de red (denominada comúnmente SNA), puede conectar una red de microcomputadoras con una computadora central anfitrión.

Redes de área local de alta velocidad tienen una tasa transmisión de datos que va desde 100 megabits por segundo hasta más de 40 gigabits por segundo. Debido a que los sitios están próximos entre sí, hay anchos de banda disponibles para atender la transmisión a muy alta velocidad para gráficas y video a todo color, animadas completamente, transmisiones digitales de voz y otras señales analógicas o digitales de tasa alta. Las topologías previamente descritas (en estrella, de anillo, de bus e híbrida) suelen usarse para construir redes de área local. El medio de transmisión usado puede variar de una topología a otra. Algunos factores que es necesario considerar al escoger un medio de transmisión son costo, tasa de transmisión de datos, confiabilidad, número de dispositivos que es posible atender, distancia entre las unidades y limitaciones técnicas.

Red de área metropolitana

Una **red de área metropolitana** (**MAN**), define una configuración que genera un área más grande que una LAN, que varía desde varias manzanas hasta toda una ciudad pero sin exceder una circunferencia de 100 kilómetros. Una red de área metropolitana puede ser propiedad y ser operada por una sola organización y suele ser usada por muchas personas y organizaciones. En algunos casos, las redes de área metropolitana son propiedad y son operadas como utilidades públicas, proporcionando los medios para interconectar en red varias redes de área local.

Una MAN es una red de alta velocidad configurada a menudo como un anillo lógico. Eso significa que dependiendo del anillo usado en el protocolo, los mensajes se transmiten en una dirección usando un solo anillo, como se ilustra en la figura 9.4, o en ambas direcciones usando dos anillos que giran en direcciones opuestas, como se ilustra en la figura 9.5. Un anillo siempre transporta los mensajes en una dirección y el otro siempre lo hace en dirección opuesta.

Red de área amplia

Internet es una red de área amplia a la que es posible acceder desde redes de área local y redes de área metropolitana.

Una **red de área amplia** (**WAN**) define una configuración que interconecta facilidades de comunicación en partes distintas de un país e incluso en partes diferentes del mundo, o que es operada como parte de una utilidad pública. Las redes de área amplia usan las líneas de comunicación de portadores comunes, que son empresas privadas reguladas por el gobierno, como compañías telefónicas que ya proporcionan facilidades de comunicación al público en general. Las redes de área amplia usan una amplia gama de medios de comunicación, incluyendo satélite y microondas; en algunos casos, la velocidad de transmisión está limitada por las capacidades de la línea de comunicación. En general, las redes de área amplia son más lentas que las redes de área local.

La primera red de área amplia, ARPAnet, fue desarrollada en 1969 por la Advanced Research Projects Agency (ARPA); la responsabilidad de su operación fue transferida en 1975 a la Defense Communications Agency. Su sucesora: internet, es la red de área amplia más reconocida, aunque existen otras redes de área amplia comerciales.

Red de área local inalámbrica

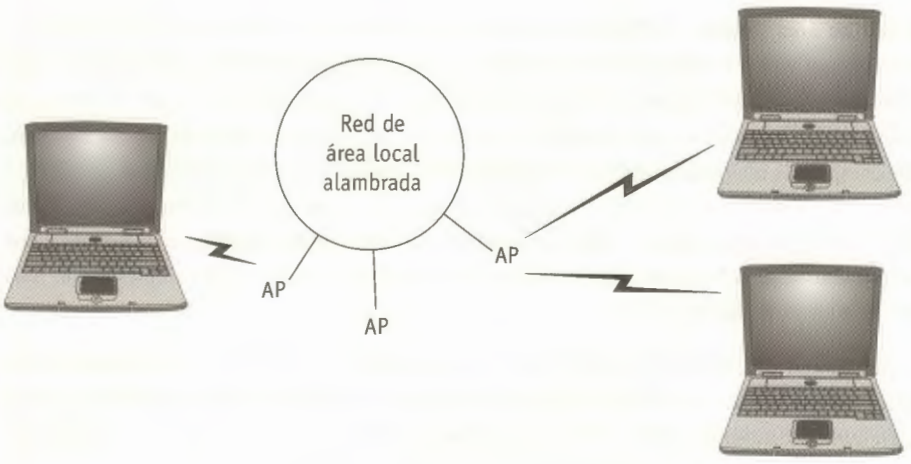
Una **red de área local inalámbrica** (**WLAN**) es una red de área local que usa tecnología inalámbrica para conectar computadoras o estaciones de trabajo dentro del alcance de la red. Como se muestra en la tabla 9.1, el Institute for Electrical and Electronics Enginners (IEEE) ha especificado varias normas para redes inalámbricas, cada una con un alcance diferente.

Norma IEEE	Velocidad	Alcance	Frecuencia	Compatibilidad
802.11a	54 Mbps	25-75 pies en interior	5 GHz	Primera norma IEEE inalámbrica
802.11b	11 Mbps	100-150 pies en interior	2.4 GHz	No es compatible con 802.11g
802.11g	54 Mbps	100-150 pies en interior	2.4 GHz	Compatible con 802.11b
802.11n	600 Mbps	300 pies en interior	5 GHz	Compatible con 802.11g

(Tabla 9.1)

Comparación de tres normas IEEE para redes inalámbricas (IEEE, 2007). La cuarta norma, 802.11n, se supone que será publicada en noviembre de 2009.

Para nodos inalámbricos (estaciones de trabajo, laptops, etc.), una red de área local inalámbrica puede proporcionar fácil acceso a una red más grande o a internet, como se muestra en la figura 9.11. Debe tomarse en cuenta que en general una red de área local inalámbrica presenta vulnerabilidades de seguridad debido a su estructura abierta y la dificultad inherente de mantener fuera intrusos no autorizados.



(Figura 9.11)

En una red de área local inalámbrica, nodos inalámbricos habilitados conectan la LAN con cable mediante puntos de acceso (AP) si están localizados dentro del alcance de los dispositivos que envían la señal.

La norma IEEE Mobile WiMAX (802.16), aprobada en 2005 por el IEEE, promete entregar datos por ancho de banda alto sobre distancias mucho más largas (hasta diez millas) que la norma actual WiFi (IEEE, 2007). Este es un tema que cambia muy rápido, por lo que alentamos al lector para que investigue en la literatura actual los últimos desarrollos.

Cuestiones de diseño de software

Hasta el momento hemos analizado las configuraciones de las componentes del hardware de una red. En esta sección analizaremos cuatro cuestiones relacionadas con el software que deben abordar los diseñadores de redes:

- ¿Cómo usan los sitios las direcciones para ubicar otros sitios?
- ¿Cómo se encaminan los mensajes y cómo se envían?
- ¿Cómo se comunican los procesos entre sí?
- ¿Cómo se resuelven las demandas conflictivas por recursos?

Convenciones de direccionamiento

Los sitios en las redes necesitan determinar cómo identificar de manera única a sus usuarios, de modo que puedan comunicarse entre sí y tener acceso a los recursos de los demás. Se requieren nombres, direcciones y rutas porque los sitios no están conectados directamente entre sí, excepto sobre enlaces de punto a punto; en consecuencia, los protocolos de direccionamiento están relacionados estrechamente con la topología de la red y con la ubicación geográfica de cada sitio. En algunos casos se hace una distinción entre un "nombre local", que se refiere al nombre por el que se conoce una unidad dentro de su propio sistema, y un "nombre global", el nombre por el que se conoce una unidad fuera de su propio sistema. Esta distinción es de utilidad porque permite a cada sitio la libertad de identificar sus unidades según sus propias normas sin imponer reglas de nombrado uniformes, algo que sería difícil de imponer a nivel local. Por otra parte, un nombre global debe obedecer longitudes y formatos normales, así como otras convenciones generales.

Usando como ejemplo típico una dirección de internet, puede verse que sigue una organización jerárquica, empezando de izquierda a derecha en la siguiente secuencia: desde el usuario lógico hasta la máquina anfitriona, de la máquina anfitriona hasta la máquina siguiente, de la máquina siguiente hasta el *cluster* y de éste a la red. Por ejemplo, en cada dirección de internet: *someone@icarus.lis.pitt.edu* o *igss12@aber.ac.uk,* los puntos se usan para separar cada componente. Estas direcciones de correo electrónico, que son muy fáciles de recordar, deben traducirse (o "resolverse", usando un concepto muy semejante al descrito en el capítulo 3) a direcciones de hardware correspondientes. Esta conversión la lleva a cabo la sección de red del sistema operativo de la computadora.

Los ejemplos anteriores siguen el protocolo **sistema de nombre de dominio** (DNS), un servicio de solicitud de datos distribuidos de propósito general cuya función principal es la resolución de direcciones de internet. Si la dirección *someone@icarus.lis.pitt.edu* se disecciona en sus componentes, se obtiene lo siguiente:

- *someone* es el usuario lógico,
- *icarus* es el anfitrión del usuario llamado someone,
- *lis* es la máquina de red para icarus,

- *pitt* es el cluster para *lis*, y
- *edu* es la red para la universidad de Pittsburgh.

No es necesario que en todas las direcciones de internet aparezcan todas las componentes. No obstante, el DNS es capaz de resolverlas al examinar cada una en orden inverso.

Estrategias de encaminamiento

Un **encaminador** es un dispositivo de red, principalmente controlado por un software que dirige el tráfico entre dos tipos diferentes de LAN o dos segmentos de redes con direcciones de protocolo. Opera a nivel de capa, lo cual se explicará después en este capítulo.

El encaminamiento permite que los datos vayan de un punto a otro sobre una red. Una vez que los datos están en la red idónea, el encaminador se asegura que los reciba el nodo correcto en la red. El rol de los encaminadores cambia en la medida que se modifica el diseño de la red. Los encaminadores se usan ampliamente para conectar sitios entre sí y al internet. También pueden usarse ampliamente para varias funciones, incluyendo el aseguramiento de la información generada en áreas predefinidas, escogiendo la ruta más rápida de un punto a otro y proporcionando conexiones de red redundantes, de modo que un problema en un área no degrade el funcionamiento de la red en otras áreas.

Los protocolos de encaminamiento deben considerar direccionamiento, resolución de direcciones, formato del mensaje y reporte de errores. La mayor parte de los protocolos de encaminamiento se basan en un formato de direccionamiento que usa una red y un número de nodo para identificar cada nodo. Cuando una red está funcionando, cada encaminador registra en una tabla la dirección de las redes que están directamente conectadas. Debido a que los protocolos de encaminamiento permiten interacción entre encaminadores, se vuelve fácil compartir destinos de red que cada encaminador puede haber adquirido cuando efectúa sus servicios. A intervalos específicos, cada encaminador en la red transmite una copia de toda su tabla de encaminamiento. Finalmente, todos los encaminadores saben cómo llegar a cada una de las diferentes red destino.

Aunque las direcciones permiten a los encaminadores enviar datos de una red a otra, no es posible usarlos para ir de un punto a otro de la red en la *misma* red. Esto debe hacerse mediante la resolución de dirección, que permite que un encaminador asigne la dirección original a una dirección de hardware y almacene la asignación en una tabla a utilizar para transmisiones futuras.

Los protocolos de encaminamiento han definido una variedad de formatos de mensajes. Estos mensajes se usan para permitir que el protocolo realice sus funciones, como encontrar nuevos nodos sobre una red, probar para determinar si están funcionando, reportar condiciones de error, intercambiar información de encaminamiento, establecer conexiones y transmitir datos.

La transmisión de datos no siempre funciona en forma suave. Por ejemplo, pueden presentarse condiciones que ocasionen errores como la incapacidad de alcanzar un destino debido a un nodo o red averiados. En estos casos, las rutas y los protocolos de encaminamiento deben reportar las condiciones de error; la corrección de los errores se deja a los protocolos a otros niveles de la arquitectura de la red.

Dos de los protocolos de encaminamiento de mayor uso en internet son el protocolo de encaminamiento de información y el protocolo de encaminamiento jerárquico de puerta de enlace interior (OSPF, por sus siglas en inglés).

Protocolo de encaminamiento de información

En el protocolo de **encaminamiento de información** (**RIP**), la selección de una ruta para transferir datos de una red a otra se basa en el número de nodos intermedios, o brincos, entre la fuente y el destino. Siempre se escoge la ruta con el menor número de brincos. Este algoritmo vector distancia es fácil de implementar, aunque puede no ser el mejor en el entorno actual de redes, ya que no toma en consideración otros factores importantes como ancho de banda, prioridad de los datos o tipo de red. Es decir, puede excluir la selección de rutas más rápidas o más confiables simplemente porque tienen más brincos. Otra limitación de RIP está relacionada con las tablas de encaminamiento. Toda la tabla se actualiza y reemite cada 30 segundos, sin importar que hayan ocurrido cambios o no; esto incrementa el tráfico en la red y afecta negativamente la entrega de los mensajes. Además, la tabla se propaga de un encaminador a otro. Así, en el caso de una interred con 15 brincos, serían necesarios más de 15 segundos para conocer un cambio ocurrido en el otro extremo de la interred. Debido a que no todos los encaminadores poseen la misma información sobre la interred, un fallo en cualquiera de los brincos podría crear un entorno inestable para todo el tráfico de mensajes.

Protocolo de encaminamiento jerárquico de puerta de enlace interior

En el **protocolo de encaminamiento jerárquico de puerta de enlace interior** (**OSPF**), la selección de una ruta de transmisión se realiza sólo después que se ha determinado el estado de la red, de modo que si un brinco intermedio está averiado, se elimina de inmediato de la consideración hasta que se restablecen sus servicios. Los mensajes de actualización de encaminamiento se envían sólo cuando ocurren cambios en el entorno de encaminamiento, reduciendo así el número de mensajes en la interred y reduciendo el tamaño de los mensajes al no enviar toda la tabla de encaminamiento. Sin embargo, el uso de la memoria aumenta porque OSPF sigue la pista de más información que RIP. Además, los ahorros en consumo de ancho de banda son compensados por el mayor uso de la CPU necesario para el cálculo de la ruta más corta, que se basa en el algoritmo de Dijkstra, planteado simplemente como encontrar la ruta más corta a partir de una fuente dada hasta todos los otros destinos al proceder por etapas y desarrollar la dirección en rutas de longitud creciente.

Cuando un encaminador usa el algoritmo de Dijkstra, calcula todas las rutas diferentes para llegar a cada destino en la interred, creando lo que se conoce como base de datos topológica.

Esta estructura de datos es mantenida por OSPF y se actualiza cada que ocurren fallos. En consecuencia, un encaminador simplemente verifica su base de datos topológica para determinar si una ruta está disponible, y luego usa el algoritmo de Dijkstra para generar un árbol de ruta más corta para evitar el enlace con falla.

Modelos de conexión

Una red de comunicación no está conectada con el contenido de los datos transmitidos, sino con el movimiento de los datos de un punto a otro. Debido a que sería prohibitivo conectar cada nodo de una red con todos los otros nodos, éstos están conectados a una red de comunicación diseñada para minimizar los costos de trasmisión y proporcionar conectividad completa entre todos los dispositivos conectados. Los datos que entran a la red en un punto son encaminados a su destino al cambiarlos de nodo a nodo, ya sea por conmutación de circuitos o de paquetes.

Conmutación de circuitos

La **conmutación de circuitos** es un modelo de comunicación donde entre dos anfitriones se establece una ruta de comunicación dedicada. La ruta es una secuencia conectada de enlaces y la conexión entre los dos puntos existe hasta que se desconecta uno de ellos. La ruta de conexión debe establecerse antes de iniciar la transmisión de datos; en consecuencia, si toda la ruta está indisponible, no es posible transmitir mensajes porque el circuito no está completo. El sistema telefónico constituye un buen ejemplo de red conmutada por circuitos.

En términos de rendimiento, hay un retraso antes de iniciar la transferencia de datos mientras se establece la conexión. Sin embargo, una vez que el circuito está completo, la red es transparente para los usuarios y la información se transmite a una tasa fija de velocidad con retrasos insignificantes en nodos intermedios.

Conmutación de paquetes

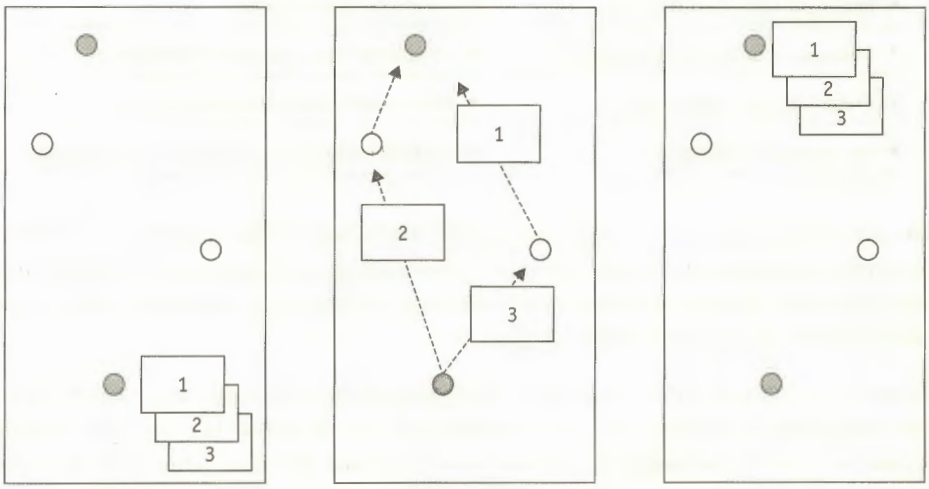
La **conmutación de paquetes** es básicamente una técnica de almacenar y enviar donde un mensaje se divide en varias unidades del mismo tamaño denominadas **paquetes**, que se envían por la red a su destino, donde vuelven a agruparse en su formato largo original, como se muestra en la figura 9.12.

La conmutación de paquetes es una tecnología efectiva para transmisión de datos a larga distancia y proporciona más flexibilidad que la conmutación de circuitos porque permite la transmisión de datos entre dispositivos que reciben o transmiten datos a tasas diferentes. Sin embargo, no hay garantía de que después que un mensaje se ha dividido en paquetes, todos éstos recorran la misma ruta a su destino o que lleguen en su orden secuencial físico.

Con conmutación de paquetes, por rutas diferentes se envían múltiples copias de cada mensaje. Así, si un paquete en una ruta se daña o pierde, para sustituirlo puede usarse un paquete idéntico que se desplaza por otra ruta.

(Figura 9.12)

Una red de conmutación de paquetes no requiere una conexión dedicada. Envía paquetes usando un procedimiento de tres pasos: a) divide los datos en paquetes direccionados; b) envía cada paquete hacia su destino; c) y, en el destino, confirma la recepción de todos los paquetes, los coloca en orden, reagrupa los datos y los entrega al receptor.



Además, los paquetes de un mensaje pueden estar salpicados con aquellos de otros mensajes a medida que se desplazan hacia su destino. En consecuencia, un encabezado que contiene información pertinente sobre el paquete se conecta a cada paquete antes de su transmisión. La información contenida en el encabezado del paquete varía según el método de encaminamiento usado por la red.

La idea es semejante a enviar una serie de 30 libros de referencia a través de un sistema de paquetería. Seis cajas contienen cinco volúmenes cada una y cada caja se identifica con su número de secuencia (por ejemplo, caja 2 de 6), así como con su último destino. A medida que hay espacio en los camiones de entrega, cada caja se envía hacia un centro de conmutación central donde se almacena hasta que hay espacio libre para enviarla al siguiente centro de conmutación más próximo a su destino. Finalmente, una vez que llegan las seis cajas, se colocan en el orden original, se desempacan los 30 volúmenes y se restituye la secuencia original.

Como se muestra en la tabla 9.2, la conmutación de paquetes es esencialmente distinta de la conmutación de circuitos, que también es una técnica de almacenar y enviar, donde un mensaje completo se acepta en un nodo central de conmutación y se envía hacia su destino cuando ocurre uno de dos eventos: todos los circuitos están libres para enviar de una vez todo el mensaje, o el nodo receptor solicita sus mensajes almacenados.

Conmutación de circuitos	Conmutación de paquetes
• Se transmite en tiempo real	• Transmite por lotes
• Es preferible en redes de bajo volumen	• Es preferible en redes de alto volumen
• Eficiencia reducida de la línea	• Eficiencia alta de la línea
• Dedicada a una sola transmisión	• Compartida para muchas transmisiones
• Preferible para transmisión de voz	• No es buena para transmisión de voz
• Se sobrecarga fácilmente	• Acomoda prioridades variables entre paquetes

(Tabla 9.2)

Comparación entre conmutación de circuitos y conmutación de paquetes.

La conmutación de paquetes proporciona mayor eficiencia en la línea porque un solo circuito de nodo a nodo puede compartirse entre varios paquetes y no permanece ocioso durante largos periodos. Aunque la entrega puede retrasarse a medida que aumenta el tráfico, sigue siendo posible aceptar y transmitir los paquetes.

También hay un contraste con las redes de conmutación de circuitos, que, cuando se sobrecargan, rehúsan aceptar nuevas conexiones hasta que disminuye la carga. ¿Ha recibido alguna vez una señal ocupada cuando intenta colocar una llamada telefónica de larga distancia durante una fiesta nacional importante? Este problema es semejante a la respuesta de sobrecarga de una red de conmutación de circuitos.

La conmutación de paquetes permite que los usuarios asignen prioridades a sus mensajes, de modo que un encaminador con varios paquetes en cola para su transmisión puede enviar primero los paquetes con prioridad más alta. Además, las redes de conmutación de paquetes son más confiables que otros tipos porque la mayor parte de los nodos están conectados por más de un enlace, de modo que si un circuito falla, entre nodos es posible establecer una ruta completamente diferente.

Hay dos métodos principales para seleccionar la ruta: datagramas y circuitos virtuales. En el enfoque de los datagramas, el destino y el número de secuencia del paquete se agregan a la información que identifica de manera única el mensaje al que pertenece el paquete; después, cada paquete se maneja de manera independiente y se selecciona una ruta a medida que cada paquete se acepta en la red. Esto es semejante a la etiqueta de embarque que se agrega a cada paquete en el ejemplo del envío de libros. En su destino, todos los paquetes que pertenecen al mismo mensaje vuelven a agruparse por número de secuencia en un mensaje continuo y, por último, se entregan al destinatario. Debido a que el mensaje no puede entregarse sino hasta que se han tomado en cuenta todos los paquetes, es el nodo receptor el que solicita la retransmisión de paquetes perdidos o dañados. Este método de encaminamiento tiene dos ventajas: ayuda a disminuir la conadministración al enviar paquetes que llegan a través de rutas menos congestionadas y proporciona más confiabilidad, ya que es posible establecer rutas alternas cuando falla un nodo.

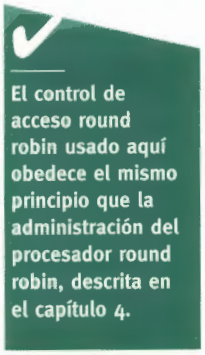
En el enfoque de circuitos virtuales, el destino y la secuencia de los paquetes no se agregan a la información que identifica el mensaje del paquete porque entre el emisor y el receptor se establece una ruta completa antes de iniciar la transmisión; todos los paquetes que pertenecen a ese mensaje usan la misma ruta. Aunque se trata de un concepto semejante, difiere de la

ruta dedicada usada en conmutación de circuitos porque cualquier nodo puede tener varios circuitos virtuales hacia cualquier otro nodo. Su ventaja sobre el método de los datagramas consiste en que su decisión de encaminamiento se toma sólo una vez para todos los paquetes que pertenecen al mismo mensaje; una característica que debe acelerar la transmisión de mensajes largos. Por otra parte, tiene la desventaja de que, si falla un nodo, todos los circuitos virtuales que usan ese nodo se vuelven indisponibles. Además, cuando en el circuito hay tráfico pesado, resulta más difícil resolver los congestionamientos.

Resolución de conflictos

Debido a que una red consta de dispositivos que comparten una capacidad de transmisión común, algunos métodos para controlar el uso del medio son necesarios para facilitar acceso equitativo y justo a este recurso común. Primero describiremos algunas técnicas comunes de control de acceso: round robin, reservación y contención. Luego analizaremos brevemente tres protocolos de control de acceso al medio usados para implementar acceso a recursos: acceso múltiple por detección de portadora (CSMA); *token passing*; y bus dual de cola distribuida (DQDB).

Técnicas de control de acceso



El control de acceso round robin usado aquí obedece el mismo principio que la administración del procesador round robin, descrita en el capítulo 4.

En redes, round robin siempre permite que cada nodo en la red use el medio de comunicación. Si el nodo tiene datos a enviar, se le proporciona cierta cantidad de tiempo para completar la transmisión, al final de la cual la oportunidad se pasa al nodo siguiente. En caso que el nodo no tenga datos a trasmitir, o si completa la transmisión antes que expire el tiempo, entonces el nodo siguiente inicia su turno. Round robin es una técnica eficiente cuando hay muchos nodos transmitiendo durante largos periodos. Sin embargo, cuando hay pocos nodos transmitiendo durante largos periodos, la sobrecarga en que se incurre al pasar turnos de nodo a nodo puede ser sustancial, haciendo preferibles otras técnicas, dependiendo de si las transmisiones son breves e intermitentes, como en sesiones interactivas terminal-anfitrión, o largas y continuas, como en sesiones de transferencia de archivos masivos.

La técnica de reservación es idónea para tráfico lento y continuo. El tiempo de acceso del medio está dividido en ranuras y un nodo puede reservar ranuras de tiempo futuras para su uso. La técnica es semejante a la que se encuentra en multiplexación por división del tiempo síncrona, usada para multiplexar flujos de voz digitalizados, donde la longitud de las ranuras de tiempo es fija y éstas son preasignadas a cada nodo. Esta técnica debería ser aceptable para una configuración con varias terminales conectadas a una computadora anfitrión mediante un solo puerto de E/S.

La técnica de contención es mejor para tráfico breve e intermitente. No se hace ningún intento por determinar a quién corresponde el turno para transmitir, de modo que los nodos compiten para tener acceso al medio. En consecuencia, funciona bien con tráfico de bajo a moderado, pero el rendimiento tiende a disminuir bajo cargas fuertes. La ventaja más importante de esta técnica es que resulta fácil de implementar. Los protocolos de acceso de uso

actual se basan en las técnicas ya mencionadas y se analizan aquí con respecto a su rol en entornos de redes de área local.

CSMA

Acceso múltiple por detección de portadora (CSMA) es un protocolo basado en contención fácil de implementar. Detección de portadora significa que un nodo en la red escucha o prueba el medio de comunicación antes de transmitir algún mensaje, previniendo así una colisión con otro nodo que esté transmitiendo en ese momento. Acceso múltiple significa que varios nodos están conectados a la misma línea de comunicación como pares, al mismo nivel, y con privilegios iguales.

Aunque un nodo no transmite sino hasta que la línea está tranquila, pueden llegar dos o más nodos de modo que terminen al mismo tiempo. Si más de una transmisión envía de manera simultánea, creando una colisión, los datos de todas las transmisiones se dañan y la línea permanece inutilizable mientras se disipan los mensajes dañados. Una vez que los nodos receptores fallan en acusar recibo de sus transmisiones, los nodos emisores se enteran que los mensajes no llegaron con éxito a sus destinos, de modo que ambos son retransmitidos. La probabilidad de esta ocurrencia aumenta si los nodos están más separados, haciendo a CSMA un protocolo de control de acceso menos atractivo para redes grandes o complicadas.

En consecuencia, el algoritmo original fue modificado a fin de incluir detección de colisiones y fue denominado acceso múltiple por detección de portadora con detección de colisiones (CSMA/CD). **Ethernet** es el protocolo CSMA/CD más conocido. La detección de colisiones no elimina las colisiones, pero sí las reduce. Cuando ocurre una colisión, una señal de embotellamiento es enviada de inmediato a ambos nodos emisores, que entonces esperan un periodo aleatorio antes de volver a intentarlo. Con este protocolo, la cantidad de capacidad de transmisión desperdiciada se reduce al tiempo necesario para detectar la colisión.

Una modificación diferente de CSMA es con evasión de colisiones (CSMA/CA). Evasión de colisiones significa que el método de acceso previene que múltiples nodos colisionen durante la transmisión. Sin embargo, las opiniones en este sentido ahora están divididas. Algunos afirman que es más eficiente que la detección de colisiones, mientras otros argumentan que disminuye el rendimiento del protocolo cuando hay grandes cantidades de nodos. El protocolo CSMA/CA está implementado en sistemas de cable LocalTalk de Apple, que usan un protocolo denominado protocolo de acceso de enlace LocalTalk. Una terminal conectada a una red CMSA/CA de Apple envía un paquete de tres bytes para indicar que desea empezar a transmitir. Este paquete dice a las otras terminales que esperen hasta que la primera termine de transmitir antes que aquellas empiecen sus transmisiones. Si ocurren colisiones, sólo implican a los paquetes de tres bytes, no a los datos actuales. Este protocolo no garantiza que los datos lleguen a su destino, aunque asegura que cualquier dato transmitido está libre de errores.

Token passing

En una red *token passing*, cuando se enciende la red se genera un mensaje electrónico especial denominado *"token"*, que luego es pasado de nodo a nodo. Sólo se permite transmitir al nodo que tiene el *token*, y una vez que ha terminado, debe pasar el *token* a otro nodo. Las redes típicamente tienen una topología de bus o de anillo y son conocidas porque el acceso es rápido y las colisiones son inexistentes.

En una red ***token-bus***, el *token* se pasa a cada nodo a su vez. Una vez que se recibe el *token*, un nodo los conecta con los datos a transmitir y envía el paquete, que contiene tanto el *token* como los datos, a su destino. El nodo receptor copia los datos, agrega un reconocimiento y regresa el paquete al nodo emisor, que luego pasa el *token* al nodo siguiente en secuencia lógica.

Inicialmente, el orden de los nodos es determinado por un algoritmo cooperativo descentralizado. Una vez que la red está arriba y funcionando, los turnos se determinan por prioridad con base en la actividad del nodo. Un nodo que requiere un *token* a menudo tiene prioridad más alta que otro que lo necesita pocas veces. La red mantiene en orden de prioridad una tabla de direcciones de los nodos. Cuando se completa una transmisión, el *token* pasa del nodo que acaba de terminar al nodo que tiene la siguiente prioridad más baja en la tabla. Una vez que el nodo con la prioridad más baja ha sido atendido, el *token* regresa a la parte superior de la tabla y el proceso se repite.

Este proceso es semejante a una locomotora que entra a una estación de ferrocarril. Si el jefe de estación tiene algún envío que hacer, los vagones conectados a la locomotora y al tren son despachados a su destino sin paradas intermedias. Cuando llegan, los carros se liberan y la locomotora se envía de vuelta al punto de origen con el mensaje de que el envío fue recibido exitosamente. Después de entregar el mensaje al remitente, el ferrocarril procede a la siguiente estación para recoger un envío.

La implementación de este protocolo dicta una mayor sobrecarga en cada nodo que el protocolo CMSA/CD, y los nodos pueden tener largas esperas en ciertas condiciones antes de recibir el *token*.

La red de anillo *token* fue desarrollada originalmente por IBM en la década de 1970.

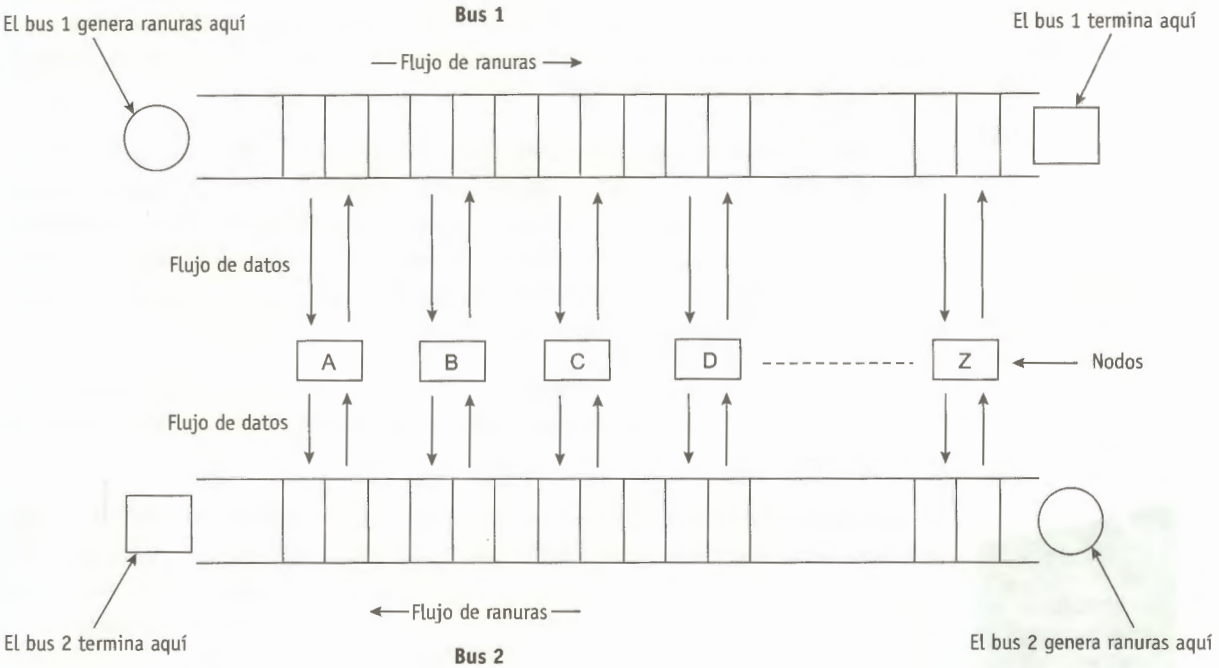
La red de anillo ***token*** es el protocolo de mayor uso para la topología de anillo; fue mejor conocido que el *token-bus* cuando IBM puso disponible comercialmente su red de anillo *token*. Se basa en el uso de un *token* que se mueve entre los nodos a su vez y sólo en una dirección. Cuando no transporta ningún mensaje, el *token* se denomina *token* "libre". Si un nodo desea emitir un mensaje, debe esperar la llegada del *token* libre. Luego cambia el *token* de libre a ocupado y envía su mensaje inmediatamente después del *token* ocupado. Mientras tanto, todos los demás nodos deben esperar que el *token* se vuelva libre y les llegue antes que sean capaces de transmitir cualquier mensaje.

El nodo receptor copia el mensaje en el paquete y habilita el bit copiado para indicar que el mensaje fue recibido exitosamente; después, el paquete continúa su camino, haciendo una vuelta completa hasta el nodo emisor, que luego suelta en la red el nuevo *token* libre. En este

punto, el siguiente nodo en la línea con datos a enviar puede recoger el *token* libre y repetir el proceso.

DQDB

Se pretendía que el uso del protocolo cola distribuida de doble red de buses (DQDB) fuese con una configuración de doble bus, donde cada bus transporta datos en una sola dirección y ha sido estandarizado por uno de los comités IEEE como parte de sus normas MAN. La transmisión sobre cada bus consta de un flujo estable de ranuras de longitud fija, como se muestra en la figura 9.13. Las ranuras generadas en un extremo de cada bus están marcadas como libres y se envían como flujo descendente, donde se marcan como ocupadas y son escritas por nodos que ya están listos para transmitir datos. Los nodos leen y copian los datos desde las ranuras, que luego continúan desplazándose hacia el final del bus, donde se disipan.



(Figura 9.13)

Protocolo cola distribuida de doble red de buses. Ranuras libres se generan en un extremo de cada bus y fluyen en una sola dirección. Usando DQDB, si el nodo C quiere enviar datos al nodo D, debe esperar una ranura libre en el bus 1 porque las ranuras fluyen hacia el nodo D en ese bus.

El protocolo de acceso distribuido está basado en un esquema de reservación distribuida y puede resumirse como sigue. Si el nodo C en la figura 9.12 quiere enviar datos al nodo D, debe usar el bus 1 porque las ranuras fluyen hacia D en ese bus. Sin embargo, si los nodos antes de C monopolizan las ranuras, entonces C no puede transmitir sus datos a D. Para resolver el problema, C puede usar el bus 2 para enviar una reservación a sus vecinos como flujo ascendente. El protocolo establece que un nodo permite a las ranuras libres proseguir hasta que se hayan satisfecho las reservaciones importantes provenientes de nodos con flujo descendente. En consecuencia, el protocolo debe contar con un mecanismo mediante el cual cada estación puede seguir la pista de las solicitudes de sus pares con flujo descendente.

El mecanismo es manipulado por un par de colas primero en entrar, primero en salir y un par de contadores, uno para cada bus, en cada uno de los nodos en la red. Este protocolo es muy efectivo que proporciona retrasos despreciables bajo cargas ligeras y colas predecibles bajo cargas pesadas. Esta combinación hace de DQDB un protocolo idóneo para MAN que gestiona grandes transferencias de datos y es capaz de satisfacer las necesidades de usuarios interactivos.

Normas de protocolos de transporte

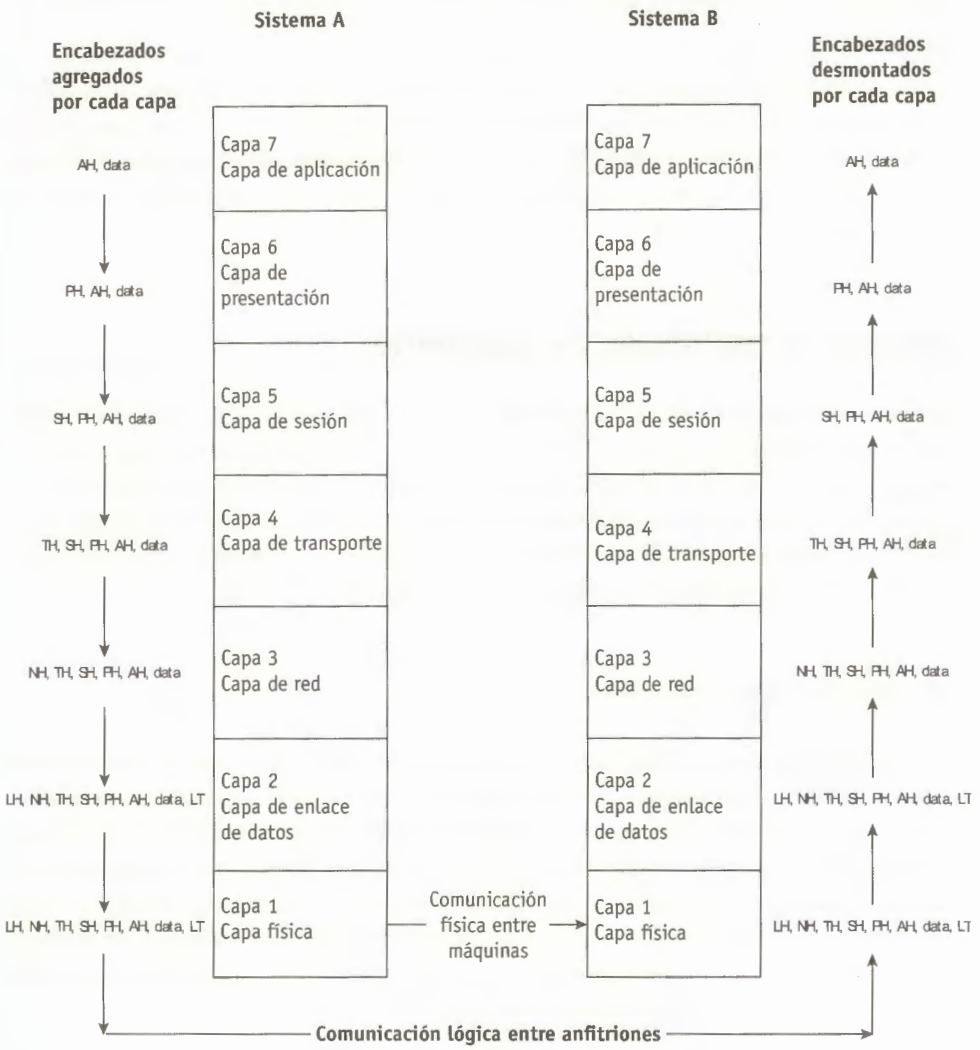
Durante la década de 1980, el uso de redes comenzó a crecer a grandes pasos, así como la necesidad de integrar dispositivos para redes distintas de diferentes vendedores; una tarea que cada vez se hizo más difícil a medida que aumentaban el número y la complejidad de los dispositivos de red. Pronto, la comunidad de usuarios presionó a la industria para crear una arquitectura única adoptada universalmente que pudiera permitir verdadera interoperatividad de múltiples vendedores. Revisaremos dos modelos: OSI y TCP/IP.

Modelo de referencia OSI

La **Organización Internacional para la Estandarización** (ISO), que hace recomendaciones técnicas sobre interfaces de comunicación de datos, asumió la tarea de crear esta arquitectura de red. Sus esfuerzos resultaron en el **Modelo de Referencia de Interconexión de Sistemas Abiertos** (OSI), que sirve como un marco de referencia para definir los servicios que una red debe proporcionar a sus usuarios. Este modelo constituye la base para conectar sistemas abiertos para procesamiento de aplicaciones distribuidas. La palabra "abiertos" significa que dos sistemas cualesquiera que se ajusten al modelo de referencia y a las normas relacionadas pueden conectarse, independientemente del vendedor.

Una vez que se identificaron todos los servicios, funciones semejantes se reunieron en siete grupos lógicos denominados capas. Una de las razones principales usadas para definir las siete capas fue agrupar funciones localizadas fácilmente de modo que cada capa pudiera rediseñarse y sus protocolos modificarse de cualquier forma para aprovechar los nuevos avances en arquitectura, hardware o software, sin cambiar los servicios esperados desde y proporcionados a las capas adyacentes. Los límites entre las capas se escogieron en puntos

que la experiencia indicaba que eran efectivos. El modelo OSI de siete capas resultante manipula la transmisión de datos desde una terminal o programa de aplicación a otra. La figura 9.14 muestra la forma en que los datos pasan a través de las siete capas y cómo están organizados: desde la capa de aplicación, la más próxima al usuario, hasta la capa física, la más próxima a los cables, módems y circuitos. A continuación presentamos una breve descripción de la función de cada capa.



(Figura 9.14)

Modelo del protocolo de transporte OSI. En cada capa de la unidad emisora, sistema A, un nuevo encabezado se asocia al paquete previo antes que pase hacia la siguiente capa más baja. Finalmente, en la capa de enlace de datos, se agrega un tráiler (LT), completando el marco, que se pasa a la capa física para su transmisión. Después, la unidad receptora elimina cada encabezado o tráiler hasta que entrega los datos al programa de aplicación en la capa 7 del sistema B.

Capa 1: Capa física

La capa 1 está en la parte inferior del modelo. Aquí es donde se describen las especificaciones mecánicas, eléctricas y funcionales para conectar un dispositivo con una red particular. La capa 1 se ocupa principalmente de la transmisión de bits sobre las líneas de comunicación, de modo que las tensiones de electricidad y los factores de sincronización son importantes. Esta es la única capa que se ocupa del hardware, y todos los datos deben pasar por ella para que ocurra la verdadera transferencia de datos entre unidades. (Las capas 2 a 7 se ocupan del software, y las comunicaciones entre unidades a estos niveles sólo es virtual.) Algunos ejemplos de especificaciones de la capa física son 100Base-T, RS449 y CCITT V.35.

Capa 2: Capa de enlace de datos

Debido a que para implementar la capa 2 se requiere software, éste debe almacenarse en algún tipo de dispositivo programable como un procesador de extremo, nodo de red o microcomputadora. El puenteo entre dos redes homogéneas ocurre en esta capa. Por un lado, la capa de enlace de datos establece y controla la ruta física de las comunicaciones antes de enviar datos a la capa física debajo de ella. Toma los datos, que se han dividido en paquetes por la capa de arriba, y físicamente agrupa el paquete para su transmisión al completar su marco. Los marcos contienen datos combinados con caracteres de control y detección de errores, de modo que la capa 1 puede transmitir un flujo continuo de bits sin preocuparse por su formato o significado. Por otra parte, verifica los errores de transmisión y resuelve los problemas ocasionados por marcos de mensajes dañados, perdidos o duplicados, de modo que la capa 3 puede trabajar con mensajes sin errores. Algunos protocolos al nivel de enlace de datos típicos son *High-Level Data Link Control* (HDLC) y *Synchronous Data Link Control* (SDLC).

Capa 3: Capa de red

La capa 3 proporciona servicios, como direccionamiento y encaminamiento que mueven datos a través de la red hacia su destino. Básicamente, el software a este nivel acepta bloques de datos provenientes de la capa 4, la capa de transporte, los redimensiona en paquetes más pequeños y los encamina al destino idóneo. Los métodos de direccionamiento que permiten identificar un nodo y su red, así como los algoritmos para manipular resolución de direcciones, se especifican en esta capa. Una base de datos de tablas de encaminamiento sigue la pista de todas las rutas posibles que puede tomar un paquete y determina cuántos circuitos diferentes existen entre dos nodos de conmutación de paquetes. Esta base de datos puede almacenarse a este nivel para proporcionar encaminamiento de paquetes eficiente y debe actualizarse dinámicamente para incluir información sobre cualquier circuito con fallo y el volumen de transmisión presente en los circuitos activos.

Capa 4: Capa de transporte

Esta capa también se denomina capa anfitrión a anfitrión o extremo a extremo porque mantiene transmisión confiable entre los usuarios extremos. Un programa en la computadora fuente puede enviar una comunicación virtual a un programa semejante en una máquina

destino usando encabezados de mensaje y mensajes de control. Sin embargo, la ruta física aún va a la capa 1 y a través de la computadora destino. El software para esta capa contiene facilidades que manipulan el direccionamiento del usuario y asegura que todos los paquetes de datos hayan sido recibidos y que no se haya perdido ninguno. Este software puede almacenarse en procesadores de extremo, nodos de conmutación de paquetes o computadoras anfitrión. Además, esta capa cuenta con un mecanismo que regula el flujo de información, de modo que un anfitrión rápido no puede rebasar a una terminal más lenta o a un anfitrión sobrecargado. Un protocolo de la capa de transporte bastante conocido es el Protocolo de Control de Transmisión (TCP).

Capa 5: Capa de sesión

Esta capa es responsable de proporcionar un servicio de conexión orientado al usuario y transferir datos sobre las líneas de comunicación. La capa de transporte es responsable de crear y mantener una conexión lógica entre los extremos. La capa de sesión proporciona una interfaz de usuario que agrega valor a la capa de transporte en la forma de administración de diálogo y recuperación de errores. Algunas veces, la capa de sesión se denomina capa de "control de flujo de datos" porque establece la conexión entre dos aplicaciones o procesos, refuerza las regulaciones para llevar a cabo la sesión, controla el flujo de datos y restablece la conexión en caso de fallo. Esta capa también puede realizar algunas funciones de contabilidad para asegurar que los usuarios reciban sus cuentas. Las funciones de la capa de transporte y la capa de sesión son muy semejantes y, debido a que en general el sistema operativo de la computadora anfitrión manipula la capa de sesión, resulta natural combinar ambas capas en una, como hace TCP/IP.

Capa 6: Capa de presentación

La capa 6 es responsable de funciones de manipulación de datos comunes a muchas aplicaciones, como formateo, compresión y cifrado. La conversión de datos, la sintaxis de la conversión, y el protocolo de la conversión son tareas comunes realizadas en esta capa. Las puertas de enlace que conectan redes con protocolos diferentes son dispositivos de la capa de presentación; una de sus funciones es acomodar interfaces totalmente diferentes según las ve una terminal en un nodo y es esperada por el programa de aplicación en la computadora anfitrión. Por ejemplo, en el Sistema de Control de Información al Cliente (CICS) de IBM, el monitor de teleprocesamiento es un servicio de la capa de presentación ubicado en una computadora central anfitrión, aunque proporciona funciones adicionales más allá de la capa de presentación.

Capa 7: Capa de aplicación

En la capa 7, los programas de aplicación, las terminales y las computadoras tienen acceso a la red. Esta capa proporciona la interfaz a los usuarios y es responsable de formatear los datos del usuario antes de pasarlos a las capas inferiores para su transmisión a un anfitrión remoto. Contiene funciones y herramientas de administración de redes para atender aplicaciones distribuidas. Dos de los protocolos y funciones de aplicación más comunes son transferencia de archivos y correo electrónico.

Una vez que se agrupa el modelo OSI, permite que los nodos se recomuniquen entre sí. Cada capa proporciona un arreglo completamente diferente de funciones a la red, pero todas las capas trabajan al unísono para asegurar que la red proporcione servicio transparente para los usuarios.

Modelo TCP/IP

El **modelo de referencia del protocolo de control de transmisión (TCP) y protocolo de internet (IP)** es tal vez la norma más antigua del protocolo de transporte. Es la base de comunicaciones por internet y es el protocolo de capas de red de mayor uso actualmente. Fue desarrollado por el Arpanet del Departamento de Defensa de Estados Unidos ARPANET y proporciona transmisión razonablemente eficiente y sin errores entre diferentes sistemas. Debido a que es un protocolo de transferencia de archivos, grandes archivos pueden enviarse a través de redes no confiables con una alta probabilidad de que los datos lleguen sin errores. Algunas diferencias entre el modelo TCP/IP y el modelo de referencia OSI son la importancia que TCP/IP asigna a la comunicación interredes y al proporcionar servicios sin conexión y su administración de ciertas funciones, como tomar en cuenta el uso de los recursos.

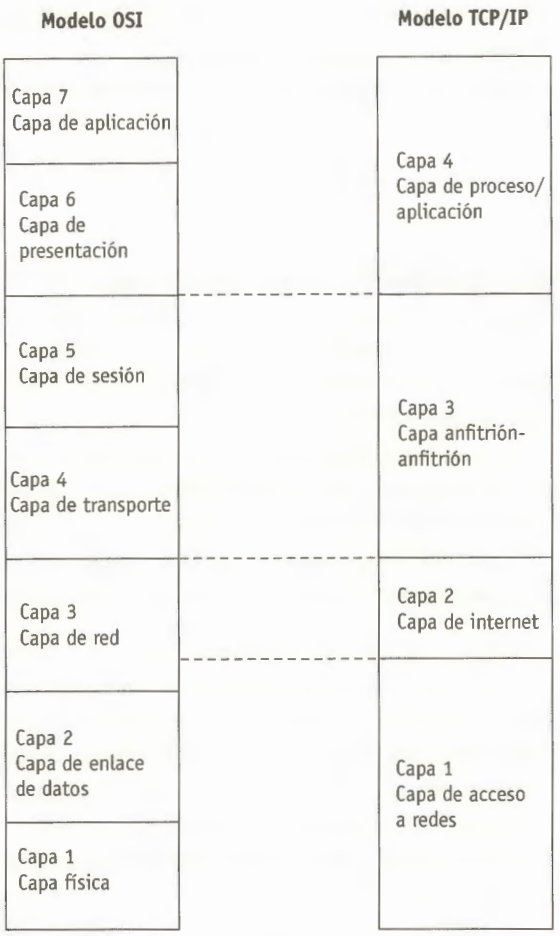
El modelo TCP/IP organiza un sistema de comunicación con tres componentes principales: procesos, anfitriones y redes. Los procesos se ejecutan en anfitriones, que a menudo pueden atender múltiples procesos simultáneos que se definen como unidades primarias que necesitan comunicarse. Estos procesos se comunican a través de redes a las que están conectados los anfitriones. Con base en esta jerarquía, el modelo puede dividirse aproximadamente en dos tareas fundamentales: una que administra la transferencia de información hacia el anfitrión en que reside el proceso y una que asegura que se obtiene el proceso correcto dentro del anfitrión. En consecuencia, una red debe ocuparse sólo con el encaminamiento de los datos entre anfitriones, en la medida en que éstos puedan después dirigir los datos a los procesos idóneos. Con esto en mente, el modelo TCP/IP puede disponerse en cuatro capas, en lugar de las siete del OSI, como se muestra en la figura 9.15. A continuación presentamos una breve descripción de las funciones de las capas y cómo se relacionan con el modelo OSI.

Capa de acceso a redes

Esta capa es equivalente al enlace de datos, físico, y forma parte de las capas de red del modelo OSI. Los protocolos en esta capa proporcionan acceso a una red de comunicación. Algunas de las funciones realizadas aquí son control de flujo, control de errores entre anfitriones, seguridad e implementación de seguridad.

Capa de internet

Esta capa es equivalente a la porción de la capa de red del modelo OSI que no se incluyó en la capa previa, específicamente el mecanismo que efectúa son funciones de encaminamiento. En consecuencia, este protocolo suele implementarse dentro de puertas de enlace y anfitriones. Un ejemplo de un conjunto estándar establecido por el Departamento de Defensa (DoD)



(Figura 9.15)
Comparación entre los modelos OSI y TCP/IP y sus capas funcionales.

de Estados Unidos es el protocolo de internet (IP), que proporciona servicio sin conexión para sistemas extremos para comunicarse a través de una o más redes.

Capa anfitrión-anfitrión

Esta capa es equivalente a las capas de transporte y sesión del modelo OSI. Como indica su nombre, esta capa atiende mecanismos para transferir datos entre dos procesos en computadoras anfitriones diferentes. Los servicios proporcionados en la capa anfitrión-anfitrión también incluyen verificación de errores, control de flujo y una capacidad para manipular señales de control de conexión. Un ejemplo de una norma establecida por el DoD es el protocolo de control de transmisión (TCP), que proporciona un servicio confiable de transferencia de datos de extremo a extremo.

Capa de proceso/aplicación

Esta capa es equivalente a las capas de presentación y aplicación del modelo OSI. Incluye protocolos para compartición de recursos de computadora a computadora y de terminal a computadora de acceso remoto. Ejemplos específicos de normas establecidas por el DoD para esta capa son el Protocolo de Transferencia de Archivos (FTP), una aplicación simple para transferencia para archivos ASCII, EBCDIC y binarios; Protocolo Simple de Transferencia de Correo (SMTP), una facilidad simple de correo electrónico; y Telnet, una capacidad síncrona que proporciona capacidades de registro remoto para usuarios que están trabajando en una terminal o en una computadora personal.

Conclusión

Aunque los sistemas operativos para redes necesariamente incluyen las funciones de los cuatro administradores analizados hasta el momento en este libro de texto (el administrador de la memoria, el administrador del procesador, el administrador de dispositivos y el administrador de archivos), también necesitan coordinar todas estas funciones entre las muy diversas piezas de hardware y software, sin importar el sitio en que se encuentren físicamente.

No hay manera simple que permita medir el éxito de un sistema operativo de red, pero un mínimo es que debe satisfacer los requerimientos de confiabilidad de sus propietarios. Es decir, cuando un nodo falla, y todas la redes experimentan fallo de nodos de vez en cuando, el sistema operativo debe detectar la falla, cambiar las instrucciones de encaminamiento para evitar ese nodo y asegurarse que todos los mensajes perdidos sean retransmitidos hasta que se reciban con éxito.

En este capítulo hemos presentado los conceptos básicos de organización de redes: terminología común, topologías de red, tipos de redes, cuestiones de diseño de redes y normas de protocolos de transporte. Sin embargo, debe tenerse en mente que este tema es complicado y que sólo hemos tocado la superficie en estas pocas páginas. Para más información sobre temas relacionados aquí analizados, consulte el o los textos de referencia que se presentan al final del libro.

Términos clave

Anfitrión: 1) Término de internet para un nodo de red capaz de comunicarse al nivel de la capa de aplicación. Cada anfitrión internet tiene una dirección IP única. 2) Computadora conectada a una red con un programa o archivos centralizados que ponen estos recursos a disposición para otras computadoras en la red.

Conmutación de circuitos: Modelo de comunicación donde una ruta de comunicación dedicada se establece entre dos anfitriones sobre los que se desplazan todos los mensajes.

Conmutación de paquetes: Modelo de comunicación donde los mensajes se encaminan individualmente entre anfitriones, sin ninguna ruta de comunicación establecida de antemano.

Encaminador: Dispositivo que envía el tráfico entre redes.

Ethernet: Conocida tecnología de red LAN donde los nodos compiten por tener acceso a una red. Barata, fácil de instalar y mantener.

Internet: La más grande colección de redes interconectadas con encaminadores. Internet es una interred de multiprotocolos.

Local: Que pertenece al nodo de red a la que está conectado el usuario.

Modelo de Referencia de Interconexión de Sistemas Abiertos (OSI): Estructura conceptual de siete capas que describe arquitecturas de redes informáticas y las formas en que los datos pasan a través de ellas.

Modelo de referencia del Protocolo de control de transmisión (TCP) y Protocolo de internet (IP): Acrónimo común para el paquete de protocolos capa de transporte y capa de aplicación que operan sobre el protocolo de internet.

Nodo: Dispositivo direccionable en la capa de red conectado a una red de computadoras.

Organización Internacional para la Estandarización (ISO): Organización voluntaria *sin fines de lucro* responsable de crear normas internacionales en muchas áreas, incluyendo computadoras y comunicación.

Paquete: Unidad de datos enviados a través de una red.

Puerta de enlace: Dispositivo o programa de comunicación que pasa datos entre redes que tienen funciones semejantes, pero protocolos diferentes.

Protocolo: Conjunto de reglas para controlar el flujo de mensajes a través de una red.

Protocolo de encaminamiento de información (RIP): Protocolo de encaminamiento usado por IP, basado en un algoritmo distancia-vector.

Protocolo de encaminamiento jerárquico de puerta de enlace interior (OSPF): Protocolo diseñado para usar en redes del protocolo de internet (IP), que se ocupa de seguir la pista del estado operacional de todas las interfaces de red.

Puente: Dispositivo de enlace de datos usado para interconectar múltiples redes usando el mismo protocolo.

Red: Colección de procesadores débilmente acoplados intercomunicados por enlaces de comunicación usando cables, tecnología inalámbrica o una combinación.

Red de área amplia (WAN): Red usualmente construida con líneas de larga distancia punto a punto, que cubre una gran área geográfica.

Red de área local (LAN): Red de datos cuya función es servir a un área que cubre sólo unos kilómetros cuadrados o menos.

Red de área local inalámbrica (WLAN): Red de área local con nodos inalámbricos.

Red de área metropolitana (MAN): Red de datos cuya función es servir un área aproximadamente igual a la de una ciudad grande.

Remoto(a): Perteneciente a un nodo en un extremo distante de una conexión de red.

Sistema de nombre de dominio (DNS): Servicio de consulta de datos replicados, distribuido, de propósito general. Su función principal es la resolución de direcciones de internet con base en nombres de dominio totalmente calificados.

Sistema operativo de red (NOS): Software que administra los recursos de una red para un nodo en la red y puede proporcionar seguridad y control de acceso.

Sistema operativo distribuido (D/OS): Sistema operativo que proporciona control para un sistema de cómputo distribuido, permitiendo que a sus recursos se tenga acceso de manera uniforme.

Sitio: Ubicación específica sobre una red, que contiene uno o más sistemas de cómputo.

***Token bus*:** Tipo de red de área local con nodos conectados a un cable común usando un protocolo CSMA/CA.

***Token ring*:** Tipo de red de área local con estaciones conectadas en una red de anillo.

Topología: En una red, la disposición geométrica de las conexiones (cables, inalámbrica o ambos) que enlaza los nodos.

Topología de anillo: Topología de red, donde cada nodo está conectado a dos nodos adyacentes.

Topología de árbol: Arquitectura de red donde los elementos están conectados en una estructura jerárquica.

Topología de bus: Arquitectura de red para conectar elementos entre sí a lo largo de una sola línea.

Topología en estrella: Topología de red donde múltiples nodos de red están conectados a través de un solo nodo central.

Topología híbrida: Arquitectura de red que combina otros tipos de topologías de red para acomodar características de operación particulares o volúmenes de tráfico.

Búsquedas de interés

- Normas inalámbricas del IEEE
- WiMAX contra Wi-Fi
- Voz sobre protocolo de internet (VoIP)
- Protocolos encaminadores

Ejercicios

Temas de investigación

A. Mencione varios sistemas operativos que ejecuten redes de área local actualmente. No incluya versiones diferentes de un solo sistema operativo. Para cada sistema operativo, escriba su nombre, la plataforma o red en que opera y su distribuidor o fabricante. Cite sus fuentes.

B. En este capítulo presentamos la norma WiMAX. Consulte literatura actual o fuentes creíbles en internet para continuar explorando el estado actual de la tecnología WiMAX. Describa cualquier barrera para el uso comercial y aplicaciones que muestren la más promisoria. Explique cuáles países esperan beneficiarse más y por qué. Asegúrese de citar sus fuentes. Si su análisis incluye términos no usados en este texto, defínalos.

Ejercicios

1. Como se mencionó en este capítulo, algunas ocasiones los clientes y los servidores intercambian roles con clientes convirtiéndose a veces en servidores. Proporcione un ejemplo donde un cliente siempre sea cliente y nunca se convierta en servidor.
2. Explique por qué los sistemas operativos de redes se desfasaron cuando los sistemas operativos distribuidos ganaron popularidad.
3. Una de las primeras topologías de red fue el token bus. Describa cómo el papel del token c en la red de token bus se compara con la red de anillo token.
4. Describa con sus propias palabras las diferencias funcionales entre un puente y una puerta de enlace. Proporcione un ejemplo de cada uno.
5. Explique las ventajas principales y las desventajas de una topología bus.
7. Describa cómo una red con una topología en estrella puede ser manipulada para que actúe como si utilizara una topología de anillo.
8. Remítase a la figura 9.9 y describa el flujo de datos de anfitrión a anfitrión, moviéndose del anfitrión 1 al 6, incluyendo cada controlador o puente.
9. Remítase a la figura 9.10 y describa el flujo de datos de anfitrión a anfitrión, moviéndose del anfitrión 1 al 6, incluyendo cada controlador o puente.
10. Describa una topología híbrida e ilustre gráficamente los dos ejemplos.
11. Analice al menos tres ventajas de una LAN alambrada comparada con una LAN inalámbrica y describa una situación en la que la LAN alambrada sea preferida.
12. Analice la principal diferencia entre un puente y una puerta de enlace y proporcione un ejemplo real de cada uno.
13. Explique el rol de los encaminadores cuando mueven datos de un anfitrión a otro en la misma red.

14. En este capítulo se describieron los encaminadores de paquetes. Proporcione dos ejemplos en los que los paquetes puedan ser encaminados y explique las diferencias entre ellos.
15. Proporcione dos ejemplos reales donde la conmutación de paquetes tenga preferencia sobre la conmutación de circuitos y explique por qué.
16. Compare el esquema de circuito virtual para la conmutación de paquetes y describa sus ventajas.
17. Compare y contraste los dos protocolos de encaminamiento más ampliamente utilizados: el protocolo de encaminamiento de información (RIP) y el protocolo de encaminamiento jerárquico de puerta de enlace interior (OSPF).
18. Identifique una topología de red que se ajuste mejor a cada uno de los siguientes entornos y explique por qué.
    a. Piso de dormitorio.
    b. Campus universitario.
    c. Aeropuerto.
    d. Estado o provincia.

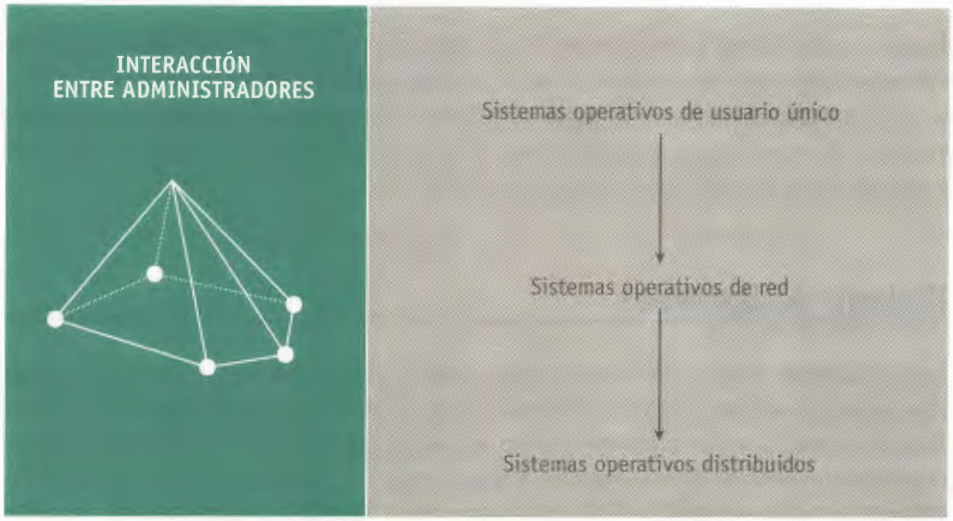
Ejercicios avanzados

19. Aunque no se analizó explícitamente en el capítulo, parece que el tamaño del paquete puede tener un impacto en el tiempo de transmisión. Analice si este hecho es cierto o no y explique por qué. Específicamente, compare el concepto del número de bits que constituyen la porción de datos del paquete y el número de bits que constituyen la parte de dirección del paquete. Recuerde que la dirección está sobrecargada e impacta las tasas de transmisión.

    Ofrezca un ejemplo extenso que compare los tamaños de los paquetes y los tiempos de transmisión resultantes. Por ejemplo, vea algunos de sus mensajes de correo electrónico y compare el número de bits que constituyen el mensaje con el número de bits de la dirección. Calcule los resultados para varios correos electrónicos y proporcione sus conclusiones.
20. Analice qué está equivocado en el siguiente argumento: La conmutación de paquetes requiere agregar a cada paquete bits de control y acceso, lo cual ocasiona considerable sobrecarga en la conmutación de paquetes. En conmutación de circuitos, se establece un circuito dedicado y no se requiere ningún bit extra.
    a. En consecuencia, en conmutación de paquetes no hay sobrecarga.
    b. Debido a que en conmutación de circuitos no hay sobrecarga, la utilización de la línea debe ser más eficiente que en conmutación de paquetes.
21. Describa las diferencias entre CSMA/CD y CSMA/CA. Proporcione un ejemplo de la vida real donde se use CSMA/CD. Proporcione un ejemplo de la vida real donde se use CSMA/CA.

22. Explique las circunstancias en que una red *token-ring* es más efectiva que una red Ethernet.
23. Aunque el modelo OSI de red especifica siete capas de funcionalidad, la mayor parte de los sistemas de cómputo usan menos capas para implementar una red. ¿Por qué usan menos capas? ¿Qué problemas puede ocasionar el uso de menos capas?

Capítulo 10 Administración de funciones de redes



"A medida que crece el conocimiento, el asombro se profundiza."

Charles Morgan (*1894-1958*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Las complejidades introducidas a los sistemas operativos por las capacidades de red.
- Sistemas operativos de red (NOS) comparados con sistemas operativos distribuidos (DO/S).
- Cómo un DO/S realiza administración de memoria, procesos, dispositivos y archivos.
- Cómo un NOS realiza administración de memoria, procesos, dispositivos y archivos.
- Características importantes de DO/S y NOS.

Cuando las organizaciones se mueven hacia sistemas completamente descentralizados, más y más dispositivos informáticos se enlazan a través de complicadas redes de comunicaciones inalámbricas, equipo de teleconferencias, computadoras anfitriones y otras tecnologías digitales. Pero con este desarrollo hay dos problemas. Primero, se impone una tremenda demanda sobre redes de comunicaciones de datos por el número asombroso de interconexiones de hardware. Segundo, la comunidad de usuarios impone cada vez más presión sobre estas redes para operar con mayor confiabilidad, seguridad y velocidad.

En este capítulo se explorarán las diferencias entre sistemas operativos de redes y sistemas operativos distribuidos. Se explicarán los modelos de sistemas operativos basados en procesos y los basados en objetos y se usarán para definir los papeles de la memoria, los dispositivos, periféricos, archivos y administradores de redes en los sistemas operativos distribuidos y se analizará el papel de los sistemas operativos de redes.

Historia de las redes

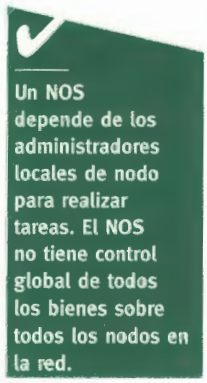
Las redes fueron creadas inicialmente para compartir recursos de hardware costosos como grandes computadoras centrales, impresoras láser y discos duros de tamaño considerable. Estas redes físicas, con sus sistemas operativos, permitieron que las organizaciones aumentaran la disponibilidad de estos recursos y distribuyeran el costo entre muchos usuarios. Sin embargo, el centro de atención de la tecnología cambió cuando los propietarios de sistemas se percataron que el recurso más valioso de una red no era el hardware, sino la información ahí almacenada. Pronto, muchos sistemas operativos fueron mejorados con capacidades de red para proporcionar a los usuarios en toda una organización un fácil acceso a los recursos de información centralizada. Primero fue desarrollado el sistema operativo de red, seguido por el más poderoso sistema operativo distribuido.

Hoy, las aplicaciones denominadas de manera colectiva como trabajo cooperativo asistido por computadora, o **groupware**, usan un conjunto de tecnologías denominadas **procesamiento distribuido** para permitir aún mayor acceso a información centralizada y permitir que los usuarios trabajen juntos para completar tareas comunes.

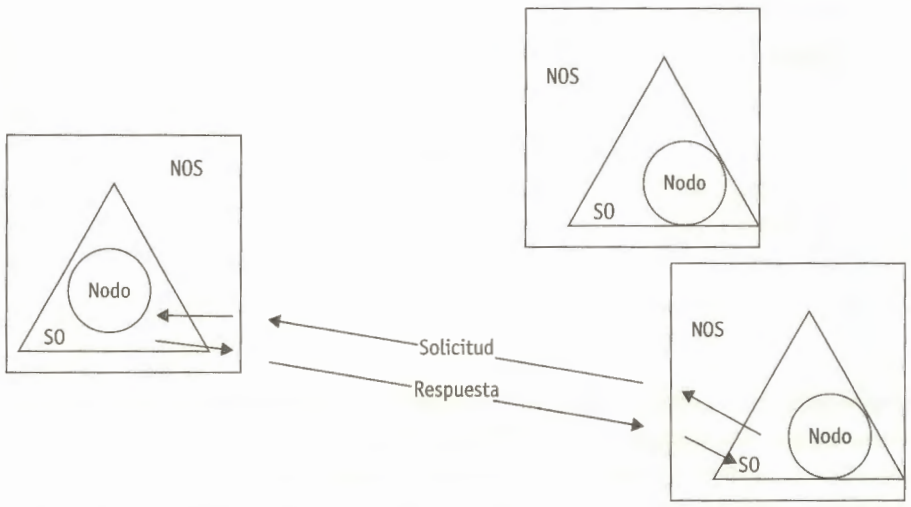
Comparación de los sistemas operativos de red y distribuidos

El **sistema operativo de red** (NOS) evolucionó a partir de la necesidad de proporcionar a los usuarios acceso global a recursos, administrar globalmente los procesos de red y hacer la red casi completamente transparente para los usuarios y sus sistemas operativos de sitios, denominados sistemas operativos locales. En la figura 10.1 se muestra un NOS típico.

Un sistema operativo de red proporciona poderes ampliados a los sistemas operativos locales. Es decir, proporciona al sistema local una nueva capacidad para aceptar una solicitud para realizar procesamiento o para tener acceso a datos que no están disponibles localmente. Empieza por determinar el lugar en que están ubicados los recursos. Luego inicia la operación y devuelve los datos o el servicio idóneos al solicitante.



Un NOS depende de los administradores locales de nodo para realizar tareas. El NOS no tiene control global de todos los bienes sobre todos los nodos en la red.



(Figura 10.1)

En un entorno NOS cada nodo, mostrado aquí como un círculo, es administrado por su propio sistema operativo local. Sus sistemas operativos de red respectivos, mostrados aquí como cuadrados, entran en juego sólo cuando un sitio en el sistema necesita trabajar con otro sitio en el sistema.

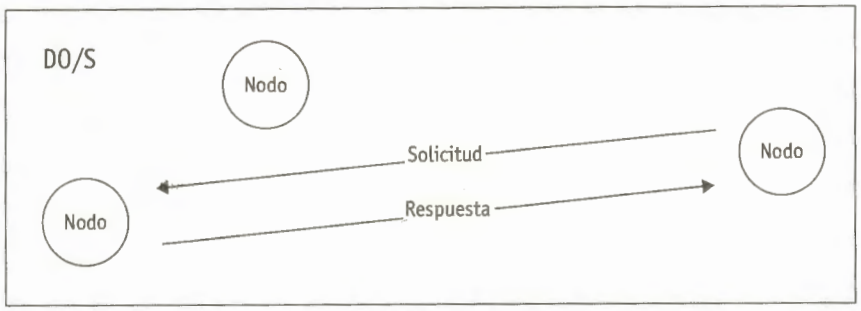
Es importante que el NOS lleve a cabo esto de manera transparente. El sistema operativo local considera la acción como si se hubiera realizado *in situ*. Esto se debe a que el sistema operativo de red maneja los detalles de interfaces y coordina el procesamiento remoto. También coordina la comunicación entre los sistemas operativos locales al seguir la pista del estado y la ubicación de todas las entidades del sistema.

Los sistemas operativos locales son sistemas operativos tradicionales diseñados para funcionar en una sola computadora. Esto significa que pueden llevar a cabo una tarea sólo si el proceso forma parte de su entorno; en caso contrario, deben pasar la solicitud hacia el sistema operativo de red para ejecutarla. Para un sistema operativo local, parece que el NOS es el servidor que ejecuta la tarea, aunque en realidad el NOS es sólo un facilitador.

La limitación más importante de un NOS es que no asume el control global sobre la administración de la memoria, la administración de procesos, la administración de dispositivos o la administración de archivos. En lugar de eso, las considera como funciones *locales* autónomas que deben interactuar entre sí. Esta visión limitada es problemática porque un sistema operativo no puede alcanzar funciones informáticas o de procesamiento realmente distribuidas sin tener el control *global* de todos los bienes, no sólo de los bienes a nivel de comunicación de red. Esta necesidad por el control global llevó al desarrollo del **sistema operativo distribuido (DO/S)**. (Aunque usan un acrónimo semejante, el DO/S descrito en este capítulo no debe confundirse con el DOS (sistema operativo de disco) para microcomputadoras, o MS-DOS, descrito en el capítulo 14.)

Los sistemas operativos distribuidos proporcionan un entorno unificado diseñado para optimizar las operaciones para la red como un todo, no sólo para sitios locales, como se ilustra en la figura 10.2.

La diferencia más importante entre un NOS y un DO/S es la manera en que cada uno considera y administra los recursos locales y globales. Un NOS cuenta con las capacidades pro-



(Figura 10.2)

En un entorno DO/S todos los nodos son parte de un sistema operativo administrado globalmente, diseñado para optimizar los recursos de todo el sistema. Las solicitudes entre nodos son manejadas por completo por el DO/S, así como toda operación en cada nodo.

porcionadas por el sistema operativo local y las amplía para satisfacer nuevas necesidades. Tiene acceso a los recursos al usar mecanismos locales y cada sistema es controlado y administrado localmente con base en esa política del sistema. Por otra parte, el DO/S considera que los recursos del sistema son su propiedad global y así los administra. Tiene acceso a los recursos usando mecanismos *globales*, más que mecanismos *locales*, que el sistema controla y administra con base en una sola política aplicable a todo el sistema. En la tabla 10.1 se presenta una comparación entre los dos tipos de sistema.

Sistema operativo de red (NOS)	Sistema operativo distribuido (DO/S)
Los recursos pertenecen a nodos locales	Los recursos pertenecen al sistema global
Los recursos locales son administrados por el sistema operativo local	Los recursos son administrados por un DO/S global
El acceso se lleva a cabo mediante un sistema operativo local	El acceso se lleva a cabo mediante el DO/S
Las solicitudes pasan de un sistema operativo local a otro mediante el NOS	Las solicitudes pasan directamente de nodo a nodo mediante el DO/S

(Tabla 10.1)

Comparación de NOS y DO/S, dos tipos de sistema operativo usados para administrar los recursos de una red.

Por ejemplo, en un entorno NOS típico, un usuario que desea ejecutar un proceso local en un sitio remoto de: 1) registrar la red local; 2) instruir al sistema local para que migre el proceso o los datos al sitio remoto, y después 3) enviar una solicitud al sitio remoto para planificar el proceso en su sistema. A partir de ahí, el sitio remoto considera el proceso como un proceso de reciente creación dentro de su entorno de sistema operativo local y lo administra sin intervención foránea. Si el proceso requiere ser sincronizado con procesos en sitios remotos, el proceso necesita tener llamadas inmersas para iniciar la acción del NOS. Estas llamadas suelen agregarse en la parte superior del sistema operativo a fin de contar con el enlace de comunicaciones entre los dos procesos en dispositivos diferentes. Esto complica la tarea de sincronización, tarea que es responsabilidad del usuario y que sólo es atendida parcialmente por el sistema operativo.

Por otra parte, un sistema administrado por un DO/S maneja el mismo ejemplo de manera diferente. Si un sitio tiene un proceso que requiere recursos de otro sitio, entonces la tarea es

presentada al DO/S simplemente como otro proceso. El usuario no asume ninguna otra responsabilidad adicional. El DO/S examina el bloque de control de procesos para determinar los requerimientos específicos para este proceso. Después, usando su planificador de procesos, el DO/S determina la mejor manera de ejecutar el proceso, con base en el conocimiento actual del sitio sobre el estado del sistema total. Luego, el planificador de procesos se hace cargo de este proceso, junto con todos los otros procesos listos para su ejecución en la red, y calcula su orden de ejecución en cada nodo a la vez que optimiza el tiempo de ejecución global y mantiene las prioridades de los procesos. El énfasis es sobre mantener la funcionalidad global, las políticas y los objetivos del sistema operativo.

Un DO/S tiene el control global de todos los bienes en el total de los nodos en la red.

Para administrar globalmente todo el conjunto de recursos de la red, un DO/S suele construirse con un sistema operativo de **núcleo** replicado; software de control de hardware de bajo nivel (firmware o programación en firme) con software a nivel de sistema para administración de recursos. Este software puede ser único o duplicado en todo el sistema. Su objetivo es asignar y administrar los recursos del sistema de modo que las políticas globales, no las políticas locales, se maximicen. El DO/S también cuenta con una capa que oculta la red y sus complejidades a los usuarios, de modo que éstos pueden usar la red como un simple sistema lógico y no como una colección de entidades cooperativas independientes.

Desarrollo del DO/S

Aunque el DO/S fue desarrollado después que el NOS, su administración global de los dispositivos de red es más fácil de comprender, de modo que la explicaremos primero.

Debido a que el DO/S administra todo el grupo de recursos dentro de la red de manera global, los recursos se asignan con base en la negociación y los compromisos entre sitios pares importantes en el sistema distribuido. Una ventaja de este tipo de sistema es su capacidad para atender el copiado de archivos, correo electrónico e impresión remota sin requerir que el usuario instale software servidor especial en máquinas locales. A continuación se describe cómo el DO/S lleva a cabo las funciones de administración del sistema operativo.

Administración de la memoria

Para cada nodo, el administrador de la memoria usa un núcleo con un algoritmo de paginación para seguir la pista de la cantidad de memoria que está disponible. El algoritmo se basa en los objetivos del sistema local pero las políticas y los mecanismos que se usan en los sitios locales son controlados por los requerimientos del sistema global. Para acomodar las necesidades locales y globales, la asignación y la liberación de la memoria dependen de los esquemas de planificación y compartición de recursos que optimizan los recursos de toda la red.

El administrador de la memoria de una red funciona de la misma forma en que lo hace para un sistema operativo independiente, aunque está ampliado para aceptar solicitudes de memoria desde fuentes locales y globales. A nivel local, el administrador de la memoria asigna páginas con base en la política local. A nivel global, recibe solicitudes del administrador de

procesos para proporcionar memoria a procesos de un cliente nuevo o en expansión o de un servidor. El administrador de la memoria también usa recursos locales para recolectar basura en la memoria, efectuar compactación, decidir cuáles son los procesos más y menos activos y determinar de cuáles procesos apropiarse para proporcionar espacio para otros.

Para controlar la demanda, el administrador de la memoria maneja las solicitudes del administrador de procesos para asignar y liberar espacio con base en los patrones de uso de la red. En un entorno distribuido, la memoria combinada para toda la red está compuesta por varios subconjuntos, uno en cada procesador, y el administrador de la memoria tiene un subcomponente que existe en cada procesador.

Cuando una aplicación intenta tener acceso a una página que no está en la memoria, ocurre un fallo de página y el administrador de la memoria automáticamente lleva la página a la memoria. Si la página es modificada mientras está en la memoria, el administrador de la memoria escribe la página modificada de regreso en el archivo cuando es el momento de sacar la página de la memoria.

Antes de asignar espacio, el administrador de la memoria examina la tabla de memoria libre total. Si es posible satisfacer la solicitud, se asigna la memoria y la tabla se modifica para mostrar la ubicación del espacio asignado.

El administrador de la memoria también administra la memoria virtual. Específicamente, asigna y libera memoria virtual, lee y escribe en la memoria virtual, cambia páginas virtuales a discos, obtiene información sobre páginas virtuales, bloquea páginas virtuales en la memoria y protege las páginas que requieren ser protegidas.

Las páginas se protegen usando hardware o software de administración de memoria de bajo nivel en el núcleo de cada sitio. Esta protección es convocada como páginas que se cargan en la memoria. En las páginas se llevan a cabo varias verificaciones de protección, como se muestra en la tabla 10.2.

Acceso permitido	Protección
Lectura/escritura	Permite a los usuarios tener completo acceso a los contenidos de la página, dándoles la posibilidad de leer y escribir.
Sólo lectura	Permite a los usuarios a leer la página, pero no se les permite modificarla.
Sólo ejecución	Permite a los usuarios utilizar la página, pero no se les permite leer o modificarla. Esto significa que, aunque un usuario esté en proceso no podrá leer o escribir en la página, pero puede saltar a una dirección en la página e iniciar el ejecutar. Esto es apropiado para compartir software de aplicación, entre editores y compiladores.
Página protegida	Utilizada para facilitar automáticamente los límites y verificación de pilas y otros tipos de estructuras de datos.
Sin acceso	Impide a los usuarios de acceder a la página. Esto es típicamente usado por depurar o programar antivirus para prevenir un proceso de lectura o escrito a una página en particular.

(Tabla 10.2)

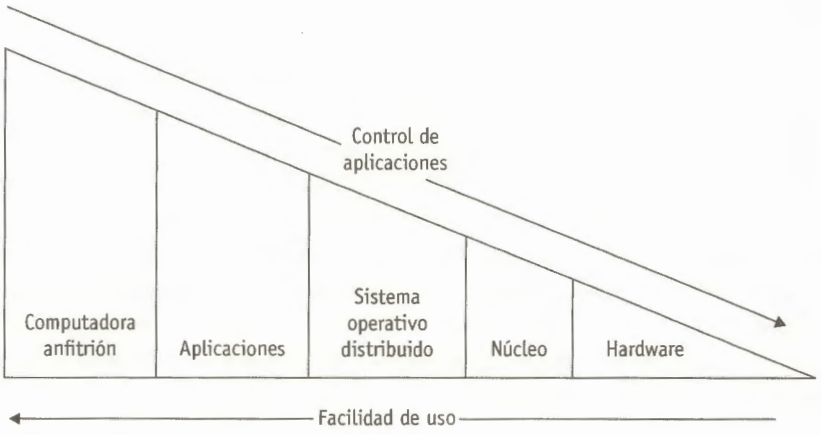
Verificaciones de protección se llevan a cabo en las páginas mientras éstas se cargan en la memoria. Los tres últimos controles mostrados en esta tabla son necesarios para asegurarse que los procesos no escriban en las páginas que deben ser de sólo lectura.

Administración de procesos

En una red, el administrador del procesador proporciona las políticas y mecanismos para crear, borrar, abortar, nombrar, renombrar, encontrar, planificar, bloquear, ejecutar y sincronizar procesos, y proporciona ejecución por prioridad en tiempo real en caso de ser necesario. Además, el administrador del procesador administra los estados de ejecución: LISTO, EN EJECUCIÓN y EN ESPERA como se describió en el capítulo 4. Para hacer esto, se espera que la CPU en la red cuente con su propio núcleo en tiempo de ejecución que administra el hardware: la operación de nivel más bajo en el dispositivo físico, como se muestra en la figura 10.3.

(Figura 10.3)

Cada núcleo controla cada pieza del hardware, incluyendo la CPU. Cada núcleo es operado por el DO/S que, a su vez es dirigido por el software de la aplicación que se ejecuta en la computadora anfitrión. De esta manera, las funciones más complicadas se ocultan al usuario.



Un núcleo controla y opera en realidad la CPU y administra las colas usadas para los estados de ejecución, aunque las políticas del sistema de nivel superior dirigen cómo los bloques de control de procesos (PCB) se almacenan en las colas y cómo se escogen para su ejecución. En consecuencia, cada núcleo asume el rol de ayudar para que el sistema alcance sus objetivos operacionales.

Los estados del núcleo dependen del planificador de procesos y del despachador del sistema global, que organizan las colas dentro de la CPU local y escogen la política de ejecución que se usa para ejecutar los procesos en esas colas. En general, la función de planificación del sistema consta de tres partes: un modo de decisión, una función de prioridad y una regla de arbitraje.

El modo de decisión determina cuáles políticas se usan al planificar un recurso. Las opciones pueden incluir de apropiación, de no apropiación, *round robin*, etcétera.

La función de prioridad proporciona al algoritmo de planificación la política que se usa para asignar un orden a los procesos en el ciclo de ejecución. La prioridad a menudo es determinada usando un cálculo que se basa en características del sistema como ocurrencia de eventos, recurrencia de tareas, niveles de carga del sistema o características del tiempo de ejecución del programa. Algunos ejemplos de estas características en tiempo de ejecución son el mayor tiempo restante (MTR), el menor tiempo restante (LTR), etcétera.

La regla de arbitraje es una política que se usa para resolver conflictos entre trabajos con la misma prioridad. Es decir, suele dictar el orden en que deben ejecutarse trabajos con la misma prioridad. Dos ejemplos de reglas de arbitraje son último en entrar, primero en salir (LIFO) y primero en entrar, primero en salir (FIFO).

La mayor parte de los avances en planificación dependen de tres teorías: teoría de colas, teoría de decisiones estadísticas y teoría de estimación. (Estas teorías de colas y de decisiones estadísticas son las mismas que las analizadas en cursos de estadística.) Un ejemplo de teoría de estimación es un planificador basado en prioridades y duración de procesos. Maximiza el resultado del sistema usando la duración para calcular y planificar la forma óptima para intercalar grupos de procesos. La planificación distribuida se logra mejor cuando la migración de la función y las políticas de planificación consideran todos los aspectos del sistema, incluyendo dispositivos de E/S, procesos y comunicaciones.

Los procesos se crean, ubican, sincronizan y borran usando procedimientos específicos. Para crear un proceso, el administrador de procesos (que forma parte del administrador del procesador) empieza por crear un bloque de control de procesos (PCB, PCB) con información semejante a los analizados en el capítulo 4, pero con información adicional que identifica la ubicación del proceso en la red. Para localizar un proceso, el administrador de procesos usa un directorio o proceso del sistema que busca todos los espacios de cola del núcleo, lo cual requiere apoyo del sistema para comunicación entre procesos. Para sincronizar procesos, el administrador de procesos usa mensajes que pasan o procedimientos de llamadas remotas. Para borrar o terminar un proceso, el administrador de procesos encuentra el bloque de control del proceso, accede a éste y lo borra.

Hay dos formas para diseñar un sistema operativo distribuido. La primera en un **DO/S basado en procesos** donde los recursos de la red son administrados como una gran colección heterogénea. La segunda y más reciente es un **DO/S basado en objetos** que agrupa cada tipo de hardware con su software operacional necesario en objetos discretos que se manipulan como una unidad. De los dos, el DO/S basado en procesos es el que se parece más a la teoría descrita en el capítulo 4.

DO/S basado en procesos

Un DO/S basado en procesos proporciona la administración de procesos mediante el uso de procesos cliente/servidor sincronizados y enlazados entre sí a través de mensajes y puertos, que también se denominan canales o tuberías. El énfasis principal es sobre los procesos y mensajes y la forma en que proporcionan las características básicas esenciales para la administración de procesos como creación de procesos, planificación, pausa, comunicación e identificación, por nombrar sólo unas cuantas.

La cooperación entre administradores es crucial para una administración de procesos exitosa en un entorno DO/S.

La cuestión de cómo proporcionar estas características puede abordarse de varias formas. Por ejemplo, los procesos pueden administrarse a partir de una sola copia del sistema operativo, a partir de múltiples pares cooperativos, o a partir de alguna combinación de los dos. Los sistemas operativos para computadoras distribuidas suelen estar configurados como un núcleo en cada sitio. Todos los otros servicios, dependientes de dispositivos particulares,

suelen encontrarse en los sitios donde se encuentran los dispositivos. A medida que los usuarios entran al sistema, se les proporciona un identificador de procesos único y después se les asigna un sitio para procesamiento por el administrador de planificación.

En un sistema distribuido, hay un alto nivel de cooperación y compartición de acciones y datos mantenidos por los sitios cuando determinan cuáles procesos deben cargarse y dónde deben ejecutarse. Esto se hace al intercambiar mensajes entre sistemas operativos de sitio. Una vez que se ha planificado un proceso para ser atendido, debe iniciarse en el sitio asignado por el despachador. Éste despachador emprende medidas a partir del planificador de procesos del sistema operativo, asigna el dispositivo al proceso y empieza su ejecución. Este procedimiento puede requerir mover un proceso de la memoria en un sitio de ésta a otro; reorganizar la asignación de un sitio en la memoria, reorganizar las colas LISTO, EN EJECUCIÓN y de ESPERA; e iniciar el proceso planificado. El administrador del procesador sólo recibe procesos y sus demandas para ser atendidos. Les responde con base en la política de planificación establecida, que determina qué debe hacerse para administrar los procesos. Como ya se mencionó en capítulos previos, las políticas de planificación deben tomar en cuenta cuestiones como balance de carga, minimización de sobrecarga, minimización de carga en la memoria y los protocolos primer llegado, primer servido y el menor tiempo restante.

La sincronización es un asunto primordial en la administración de procesos de red. Por ejemplo, los procesos pueden coordinar sus actividades al pasarse mensajes entre sí. Además, los procesos pueden pasar parámetros de sincronización de un puerto a otro usando primitivos mecanismos del sistema operativo de bajo nivel bien definidos como "enviar y recibir", para realizar la logística idónea a fin de sincronizar acciones dentro de un proceso. Por ejemplo, cuando un proceso llega a un punto en que requiere servicio de una fuente externa, como una solicitud de E/S, envía un mensaje en busca del servicio. Mientras está en espera de alguna respuesta, el procesador servidor coloca el proceso en estado de ESPERA.

Las interrupciones, que ocasionan que un procesador sea asignado a otro proceso, también están representadas como mensajes que se envían al proceso idóneo para su servicio. Por ejemplo, una interrupción puede provocar que el proceso activo se bloquee y desplace hacia un estado en ESPERA. Más tarde, cuando termina la causa de la interrupción, cada procesador servidor desbloquea el proceso interrumpido y lo restituye en un estado LISTO.

DO/S basado en objetos

Un DO/S basado en objetos cuenta con varias maneras para tratar con el sistema de cómputo. En lugar de considerar al sistema como una colección de recursos y procesos individuales, el sistema se considera como una colección de **objetos**. Un objeto puede representar hardware (como CPU y memoria), software (como archivos, programas, unidades de cinta, semáforos y datos), o una combinación de los dos (impresoras, escáners, unidades de cinta y discos, cada uno reunido en el software necesario para operarlo). Cada objeto en el sistema tiene un identificador único para diferenciarlo de todos los otros objetos en el sistema.

Los objetos son considerados como entidades abstractas, tipos de datos que experimentan una serie de cambios de estado, que actúan según patrones establecidos, son manipulados o

existen en relación con otros objetos de una manera única idónea para la semántica de los objetos en el sistema. Esto significa que los objetos tienen un conjunto de propiedades inminentes que los define a ellos y a su comportamiento en el contexto de sus parámetros definidos. Por ejemplo, una unidad de CD grabable tiene propiedades que no cambian que incluyen lo siguiente: los datos pueden escribirse en un disco, los datos pueden leerse desde un disco, la lectura y la escritura pueden realizarse de manera concurrente y no es posible comprometer los puntos inicial y final de los datos. Si estas simples reglas se siguen para construir una simulación de una unidad de CD, es posible crear una representación exacta de este objeto.

Para determinar el estado de un objeto, es necesario realizar una operación sobre éste, como leer o escribir en un disco duro, porque el objeto es identificado por el conjunto de operaciones que puede enviársele. La combinación de las operaciones con sus estructuras de datos y cálculos definidos internamente representa una instanciación del objeto. En general, los sistemas que usan este concepto cuentan con un gran número de objetos pero un pequeño número de operaciones sobre éstos. Por ejemplo, una impresora puede tener tres operaciones: una para avanzar una página completa, una para avanzar una línea y una para avanzar un caracter.

En consecuencia, en un DO/S basado en objetos, la administración de procesos se vuelve una administración de objetos, donde los procesos actúan como objetos discretos. La administración de procesos, en este caso, trata sobre las políticas y los mecanismos para controlar las operaciones, la creación y la destrucción de objetos. Por tanto, la administración de procesos tiene dos componentes: el nivel de núcleo y el administrador de procesos.

Nivel de núcleo

El **nivel de núcleo** proporciona los mecanismos fundamentales para construir el sistema operativo al crear, administrar, planificar, sincronizar y borrar objetos, lo cual realiza en forma dinámica. Por ejemplo, cuando se crea un objeto, se le asignan todos los recursos necesarios para su operación y se le proporciona el control hasta que se completa la tarea. Después, el objeto regresa el control al núcleo, que escoge el siguiente objeto a ejecutar.

El núcleo también tiene la responsabilidad final de las listas de capacidad de la red, analizadas en el capítulo 8. Cada sitio tiene tanto un administrador de capacidad que mantiene la lista de capacidad de sus objetos y un directorio que enumera la ubicación de todas las capacidades del sistema. Este directorio orienta las solicitudes locales por capacidades a otros sitios en los que están ubicados.

Por ejemplo, si un proceso solicita acceso a una región de la memoria, el administrador de capacidad primero determina si al proceso solicitante ya le han sido otorgados derechos. En caso afirmativo, entonces puede continuar. En caso contrario, procesa la solicitud por derechos de acceso. Cuando el solicitante tiene derechos de acceso, el administrador de capacidad otorga al solicitante acceso al objeto nombrado, en este caso la región en la memoria. Si el objeto nombrado está en un sitio remoto, el administrador de capacidad local dirige al solicitante, usando un nuevo cálculo y mensaje de dirección, hacia el administrador de capacidad local.

El núcleo también es responsable de los mecanismos de sincronización de procesos y apoyo para comunicación. En general, la sincronización se implementa como alguna forma de

variable compartida, como los códigos WAIT y SIGNAL analizados en el capítulo 6. La comunicación entre objetos distribuidos puede ser en la forma de objetos de datos compartidos, objetos mensaje o interacciones de control. La mayor parte de los sistemas cuentan con primitivos de comunicaciones diferentes hacia sus objetos, que son síncronos (el emisor y el receptor están enlazados y listos para emitir y recibir) o asíncronos (hay algún área compartible como un buzón, una cola o una pila a los que se envía la información comunicada). En algunos casos, el receptor periódicamente verifica si alguien ha enviado algo. En otros casos, la información comunicada llega a la estación de trabajo del receptor sin ningún esfuerzo de parte del receptor; éste simplemente espera. También puede haber una combinación de lo anterior. Un ejemplo de este modelo de comunicación podría contar con un mecanismo que indique al receptor cada que llega una comunicación al área compartible, de modo que sea posible buscar la información siempre que sea conveniente. La ventaja de este sistema es que elimina la verificación innecesaria cuando no hay mensajes en espera.

Finalmente, el entorno del núcleo para sistemas distribuidos debe tener un planificador con un mecanismo robusto y consistente para planificar objetos dentro del sistema según sus objetivos de operación.

El administrador de procesos

Si el núcleo aún no cuenta con primitivos (prueba y orden, P y V, etcétera) para trabajar con el hardware, entonces el administrador de procesos debe crear sus propios primitivos antes de proseguir con su trabajo. El administrador de procesos es responsable de las siguientes tareas: crear objetos, despachar objetos, planificar objetos, operaciones de sincronización sobre objetos, comunicación entre objetos y borrado de objetos. Para llevar a cabo estas tareas, el administrador de procesos usa el entorno del núcleo, que le proporciona los primitivos que necesita para capturar el hardware de bajo nivel en el sistema.

Por ejemplo, para ejecutar un objeto base de datos, el administrador de procesos debe seguir en orden los pasos siguientes en orden:

1. Determinar si el objeto está en la memoria o no. En caso afirmativo, ir al paso 3.
2. Si no está en la memoria, encontrar el objeto en almacenamiento secundario, asignarle espacio en la memoria y registrarlo en las ubicaciones idóneas.
3. Proporcionar al planificador de objetos la información del objeto.
4. Una vez que se ha planificado el objeto, esperar que el despachador del núcleo lo empuje y coloque en el estado de EJECUCIÓN.

Hasta el momento hemos analizado las semejanzas entre los administradores basado en objetos y procesos. La diferencia más importante entre ellos es que los objetos contienen toda su información de estado. Esto significa que la información se almacena *con* el objeto, no por separado en otra parte del sistema, como en un bloque de control de procesos (PCB) o alguna estructura de datos separada del objeto.

Administración de dispositivos

En todos los sistemas distribuidos, los dispositivos deben abrirse, leerse, escribirse y cerrarse. Además, es necesario iniciar los parámetros del dispositivo y establecer o borrar bits de estado, de la misma forma que en los sistemas independientes. Todo lo anterior puede hacerse de manera global, por grupos o localizada.

En general, los usuarios prefieren escoger los dispositivos según su nombre y dejar que el sistema operativo distribuido escoja y opere el mejor dispositivo de entre todos los disponibles. Por ejemplo, si los usuarios necesitan el control específico de un dispositivo, deben poder llamar a un dispositivo por su nombre, como DISCO 12. Una vez que se ha hecho la elección, el DO/S asume el control, asignando la unidad cuando esté disponible, asignándolo al usuario cuando se emite el comando OPEN, operándolo y luego liberándolo cuando termina el trabajo.

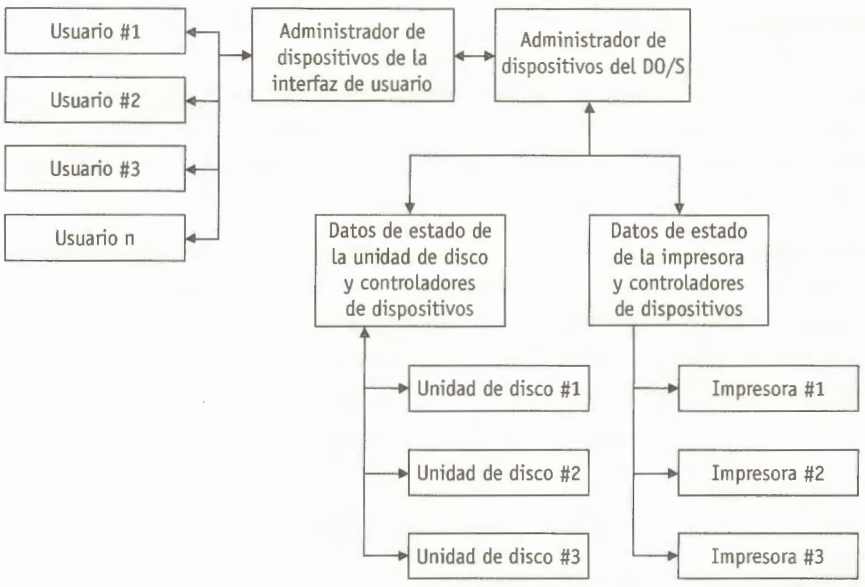
No es posible asignar el dispositivo sino hasta que el administrador de dispositivos analiza el estado del dispositivo y, cuando está libre, envía al proceso solicitante un identificador de dispositivo único: un nombre que se usa para todas las comunicaciones entre el proceso y el dispositivo. Más tarde, cuando el proceso emite un comando CLOSE, el dispositivo es liberado. Esto es cuando el DO/S rehabilita la información de estado del dispositivo y regresa su bloque de control del dispositivo a la cola LISTO del dispositivo. Por ejemplo, cuando un usuario desea imprimir un archivo al ejecutar un comando de impresión, el DO/S sigue un proceso semejante a éste:

1. El administrador de archivos del usuario coloca una copia del archivo en el directorio *spooler* del DO/S.
2. El *spooler* escoge el archivo del directorio *spooler* e inicia una solicitud abierta para el administrador de archivos del DO/S.
3. Una vez que se ha satisfecho la solicitud, el *spooler* inicia otra solicitud abierta hacia un controlador de dispositivo de impresión de red en línea.
4. Cuando se ha satisfecho la segunda solicitud abierta, el *spooler* envía el archivo al *búfer* de entrada de la impresora. Esto puede llevarse a cabo mediante la transferencia directa de un mensaje o la transferencia de un paquete, como se describió en el capítulo 9.
5. Cuando se ha completado la impresión, el administrador de archivos del DO/S borra la copia del archivo del *spooler*.
6. Finalmente, el dispositivo se habilita y cierra.

Este sistema funciona sólo si el DO/S mantiene una contabilidad global de cada dispositivo de la red y su disponibilidad, manteniendo un registro del estado y bloque de control de cada dispositivo, y distribuyendo esta información a todos los sitios. Como se muestra en la figura 10.4, el administrador de dispositivos del DO/S es una colección de controladores de dispositivos remotos conectados y asociados con los dispositivos, aunque controlado por datos de estado que son proporcionados por el administrador de dispositivos del DO/S.

DO/S basado en procesos

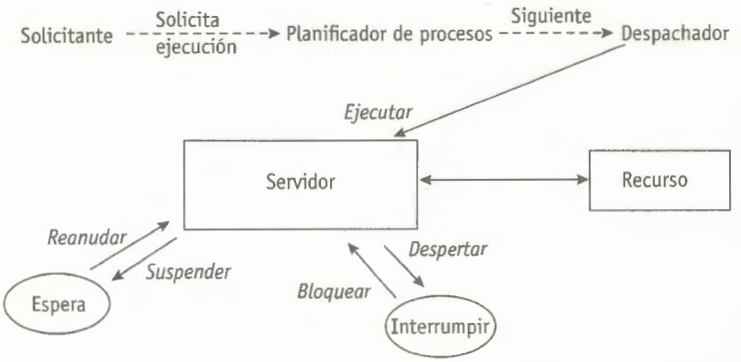
Todos los recursos en el DO/S basado en procesos son controlados por servidores denominados "guardianes" o "administradores". Estos servidores son responsables de aceptar las



(Figura 10.4)
Todos los dispositivos son operados por sus odministradores de dispositivos o controladores de dispositivos individuales usando datos de estado específicos controlados por el administrador de dispositivos del DO/S. En este ejemplo, la red tiene tres unidades de disco y tres impresoras.

solicitudes de servicio en los dispositivos individuales que controlan, procesar cada solicitud de manera justa, proporcionar servicio al solicitante y volver a servir a otros, como se muestra en la figura 10.5.

Sin embargo, no todos los sistemas cuentan con una colección de recursos simple. Muchos tienen grupos de impresoras, unidades de disco, cintas, etcétera. Para controlar estos grupos, la mayor parte de los sistemas basados en procesos están configurados alrededor de complicados **procesos de servidor**, que administrar múltiples recursos o dividen el trabajo entre procesos subordinados. El administrador de procesos está configurado como un administrador de dispositivos e incluye el software necesario para aceptar solicitudes locales y remotas para su servicio, descifrar su significado y actuar sobre ellas. En general, un proceso servidor consta de uno o más controladores de dispositivos, un administrador de dispositivos y una de componentes de servidor de red.



(Figura 10.5)
En un DO/S basado en procesos, las solicitudes se desplazan desde el solicitante hasta el servidor pasando por el planificador de procesos y el despachador. El procesamiento de interrupciones administra E/S o problemas de procesamiento. El estado WAIT se usa para suspender y reanudar el procesamiento. Funciona de manera idéntica al estado WAIT descrito en el capítulo 4.

DO/S basado en objetos

En un DO/S basado en objetos, cada dispositivo es administrado de la misma forma a través de toda la red. El dispositivo físico se considera un objeto, justo como cualquier otro recurso de la red, y está rodeado por una capa de software que proporciona a los otros objetos una visión completa del objeto dispositivo.

El dispositivo físico es manipulado por un conjunto de operaciones, comandos explícitos que movilizan el dispositivo para que desempeñe sus funciones diseñadas. Por ejemplo, un objeto para controlar una unidad de cinta requiere las operaciones de rebobinado, adelantamiento rápido y escaneo. Para usar una unidad de cinta, los usuarios emiten una operación en un objeto cinta como éste:

```
WITH TAPE 1 DO FAST FORWARD (N) RECORDS
```

Esto hace que la unidad de cinta avance *N* registros. Para lo anterior, se supone, por supuesto, que el sistema operativo ya ha otorgado autorización al usuario para que use el objeto cinta.

Una unidad de disco funciona de la misma manera. Los usuarios tienen acceso a la unidad al enviar operaciones al administrador de dispositivos para que cree un nuevo archivo, destruya el archivo anterior, abra o cierre un archivo existente, lea información desde un archivo, o escriba ahí. Los usuarios no necesitan conocer los mecanismos subyacentes que implementan las operaciones: les basta saber cuáles operaciones funcionan.

Los detalles requeridos para operar cada unidad de disco están contenidos en el software del controlador de dispositivos.

Una ventaja de un DO/S basado en objetos es que los objetos pueden reunirse para comunicarse y sincronizarse entre ellos a fin de proporcionar una red distribuida de recursos, donde cada objeto conoce la ubicación de sus pares distribuidos. Entonces, si el administrador de dispositivos local no puede satisfacer una solicitud de un usuario, la solicitud es enviada a otro administrador de dispositivos, un par. De nuevo, los usuarios no necesitan saber si los recursos de la red están centralizados o distribuidos: sólo que sus solicitudes han sido cumplidas.

Para que este sistema sea exitoso, el objeto administrador de dispositivos en cada sitio necesita mantener un directorio actual de objetos dispositivo en todos los sitios. Luego, cuando un objeto solicitante necesita usar la impresora, por ejemplo, la solicitud es presentada a su administrador de dispositivos local. Si el administrador local cuenta con los medios y la oportunidad de proporcionar el servicio idóneo, imprime la solicitud. En caso de que no pueda satisfacer localmente la solicitud, envía la solicitud a un administrador de dispositivos par que tenga los recursos apropiados. Es este administrador de dispositivos remoto el que procesa la solicitud y realiza la operación.

Administración de archivos

La administración de archivos distribuida da a los usuarios la impresión de que la red es un sistema de archivos lógico único implementado con base en un surtido de dispositivos y computadoras. En consecuencia, la función más importante de un administrador de archivos DO/S consiste en proporcionar mecanismos transparentes para encontrar y abrir, leer, escribir, cerrar, crear y borrar archivos, sin importar dónde se encuentren ubicados en la red, como se muestra en la tabla 10.3

(Tabla 10.3)
Funciones de administración de archivos típicas y las acciones necesarias del administrador de archivos.

Función deseada del archivo	Acción del gestor de archivos
Encontrar y abrir	Usa un directorio maestro con información sobre todos los archivos almacenados en cualquier parte en el sistema y establece un canal hacia el archivo.
Leer	Establece un canal hacia el archivo e intenta leerlo usando esquemas simples de acceso a archivos. Sin embargo, una operación de lectura no funciona si el archivo está siendo creado o modificado en ese momento.
Escribir	Establece un canal hacia el archivo e intenta escribir en éste usando esquemas simples de acceso a archivos. Para escribir en un archivo, el proceso solicitante debe llevarse a cabo al bloquear el archivo, una técnica de uso común en sistema de bases de datos. Mientras un archivo está bloqueado, todos los otros procesos solicitantes deben esperar hasta que el archivo se desbloquee antes de poder escribir o leer desde un archivo.
Cerrar	Envía un comando al servidor remoto para desbloquear ese archivo. Esto suele llevarse a cabo al cambiar la información en el directorio en el sitio de almacenamiento del archivo.
Crear	Crea un identificador de archivo único en el directorio maestro de la red y le asigna espacio en un dispositivo de almacenamiento.
Borrar	Borra el identificador de archivo único en el directorio maestro y libera el espacio reservado para el archivo en el dispositivo de almacenamiento.

Los sistemas de administración de archivos son un subconjunto de los administradores de bases de datos, lo cual proporciona más capacidades a los procesos del usuario que los sistemas de archivos y se implementan como sistemas de administración de bases de datos distribuidos como parte de sistemas de red de área local.

En consecuencia, las tareas requeridas por un DO/S incluyen las tareas que suelen encontrarse en un entorno de bases de datos distribuido. Algunas implican un anfitrión de controles y mecanismos necesarios para proporcionar una administración consistente, sincronizada y confiable del sistema y bienes del usuario, incluyendo lo siguiente:

- Control de concurrencia.
- Redundancia de datos.
- Transparencia de ubicación y directorio distribuido.
- Resolución o recuperación de bloqueos.
- Procesamiento de consultas.

Control de concurrencia

Las técnicas de control de concurrencia otorgan al sistema la capacidad de realizar lecturas y escrituras concurrentes, en la medida en que los resultados de estas acciones no pongan en peligro el contenido de la base de datos. Esto significa que los resultados de todas las transacciones concurrentes son los mismos que si éstas se hubieran ejecutado una a la vez, en algún orden arbitrario en serie, proporcionando así la visión de ejecución en serie sobre una base de datos. El mecanismo de control de concurrencia mantiene la base de datos en un estado consistente a medida que se procesan las transacciones.

Por ejemplo, suponga que un grupo de agentes de reservaciones para aerolíneas hace arreglos para los clientes que llaman por teléfono. Al usar técnicas de control de concurrencia, el administrador de archivos permite que cada agente lea y escriba en la gran base de datos de la aerolínea si, y sólo si, cada lectura y escritura no interfiere con ninguna otra que se esté llevando a cabo. Estas técnicas proporcionan una visión de ejecución en serie sobre la base de datos.

El bloqueo (como a nivel de campo o registro de una base de datos) es una forma de control de concurrencia.

Redundancia de datos

La redundancia de datos puede hacer que la lectura de archivos sea más rápida y fácil. Esto se debe a que el administrador de archivos permite un proceso de leer la copia más próxima o fácil de acceder. O, si el archivo es muy grande, la solicitud de lectura puede dividirse en varias solicitudes diferentes, cada una de las cuales se cumple en una ubicación de archivo distinta. Por ejemplo, si un sistema de reservaciones aéreas recibe una solicitud de información sobre los pasajeros en cierto vuelo, y la base de datos está almacenada en tres ubicaciones diferentes, entonces una solicitud de lectura podía buscar los pasajeros por apellido empezando con A-K, la segunda solicitud podría buscar L-R y la tercera, S-Z. Los resultados de las tres solicitudes se combinan antes de regresar el resultado al solicitante.

La redundancia de datos también tiene aspectos benéficos desde un punto de vista de recuperación de desastres porque si un sitio falla, las operaciones pueden volver a iniciarse en otro sitio con los mismos recursos. Más tarde, el sitio con falla puede reinstalarse al copiar todos los archivos que se actualizaron a partir del momento de la falla. La desventaja de datos redundantes es la tarea de mantener todo el tiempo actualizadas múltiples copias del mismo archivo. Cada que se actualiza un archivo, todas las otras copias del archivo deben actualizarse de la misma forma y la actualización debe efectuarse según las normas de confiabilidad del sistema.

Con base en el algoritmo usado y en el método de recuperación, el sistema puede requerir que las actualizaciones se realicen en todos los sitios antes que ocurra alguna lectura en un sitio maestro o en una mayoría de los sitios. Algunos algoritmos de actualización de uso común con acuerdo unánime, copia en el sitio primario, movimiento del sitio primario y acuerdo de mayoría de sitios.

Transparencia de la ubicación y directorio distribuido

Transparencia de la ubicación significa que a los usuarios no les concierne la ubicación física de sus archivos. En lugar de ello, tratan con la red como si fuese un solo sistema. La transparencia de la ubicación es proporcionada por mecanismos y directorios que asignan artículos de datos lógicos a ubicaciones físicas. Los mecanismos suelen usar información sobre los datos que está almacenada en todos los sitios en forma de directorios.

El directorio distribuido administra la transparencia de la ubicación de los datos y mejora la recuperación de datos para los usuarios. El directorio contiene definiciones relacionadas con la estructura física y lógica de los datos almacenados, así como las políticas y los mecanismos para asignar entre ambos. Además, contiene los nombres de todos los recursos del sistema y los mecanismos de direccionamiento para ubicarlos y tener acceso a ellos.

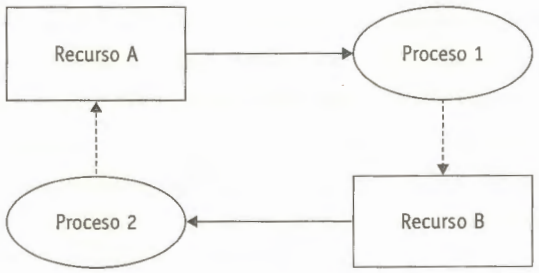
Resolución o recuperación de bloqueos mutuos

La detección y recuperación de bloqueos mutuos, descritas en el capítulo 5, son cuestiones críticas en sistemas distribuidos. La función más importante es la detección y recuperación de una espera circular. Esto ocurre cuando un proceso solicita un recurso (como un archivo, una unidad de disco, un módem o una unidad de cinta), que se denomina recurso B, mientras mantiene el control exclusivo sobre otro recurso, que se denomina recurso A. Mientras tanto, un segundo proceso solicita el uso del recurso A mientras mantiene el control exclusivo sobre el recurso B. En la figura 10.6 se muestra una gráfica dirigida para este ejemplo (las líneas continuas representan recursos asignados a los procesos, y las líneas punteadas, solicitudes de recursos por los procesos).

Sin embargo, la mayor parte de los ejemplos de la vida real de espera circular son mucho más complicados y difíciles de detectar porque implican múltiples procesos y múltiples recursos: todos en espera de un recurso que es retenido exclusivamente por otro proceso.

(Figura 10.6)

Este ejemplo de espera circular fue creado cuando el proceso 1 solicitó el recurso B sin liberar su control exclusivo sobre el recurso A. En forma semejante, el proceso 2 solicitó el recurso A sin liberar el recurso B.



Detección, prevención, evasión y recuperación son –todas– estrategias usadas por un sistema distribuido. Para reconocer esperas circulares, el sistema usa gráficas dirigidas de recursos y busca ciclos. Para prevenir esperas circulares, el sistema intenta retrasar el inicio de una transacción hasta que tiene todos los recursos que solicita durante su ejecución. Para evitar esperas circulares, el sistema trata de permitir la ejecución sólo cuando sabe que la transacción puede ejecutarse hasta su terminación. Para recuperarse de un bloqueo ocasionado por esperas circulares, el sistema escoge la mejor víctima; una que puede reiniciarse sin mucha dificultad y una que, al terminar, libere suficientes recursos de modo que otras puedan terminar. Después, el sistema mata a la víctima, obliga al proceso a reiniciarse desde el principio y reasigna sus recursos a los procesos en espera.

Procesamiento de consultas

El procesamiento de consultas es la función de procesamiento de solicitudes de información. Las técnicas para el procesamiento de consultas intentan incrementar la efectividad de la ejecución de secuencias de consultas globales, secuencias de procesamiento en un sitio local y secuencias de procesamiento de dispositivos. Todo esto se relaciona directamente con el problema de planificación de procesos global de la red. En consecuencia, para asegurar consistencia del esquema de planificación de todo el sistema, la estrategia del procesamiento de consultas debe ser parte integral de la estrategia de planificación de procesamiento.

Administración de redes

La función de administración de redes es una función de comunicación única de los sistemas de red porque los sistemas independientes no requieren comunicarse con otros sistemas. Para un DO/S, el administrador de la red cuenta con las políticas y los mecanismos necesarios para proporcionar comunicación intrasitios y comunicación entre sitios entre procesos concurrentes. Para procesos intrasitios dentro de la red, el administrador de la red proporciona identificadores de proceso –en una manera uno a uno, uno a pocos, uno a muchos y uno a todos– a la vez que administran dinámicamente estas rutas.

El administrador de la red tiene muchas responsabilidades diversas. Debe poder localizar procesos en la red, enviar mensajes en toda ésta y seguir la pista del uso de los medios. Además, debe poder transferir datos de manera confiable, codificar y decodificar mensajes, retransmitir errores, efectuar verificaciones de paridad, de redundancia cíclicas, establecer enlaces redundantes y reconocer mensajes y respuestas, en caso de ser necesario.

El administrador de la red empieza por registrar cada proceso en la red tan pronto como se le ha asignado un identificador físico único. Envía este identificador a todos los otros sitios en la red. A partir de ese momento, el proceso es registrado en todos los sitios de la red.

Cuando los procesos u objetos requieren comunicarse entre sí, el administrador de la red los enlaza entre sí mediante un puerto: una puerta lógica sobre un proceso que puede enlazarse con un puerto sobre otro proceso, estableciendo así una ruta lógica para que se comuniquen los dos. En general, los puertos tienen *búfers* y canales de E/S físicos, que representan bienes físicos que es necesario usar prudentemente por el administrador de la red. Los puertos pueden asignarse a un proceso o muchos.

Los procesos requieren encaminamiento debido a la topología de red subyacente y a la ubicación de los procesos. El encaminamiento puede ser tan simple como una dirección de un par proceso-dispositivo que asocia un proceso lógico con un sitio físico. O puede incorporar muchos niveles que atraviesan múltiples enlaces ya sea en forma directa o por saltos, como se describió en el capítulo 9.

Además del encaminamiento, otras funciones requeridas por el administrador de la red son mantener estadísticas sobre el uso de la red (para su uso en planificación de mensajes, localización de fallos, y rencaminamiento), así como para proporcionar mecanismos para ayudar a la sincronización de tiempo de los procesos. Este mecanismo de estandarización suele denominarse reloj para todo el sistema, un dispositivo que permite que los componentes del sistema en varios sitios compensen las variaciones de tiempo debido a retrasos ocasionados por el sistema de comunicación distribuido.

DO/S basado en procesos

En un DO/S basado en procesos, las comunicaciones interprocesos son transparentes para los usuarios. El administrador de la red asume toda la responsabilidad de asignar puertos a

los procesos que los solicitan, identificar cada proceso en la red, controlar el flujo de mensajes y garantizar la transmisión y aceptación de mensajes sin errores.

El administrador de la red actúa rutinariamente como el mecanismo de interfaz de todos los procesos en el sistema y manipula el tráfico de mensajes, aliviando a los usuarios de la necesidad de conocer dónde están ubicados físicamente sus procesos en la red. Como operador del tráfico, el administrador de la red acepta e interpreta cada comando de cada proceso para enviar y recibir. Luego transforma estos comandos en acciones de bajo nivel que realmente transmiten los mensajes a través de enlaces de comunicación.

DO/S basado en objetos

Un objeto administrador de la red realiza fácilmente comunicaciones intermodos e intramodos entre objetos cooperativos. Un proceso, los elementos del código activo dentro de los objetos, puede iniciar una operación en una instancia específica de un objeto al enviar un mensaje de solicitud. El usuario no necesita conocer la ubicación del receptor. El usuario sólo requiere conocer el nombre del receptor y el administrador de la red se hace cargo, proporcionando al receptor el encaminamiento idóneo del mensaje. Un proceso también puede invocar una operación que es parte de su entorno de objeto local.

En general, las redes de comunicaciones permiten algún nivel de las siguientes funciones: enviar, recibir, solicitar y contestar, como se muestra en la tabla 10.4.

(Tabla 10.4)

Las comunicaciones enviadas por el administrador de la red permiten que los objetos realicen por lo menos una de cuatro funciones.

Función	Propósito
Enviar	Permite que los objetos envíen un mensaje con operaciones a cualquier objeto en el sistema.
Recibir	Advierte a los objetos de comunicaciones provenientes de un objeto en el sistema.
Solicitar	Proporciona un mecanismo para que los objetos soliciten servicios particulares. Por ejemplo, pueden solicitar que se envíe un mensaje.
Contestar	Permite a los objetos hacer una de tres cosas: • Contestar la solicitud de comunicaciones desde otro objeto • Contestar a un comando de enviar que no están preparados para • Indicar que están listos para aceptar un comando de enviar; ahora es posible enviar un mensaje sabiendo que el receptor está esperando.

Los servicios del administrador de la red suelen proporcionarse a nivel de núcleo para acomodar mejor muchos objetos que los usan y ofrecer servicio eficiente. Sin embargo, dependiendo del sistema, el administrador de la red puede ser una simple utilidad que maneja sólo primitivos de envío y recepción. O quizás está construido usando puertos, o canales. Una simple utilidad de emisión-recepción requiere que los usuarios conozcan el nombre de los objetos con los que necesitan comunicarse, mientras un sistema de puertos o canales

requiere que los usuarios sólo conozcan el nombre del puerto o canal con que está asociado cada objeto. Por ejemplo, si una facilidad de comunicaciones es atendida por un mecanismo del tipo puerto-objeto, entonces los objetos se agrupan como emisores, receptores, puertos y mensajes, y están enlazados entre sí mediante el uso de listas de capacidad.

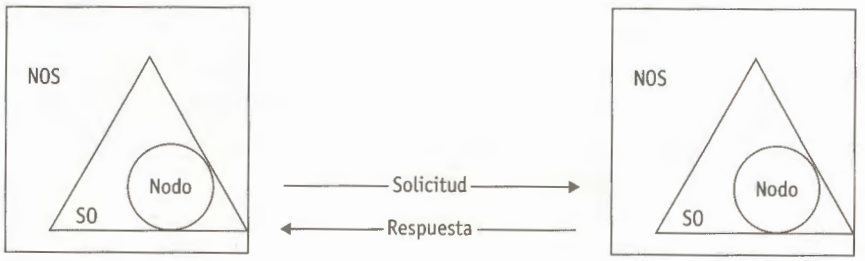
Desarrollo del NOS

Un sistema operativo de red (NOS) típicamente se ejecuta en una computadora denominada **servidor** y efectúa servicios para estaciones de trabajo de red denominadas **clientes**. Aunque las computadoras pueden asumir el rol de cliente la mayor parte del tiempo, cualquier computadora dada puede asumir el rol de servidor (o cliente), dependiendo de los requerimientos de la red. Cliente y el servidor no son términos específicos del hardware. En lugar de eso, son términos específicos de roles.

Antes que un nodo pueda comunicarse con otro nodo sobre la red, el NOS del primer nodo debe abrir comunicaciones con el NOS del nodo objetivo en la red.

Muchos sistemas operativos de red modernos son verdaderos sistemas operativos que incluyen las cuatro funciones administrativas: administración de la memoria, planificación de procesos, administración de archivos y administración de dispositivos, incluyendo operaciones del disco y de E/S. Además, cuentan con una función de administración de la red con responsabilidad de comunicaciones de la red, protocolos, etcétera. En un NOS, las funciones de administración de la red entran en juego sólo cuando el sistema requiere usar la red, como se muestra en la figura 10.7. El resto del tiempo, el administrador de la red permanece dormido y el sistema operativo opera como si fuese un sistema independiente.

Aunque los NOS pueden ejecutar aplicaciones, así como otros sistemas operativos, su foco de interés consiste en asegurarse de compartir recursos, más que en ejecutar programas. Por ejemplo, un sistema operativo de usuario único como las primeras versiones de Windows, e incluso



(Figura 10.7)
En un entorno NOS los cuatro administradores del sistema operativo administran todos los recursos del sistema en ese sitio, a menos que y hasta que el nodo necesite comunicarse con otro nodo de la red.

un sistema multiusuario como UNIX o Linux, se enfocan en la habilidad del usuario para ejecutar aplicaciones. Por otra parte, los sistemas operativos de red se enfocan en las capacidades de las estaciones de trabajo para compartir los recursos del servidor incluyendo aplicaciones y datos, así como recursos compartidos costosos como impresoras y módems de alta velocidad.

En las páginas siguientes describiremos algunas de las características comúnmente encontradas en un sistema operativo de red sin enfocarnos en ninguno en particular. La mejor elección del NOS depende de muchos factores, incluyendo las aplicaciones que se ejecutarán en el servidor, el soporte técnico requerido, el nivel de capacitación del usuario y la compatibilidad del hardware con otros sistemas operativos de red.

Características importantes del NOS

La mayor parte de los sistemas operativos están implementados como las primeras versiones de software de 32 o 64 bits. Sin embargo, más importante que el procesador es el soporte proporcionado por el NOS para tecnologías normales de redes de área local y sistemas operativos cliente de escritorio.

Las redes se están volviendo cada vez más heterogéneas. Es decir, atienden estaciones de trabajo que ejecutan una amplia gama de sistemas operativos. Por ejemplo, una sola red podría incluir estaciones de trabajo que ejecutan Windows, el sistema operativo Macintosh y Linux. Para que un NOS sirva como elemento aglutinante, debe proporcionar un fuerte apoyo para cada sistema operativo en la red de información corporativa, sosteniendo tantas normas actuales como sea necesario. En consecuencia, debe contar con una arquitectura robusta que se adapte fácilmente a otras tecnologías.

Por ejemplo, un NOS debe preservar las expectativas del usuario para un sistema de escritorio. Esto significa que los recursos de la red deben verse como simples extensiones de las del sistema operativo del usuario. Por ejemplo, en una computadora Windows, una unidad de red debe verse justo como otro disco duro, pero con otro nombre de volumen y una letra de unidad diferente. En una computadora Macintosh, la unidad de red debe verse como un icono para un volumen sobre el escritorio. Y en un sistema Linux, la unidad debe parecer como un sistema de archivos montable.

Un NOS está diseñado para operar una amplia gama de aplicaciones de software de tercer partido y dispositivos de hardware, incluyendo unidades de discos duros, módems, unidades de CD-ROM y tarjetas de interfaz de red. Un NOS también atiende software para aplicaciones de red multiusuario, como mensajes electrónicos, así como expansiones de red como nuevas pilas de protocolos.

Finalmente, el NOS debe combinar eficiencia con seguridad. Su objetivo constante es proporcionar a los clientes de la red acceso fácil y rápido a los datos y recursos de la red sin comprometer la seguridad de la red. Una red comprometida pierde de manera instantánea integridad de los datos y la confianza de sus usuarios.

Funciones primordiales del NOS

Una función importante del NOS consiste en dejar que los usuarios transfieran archivos de una computadora a otra. En ese caso, el sistema controla y administra sus propios sistemas de archivos. Por ejemplo, internet cuenta con el programa **protocolo de transferencia de archivos** (FTP). Entonces, si los estudiantes en una clase de programación de UNIX necesitan copiar un archivo de datos de la computadora de la universidad a sus computadoras personales para completar una asignación, entonces cada estudiante empieza por emitir el siguiente comando para crear la conexión FTP:

```
ftp unixs.cis.pitt.edu
```

Esto abre el programa FTP, que luego pide al estudiante un nombre de registro y una contraseña. Una vez que esta información la verifica el sistema operativo UNIX, a cada estudiante se permite copiar el archivo de la computadora anfitrión usando este comando:

```
get filename.ext
```

En este ejemplo, *filename.ext* es el nombre absoluto del archivo y la extensión requerida por el nombre del archivo. Esto significa que el usuario debe saber *exactamente* dónde está ubicado el archivo: en qué directorio y en qué subdirectorio está almacenado. Esto se debe a que la ubicación del archivo no es transparente para el usuario.

Esta técnica de encontrar y copiar no se considera una verdadera compartición de archivos porque todos los usuarios que desean tener acceso al archivo de datos deben copiar el archivo en sus propios sistemas, duplicando así el código y desperdiciando espacio. Esta práctica también agrega dificultades al **control de la versión**, ya que cuando un usuario modifica de cualquier manera un archivo, los cambios se reflejan en otras copias ya almacenadas en otros directorios, a menos que cada usuario sustituya la versión anterior con la nueva.

Para que una colección de archivos esté a disposición del público en general, los archivos deben colocarse en un directorio FTP público sobre un servidor FTP. Luego, cualquier persona con un cliente FTP o navegador de red puede descargar copias de los archivos sobre demanda usando un proceso denominado **FTP anónimo**. La ventaja de una conexión FTP es que no obliga a que los archivos pasen a través de programas de correo que pudieran codificar o decodificar el documento. En consecuencia, todos los documentos FTP suelen descargarse más rápido a la vez que preservan sus características necesarias, incluyendo plantillas, formateo y gráficas.

Los visitantes de la web no suelen acudir al servidor FTP hasta que desean descargar un archivo. Cuando hacen clic en la liga de descargar, son transferidos al servidor FTP. Puesto que los sitios FTP no suelen estar diseñados para una navegación fácil sin que los navegadores de la web puedan usarse para manipular archivos FTP con algunas limitaciones. La mayor parte de los navegadores de la web exhiban directorios FTP usando iconos gráficos para representar archivos y directorios disponibles. Es posible usar comandos y características típicos de un navegador de la web para descargar archivos FTP justo como hacen otros objetos de la web. La ventaja de usar la web para FTP es que la mayoría de los usuarios conocen la forma en que trabaja el navegador de la web, de modo que no requieren aprender más software.

Conclusión

Los primeros sistemas operativos para red fueron NOS. Aunque son muy adecuados, no aprovechaban por completo los recursos globales disponibles para todos los sitios conectados. El desarrollo de sistemas operativos distribuidos (DO/S) abordó específicamente esta necesidad.

Todo sistema de red, NOS o DO/S, tiene requerimientos específicos. Cada uno es seguro en cuanto a acceso no autorizado sin dejar de ser accesible para los usuarios autorizados. Cada uno debe monitorear sus recursos del sistema disponibles, incluyendo memoria, procesadores, dispositivos y archivos (según se describió en los capítulos 2 a 8), así como sus enlaces de comunicación. Además, debido a que se trata de un sistema operativo, debe efectuar las tareas de red requeridas descritas en el capítulo 9.

Cada uno de los avances tecnológicos que hemos analizado hasta el momento ha creado vulnerabilidades de seguridad. A los expertos en seguridad de los sistemas les agrada decir que la única computadora segura es aquella que está sin conectar a la fuente de energía. En el capítulo siguiente abordaremos los elementos clave de seguridad y ética de los sistemas, y el rol del administrador del sistema en cuanto a resguardar el recurso más valioso de la red: los datos.

Términos clave

Cliente: Un nodo usuario que solicita y usa varios servicios de la red.

Control de versión: Rastreo y actualización de alguna versión específica de una pieza de hardware o software.

DO/S basado en objetos: Una visión de sistemas operativos donde cada unidad de hardware está agrupada con su software operacional requerido, constituyendo un objeto discreto que se manipula como una entidad.

DO/S basado en procesos: Visión de un sistema operativo distribuido que abarca todos los recursos y procesos del sistema.

FTP anónimo: Una utilización del protocolo de transferencia de archivos que permite al usuario recuperar documentos, archivos, programas y otros datos desde cualquier parte en internet sin necesidad de establecer una identificación y una contraseña del usuario.

Groupware: Aplicaciones de software que atienden trabajo corporativo a través de una red.

Nivel de núcleo: En un sistema operativo distribuido basado en objetos, proporciona los mecanismos básicos para construir dinámicamente partes del sistema operativo al crear, administrar, planificar, sincronizar y borrar objetos.

Núcleo: Parte del sistema operativo que reside en la memoria principal todo el tiempo y realiza las tareas más esenciales, como administración de la memoria y manipulación de entrada y salida de discos.

Objeto: Cualquiera de muchas entidades que constituyen un sistema de cómputo, como CPU, terminales, unidades de disco, archivos o bases de datos.

Primitivas: Mecanismos del sistema operativo de bajo nivel bien definidos y predecibles que permiten que las componentes del sistema operativo de alto nivel ejecuten sus funciones sin considerar manipulación directa del hardware.

Procesamiento distribuido: Método de procesamiento de datos donde los archivos se almacenan en muchas ubicaciones diferentes y donde el procesamiento se lleva a cabo en sitios diferentes.

Proceso servidor: Unidad lógica compuesta por uno o más controladores de dispositivos, un administrador de dispositivos y un módulo servidor de red necesarios para controlar grupos o dispositivos semejantes en un entorno de sistema operativo distribuido basado en procesos.

Protocolo de transferencia de archivos (FTP): Protocolo que permite a un usuario en un anfitrión que tenga acceso y transfiera archivos hacia o desde otro anfitrión sobre una red TCP/IP.

Servidor: Nodo que proporciona a los clientes varios servicios de red como recuperación de archivos, impresión o servicios de acceso a bases de datos.

Sistema operativo de red (NOS): Software que administra los recursos de la red para un nodo en la red y puede proporcionar seguridad y control de acceso.

Sistema operativo distribuido (DO/S): Sistema operativo que proporciona el control para un sistema de cómputo distribuido, permitiendo tener acceso a sus recursos en forma unificada.

Búsquedas de interés

- Control de concurrencia
- Procesamiento en red
- Sincronización de procesos en red
- Núcleo de sistemas operativos
- Protocolo de transferencia de archivos

Ejercicios

Tema de investigación

A. Investigue el concepto de procesamiento de consultas en sistemas operativos distribuidos. Abunde en la breve descripción proporcionada en este capítulo y explique qué es y (en términos simples) cómo funciona en una red distribuida. ¿Es una herramienta necesaria? ¿El procesamiento de consultas incrementa la velocidad de procesamiento? ¿Por qué? ¿Cuál es el efecto que tiene el tamaño de las redes sobre la velocidad del procesamiento de consultas? Cite sus fuentes.

B. Identifique cuatro sistemas operativos iniciales para redes y explique a qué grupo pertenecen (como se definió en este capítulo), NOS o DO/S. Establezca las razones para su respuesta y cite sus fuentes.

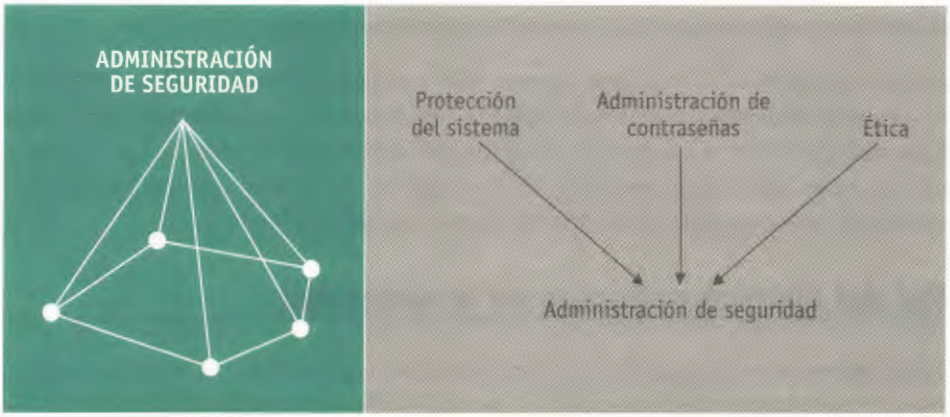
Ejercicios

1. Compare brevemente las ventajas de un NOS con un DO/S y explique cuál preferiría para operar y por qué.
2. Si su DO/S tiene cuatro nodos, ¿cuántos sistemas operativos puede tener? Explique su respuesta.
3. Si su NOS tiene cuatro nodos, ¿cuántos sistemas operativos puede tener? Explique su respuesta.
4. En este capítulo analizamos varios niveles de archivos de acceso incluyendo lectura/escritura, sólo lectura, sólo ejecución, protección de página y sin acceso. Si usted mantuviera una base de datos que contiene información confidencial de pacientes y necesita restringir el acceso UPDATE para cinco individuos, restringir el acceso READ a otros 15 e impedir el acceso a cualquier otro usuario, entonces ¿qué nivel de acceso puede usted asignar a los individuos en cada uno de los tres grupos: los cinco elegidos, los otros 15 y cualquier otro? Explique su respuesta.
5. Explique con sus propias palabras los pasos que un administrador de archivos DO/S sigue para abrir un archivo, leer los datos de éste, actualizarlos y cerrar el archivo. ¿Cambiarán estos pasos si los datos del archivo no cambian? Explique las razones de su respuesta.
6. Mencione tres beneficios de la redundancia de datos como se describió en este capítulo.
7. Describa en detalle cómo un DO/S protege un archivo de acceso o modificación de un usuario no autorizado. Compare lo anterior con la protección de archivos NOS.
8. Los bloques de control de procesos (PCB) se analizaron en el capítulo 4 en el contexto de sistemas no conectados en red. En un sistema operativo distribuido, ¿qué información adicional es necesaria observar en el PCB para que el administrador de procesamiento administre exitosamente los procesos correctos?
9. Describa cómo el administrador de procesos del DO/S actúa para prevenir bloqueos mutuos en la red. Luego describa cómo éste intenta evitar esta situación y si es necesario, recuperarse de un bloqueo mutuo.
10. Explique por qué el proceso de sincronización es crítico en la administración de procesos en red.

Ejercicios avanzados

11. ¿Qué tipo de estructura de red (NOS o DO/S) cree que es menos probable que colapse o congele si un mal funcionamiento del spooler causa un bloqueo mutuo en un nodo? Explique su conclusión y diga si un sistema robusto es o no razón suficiente para seleccionar una estructura sobre la otra.
12. Compare y contraste las dos variedades de sistemas operativos distribuidos analizados en este capítulo: DO/S basado en procesos y DO/S basado en objetos.
13. Si usted administrara la red de un hospital, ¿qué políticas implementaría para proteger su sistema? Tenga en cuenta que, como administrador de un sistema, su trabajo consiste en proporcionar el nivel de accesibilidad correcto a usuarios autorizados, a la vez que rehúsa el acceso a usuarios no autorizados.
14. Al recordar el análisis de bloqueo en el capítulo 5, si usted estuviera diseñando un sistema de red, ¿cómo manipularía la amenaza de bloqueos en su red? Considere todo lo siguiente: prevención, detección, evasión y recuperación.
15. Describa las 10 funciones críticas de un administrador de redes para un sistema operativo distribuido. Identifique el nivel de acceso requerido por el administrador y explique por qué. Describa cómo administraría el control de acceso para una red con múltiples administradores del sistema.

Capítulo 11 Seguridad y ética



"La perfección de los medios y la confusión de los objetivos parecen, en mi opinión, caracterizar nuestra era."

Albert Einstein (*1879-1955*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- El rol del sistema operativo respecto a la seguridad del sistema.
- Los efectos de prácticas de seguridad del sistema sobre el rendimiento global del sistema.
- Los niveles de seguridad del sistema que pueden implementarse y las amenazas planteadas por tecnologías en evolución.
- Las diferencias entre virus, gusanos y amenazas combinadas para las computadoras.
- El rol de la educación y prácticas éticas en la seguridad del sistema.

Todo sistema de cómputo tiene necesidades contradictorias: compartir recursos y protegerlos. En los primeros días, la seguridad constaba de un candado seguro y algunas llaves. Es decir, el sistema estaba protegido físicamente y el acceso sólo se permitía a usuarios autorizados, y eso era suficiente cuando el grupo de usuarios estaba limitado a unas cuantas docenas de personas. Sin embargo, con la aparición de la comunicación de datos, redes, la proliferación de computadoras personales, software de telecomunicaciones, sitios web y el correo electrónico, la comunidad de usuarios ha crecido hasta llegar a millones de personas, lo cual hace mucho más difícil la seguridad en las computadoras.

Una parte integral tanto de la seguridad como de la administración del sistema es la aplicación oportuna de parches. El tema se analiza con detalle en el siguiente capítulo.

La seguridad del sistema constituye un tema vasto y complicado en sí. Aunque no es posible hacer justicia a un tema así en un solo capítulo, aquí presentamos los conceptos importantes para animar al lector a que revise investigaciones actuales sobre el tema. Tenga en cuenta que este tema cambia a la velocidad de la luz y que bien merece la pena que propietarios, operadores y usuarios del sistema lo monitoreen de manera constante.

Rol del sistema operativo en la seguridad

El sistema operativo desempeña un rol fundamental en la seguridad del sistema de cómputo porque tiene acceso a todas las partes del sistema. Cualquier vulnerabilidad a nivel del sistema operativo abre todo el sistema a ataques. Mientras más complicado y poderoso es el sistema operativo, es más probable que presente vulnerabilidades a ataques. Como resultado, los administradores del sistema deben estar preparados para proporcionar a sus sistemas operativos todas las defensas disponibles contra ataques y posibles fallos.

Supervivencia del sistema

La **supervivencia del sistema** se define como "la capacidad de un sistema para cumplir su misión, de manera oportuna, en presencia de ataques, fallos o accidentes" (Linger, 2002).

- El término *sistema* se refiere a cualquier sistema. Aquí se usa en el sentido más amplio posible, desde laptops hasta supercomputadoras, pasando por sistemas distribuidos.
- Una *misión* es un conjunto de muy alto nivel de requerimientos u objetivos.
- *De manera oportuna* se refiere al tiempo de respuesta del sistema, un factor crítico para la mayor parte de los sistemas.
- Los términos *ataques*, *fallos* y *accidentes* se refieren a cualquier incidente potencialmente dañino, sin importar la causa; si es intencional o no.

Antes que un sistema sea considerado capaz de sobrevivir, debe satisfacer los requisitos anteriores, especialmente con respecto a servicios que son considerados esenciales para la organización de frente a desafíos adversos. Las cuatro características clave de los sistemas capaces de sobrevivir son: resistencia, reconocimiento, daños resultantes y recuperación de servicios esenciales después de los ataques; adaptación y evolución de mecanismos de defensa del sistema para mitigar ataques futuros.

Con los elevados riesgos en años recientes de intrusión y puesta en peligro de los sistemas, los diseñadores de éstos han reconocido la necesidad crítica de supervivencia del sistema

que es incorporada en el desarrollo del sistema. Ya no es satisfactorio agregar factores de supervivencia al sistema sólo como una idea tardía. En la tabla 11.1 se muestran algunos ejemplos de estrategias que pueden integrarse a los sistemas para mejorar su capacidad de supervivencia.

(Tabla 11.1)

Cuatro propiedades clave de un sistema copaz de sobrevivir y estrategias para alcanzarlas (Linger, 2002). En la página www.cert.org se muestran detalles sobre las estrategias de ejemplo mostradas aquí.

Propiedad clave	Descripción	Estrategias ejemplo
Resistencia a ataques	Estrategias para repeler ataques	Autenticación Controles de acceso Cifrado Filtrado de mensajes Diversificación del sistema Aislamiento funcional
Reconocimiento de ataques y daños	Estrategias para detectar ataques y evaluar daños	Detección de intrusos Verificación de integridad
Recuperación de servicios esenciales y completos después del ataque	Estrategias para limitar daños, restituir información comprometida o funcionalidad, mantenimiento o restitución de servicios especiales respetando restricciones del tiempo de la misión, restitución de todos los servicios	Componentes redundantes Replicación de datos Respaldo y restitución del sistema Planificación de contingencias
Adaptación y evolución para reducir la eficacia de ataques futuros	Estrategias para mejorar la capacidad de supervivencia del sistema con base en conocimientos adquiridos a partir de intromisiones	Patrones de reconocimiento de intromisiones

Niveles de protección

Una vez que un sistema ha sido violado, ya no es posible confiar en la integridad de ningún archivo o ningún dato en el sistema. Para cada configuración de computadoras, el administrador del sistema debe evaluar el riesgo de intromisiones, lo cual, a su vez, depende del nivel de conectividad otorgado al sistema, como se muestra en la tabla 11.2.

(Tabla 11.2)

Comparación simplificada de la protección de la seguridad requerida por tres configuraciones de computadoras. Observe que las vulnerabilidades del sistema se acumulan a medida que aumenta el nivel de conectividad de la computadora.

Configuración	Facilidad de protección	Riesgo relativo	Vulnerabilidades
Computadora única (sin correo electrónico o internet)	Alta	Baja	Contraseñas comprometidas, virus
LAN conectada (sin internet)	Media	Media	Husmeadores, suplantación de identidad (+contraseña, virus)
LAN conectada (con internet)	Baja	Alta	Correo electrónico, servidores de red, FTP, Telnet (+husmeadores, suplantación de identidad, contraseña, virus)

Todas las vulnerabilidades mostradas en la tabla 11.2 se analizarán más tarde en este capítulo.

Respaldo y recuperación

Una vez que en el sitio se cuenta con suficientes políticas de respaldo y recuperación, la realización de otras técnicas relacionadas con los archivos son un procedimiento operativo normal para la mayor parte de los sistemas de cómputo. Muchos administradores de sistemas usan una planificación de respaldo por capas. Es decir, respaldan todo el sistema una vez a la semana y sólo respaldan a diario los archivos que fueron modificados ese día. Como una medida de seguridad extra, copias de respaldos del sistema completo se almacenan de tres a seis meses en un lugar seguro fuera del sitio.

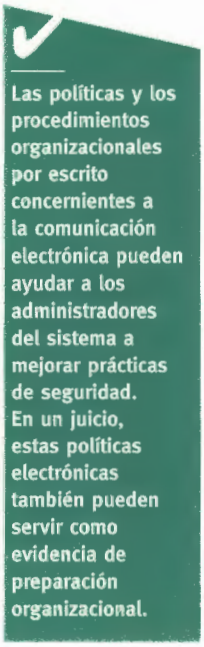
Los respaldos se vuelven esenciales cuando el sistema se hace indisponible a causa de un desastre natural o cuando un virus de computadora infecta el sistema. Si se descubre pronto, es posible ejecutar software para erradicarlo y volver a cargar archivos dañados desde las copias de respaldo. Por supuesto, cualquier cambio realizado desde que los archivos fueron respaldados debe regenerarse.

Los respaldos son un conjunto almacenado fuera del sitio, también son cruciales para recuperación de desastres. Algunas causas de desastres pueden ser las siguientes:

- agua de un incendio escaleras arriba.
- un incendio en una conexión eléctrica.
- un servidor que funciona mal.
- medios de almacenamiento corruptos.
- intromisiones de parte de usuarios no autorizados.

La importancia de los respaldos adecuados se ilustra con un ejemplo del reporte CERT 2005 acerca de un administrador de sistemas molesto por las decisiones de trabajo de la firma de defensa para la que trabajaba. Una vez que comprendió que él podía tener un rol pequeño administrando la red de cómputo que él solo desarrolló y administró, tomó la iniciativa de centralizar los programas de soporte de fabricación de la empresa en un único servidor. Entonces intimidó a un compañero de trabajo para que le diera las únicas cintas de respaldo para estos programas. Cuando el administrador del sistema terminó, una bomba lógica previamente colocada detonó y borró la única copia que quedaba del programa crítico en el servidor de la empresa. ¿El costo? La compañía estimó que el daño excedía los $10 millones, lo que llevó, a su vez, al despido de unos 80 empleados. Más historias como ésta pueden encontrarse en *www.cert.org*.

Políticas y procedimientos por escrito y entrenamiento regular del usuario son elementos esenciales de la administración del sistema. La mayor parte de las fallas en el sistema son ocasionadas por errores honestos cometidos por usuarios bien intencionados, no por intrusos maliciosos. Los procedimientos de seguridad por escrito deben recomendar cambios frecuentes de contraseñas, procedimientos de respaldo confiables, directrices para cargar nuevo software, cumplimiento con licencias de software, salvaguardas de red, directrices para monitorear la actividad de redes y reglas para acceso a terminales.



Violaciones de seguridad

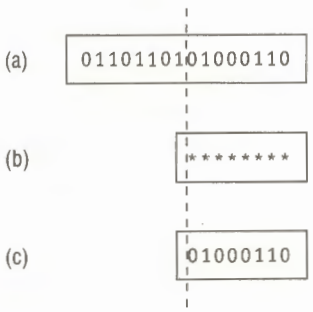
Una violación en la seguridad del sistema puede ser maliciosa o no. Por ejemplo, algunas intromisiones son el resultado de un usuario no educado y el acceso no autorizado a los recursos del sistema. Pero otras surgen del trastorno intencional de la operación del sistema, y aún otras son puramente accidentales, como mal funcionamiento del hardware, errores no detectados en el sistema operativo o en aplicaciones, o desastres naturales. Maliciosa o no, una violación de seguridad afecta gravemente la credibilidad del sistema. A continuación se presentan algunos tipos de violaciones de seguridad que pueden ocurrir.

Intromisiones no intencionales

Un ataque no intencional se define como cualquier violación de seguridad o modificación en los datos que no es resultado de una intromisión planeada.

Cuando procesos no sincronizados tienen acceso a datos y modifican algunos de los campos de un registro, este hecho se denomina "modificación accidental incompleta de los datos". Un ejemplo se proporcionó en el capítulo 5 cuando analizamos el tema de una competencia en una base de datos con dos procesos funcionando en el mismo registro de estudiante y se escribieron versiones diferentes de éste en su base de datos.

Es posible que ocurran errores cuando los valores de los datos se almacenan incorrectamente porque el campo no es suficientemente grande para dar cabida al valor numérico ahí almacenado. Por ejemplo, cuando un campo es demasiado pequeño para dar cabida a un valor numérico, FORTRAN sustituye el número con una cadena de asteriscos, y COBOL simplemente trunca los dígitos de orden superior, como se muestra en la figura 11.1.



(Figura 11.1)
El valor original de los datos se muestra: a) en un campo suficientemente grande para darle cabida. Si el campo es demasiado pequeño, FORTRAN, b) sustituye los datos con asteriscos, COBOL, c) trunca los dígitos de orden superior (valores a la izquierda de la línea punteada) y almacena sólo los dígitos que quedan.

En el momento del almacenamiento ningún error está marcado y no es posible activar un mensaje de error; los errores sólo pueden descubrirse cuando se recupera el valor. Aunque no hay necesidad de mencionarlo, ese momento es inconveniente para hacer un descubrimiento así de desagradable.

Ataques intencionales

El acceso intencional no autorizado incluye los ataques de denegación de servicio, ojeo, intervenciones de la línea, ensayos repetidos, trampas y recolección de basura. Estos ataques difieren fundamentalmente de los virus y gusanos y cuestiones semejantes (que se abordarán en breve), en que infligen un daño distribuido a numerosas compañías y organizaciones, sin especificar ninguna como objetivo.

Este ejemplo proviene del reporte CERT 2005 sobre amenazas internas. Un desarrollador de aplicaciones, quien perdió su trabajo en el sector IT como resultado de la reducción del tamaño de la empresa, expresó su descontento justo antes de las fiestas de Navidad lanzando un ataque sistemático a la red informática de su ex empleador. Tres semanas después de su despido, el empleado utilizó el nombre de usuario y contraseña de uno de sus ex compañeros para obtener acceso remoto a la red y modificar varias páginas Web de la empresa, reemplazando el texto por imágenes pornográficas. También envió a cada uno de los clientes de la empresa un mensaje por correo electrónico advirtiéndoles que la página de la compañía había sido hackeada. Cada mensaje de correo electrónico contenía los nombres de usuario y contraseña de los clientes para la página Web. Se inició una investigación, pero no se logró identificar al empleado perpetrador. Un mes y medio después, volvió a ingresar a la red de manera remota, ejecutando un script para anular todas las contraseñas de red y cambió 4000 registros de precios para reflejar información falsa. Al final este ex empleado fue identificado y procesado. Se le sentenció a servir por cinco meses en prisión y por dos años en libertad condicional, además de pagar $48 600 como reparación del daño a su ex empleador.

Acceso intencional no autorizado

Los **ataques de denegación de servicio** (**ataques DOS**) son intentos sincronizados por rehusar el servicio a usuarios autorizados al ocasionar que una computadora (que suele ser un servidor de la red) lleve a cabo una tarea (a menudo improductiva) una y otra vez, haciendo así que el sistema esté indisponible para realizar el trabajo para el que fue diseñado. Por ejemplo, si un servidor de la web está diseñado para aceptar órdenes de clientes sobre internet se desvía de su tarea acordada con comandos repetidos para identificarse a sí mismo, entonces la computadora se vuelve indisponible para servir a los clientes en línea.

El ojeo ocurre cuando usuarios no autorizados adquieren la capacidad de buscar a través del almacenamiento, directorios o archivos información que no tienen el privilegio de leer. El término "almacenamiento" se refiere a la memoria principal o al espacio sin asignar en discos y cintas. Algunas veces el ojeo ocurre después que se ha terminado el primer trabajo. Cuando una sección de la memoria principal se asigna a un proceso, los datos de un trabajo previo a menudo permanecen en la memoria –no suelen ser borrados por el sistema– por lo que están disponibles para un ojeador. Lo mismo es válido para datos en almacenamiento secundario.

La **intervención de la línea** no es algo nuevo. De la misma forma en que es posible intervenir las líneas telefónicas, también es posible hacerlo con la mayor parte de las líneas de comunicación. Cuando la intervención de la línea es pasiva, el usuario no autorizado simplemente escucha la transmisión pero no modifica el contenido. Hay dos razones por las que existe la

intervención de la línea pasiva: copiar datos mientras se evita cualquier procedimiento de autorización y para recolectar información específica (como **contraseñas**) que permiten al escucha entrar al sistema en una fecha posterior. En un entorno de red inalámbrica inseguro, no es difícil intervenir la línea.

La intervención de la línea activa es cuando se modifican los datos enviados. Dos métodos de intervención de la línea activa son "entre líneas de transmisión" y "la entrada a cuestas". Entre líneas de transmisión no altera los mensajes enviados por el usuario legítimo, sino que inserta mensajes adicionales en la línea de comunicación mientras el usuario legítimo se encuentra en una pausa. La entrada a cuestas intercepta y modifica los mensajes originales. Esto puede lograrse al romper la línea de comunicación y encaminando el mensaje a otra computadora que actúa como el anfitrión. Por ejemplo, el escucha puede interceptar un mensaje para salir del sistema, regresar el reconocimiento esperado al usuario y luego continuar la sesión interactiva con todos los privilegios del usuario original, sin que nadie se entere.

Ensayos repetidos describen el método usado para entrar a los sistemas mediante la obtención de contraseñas auténticas. Si un intruso conoce el esquema básico para crear contraseñas, como la longitud de la contraseña y los símbolos permitidos para formarla, entonces el sistema puede comprometerse con un programa que de manera sistemática recorre todas las combinaciones posibles hasta encontrar una combinación válida. Como puede imaginarse, este proceso no es largo si las contraseñas son cortas o si el intruso aprende lo suficiente acerca del presunto víctima/usuario, como se muestra en la tabla 11.3. Debido a que el intruso no necesita decodificar una contraseña específica, la conjetura de cualquier contraseña permite la entrada al sistema y tener acceso a sus recursos.

(Tabla 11.3)
Tiempo promedio requerido para que un humano y una computadora adivinen contraseñas de hasta 10 caracteres alfabéticos usando fuerza bruta (A-Z).

No. de caracteres alfabéticos	Combinaciones posibles	Tiempo promedio del intento humano para descubrir en un intento por segundo	Tiempo promedio del intento de la computadora para descubrir en un millón de intentos por segundo
1	26	13 segundos	.000013 segundo
2	$26^2 = 676$	6 minutos	.000338 segundo
3	$26^3 = 17576$	2.5 horas	.008788 segundo
8	$26^8 = 208,827,064,576$	6 640 años	58 horas
10	$26^{10} = (1.4 \times 10)^{14}$	4.5 millones de años	4.5 años

Las **trampas**, incluyendo contraseñas de puerta trasera, se definen como puntos de entrada no específicos e indocumentados al sistema. Tal vez las trampas puedan ser originadas por una falla en el diseño del sistema. Con más probabilidad, son instaladas por un diagnosticador o programador del sistema para su uso futuro. O bien, son incorporadas en el código del sistema mediante un virus destructivo o por un programa Troyano: un programa aparentemente inocuo pero que ejecuta instrucciones ocultas. Sin importar la razón de su existencia, una trampa deja vulnerable al sistema para intromisiones futuras.

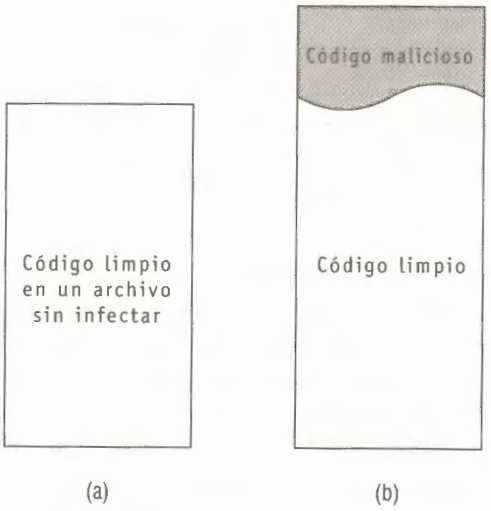
La recolección de basura, también denominada hurgar en la basura, es un pasatiempo nocturno de quienes disfrutan ojear en todos y cada uno de los objetos tirados a la basura por los usuarios del sistema: discos desechados, discos compactos, faxes, cintas de impresora, así como impresiones del programa fuente, programas, archivos pequeños de volcado de memoria y notas. Todo lo anterior contiene información importante que puede usarse para entrar ilegalmente al sistema. Se recomienda que los administradores del sistema adopten una política de triturar todo el trabajo que pudiera contener datos, códigos, contraseñas, información de acceso, información de desarrollo de sitios web o pistas para trabajos financieros de la organización. Jamás puede exagerarse esta precaución evidente.

Legalmente, es importante saber que los ataques maliciosos a las computadoras pueden violar leyes federales y estatales de Estados Unidos, lo cual amerita sanciones penales. En general, los convictos en Estados Unidos han perdido sus sistemas de cómputo y pueden haber sido condenados a pagar multas importantes, a prisión o ambas cosas.

Virus

Un **virus** se define como un pequeño programa escrito para modificar la forma de operación de una computadora, sin permiso o conocimiento del usuario. Un virus debe cumplir dos criterios:

- Debe ser autoejecutable. A menudo, esto significa colocar su propio código en la ruta de otro programa.
- Debe ser capaz de autorreplicarse. Normalmente, esto se logra al copiarse a sí mismo desde archivos infectados a archivos limpios, como se muestra en la figura 11.2. Los virus pueden infectar computadoras de escritorio y servidores de red en forma semejante y distribuirse cada que se ejecuta el archivo anfitrión.



(Figura 11.2)
Un virus que infecta archivos ataca un archivo limpio; a) al conectarse a un pequeño programa; b) que se ejecuta cada que funciona el archivo infectado.

Los virus suelen escribirse para atacar cierto sistema operativo. En consecuencia, es raro que el mismo código de virus ataque exitosamente una estación de trabajo Linux y un servidor de Windows. Los escritores de virus suelen explotar una vulnerabilidad conocida en el software del sistema operativo; de ahí la necesidad de actualizarlo correctamente con parches, algo que se analizará en el capítulo 12.

Algunos virus están diseñados para dañar significativamente la computadora infectada al borrar o corromper archivos o reformatear el disco duro. Otros no son tan maliciosos, ya que simplemente hacen saber de su presencia al entregar mensajes de texto, video o audio a la computadora del usuario. Sin embargo, ningún virus puede considerarse benigno porque todos confiscan espacio valioso de memoria y almacenamiento requerido por programas legítimos y a menudo ocasionan fallas en el sistema y pérdida de datos. Hay cinco tipos de virus identificados, como se muestra en la tabla 11.4.

(Tabla 11.4)
Cinco tipos de virus.

Tipo de virus	Descripción
Virus infector de archivos	Infecta archivos en la computadora, normalmente archivos ejecutables como .exe y .com que suelen encontrarse en sistemas operativos Microsoft. Estos virus normalmente se vuelven residentes en la memoria y luego infectan cualquier programa ejecutable limpio que se ejecute en esa computadora.
Virus en el sector de arranque	Infecta el registro de arranque, el área del sistema de un disquete o una unidad de disco duro. Estos virus se activan siempre que el usuario inicia (enciende) la computadora. La mayor parte de los virus del sector de arranque fueron escritos para MS-DOS, aunque otros sistemas operativos son blancos potenciales.
Virus en el registro de arranque maestro	Infecta el registro de arranque de un disco, guardando una copia legítima del registro de arranque maestro en una ubicación diferente en el volumen.
Virus multipartido	Infecta tanto el registro de arranque como los archivos del programa, haciendo especialmente difícil su reparación. Para eliminarlo con éxito se requiere eliminar de una vez todas las instancias del virus: en los registros de arranque y en todas las instancias de los archivos infectados con el virus. En caso de que quede alguna instancia de la infección, el virus infecta nuevamente el sistema.
Macrovirus	Infecta archivos de datos (como documentos de procesadores de palabras, hojas de cálculo, etc.), aunque versiones más recientes ahora también infectan otros archivos. Los usuarios de computadoras saben que deben inhabilitar la ejecución automática de macros sobre archivos en que no confían plenamente.

Los virus se distribuyen a través de una amplia gama de aplicaciones e incluso se han encontrado en un software empaquetado. En una ocasión, un virus fue detectado inadvertidamente en una exhibición comercial de un desarrollador que sin saberlo permitió que el virus infectara el código terminado de un paquete de software comercial completo justo antes de lanzarlo al mercado. El paquete fue retirado de inmediato.

En un ejemplo extraño, un virus fue distribuido a turistas en Medio Oriente, embebido en parte de un paquete de software copiado ilegalmente (y comprado ilegalmente). Según se dice, los vendedores manifestaron a las autoridades que ellos no enseñaban lecciones de ética a los compradores.

Un macrovirus funciona al introducirse en la plantilla (como NORMAL.DOT) que, a la vez, penetra en documentos de procesamiento de palabras. Una vez que el archivo de la plantilla está infectado, todos los documentos subsecuentes creados en esa plantilla se infectan.

Gusanos

Un **gusano** es un programa residente en la memoria que se copia a sí mismo de un sistema al siguiente sin requerir la ayuda de un archivo de programa infectado. El resultado inmediato de un gusano es tiempo de procesamiento más lento de trabajo legítimo porque el gusano actúa como un embudo para el tiempo de procesamiento y espacio en la memoria. Los gusanos son especialmente destructivos en las redes, donde ellos acaparan recursos críticos del sistema, tales como la memoria principal y tiempo de procesador.

Caballos de Troya

Un **Troyano** (originalmente llamado caballo de Troya) es un programa destructivo que está disfrazado como un programa legítimo o inofensivo que algunas veces transporta en sí mismo los medios para que el creador del programa tenga acceso secreto al sistema del usuario. Se ha sabido de intrusos que capturan las contraseñas del usuario al usar un Troyano para sustituir el programa de registro normal en la computadora con un registro falso que captura combinaciones de teclas. Una vez instalado, funciona como sigue:

1. El usuario ve un indicador de inicio de sesión y teclea la identificación del usuario.
2. El usuario ve una petición de contraseña e introduce la contraseña.
3. El programa bandido registra tanto la identificación como la contraseña del usuario y envía un mensaje de fallo de registro al usuario. Después, el programa deja de ejecutarse y regresa el control al programa legítimo.
4. En seguida, el usuario ve el indicador de inicio de sesión legítimo y vuelve a introducir la identificación del usuario.
5. El usuario ve la identificación como la contraseña legítima y vuelve a introducirla.
6. Finalmente, el usuario obtiene acceso al sistema, sin darse cuenta que el programa bandido ha almacenado el primer intento, registrado la identificación y la contraseña del usuario.

Esta serie de eventos se ve como el diálogo que se muestra en la figura 11.3. También es fácil programar una técnica semejante usando ventanas simuladas que son fáciles de crear.

Después, en el momento idóneo, el propietario del Troyano recupera el archivo con su lista de identificaciones y contraseñas del usuario, quizás una contraseña raíz, que le permitirá tener acceso a funciones de administración delicadas.

(Figura 11.3)

Dos intentos de un usuario legítimo por tener acceso al sistema. El primer intento (sombreado) muestra cómo un Troyano ha capturado la identificación y la contraseña del usuario. Después, el programa regresa al usuario un indicador de inicio de sesión del sistema real (registro:) para intentarlo de nuevo.

```
login: kat1422
password: ********
Error:login incorrect, please try again.
Login: kat1422
password: ********
Last login: Wed Nov 17 11:12:13
```

El software bandido no necesariamente tiene que ser sembrado por alguien externo a la organización. En un caso en California, un empleado sembró un Troyano en su sistema de empleado antes de despedirse del trabajo. El código fue diseñado para borrar un programa que había escrito el empleado en el trabajo para seguir la pista de la disponibilidad y el precio de partes para un programa de misiles orbitales. En su conclusión, el programa tenía instrucciones para borrarse a sí mismo. Según se dice, el programador esperaba convertirse en consultor, y planeaba trabajar para su empleador anterior reconstruyendo el programa que había escrito y borrado. El plan fracasó cuando otro técnico descubrió el código malicioso y lo eliminó antes de la fecha planificada de operación. El programador fue acusado en la corte federal por sus actos.

Bombas

Una **bomba lógica** es un programa destructivo con un fusible: cierto evento de activación (como una combinación de teclas o alguna conexión con internet). Una bomba lógica a menudo se distribuye de manera inadvertida en toda una red hasta que ocurre un evento predeterminado cuando la bomba se activa y ocasiona el daño.

Una **bomba de tiempo** es una bomba lógica, pero que es activada en un momento específico, como un día o un año. Por ejemplo, la bomba de tiempo denominada virus Miguel Ángel, descubierta por primera vez en 1991, fue una de las primeras bombas lógicas en exhibir advertencias a todos los usuarios de computadoras por los medios conocidos. El código fue diseñado para ejecutarse en el aniversario del natalicio del artista Miguel Ángel (6 de marzo de 1475). El código fue diseñado para ejecutarse cuando una computadora infectada se iniciara en la fecha de activación (el 6 de marzo de cualquier año) y sobrescribir los 17 primeros sectores en las cabezas 0-3 de las 256 primeras pistas del disco en la máquina infectada.

Amenazas combinadas

Una **amenaza combinada** inserta en un programa las características de otros ataques, incluyendo virus, gusanos, troyanos, spyware, registradores de teclas y otros códigos maliciosos. Es decir, este simple programa usa una variedad de herramientas para atacar sistemas y distribuirse a otros. Debido a que la amenaza de estos programas es tan variada, la única defensa en su contra es un extenso plan de seguridad que incluya múltiples capas de protección. Una amenaza combinada muestra las siguientes características:

- Daña el sistema afectado. Por ejemplo, podría lanzar un ataque de denegación de servicio en una dirección IP objetivo, dañar un servidor de la web, o sembrar un programa Troyano para su ejecución posterior.
- Se distribuye hacia otros sistemas usando métodos múltiples. Por ejemplo, puede buscar vulnerabilidades para comprometer un sistema, como código embebido en archivos de páginas Web en un servidor, infectar los sistemas de visitantes que visitan un sitio comprometido en la web, o enviar correos electrónicos no autorizados desde servidores comprometidos con un gusano adjunto.
- Ataca otros sistemas desde varios puntos. Por ejemplo, inyecta códigos maliciosos en los archivos .exe de un sistema, eleva el nivel de privilegio de la cuenta de un anfitrión intruso, crea acceso legible y grabable para todo el mundo a archivos privados, y agrega código script falso a archivos de páginas Web. (Este tema se analizará más tarde en este capítulo.)
- Se propaga sin intervención humana. Por ejemplo, puede explorar de manera continua internet en busca de servidores vulnerables que no tienen parches y están abiertos a ataques.
- Explota las vulnerabilidades de sistemas objetivo. Por ejemplo, podría aprovechar problemas del sistema operativo (como desbordamiento de *búfers*), vulnerabilidades de validación de páginas en la web en pantallas de entrada y contraseñas comunes predeterminadas a fin de obtener acceso no autorizado.

Cuando la amenaza incluye todas o muchas de estas características, ninguna herramienta simple es capaz de proteger un sistema. Sólo una combinación de defensas en combinación con administración de parches regular (analizada en el capítulo 12) puede esperar proteger idóneamente el sistema.

El software para la detección de registradores de teclas almacena cada combinación de teclas y la guarda para referencia posterior. Cuando el software para la detección de registradores de teclas se instala sin el conocimiento del usuario de la computadora, puede constituir una forma de intromisión al sistema. Este tópico puede profundizarse más como tema de investigación al final de este capítulo.

Protección del sistema

Las amenazas pueden provenir de externos (aquellos fuera de la organización) como de internos (empleados u otros con acceso al sistema) y pueden incluir robo de propiedad intelectual u otra información confidencial o sensible, fraude y actos de sabotaje del sistema.

La protección del sistema es polifacética. Aquí se analizan cuatro métodos de protección: instalación de software antivirus (y su ejecución regular), el uso de cortafuegos (y su manutención actualizada), asegurarse que al sistema sólo tengan acceso personas autorizadas y aprovechar la tecnología de cifrado cuando la carga, requerida para su implementación sean algo obligatorio debido al riesgo.

Software antivirus

Puede comprarse **software antivirus** para proteger sistemas de ataques de software de códigos maliciosos. El nivel de protección suele ser proporcional a la importancia de sus datos. Los datos médicos deben estar altamente protegidos. Los programas escritos por estudiantes de computación tal vez no merezcan el mismo nivel de protección.

El software para combatir virus puede ser preventivo o diagnóstico, o ambas cosas. Los programas preventivos pueden calcular una suma de control para cada programa de producción, poniendo los valores en un archivo maestro. Después, antes de la ejecución de un programa, su suma de control se compara con el maestro. En general, el software de diagnóstico compara tamaños de archivos (verificando códigos agregados cuando no se espera ninguno), busca instrucciones de replicación y busca una actividad inusual de archivos. Algún software puede buscar ciertas instrucciones específicas y monitorear la forma en que se ejecutan los programas. Pero recuerde: poco después que estos paquetes están en el mercado, los intrusos de sistemas empiezan a buscar formas de frustrarlos. Por tanto, es de esperar que sólo el software más actual puede descubrir los virus más recientes. En otras palabras, el software antiguo sólo encuentra virus antiguos.

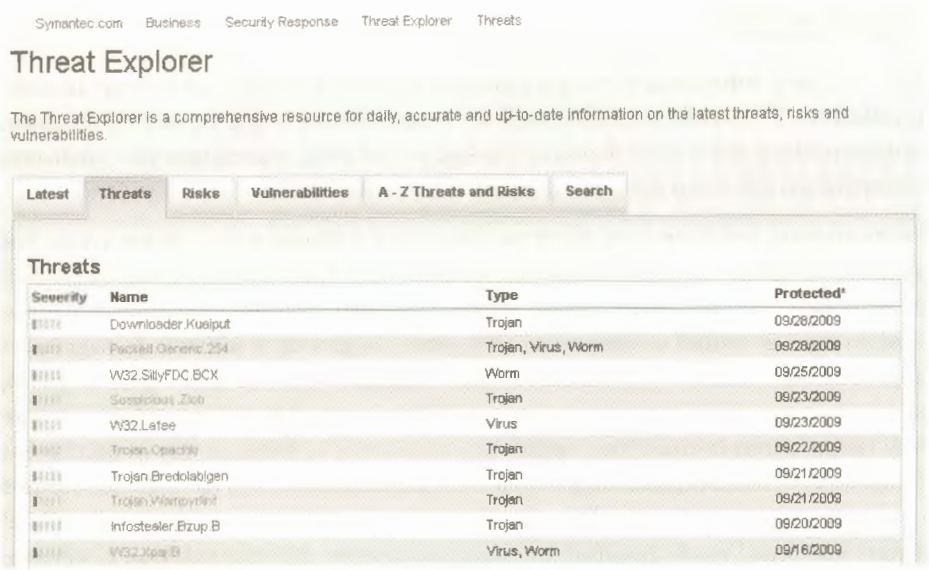
Información sobre virus actuales está disponible de parte de vendedores y agencias gubernamentales dedicadas a seguridad de sistemas, como las que se enumeran en la tabla 11.5.

(Tabla 11.5)
La seguridad de sistemas es un campo que cambia rápidamente. Información actual puede encontrarse en estos sitios de la web y en muchos más. Los sitios aparecen en orden alfabético.

Sitio web	Organización
http://csrc.nist.gov	Computer Security Division del National Institute of Standards and Technology
www.cert.org	Centro de coordinación CERT
www.mcafee.com	McAfee, Inc.
www.sans.org	SANS Institute
www.symantec.com	Symantec Corp.
www.us-cert.gov	U.S. Computer Emergency Readiness Team

Aunque el software antivirus es capaz de reparar archivos infectados con un virus, en general es incapaz de reparar gusanos, Troyanos o amenazas combinadas debido a las diferencias estructurales entre virus y gusanos o Troyanos. Un virus trabaja al infectar un archivo que de otra forma está limpio, como se muestra en la figura 11.4. En consecuencia, el software antivirus algunas veces puede remover la infección y dejar intacto el resto. Por otra parte, un gusano o un Troyano es un código malicioso en su totalidad. Es decir, todo el cuerpo del código de software contenido en un gusano o en un Troyano constituye una amenaza y debe ser eliminado como un todo.

La única forma de eliminar un Troyano es eliminando todo el cuerpo del programa malicioso. Por ejemplo, si un juego de computadora, disponible para ser descargado gratuitamente de internet, es un gusano que roba identificaciones y contraseñas del sistema, no hay ninguna forma de limpiar el juego del mal código y salvar el resto. El juego debe irse.



(Figura 11.4)

Esta pantalla del sitio Web de Symantec.com muestra las amenazas (virus, gusanos y Troyanos) hasta septiembre de 2009.

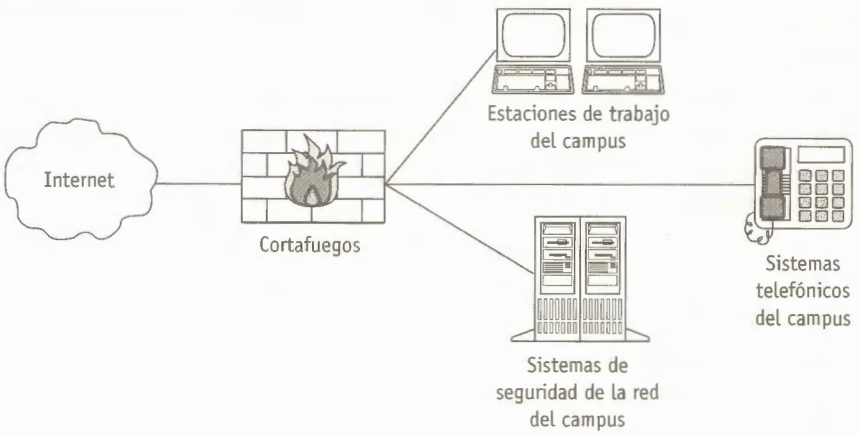
Cortafuegos

Los asaltos a redes incluyen servidores de red comprometidos, cortafuegos eludidos y sitios FTP y Telnet a los que tienen acceso usuarios no autorizados, por mencionar sólo unos cuantos.

Un **cortafuegos** es un conjunto de hardware y/o software diseñado para proteger un sistema o disfrazar sus direcciones IP de extraños que no tienen autorización para acceder al sistema o solicitan información para hacerlo. Un cortafuegos se ubica entre internet y la red, como se muestra en la figura 11.5, bloqueando consultas curiosas e intromisiones potencialmente peligrosas desde fuera del sistema. Las tareas típicas de un cortafuegos son:

- registrar las actividades que tienen acceso al sistema.
- mantener **control de acceso** con base en las direcciones IP de los emisores y los receptores.
- mantener control de acceso con base en los servicios solicitados.
- ocultar la red interna de usuarios no autorizados que solicitan información de la red.
- verificar que la protección contra virus está instalada y se ha reforzado.
- realizar autenticación con base en la fuente de la solicitud desde internet.

Los dos mecanismos fundamentales usados por un cortafuegos para llevar a cabo estas tareas son el filtrado de paquetes y los servidores proxy. Al usar **filtrado de paquetes**, el cortafuegos revisa la información del encabezado para paquetes que llegan y salen de internet a fin de verificar que la dirección de origen, la dirección de destino y el protocolo estén –todos– correctos. Por ejemplo, si un paquete llega por internet (fuera de la red) con una dirección de origen interna (que debe ser de dentro de la red) es de esperar que el cortafuegos les rehúse su entrada.



(Figura 11.5)

En este ejemplo de un sistema universitario, el cortafuegos se ubica entre las redes del campus y la internet, filtrando solicitudes para otorgarles acceso o no.

Un **servidor proxy** oculta información importante de la red a extraños al hacer invisible el servidor de la red. Para lograr lo anterior, el servidor proxy intercepta las solicitudes de acceso a la red, decide si se trata de una solicitud válida y, en caso afirmativo, pasa la solicitud al servidor idóneo que puede satisfacerla: todo sin revelar la disposición de la red, los servidores u otra información que pudiera residir en éstos. En forma semejante, si se va a pasar información desde la red hacia internet, el servidor proxy enlaza la transmisión pero sin revelar nada respecto a la red. Los servidores proxy son invisibles para los usuarios, pero son críticos para el éxito del cortafuegos.

Autenticación

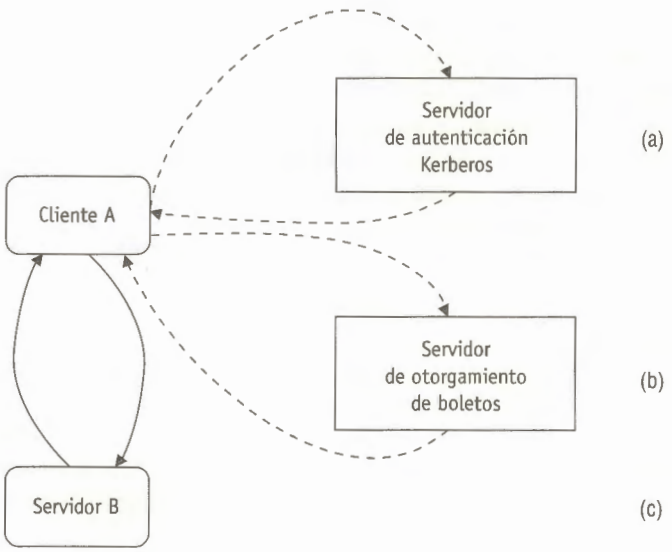
La **autenticación** es la verificación de que una persona que está intentando tener acceso tiene la autorización para hacerlo. Una herramienta de autenticación conocida es **Kerberos**, un protocolo de autenticación de red desarrollado como parte del Athena Project en el MIT. El nombre proviene del perro de tres cabezas que según la mitología griega resguardaba la entrada de Hades.

Para responder a la necesidad de cifrado de contraseñas a fin de mejorar la seguridad del sistema, Kerberos fue diseñado para proporcionar una fuerte autenticación para aplicaciones cliente/servidor. Una implementación libre de fuente abierta de este protocolo (bajo permisos de copyright) está disponible en la página web del MIT (*http://web.mit.edu/kerberos/*) o puede comprarse en muchos distribuidores.

El protocolo Kerberos usa **criptografía** (la ciencia de codificar mensajes) fuerte de modo que un cliente puede probar su identidad a un servidor, y viceversa, a través de una conexión insegura a la red. Luego, una vez que se ha completado la autenticación, tanto el cliente como el servidor pueden cifrar todas sus comunicaciones subsecuentes para asegurar privacidad e integridad de los datos.

A continuación se muestra cómo funciona Kerberos en su forma más simple, como se ve en la figura 11.6.

1. Cuando el cliente (usted) quiere tener acceso a un servidor que requiere un boleto Kerberos, usted solicita autenticación al servidor de autenticación Kerberos, que crea una clave de sesión con base en su contraseña (la de usted). Esta clave también sirve como su clave de cifrado.
2. Después, usted es enviado a un servidor de otorgamiento de boletos (que puede ser el mismo servidor físico pero una unidad lógica diferente), que crea un boleto válido para tener acceso al servidor.
3. En seguida, su boleto es enviado al servidor, donde puede ser aceptado o rechazado. Una vez aceptado, usted es libre de interactuar con el servidor durante el periodo especificado. El boleto tiene una marca de tiempo, de modo que usted puede hacer solicitudes adicionales usando el mismo boleto durante cierto periodo, aunque debe ser autenticado una vez que expira ese periodo. Esta característica de diseño es para limitar la probabilidad de que alguien use más tarde su boleto sin que usted lo sepa.



(Figura 11.6)

Al usar Kerberos, cuando el cliente A intenta tener acceso al servidor B, el usuario es autenticado (a) y recibe un boleto para la sesión (b). Una vez que se emite el boleto, el cliente y el servidor pueden comunicarse a voluntad (c). Sin el boleto no se permite el acceso.

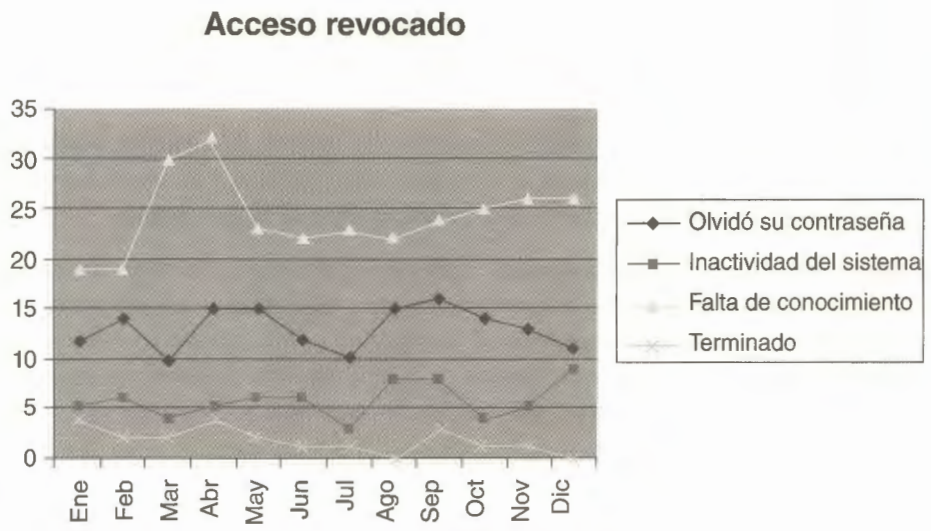
Debido a que el usuario obtiene acceso usando un boleto, no hay necesidad de una contraseña del usuario para pasar a través de la red, mejorando así la protección de contraseñas para la red.

Una parte importante del mantenimiento del protocolo Kerberos es la revocación sistemática de derechos de acceso de parte de clientes que ya no merecen tener acceso. Es por esto que los administradores del servidor de autenticación Kerberos, así como del servidor de otorgamiento de boletos deben mantener actualizadas y seguras sus bases de datos. Al mantener registros de revocación de acceso, el administrador puede indicar tendencias y anticipar la necesidad para la educación del usuario.

Por ejemplo, si la organización contrata nuevos consultores en febrero de cada año, lo cual es seguido por numerosas llamadas de revocación de acceso en marzo y abril, como se muestra en la figura 11.7, entonces planificar la educación del usuario para nuevos empleados en febrero puede ser un movimiento proactivo y rentable, reduciendo así la razón para la revocación.

(Figura 11.7)

Reporte muestra del administrador del sistema. Al seguir la pista de las razones para revocar el acceso al usuario, el administrador puede orientar sus actividades a la población de usuarios, como reentrenamiento y administración de contraseñas.



Encriptado

La protección más extrema para datos sensibles es el **encriptado** (o cifrado): escribirlos en una clave secreta. El cifrado total de la red, también denominada encriptado de comunicaciones, constituye la forma más extrema; que es cuando todas las comunicaciones con el sistema están cifradas. Después, el sistema las descifra para su procesamiento. Para comunicarse con otro sistema, los datos se cifran, transmiten, descifran y procesan.

El cifrado parcial es menos extremo y puede usarse entre puntos de entrada y salida a la red o algunas otras partes vulnerables de su sistema de comunicación. Almacenar lo cifrado significa que la información se almacena en forma cifrada y que se descifra antes de leerse o usarse.

El cifrado tiene dos desventajas: aumenta la carga extra del sistema y el sistema se vuelve totalmente dependiente del proceso de cifrado en sí; si se pierde la llave, los datos se pierden para siempre. Pero si el sistema debe mantenerse seguro, entonces es posible garantizar este procedimiento.

Cómo funciona el cifrado

La forma de entender la criptografía consiste en comprender primero el rol de una llave pública y una llave privada. Una **llave privada** es un par de números primos (que suelen ser de 75 o más dígitos cada uno) escogidos por la persona que desea recibir un mensaje privado. Los dos números primos se multiplican entre sí, formando un tercer número con 150 o más dígitos. La persona que crea esta llave privada es la única que sabe cuáles fueron los dos números primos usados para crearla. ¿Cuáles son las posibilidades de que alguien más pueda adivinar los dos números primos? Muy pocas, ya que no existe ninguna fórmula para factorizar grandes números de más de 80 dígitos, y este producto tiene por lo menos 150 dígitos. Incluso con cálculos de fuerza bruta en una supercomputadora, la tarea podría llevarse décadas (por lo menos desde que se escribió esto).

Una vez que el mensaje receptor tiene el producto, denominado **llave pública**, puede colocarse en cualquier sitio público, incluso en un directorio telefónico, para que todos puedan verlo, ya que no es posible descifrar la llave privada a partir de la llave pública.

Entonces, quien quiera enviar un mensaje confidencial al receptor usa software de encriptado e inserta la llave pública como una variable. Después, el software mezcla el mensaje antes de enviarlo al receptor. Una vez recibido, el receptor usa la llave privada en el software de encriptado y se revela el mensaje confidencial. Si alguien más debe recibir el mensaje cifrado e intenta abrirlo con una llave privada que es incorrecta, el mensaje resultante es un mensaje distorsionado e ininteligible.

Husmeadores y suplantación de identidad

Si por una red se envían datos sensibles o por internet se envían en **texto plano**, sin cifrado, se vuelven vulnerables en numerosos sitios a través de la red. Los **husmeadores de paquetes**, también denominados husmeadores, son programas que residen en computadoras conectadas a la red.

Ojean paquetes de datos a medida que pasan, examinan cada uno en busca de información específica y registran copias de paquetes interesantes para un examen más detallado. Husmear es particularmente problemático en redes inalámbricas. Cualquiera con un dispositivo inalámbrico puede detectar una red inalámbrica que esté a su alcance. Si la red está pasando paquetes de texto plano, éstos son fáciles de interceptar, leer, modificar y reenviar. La información busca desde intervalos de contraseñas, números de seguridad social y números de tarjetas de crédito hasta secretos industriales e información de mercado. Esta vulnerabilidad es una desventaja fundamental de la transmisión de texto plano por internet.

La seguridad inalámbrica es un área especial de la seguridad de un sistema, demasiado amplia para ser cubierta aquí. Alentamos a los lectores para que profundicen en este tema consultando en la literatura actual las investigaciones, herramientas y mejores prácticas más recientes.

La **suplantación de identidad** es una amenaza a la seguridad que depende de la transmisión de texto plano donde el agresor falsifica las direcciones IP de un servidor de internet al modificar las direcciones registradas en los paquetes que envía por internet. Esta técnica es de utilidad cuando usuarios no autorizados desean disfrazarse como sitios amigables. Por ejemplo, para guardar información confidencial en una red interna (intranet), algunos servidores de la red permiten tener acceso sólo a usuarios de ciertos sitios; sin embargo, al suplantar la dirección IP, el servidor puede admitir de manera inadvertida a usuarios no autorizados.

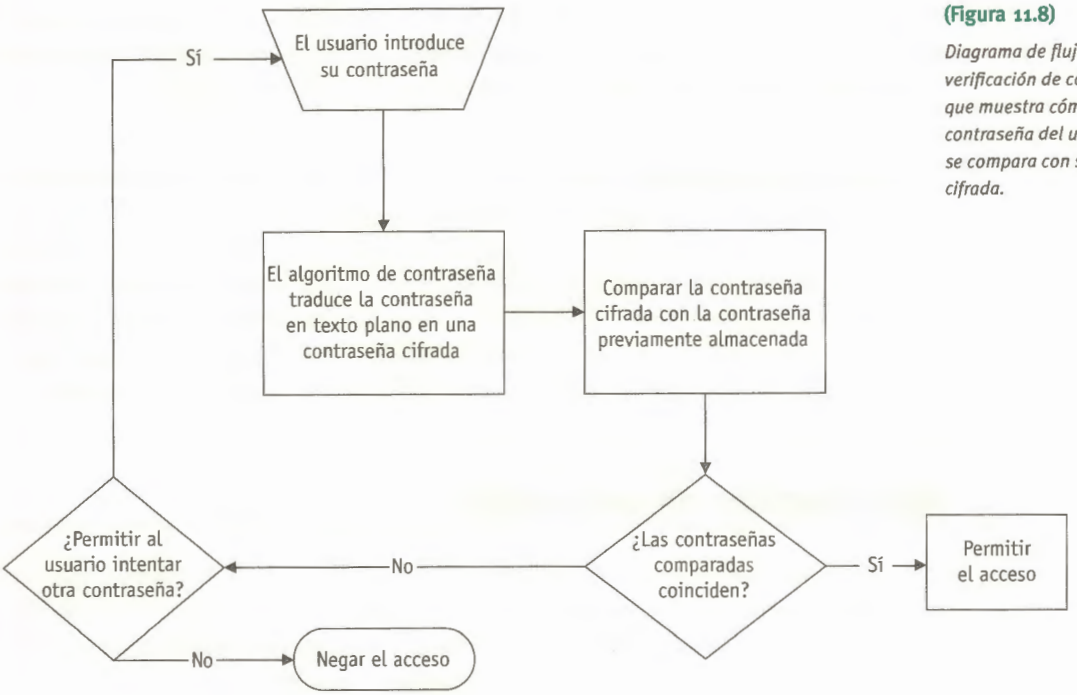
Administración de contraseñas

Las técnicas más fundamentales usadas para proteger inversiones de hardware y software son buenas contraseñas y entrenamiento cuidadoso del usuario, aunque esto no es tan simple como parece. Las contraseñas pueden olvidarse, a menudo no es posible cambiarlas, se comparten en común y muchas personas las consideran engorrosas.

Construcción de contraseñas

Las contraseñas constituyen uno de los esquemas de protección más sencillos y efectivos de implementar, aunque sólo si se usan correctamente. Una buena contraseña es inusual, memorable y se cambia a menudo, normalmente entre cada 30 y 90 días. Idealmente, la contraseña debe ser una combinación de caracteres y números, algo fácil de recordar para el usuario, pero difícil para que alguien más la adivine. La contraseña debe aprenderse de memoria, jamás escribirse y tampoco incluirse en un archivo script para registrarse en la red.

Los archivos de contraseñas suelen almacenarse en forma cifrada, de modo que no sean legibles para ojeadores casuales. Para verificar una contraseña, el sistema acepta la entrada del usuario en texto plano y compara la nueva secuencia con la versión cifrada almacenada en el archivo de contraseña para ese usuario, como se muestra en la figura 11.8.



(Figura 11.8)

Diagrama de flujo de verificación de contraseña que muestra cómo la contraseña del usuario se compara con su forma cifrada.

Hay varias técnicas confiables para generar una buena contraseña:

- Usar un mínimo de ocho caracteres, incluyendo números y caracteres no alfanuméricos.
- Crear una palabra impronunciable o unir bits en una palabra fácil de recordar.
- Seguir cierto patrón en el teclado, generando nuevas contraseñas fácilmente al empezar la secuencia con una letra diferente cada vez.
- Crear acrónimos de frases memorables, como MPC4AEA, que significa "mi perro cumplirá cuatro años en abril".
- Si el sistema operativo distingue entre mayúsculas y minúsculas (como lo hacen UNIX y Linux), los usuarios deben aprovechar este hecho al usar ambos tipos de letra en la contraseña: MPc4aeA.
- Evitar cualquier palabra que aparezca en un diccionario.

La longitud de la contraseña tiene un efecto directo sobre la capacidad de que la contraseña sobreviva intentos por romperla. Mientras más largas sean las contraseñas, mejor. Por ejemplo, si el administrador del sistema ordena que todas las contraseñas deben tener exactamente ocho caracteres y contener cualquier letra minúscula como "abcdefgh", el número de las combinaciones posibles es 26^8. En forma semejante, si la política determina 10 caracteres de letras minúsculas, el total de combinaciones es 26^{10}. No obstante, si la regla permite ocho caracteres de entre 95 que es posible imprimir, las combinaciones aumentan a 95^8, como se muestra en la tabla 11.6.

(Tabla 11.6)

Número de combinaciones de contraseñas, dependiendo de su longitud y del conjunto de caracteres disponible.

	Longitud de la contraseña en caracteres					
	2	4	6	8	10	12
Caracteres ASCII	128^2	128^4	128^6	128^8	128^{10}	128^{12}
Caracteres imprimibles	95^2	95^4	95^6	95^8	95^{10}	95^{12}
Caracteres alfanuméricos	62^2	62^4	62^6	62^8	62^{10}	62^{12}
Letras minúsculas y números	36^2	36^4	36^6	36^8	36^{10}	36^{12}
Letras minúsculas	26^2	26^4	26^6	26^8	26^{10}	26^{12}

Ataque de diccionario es el término usado para describir un método para descifrar contraseñas encriptadas. Sus requerimientos son simples: una copia del archivo de la contraseña encriptada y el algoritmo usado para cifrar las contraseñas. Con estas dos herramientas, el intruso ejecuta un programa de software que toma todas las palabras en el diccionario, las ejecuta a través del algoritmo de cifrado de la contraseña y compara el resultado cifrado con las contraseñas cifradas contenidas en el archivo. Si ambas versiones cifradas coinciden, entonces el intruso sabe que esta palabra del diccionario ha sido usada como una contraseña legítima.

Una técnica usada por algunos sistemas operativos para hacer más difíciles de adivinar las contraseñas es "salar" contraseñas de usuario con bits aleatorios extra a fin de hacerlas menos vulnerables a ataques de diccionario. He aquí cómo funciona. El usuario introduce la contraseña deseada, que después se codifica. Después, el sistema asigna al usuario una combinación única de bits (denominada la sal) que se integran al final de la contraseña codificada. Esto significa que la combinación de ceros y unos almacenada contiene la contraseña codificada combinada con una sal única. Entonces, si un intruso descarga la lista de contraseñas codificadas, el intruso no sólo requiere conjeturar la contraseña, sino también la sal aleatoria. Esto es algo mucho más difícil de lograr y consume más tiempo que simplemente conjeturar contraseñas obvias.

Alternativas de contraseñas

Como una alternativa a las contraseñas, algunos sistemas usan una **tarjeta inteligente**: una calculadora del tamaño de una tarjeta de crédito que requiere "algo que usted tiene y algo que usted sabe". La tarjeta inteligente exhibe un número con varios dígitos que cambia constantemente y está sincronizado con un generador de un número idéntico en el sistema. Para introducir la contraseña correcta, el usuario debe teclear el número que aparece en ese momento en la tarjeta inteligente. Para protección adicional, después el usuario introduce un código secreto. El usuario es admitido en el sistema sólo si se validan tanto el número como el código. Usando este esquema, un intruso necesita más que sólo la tarjeta o sólo el código, ya que a menos que ambos sean válidos, se niega la entrada.

Otra alternativa es la **biometría**, ciencia y tecnología de identificación de personas con base en las características biológicas únicas de cada persona. La investigación actual está centrada en el análisis del rostro humano, huellas digitales, medidas de las manos, iris/retina e impresión de voz. Los dispositivos biométricos a menudo constan de un escáner o algún otro dispositivo para reunir los datos necesarios sobre el usuario, software para convertir los datos a alguna forma que sea posible comparar y almacenar, y una base de datos para mantener la información almacenada de todos los usuarios autorizados. Una de las fortalezas de la biometría es que identifica positivamente a la persona escaneada.

Por ejemplo, una huella digital cuenta con alrededor de 35 discriminadores: factores que diferencian una huella digital de otra. Aún mejor, el iris cuenta con alrededor de 260 discriminadores y la de cada ojo es única. Ni siquiera dos gemelos idénticos tienen el mismo patrón. Un factor crítico con la biometría es reducir el margen de error de modo que usuarios autorizados rara vez son rechazados y quienes no están autorizados son atrapados en la puerta. Al momento de escribir esto, la tecnología para implementar autenticación biométrica es costosa, pero hay indicaciones que en los años por venir su uso se distribuirá ampliamente.

Otra alternativa a las contraseñas implica el uso de gráficos y patrones de clics usando un ratón, un estilete, una pantalla sensible al tacto o algún otro dispositivo apuntador, como se muestra en la figura 11.9. Con esta tecnología, el usuario establece cierta secuencia de clics sobre una fotografía o ilustración y luego la repite para tener acceso. Debido a que este sistema elimina todas las entradas desde el teclado, es resistente a ataques de diccionario (que implican probar todas las palabras en un diccionario para intentar descifrar la contraseña objetivo). Uno de los temas de investigación al final de este capítulo permite explorar los últimos desarrollos en este tema en evolución.

(Figura 11.9)

Contraseña gráfica creada al hacer clic en secuencia en ciertas áreas de la fotografía.

Ingeniería social

Históricamente, los intrusos obtenían acceso a los sistemas al usar técnicas innovadoras para descifrar contraseñas de usuarios como el uso de contraseñas predeterminadas y genéricas, o intentar palabras encontradas en un archivo de términos de diccionario. También usan una técnica denominada **ingeniería social**, que significa buscar en y alrededor del escritorio del usuario algún recordatorio escrito, intentando la identificación de registro del usuario como contraseña, e incluso telefoneando a amigos y colaboradores para conocer los nombres de los miembros y las mascotas de la familia del usuario, los destinos de vacaciones, los pasatiempos preferidos, el modelo de automóvil, etcétera.

Esto funciona incluso con objetivos militares. Uno de tales casos implicaba un sistema de cómputo en una ubicación remota que se reportaba como excepcionalmente segura. Para probar su seguridad, se contrató una consultora de sistemas para tratar de entrar al sistema. Ella empezó por abrir un documento no protegido que indicaba dónde se ubicaba la computadora. Después, llamó por teléfono a la base y se aprendió el nombre del oficial comandante de la base. En seguida se aprendió el nombre de la secretaria del comandante. A continuación, llamó al centro de datos, enmascarada como secretaria, para informar que el comandante tenía dificultades para acceder a los datos. Cuando el personal del centro de datos estaba renuente a ayudarla, la consultante montó en cólera y exigió acción. Le respondieron otorgándole acceso, de modo que el "comandante" podía usar la red. Pronto, la consultora estaba en el sistema con lo que aparentemente eran datos clasificados. Con eso, los oficiales a cargo del ejercicio lo suspendieron y la consultora fue enviada a casa.

La **suplantación de identidad** es una forma de ingeniería social en la cual un intruso pretende ser una entidad legítima y entra en contacto con usuarios despistados pidiéndoles que reconfirmen su información personal y/o financiera. En 2003, muchos clientes de eBay, Inc., recibieron un correo electrónico que decía que su cuenta estaba comprometida y era necesario cerrarla. Adjunta al correo iba una liga a una página que parecía una página web genuina de eBay donde las personas podían reintroducir los datos de sus tarjetas de crédito, sus números de identificación personal para cajeros automáticos, su número de seguridad social, fecha de nacimiento y el nombre de soltera de su madre. Así, el propietario del sitio contaba con toda la información necesaria para robar la identidad de quienes respondieron.

Las contraseñas predeterminadas presentan vulnerabilidades únicas porque son ampliamente conocidas entre los atacantes de sistemas, aunque son una herramienta necesaria para los vendedores. Las contraseñas predeterminadas suelen embarcarse de manera rutinaria con hardware o software. Son convenientes para los fabricantes porque proporcionan a los trabajadores de servicio de campo acceso a nivel de supervisión para reparar equipo con problemas *in situ* o usando una conexión de telecomunicaciones. Los intrusos del sistema también las encuentran útiles porque si no se han modificado, proporcionan acceso poderoso al sistema. Listas de las contraseñas predeterminadas suelen pasarse rutinariamente de un pirata cibernético a otro, sirviendo a menudo como punto de partida para un ataque. Para proteger el sistema, los administradores deben identificar y cambiar de manera periódica todas las contraseñas predeterminadas en su hardware y software.

Ética

¿Qué es un comportamiento ético? En su forma más breve es esto: Ser bueno. Hacer el bien.

Muchas asociaciones profesionales han abordado el tema de la ética: las reglas o normas de comportamiento que se espera sigan los miembros de la comunidad de usuarios de computadoras, demostrando los principios del bien y el mal. EN 1992, la IEEE y la Association for Computing Machinery (ACM) emitieron una norma de ética para la comunidad informática global.

La aparente falta de ética en el campo de la informática constituye una desviación importante con respecto a otras profesiones. Por ejemplo, damos por hecho que nuestro médico mantendrá la privacidad de nuestros expedientes, pero no todos tenemos la misma confianza en quienes trabajan para compañías que poseen historiales de créditos o en los intrusos que irrumpen en estos sistemas. Aunque la ética constituye un tema que no se aborda a menudo en cursos de informática, las implicaciones son tan vastas que no pueden ser ignoradas por los administradores o los usuarios de sistemas.

En la mesa de discusión están las necesidades aparentemente contradictorias de los usuarios: la necesidad individual de privacidad, la necesidad de la organización de proteger su información registrada, y el derecho público a conocer, como se ilustra en la libertad de las leyes de información.

Para el propietario del sistema, deslices éticos de parte de usuarios autorizados o no autorizados pueden tener graves consecuencias:

- Software copiado ilegalmente puede resultar en demandas y multas equivalentes a varias veces el precio al menudeo de cada producto por cada transgresión. Varias asociaciones industriales publican números gratuitos dedicados a alentar a empleados descontentos para entregar a la policía a sus empleadores que usen software ilegal.
- El plagio, la copia no autorizada de trabajo registrado (incluyendo, pero sin estar limitado a música, películas, material de textos, bases de datos, etc.) es ilegal y punible por ley en muchas naciones. Cuando el trabajo original esté en papel, la mayoría de los usuarios conocen el curso de acción idóneo, pero cuando el original está en forma electrónica, algunas personas no reconocen las cuestiones éticas implicadas.
- Escuchar conversaciones ajenas en correo electrónico, datos o comunicaciones de voz algunas veces es ilegal y suele no estar garantizada, excepto en ciertas circunstancias. Si es necesario monitorear llamadas o mensajes, los participantes siempre deben ser notificados antes de iniciar el monitoreo.
- Quebrar la seguridad del sistema, algunas veces denominado *hacking*, consiste en obtener acceso a otro sistema de cómputo para monitorear o modificar datos, lo que rara vez constituye una actividad ética. Aunque algunas personas lo consideran un deporte, cada intromisión debe hacer que el propietario y los usuarios del sistema cuestionen la validez de los datos del sistema.
- El uso no ético de la tecnología, definido como acceso no autorizado a sistemas de cómputo privados o protegidos o a información electrónica, es un área oscura de la ley, pero es claramente lo que no hay que hacer. Legalmente, el sistema de justicia tiene bas-

tantes dificultades para mantener cada forma específica de acceso no autorizado porque la tecnología cambia muy rápido. En consecuencia, los propietarios del sistema no pueden depender de la ley para orientación. En lugar de ello, deben enseñar agresivamente a sus usuarios sobre lo que es o no un comportamiento ético.

¿Cómo puede enseñarse a los usuarios a comportarse éticamente? Una serie continua de advertencias de seguridad y comunicaciones de ética para los usuarios de computadoras es más eficaz que un solo anuncio. Algunas actividades específicas podrían incluir lo siguiente:

- Publicar políticas que establezcan con claridad cuáles acciones serán justificadas y cuáles no.
- Establecer un seminario regular sobre el tema, que incluya casos de la vida real.
- Conducir discusiones abiertas sobre cuestiones de ética como: ¿es correcto leer el correo electrónico de alguien? ¿Es correcto que alguien lea el correo electrónico de otra persona? ¿Es ético que un competidor lea datos de otra persona? ¿es correcto que alguien explore la cuenta bancaria de otra persona? ¿Es correcto que alguien cambie el resultado de un examen médico? ¿Es aceptable que alguien copie el programa de software de otra persona y lo suba a internet? ¿Es aceptable que alguien copie un documento del gobierno y lo suba a internet?

Para una guía de comportamiento ético, consulte extractos del Código de ética y conducta profesional ACM en el apéndice de este libro o visite la página *www.acm.org*.

Conclusión

El sistema es sólo tan bueno como la integridad de los datos ahí almacenados. Una sola violación de seguridad –ya sea catastrófica o no, accidental o no- daña la integridad del sistema. Y la integridad dañada amenaza la viabilidad de los sistemas mejor diseñados, de sus administradores, diseñadores y usuarios. En consecuencia, son esenciales precauciones de seguridad vigilante.

Hasta el momento en este texto hemos analizado la función de cada administrador y cada sistema operativo de manera aislada, pero en realidad el rendimiento del sistema depende del esfuerzo combinado de cada parte. En el siguiente capítulo abordaremos el sistema como un todo y analizaremos cómo cada parte contribuye al rendimiento global del sistema o lo merma.

Términos clave

Amenaza combinada: Amenaza al sistema que combina en un programa las características de otros ataques, incluyendo un virus, gusano, Troyano, Spyware y otros códigos maliciosos.

Ataque de denegación de servicio (DoS): Ataque a una red que imposibilita la ejecución de funciones para las que estaba diseñada. Esto puede hacerse al inundar al servidor con solicitudes o información sin sentido.

Ataque de diccionario: Técnica mediante la cual un intruso trata de adivinar contraseñas de usuarios al intentar palabras encontradas en un diccionario.

Autenticación: Medios por los cuales un sistema verifica que el intento individual por tener acceso al sistema está autorizado para hacerlo.

Biometría: Ciencia y tecnología de identificar usuarios no autorizados con base en sus características biológicas.

Bomba de tiempo: Virus con un activador vinculado con una fecha dada: año, mes, día o momento que ocasiona su ejecución.

Bomba lógica: Virus con un activador, normalmente un evento, que ocasiona su ejecución.

Contraseña: Método de control definido por el usuario. Normalmente una palabra o cadena de caracteres que un usuario debe especificar para que se le permita registrarse en un sistema de cómputo.

Control de acceso: El control del acceso del usuario a una red o a un sistema de cómputo.

Cortafuegos: Conjunto de hardware y software que disfraza la dirección de la red interna de una computadora o red para controlar la forma en que los clientes desde fuera pueden tener acceso a los servidores de la red interna.

Criptografía: Ciencia de codificar mensajes o texto de modo que usuarios no autorizados no puedan leerlos.

Encriptado: Traducción de un mensaje o datos de su forma original a una forma codificada, escondiendo así su significado y haciéndolo ininteligible sin la llave para descifrarlo. Se usa para mejorar la seguridad del sistema y para protección de datos.

Ética: Reglas o normas de comportamiento que se espera sigan las personas, a fin de demostrar los principios del bien y el mal.

Filtrado de paquetes: Revisión de paquetes que llegan y salen por internet para verificar que la dirección de origen, la dirección de destino y el protocolo sean correctos. Suele ser la función de un cortafuegos.

Gusano: Programa de computadora que se replica a sí mismo y se autopropaga en la memoria principal.

Husmeador de paquetes: Software que intercepta paquetes de datos por internet en texto plano y busca en ellos información como contraseñas.

Ingeniería social: Técnica con la que los intrusos del sistema obtienen acceso a información sobre un usuario legítimo para conocer contraseñas activas, algunas veces mediante llamadas al usuario y presentándose como un técnico del sistema.

Intervención de la línea: Violación a la seguridad del sistema donde usuarios no autorizados monitorean o modifican la transmisión de un usuario.

Kerberos: Sistema de autenticación desarrollado en el MIT que permite a los administradores de la red y administrar la autenticación del usuario a nivel de red.

Llave privada: Herramienta usada para descifrar un mensaje que fue cifrado usando una llave pública.

Llave pública: Herramienta usada para codificar un mensaje, que será descifrado más tarde usando una llave privada.

Ojear: Violación del sistema de seguridad donde a usuarios no autorizados se les permite buscar en directorios o archivos en almacenamiento secundario información que no tienen el privilegio de leer.

Recuperación: Pasos que deben emprenderse cuando un sistema es asaltado, para recuperar la operabilidad del sistema y, en el mejor de los casos, recuperar cualquier dato perdido.

Respaldo: Proceso de hacer copias para almacenamiento de archivos a largo plazo en el sistema.

Servidor proxy: Servidor posicionado entre una red interna y una red externa o en internet para mostrar todas las solicitudes de información e impedir acceso no autorizado a recursos de la red.

Software antivirus: Software diseñado para detectar y recuperarse de ataques de virus y gusanos. Suele ser parte de un paquete de software de protección del sistema.

Spyware: Amenaza combinada que recolecta datos de manera encubierta sobre usuarios del sistema y la envía a un depositario designado.

Supervivencia del sistema: Capacidad de un sistema para lograr su misión, de manera oportuna, en presencia de ataques, fallas o accidentes.

Suplantación de identidad: Técnica usada para engañar a los consumidores para que revelen información personal, al presentarse como una entidad legítima.

Tarjeta inteligente: Dispositivo del tamaño de una tarjeta de crédito que usa tecnología de encriptado para controlar el acceso a computadoras y redes de computadoras. Cada tarjeta inteligente tiene su propio identificador personal, que sólo conoce el usuario, así como su propia contraseña almacenada y codificada.

Texto plano: En criptografía, un método de transmisión de datos sin encriptado, en texto que es legible para quien lo lea.

Trampa: Punto de entrada no especificado y sin documentar al sistema, lo cual representa un riesgo importante para la seguridad.

Troyano: Programa de computadora malicioso con efectos colaterales no intencionales de parte del usuario que ejecuta el programa.

Virus: Programa que se replica a sí mismo en otros programas, incluyendo los que están en almacenamiento secundario, que se comparten entre otros sistemas de cómputo.

Búsquedas de interés

- Herramientas de seguridad para redes
- Contraseña de seguridad fuerte
- Prácticas de encriptamiento
- Bloqueo de amenazas internas
- Asuntos legales contra asuntos éticos

Ejercicios

Temas de investigación

A. Investigue la práctica del registrador de teclas en literatura actualizada. Describa qué es y explique por qué algunas personas defienden esta práctica mientras que otras no. Finalmente, escriba su propia opinión y defiéndala. Cite sus fuentes.

B. En literatura actualizada investigue el uso de gráficas para sustituir contraseñas tecleadas. Describa los hitos históricos de la tecnología y las ventajas y desventajas de la tecnología. Por último, encuentre por lo menos dos aplicaciones de la vida real que usen esta tecnología, citando sus fuentes para cada una.

Ejercicios

1. Proporcione tres ejemplos de contraseñas excelentes y explique por qué cada una sería una buena opción para proteger un sistema de usuarios no autorizados.
2. Proporcione tres ventajas y desventajas de un programa generador de contraseñas. ¿Recomendaría el uso de tal programa en su propio sistema? Explique por qué sí o por qué no.
3. Los administradores del sistema no pueden proteger sus recursos sin reconocer todas las amenazas e incluso aprendiendo a pensar como un intruso. Sabiendo lo anterior y si se sabe que no es ético usar sistemas de cómputo sin autorización idónea, imagine que usted es un usuario no autorizado que quiere irrumpir en su sistema. Describa cómo empezaría a conjeturar la contraseña de un usuario legítimo.
4. Como continuación de la pregunta previa, identifique a un amigo que haya escogido por lo menos una contraseña de computadora. En una hoja de papel, escriba por lo menos 20 contraseñas posibles que usted pudiera usar si estuviese intentando tener acceso al sistema de un amigo. Luego muestre la lista a su amigo y pregúntele si alguna de las contraseñas es correcta. Puede intentar combinaciones de nombres de miembros y amigos de la familia, pasatiempos favoritos, automóviles, mascotas, cumpleaños, lenguaje coloquial usado, dichos o expresiones favoritas, etcétera.

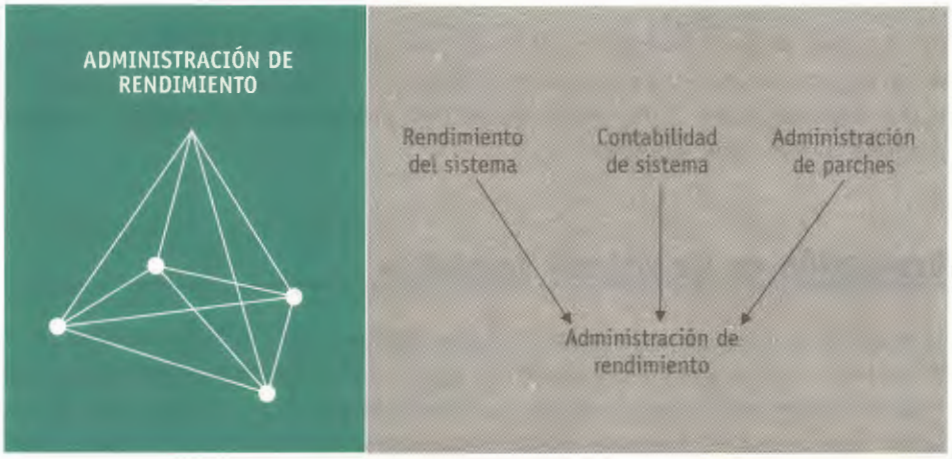
5. Imagine que usted es el gerente de un pequeño negocio de computación. Enliste al menos tres razones que usaría para convencer a un ocupado y renuente miembro del equipo, para ejecutar respaldos regulares y elaborar y administrar sus archivos de sistema apropiadamente.
6. En Estados Unidos, la legislación derivada del Acta de Seguridad, Portabilidad y Contabilidad para la Salud de 1996 (HIPAA, por sus siglas en inglés) protege la privacidad de los datos de los pacientes médicos. Aun en una emergencia, es por su bien que los profesionales que atienden el cuidado a la salud tengan acceso a su expediente, historia médica, etc. Al reconocer la necesidad de privacidad y de otorgar acceso, describa algunas ventajas y desventajas de mantener estrictamente protegido el expediente médico de los pacientes.
7. A muchos usuarios se les pide registrarse en varias redes y así contar con varias contraseñas, que son difíciles de recordar. Mencione tres maneras en las que un usuario puede administrar estos requerimientos de contraseñas y compare las ventajas y las desventajas de cada una. Finalmente, decida cuál usaría usted y explique por qué.

Ejercicios avanzados

8. Describa las únicas amenazas para un centro de datos colocada por empleados descontentos. Describa cómo identificaría usted a esta gente si es posible, y cómo protegería usted su sistema de estas amenazas.
9. Enumere 20 virus descubiertos en los últimos 12 meses e investigue tres en detalle, describiendo los archivos que infectan, la forma en que se distribuyen y sus efectos pretendidos.
10. Identifique tres conjuntos de parámetros de seguridad (para protección aceptable, mejor y óptima) para una computadora que contiene información de registros universitarios. Considere no sólo el sistema operativo, sino el software de protección, controles de acceso y el espacio en que está la computadora. Después, haga una recomendación con base en la necesidad de seguridad contra el costo de esa seguridad.
11. Use información de CERT *Coordination Center* (*www.cert.org*) para identificar la vulnerabilidad más reciente para un sistema operativo de su elección. Escriba la amenaza, la gravedad de la amenaza, el impacto potencial, la solución sugerida, los sistemas afectados y las acciones que llevaría a cabo como administrador del sistema.
12. La HIPAA (Health Insurance Portability and Accountability Act de 1996) impuso requisitos astringentes de seguridad a la industria del cuidado de la salud en Estados Unidos. Identifique los requerimientos para transmisión de datos entre redes y el rol de la codificación, en caso de haber alguno, en estas transmisiones.
13. Las redes de área local inalámbricas plantean desafíos únicos para los operadores del sistema debido a su accesibilidad. Imagine que usted es el administrador de sistemas de una red inalámbrica que se usa en una instalación de investigación científica. Identifique los cinco desafíos más importantes a la seguridad y cómo abordaría cada uno a pesar de su presupuesto limitado.

14. Con el robo de identidad cada vez más extendido, muchas organizaciones han decidido asegurar los números de seguridad social de sus clientes, abastecedores y empleadores. Imagine que usted es el administrador del sistema de un campus universitario y a los registros de los estudiantes se tuviera acceso usando el número de identificación del estudiante, que es idéntico al número de seguridad social (haciendo que usted necesite extender su protección a varias décadas de registros históricos de los estudiantes). Describa los pasos que seguiría para cambiar el sistema. Asegúrese de que su solución también elimine el número de seguridad social de los estudiantes en las transcripciones, formas de registro a los cursos, pantallas de datos accesibles a los estudiantes, tarjetas de identificación, registros de centros de salud y otros sistemas de conservación de registros. Finalmente, identifique qué personas en el campus pueden retener el acceso a los números de seguridad social y explique por qué.

Capítulo 12 Administración del sistema



"No existe ningún problema aislado; . . . todos están interrelacionados."

Saul D. Alinsky (*1909-1972*)

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Las desventajas a considerar cuando se intenta mejorar el rendimiento global del sistema.
- Los roles de las herramientas de medición del sistema, como ciclos de realimentación positiva y negativa.
- Dos técnicas de monitoreo del sistema.
- Los fundamentos de la administración de parches.
- La importancia de las prácticas de contabilidad profundas por parte de administradores del sistema.

En el capítulo 1 presentamos el sistema operativo global. En los capítulos 2 al 8 estudiamos cómo funciona cada uno de sus componentes, y de los capítulos 9 a 11 analizamos las redes y su seguridad. En estos capítulos analizamos cada parte por separado. En un sistema operativo real, no obstante, no trabajan de manera aislada. Cada componente depende de los otros componentes.

Este capítulo muestra cómo funcionan juntas y la forma en que el diseñador del sistema debe considerar las desventajas para mejorar la eficiencia global del sistema. Empezamos al mostrar la manera en que el diseñador puede mejorar el rendimiento de un componente, el costo de esa mejora y cómo podría afectar el resto del sistema. Concluimos con algunos métodos usados para monitorear el rendimiento del sistema, incluyendo la necesidad de mantener parchado correctamente el sistema.

Evaluación de un sistema operativo

La mayor parte de los sistemas operativos fueron diseñados para trabajar con cierta pieza de hardware, una categoría de procesadores o un grupo específico de usuarios. Aunque casi todo evolucionó con el tiempo para operar sistemas múltiples, la mayor parte sigue favoreciendo algunos usuarios y algunos entornos de computación sobre otros. Por ejemplo, si el sistema operativo fue escrito para que usuarios novatos cumplieran requerimientos básicos, podría no satisfacer las demandas de los más conocedores. Al revés, si fue escrito por programadores, entonces un operador de una computadora de oficina puede encontrar oscuros sus comandos. Si atienden las necesidades de un centro de cómputo multiusuario, podría ser inapropiado para un centro de cómputo pequeño. O bien, si fue escrito para proporcionar tiempo de respuesta rápido, podría proporcionar un resultado deficiente.

Para evaluar un sistema operativo, es necesario comprender sus objetivos de diseño, su historia, cómo se comunica con los usuarios, la forma en que se administran sus recursos y qué sacrificios se hicieron para alcanzar sus objetivos. En otras palabras, las fortalezas y las debilidades de un sistema operativo deben ponderarse en relación con quién lo usará, sobre qué hardware y para qué propósito.

Cooperación entre componentes

La utilidad de cualquiera de los recursos depende del rendimiento de los otros recursos en el sistema. Por ejemplo, la administración de la memoria está vinculada intrínsecamente con la administración de dispositivos cuando la memoria se usa para almacenar datos entre un procesador muy rápido y dispositivos de almacenamiento secundario más lentos. A lo largo de los capítulos precedentes se han presentado muchos otros ejemplos de interdependencia.

Si usted administrara el sistema de cómputo de una organización y contara con dinero para actualizarlo, ¿cómo invertiría el dinero de la mejor forma? Quizá tuviera varias opciones:

- Más memoria.
- Una CPU más rápida.
- Procesadores adicionales.
- Más unidades de disco.
- Un sistema RAID.
- Nuevo software para administración de archivos.

O bien, si compra un nuevo sistema, ¿qué características buscaría para que fuera más eficiente que el anterior?

Cualquier mejora del sistema puede hacerse sólo al cabo de un análisis detallado de las necesidades de los recursos, requerimientos, administradores y usuarios del sistema. Pero cada que se hacen cambios a uno, a menudo se sacrifica un conjunto de problemas por otro. La clave consiste en el rendimiento de todo el sistema y no sólo en el de las componentes individuales.

Rol de la administración de la memoria

En los capítulos 2 y 3 se analizaron esquemas de administración de la memoria. Si se aumenta la memoria o se cambia a otro esquema de asignación, es necesario considerar el entorno operativo actual en que residirá el sistema. Hay una negociación entre uso de la memoria y la sobrecarga de la CPU.

Por ejemplo, si el sistema sólo ejecutará programas de estudiantes y la tarea media es de sólo tres páginas de longitud, la decisión de incrementar el tamaño de la memoria virtual podría no acelerar el resultado porque la mayor parte de los trabajos son demasiado pequeños para usar de manera efectiva la memoria virtual. Recuerde que a medida que los algoritmos de administración de la memoria son más complicados, la sobrecarga de la CPU aumentan y el rendimiento global puede sufrir. Por otra parte, algunos sistemas operativos se desempeñan de manera extraordinariamente mejor con memoria adicional.

Rol de la administración del procesador

La administración del procesador se cubrió en los capítulos 4, 5 y 6. Suponga que usted ha decidido implementar un sistema multiprogramación a fin de incrementar la utilización de su procesador. En este caso, debe recordar que la multiprogramación requiere bastante sincronización entre el administrador de la memoria, el administrador del procesador y los dispositivos de E/S. El sacrificio: mejor uso de la CPU contra la sobrecarga incrementada, tiempo de respuesta más lento y salida disminuida.

Hay varios problemas a considerar, algunos de los cuales son los siguientes:

- Un sistema puede alcanzar un punto de saturación si la CPU se utiliza completamente aunque se permite que acepte trabajos adicionales, lo cual puede resultar en mayores sobrecargas y menos tiempo para ejecutar programas.
- Bajo cargas pesadas, el tiempo de CPU requerido para administrar colas de E/S (lo que en circunstancias normales no requiere gran cantidad de tiempo), puede aumentar drásticamente el tiempo necesario para ejecutar los trabajos.
- Con la formación de largas colas en los canales, unidades de control y dispositivos de E/S, la CPU podría permanecer ociosa esperando que los procesos terminen sus operaciones de E/S.

En forma semejante, aumentar el número de procesadores necesariamente aumenta la sobrecarga requerida para administrar múltiples trabajos entre múltiples procesadores. Pero en ciertas circunstancias, el pago puede ser un tiempo de respuesta más rápido.

Rol de la administración de dispositivos

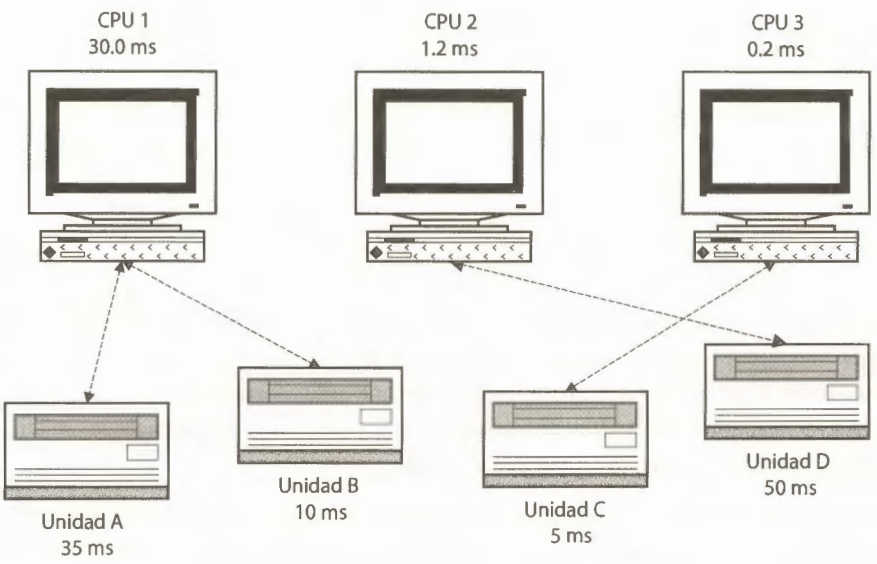
La administración de dispositivos, cubierta en el capítulo 7, contiene varias formas de mejorar la utilización de los dispositivos de E/S incluyendo almacenamiento intermedio, bloqueos y reprogramación de solicitudes de E/S para optimizar los tiempos de acceso. Pero hay desventajas: cada una de estas opciones también incrementan la sobrecarga de la CPU y usa espacio de memoria adicional.

¿Cuán grandes pueden ser los *búfers*? Si son pequeños (64K), pueden llenarse y vaciarse rápidamente. Si son más grandes (256K), pueden requerir menos comandos de lectura y escritura.

El bloqueo reduce el número de solicitudes de E/S, lo cual está bien. Pero es responsabilidad de la CPU bloquear y desbloquear después los registros, y estas son sobrecargas.

El almacenamiento intermedio ayuda a que la CPU haga coincidir la velocidad más lenta de los dispositivos de E/S y viceversa, pero requiere espacio de la memoria para los *búfers*, ya sea espacio dedicado o una sección de la memoria principal asignada temporalmente, lo cual, a la vez, reduce el nivel de procesamiento que puede llevarse a cabo. Por ejemplo, si cada *búfer* requiere 64K de memoria y el sistema requiere dos conjuntos de *búfers* dobles, entonces a los *búfers* se les están dedicando 256K de memoria. La negociación es multiprogramación reducida contra un mejor uso de los dispositivos de E/S.

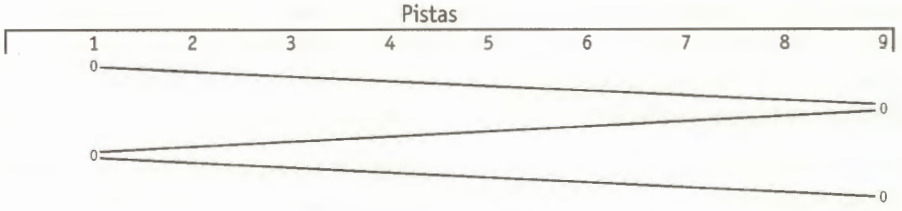
La reprogramación de solicitudes es una técnica que puede ayudar a optimizar los tiempos de E/S; es una técnica de reordenamiento de colas. Pero también es una función de sobrecarga, de modo que la velocidad tanto de la CPU como de los dispositivos de E/S debe ponderarse contra el tiempo necesario para ejecutar el algoritmo de reordenamiento. El siguiente ejemplo ilustra este punto. En la figura 12.1 se muestran tres CPU diferentes con la velocidad para ejecutar 1 000 instrucciones y cuatro unidades de disco con sus velocidades de acceso medias.



(Figura 12.1)
Un sistema con tres CPU y cuatro unidades de disco de diferentes velocidades. En el supuesto de que el sistema requiere 1 000 instrucciones para reordenar solicitudes de E/S, las ventajas del reordenamiento varían dependiendo de la combinación de la CPU y el disco.

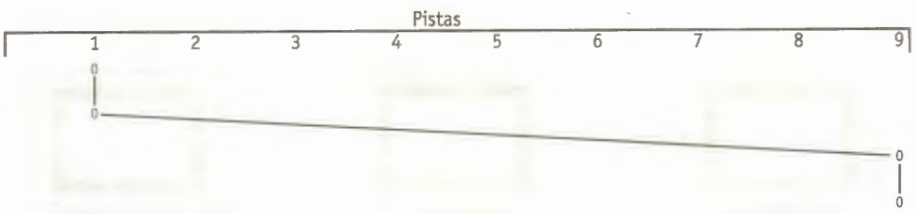
Con los datos en la figura 12.1, y en el supuesto de que un módulo típico de reordenamiento consta de 1 000 instrucciones, ¿qué combinación de una CPU y una unidad de disco garantiza un módulo de reordenamiento? Para conocer la respuesta es necesario comparar las velocidades de acceso del disco antes y después del reordenamiento.

Por ejemplo, suponga que un sistema que consta de la CPU 1 y una unidad de disco A tiene acceso a las pistas 1, 9, 1 y luego 9, y que el brazo ya está colocado en la pista 1. Sin reordenamiento, la unidad A requiere aproximadamente 35 ms para cada acceso: 35 + 35 + 35 = 105, como se muestra en la figura 12.2.



(Figura 12.2)
Al usar la combinación de la CPU 1 y la unidad de disco A sin reordenamiento, el acceso a los datos requiere 105 ms.

Después del reordenamiento (que requiere 30 ms), el brazo puede efectuar ambos accesos a la pista 1 antes del recorrido, en 35 ms, a la pista 9 para los otros dos accesos, resultando en una velocidad casi dos veces más rápida: 30 + 35 = 65, como se muestra en la figura 12.3. En este caso, el reordenamiento mejora la velocidad global.



(Figura 12.3)
Al usar la CPU 1 y la unidad de disco A después del reordenamiento (que requiere 30 ms), el acceso a los datos requiere 35 ms.

Sin embargo, cuando la misma situación es enfrentada por la CPU 1 y la unidad de disco C mucho más rápida, encontramos que el disco empieza de nuevo en la pista 1 y efectúa los 4 accesos en 15 ms (5 + 5 + 5), pero luego se detiene para reordenar estos accesos (que requieren 30 ms), toma 35 ms (30 + 5) para completar la tarea. En consecuencia, las solicitudes de reordenamiento no siempre están garantizadas.

Recuerde que cuando el sistema está configurado, el algoritmo de reordenamiento siempre está encendido o siempre está apagado. El operador de sistemas no puede modificarlo sin reconfiguración, de modo que la disposición inicial, encendido o apagado, debe determinarse al evaluar el sistema con base en el rendimiento medio del sistema.

Rol de la administración de archivos

El análisis de la administración de archivos en el capítulo 8 consideró la manera en que los esquemas de asignación de almacenamiento secundario ayudan al usuario a organizar y tener acceso a los archivos en el sistema. Casi todos los factores analizados en ese capítulo pueden afectar el rendimiento global del sistema.

Por ejemplo, la organización de archivos es una consideración importante. Si un archivo está almacenado de manera no contigua y tiene varias secciones que residen en cilindros bastante separados en un paquete de discos, el acceso secuencial a todos sus registros sería una tarea consumidora de tiempo. Este caso debe sugerir que los archivos deben compactarse, lo cual también se denomina desfragmentación, de modo que cada sección del archivo resida cerca de los otros. Sin embargo, la recompactación requiere tiempo de la CPU y hace que los archivos no estén disponibles para los usuarios mientras se lleva a cabo lo anterior.

Otra cuestión de administración de archivos que puede afectar el tiempo de recuperación es la ubicación del directorio de un volumen. Por ejemplo, algunos sistemas leen el directorio en la memoria principal y ahí lo mantienen hasta que el usuario termina la sesión. Si volvemos a nuestro ejemplo en la figura 12.1 de las cuatro unidades de disco de velocidad variable, la primera recuperación toma 35 ms cuando el sistema recupera el directorio de la unidad A y lo carga en la memoria. Pero cada acceso subsecuente debe realizarse a una velocidad mucho más rápida de la CPU sin necesidad de tener acceso al disco. Resultados semejantes se obtendrían con cada una de las otras unidades de disco, como se muestra en la tabla 12.1.

Esto plantea un problema si el sistema se cae antes de que cualquier modificación se haya registrado permanentemente en almacenamiento secundario. En tal caso, el tiempo de E/S que

(Tabla 12.3)

Un sistema con cuatro unidades de disco de diferentes velocidades y una CPU con velocidad de 1.2 ms. Si el directorio del archivo se carga en la memoria, la velocidad de acceso afecta sólo la recuperación inicial y ninguna de las recuperaciones subsecuentes.

Unidad de disco	Velocidad de acceso para la primera recuperación	Recuperaciones subsecuentes
A	35 ms	1.2 ms
B	10 ms	1.2 ms
C	5 ms	1.2 ms
D	50 ms	1.2 ms

se ahorró al no tener acceso al almacenamiento secundario cada vez que el usuario solicitó ver el directorio se negaría al no tener información actual en el directorio del usuario.

En forma semejante, la ubicación del volumen de un directorio en el disco podría hacer una diferencia importante en el tiempo que se requiere para tener acceso a él. Por ejemplo, si los directorios están almacenados en la pista más externa entonces, en promedio, el brazo de la unidad de disco debe desplazarse más lejos para tener acceso a cada archivo que si los directorios estuviesen guardados en las pistas centrales.

En general, la administración de archivos está estrechamente relacionada con el dispositivo en que están almacenados los archivos, por lo que los diseñadores deben considerar ambas cuestiones al mismo tiempo al evaluar o modificar sistemas de cómputo. Esquemas diferentes ofrecen flexibilidad diferente, pero la desventaja de flexibilidad incrementada del archivo son sobrecargas incrementadas de la CPU.

Rol de la administración de redes

El análisis de administración de redes en los capítulos 9 y 10 examinó el impacto de añadir capacidad de red al sistema operativo y el efecto global sobre el rendimiento del sistema. El administrador de la red sincroniza de manera rutinaria la carga entre procesadores remotos, determina la prioridad de los mensajes e intenta escoger las rutas de comunicación más eficientes sobre múltiples líneas de comunicación de datos.

El rendimiento del sistema para redes puede dar mayor prioridad a ciertos servidores, como correo electrónico o servidores web, si son críticos para la operación de la organización.

Por ejemplo, cuando un programa de aplicación requiere datos de una unidad de disco en ubicaciones diferentes, el administrador de la red intenta proporcionar este servicio de manera fluida. Cuando se requieren dispositivos de la red, como impresoras, plotters o unidades de disco, el administrador de la red tiene la responsabilidad de asignar y liberar correctamente los recursos.

Además de las tareas de rutina realizadas por sistemas operativos independientes, el administrador de recursos permite que un administrador de la red supervise el uso de computadoras individuales y hardware compartido, y asegure el cumplimiento de acuerdos con licencias de software. El administrador de recursos también simplifica el proceso de actualizar archivos de datos y programas en computadoras conectadas a una red al coordinar cambios a través de servidores de comunicaciones en vez de efectuar los cambios en cada computadora individual.

Medición del rendimiento del sistema

El rendimiento total del sistema puede definirse como la eficiencia con la cual un sistema de cómputo cumple sus objetivos; es decir, cuán bien sirve a los usuarios. Sin embargo, la eficiencia del sistema no es fácil de medir porque se ve afectada por tres componentes fundamentales: programas del usuario, programas del sistema operativo y hardware. Además, el rendimiento del sistema puede ser muy subjetivo y difícil de cuantificar: ¿cómo, por ejemplo, puede calibrarse objetivamente la facilidad de uso? Mientras algunos aspectos de facilidad de uso pueden cuantificarse, por ejemplo el tiempo para registrarse, el concepto global es difícil de cuantificar.

Incluso cuando el rendimiento es cuantificable, como el número de accesos a disco por minuto, no es una medida absoluta sino relativa basada en las interacciones de las tres componentes y la carga de trabajo manipulada por el sistema.

Herramientas de medición

La mayoría de diseñadores y analistas dependen de ciertas medidas del rendimiento del sistema: volumen de trabajo, capacidad, tiempo de respuesta, utilización de recursos, disponibilidad y confiabilidad.

El **volumen de trabajo** constituye una medida compuesta que indica la productividad del sistema como un todo; el término a menudo es usado por los administradores del sistema. Suele medirse en condiciones de estado estable y refleja cantidades como "el número de trabajos procesados por día" o "el número de transacciones en línea manipuladas por hora". El volumen de trabajo también puede ser una medida del volumen de trabajo manipulado por una unidad del sistema de cómputo, una diferenciación que resulta de utilidad cuando los analistas buscan cuellos de botella en el sistema.

Los cuellos de botella tienden a desarrollarse cuando los recursos alcanzan su **capacidad** o nivel máximo de resultados; el recurso se satura y los procesos en el sistema dejan de pasarse. *Thrashing* es el resultado de un disco saturado. Los cuellos de botella también ocurren cuando la memoria principal ha sido sobrecomprometida y el nivel de multiprogramación ha alcanzado un punto pico. Cuando ocurre esto, los conjuntos de trabajo para los trabajos activos no pueden mantenerse en la memoria principal, de modo que el administrador de la memoria intercambia páginas de manera continua entre la memoria principal y almacenamiento secundario. La CPU procesa los trabajos de manera muy lenta porque se encuentra muy ocupada lanzando páginas.

El volumen de trabajo y la capacidad pueden monitorearse por medio del hardware o del software. Los cuellos de botella pueden detectarse al monitorear las colas que se forman en cada recurso; cuando una cola comienza a crecer rápidamente, es una indicación de que la tasa de llegada es mayor que o está muy próxima a la tasa de servicio y que el recurso está

saturado. Esto se denomina ciclo de realimentación, que se analizará después de este capítulo. Una vez que se detecta un cuello de botella, es posible emprender la acción idónea para resolver el problema.

Para usuarios interactivos en línea, el **tiempo de respuesta** es una medida importante del rendimiento del sistema. Se trata del intervalo de tiempo requerido para procesar una solicitud de un usuario: desde que el usuario oprime una tecla para enviar el mensaje hasta que el sistema acusa recibo del mensaje. Para trabajos por lotes, esto se denomina **tiempo de espera**: el tiempo desde que se presenta el trabajo hasta que al usuario se regresa un resultado. Sin importar que el contexto sea por lotes o en línea, esta medida depende de la carga de trabajo que está manipulando el sistema y el tiempo de la solicitud y el tipo de trabajo o solicitud que se está presentando. Algunas solicitudes, por ejemplo, se manipulan más rápido que otras porque requieren menos recursos.

Para ser una medida precisa de la previsibilidad del sistema, los datos de medición que muestran el tiempo de respuesta y el tiempo de espera deben incluir no sólo sus valores medios, sino también su varianza.

¿El tiempo de inactividad debe incluirse o excluirse en las estadísticas de disponibilidad? Algunos dicen que no debe incluirse porque el apagón está planeado. Otros dicen que sí debe incluirse porque cualquier apagón significa que el sistema está indisponible.

La **utilización de recursos** es una medida de con cuánto contribuye cada unidad a la operación global. Suele darse como un porcentaje del tiempo que un recurso está realmente en uso. Por ejemplo: ¿la CPU está ocupada 60% del tiempo? ¿La línea de impresión está ocupada 90% del tiempo? ¿Y qué hay respecto a cada una de las terminales? ¿Y del mecanismo de búsqueda en un disco? Estos datos son de utilidad para que el analista determine si entre las unidades de un sistema hay equilibrio o si el sistema es **E/S amarrado** o está limitado por la CPU.

La **disponibilidad** indica la probabilidad de que un recurso esté listo cuando un usuario lo necesite. Para usuarios en línea puede significar la probabilidad de que un puerto esté libre o que una terminal esté disponible cuando intenten registrarse. Para quienes ya están en el sistema, puede significar la probabilidad de que uno o varios recursos específicos, como un plotter o un grupo de unidades de cinta, estén listos cuando su programa haga solicitudes. En su forma más simple, disponibilidad significa que una unidad es operacional y no está fuera de servicio cuando un usuario la necesite.

La disponibilidad es afectada por dos factores: el **tiempo medio entre fallas** (**MTBF**) y el **tiempo medio de reparación** (**MTTR**). El MTBF es el tiempo promedio que una unidad es operacional antes de descomponerse, y MTTR es el tiempo necesario para reparar una unidad con fallo y ponerla en servicio. Estos valores se calculan con ecuaciones aritméticas simples.

Por ejemplo, si se compra una terminal con MTBF de 4 000 horas (la cifra la proporciona el fabricante) y se piensa usarla 4 horas al día durante 20 días al mes (u 80 horas al mes), entonces es de esperar que falle una vez cada 50 meses (4 000/80), lo cual no está mal. El MTTR es el tiempo promedio necesario para reparar una pieza de hardware dañada, y depende de varios factores: la gravedad del daño, la ubicación del taller de reparaciones, la rapidez con que se quiera volver a usarla, la cantidad que desea pagarse, etcétera. En gene-

ral, esta cifra es aproximada. Cuando calcule la disponibilidad, asegúrese de que todas sus variables estén en las mismas unidades (todas en horas, días, etc.).

La fórmula usada para calcular la disponibilidad de la unidad es:

$$\text{Disponibilidad} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Como se indicó, la disponibilidad es una razón entre el MTBF de la unidad y el tiempo total (MTBF + MTTR). Para nuestra terminal, suponga que MTTR es 2 horas; entonces

$$\text{Disponibilidad} = \frac{4000}{4000 + 2} = 0.9995$$

Así, en promedio, esta unidad está disponible 9 995 de cada 10 000 horas. En otras palabras, se espera que falle 5 veces de cada 10 000 usos.

La **confiabilidad** es semejante a la disponibilidad, aunque mide la probabilidad de que una unidad no falle *durante un periodo determinado* (t), y es una función del MTBF. La fórmula introducida por Nickel en 1978 usada para calcular la confiabilidad de una unidad (donde e es una constante matemática aproximadamente igual a 2.71828) es:

$$\textit{Confiabilidad}\ (t) = e^{-(1/\text{MTBF})(t)}$$

Para ilustrar cómo funciona esta fórmula, suponga que es absolutamente necesario usar la terminal durante los 10 minutos antes que se venza el plazo. Con el tiempo expresado en horas, la confiabilidad de la unidad está dada por:

$$\begin{aligned}\textit{Confiabilidad}\ (t) &= e^{-(1/4000)(10/60)} \\ &= e^{-(1/24\,000)} \\ &= 0.9999584\end{aligned}$$

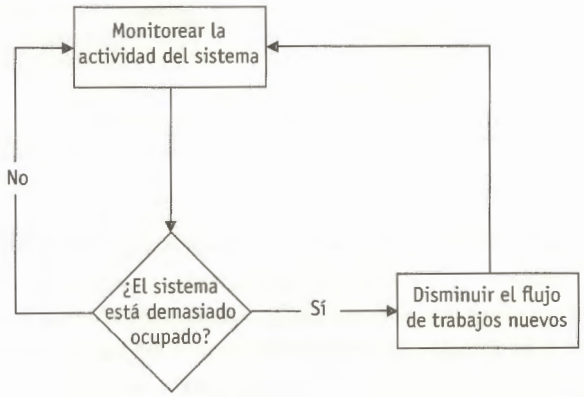
Esta es la probabilidad de que la unidad esté disponible (que no falle) durante el tiempo crítico de 10 minutos y 0.999584 es un número muy alto. En consecuencia, si la terminal está lista en el inicio de la transacción, tal vez permanezca en orden de trabajo durante todo este periodo.

Esas medidas de rendimiento no pueden considerarse de manera aislada de la carga de trabajo manipulada por el sistema, a menos que simplemente se esté afinando una porción específica del sistema. El rendimiento global del sistema varía de vez en cuando, de modo que es importante definir el entorno de trabajo real antes de hacer generalizaciones.

Ciclo de realimentación

Para prevenir que el procesador pase más tiempo haciendo sobrecarga que ejecutando trabajos, el sistema operativo debe monitorear de manera continua el sistema y alimentar esta información al planificador de trabajos. Después, el planificador puede permitir que más trabajos entren al sistema o evitar su entrada hasta que se alivie algo del congestionamiento. Esto se denomina **ciclo de realimentación** y debe diseñarse con sumo cuidado; la realimentación puede ser negativa o positiva.

Un **mecanismo de ciclo de realimentación negativa** monitorea al sistema y después, cuando se congestiona demasiado, indica al planificador que aminore la tasa de llegada de los procesos, como se muestra en la figura 12.4.

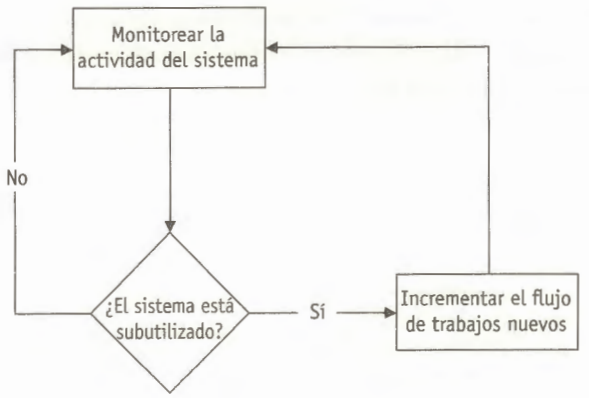


(Figura 12.4)
Ciclo simple de realimentación negativa. Monitorea la actividad del sistema y entra en acción sólo cuando el sistema está demasiado ocupado.

Las personas de vacaciones usan ciclos de realimentación negativa todo el tiempo. Por ejemplo, si usted está buscando una gasolinera y en la primera que encuentra hay demasiados automóviles esperando su turno, usted registra esos datos y reacciona negativamente. En consecuencia, su procesador le sugiere ir a otra gasolinera (en el supuesto, claro, de que no ha demorado demasiado y está a punto de quedarse sin gasolina).

En un sistema de cómputo, un ciclo de realimentación negativa que monitorea dispositivos de E/S informa al administrador de dispositivos que la impresora 1 tiene demasiados trabajos en su cola, ocasionando que el administrador de dispositivos dirija todos los trabajos nuevos que llegan hacia la impresora 2, que no está tan ocupada. La realimentación negativa ayuda a estabilizar el sistema y mantiene la longitud de las colas cerca de los valores medios.

Un **mecanismo de ciclo de realimentación positiva** trabaja en sentido opuesto: monitorea el sistema, y cuando éste se vuelve subutilizado, el ciclo de realimentación positiva provoca un incremento en la tasa de llegada, como se muestra en la figura 12.5. Los ciclos de realimentación positiva se usan en sistemas paginados de memoria virtual, aunque deben usarse con cautela porque son más difíciles de implementar que los bucles negativos.



(Figura 12.3)
Ciclo simple de realimentación positiva. Monitorea la actividad del sistema y entra en acción sólo cuando el sistema no está demasiado ocupado. El monitoreo de la actividad del sistema es crítica aquí porque el sistema puede volverse inestable.

He aquí cómo funciona. El ciclo de realimentación positiva que monitorea a la CPU informa al planificador de trabajos que la CPU está subutilizada, de modo que el planificador permite la entrada de más trabajos al sistema para dar más trabajo a la CPU. Sin embargo, a medida que entran más trabajos, la cantidad de memoria principal asignada a cada trabajo disminuye. Si se permite que demasiados trabajos nuevos entren en un flujo de trabajo, el resultado puede ser un aumento en fallos de página. Y esto, a su vez, puede ocasionar que se deteriore la utilización de la CPU. De hecho, si el sistema operativo está diseñado de manera deficiente, los ciclos de realimentación positiva realmente pueden colocar al sistema en un modo de operación inestable. En consecuencia, los mecanismos de monitoreo para ciclos de realimentación positiva deben diseñarse con extremo cuidado.

Como se muestra con este ejemplo, un algoritmo para un ciclo de realimentación positiva debe monitorear el efecto de nuevas llegadas en dos sitios: el control del administrador del procesador de la CPU y las operaciones de lectura y escritura del administrador de dispositivos. Esto se debe a que ambas áreas experimentan los cambios más dinámicos, lo cual puede conducir a condiciones inestables. Un algoritmo así debe verificar si las llegadas producen el resultado anticipado y si el rendimiento del sistema ha mejorado realmente. Si las llega-

das ocasionan que el rendimiento se deteriore, entonces el algoritmo de monitoreo puede ocasionar que el sistema operativo ajuste sus estrategias de asignación hasta que vuelva a alcanzarse un modo de operación estable.

Administración de parches

La **administración de parches** es la actualización sistemática del sistema operativo y otro software del sistema. Normalmente, un **parche** es una pieza de código de programación que sustituye o modifica el código que constituye el software. Hay tres razones primarias para recalcar los parches de software para una administración profunda del sistema: la necesidad de precauciones de seguridad, vigilantes contra amenazas al sistema que cambian constantemente; la necesidad de asegurar que el sistema cumpla regulaciones gubernamentales con respecto a privacidad y contabilidad financiera y la necesidad de mantener funcionando los sistemas a eficiencia pico.

Los parches no son algo nuevo; se han escrito y distribuido desde que el código de programación se creó por primera vez. Sin embargo, la tarea de mantener los sistemas de cómputo parchados correctamente se ha vuelto un desafío debido a la complejidad de todo el sistema (incluyendo el sistema operativo, la red, varias plataformas y usuarios remotos), y la velocidad con que las vulnerabilidades del sistema son explotadas por gusanos, virus y otros asaltos al sistema descritos más adelante en este capítulo.

Según la encuesta 2004 E-Crime Watch de seguridad y aplicación de la ley, la administración manual y automática de parches estaba entre las ocho tecnologías más usadas en sus organizaciones, como se muestra en la tabla 12.2. La encuesta fue conducida por CSO Online en cooperación con el servicio secreto de Estados Unidos y el CERT Coordination Center.

¿Quién tiene la responsabilidad global de mantener actualizado el software de una organización? Depende de la organización. Para muchos, la responsabilidad global depende del jefe de información o del jefe de seguridad. Para otros, recae en los hombros del administrador de la red o del oficial de seguridad del sistema. Aun para otros, los usuarios individuales asumen ese papel. Sin importar qué persona haga este trabajo, es sólo mediante parches

(Tabla 12.2)

Resultados de una encuesta muestran que la mayoría de los encuestados clasificó las tecnologías de administración de parches entre las herramientas más efectivas para detectar y/o contra el mal uso o abuso de sistemas de cómputo y redes (CSO Online, 2004).

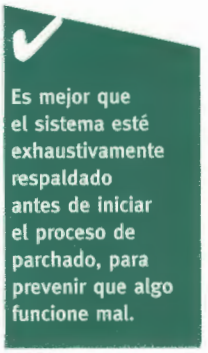
Tecnologías en uso	Clasificación	Porcentaje
Cortafuegos	1	98.2%
Sistemas de seguridad física	2	94.2%
Administración manual de parches	3	91.0%
Administración automática de parches	8	74.4%

rigurosos que los recursos del sistema pueden alcanzar un rendimiento máximo, y su información puede protegerse mejor.

Fundamentos de parches

Aunque la instalación del parche es el evento más público, hay varios pasos esenciales que se llevan a cabo antes de hacer lo anterior:

1. Identificar el parche necesario.
2. Verificar la fuente e integridad del parche.
3. Probar el parche en un entorno seguro.
4. Desplegar el parche a través del sistema
5. Auditar el sistema para calibrar el éxito del desplegado del parche.



Aunque este análisis está limitado a la administración de parches de un sistema operativo, todos los cambios en el sistema operativo o algún otro software crítico deben emprenderse en un entorno que haga respaldos regulares del sistema y pruebe la restauración de los respaldos.

Disponibilidad de parches

Suponga que usted recibe una notificación de que en su red hay disponible un parche para un sistema operativo. Su primera tarea consiste en identificar la importancia del parche, información disponible de parte del vendedor. Para estar a la vanguardia de la información del parche, muchos administradores de sistema se inscriben en servicios de anuncios automáticos ofrecidos por grupos gubernamentales e industriales o al entrar en contacto con vendedores de software.

Si el parche es crítico, es necesario aplicarlo lo más pronto posible. Recuerde que con cada anuncio de un parche, los atacantes del sistema también están armados con esta información crucial y se moverán rápido a fin de comprometer su sistema. Si la naturaleza del parche no es crítica, puede escoger posponer su instalación hasta el inicio de un parche regular. Los ciclos de parches se analizarán después en este capítulo.

Integridad de los parches

Debido a que los parches de software tienen la autorización para escribir sobre código de programación existente, pueden ser especialmente dañinos en caso de no ser válidos. Los parches auténticos cuentan con una firma digital o herramienta de validación. Antes de aplicar un parche, es necesario validar la firma digital usada por el vendedor para enviar el nuevo software.

Prueba de parches

Antes de instalar un parche en un sistema vivo, el parche debe probarse en un sistema muestra o en una máquina aislada para verificar que merece la pena. En caso de no contar con una máquina sin conexión a la red, el parche debe probarse en un sistema desarrollado en lugar de en el sistema operativo.

Primero, debe probarse si el sistema se reinicia después que se instala el parche. Después, debe comprobarse si el software parchado efectúa sus tareas asignadas; si ahuyenta intrusos o mejora el rendimiento del sistema. Aunque a menudo no es factible probar el parche en un sistema duplicado, el sistema probado debe semejar la complejidad de la red objetivo lo más posible.

Este también es el momento para hacer planes detallados respecto a lo que se hará si algo va terriblemente mal durante la instalación. Pruebe sus planes de contingencia para desinstalar el parche y recuperar el software anterior en caso de ser necesario.

Desplegado de parches

En una computadora de usuario único, la implantación de parches es una tarea simple: se instala el software y se reinicia la computadora. Sin embargo, en un sistema multiplataforma con cientos o miles de usuarios, la tarea se vuelve excepcionalmente complicada.

Para garantizar el éxito, es necesario mantener un inventario exacto de todo el hardware y software en esas computadoras que es necesario parchar. En una gran red, esta información puede averiguarse a partir de software de asignación de red que supervisa la red y elabora un inventario detallado del sistema. A su vez, esto supone que todo el hardware del sistema está conectado a la red.

Finalmente, la implantación puede lanzarse en etapas, de modo que el escritorio de ayuda pueda tratar con llamadas telefónicas. A menudo, debido a que es imposible usar el sistema durante el proceso de parchado, se planifica para ocasiones en que el sistema está en uso poco intenso, como por la noche o fines de semana.

Auditar el sistema terminado

Antes de anunciar el éxito de la implantación, es necesario confirmar que el sistema resultante satisface sus expectativas al emprender pasos como los siguientes:

- verificar que todas las computadoras están parchadas correctamente y realizan sus tareas fundamentales según se esperaba.
- verificar que ningún usuario tenga versiones inesperadas o no autorizadas de software que pudiera no aceptar el parche.
- verificar que ningún usuario quede fuera del desplegado.

El proceso debe incluir documentación de los cambios hechos al sistema y el éxito o fracaso de cada etapa del proceso. Es necesario mantener un registro de todos los cambios en el sistema para referencia futura. Finalmente, es necesario obtener realimentación de parte de los usuarios para verificar el éxito del despliegue.

Opciones de software

Los parches pueden instalarse manualmente, uno a la vez, o mediante software escrito para realizar la tarea de manera automática. Organizaciones que escogen software pueden decidir construir su propio software de implantación o comprar software ya elaborado para efectuar la tarea por ellas. La implantación de software se ubica en dos grupos: aquellos programas que requieren un agente (denominado software basado en agente) y los que no (software sin agente).

Service pack es un término usado por algunos vendedores para parches de sistemas operativos.

Si la implantación de software usa un agente, que es software que asiste en la instalación de parches, entonces el agente debe instalarse en todo sistema de cómputo objetivo antes de implantar los parches. En un sistema muy grande o dinámico, esta tarea puede ser desalentadora. En consecuencia, para administradores de grandes redes complicadas, el software sin agente puede ofrecer algunas eficiencias de ahorro de tiempo.

Sincronización del ciclo de parches

Aunque los parches de sistemas críticos deben aplicarse de inmediato, parches menos críticos pueden planificarse a conveniencia del grupo de sistemas. Estos ciclos de parches pueden basarse en eventos de calendario o eventos del vendedor. Por ejemplo, parches de rutina pueden aplicarse mensual o trimestralmente, o pueden programarse de modo que coincidan con la emisión de un **service pack** del vendedor. La ventaja de contar con ciclos de parches de rutina es que permiten una revisión exhaustiva del parche y los ciclos de parche antes de la implantación.

Monitoreo

A medida que han evolucionado los sistemas de cómputo, se han desarrollado varias técnicas para medir el rendimiento de un sistema de trabajo, que pueden implementarse usando ya sea componentes de hardware o software. Los monitores de hardware son más costosos, aunque cuentan con la ventaja de tener un impacto mínimo sobre el sistema porque son externos y están conectados electrónicamente. Incluyen contadores con cables, relojes y elementos comparativos.

Los monitores de software son relativamente poco costosos porque como forman parte del sistema pueden distorsionar los resultados del análisis. Después de todo, el software debe usar los recursos que está intentando monitorear. Además, para cada sistema es necesario desarrollar herramientas de software, por lo que es difícil moverse de un sistema a otro.

En los primeros sistemas, el rendimiento se medía simplemente al sincronizar el procesamiento de instrucciones específicas. Los analistas de sistemas hubieran podido calcular el número de veces que una instrucción ADD podía hacerse en un segundo. O hubieran podido medir el tiempo de procesamiento de un conjunto de instrucciones típico (típico en el sentido de que hubiera representado las instrucciones comunes al sistema). Estas mediciones sólo monitoreaban la velocidad de la CPU porque en esa época la CPU era el recurso más importante, de modo que el resto del sistema se ignoraba.

Hoy, las mediciones del sistema deben incluir las otras unidades de hardware, así como el sistema operativo, los compiladores y otro software del sistema. Las mediciones se hacen de varias formas. Algunas se hacen usando programas reales, normalmente programas de producción de amplio uso por parte de los usuarios del sistema, que se ejecutan en configuraciones diferentes de CPU, sistemas operativos y otras componentes. Los resultados se denominan **pruebas comparativas** y son de utilidad cuando se comparan sistemas que han experimentado extensos cambios. Los vendedores suelen usar pruebas comparativas para mostrar a clientes potenciales las ventajas específicas de una CPU, un sistema operativo, un compilador o una parte de hardware nuevas. En la tabla 12.3 se muestra el ejemplo de una tabla de pruebas comparativas.

Recuerde que los resultados de las pruebas comparativas son altamente dependientes de la carga de trabajo del sistema, su diseño e implementaciones, así como de los requerimientos específicos de las aplicaciones cargadas en el sistema. Los datos de rendimiento suelen obtenerse en un entorno controlado rigurosamente, de modo que los resultados probablemente difieran en la operación de la vida real. Aun así, las pruebas comparativas ofrecen valiosos datos de comparación, un sitio donde iniciar una evaluación del sistema.

Si no es recomendable o posible experimentar con el sistema en sí, para medir el rendimiento puede usarse un modelo de simulación. Este suele ser el caso cuando se está desarrollando nuevo hardware. Un modelo de simulación es una abstracción computarizada de lo que está representado en la realidad. La cantidad de detalles integrados en el modelo es determinada por el tiempo y el dinero: el tiempo necesario para desarrollar el modelo y el costo de su ejecución.

Los diseñadores de los modelos de simulación deben tener cuidado en evitar los extremos de muchos detalles, lo cual se vuelve demasiado costoso para ejecutar, o muy pocos detalles, lo cual no produce suficiente información útil. Si usted quiere ejecutar un programa que es un ejemplo de un modelo de simulación, consulte el primer ejercicio de programación en el capítulo 2.

Las diez primeras TPC-C por rendimiento, (como las de septiembre de 2009)

Clasificación	Compañía	Sistema	tpmC	Precio/tpmC	Disponibilidad del sistema	Base de datos	Sistema operativo	Monitor TP	Fecha de presentación	Grupo
1	IBM	IBM Power 595 Server Model 9119-FHA	6,085,166	2.81 USD	12/10/08	IBM DB2 9.5	IBM AIX 5L V5.3	Microsoft COM+	06/10/08	N
***	Bull	Bull Escala PL6460R	6,085,166	2.81 USD	12/15/08	IBM DB2 9.5	IBM AIX 5L V5.3	Microsoft COM+	06/15/08	N
2	HP	HP Integrity Superdome-Itanium2/ 1.6GHz/ 24MB iL3	4,092,799	2.93 USD	08/06/07	Oracle Database 10g R2 Enterprise Edt w/Partitioning	HP-UX 11i v3	BEA Tuxedo 8.0	02/27/07	N
3	IBM	IBM System p5 595	4,033,378	2.97 USD	01/22/07	IBM DB2 9	IBM AIX 5L V5.3	Microsoft COM+	01/22/07	N
4	IBM	IBM eServer p5 595	3,210,540	5.07 USD	05/14/05	IBM DB2 UDB 8.2	IBM AIX 5L V5.3	Microsoft COM+	11/18/04	N
5	Fujitsu	PRIMEQUEST 580A 32p/64c	2,382,032	3.76 USD	12/04/08	Oracle Database 10g R2 Enterprise Edt w/Partitioning	Red Hat Enterprise Linux 4 AS	BEA Tuxedo 8.1	12/04/08	N
6	Fujitsu	PRIMEQUEST 580 32p/64c	2,196,268	4.70 USD	04/30/08	Oracle 10g Enterprise Ed R2 w/ Partitioning	Red Hat Enterprise Linux 4 AS	BEA Tuxedo 8.1	10/30/07	N
7	IBM	IBM System p 570	1,616,162	3.54 USD	11/21/07	IBM DB2 Enterprise 9	IBM AIX 5L V5.3	Microsoft COM+	05/21/07	N
***	Bull	Bull Escala PL1660R	1,616,162	3.54 USD	12/16/07	IBM DB2 9.1	IBM AIX 5L V5.3	Microsoft COM+	12/17/07	N
8	IBM	IBM eServer p5 595	1,601,784	5.05 USD	04/20/05	Oracle Database 10g Enterprise Edition	IBM AIX 5L V5.3	Microsoft COM+	04/20/05	N
9	Fujitsu	PRIMEQUEST 540A 16p/32c	1,354,086	3.25 USD	11/22/08	Oracle Database 10g release2 Enterprise Edt	Red Hat Enterprise Linux 4 AS	BEA Tuxedo 8.1	11/22/08	N
10	NEC	NEC Express5800/ 1320Xf (16p/32c)	1,245,516	4.57 USD	04/30/08	Oracle Database 10g R2 Enterprise Edt w/Partitioning	Red Hat Enterprise Linux 4 AS	BEA Tuxedo 8.1	01/21/08	N

*** Los resultados duplicados se muestran con un (*) en la columna de Clasificación.
Nota 1: El TPC no cree en la validez de esto para comparar precios o precio/desempeño de los resultados en diferentes monedas.

(Tabla 12.3)
Resultados de pruebas comparativas publicados por el Transaction Processing Performance Council en su página web. Este reporte incluye transacciones por minuto (Tpac), así como precio/rendimiento normalizado ($/tpmC). Este es sólo un reporte muestra. Para resultados de pruebas comparativas recientes y detalles de las condiciones de prueba usadas para generar este reporte, consulte por favor la página web TPC (www.tpc.org). Reproducido con autorización del Transaction Processing Performance Council.

Contabilidad

La función de contabilidad del sistema operativo podría parecer algo prosaico, pero no lo es; paga las cuentas y mantiene financieramente operable al sistema. Desde un punto de vista práctico, puede ser uno de los elementos más importantes del sistema.

La mayor parte de los recursos de un sistema de cómputo son pagados por los usuarios. En el caso más simple, el de un usuario único, es fácil calcular el costo del sistema. Pero en un entorno multiusuario, los costos de computación suelen distribuirse entre los usuarios con base en cuánto usa cada uno los recursos del sistema. Para hacer esta distribución, el sistema operativo debe poder establecer las cuentas de los usuarios, asignar contraseñas, identificar los recursos que están disponibles para cada usuario, y definir cuotas para los recursos disponibles, como espacio de disco o máximo tiempo de CPU permitido. En una universidad, por ejemplo, a los estudiantes a veces se les fijan cuotas que incluyen número máximo de páginas por trabajo, tiempo máximo de utilización de la computadora y número máximo de trabajos durante un periodo dado. Para calcular el costo de todo el sistema, el programa de contabilidad debe reunir información sobre cada usuario activo.

Las políticas de precios varían de un sistema a otro. Mediciones típicas incluyen algunas o todas las cuestiones siguientes:

- *Cantidad total de tiempo* transcurrido entre la presentación y la terminación del trabajo.
- *Tiempo de CPU* dedicado por el procesador para ejecutar el trabajo.
- El *uso de la memoria principal* está representado por unidades de tiempo, bytes de almacenamiento o bytes de almacenamiento multiplicado por unidades de tiempo, lo cual depende de la configuración del sistema operativo. Por ejemplo, un trabajo que requiere 200K por 4 segundos seguido de 120K por 2 segundos puede cobrarse como 6 segundos de uso de la memoria principal, o 320K de uso de memoria, o una combinación de K/segundo de uso de memoria calculado como sigue:

$$[(200 * 4) + (120 * 2)] = 1040\text{K/segundo de uso de memoria}$$

- El *almacenamiento secundario usado durante la ejecución del programa*, como el uso de la memoria principal, puede darse en unidades de tiempo o espacio, o ambas cosas.
- El *almacenamiento secundario usado durante el periodo de facturación* suele darse en términos del número de pistas de disco asignadas.
- El *uso del software del sistema* incluye paquetes de utilidades, compiladores y/o bases de datos.
- El *número de operaciones de E/S* suele agruparse por clase de dispositivo: línea de impresión, terminal y discos.
- *Tiempo dedicado a esperar el completamiento de E/S.*
- El *número de registros de entrada leídos* suele agruparse por tipo de dispositivo de entrada.
- El *número de registros de salida impresos* suele agruparse por tipo de dispositivo de salida.
- El *número de fallos de página* se reporta en sistemas de paginación.

Las políticas de precios algunas veces se usan como una forma para alcanzar objetivos operacionales específicos. Al variar el precio de los servidores del sistema, es posible convencer a los usuarios para que distribuyan su carga de trabajo en beneficio del administrador del sistema. Por ejemplo, al ofrecer tarifas reducidas durante horas libres, algunos usuarios podrían ejecutar trabajos largos en modo por lotes de manera poco costosa durante la noche en lugar de hacerlo de manera interactiva durante horas pico. También es posible usar incentivos de precios para alentar a los usuarios a que tengan acceso a recursos más abundantes y baratos en vez de escasos y costosos. Por ejemplo, al fijar un precio alto en la salida de impresión, los usuarios podrían preferir ordenar un número mínimo de impresiones.

¿El sistema debe proporcionar a cada usuario información al final de cada trabajo o al final de cada sesión en línea? La respuesta depende del entorno. Algunos sistemas sólo proporcionan información sobre uso de recursos. Otros sistemas también calculan el precio de las cuestiones más costosas, como la utilización de la CPU, el uso de almacenamiento en disco, y material de abastecimiento (es decir, papel usado en las impresoras) al final de cada trabajo. Esto proporciona al usuario un reporte actualizado de los gastos y, en caso de ser procedente, calcula el saldo en la cuenta del usuario. Algunas universidades usan esta técnica para advertir a los estudiantes que pagan sobre recursos agotados.

La ventaja de mantener registros de facturación en línea es que el estado de cada usuario puede comprobarse antes de permitir la entrada del trabajo del usuario a la cola LISTO.

La desventaja es la sobrecarga. Cuando se mantienen registros de facturación en línea y se mantiene activo un programa de contabilidad, se usa espacio de memoria y se incrementa el procesamiento de CPU. Un compromiso consiste en posponer el programa de contabilidad hasta fuera de horas, cuando la carga del sistema es ligera.

Conclusión

El sistema operativo es más que la suma de sus partes: es la cooperación orquestada de cada pieza de hardware y cada pieza de software. Como hemos mostrado, cuando una parte del sistema se favorece, a menudo es a costa de las otras. Entonces, si es necesario hacer algún sacrificio, los administradores del sistema deben asegurarse que están reutilizando las herramientas y técnicas de medición idóneas para verificar la eficacia del sistema antes y después de las modificaciones y luego evaluar el grado de mejora.

Con este capítulo concluimos la primera parte de este libro. Hasta el momento hemos visto las semejanzas de los sistemas operativos. En la segunda parte estudiaremos sistemas operativos reales y sus diferencias, así como la forma en que cada uno administra las componentes comunes a todos los sistemas operativos. En otras palabras, veremos cuán estrechamente la realidad se aproxima a los conceptos aprendidos hasta ahora.

Términos clave

Administración de parches: Instalación oportuna de parches de software para hacer reparaciones y mantener en funcionamiento el software del sistema operativo.

Ciclo de realimentación: Mecanismo para monitorear la utilización de los recursos del sistema de modo que sea posible hacer ajustes.

Ciclo de realimentación negativa: Mecanismo para monitorear los recursos del sistema y, cuando esté demasiado congestionado, para indicar al administrador idóneo que aminore la tasa de llegada de los procesos.

Ciclo de realimentación positiva: Mecanismo usado para monitorear el sistema. Cuando el sistema está subutilizado, la realimentación ocasiona el incremento de la tasa de llegada.

Capacidad: Volumen de trabajo máximo de cualquiera de los componentes del sistema.

Confiabilidad: Norma que mide la probabilidad de que una unidad no falle durante un periodo dado. Es una función del tiempo medio entre fallas.

Disponibilidad: Herramienta para la medición de recursos que indica la probabilidad de que el recurso esté listo cuando lo necesite un usuario. Es afectada por el tiempo medio entre fallas y el tiempo medio de reparación.

Parche: Software ejecutable que repara errores u omisiones en otro programa o parte de software.

Pruebas comparativas: Herramienta para la medición usada para medir y evaluar objetivamente el rendimiento de un sistema al ejecutar un conjunto de trabajos representativos de la función que normalmente realiza un sistema informático.

Service pack: Término usado por algunos vendedores para describir una actualización del software de un cliente para reparar problemas existentes y/o entregar mejoras.

Tiempo de respuesta: Medida de la eficiencia de un sistema interactivo que sigue la pista de la velocidad con que el sistema responde a un comando del usuario.

Tiempo de respuesta: Medida de la eficiencia de un sistema que rastrea el tiempo requerido para ejecutar un trabajo y devolver el resultado o salida al usuario

Tiempo medio de reparación (MTTR): Herramienta para la medición de recursos; tiempo promedio necesario para reparar una unidad y volver a ponerla en servicio.

Tiempo medio entre fallas (MTBF): Herramienta para la medición de recursos; tiempo promedio que una unidad es operacional antes de descomponerse.

Utilización de recursos: Medida de cuánto contribuye cada unidad a la operación global del sistema.

Volumen de trabajo: Medida compuesta de la eficiencia de un sistema que cuenta el número de trabajos atendidos en una unidad de tiempo dada.

Búsquedas de interés

- Monitoreo de desempeño
- Comparativos de sistemas
- Parches OS críticos
- Confiabilidad y MBTF
- Auditoría del sistema
- Pérdidas de memoria

Ejercicios

Temas de investigación

A. Visite la página web de un vendedor importante de sistemas operativos e identifique los parches que se han emitido para ese software en los últimos 12 meses. Para cada parche, encuentre su importancia y tamaño. Después, como si a usted se le hubiera asignado la tarea de instalar esos parches, decida el momento de su instalación. ¿Cuántos necesita instalar de inmediato y cuáles podrían esperar el próximo ciclo de parches (en el supuesto de que hay un ciclo de cuatro al año)? Cite sus fuentes.

B. Investigue en la literatura actual para encontrar las estadísticas de disponibilidad de un conjunto de servidores o de otra colección de servidores que es compatible con un sitio Web. Haga una lista de los principales sistemas operativos utilizados y la disponibilidad calculada del sistema como un todo. Mencione si las interrupciones planificadas se incluyen (o no) en las estadísticas de disponibilidad. Cite sus fuentes.

Ejercicios

1. Describa cómo el gerente de un banco puede usar un ciclo de realimentación positivo directamente con clientes en espera para cinco oficinas de préstamos, teniendo cuidado al minimizar el tiempo de espera de los clientes y maximizar la rapidez del procesamiento de los préstamos. Incluya una descripción de cómo podría monitorear el sistema y medir su éxito.
2. Describa cómo usaría un ciclo de realimentación negativa para administrar el balance de su banco. Describa cómo lo haría con un bucle de realimentación positivo. Explique cuál preferiría y por qué.
3. Al recordar que hay una negociación entre el uso de la memoria y la sobrecarga de la CPU, proporcione un ejemplo donde el incremento del tamaño de la memoria virtual mejora el volumen de trabajo. Después, proporcione un ejemplo donde lo anterior ocasiona que el volumen de trabajo sufra.

4. Imagine que administra un sistema en una universidad que llega al límite de la CPU cada final de semestre. ¿Qué efecto en el rendimiento esperaría si le permitieran el doble de procesadores? Si usted pudiera hacer un cambio al sistema, ¿cuál sería?
5. Imagine que administra el sistema de una empresa de consultoría que llega al límite de E/S al final de cada año fiscal. ¿Qué efecto en el rendimiento esperaría si le permitieran el doble de procesadores? Si usted pudiera hacer un cambio al sistema, ¿cuál sería?
6. Use la información proporcionada en la figura 12.1 para calcular la velocidad de acceso de E/S usando la CPU y cada una de las cuatro unidades de disco a medida que evalúan las siguientes solicitudes de pistas: 0, 31, 20, 15, 20, 31, 15. Luego, en cada caso, calcule las velocidades de acceso después de que las pistas solicitadas fueran reordenadas y clasifique las cuatro unidades de disco antes y después del reordenamiento.
7. Describa cómo convencería a un colega para administrar mejor una computadora personal, haciendo respaldos regulares y mantener parchado el sistema.
8. Compare y contraste disponibilidad y confiabilidad. En su opinión, ¿cuál es más importante para el administrador del sistema? Pruebe su respuesta.
9. Calcule la disponibilidad de un servidor con un MTBF de 80 horas y un MTTR de 3 días (72 horas).
10. Calcule la confiabilidad de una unidad de disco duro con un MTBF de 1 050 horas durante las últimas 40 horas de este mes. Suponga $e = 2.71828$ y use la fórmula de Nickel:

$$Confiabilidad\ (t) = e^{-(1/\mathrm{MTBF})(t)}$$

Ejercicios avanzados

11. Calcule la confiabilidad de una unidad de disco duro con un MTBF de 1 050 horas y un MTTR de 8 horas.
12. Calcule la disponibilidad de un servidor con un MTBF de 28 semanas y un MTTR de 504 horas.
13. En este capítulo describimos las negociaciones entre todos los administradores en un sistema operativo. Estudie un sistema al cual tenga acceso, suponiendo que cuenta con fondos suficientes para actualizar sólo un componente del sistema, explique cuál componente escogería para actualizar a fin de mejorar el rendimiento global del sistema, y por qué.
14. Lleve a cabo un inventario de software de su sistema de cómputo para identificar todas las aplicaciones residentes en la computadora. Si tiene más de una docena de aplicaciones, escoja 12 de ellas y verifique los parches que hay disponibles para su software. Para cada vendedor, identifique cuántos parches hay disponibles para su software, el número de parches críticos y qué ciclo de parches recomendaría para el vendedor de parches: actualizaciones anuales, trimestrales, mensuales o semanales.

15. Realice un inventario de su sistema de cómputo (similar al mostrado en la figura 12.6) para descubrir cuántos procesos están activos cuando éste se encuentra conectado al internet. ¿Cuál es el número de procesos ejecutados normalmente? ¿Para cuáles procesos puede identificar la aplicación? ¿Cuántos procesos están conectados a aplicaciones que no puede identificar? Analice cómo los procesos no identificables representan un reto en cuanto a la administración efectiva del sistema y qué haría usted para abordar este reto.

Parte dos

Sistemas operativos en la práctica

Hasta ahora en este texto hemos explorado la forma en que teóricamente funciona el software del sistema operativo: los roles del administrador de la memoria, del administrador del procesador, del administrador de dispositivos, del administrador de archivos y del administrador de la red. Para lograr lo anterior, hemos restado importancia a la intrincada coreografía requerida por todos los administradores a medida en que trabajan juntos para realizar incluso las tareas más simples. También hemos descrito los sistemas operativos en su forma más fundamental: sistemas operativos que ejecutan configuraciones de hardware relativamente simples.

En la segunda parte de este libro, exploramos los sistemas operativos en la práctica al familiarizarnos con algunos de los sistemas operativos más conocidos. Cada uno de los siguientes capítulos describe un sistema operativo; los capítulos están presentados en el orden en que fue lanzado el software. Los cuatro sistemas operativos descritos aquí son –todos– importantes desde un punto de vista histórico, y tres siguen siendo de uso bastante común.

Para cada capítulo en la parte dos, nuestro discurso incluye la historia del desarrollo del sistema operativo; sus objetivos de diseño, las propiedades únicas de sus subadministradores; y su interfaz de usuario, la porción del sistema operativo que interactúa con los usuarios. Los comandos y los formatos de interfaz del usuario varían de sistema a sistema, como se muestra con los ejemplos presentados en esta sección. La interfaz del usuario tal vez es la componente más particular de un sistema operativo, razón por la cual no ha sido presentada hasta ahora.

La historia del desarrollo de un sistema operativo a menudo ilustra sus fortalezas y debilidades intrínsecas. Por ejemplo, un sistema que ha evolucionado desde un rudimentario sistema de usuario único hasta un sistema multifuncional de multiusuarios puede realizar tareas simples bien, pero sufre cuando trata de satisfacer las necesidades de un entorno informático grande. Por otra parte, un elegante sistema de computadoras centrales que después se escala hacia abajo puede ser excelente para tareas complicadas aunque tal vez esté sobrediseñado y sea engorroso al ejecutar tareas en el entorno más pequeño.

Nuestro análisis en esta parte no puede servir como una evaluación a profundidad de ningún sistema operativo, aunque presenta cada sistema en formato normal para ayudar al lector a comparar sus fortalezas y debilidades relativas.

Para empezar, la tabla 1 presenta algunos de los eventos importantes en software de sistemas operativos durante los últimos 50 años.

(Tabla 1)

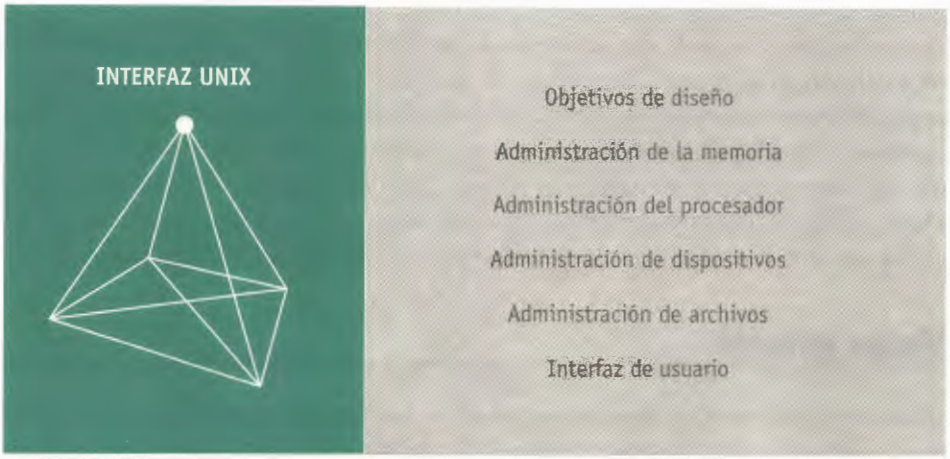
Sistemas operativos selectos y versiones selectas que muestran la evolución del software del sistema operativo.

Sistema operativo	Fecha aproximada	Importancia
OS/PCP (IBM)	1964	Programa de control primario (PCP). Diseñado por IBM para su ejecución en la computadora IBM 360; era un sistema operativo por lotes de tarea única.
OS/MVT (IBM)	1967	Multiprogramación con un número variable de tareas (MVT). Descendiente de la PCP de IBM; introdujo spooling; usaba administración de memoria dinámica.
UNIX	1969	Ken Thompson escribió la primera versión de UNIX.
UNIX (AT&T)	1971	Basado en MULTICS; introdujo el concepto de shell, escrito en lenguaje ensamblador para su ejecución en la computadora PDP-11 de DEC.
VM (IBM)	1973	IBM introdujo el concepto de máquina virtual (VM). Construido para la computadora IBM 370.
MVS (IBM)	1974	Sistema de almacenamiento virtual múltiple (MVS) para computadoras de IBM.
CP/M	1975	Programa de control/Microcomputadora (CM/M). Primer sistema operativo para computadoras pequeñas (PC) vendido comercialmente.
VMS (DEC)	1978	Escrito específicamente para la computadora VAX-11 para aprovechar mejor su arquitectura.
MS-DOS (Microsoft)	1981	Primer sistema operativo para la familia de computadoras IBM PC. La versión 1 era compatible con CP/M y sólo soportaba un directorio.
UNIX System III (AT&T)	1981	Primera versión de UNIX disponible para usar en una computadora central.
Mac OS (Apple)	1984	Primer sistema operativo ampliamente distribuido con una interfaz gráfica de usuario. También introdujo el ratón para computadoras caseras.
Windows 3.0 (Microsoft)	1990	Primera versión utilizable de una interfaz gráfica de usuario para la PC. Requería el sistema operativo MS-DOS.
Linux	1991	Linus Torvald lanzó el núcleo Linux.
OpenVMS Alpha (DEC)	1992	Introdujo direccionamiento de 64 bits, múltiples compiladores y las redes para el sistema Alpha.
Novell UnixWare	1993	Versión de Novell de la versión 4 del sistema V de UNIX, diseñado para trabajar con NetWare.

Sistema operativo	Fecha aproximada	Importancia
FreeBSD 2 (UNIX) (freeBSD.org)	1994	Sistema operativo completo de 32 bits para chips Intel: 386, 486 y clase Pentium.
Solaris (Sun Microsystems)	1994	Versión de Sun de UNIX diseñada para ejecutar en estaciones de trabajo Sun, derivado de la versión 4 del sistema V UNIX de AT&T.
Linux 1.0 (Red Hat)	1995	Primera versión no beta de Linux por Red Hat.
Windows 95 (Microsoft)	1995	Para usuarios de PC. Primer sistema operativo verdadero de Windows. Técnicamente, Windows 95 no estaba relacionado con Windows NT, que tenía capacidades de red.
Linux 3.0.3 (Red Hat)	1996	Primera versión multiarquitectura aproximadamente concurrente; soportaba la plataforma Digital Alpha.
OS/390, ver 2 (IBM)	1997	Capacidad de procesamiento de transacciones escalable, planificador de cargas de trabajo por lotes, seguridad cortafuegos, capacidad de comercio electrónico mejorada.
OpenVMS Alpha, version 7 (DEC)	1997	Diseñado para aplicaciones de memoria muy grande en el sistema Alpha.
Windows 98	1998	Para usuarios de PC. Soporte de hardware más extendido, y completamente de 32 bits. No está relacionado directamente con Windows NT.
Windows 2000	1999	Para redes. El núcleo contenía 45 millones de líneas de código, en comparación con 15 millones para Windows NT. No está relacionado directamente con Windows 95 o Windows 98.
FreeBSD 4 (freeBSD.org)	2000	Sistemas mejorados de administración de archivos y características de seguridad.
Windows XP (Microsoft)	2001	La versión Home fue la sucesora de Windows 95 y Windows 98 para usuarios individuales. Versión Pro disponible en versiones de 32 bits y 64 bits.
Mac OS X 10.0 (Apple)	2001	Combinaba un núcleo basado en UNIX con gráficas Macintosh y una nueva GUI.
Fedora Core 1	2003	Producto Linux basado en Red Hat Linux 9; creado por el Fedora Project en cooperación con Red Hat, Inc.
FreeBSD 6.0	2005	Sistema de archivos multihilo, soporte expandido para tecnología inalámbrica.
FreeBSD 6.2	2007	Mejora de estabilidad, actualizaciones GUI.
Windows Vista (Microsoft)	2007	Seguridad mejorada y nuevas características de rendimiento y estabilidad.

Sistema operativo	Fecha aproximada	Importancia
Fedora 7	2007	Nuevo nombre. Personalización permitida. Accesibilidad ampliada por contribuciones en la comunidad Fedora.
Mac OS X 10.5 (Apple)	2007	Sistema basado en UNIX que permite procesamiento de 64 bits con compatibilidad para aplicaciones y controladores Mac OS X de 32 bits.
RHEL 5 (Red Hat)	2007	Rendimiento, seguridad y flexibilidad mejoradas, con virtualización de almacenamiento.
FreeBSD 6.4	2008	Permite la escalabilidad lineal para más de ocho núcleos de CPU. Apoyo inalámbrico mejorado.
Fedora 11	2009	Rápida inicialización (20 s) desde el encendido hasta la total operación del sistema. Puede manejar archivos de más de 16 TB.
Mac OS X 10.6 Server (Apple)	2009	El núcleo de 64 bits incrementa el número total de procesos simultáneos del sistema, hilos y conexiones de red para el uso del servidor. Soporta el modelo de datos LP64 así como códigos escritos para otros sistemas basados en UNIX de 64 bits que se pueden portar a un servidor Mac OS X.
Windows 7 (Microsoft)	2009	Un sistema operativo de 64 bits fácil de usar y con mejoras para redes sobre Windows Vista.

Capítulo 13 Sistema operativo UNIX



"UNIX es simple. Sólo se requiere un genio para comprender su simpleza".

Dennis Ritchie

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Los objetivos de los diseñadores de UNIX.
- La importancia de usar archivos para manipular dispositivos.
- Las fortalezas y debilidades de contar con versiones competitivas de UNIX.
- Las ventajas de interfaces de usuario controladas por comandos.
- Los roles de los administradores de la memoria, del procesador, de dispositivos y archivos en UNIX.

A diferencia de muchos sistemas operativos, UNIX no está limitado a computadoras específicas que usan un procesador particular como una CPU. Hay muchas versiones de UNIX que se ejecutan en todos los tamaños de computadora, usando una amplia gama de microprocesadores. Además, versiones actuales de muchos otros sistemas operativos han sido revisadas a fin de incluir la capacidad de ejecutar aplicaciones UNIX y conectarse de manera sencilla con redes UNIX.

Aunque Linux (analizado en el capítulo 16) y UNIX son sistemas operativos diferentes, se parecen bastante entre sí de muchas formas. Y aunque durante décadas UNIX ha ejercido un rol dominante en el mercado, hay indicios de que Linux y UNIX están desafiándose mutuamente ahora como pares en todo el mercado. Desde 2001, cuando Apple introdujo el sistema operativo Macintosh OSX basado en la versión libre BSD de UNIX, la compañía publicitaba que este sistema operativo está basado en una plataforma probada y poderosa de UNIX (Apple, 2009). La continua competencia y cooperación entre estos dos sistemas operativos es una que probablemente haga su aparición en el mercado durante los años por venir.

Visión general

UNIX (cuyo autor es Ken Thompson) cuenta con tres ventajas principales: es portable de grandes sistemas a sistemas pequeños, tiene utilidades muy poderosas y proporciona independencia de dispositivos a los programas de aplicación. Su portabilidad se atribuye al hecho de que en su mayor parte está escrito en un lenguaje de alto nivel, C (cuyo autor es Dennis Ritchie), en lugar de un lenguaje ensamblador. Las utilidades son comandos breves de una sola operación que pueden combinarse en una sola línea de comandos para lograr casi cualquier resultado deseado, una característica que muchos programadores encuentran encantadora. Y puede configurarse para operar virtualmente en cualquier tipo de dispositivo.

A lo largo de nuestro análisis de UNIX describiremos la versión de AT&T, a menos que especifiquemos otra cosa. Nota: UNIX es sensible a mayúsculas y minúsculas y está fuertemente orientado hacia caracteres de letras minúsculas, que son más rápidos y fáciles de escribir. En consecuencia, a lo largo de este capítulo todos los nombres de archivo y comandos se muestran con minúsculas.

Historia

La historia de UNIX, resumida en la tabla 13.1 se origina con un proyecto de investigación iniciado en 1965. Era una empresa conjunta entre los laboratorios Bell (el grupo de investigación y desarrollo de AT&T), General Electric y MIT (Massachusetts Institute of Technology).

Cuando los laboratorios Bell abandonaron el proyecto en 1969, la gerencia de AT&T decidió no continuar el desarrollo de ningún otro sistema operativo. Pero esto no detuvo a dos jóvenes veteranos del proyecto MULTICS, Ken Thompson y Dennis Ritchie. Algunas personas afirman que ellos necesitaban un nuevo sistema operativo para atender su juego de computadora favorito. Sin importar las razones por las que lo desarrollaron, UNIX creció hasta convertirse en uno de los sistemas operativos más usados en la historia.

(Tabla 13.1)

Raíces históricas de UNIX y algunas de sus características y modificaciones. Para información actual sobre la historia de UNIX, consulte la página www.theopengroup.org.

Año	Versión	Características
1969	–	Ken Thompson escribió la primera versión de UNIX.
1971	UNIX V1	Basado en MULTICS; introdujo el concepto de shell, escrito en lenguaje ensamblador.
1975	UNIX V6	Primera versión en volverse ampliamente disponible para la industria y la academia.
1980	UNIX System III	Primera versión usada en microcomputadoras de 16 bits.
1984	UNIX System V Release 2	Agregó características de la versión de Berkeley: memoria compartida, más comandos, editor vi, base de datos termcap (capacidad de la terminal), nombres de archivos flexibles.
1991	UNIX System V Release 4	Características combinadas de BSD, SunOS y Xenix.
1991	Solaris 1.0	Versión de Sun diseñada para ejecutarse en estaciones de trabajo Sun, derivada de la versión 4 del sistema V UNIX de AT&T.
1994	Single UNIX Specification 1	Separación de la marca registrada UNIX del flujo de código actual, abriendo la puerta a la estandarización.
1997	Single UNIX Specification 2	Agregó soporte para procesamiento en tiempo real, hilos y procesadores de 64 bits.
2000	Free BSD 4	Sistema de administración de archivos y características de seguridad mejoradas.
2001	Single UNIX Specification 3	Unificó IEEE POSIX y esfuerzos de la industria por estandarizar.
2001	Mac OS X	Apple combinó un núcleo basado en UNIX con gráficas y GUI Macintosh.
2003	ISO/IEC 9945	Norma internacional aprobada para los volúmenes de núcleos de la versión 3 de la especificación única UNIX.
2005	Free BSD 6.0	Sistema de archivos multihilos, con soporte extendido para tecnología inalámbrica.
2007	Mac OS X 10.5	Permite el procesamiento de 64 bits compatible con aplicaciones y controladores Mac OS X de 32 bits.
2008	Free BSD 6.4	Permite la escalabilidad lineal para más de ocho núcleos de CPU; soporte inalámbrico mejorado.
2009	Mac OS X 10.6 Server	El núcleo de 64 bits incrementa el número total de procesos simultáneos del sistema, hilos y conexiones de red para el uso del servidor. Soporta el modelo de datos estándar LP64 así como códigos escritos para otros sistemas basados en UNIX de 64 bits que se pueden portar a un servidor Mac OS X.

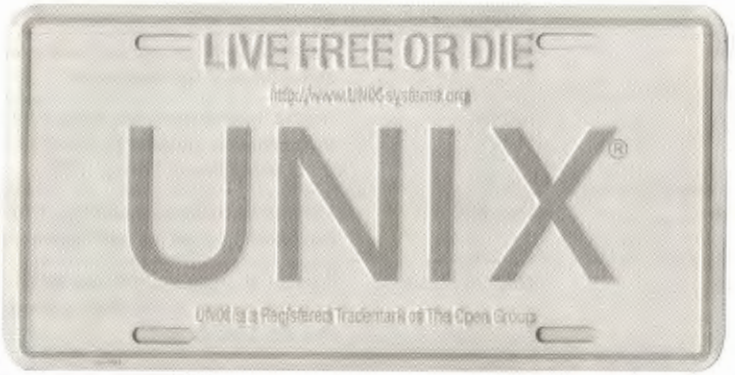
En la tabla 13.1 no se incluye UNIX de Berkeley; originalmente fue desarrollado a partir de la versión 7, aunque algunas características de la versión de Berkeley fueron integradas después en el sistema V, como el sistema de archivo rápido, administración de la memoria paginada, redes, C shell, Vi y otras.

La evolución de UNIX

Thompson y Ritchie escribieron originalmente UNIX en lenguaje ensamblador para una computadora PDP-7 de Digital Equipment Corporation. Brian Kerninghan, un colega, la denominó "UNIX" como un juego de palabras de MULTICS.

La primera versión oficial, presentada en 1971 por Thompson y Ritchie, fue diseñada para "hacer bien una cosa" y ejecutarse en la popular minicomputadora DEC. No mucho después, UNIX llegó a conocerse como un sistema operativo formidable. Para la versión 3, Ritchie dio el paso innovador de desarrollar un nuevo lenguaje de programación, denominado C, y escribió un compilador para el lenguaje C, lo cual hizo más fácil y rápido para los diseñadores escribir código.

A medida que crecían la fama y popularidad de UNIX (vea la figura 13.1, para un ejemplo), AT&T se encontró en una situación difícil. En la época, según reglamentaciones antimonopolio establecidas por el gobierno federal de Estados Unidos, estaba prohibido vender software, aunque era posible, mediante el pago de una cantidad nominal, poner a disposición el sistema operativo, primero para universidades y después para desarrolladores independientes quienes, a su vez, lo transformaron en un producto comercial. Entre 1973 y 1975, se desarrollaron varias versiones mejoradas, de las cuales la más conocida fue creada en la Universidad de California en Berkeley y llegó a conocerse como BSD. Su popularidad en universidades creó una demanda por esa versión en los negocios y la industria, demanda que AT&T era capaz de cumplir a principios de 1984 con la desregulación gubernamental de sus negocios.



(Figura 13.1)
UNIX es una marca registrada de The Open Group. Imagen cortesía de The Open Group

AT&T ingresó en la industria de las computadoras al ofrecer una línea personal de computadoras accionadas por el sistema V de UNIX: su versión de UNIX con características adicionales a la versión de Berkeley. En esa época, AT&T intentó promover su versión de UNIX como la versión normal, pero para entonces UNIX ya había sido adoptado y adaptado por muchos diseñadores para demasiadas máquinas. En 1990, había alrededor de dos docenas

de versiones de UNIX, entre ellas la del sistema V de AT&T; A/UX (sistema V de UNIX para la Macintosh II); Ultrix (UNIX Para el sistema VAX de DEC); XENIX de Microsoft (un sistema operativo basado en UNIX para microcomputadoras que usan procesadores Intel); y las versiones 4.1, 4.2 y 4.3 basadas en UNIX. UNIX de Berkeley es una versión ampliada de la versión 7 de AT&T. Originalmente fue diseñada para ejecutarse en computadoras VAX y se volvió bastante popular en muchos círculos académicos. Aunque era un derivado de UNIX, en algunas áreas –como la estructura del sistema de archivos y el software de red– difería bastante de la versión 1 del sistema V de AT&T.

En 1991, IBM y Hewlett-Packard estuvieron entre las empresas que establecieron The Open Group, propietario de la marca registrada del nombre UNIX, para crear una versión normal del sistema operativo.

En 1993, Berkeley lanzó la 4.4 bsd basada en UNIX de AT&T, requiriendo que los clientes adquirieran licencias de AT&T para usarla. Poco después, Novell adquirió UNIX de AT&T y lanzó su propia versión denominada UNIX Ware diseñada para interactuar con el sistema NetWare de Novell.

La posición original "hacer bien una cosa" de los primeros comandos fue modificada en versiones actuales, y los comandos recientes ofrecen muchas opciones y controles. Este factor tiene sus pros y sus contras: aunque los comandos se vuelven más difíciles de usar, es posible adaptarlos a nuevas situaciones con relativa facilidad. En esencia, las características clave de los primeros sistemas, como tuberías, se han preservado, mientras que el potencial de los comandos ha aumentado para satisfacer nuevas necesidades.

UNIX ofrece apoyo total para redes de área local, y cumple con normas internacionales para los sistemas operativos. Además, la seguridad del sistema ha mejorado bastante y cumple con muchos de los requerimientos de seguridad del gobierno de Estados Unidos. La mayor parte de los sistemas UNIX se caracterizan por tener una interfaz gráfica de usuario diseñada para proporcionar una apariencia uniforme a los sistemas UNIX.

Algunos de los comandos UNIX parecen crípticos a los usuarios no familiarizados con el sistema operativo. Por ejemplo, "grep" es el comando para encontrar una cadena específica de caracteres en un archivo.

Para resolver esta cuestión de normas múltiples, un problema que ha molestado a UNIX desde sus principios, la industria continúa en su intento por estandarizar los programas de un sistema a otro. La versión de la ISO/IEC 9945:2003 fue un gran paso en esa dirección, con más avances esperados en los años por venir.

Objetivos de diseño

Desde el principio, Thompson y Ritchie consideraron a UNIX como un sistema operativo creado por programadores para programadores. Debía ser fácil, flexible y fácil de usar.

Los objetivos inmediatos eran por partida doble: desarrollar un sistema operativo que pudiera atender desarrollo de software y mantener sus algoritmos lo más simples posible (sin volverlos rudimentarios). Para alcanzar su primer objetivo, incluyeron utilidades en el sistema

operativo para el cual los programadores de esa época requerían escribir un código personalizado. Cada unidad estaba diseñada por su simpleza: realizar una sola función pero hacerlo muy bien. Estas utilidades estaban diseñadas para su uso en combinación con otras, de modo que los programadores pudieran decidir y combinar cualesquiera utilidades idóneas que pudieran requerirse para realizar trabajos específicos. Este concepto de pequeñas secciones gestionables de código se ajustaba perfectamente con el segundo objetivo: mantener simple el sistema operativo. Para ello, Thompson y Ritchie escogieron los algoritmos con base en su sencillez, en lugar de hacerlo con base en su velocidad o sofisticación. Como resultado, programadores con experiencia pueden llegar a dominar UNIX en cuestión de semanas.

El objetivo a largo plazo de los programadores era hacer el sistema operativo, y cualquier aplicación de software desarrollada para éste, portable de una computadora a otra. La ventaja evidente de la portabilidad es que reduce los costos de conversión y no ocasiona que los paquetes de aplicaciones se vuelvan obsoletos con cada cambio de hardware. Este objetivo fue alcanzado finalmente con la versión 4 de UNIX porque es independiente del hardware. En esa época, esto era algo innovador.

Numerosas versiones de UNIX cumplen el elemento adicional de diseño de conformarse a las especificaciones de **Interfaz portable de sistema operativo para entornos informáticos** (POSIX), una marca registrada de IEEE. POSIX forma parte de una familia de normas IEEE que define una interfaz de sistema operativo portable para mejorar la portabilidad de programas de un sistema operativo a otro.

Con POSIX, UNIX se ha desarrollado en un sistema operativo poderoso y flexible para entornos multiusuario o sistemas de programación de usuario único.

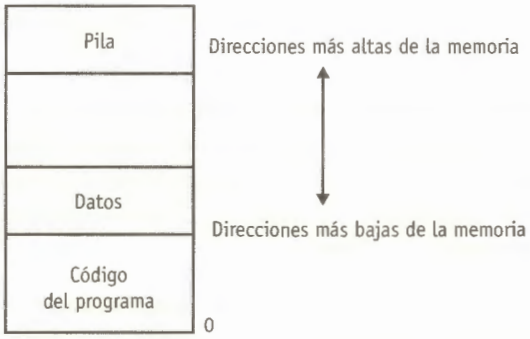
Administración de la memoria

Para sistemas de multiprogramación, la mayor parte de los sistemas operativos UNIX usan técnicas de administración de la memoria ya sea por intercambio de páginas o demanda de paginación (o ambas cosas). La mejor opción depende del tipo de aplicaciones que se ejecutarán en el sistema: si la mayor parte de los trabajos son pequeños, entonces la mejor opción podría ser intercambio de páginas, pero si el sistema está ejecutando muchos trabajos grandes, entonces lo mejor es paginación por demanda.

El intercambio de páginas requiere que todo el programa esté en la memoria principal antes que pueda ser ejecutado, lo cual impone una restricción de tamaño sobre los programas. Por ejemplo, si hay 256 MB de memoria y el sistema operativo toma la mitad para sí mismo (125 MB), entonces el tamaño de los programas debe ser menor que 125 MB. El intercambio de páginas usa una política de *round robin*: cuando la rebanada de tiempo de un programa ha terminado, o cuando se genera una interrupción de E/S, todo el trabajo se intercambia hacia el almacenamiento secundario a fin de hacer espacio para otros trabajos que están esperando en la cola LISTO. Esto está bien cuando en el sistema hay relativamente pocos trabajos, pero cuando el tráfico es pesado, este intercambio de un lado a otro hace lento el sistema.

La paginación por demanda requiere configuraciones de hardware más complicadas; incrementa la sobrecarga del sistema y bajo cargas pesadas puede conducir a *thrashing*. Pero tiene la ventaja de implementar el concepto de memoria virtual.

La figura 13.2 muestra la disposición interna típica de la memoria para una imagen de parte de la memoria de usuario único. Una "imagen" es un concepto abstracto que puede definirse como un entorno de ejecución informático compuesto de una parte de memoria de un usuario (toda la cual se representa en la figura 13.2), valores de registros generales, estados de archivos abiertos y directorio actual. Esta imagen debe permanecer en la memoria durante la ejecución de un proceso.



(Figura 13.2)
Así es como la parte de la memoria de un usuario de una imagen se almacena en la memoria principal.

En la figura 13.2, el segmento denominado código del programa es la porción compartible del programa. Debido a que este código puede compartirse físicamente por varios procesos, debe escribirse en **código reentrante**. Esto significa que el código está protegido, de modo que sus instrucciones no se modifican para nada durante su ejecución normal. Además, todas las referencias a datos se hacen sin el uso de direcciones físicas absolutas.

El administrador de la memoria otorga al código del programa un tratamiento especial. Debido a que varios procesos lo van a compartir, el espacio asignado al código del programa no puede liberarse sino hasta que todos los procesos que lo están usando terminan su ejecución. UNIX usa una tabla de texto para seguir la pista de cuáles procesos están usando qué código del programa, y la memoria no se libera sino hasta que el código del programa ya no es necesario. La tabla de texto se explica con más detalle en la siguiente sección sobre administración del procesador.

El segmento de datos mostrado en la figura 13.2 empieza después del código del programa y crece hacia ubicaciones superiores de la memoria según es necesario por el programa. El segmento de pila empieza en la dirección más alta de la memoria y crece hacia abajo a medida que llamadas a rutinas e interrupciones le agregan información. Una pila es una sección de la memoria principal donde se guarda información del proceso cuando se interrumpe un proceso, o para almacenamiento temporal. Los datos y la pila no son secciones compartibles de la memoria, de modo que cuando el programa original termina la memoria, se libera espacio.

El **núcleo** UNIX, que reside permanentemente en la memoria, es la parte del sistema operativo que implementa las "llamadas al sistema" para establecer los límites de la memoria de modo que varios procesos pueden coexistir en la memoria al mismo tiempo. Los procesos también usan llamadas al sistema para interactuar con el administrador de archivos y para solicitar servicios de E/S.

Mientras cada proceso está en la memoria, el administrador de la memoria los protege uno de otro, de modo que las secciones de datos y de pila de múltiples procesos no se superpongan.

El núcleo es el conjunto de programas que implementan las funciones más primitivas de ese sistema, y es la única parte del sistema operativo que reside permanentemente en la memoria. Las secciones restantes del sistema operativo son manipuladas de la misma forma que en cualquier programa grande. Es decir, páginas del sistema operativo se llevan a la memoria sobre demanda, sólo cuando son necesarias, y su espacio de memoria se libera a medida que se llaman otras páginas. UNIX usa el algoritmo de sustitución de páginas menos usada recientemente (LRU).

Aunque este análisis se ha dirigido hacia grandes sistemas de cómputo multiusuario, UNIX usa los mismos conceptos de administración de la memoria para computadoras de red y sistemas de usuario único. Por ejemplo, una sola computadora con un sistema operativo UNIX, que usa un esquema de paginación por demanda, puede atender usuarios múltiples en un entorno **multitarea** verdadero.

Con la arquitectura de 64 bits agregada en las versiones modernas de UNIX, incluyendo la Mac OS X, el sistema operativo puede hacer llamadas más rápidas al sistema, las que, a su vez, mejoran el desempeño de las aplicaciones E/S y la respuesta de la red. El esquema de direccionamiento de 64 bits es uno de los más compartidos por la mayoría de los sistemas operativos, como el de este artículo.

Administración de procesos

El administrador del procesador de un núcleo de sistema UNIX manipula la asignación de la CPU, la planificación de procesos y la satisfacción de solicitudes de proceso. Para efectuar estas tareas, el núcleo mantiene varias tablas para coordinar la ejecución de procesos y la asignación de dispositivos.

Al usar una política predefinida, el planificador de procesos escoge un proceso de la cola LISTO e inicia su ejecución durante una rebanada de tiempo dada. Recuerde, como lo analizamos en el capítulo 4, los procesos en un entorno de tiempo compartido pueden estar en uno de cinco estados: RETENIDO, LISTO, en ESPERA, en FUNCIONAMIENTO o TERMINADO.

El algoritmo de planificación de procesos escoge el proceso con la prioridad más alta para ser el primero en ejecutar. Puesto que uno de los valores usados para calcular la prioridad es el tiempo acumulado de CPU, cualquier proceso que haya usado bastante del tiempo de CPU obtiene una prioridad inferior que los que no. El sistema actualiza cada segundo la razón-cálculo-a-tiempo-total de cada trabajo. Esta razón divide la cantidad de tiempo de CPU que ha usado un proceso entre el tiempo total que el mismo proceso ha pasado en el sistema. Un resultado próximo a 1 indica que el proceso es limitado por CPU. Si varios procesos tienen

la misma prioridad calculada, se manipulan en modo *round robin* (los procesos de prioridad baja son adueñados por procesos de prioridad alta). Los procesos interactivos suelen tener una baja razón-cálculo-a-tiempo-total, de modo que la respuesta interactiva se mantiene sin ninguna política especial.

El efecto global de esta realimentación negativa es que el sistema equilibra los **trabajos limitados por E/S** con **trabajos limitados por CPU** para mantener ocupado al procesador y minimizar la sobrecarga para procesos en espera.

Cuando el administrador del procesador está decidiendo cuáles procesos de la cola LISTO serán cargados en la memoria para su ejecución, escoge los procesos que han pasado el mayor tiempo en almacenamiento secundario.

Cuando el administrador del procesador está decidiendo cuáles procesos (actualmente en la memoria y en espera o listos para ser ejecutados) serán movidos temporalmente para hacer espacio para una nueva llegada, escoge el proceso que está esperando E/S de disco o que actualmente está ocioso. Si hay varios procesos de los cuales escoger, primero se mueve el que ha estado más tiempo en la memoria.

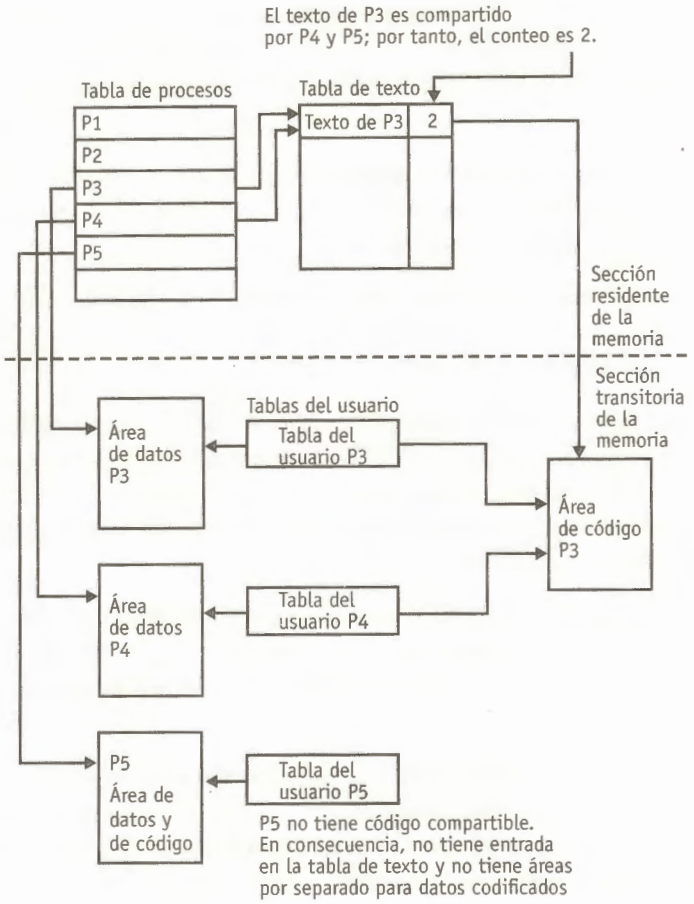
Las técnicas para administración de procesos que se describen aquí se refieren a un sistema de intercambio de páginas puro como el que se usa en UNIX V 7.

Si un proceso que espera el completamiento de una solicitud de E/S no está listo para su ejecución una vez que es escogido, UNIX recalcula dinámicamente todas las prioridades de los procesos para determinar cuál proceso inactivo, pero listo, puede empezar su ejecución una vez que el procesador esté disponible.

Parece que estas políticas funcionan bien y no impactan los procesos en ejecución. Sin embargo, si un disco se usa para almacenamiento secundario de archivos, así como un "área de intercambio", entonces el tráfico pesado puede hacer más lento a E/S del disco porque el intercambio de trabajos puede tener precedencia sobre el almacenamiento de archivos.

Tabla de procesos contra tabla del usuario

UNIX usa varias tablas para mantener funcionando de manera sencilla el sistema, como se muestra en la figura 13.3. Información de procesos simples, los que tienen un código no compartible, se almacena en dos conjuntos de tablas: la tabla de procesos, que siempre reside en la memoria, y la tabla del usuario, que reside en la memoria sólo mientras el proceso está activo. En la figura 13.3, los procesos P3 y P4 muestran con el número 2 el mismo código del programa indicado en la tabla de texto. Sus áreas de datos y las tablas del usuario se mantienen por separado mientras al área de código se comparte. El proceso P5 no está compartiendo su código con ningún otro proceso; en consecuencia, no está registrado en la tabla de texto, y sus datos y área de código se mantienen juntos como una unidad.



(Figura 13.3)

La estructura de control de procesos muestra cómo la tabla de procesos y la tabla de texto interactúan para procesos con código compartible, así como para aquellos sin código compartible.

Cada entrada en la tabla de procesos contiene la siguiente información: número de identificación del proceso, dirección de memoria del proceso o dirección de almacenamiento secundario, tamaño del proceso e información de planificación. Esta tabla se establece cuando se crea el proceso y se borra cuando termina el proceso.

Para procesos con **código compartible**, la tabla de procesos mantiene una subtabla, denominada tabla de texto, que contiene la siguiente información: dirección de memoria o dirección de almacenamiento secundario del segmento de texto (código compartible) y una cuenta para seguir la pista del número de procesos que están usando este código. Cada vez que un proceso empieza a usar este código, la cuenta aumenta por uno; y cada que un proceso deja de usar este código, la cuenta disminuye por uno. Cuando la cuenta es igual a cero, el código ya no es necesario y la entrada de la tabla se libera junto con cualquier ubicación en la memoria que haya sido asignada al segmento de código.

A cada proceso activo se asigna una tabla del usuario. Esta tabla se mantiene en el área transitoria de la memoria en la medida en que el proceso esté activo, y contiene información que debe ser accesible cuando el proceso está en ejecución. Esta información incluye: los números de identificación del usuario y del grupo para determinar privilegios de acceso a archivos, apuntadores a la tabla del sistema de archivos para todo archivo usado por el proceso, un apuntador hacia el directorio actual, y una lista de respuestas para varias interrupciones. Esta tabla, junto con el segmento de datos del proceso y su segmento de código, pueden intercambiarse hacia dentro o hacia fuera de la memoria principal, según sea necesario.

Sincronización

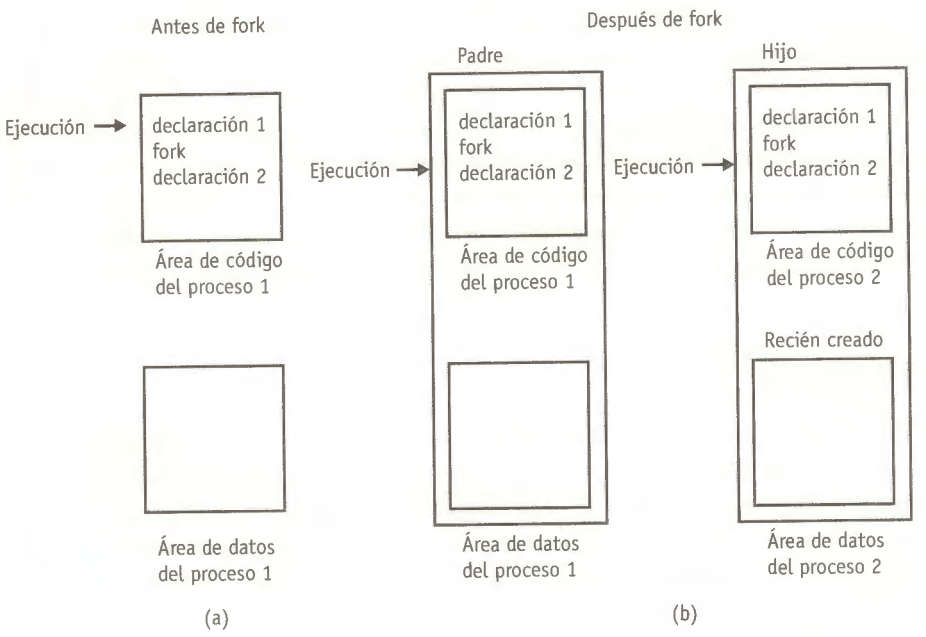
UNIX es un sistema operativo verdaderamente multitarea. Logra sincronización de procesos al requerir que los procesos esperen ciertos eventos. Por ejemplo, si un proceso requiere más memoria, se le pide esperar por un evento asociado con asignación de memoria. Después, cuando la memoria está disponible, el evento es el indicado y el proceso puede continuar. Cada evento está representado por enteros que, por convención, son iguales a la dirección de la tabla asociada con el evento.

Una condición de competencia puede ocurrir si un evento se genera durante la transición de un proceso entre la decisión de esperar por un evento y de entrar en el estado WAIT. En este caso el proceso está esperando por un evento que ha ocurrido previamente y que podría no volver a ocurrir. La **competencia** está esperando un evento que ya ha ocurrido y puede no volver a ocurrir.

fork

Una característica inusual de UNIX es que proporciona al usuario la capacidad de ejecutar un programa a partir de otro programa usando el comando fork. Este comando proporciona al segundo programa todos los atributos del primer programa, como cualquier archivo abierto, y guarda el programa en su forma original.

La llamada al sistema, fork separa el programa en dos copias, ambas en ejecución a partir de la declaración del comando fork. Cuando se ejecuta un comando fork, para el nuevo proceso se genera la "identificación del proceso" (denominada "pid", en forma abreviada). Esto se efectúa de una manera que asegura que cada proceso tenga su número de identificación único. La figura 13.4 muestra qué ocurre después del fork. El proceso original (proceso 1) se denomina **proceso padre** y el proceso resultante (proceso 2) es el **proceso hijo**. Un hijo hereda los archivos abiertos del padre y funciona de manera asíncrona con éste, a menos que el padre tenga instrucciones para esperar la terminación del proceso hijo.

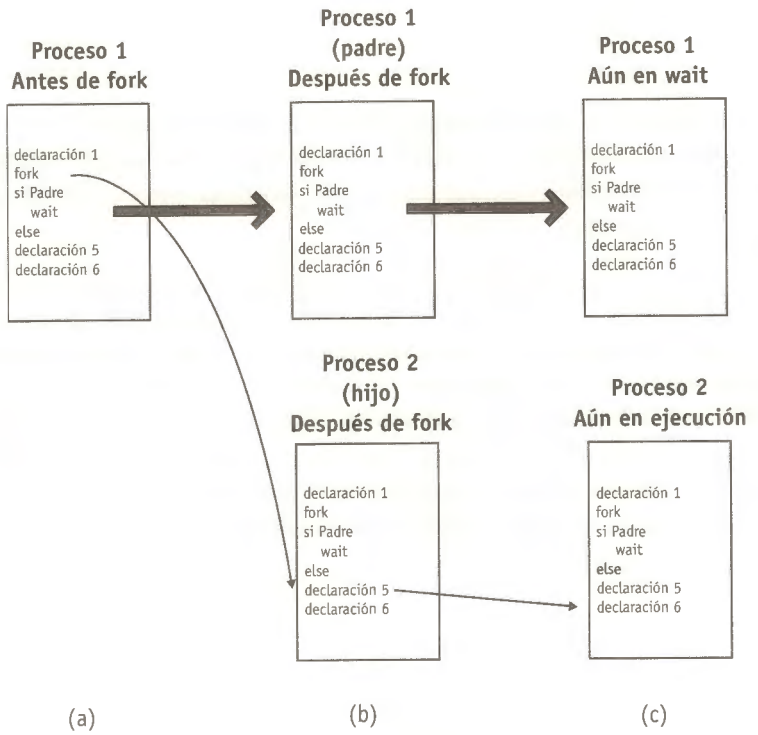


(Figura 13.4)

Cuando se recibe el comando fork, el proceso padre mostrado en (a) genera el proceso hijo mostrado en (b) y la declaración 2 se ejecuta dos veces.

wait

Un comando relacionado, wait, permite que el programador sincronice la ejecución del proceso al suspender al padre hasta que el hijo termine, como se muestra en la figura 13.5.



(Figura 13.5)

El comando wait usado junto con el comando fork sincroniza los procesos padre e hijo. a) muestra el proceso padre, b) muestra los procesos padre e hijo después de un comando tenedor, y c) muestra los procesos padre e hijo durante el comando espera.

En la figura 13.5, la estructura IF-THEN-ELSE es controlada por el valor asignado al pid, que es regresado por la llamada fork al sistema. Un pid mayor que cero indica un proceso padre, un pid igual a cero indica un proceso hijo, y un pid negativo indica un error en la llamada tenedor.

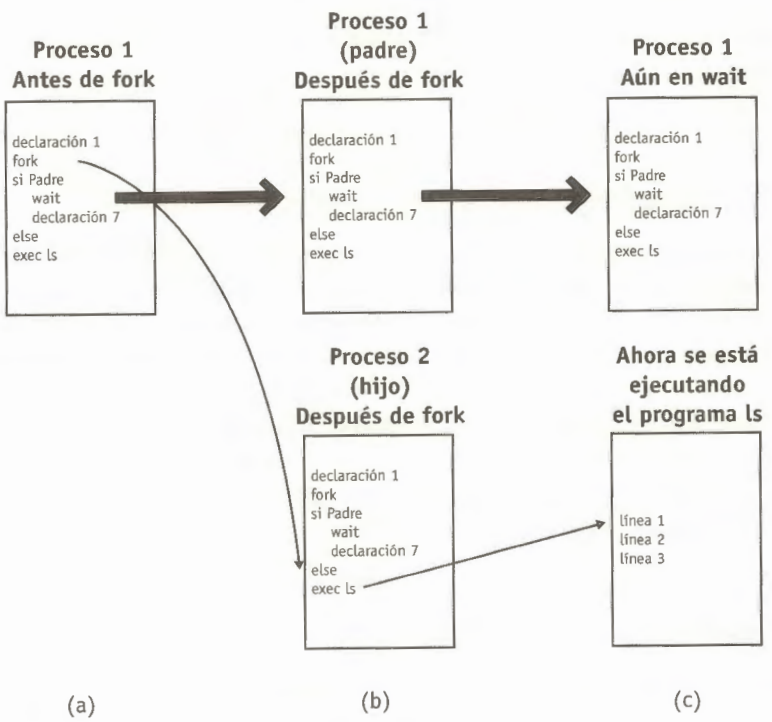
exec

La familia de comandos –execl, execv, execle, execlp y exevp– se usa para empezar la ejecución de un nuevo programa y otro programa, a diferencia del comando fork que resulta en dos procesos que ejecutan el mismo programa que está en la memoria, una llamada exec exitosa superpone el segundo programa al primero, dejando en la memoria sólo al segundo programa. El código y los datos del segundo programa ahora forman parte del proceso original cuyo pid no cambia.

Observe que una llamada exec exitosa no regresa nada; en consecuencia, aquí no se cumple el concepto de padre-hijo. Sin embargo, un programador puede usar los comandos fork, wait y exec en ese orden para crear una relación padre-hijo y luego superponer en el hijo otro programa que, al terminar, despierte al padre de modo que pueda continuar su ejecución, como se muestra en la figura 13.6.

(Figura 13.6)

El comando exec se usa después de la combinación fork y wart. En (a) se muestra al padre antes de fork, (b) muestra los procesos padre e hijo después de un comando fork, y (c) muestra cómo el proceso hijo (proceso 2) es revestido por el programa ls después del comando exec.



El comando ls puede generar un listado del directorio actual. Cuando se ha ejecutado exitosamente la llamada al sistema, exec ls el procesamiento empieza en la primera línea del programa ls. Una vez que el programa ha terminado en el hijo, el control regresa a la declaración ejecutable que sigue a la espera en el proceso padre.

Estas llamadas al sistema ilustran la flexibilidad de UNIX que los programadores encuentran extremadamente útil. Por ejemplo, es posible crear un proceso hijo para ejecutar un programa mediante el proceso padre, como se hizo en la figura 13.6., sin necesidad de que el programador escriba código para cargar o encontrar espacio de memoria para un programa por separado (en este caso, el programa ls).

Administración de dispositivos

Una característica innovadora es el tratamiento de los dispositivos: UNIX está diseñado para proporcionar independencia a los dispositivos y a las aplicaciones que se ejecutan en éstos. Esto se logra al tratar a cada dispositivo de E/S como un tipo de archivo especial. A cada dispositivo que se instala en un sistema UNIX se asigna un nombre semejante al nombre otorgado a otro archivo, que son descriptores denominados. Estos descriptores identifican a los dispositivos, contienen información sobre ellos y se almacenan en el directorio de dispositivos. Las subrutinas que funcionan con el sistema operativo para supervisar la transmisión de datos entre la memoria principal y una unidad periférica se denominan **controladores de dispositivos.**

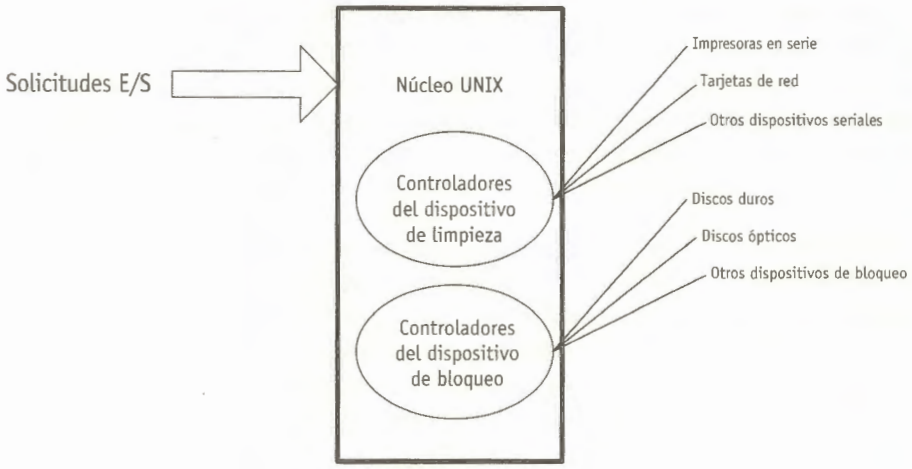
Si el sistema de cómputo usa controladores de dispositivos que no vienen con el sistema operativo, es necesario que un programador experimentado los escriba; o bien, es necesario obtenerlos de una fuente confiable e instalarlos en el sistema operativo.

La incorporación real de un controlador de dispositivos en el núcleo se lleva a cabo durante la configuración del sistema. UNIX cuenta con un programa denominado config que crea en forma automática un archivo config.c para cualquier configuración de hardware. El archivo config.c contiene los parámetros que controla recursos como el número de *búfers* internos para el núcleo y el espacio del espacio de intercambio. Además, el archivo config.c contiene dos tablas: bdevsw (abreviatura de "conmutador para dispositivo de bloque") y cdevsw (abreviatura de "conmutador para dispositivo de caracter"), que proporcionan al núcleo del sistema UNIX la capacidad de adaptarse fácilmente a diferentes configuraciones de hardware al instalar diferentes módulos controladores.

Clasificación de dispositivos

El sistema de E/S está dividido en el sistema de bloques de E/S (algunas veces denominado "sistema estructurado de E/S") y el sistema de caracteres de E/S (algunas veces denominado "sistema no estructurado de E/S").

Cada dispositivo físico es identificado por un número menor de dispositivo, un número mayor de dispositivo y una clase –bloque o caracter– como se muestra en la figura 13.7.



(Figura 13.7)
Cuando un proceso envía una solicitud de E/S, ésta va al núcleo de UNIX donde reside el controlador del dispositivo de limpieza y bloqueo.

Cada clase tiene una tabla de configuración que contiene un arreglo de puntos de entrada hacia los controladores de dispositivos. Esta tabla constituye la única conexión entre el código del sistema y los controladores de dispositivos, y es una característica importante del sistema operativo. Esta tabla permite a los programadores de sistemas crear rápidamente nuevos controladores de dispositivos para dar cabida a sistemas configurados de manera diferente. El número mayor del dispositivo se usa como un índice del arreglo para tener acceso al código idóneo de un controlador de dispositivo específico.

El número menor del dispositivo se pasa al controlador de dispositivos como un argumento y se usa para tener acceso a uno de varios controladores de dispositivos manipulados por el controlador.

Como indica su nombre, el sistema de bloques de E/S es usado por dispositivos que pueden tratarse como una secuencia de bloques de tamaño idéntico. Esto permite que el administrador de dispositivos use almacenamiento intermedio para reducir la E/S física del disco. UNIX cuenta con desde 10 hasta 70 *búfers* para E/S, e información relacionada con estos *búfers* se mantiene en una lista.

Cada que se emite un comando de lectura, se busca en la lista del *búfer* de E/S. Si los datos solicitados ya están en un *búfer*, entonces se ponen a disposición del proceso solicitante. En caso contrario, entonces se mueve físicamente de almacenamiento secundario hacia un *búfer*. En caso que haya un *búfer* disponible, el movimiento se lleva a cabo. Si todos los *búfers* están ocupados, entonces es necesario vaciar uno a fin de hacer espacio para el nuevo bloque. Esto se logra usando una política LRU, de modo que permanece intacto el contenido de *búfers* usados a menudo, lo que a su vez debe reducir espacio físico de E/S en el disco.

Los dispositivos en la clase de caracter son manipulados por controladores de dispositivos que implementan listas de caracteres. He aquí cómo funciona: una subrutina coloca un caracter en la lista, o cola, y otra subrutina recupera el caracter de la lista.

Una terminal es un dispositivo típico de caracter que tiene dos colas de entrada y una de salida. Las dos colas de entrada se identifican como "cola pura" y "cola canónica". Funcio-

na como sigue: a medida que el usuario escribe cada caracter, se reúne en la cola pura de entrada. Cuando se ha completado la línea y se oprime la tecla Enter, la línea se copia de la cola pura de entrada a la cola canónica de entrada y la CPU interpreta la línea. En forma semejante, la sección del controlador de dispositivos que manipula los caracteres que van del módulo de salida a una terminal los almacena en la "cola de salida" hasta que contiene el número máximo de caracteres.

El procedimiento de E/S es sincronizado mediante hardware de conclusión de interrupciones. Cada que hay una interrupción de conclusión, el controlador de dispositivos obtiene del siguiente caracter de la cola y lo envía al hardware. Este proceso continúa hasta que la cola está vacía. Algunos dispositivos pueden pertenecer realmente a ambas clases: de bloque y caracter. Por ejemplo, es posible tener acceso a las unidades de disco y las unidades de cinta en modo de bloques al usar *búfers*, o la cinta puede evitar los *búfers* al tener acceso a los dispositivos en modo de caracter.

Controladores de dispositivos

Cada dispositivo tiene una sección especial en el núcleo denominada controlador de dispositivos. Los controladores de dispositivos de las unidades de disco usan una estrategia de búsqueda para minimizar el movimiento del brazo, como se explicó en el capítulo 7.

Cada controlador de dispositivos incluye todas las instrucciones necesarias para que UNIX se comunique con el dispositivo.

Los controladores de dispositivos se mantienen en un conjunto de archivos que pueden incluirse según sea necesario. Cuando en los periféricos se llevan a cabo actualizaciones, pequeños cambios en los controladores pueden vincularse en el núcleo para que el sistema operativo valore las nuevas características y capacidades. Aunque los archivos de dispositivos pueden mantenerse en cualquier parte del sistema, por defecto y convención se mantienen en el directorio colocar el espacio/dev. El hecho de mantenerlos en este directorio los señala de manera inequívoca como archivos de dispositivos.

El sistema operativo Mac OS X contiene un soporte para RAID 0, RAID 1, RAID 01 y RAID 10, como se muestra en la figura 13.8.

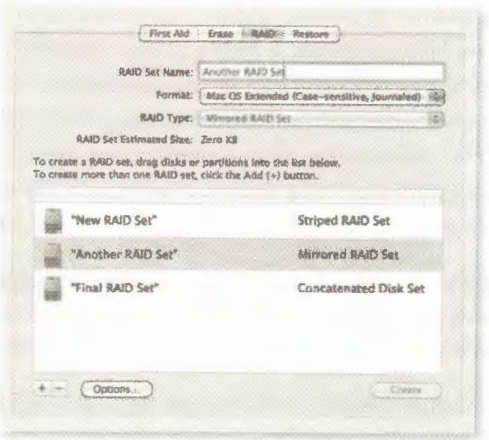


Figura 13.8
Establecimiento de una red de espejo RAID usando Mac OS X.

Administración de archivos

UNIX tiene tres tipos de archivos: directorios, normales y especiales. Cada archivo disfruta de ciertos privilegios.

Los *directorios* son archivos usados por el sistema para mantener la estructura jerárquica del sistema de archivos. Los usuarios tienen permitido leer información en estos archivos, pero sólo el sistema puede modificarlos.

Los *archivos normales* son aquellos en que los usuarios almacenan información. Su protección está basada en las solicitudes del usuario y está relacionada con funciones de lectura, escritura, ejecución y borrado que pueden efectuarse en un archivo.

Los *archivos especiales* son controladores de dispositivos que proporcionan la interfaz al hardware de E/S. Los archivos especiales aparecen como entradas en directorios. Son parte del sistema de archivos, y la mayor parte reside en el directorio colocar el espacio/dev. El nombre de cada archivo especial indica el tipo de dispositivo con el que está asociado. La mayoría de los usuarios no necesitan saber mucho sobre archivos especiales, pero los programadores del sistema deben saber dónde están y cómo usarlos.

UNIX almacena archivos como secuencias de bytes y no les impone ninguna estructura. En consecuencia, los archivos de texto (los escritos usando un editor) son cadenas de caracteres con líneas delimitadas por un caracter de línea de alimentación, o de nueva línea. Por otra parte, los archivos binarios (los que contienen un código ejecutable generado por un compilador o un ensamblador) son secuencias de dígitos binarios agrupados en palabras como aparecen en la memoria durante la ejecución del programa. Por tanto, la estructura de los archivos la controlan los programas que los usan, no el sistema.

El sistema de administración de archivos organiza el disco en bloques de 512 bytes cada uno y divide el disco en cuatro regiones básicas:

- La primera (empezando en la dirección 0) está reservada para el arranque.
- La segunda, denominada "superbloque" contiene información sobre el disco como un todo, como su tamaño y los límites de las otras regiones.
- La tercera incluye una lista de definiciones de archivos, denominada "lista i", que es una lista de descriptores de archivos, uno por cada archivo. Los descriptores se denominan "nodos i". La posición de un nodo i en la lista se denomina "número i" y es este número i el que identifica de manera única a un archivo.
- La cuarta contiene los bloques libres disponibles para almacenamiento de archivos. Los bloques libres se mantienen en una lista enlazada donde cada bloque apunta al siguiente bloque vacío disponible. Así, a medida que un archivo crece, bloques no contiguos se enlazan con la cadena existente.

Siempre que es posible, los archivos se almacenan en bloques vacíos contiguos. Y como toda la asignación de disco está basada en bloques de tamaño fijo, la asignación es muy simple y no hay necesidad de compactar los archivos.

Cada entrada en la lista i se denomina "nodo i" (o inodo) y contiene 13 direcciones de disco. Las diez primeras direcciones apuntan a los diez primeros bloques. La undécima apunta hacia un bloque que contiene un archivo. Sin embargo, si un archivo es más grande que diez bloques, la undécima dirección apunta a un bloque que contiene las direcciones de los 128 bloques siguientes del archivo. Para archivos más grandes, la duodécima dirección apunta hacia otro conjunto de 128 bloques, cada uno apuntando a 128 bloques. Para archivos más grandes que 8 MB hay una decimotercera dirección que permite un tamaño máximo de archivo de más de 1 GB.

Cada nodo i contiene información de un archivo específico, como la identificación del propietario, los bits de protección, el tamaño del archivo, el último uso y la última actualización, el número de enlaces y si el archivo es un directorio, un archivo normal o un archivo especial.

Convenciones de nomenclatura de archivo

Los nombres de archivo son sensibles a mayúsculas y minúsculas, de modo que reconocen letras mayúsculas en nombres de archivo. Por ejemplo, estos nombres son legítimos para archivos diferentes en un solo directorio: `FIREWALL`, `firewall`, `FireWall` y `fireWALL`.

La mayor parte de las versiones de UNIX permiten nombres hasta de 255 caracteres de longitud. Aunque el sistema operativo no impone ninguna convención para la nomenclatura de archivos, algunos programas del sistema, como compiladores, esperan que los archivos tengan "sufijos" específicos (que son lo mismo que las "extensiones" descritas en el capítulo 8). Por ejemplo, prog1.bas podría indicar que el archivo es un programa BASIC debido a su sufijo .bas, mientras el sufijo en backup.sh indica que el archivo es un programa "shell".

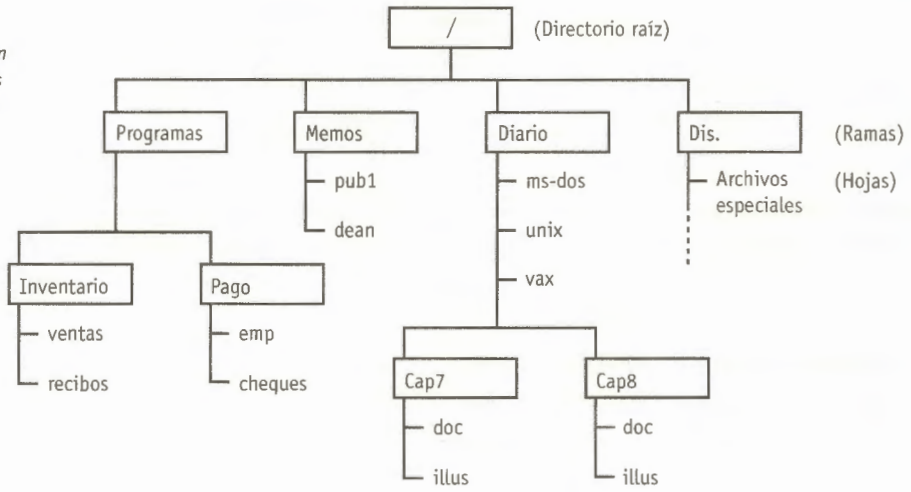
UNIX atiende una estructura de directorio de árbol jerárquica. El directorio raíz se identifica con un slash (/); los nombres de otros directorios están precedidos por el símbolo de slash (/), que se usa como delimitador. Se tiene acceso a un archivo empezando en un punto dado en la jerarquía y descendiendo a través de las ramas del árbol (subdirectorios) hasta llegar a la hoja (archivo). Esta ruta puede volverse muy larga y algunas veces es ventajoso cambiar directorios antes de tener acceso a un archivo. Esto puede hacerse rápidamente al escribir dos puntos ("..") si el archivo se necesita en un nivel arriba del directorio de trabajo en la jerarquía. Al escribir ../.. se efectúa un movimiento de dos ramas hacia la raíz en la estructura de árbol.

Para tener acceso al archivo checks en el sistema ilustrado en la figura 13.9, el usuario puede escribir lo siguiente:

```
/Programas/Pago/cheques
```

(Figura 13.9)

Jerarquía de los archivos con (/) como raíz, los directorios como ramas y los archivos como hojas.



El primer slash indica que esta es una ruta absoluta que empieza en el directorio raíz. Por otra parte, una ruta relativa es aquella que no empieza en el directorio raíz. A continuación se presentan dos ejemplos de rutas relativas de la figura 13.9:

```
Pago/cheques
Diario/Cap8/illus
```

Algunas reglas son válidas para todos los nombres de ruta:

- Si la ruta empieza con un slash, la ruta empieza en el directorio raíz.
- Una ruta puede ser un nombre o una lista de nombres separados por slashes. El último nombre en la lista es el nombre del archivo solicitado.
- Al usar dos puntos (..) en una ruta se realiza un movimiento hacia arriba en la jerarquía (más cerca de la raíz). Esta es la única manera de ascender en la jerarquía; todas las otras rutas descienden por el árbol.
- Dentro de las rutas no se permiten espacios.

Listas de directorios

Como se muestra en la tabla 13.2, un "listado largo" de archivos en un directorio muestra ocho partes de información para cada archivo; el control de acceso, el número de enlaces, el nombre del grupo y del propietario, el tamaño de bytes del archivo, la fecha y hora de la última modificación y, finalmente, el nombre del archivo. Observe que la lista está en orden alfabético por nombre de archivo.

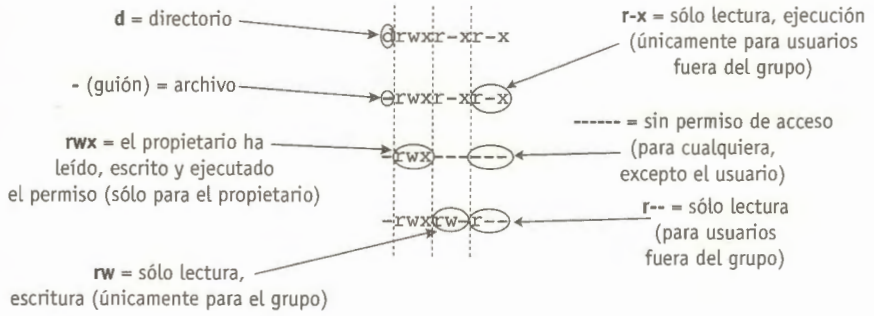
Control de acceso	No. de enlaces	Grupo	Propietario	No. de bytes	Fecha	Hora	Nombre de archivo
drwxrwxr-x	2	Periódico	comp	128	10 de ene	19:32	Cap7
drwxrwxr-x	2	Periódico	comp	128	15 de ene	09:59	Cap8
-rwxr-xr-x	1	Periódico	comp	11904	6 de ene	11:38	ms-dos
-rwxr--r-	1	Periódico	comp	12556	20 de ene	18:08	unix
-rwx-----	1	Periódico	comp	10362	17 de ene	07:32	vax

(Tabla 13.2)

Esta tabla muestra la lista de archivos almacenados en el periódico del directorio del sistema ilustrado en la figura 13.9. Para generar esta lista se usó el comando ls –1 (abreviatura de "listado largo").

La primera columna muestra el tipo de archivo y los privilegios de acceso para cada archivo, como se muestra en la figura 13.10. El primer caracter en la primera columna describe la naturaleza del archivo o directorio; d indica un directorio y – indica un archivo normal. Otros códigos que pueden usarse son:

- *b* para indicar un archivo especial de bloque.
- *c* para indicar un archivo especial de caracter.
- *p* para indicar un archivo de tubería nombrado.



(Figura 13.10)

Representación gráfica de una lista de permisos para archivos y directorios en UNIX.

Los tres caracteres siguientes (rwx) muestran los privilegios de acceso otorgados al propietario del archivo; r significa lectura; w, escritura y x, ejecución. En consecuencia, si la lista incluye rwx, el usuario puede leer, escribir y/o ejecutar ese programa.

En forma semejante, los tres caracteres siguientes describen los privilegios de acceso otorgados a los otros miembros del grupo del usuario. (En UNIX, un "grupo" se define como un conjunto de usuarios que tienen algo en común: el mismo proyecto, el mismo curso, el mismo departamento, etc.) En consecuencia, rwx para los caracteres 5-7 significa que otros usuarios también pueden leer, escribir y/o ejecutar ese archivo. Sin embargo, el guión – indica que el acceso se ha negado para esa operación. En la taba 13.2, el r-- significa que el archivo denominado unix puede ser leído por otros miembros del grupo, pero que no pueden modificarlo ni ejecutarlo.

Finalmente, los tres últimos caracteres en la columna uno describen los privilegios de acceso otorgados a los usuarios en general; quienes no son el propietario o parte del grupo del propietario. Así, los usuarios en general no pueden modificar los archivos enumerados

en la tabla 13.2, y tampoco pueden leer, modificar o ejecutar el archivo denominado unix. Aún más, el archivo vax no puede leerse, modificarse o ejecutarse por nadie que no sea el propietario.

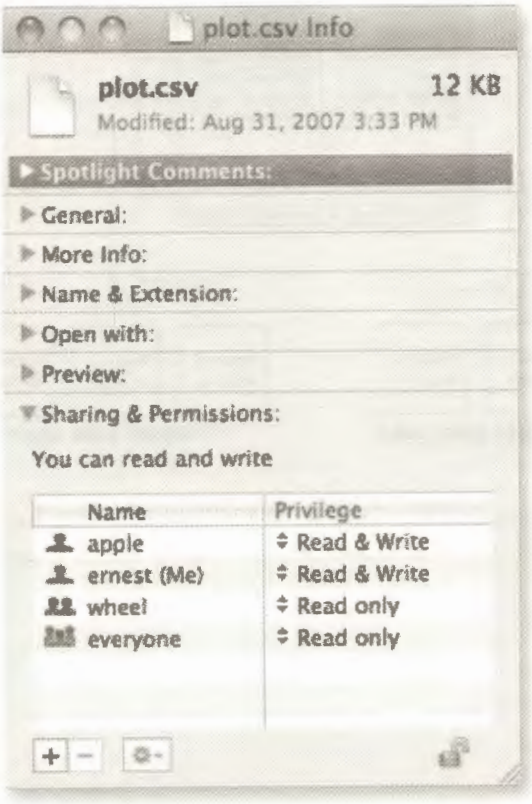
La segunda columna en el listado del directorio indica el número de enlaces, también denominado número de alias, que se refieren al mismo archivo físico. Los alias son una característica importante de UNIX; soportan la compartición de archivos cuando varios usuarios trabajan juntos en el mismo proyecto. En este caso es conveniente que los archivos compartidos aparezcan en diferentes directorios pertenecientes a distintos propietarios aunque para ese archivo sólo existe un descriptor de archivo central (que contiene la información sobre el archivo). El nombre de archivo puede ser diferente de un directorio a otro, puesto que estos nombres no se mantienen en el descriptor de archivo, aunque el número de enlaces ahí mantenidos se actualiza de modo que el sistema sabe cuántos usuarios están compartiendo ese archivo. Eventualmente, este número puede bajar a 0 indicando que el archivo no es de la longitud necesaria y puede ser eliminado.

Las tres columnas siguientes muestran, respectivamente, el nombre del grupo, el nombre del propietario y el tamaño del archivo en bytes. Las columnas sexta y séptima muestran la fecha y la hora de la última modificación, y la última columna muestra el nombre del archivo.

La figura 13.11 muestra que el Mac OS X utiliza una lista de control de acceso para limitar el acceso a los archivos para ciertos usuarios y grupos.

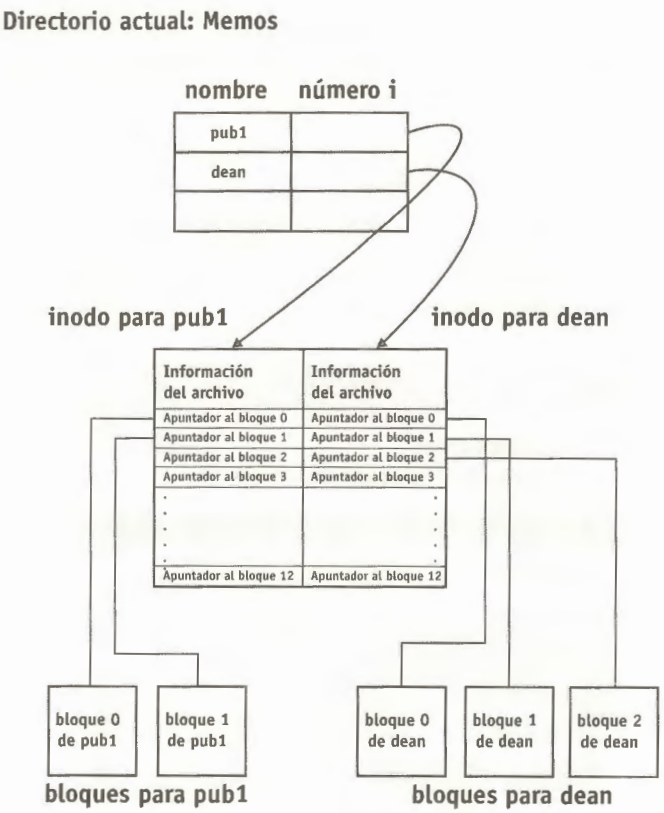
(Figura 13.11)

En el Mac OS X, el buscador (Finder) usa una lista de control de acceso para agrupar los permisos para el acceso a los archivos.



Estructuras de datos

No toda la información presentada en el directorio se mantiene en la misma ubicación. UNIX divide la información del archivo en dos partes: parte en las entradas del directorio y parte en los inodos. En consecuencia, todo lo que se ve en la tabla 13.2, con excepción del nombre del archivo y la adición de las direcciones de los archivos físicos para el contenido del archivo, se mantiene en el inodo. Todos los inodos se almacenan en una parte reservada del dispositivo donde reside el directorio, normalmente en el bloque 1. Esta estructura se ilustra en la figura 13.12, que usa los memos del directorio de la figura 13.9 como ejemplo.



(Figura 13.12)

Ejemplo de jerarquía para directorios, inodos y bloques de archivos. Aunque los bloques de archivos están representados aquí en orden físico en serie, en realidad pueden estar almacenados de manera no contigua.

Cada inodo tiene espacio para 13 apuntadores (0-12). Los diez primeros números de bloques almacenados en la lista de inodos de relacionan con los diez primeros bloques en un archivo.

Para el archivo denominado pub1 en la figura 13.12, sólo las dos primeras entradas tienen apuntadores hacia bloques de datos y todas las otras son ceros porque este es un archivo pequeño que ocupa sólo dos bloques de almacenamiento. Si un archivo es más grande que diez bloques, entonces la undécima entrada apunta hacia un bloque que contiene una lista de los 128 bloques siguientes en el archivo. Debido a que este bloque es un paso extra en la ruta hacia los datos, se denomina "bloque indirecto".

Para archivos más grandes que 138 bloques, la duodécima entrada apunta hacia un bloque que contiene 128 bloques indirectos (cada uno de los cuales contiene apuntadores hacia 128 bloques de archivos). Debido a que este bloque introduce dos pasos extra en la ruta hacia los datos, se denomina "bloque indirecto doble". Finalmente, para archivos extremadamente grandes de más de 16 522 bloques, la decimotercera entrada apunta hacia un "bloque indirecto triple". Este esquema permite la asignación de 2 113 674 bloques en un sólo archivo, para un total de 1 082 201 088 bytes.

En consecuencia, al llevar más adelante este paso puede verse que los bytes numerados por abajo de 5 120 pueden recuperarse con un sólo acceso al disco. Los que están entre 5 120 y 70 656 requieren dos accesos al disco. Los que están entre 70 656 y 8 459 264 requieren tres accesos al disco, y los que están más allá de 8 459 264 requieren cuatro accesos al disco. Con lo anterior, el acceso a grandes archivos de datos sería muy lento pero, en realidad, el sistema mantiene un mecanismo de almacenamiento intermedio más bien complicado que reduce considerablemente el número de operaciones de E/S necesarias para tener acceso a un archivo.

Cuando se crea un archivo, se le asigna un inodo, y se crean una entrada del directorio con el nombre del archivo y el número de su inodo. Cuando se enlaza un archivo (lo que ocurre cuando otro usuario empieza a compartir el mismo archivo), se crea una entrada de directorio con el nuevo nombre y el número del inodo original, y el campo de conteo en el inodo se incrementa por uno.

Cuando se borra un archivo compartido, el campo de conteo en el inodo se disminuye por uno. Y cuando la cuenta llega a cero, la entrada de directorio se borra y se liberan todas las asignaciones de bloques de disco asignadas al archivo, junto con su entrada del inodo en la lista i del disco.

Interfaz de usuario

UNIX fue creado como un sistema controlado por comandos y sus comandos de usuario suelen ser muy cortos (lo cual se muestra en la tabla 13.3): de un caracter (normalmente la primera letra del comando) o un grupo de caracteres (un acrónimo de las palabras que integran el comando). En modo de comando, el caracter prompt del sistema es muy económico, a menudo de un solo caracter, como un signo de dólar ($) o porcentaje (%).

Comando	Significa	Acción a ejecutar
(nombre del archivo)	Ejecutar el archivo	Ejecutar el archivo con ese nombre
ls	Lista de directorios	Mostrar una lista de los nombres de archivo en el directorio
ls -l	Lista larga	Mostrar una lista exhaustiva de directorios
cd	Cambiar de directorio	Cambiar el directorio de trabajo
cp	Copiar	Copiar un archivo en otro archivo o en un directorio
rm	Remover	Remover/eliminar un archivo o un directorio
mv	Mover	Mover o renombrar un archivo o un directorio
more	Ver más	Escribir el contenido de un archivo en la pantalla
lpr	Imprimir	Imprimir un archivo
date	Fecha	Mostrar la fecha y la hora
date-u	Fecha/hora universal	Mostrar la fecha y la hora en formato universal (tiempo del meridiano de Greenwich)
mkdir	Hacer un directorio	Hacer un directorio nuevo
grep	"Expresión regular global/imprimir"	Encontrar una cadena específica en el directorio
cat	Concatenar	Concatenar archivos
format	Formatear	Dar formato a un volumen
diff	Diferente	Comparar dos archivos
pwd	Imprimir el directorio de trabajo	Mostrar el nombre del directorio de trabajo

(Tabla 13.3)

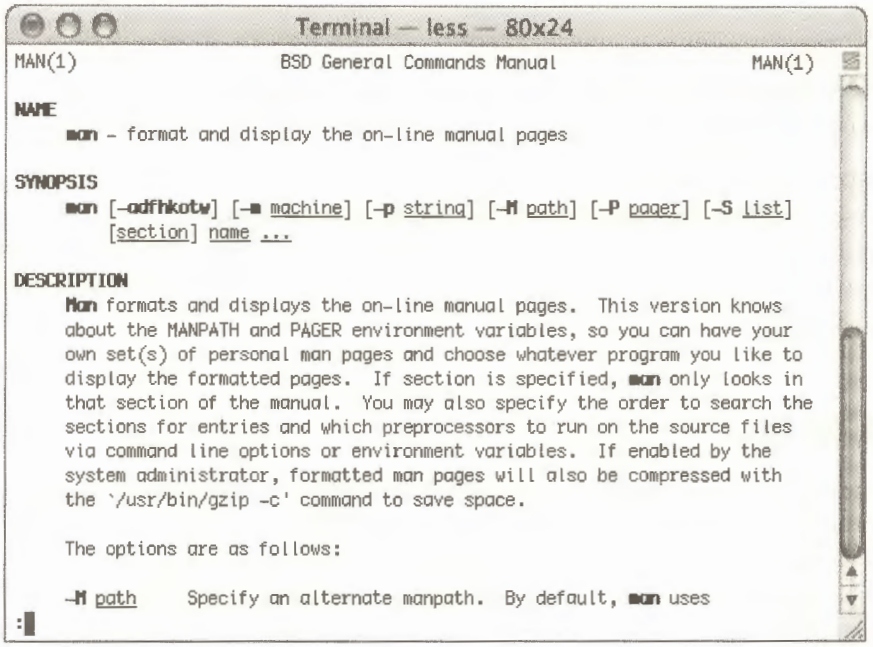
Los comandos de usuario no pueden abreviarse o ampliarse y deben escribirse con el tipo de letra correcto (en general, los comandos deben escribirse sólo con minúsculas). Muchos comandos pueden combinarse para potencia y flexibilidad adicionales. Para conocer la ortografía y sintaxis correctas en su sistema, consulte la documentación técnica.

La sintaxis general de los comandos es como sigue:

```
command arguments file_name
```

El "comando" es cualquier comando legal del sistema operativo, como se muestra en la figura 13.13. Los "argumentos" son necesarios para algunos comandos y opcionales para otros. El "file_name" puede ser una ruta relativa o absoluta. Los comandos se interpretan y ejecutan por el programa shell, uno de los dos programas más ampliamente usados. El shell se conoce técnicamente como intérprete de la línea de comandos porque esa es su función. Sin embargo, no es sólo un intérprete de comandos interactivo, también es la clave para la coordinación y combinación de los programas del sistema. De hecho, es un sofisticado lenguaje de programación en sí.

En este capítulo analizamos las estructuras de comandos comunes a UNIX y a Linux. En el capítulo 16 analizamos interfaces gráficas de usuario comunes a algunas versiones de ambos sistemas operativos.



(Figura 13.13)

Interfaz de comandos en una computadora Macintosh que ejecuta MAC OS X, un sistema operativo basado en UNIX.

Archivos de script

Los archivos de comandos, a menudo denominados archivos shell o de script, pueden usarse para tareas que se repiten de manera automática. Cada línea del archivo es un comando válido y el archivo de script puede ejecutarse simplemente al escribir *sh* y el nombre del archivo de script. Otra forma de ejecutarlo consiste en definir el archivo como un comando ejecutable y escribir simplemente el nombre del archivo como caracter prompt del sistema.

Otros shells comunes son csh (el shell C, que fue desarrollado originalmente para atender entornos de desarrollo en lenguaje C en Berkeley y la NASA) y ksh (el shell Korn, que fue desarrollado después de los shells Bourne y C). El shell Korn suele estar disponible como producto que no forma parte de ningún paquete.

Los archivos de script se usan para tareas que se repiten de manera automática y para simplificar procedimientos complicados. A continuación se presenta un ejemplo de un archivo de script simple diseñado para configurar el sistema para cierto usuario:

```
setenv DBPATH /u/lumber:.:/zdlf/product/central/db
setenv TERMCAP $INFODIR/etc/termcap
stty erase '^H'
set savehistory
set history=20
alias h history
alias 4gen infogen -f
setenv PATH /usr/info/bin:/etc
```

En este ejemplo se fijan las rutas del directorio de trabajo, la historia se establece en 20 líneas y se le asigna el alias de "h" (de modo que el usuario pueda realizar el comando "history" al escribir simplemente h). En forma semejante, "4gen" se establece como un alias para el comando "infogen–f". Por último, la ruta se define como: /usr/info/bin:/etc.

Si este archivo de script se incluye en el archivo de configuración del usuario, se ejecuta de manera automática tantas veces como el usuario entre al sistema. El nombre exacto del archivo de configuración del usuario varía de un sistema a otro, aunque dos nombres comunes son .profile y .login. Consulte la documentación de su sistema para detalles específicos.

Redirección

Si usted es un usuario interactivo, la mayor parte de los comandos usados para producir resultados se envían de manera automática a su pantalla y el editor acepta entrada desde el teclado. Hay veces en que usted desea enviar resultados a un archivo o a otro dispositivo. Esto se hace usando el símbolo > entre los comandos y el destino al que deseen dirigirse los resultados. Por ejemplo, para enumerar los archivos en su directorio actual hacia el archivo denominado myfiles en lugar de enlistarlos en la pantalla, use este comando:

```
ls > myfiles
```

El siguiente comando copia el contenido de dos archivos, chapt1 seguido por chapt2, en un archivo denominado sectiona:

```
cat chapt1 chapt2 > sectiona
```

El comando cat es una abreviatura de "concatenar". Si `sectiona` es un nuevo archivo, entonces se crea de manera automática. Si ya existe, entonces en el contenido previo se sobreescribe. (Cuando cat se usa con un solo archivo y no se indica redirección, entonces exhibe el contenido de ese archivo en la pantalla). Otra forma de lograr el mismo resultado es con el símbolo "comodín" (*) como sigue:

```
cat chapt* > sectiona
```

El símbolo asterisco (*) indica que un comando aplica a todos los archivos que empiezan con "chap": en este caso, significa chapt1 y chapt2.

El símbolo >> añade el nuevo archivo a un archivo existente. En consecuencia, cualquiera de los dos comandos siguientes copia el contenido de chapt1 y chapt2 sobre el final de lo que ya exista en el archivo denominado sectiona.

```
cat chapt1 chapt2 >> sectiona
cat chapt* >> sectiona
```

Si sectiona no existe, entonces el archivo se crea como un archivo vacío y entonces se llena con chapt1 y chapt2, en ese orden.

La dirección inversa consiste en tomar datos de entrada de un programa de un archivo existente en lugar de hacerlo desde el teclado. Por ejemplo, si usted ha escrito un memo y debe enviarlo por correo a varias personas, el siguiente comando envía el contenido del archivo memo a las personas enumeradas entre el comando mail y el símbolo <.

```
mail ann roger < memo
```

Al combinar el poder de redireccionar con comandos del sistema, es posible lograr resultados que de otra forma no sería posible. Por ejemplo, el siguiente comando almacena en el archivo denominado temporary los nombres de todos los usuarios registrados en su sistema:

```
who > temporary
```

Y el comando sort ordena la lista almacenada temporalmente y exhibe la lista ordenada en la pantalla a medida que es generada.

En cada uno de estos ejemplos, es importante notar que la interpretación de < y > la hace el programa shell y no el programa individual (como mail, who o sort). Esto significa que la redirección de los datos de entrada y salida puede usarse con cualquier programa porque éste no está enterado de que algo extraordinario está ocurriendo. Esta es una instancia del poder de UNIX: la flexibilidad que proviene de la combinación de muchas operaciones en un solo comando breve.

Tuberías

Las tuberías y los filtros hacen posible redirigir datos de salida y entrada hacia archivo o dispositivos selectos con base en los comandos proporcionados al intérprete de comandos. UNIX hace lo anterior al manipular los dispositivos de E/S como archivos especiales.

Para el ejemplo que acaba de presentarse, el número de usuarios en línea se enumeran en un archivo denominado temporary y luego se ordena el archivo. Para crear este archivo, la única razón es el hecho de que se necesita para completar la operación de dos pasos requerida para ver la lista en orden alfabético en la pantalla. Sin embargo, una "tubería" puede hacer lo mismo en un solo paso.

Una tubería es la forma en que el sistema operativo conecta los resultados de un programa con los datos de entrada de otro sin necesidad de archivos temporales o intermedios. Una tubería es un tipo especial de archivo que conecta dos programas: información escrita en él por un programa y leída de inmediato por el otro, con programación, planificación y almacenamiento intermedio manipuladas por el sistema. En otras palabras, los programas se ejecutan de manera concurrente, uno después del otro. Al usar una tubería, indicado por el símbolo vertical (|), el último ejemplo puede volver a escribirse como:

```
who | sort
```

Como resultado, en la pantalla se exhibe una lista ordenada de todos los usuarios en el sistema.

Una tubería son varios programas que procesan de manera simultánea el mismo flujo de E/S. Por ejemplo, el siguiente comando es una tubería que toma los datos de salida de quién (una lista de todos los usuarios registrados), la ordena e imprime.

```
who | sort | lpr
```

Filtros

UNIX tiene muchos programas que leen algunos datos de salida, los manipulan de alguna manera y generan datos de salida; estos programas se denominan "filtros". Un ejemplo es *wc*, que significa "conteo de palabras" y cuenta las líneas, palabras y caracteres en un archivo. Por ejemplo, el siguiente comando ejecuta el comando contar palabras en el archivo journal y envía el resultado a la terminal.

```
wc journal
```

Como resultado, el sistema respondería con: 10 140 700, lo cual significa que el archivo del periódico tiene 10 líneas, 140 palabras y 700 caracteres. (Una "palabra" se define como una cadena de caracteres separada por espacios en blanco). Una versión más corta (wc -1) cuenta sólo el número de líneas en el archivo.

Otro comando filtro es *sort* (el mismo comando que usamos para mostrar las tuberías). Si un nombre de archivo se proporciona con el comando, el contenido del archivo se almacena y exhibe en la pantalla. Si ningún nombre de archivo se proporciona con el comando, entonces sort acepta datos de entrada desde el teclado y dirige los datos de salida hacia la pantalla. Cuando se usa con redirección, sort acepta datos de entrada desde un archivo y escribe los datos de salida en otro archivo. Por ejemplo,

```
sort names>sortednames
```

ordena el contenido del archivo denominado names y envía los datos de salida al archivo con los nombres ordenados. Los datos en los archivos se almacenan en orden ASCII; es decir, usando una secuencia adherente normal de modo que las líneas con espacios en blanco principales aparezcan primero (ya en orden), luego vayan las líneas con minúsculas, y las líneas que empiezan con caracteres en mayúsculas vayan al final. Para ordenar la lista en orden alfabético pero sin hacer caso del tipo de letra, el comando es:

```
sort -f names>sortednames
```

Para obtener un orden numérico en orden creciente, el comando es:

```
sort -n names>sortednums
```

Para obtener un orden numérico en orden decreciente, el comando es:

```
sort -nr names>sortednums
```

En todos los ejemplos presentados aquí, *sort* usa cada línea entera del archivo para conducir el ordenamiento. Sin embargo, si se conoce la estructura de los datos almacenados en el archivo, entonces el ordenamiento puede usar otros campos clave.

Por ejemplo, suponga que un archivo denominado empl tiene datos que siguen el mismo formato de columna: el número de identificación empieza en la columna 1, los números telefónicos empiezan en la columna 10, y los apellidos empiezan en la columna 20. Para ordenar el archivo por apellido (el tercer "campo"), el comando sería:

```
sort +2f empl>sortedempl
```

En este ejemplo, el archivo empl se ordena alfabéticamente por el tercer campo y el resultado se envía al archivo denominado sortedempl. (Un campo se define como un grupo de caracteres separados por al menos un espacio en blanco). El +2 indica al comando *sort* que omita los dos primeros campos y la f indica que la lista debe ordenarse en orden alfabético. La integridad del archivo se preserva porque toda la línea está ordenada: de modo que cada nombre mantiene el número telefónico y el número de identificación correctos.

Comandos adicionales

En esta sección se representan varios otros comandos de uso común.

man

Este comando exhibe el manual en línea proporcionado con el sistema operativo. Se llama con un argumento que especifica qué página en el manual en línea se quiere ver. Por ejemplo, para mostrar la página para el comando comparar (cmp), el comando se vería como sigue:

```
man cmp
```

Si la entrada cmp aparece más de una vez en el manual, se exhiben todas las páginas, una después de otra. Es posible redirigir los datos de salida a un archivo, que luego puede enviarse a una impresora para referencia futura.

grep

Uno de los comandos más usados es grep, que significa "expresión regular global e imprimir" y busca un patrón específico de caracteres. Es uno de los comandos más útiles (y extrañamente llamados). Es el equivalente de los comandos FIND y SEARCH usados en otros sistemas operativos. Una vez que se encuentra el patrón de caracteres, la línea que lo contiene se muestra en la pantalla.

He aquí un ejemplo: si se requiere recuperar los nombres y las direcciones de todas las personas con una dirección en Pittsburgh de un gran archivo denominado maillist, el comando se vería como sigue:

```
grep Pittsburgh maillist
```

Como resultado, en la pantalla se observan las líneas de maillist para entradas que incluyen Pittsburgh. Y si desea enviar los resultados a un archivo para su uso futuro, es posible agregar el comando redirección.

Este comando, grep, también puede usarse para enlistar todas las líneas que no contienen cierta cadena de caracteres. Usando el mismo ejemplo, el siguiente comando exhibe en la pantalla los nombres y las direcciones de todas las personas que no tienen una dirección en Pittsburgh:

```
grep -v Pittsburgh maillist
```

En forma semejante, el siguiente comando cuenta a todas las personas que viven en Pittsburgh y muestra el número en la pantalla sin imprimir cada línea:

```
grep -c Pittsburgh maillist
```

Como ya se anotó, el poder de este sistema operativo proviene de su capacidad para combinar comandos. A continuación se muestra la forma en que es posible combinar el comando grep con el comando who. Suponga que usted desea averiguar si su amigo Sam está en el sistema. El comando para mostrar el nombre, el dispositivo, la fecha y hora en que Sam se registró sería:

```
who | grep sam
```

La combinación de comandos, aunque es efectiva, puede parecer confusa para un observador ocasional. Por ejemplo, si se quiere tener una lista de todos los subdirectorios (pero no los archivos) encontrados en el directorio raíz, el comando sería:

```
ls -l / | grep '^d'
```

Este comando es la combinación de varios comandos más simples:

- ls para lista de directorios.
- -l, que es la opción larga de ls e incluye la información que se muestra en la tabla 13.3.
- / para indicar el directorio raíz.
- | para establecer una tubería.
- grep para encontrar
- '^d', que indica que d es el caracter que se está buscando (ya que sólo se quieren los directorios), la ^ indica que la d está al principio de cada línea (y las comillas se requieren porque se usó el símbolo ^).

nohup

Si se espera que la ejecución de un programa consuma mucho tiempo, es posible empezar su ejecución y después salir del sistema sin tener que esperar que termine la ejecución. Esto se hace con el comando nohup, que es una abreviatura de "no hangup". Suponga, por ejemplo, que se quiere una copia de un archivo muy grande pero no es posible esperar en la terminal hasta que finalice el trabajo. El comando es:

```
nohup cp oldlargefile newlargefile &
```

El comando copy (cp) continúa la ejecución de copiar *oldlargefile* en *newlargefile* en segundo plano incluso si usted no está en el sistema. Para este ejemplo, hemos indicado que la ejecución debe continuar en segundo plano; el ampersand (&) es el símbolo para ejecutar el programa en el modo "de segundo plano".

nice

Si su programa usa un gran número de recursos y usted no tiene prisa por obtener los resultados, puede ser "agradable" para otros procesos al disminuir su prioridad con el comando nice. Este comando con un ampersand al final libera a su terminal para otros trabajos. Por ejemplo, usted desea copiar `oldlargefile` en `newlargefile` y continuar trabajando en otro proyecto al mismo tiempo. Para hacer lo anterior, use este comando:

```
nice cp oldlargefile newlargefile &
```

Sin embargo, tal vez usted no registre su salida del sistema cuando use el comando nice sino hasta que la copia esté terminada porque fue necesario detener la ejecución del programa.

El comando nohup activa de manera inmediata a nice al disminuir la prioridad del proceso. Supone que puesto que usted ya salió del sistema, ya no tiene prisa por los resultados. Lo contrario no es cierto: cuando se emite nice, no activa de manera inmediata a nohup. En consecuencia, si usted quiere poner en segundo plano un trabajo muy largo, trabajar en algunos otros trabajos y salir del sistema antes que termine el trabajo, la combinación que debe usar es nohup con un ampersand & al final.

Aquí sólo hemos incluido algunos comandos. Para una lista completa de comandos para una versión específica de este sistema operativo, la sintaxis exacta y más detalles además de los que hemos analizado aquí, consulte en un manual técnico la versión idónea.

Cuando el ampersand "&" se agrega a un comando, ese comando se ejecuta y se obtiene el prompt del sistema de inmediato mientras el comando se ejecuta en modo de segundo plano. Esto le permite a usted la ejecución de otro comando sin esperar que termine la primera lista.

Conclusión

Es difícil medir el impacto que UNIX ha tenido en el mundo de la informática. Desde que fue escrito en 1969, ha sido una fuerza fundamental en el campo de los sistemas operativos y se espera que siga así durante muchos años.

UNIX sigue siendo un sistema operativo dirigido por comandos, escrito por programadores y para programadores quienes se sienten cómodos con este sistema operativo famoso por sus comandos breves, pero éste ha evolucionado hacia uno de los más significativos sistemas operativos disponibles. Con la adición de una interfaz gráfica para el usuario, UNIX se ha convertido en un sistema operativo muy flexible ejecutable en computadoras de cualquier tamaño y complejidad, desde teléfonos hasta supercomputadoras.

Desde que fue adoptado por Apple para sus computadoras Macintosh, UNIX ha ganado una nueva legión de usuarios que ciertamente crecerá en popularidad en el futuro.

Términos clave

Argumento: En un sistema operativo controlado por comandos, un valor u opción colocado en el comando, que modifica la manera en que debe efectuarse el comando.

Código compartible: Código ejecutable en el sistema operativo que puede compartirse por varios procesos.

Código reentrante: Código que puede usarse por dos o más procesos el mismo tiempo; cada uno comparte la misma copia del código ejecutable pero tiene áreas de datos por separado.

Controlador de dispositivos: Módulo de programa específico que manipula las interrupciones y controla un tipo particular de dispositivo.

Directorio: Unidad de almacenamiento lógico que contiene archivos.

Independiente de dispositivos: Programas que pueden funcionar en una variedad de computadoras y con una variedad de dispositivos.

Interfaz portable de sistema operativo para entornos informáticos (POSIX): Conjunto de normas IEEE que definen al usuario normal y las interfaces de programación para sistemas operativos, de modo que los desarrolladores pueden portar programas de un sistema operativo a otro.

Limitado por E/S: Trabajo que requiere un gran número de operaciones de entrada/salida, resultando en mucho más tiempo libre para la CPU.

Limitado por CPU: Trabajo que realiza una gran cantidad de procesamiento sin detenerse antes de emitir una interrupción. Un trabajo limitado por CPU puede estar atado durante largos periodos.

Núcleo: Parte del sistema operativo que reside en la memoria principal todo el tiempo y que realiza las tareas más esenciales, como administración de la memoria y manipulación de datos de entrada y salida al disco.

Proceso hijo: En sistemas operativos semejantes a los de UNIX, los procesos subordinados que son creados y controlados por un proceso padre.

Proceso padre: En sistemas operativos semejantes a los de UNIX, un trabajo que controla uno o más procesos hijo, creados por aquél.

Script: Serie de comandos ejecutables escritos en texto plano que pueden ejecutarse por el sistema operativo en secuencia como un solo procedimiento.

Búsquedas de interés

- UNIX para supercomputadoras
- Administración de archivos en UNIX
- UNIX embebido
- Historia de UNIX y Linux
- Macintosh OS X

Ejercicios

Temas de investigación

A. Investigue en la literatura actualizada para descubrir el estado actual de las normas IEEE POSIX y liste 10 sistemas operativo UNIX que sean compatibles al 100% con POSIX. Explique el significado de esta compatibilidad y por qué piensa que algunos de los sistemas operativos más populares pueden elegir no ser compatibles.

B Explore temas de seguridad alrededor de UNIX. Específicamente, identifique las mayores amenazas para sistemas que corren en UNIX y los pasos que los administradores de sistemas deben seguir para proteger al sistema de accesos no autorizados. Compare los problemas prácticos cuando se balancea la necesidad de accesibilidad con la necesidad de restricción de acceso y sugiera la primera acción que tomaría para asegurar la información de un sistema UNIX si usted administrara uno.

Ejercicios

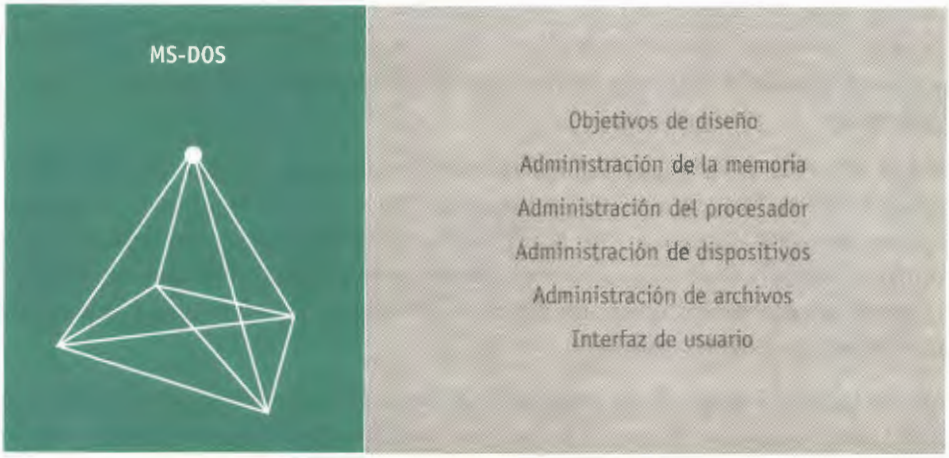
1. Describa la importancia de la independencia de hardware.
2. Las primeras versiones de UNIX estaban disponibles sólo una interfaz de comandos de control. En los años recientes, el uso de interfaces gráficas se volvió popular. Explique por qué estas GUIs causaron impacto en la popularidad de este sistema operativo.
3. El papel del núcleo UNIX es crucial. Explique por qué esto es así y cómo es tratado de manera diferente de otras partes del sistema operativo.
4. Explique cómo UNIX identifica la CPU (enlazada a E/S) y puestos de trabajo y utiliza esta información para mantener el sistema equilibrado.
5. UNIX trata todos los dispositivos como archivos. Explique por qué esto fue una característica innovadora cuando UNIX se introdujo por primera vez y cómo esto agrega flexibilidad a este sistema operativo.
6. En sus propias palabras, compare y contraste tres tipos de archivos: directorios, archivos ordinarios y archivos especiales. Explique qué puede suceder si un usuario modifica un archivo de directorios.

7. Genere una ilustración que muestre cómo un inodo puede seguir las direcciones del disco para un solo archivo de 1GB de tamaño.
8. Describa al menos una circunstancia en la que usted desearía ejecutar comandos en el modo de respaldo.
9. Explique por qué usted podría desear cambiar los permisos en algunos de sus directorios para hacerlos invisibles a otros usuarios.
10. Compare y contraste los dispositivos de bloque y caracteres y cómo éstos son manipulados por el administrador de dispositivos de UNIX.

Ejercicios avanzados

11. Describa cómo un proceso padre y un proceso hijo son creados desde el comando fork y por qué piensa que el proceso hijo puede o no continuar si el proceso padre está terminado.
12. Utilizando el comando /dev, identifique qué dispositivos están disponibles e indique cuáles de estos dispositivos son orientados a caracteres y cuáles son orientados a bloques. Explique su razonamiento.
13. Si usted usara un sistema y un operador del mismo borrara el comando listar (`ls`) accidentalmente, ¿se le ocurriría otra forma de para listar todos los archivos en un directorio?
14. En su sistema UNIX, identifique cuáles de los siguientes archivos de sistemas están soportados e identifique el formato que mantiene cada archivo de sistema: FAT32, NTFS, ISO 9660, HFS$_+$, UDF 2.5.
15. Compare y contraste el estado de la seguridad de UNIX en 1995 y en 2010 y describa algunas amenazas importantes para los sistemas ejecutables en UNIX en ese momento.

Capítulo 14 Sistema operativo MS-DOS



"Las computadoras personales proporcionaron al mundo una forma totalmente nueva de trabajar, jugar y comunicarse."

Bill Gates

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Cómo acceder a emuladores del MS-DOS desde otros sistemas operativos.
- La forma en que MS-DOS proporcionó una base para las primeras versiones de Microsoft Windows.
- Los fundamentos de los sistemas controlados por comandos y cómo construir archivos por lotes simples.
- Cómo varios procesos pueden compartir un procesador.
- Las limitaciones del MS-DOS.

MS-DOS, también conocido simplemente como DOS, fue desarrollado para su ejecución en computadoras de escritorio de usuario único e independiente. Cuando el mercado de computadoras personales explotó en la década de 1980, MS-DOS era el sistema operativo normal entregado con millones de estas máquinas.

Este sistema operativo es uno de los más sencillos de comprender. En muchas formas, MS-DOS ejemplifica los primeros sistemas operativos porque administra trabajos en forma secuencial desde un usuario único. Sus ventajas son su operación fundamental y sus comandos de usuario directos. Con sólo unas cuantas horas de instrucción, un usuario por primera vez puede aprender a manipular exitosamente una computadora personal y sus archivos y dispositivos.

MS-DOS Fue escrito para una sola familia de microprocesadores: los chips Intel 8086, 8088, 80186 y 80286. Cuando estos microprocesadores dominaron el mercado de las computadoras personales, también lo hizo MS-DOS. Cuando aparecieron nuevos chips, MS-DOS perdió su ventaja en el mercado sobre otros sistemas operativos más sofisticados. MS-DOS fue el primer sistema operativo para una generación de microcomputadoras, por lo que en este texto se incluye como un ejemplo.

Merece la pena observar que las primeras versiones de Windows (versiones 1.0 a 3.1) eran meras interfaces gráficas de usuario que se ejecutaban en la parte superior del sistema operativo MS-DOS. Consulte el capítulo 15 sobre Windows para más información.

No debe confundirse MS-DOS con software que tiene acrónimos semejantes, como "distributed operating systems" o un "disk operating system" comercializado por otros fabricantes como IBM.

Historia

MS-DOS fue el sucesor del sistema operativo CP/M. CP/M (de Control Program for Microcomputers) se ejecutaba en las primeras computadoras personales, máquinas de 8 bits comercializadas por Apple Computer y Tandy Corporation. Pero cuando las primeras computadoras personales de 16 bits fueron desarrolladas en 1980, requerían un sistema operativo con más capacidad que CP/M, y muchas compañías se apresuraron a desarrollar el sistema operativo que se volvería la norma para la nueva generación de hardware.

IBM fue el catalizador. Cuando investigaba un sistema operativo para su próxima versión a entregar de las computadoras personales de 16 bits, Digital Research ofreció el nuevo sistema operativo CP/M-86 y Softech ofreció su sistema P. IBM analizó con cuidado ambos sistemas y comenzó a negociar con Digital Research para adquirir el "nuevo y mejorado sistema CP/M". Mientras tanto, Microsoft, una naciente compañía iniciada por Bill Gates y Paul Allen, descubrió un sistema operativo innovador, denominado 86-DOS, diseñado por Tim Patterson de Seattle Computer Products para ejecutar esa línea de productos de computadoras personales de 16 bits. Microsoft lo compró, lo denominó MS-DOS que significa *Microsoft Disk Operating System*, y lo puso a disposición para IBM.

IBM escogió MS-DOS en 1981, lo denominó PC-DOS y lo proclamó la norma para su línea de computadoras personales. Finalmente, con el peso del endoso de IBM, MS-DOS se volvió el sistema operativo normal para la mayor parte de las computadoras personales en la década de 1980.

Este sistema operativo experimentó varias modificaciones después de su aparición en Seattle. Unas fueron necesarias para reparar algunas deficiencias, mientras otras se realizaron para permitir cambios fundamentales en el hardware, como capacidades aumentadas de la unidad de disco o formatos diferentes. En la tabla 14.1 se muestran algunas de las versiones más importantes.

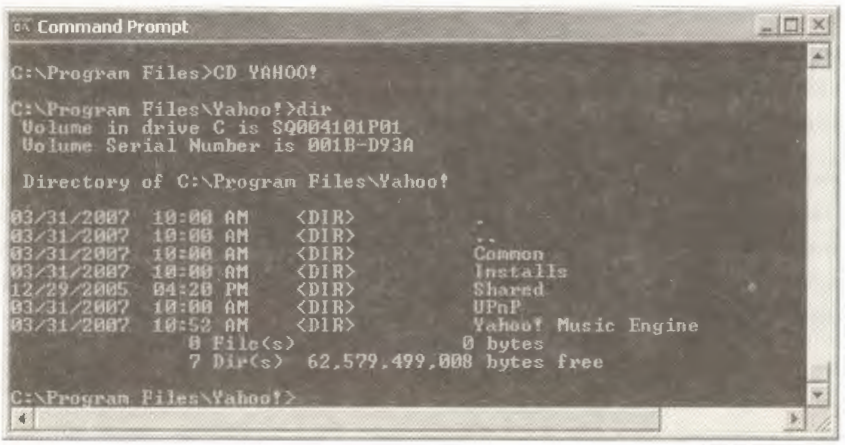
(Tabla 14.1)
Evolución de MS-DOS.

Versión no.	Fecha de lanzamiento	Características
1.0	1981	Compatible con CP/M, sólo atendía un directorio
1.1	1982	Permitía discos de 5¼ pulg con dos lados
2.0	1983	Eliminó algunos defectos de la primera versión
3.0	1984	Aumentó el requerimiento de memoria a 36K, atendía PC/AT
3.1	1984	Primera versión que atendía redes
3.2	1986	Atendía token ring y discos de 3½ pulg
3.3	1987	Atendía la computadora IBM PS/2
4.0	1988	Atendía grandes discos de más de 32 MB
5.0	1991	Mejor uso de la memoria extendida
6.0	1993	Mejor uso de la memoria convencional
6.22	1994	Proporciona a los usuarios capacidades previamente disponibles sólo como aplicaciones de un tercer partido

Aunque los emuladores de MS-DOS encontraron que Windows permite nombres de archivo superiores de 8 caracteres en los comandos auténticos de MS-DOS no.

Cada versión de MS-DOS era normal, de modo que versiones posteriores de éste fueron compatibles con las primeras. En consecuencia, los programas escritos para su ejecución en la versión 5.0 también podían ejecutarse en la versión 6.2. También significaba que, entre diferentes fabricantes, con los mismos comandos se obtenía la misma respuesta del sistema operativo sin importar quién había fabricado el hardware para ejecutarlo; esta característica era importante en esa época.

Aunque MS-DOS ya no se usa ampliamente, muchos sistemas operativos Windows ofrecen un emulador DOS para permitir que cualquier persona introduzca algunos comandos semejantes a los de DOS. Para encontrarlos en Windows, haga clic en el botón de inicio, luego todos los programas, accesorios, símbolos del sistema para ver una pantalla como la que se muestra en la figura 14.1.

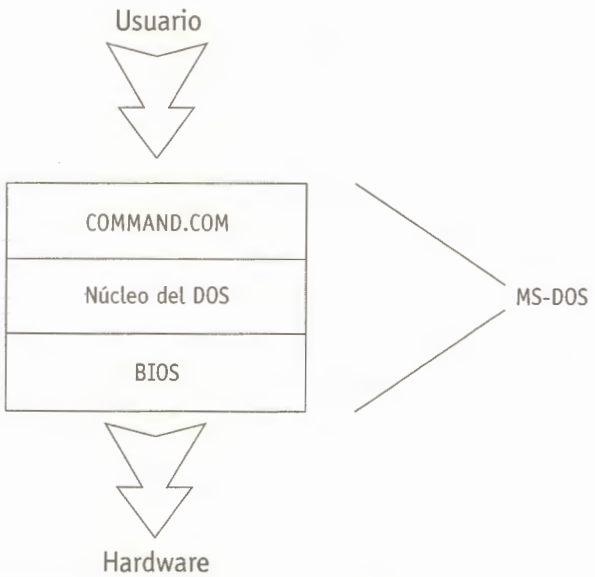


(Figura 14.1)

Ventana de símbolos del sistema abierta desde un sistema operativo Windows. A partir de los símbolos del sistema, los usuarios pueden introducir comandos MS-DOS.

Objetivos de diseño

MS-DOS fue diseñado para atender a un usuario novato único en un entorno de proceso único, como se muestra en la figura 14.2. Su soporte normal de E/S incluye teclado, monitor, impresora y unidad de almacenamiento secundario. Sus comandos de usuario están basados en palabras y frases del inglés y son indicativos de la acción a realizar. Estos comandos son interpretados por el procesador de comandos, en general la única porción del sistema operativo con la que interactúa la mayoría de los usuarios.



(Figura 14.2)

Las tres capas del MS-DOS. La capa superior es el procesador de comandos, un programa denominado COMMAND.COM, que proporciona independencia de dispositivo. El núcleo de software de DOS proporciona servicios de administración de archivos. El software BIOS proporciona servicios de administración de dispositivos.

El enfoque por capas es fundamental para el diseño de todo el sistema MS-DOS, que es "proteger" al usuario de tener que trabajar con los bits y los bytes de la máquina desnuda

que integran la capa inferior; el hardware que incluye los circuitos eléctricos, los registros y otros componentes básicos de la computadora. Cada capa está construida sobre la capa precedente, empezando de abajo arriba.

La capa en la parte inferior del MS-DOS es el **BIOS** (Basic Input/Output System). Esta capa del sistema operativo se interconecta directamente con los diversos dispositivos de E/S como impresoras, teclados y monitores. BIOS contiene los **controladores de dispositivos** que controlan el flujo de datos hacia y desde cada dispositivo excepto las unidades de disco. Recibe información de estado sobre el éxito o fracaso de cada operación de E/S y la pasa al procesador. BIOS se ocupa de las pequeñas diferencias entre unidades de E/S, de modo que el usuario puede comprar una impresora de cualquier fabricante sin tener que escribir un controlador de dispositivo para ella: BIOS hace que trabaje como debe hacerlo.

La capa de en medio, el núcleo del DOS, contiene las rutinas necesarias para conectarse con las unidades de disco. Se lee en la memoria en el momento de inicio desde el archivo MSDOS.SYS que reside en el disco de arranque. El núcleo del DOS es un programa registrado proporcionado por Microsoft Corporation que implementa MS-DOS. Al núcleo tienen acceso los programas de aplicación y el núcleo proporciona una colección de servicios independientes del hardware, como administración de la memoria y administración de archivos y registros. Éstas se denominan funciones del sistema. Como BIOS, el núcleo DOS compensa diferencias de un fabricante a otro, de modo que todas las unidades de disco funcionan del mismo modo. En otras palabras, el núcleo hace que la administración de archivos sea transparente para el usuario, de modo que no es necesario recordar en qué pistas y sectores están almacenados sus archivos; y qué sectores del disco están dañados y deben evitarse. El núcleo hace esto por el usuario; administra el almacenamiento y la recuperación de archivos y asigna y libera dinámicamente el almacenamiento secundario según sea necesario.

La tercera capa, el procesador de comandos, algunas veces se denomina shell. Esta es la parte del sistema que envía desplegados al usuario, acepta los comandos escritos, ejecuta los comandos y emite las respuestas idóneas. El procesador de comandos reside en un archivo denominado COMMAND.COM, que consta de dos partes almacenadas en dos secciones diferentes de la memoria principal. Algunos usuarios consideran de manera errónea que el archivo COMMAND.COM es todo el sistema operativo porque es la única parte que aparece en el directorio público. En realidad, es sólo uno de varios programas que integran el MS-DOS; los demás están ocultos.

El trabajo del procesador de comandos consiste en llevar a cabo los comandos del usuario introducidos desde el **signo del sistema** sin tener que esperar las instrucciones de algún controlador de dispositivos. Por ejemplo, cuando un usuario emite un comando PRINT, el procesador de comandos dirige el resultado hacia la línea de impresión mediante el BIOS. En forma semejante, cuando un usuario ordena ESCRIBIR un archivo, el procesador de comandos dirige el resultado hacia el monitor. En estos casos, el usuario no necesita compensar la baja velocidad de la impresora y la alta velocidad de la terminal; el usuario puede interactuar con dispositivos y con archivos de la misma manera.

La debilidad del procesador de comandos es que no es interpretativo. Es decir, los programadores no pueden tomar atajos para abreviar los comandos. Los nuevos usuarios deben aprender a introducir cada comando de manera completa y correcta. Es implacable para quienes no pueden escribir, deletrear o construir comandos perfectamente.

La versión 4.0 de MS-DOS introdujo un shell DOS controlado por menú para facilitar la interacción del usuario con el sistema, aunque no tuvo gran aceptación. Cuando se lanzó la versión 5.0, IBM y Microsoft también lanzaron un nuevo sistema operativo, OS/2, que fue diseñado para sustituir a MS-DOS. Aunque OS/2 tenía varias ventajas sobre MS-DOS, como usar toda la memoria disponible y atender multiprogramación, fracasó en generar el interés que esperaban ambas compañías.

En su apogeo, MS-DOS ejecutó una gran colección de paquetes de software. De las aplicaciones notables fueron Lotus 1-2-3 (un popular programa de cálculo en ese momento) y WordPerfect (un procesador de palabras). La adopción de la hoja de cálculo de estos dos productos ayudó a los usuarios novatos a apreciar la potencia de una computadora personal y fue un acicate para el crecimiento de la industria.

Microsoft ha continuado incorporando acceso a un emulador MS-DOS con un "símbolo del sistema DOS", una capacidad semejante a un comando DOS, sobre muchos de sus productos Windows (mostrado en la figura 14.1) aún cuando MS-DOS ya ha sido retirado oficialmente del mercado.

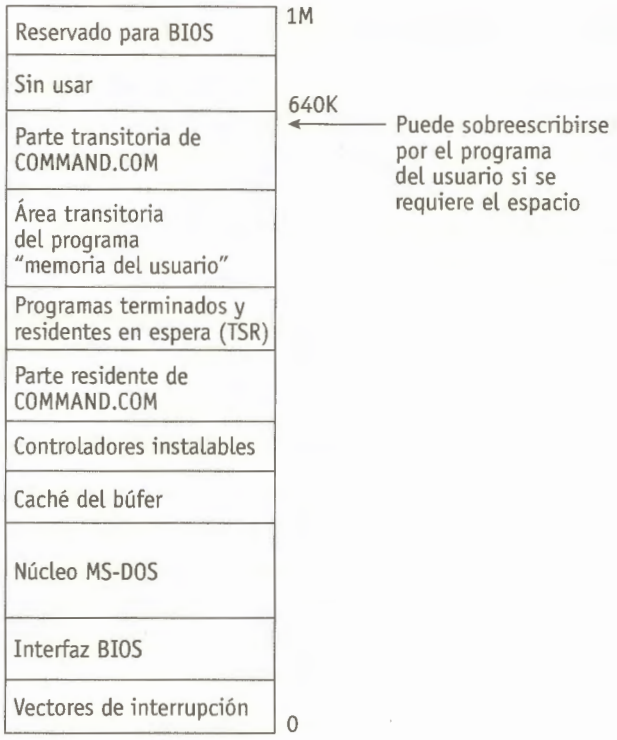
Administración de la memoria

El administrador de la memoria tiene un trabajo relativamente simple porque administra un solo trabajo para un usuario único. Para ejecutar un segundo trabajo, el usuario debe cerrar o hacer una pausa en el primer archivo antes de abrir el segundo. El administrador de la memoria usa un **esquema de asignación de memoria del primer ajuste**. El primer ajuste fue escogido para las primeras versiones de DOS porque es la estrategia más eficiente en un entorno de usuario único. (Algunas versiones permiten características de memoria extendida y **multitarea**, características disponibles con hardware y software que puede agregarse, aunque para que este análisis sea breve no los incluiremos aquí.)

Antes de ver cómo se asigna la memoria, veamos cómo está estructurada. La memoria principal viene en dos formas: memoria de sólo lectura (ROM) y memoria de acceso aleatorio (RAM).

La ROM suele ser de tamaño muy pequeño y contiene un programa, una sección de BIOS, con la única tarea de iniciar el sistema. El proceso de inicialización es llamado rutina de arranque debido a que efectivamente el sistema sale adelante por sí mismo por medio de sus rutinas de arranque. Este programa en ROM inicia la computadora. También recupera el resto de la porción residente del sistema operativo de almacenamiento secundario y la carga en RAM.

RAM es la parte de la memoria principal donde se cargan y ejecutan los programas. La disposición de la RAM para una computadora con 1 MB de memoria se muestra en la figura 14.3.



(Figura 14.3)

Un megabyte de memoria principal RAM en MS-DOS. Los vectores de interrupción están ubicados en la memoria baja direccionable y el COMMAND.COM está superpuesto en la memoria alta direccionable.

La parte inferior de la RAM: denominada memoria baja direccionable porque es donde empieza la memoria: 0, 1, 2, . . ., está ocupada por 256 vectores de interrupción y sus tablas. Un vector de interrupción especifica dónde está ubicado el programa **manejador de interrupciones** para un tipo de interrupción específico. El uso de manipuladores de interrupciones se analizó en el capítulo 4. Lo anterior es seguido por tablas BIOS y tablas DOS, y por el núcleo DOS con controladores adicionales instalables, en caso de haber alguno, que se especifican en el archivo de configuración del sistema denominado CONFIG.SYS. Esto es seguido por la parte residente del intérprete de comandos COMMAND.COM; la sección necesaria para ejecutar programas de aplicación.

Cualquier programa de aplicación del usuario puede cargarse ahora en el área transitoria de programas (TPA). Si un gran programa requiere más espacio, el área superpuesta de COMMAND.COM, localizada en la ubicación de la memoria con numeración alta, también puede usarse por el programa de aplicación. Los programas COMMAND.COM en esta área se consideran transitorios porque se usan para ejecutar comandos, como FORMAT, que no puede ejecutar el usuario cuando está ejecutándose un programa de aplicación, de modo que es posible que otros programas se superpongan (o sobreescriban).

Asignación de la memoria principal

Las primeras versiones de MS-DOS proporcionaban toda la memoria disponible al programa de aplicación residente, pero esto demostró ser insuficiente porque el esquema simple de asignación de memoria contigua no permitía que los programas de aplicación asignaran y liberaran de manera dinámica bloques de memoria. Con la versión 2.0, MS-DOS comenzó a atender asignación y modificación dinámica, y a liberar de la memoria principal bloques por medio de programas de aplicación.

La cantidad de memoria que realmente pertenece a cada programa de aplicación depende tanto del tipo de archivo como del programa que se cargue y del tamaño del área transitoria de la TPA.

Toda la TPA se proporciona a los programas con la **extensión** COM, la necesiten o no.

A los programas con la extensión EXE sólo se les proporciona la cantidad de memoria que necesitan. Estos archivos tienen un encabezado que indica la cantidad mínima y máxima de memoria necesarias para la ejecución de un programa. Idealmente, MS-DOS proporciona al programa la cantidad máxima de memoria solicitada. En caso de no ser posible, trata de satisfacer el requerimiento mínimo. Si el mínimo es más que la cantidad de espacio de memoria disponible, entonces no es posible ejecutar el programa.

Excepto por archivos COM, en la TPA puede haber cualquier número de archivos a la vez. Pero esto plantea una pregunta interesante: ¿Por qué un sistema puede tener dos programas en la memoria cuando sólo puede ejecutar uno? Respuesta: al contar con varios archivos en la memoria a la vez, un usuario puede abrir rápidamente uno, trabajar con él y cerrarlo antes de abrir el siguiente. No es posible abrirlos al mismo tiempo, pero al abrirlos y cerrarlos de manera alterna, el usuario puede usar ambos programas rápida y fácilmente.

Por ejemplo, un programa procesador de palabras podría permitir que un usuario exhibiera dos archivos en la pantalla al mismo tiempo al abrir un área de trabajo por separado para cada uno. Estas dos áreas de trabajo dividen en secciones la pantalla; en este ejemplo una mostraría el archivo activo y la otra el archivo dormido. Si el usuario indica cuál trabajo debe empezar en el segundo archivo (dormido), entonces el primer archivo (activo) se cierra rápidamente y se activa el segundo archivo.

He aquí un segundo ejemplo: suponga que su programa principal del procesador de palabras incluye el código requerido para componer e imprimir un documento, pero si usted desea verificar la ortografía, es necesario cargar en el disco el programa de revisión ortográfica. Una vez que se ha cargado, la porción principal del procesador de palabras se mantiene en la memoria y el segundo programa se agrega sin borrar nada del primero que ya estaba ahí. Ahora usted tiene dos programas en la memoria, pero sólo uno se ejecuta en un momento dado. Este hecho se analizará en la sección de administración de procesos después en este capítulo.

Si un programa que ya está en ejecución requiere más memoria, como para *búfers* adicionales de E/S, el administrador de la memoria verifica si queda suficiente memoria. En caso afirmativo, la asigna al programa mientras actualiza la tabla de asignación de bloques de

memoria para ese programa. En caso negativo, entonces se regresa un mensaje de error al usuario y el programa se detiene. Aunque la asignación inicial de memoria es manipulada de manera automática por los programas escritos en BASIC, Pascal o en cualquier lenguaje atendido por MS-DOS, la reducción y ampliación de la asignación de la memoria durante la ejecución puede hacerse sólo desde programas escritos en lenguaje ensamblador o en C.

Asignación de bloques de memoria

El administrador de la memoria asigna la memoria usando un algoritmo del primer ajuste y una lista enlazada de bloques de memoria. Pero con la versión 3.3 y versiones posteriores, es posible escoger la estrategia del primero o del último ajuste. Cuando se usa el último ajuste, DOS asigna el bloque de memoria direccionable más alto suficientemente grande para satisfacer la solicitud del programa.

El tamaño de un bloque puede variar desde tan pequeño como de 16 bytes (denominado "párrafo") hasta tan grande como la máxima memoria disponible. Cuando se forma un bloque, sus cinco primeros bytes contienen la información mostrada en la tabla 14.2.

(Tabla 14.2)
Los cinco primeros bytes de un bloque de memoria definen las características estructurales del bloque.

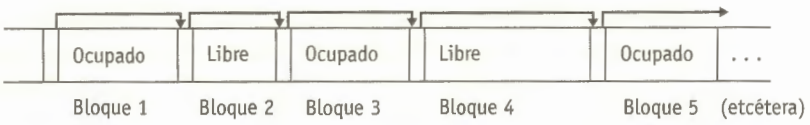
Byte	Contenido
Byte 0	ASCII 90h si es el último bloque, o ASCII 77H en caso de no serlo.
Bytes 1-2	Se incluye el número cero para indicar un bloque ocupado, y el apuntador hacia al prefijo del segmento del programa (PSP) que se crea durante la función EXEC cuando se carga el programa.
Bytes 3–4	Proporciona el número de párrafos contenido en el bloque.

En consecuencia, si un bloque contiene cuatro párrafos y es el primer bloque, entonces su código sería 7700000004h (como se explica en la tabla 14.3) La letra h que aparece al final del valor anterior es una notación hexadecimal y no está registrado.

(Tabla 14.3)
Bloque de memoria muestra con los cinco primeros bytes que contienen 7700000004h.

Byte	Contenido	Significado
Byte 0	77	Indica que este no es el último bloque.
Bytes 1-2	0000	Indica que este es un bloque ocupado y que su apuntador hacia la PSP es cero.
Bytes 3-4	0004	Indica que este bloque contiene cuatro párrafos.

Siempre que llega una solicitud por memoria, DOS busca en la lista de bloques libre/ocupado, como se muestra en la figura 14.4, hasta que encuentra un bloque libre que se ajusta a la solicitud. Si la lista de bloques se desconecta, se genera un mensaje de error y el sistema se detiene. Para recuperar el sistema, es necesario reiniciarlo.



(Figura 14.4)
La lista enlazada de bloques de memoria.

Un programa de aplicación bien diseñado libera el bloque de memoria que ya no necesita. Si dos bloques de memoria libres son contiguos, de inmediato se fusionan en un bloque y se enlazan a la lista. Un programa que no está bien diseñado, sin embargo, amontona sus bloques de memoria hasta que termina su ejecución; sólo entonces MS-DOS puede liberar los bloques de memoria usados por ese programa.

Administración del procesador

El administrador del procesador tiene la tarea relativamente fácil de asignar el procesador al trabajo residente cuando está listo para su ejecución.

Administración de procesos

MS-DOS no fue escrito en código reentrante, analizado en la sección sobre memoria virtual en el capítulo 3, porque fue diseñado para un entorno unitarea de usuario único. El código reentrante es la base de la multitarea, y MS-DOS no lo atiende; en consecuencia, los programas no pueden irrumpir en medio de una rutina interna DOS y luego volver a empezar la rutina a partir de cualquier parte.

En nuestro ejemplo del procesador de procesamiento/verificación de ortografía, el programa padre del procesador llamaba al programa hijo verificador de ortografía. El padre pasaba a dormir y así permanecía mientras se ejecutaba el hijo. No hay intercalamiento, por lo que no hay necesidad de que algoritmos o políticas sofisticadas determinen el siguiente trabajo a ejecutar o durante cuánto tiempo. Cada trabajo se ejecuta en segmentos completos y no se interrumpe a la mitad. En otras palabras, no hay necesidad de mantener una buena mezcla de trabajos para equilibrar la utilización del sistema.

Sin embargo, aunque no es posible ejecutar dos trabajos al mismo tiempo, algunos programas de software dan esa ilusión. Tanto Microsoft Windows como *SideKick* de Borland, por ejemplo, parecen interrumpir el programa padre, cambiar lo que aparece en pantalla, ejecutar programas no relacionados y luego regresar al padre; pero esto no es multitarea. (Multitarea es el sinónimo de la industria de microcomputadoras para multiprogramación.) Estos programas se ven y sienten como operaciones multitarea porque retienen su área de la memoria y ejecutan programas ejecutables, pero no están en el estado de ejecución al mismo tiempo. En cada caso el programa padre duerme mientras el hijo es ejecutado en sí. Esta sincronización es posible porque los manejadores de interrupciones integrados en MS-DOS proporcionan a los programadores la capacidad de guardar toda la información sobre su programa padre, lo cual permite su reinicio idóneo una vez que termina el programa hijo.

Manejadores de interrupciones

Los manejadores de interrupciones son una parte crucial de los sistemas. Podría decirse que son responsables de la sincronización de procesos. Una computadora personal cuenta con 256 interrupciones y manejadores de interrupciones, a los que se tiene acceso por medio de la tabla de vector de interrupciones residente en los bytes inferiores de la memoria, como se muestra en la figura 14.3. Las interrupciones pueden dividirse en tres grupos: interrupciones internas de hardware, interrupciones externas de hardware e interrupciones de software. Las interrupciones internas de hardware son generadas por ciertos eventos que ocurren durante la ejecución de un programa, como división entre cero. La asignación de tales eventos a números específicos de interrupciones se conecta electrónicamente hacia el procesador y no es modificable por instrucciones de software.

Las interrupciones externas de hardware son ocasionadas por controladores de dispositivos periféricos o por coprocesadores como el 8087/80287. La asignación de dispositivos externos a niveles de interrupciones específicos la hace el fabricante del sistema de cómputo o el fabricante del dispositivo periférico. El software no puede modificar estas asignaciones porque están conectadas directamente; implementadas como conexiones eléctricas físicas.

Las interrupciones de software son generadas por el sistema y por programas de aplicación. Tienen acceso a funciones del DOS y del BIOS, las que, a su vez, tienen acceso a los recursos del sistema.

Algunas interrupciones de software se usan para activar programas de aplicación especializados que asumen el control de la computadora. *SideKick* de Borland es uno de estos programas. Este tipo de manejador de interrupciones se denomina *Terminate* y *Stay Resident* (TSR). Su función consiste en terminar un proceso sin liberar su memoria, proporcionando así facilidades de programación residentes en la memoria. El TSR suele ser usado por bibliotecas de subrutinas que son llamadas una vez desde el nivel de comandos MS-DOS y luego son puestas a disposición para proporcionar servicios a otras aplicaciones a través de una interrupción de software. Cuando un TSR empieza a funcionar, establece sus tablas de la memoria y se prepara para su ejecución al conectarse a una interrupción DOS; cuando todo está listo, el programa determina cuánta memoria debe mantener. Luego, cuando el programa sale, se hace pasar un código de regreso al padre.

¿Cómo se sincronizan estas interrupciones? Cuando la CPU detecta una interrupción, hace dos cosas: 1) coloca en una pila el contenido del PSW (programa de estado de palabra), el segmento del código de registro y el registro del apuntador de instrucciones; y 2) deshabilita el sistema de interrupciones de modo que todas las otras interrupciones se apaguen hasta que se haya resuelto la interrupción actual. La CPU usa el número de 8 bits colocado sobre el bus del sistema por el dispositivo de interrupciones a fin de obtener la dirección del manejador de interrupciones idóneo de la tabla del vector de interrupciones y obtiene ejecución en esa dirección.

Finalmente, el manejador de interrupciones reactiva la interrupción del sistema para permitir que interrupciones de alta prioridad ocurran, salvando cualquier registro que éste necesite usar y procesando la interrupción lo más rápido posible.

Resulta evidente que este procedimiento es delicado. La sincronización de las actividades del TSR con funciones del DOS ya en progreso debe diseñarse e implementarse con cuidado para evitar modificar cosas que no deben modificarse o la caída del sistema.

Administración de dispositivos

La capacidad de reordenar solicitudes para optimizar el tiempo de espera y el tiempo de búsqueda no es una característica del MS-DOS porque está diseñado para un entorno de usuario único. Todas las solicitudes se manejan sobre la base de primero en llegar, primero en servirse. Pero, desde la versión 3.0, BIOS puede atender *spooling* al permitir que los usuarios planifiquen la impresión de varios archivos uno después de otro. Para hacer esto, BIOS transfiere de manera continua datos desde un *búfer* específico de la memoria hacia la impresora hasta que se vacía el *búfer*.

MS-DOS fue escrito para sistemas simples que usan teclado, monitor, impresora, ratón, uno o dos puertos en serie y tal vez una segunda impresora. Para almacenamiento, la mayor parte de los sistemas de cómputo personales usan dispositivos de almacenamiento con acceso directo, por lo general disquetes o discos duros. Algunos sistemas también atienden un sistema de archivamiento de cinta magnética con acceso secuencial. El administrador de dispositivos del MS-DOS puede trabajar con cualquiera.

Estos sistemas sólo usan un tipo de dispositivo de E/S para cada puerto, de modo que los canales de dispositivos no forman parte del MS-DOS. Y debido a que cada dispositivo cuenta con su propia unidad de control dedicada, los dispositivos no requieren administración especial del sistema operativo. En consecuencia, los manejadores de dispositivos son los únicos dispositivos necesarios para que el administrador de dispositivos haga funcionar el sistema. Un controlador de dispositivos es un módulo de software que controla un dispositivo de E/S aunque maneja sus interrupciones. Cada dispositivo tiene su propio controlador de dispositivos. BIOS es la porción del administrador de dispositivos que manipula el software del controlador de dispositivos.

El BIOS está almacenado en ROM y en RAM. En muchos sistemas MS-DOS, las partes más primitivas de los controladores de dispositivos están ubicadas en ROM, de modo que pueden usar las aplicaciones y diagnósticos independientes, así como el programa *bootstrapping* del sistema. Una segunda sección se carga desde el disco hacia la RAM y extiende las capacidades de las funciones básicas almacenadas en ROM, de modo que BIOS puede manejar todas las solicitudes de entrada y salida al sistema.

Normalmente, BIOS es proporcionado por el fabricante del sistema apegado a las especificaciones de Microsoft para MS-DOS y, puesto que es el enlace entre el hardware y DOS, usa núcleos normales de sistema operativo sin importar el hardware. Esto significa que los programas con las interfaces normales DOS y BIOS para sus funciones dependientes del sistema pueden usarse en toda máquina DOS sin importar el fabricante.

El BIOS responde a interrupciones generadas por hardware o software. Por ejemplo, una interrupción de hardware es generada cuando un usuario oprime la tecla *Print Screen*; lo que ocasiona que BIOS active una rutina que envía el contenido ASCII de la pantalla a la impresora.

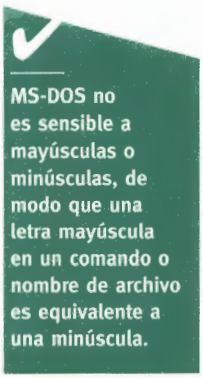
En forma semejante, una interrupción de software es generada cuando un programa emite un comando para leer algo desde un archivo de disco. Esto hace que la CPU indique a BIOS que active una rutina para leer datos desde el disco y le proporciona lo siguiente: el número de sectores a transferir, el número de pista, el número de sector y el número de unidad. Una vez que la operación ha sido completada con éxito, indica a BIOS el número de sectores transferidos y envía un comando para limpiar el código. En caso de que durante la operación ocurra un error, el código de error se regresa de modo que BIOS puede exhibir el mensaje de error idóneo en la pantalla.

La mayor parte de los controladores de dispositivos forman parte de un MS-DOS normal. Por supuesto, siempre es posible que usted escriba su propio controlador de dispositivos. Todo lo que se requiere son conocimientos del lenguaje ensamblador, información sobre el hardware y algo de paciencia. Esta opción puede ser necesaria si se está usando un sistema con una combinación inusual de dispositivos. Por ejemplo, en sus primeros años de disponibilidad comercial, no había una gran demanda para interconectar una computadora con un reproductor de discos, de modo que sus controladores de dispositivos no estaban incorporados en BIOS. En consecuencia, los usuarios que deseaban usar un disco como dispositivo de E/S debían escribir o comprar sus propios controladores de dispositivos y cargarlos cuando se arrancaba el sistema. Estos controladores de dispositivos se denominan instalables porque pueden incorporarse en el sistema operativo según sea necesario sin tener que parchar o modificar el sistema operativo existente. Los controladores de dispositivos instalables son una característica sobresaliente del diseño del MS-DOS.

Administración de archivos

MS-DOS atiende organizaciones de archivos secuenciales, directos e indexados secuenciales. Los archivos secuenciales pueden tener registros de longitud variable o fija. Sin embargo, los archivos directos y los indexados secuenciales sólo pueden tener registros de longitud fija.

Convenciones de nomenclatura de archivos



MS-DOS no es sensible a mayúsculas o minúsculas, de modo que una letra mayúscula en un comando o nombre de archivo es equivalente a una minúscula.

Un nombre de archivo no contiene espacios en blanco y consta de la identificación de la unidad, el directorio, cualquier subdirectorio, un nombre base y una extensión opcional. (DOS no es sensible a mayúsculas ó minúsculas, de modo que los nombres de archivo pueden escribirse con letras mayúsculas, minúsculas o una combinación de ambas.)

La identificación de la unidad (normalmente A, B, C o D), está seguida por dos puntos (:). Los directorios o los subdirectorios pueden tener entre uno y ocho caracteres de longitud y van precedidos por un backslash (\). El nombre de archivo base puede tener entre uno y ocho caracteres de longitud y la extensión, de uno a tres caracteres. El nombre base y las extensiones van separadas por un punto. Una extensión de archivo puede tener un significado especial para DOS; el usuario debe conocer las extensiones y sus usos.

Si en el nombre no se incluyen directorios o subdirectorios, se supone que el archivo está en el directorio de trabajo actual. En caso que no se designen unidades, se supone que el archivo

está en la unidad actual. El directorio raíz (consulte la siguiente sección, Administración de archivos, para un análisis de esto) se llama con un simple *backslash* (\). Los nombres de otros directorios van precedidos por el símbolo *backslash*, que también se usa como delimitador entre nombres.

Un nombre relativo de archivo consta de su nombre base y una extensión, en caso de usarse. Un nombre absoluto de archivo consta de su identificación de unidad y la ubicación de su directorio (denominada **ruta**) seguidas por su nombre relativo. Cuando un usuario está trabajando en un directorio o en un subdirectorio, el directorio en cuestión se denomina directorio de trabajo y a cualquier archivo en ese directorio puede tenerse acceso mediante su nombre relativo. Sin embargo, para tener acceso a un archivo que está en otro directorio se requiere el nombre absoluto.

Por ejemplo, si su directorio de trabajo incluye un archivo denominado JORDAN.DOC, entonces es posible identificar el archivo al escribir:

```
JORDAN.DOC
```

No obstante, si usted ha cambiado a otro directorio de trabajo, entonces tal vez deba incluir el nombre del directorio cuando llame al archivo:

```
\JOURNAL\CHAP9\JORDAN.DOC
```

Y si ha cambiado a otra unidad, como la unidad A, y desea llamar al archivo, quizá también deba incluir la identificación de la unidad:

```
C:\JOURNAL\CHAP9\JORDAN.DOC
```

Los nombres de archivos DOS no pueden tener espacios entre ellos. En consecuencia, para copiar un archivo de la unidad C a la unidad B, este es el comando:

```
COPY  C:\MEMO\DEAN.DOC  B:DEAN.DOC
```

Una forma más simple para tener acceso a archivos consiste en escoger primero un directorio y luego acceder a los archivos dentro de ese directorio mediante sus nombres relativos. Después, cuando haya terminado con un directorio, puede emitir el comando de cambio de directorio (consulte la tabla 14.4 más tarde en este capítulo) para dirigirse a otro directorio de trabajo.

Por supuesto, hay muchas variantes. Para detalles completos, consulte un manual técnico para su versión de MS-DOS.

Administración de archivos

Las primeras versiones de MS-DOS mantenían cada archivo en un solo directorio. Esto era lento y engorroso, especialmente a medida que los usuarios agregaban más y más archivos.

Para recuperar un solo archivo, el administrador de archivos buscaba desde el principio de la lista hasta encontrar el archivo o llegar al final de la lista. Si un usuario olvidaba el nombre del archivo, había muchas posibilidades de no volver a verlo jamás.

Para resolver este problema, Microsoft implementó una estructura jerárquica de directorios en la versión 2.0; una estructura de árbol invertida. (Es invertida porque la raíz está en la parte superior y las hojas, en la parte inferior.)

Cuando se formatea un disco (usando el comando FORMAT) sus pistas se dividen en sectores de 512 bytes cada uno. (Esto corresponde a un tamaño de *búfer* de 512 bytes). Los discos de un solo lado tiene una superficie grabable; los de dos lados tienen dos superficies grabables, y los discos duros tienen de dos a cuatro platos, cada uno con dos superficies grabables. El concepto de cilindros, presentado en el capítulo 7, es válido para estos discos duros porque las cabezas de lectura/escritura se mueven al unísono.

Los sectores (de dos a ocho) están agrupados en *clusters*, y esa es la manera en que el administrador de archivos asigna espacio a los archivos. Cuando un archivo requiere espacio adicional, DOS le asigna más *clusters*. Además de dividir el espacio del disco, FORMAT crea tres áreas especiales sobre el disco: el registro de arranque, el directorio raíz y la FAT, que significa **tabla de asignación de archivos**.

El *registro de arranque* es el primer sector de todo disco lógico, ya sea una unidad-entidad totalmente física (como un disquete o un disco duro) o sólo un disco virtual (como un disco RAM). Empezando con la versión 2.0, el registro de arranque contiene el programa del disco de arranque y una tabla con las características del disco.

El *directorio raíz* es donde el sistema empieza su interacción con el usuario cuando arranca. El directorio raíz contiene una lista de los subdirectorios y archivos primarios del sistema, incluyendo cualquier archivo de configuración generado por el sistema y cualquier instrucción de arranque generada por el usuario que pudiera estar incluida en un archivo AUTOEXEC.BAT. Se trata de un **archivo por lotes** que contiene una serie de comandos definidos por el usuario. Cada que se enciende o restablece la CPU, el sistema ejecuta de manera automática los comandos en este archivo. Después, en este capítulo se analiza un archivo muestra AUTOEXEC.BAT.

La información que contiene el directorio raíz es: 1) el nombre del archivo, 2) la extensión del archivo, 3) el tamaño del archivo en bytes, 4) la fecha y hora en que se hizo la última modificación al archivo, 5) el número del *cluster* inicial para el archivo y 6) los códigos de atributos del archivo. De estos seis elementos, los cuatro primeros se exhiben en respuesta al comando DIR, como se muestra en la figura 14.5.

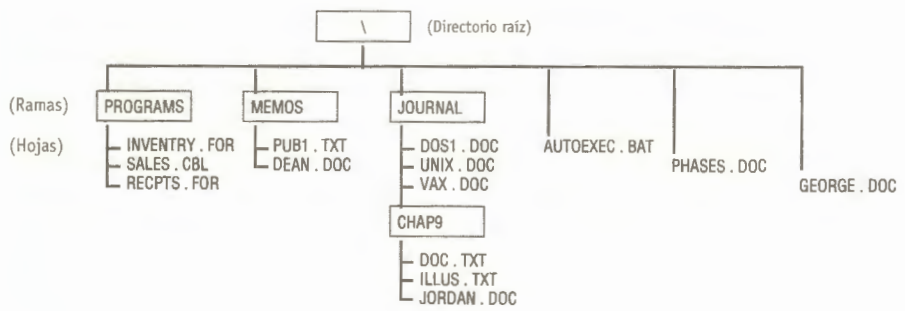
El número de entradas en el directorio raíz es fijo. Por ejemplo, para un disco duro de 20 MB sólo se permiten 512 entradas. El tamaño del directorio raíz es limitado porque DOS requiere saber dónde empieza el área de datos del disco. Empezando con la versión 2.0, los usuarios pueden evitar esta limitación al crear subdirectorios sin límite de tamaño.

(Figura 14.5)

Lista de un directorio DOCS que muestra tres subdirectorios y dos archivos.

La lista del directorio que se muestra en la figura 14.5 fue generada por el comando: DIR. Observe la manera en que los tres subdirectorios se distinguen de los dos archivos. Observe también que el sistema mantiene la fecha y la hora en que fue modificado la última vez. Algunos programas de seguridad del software usan estos datos para detectar cualquier virus o alguna modificación no autorizada o inusual del software del sistema.

Cada subdirectorio puede contener sus propios subdirectorios y/o archivos, como se muestra en la figura 14.6.



(Figura 14.6)

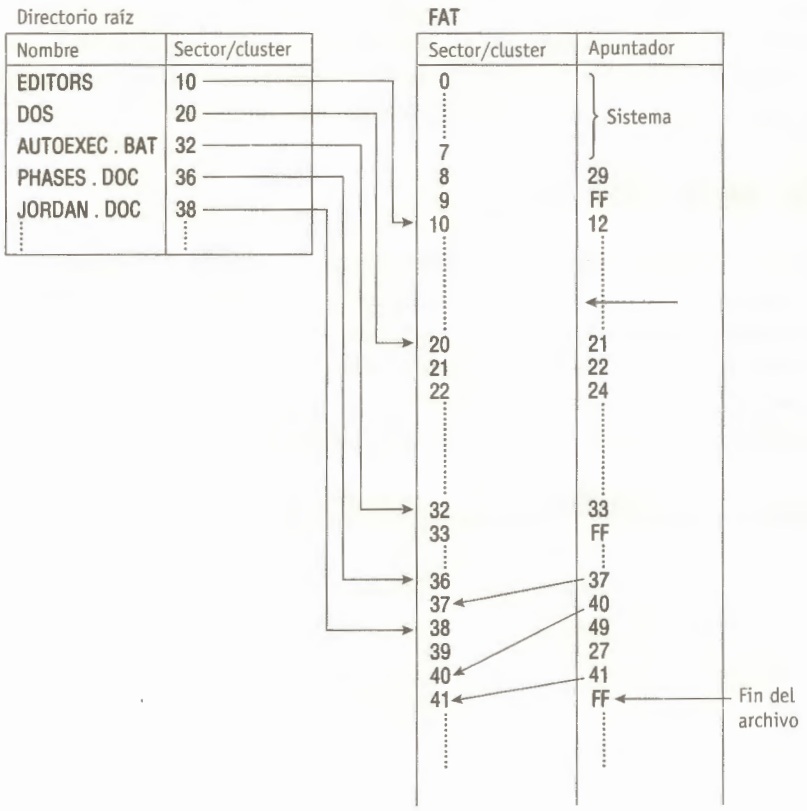
Sistema de directorios: la lista del directorio raíz tiene seis entradas. La lista del directorio para JOURNAL tiene cuatro entradas: sus tres archivos y su subdirectorio CHAP9. La lista del directorio para CHAP9 tiene tres entradas para sus tres archivos.

MS-DOS atiende archivos ocultos; archivos que son ejecutables aunque no se exhiben en respuesta a comandos DIR. Algunos archivos del sistema MS-DOS son ocultos; se usan para ejecutar el sistema operativo pero no aparecen en las listas de directorios. COMMAND. COM es el único archivo del sistema que no es oculto, de modo que siempre aparece en directorios públicos.

La tabla de asignación de archivos (FAT) contiene información crítica de estado, incluyendo la capacidad de cada sector para ser asignado o no, lo que puede ser el resultado de errores de formateo que interpretan al sector como no utilizable.

El directorio nota el número del primer sector o *cluster* del archivo; este número se registra en el directorio cuando se crea el archivo. Todos los sectores o *clusters* sucesivos asignados a

ese archivo se registran en la FAT, y se enlazan entre sí para formar una cadena, donde cada entrada de la FAT proporciona el número de sector/*cluster* a la siguiente entrada. La última entrada de cada cadena contiene el valor hexadecimal FF para indicar el final de la cadena. Como puede verse en la figura 14.7, los sectores de un archivo no tienen que ser contiguos.



(Figura 14.7)

Para cada archivo, el directorio incluye la ubicación del primer sector/cluster en la tabla de asignación de archivos, de modo que puede tenerse acceso rápido a éste. La FAT enlaza cada sector para cada archivo. Observe que los sectores para el archivo PHASES.DOC no son contiguos (las flechas son una ayuda visual para mostrar su enlace).

MS-DOS busca en los datos en un archivo del disco como una cadena continua de bytes. En consecuencia, las operaciones de E/S solicitan los datos por byte relativo (relativo con respecto al inicio del archivo) en vez de por sector relativo. La transformación de un sector físico (o *cluster*) en una dirección relativa de byte la efectúa el administrador de archivos de modo que parece que el acceso a los datos sobre un disco es justo como el acceso a los datos en la memoria principal.

Como ya mencionamos, MS-DOS atiende el almacenamiento de archivos no contiguo y asigna de manera dinámica espacio de disco a un archivo, en el supuesto de que en el disco hay suficiente espacio. Desafortunadamente, a medida que en el disco se agregan y borran archivos, un archivo puede volverse demasiado fragmentado, haciendo cada vez más engorrosa y consumidora de tiempo su recuperación.

La **compactación** se volvió una característica de la versión 6.0 de MS-DOS con la inclusión de DEFRAG.EXE, una utilidad usada para desfragmentar discos al escoger los fragmentos de un archivo y reposicionarlos como una sola pieza en espacio contiguo.

Para determinar la necesidad de compactación puede usarlas otro comando. Dado CHKDSK (nombre del archivo), el sistema responde con el número de bloques no contiguos donde está almacenado el archivo. Es decisión del usuario compactar al archivo, en caso de ser necesario, de modo que se almacena en el menor número de bloques no contiguos para acelerar el tiempo de acceso y reducir el mantenimiento sobre el mecanismo de búsqueda.

La habilidad para restringir el acceso del usuario al sistema de cómputo y sus recursos no está incluida en MS-DOS. Se cuenta con software de seguridad que puede agregarse aunque, para la mayoría de los usuarios, los datos se mantienen seguros al mantener la computadora cerrada físicamente o al retirar los discos y guardarlos en un sitio seguro.

Interfaz de usuario

MS-DOS usa una **interfaz controlada por comandos**. En la tabla 14.4 se muestran algunos de los comandos más comunes. Los usuarios escriben sus comandos en el símbolo del sistema. El símbolo del sistema por defecto es el indicador de la unidad (como C: y el caracter >; en consecuencia, C:> es el símbolo del sistema normal para un sistema con disco duro y A:> es el símbolo del sistema para una computadora con una unidad de disquete. El símbolo del sistema predeterminado puede cambiarse usando el comando PROMPT.

Comando	Significa	Acción a ejecutar
DIR	Directorio	Hacer una lista de lo que está en el directorio.
CD o CHDIR	Cambiar directorio	Cambiar el directorio de trabajo.
COPY	Copiar	Copiar un archivo. Agregar uno a otro.
DEL o ERASE	Borrar	Borrar el archivo o siguiente o los archivos siguientes.
RENAME	Renombrar	Renombrar un archivo.
TYPE	Escribir	Mostrar en la pantalla el texto del archivo.
PRINT	Imprimir	Imprimir uno o más archivos.
DATE	Fecha	Mostrar y/o cambiar la fecha del sistema.
TIME	Hora	Mostrar y/o cambiar la hora del sistema.
MD o MKDIR	Hacer directorio	Crear un nuevo directorio o subdirectorio.
FIND	Encontrar	Encontrar una cadena. Buscar en archivos una cadena.
FORMAT	Formatear disco	Preparar lógicamente un disco para almacenamiento de archivos.
CHKDSK	Comprobar disco	Comprobar disco respecto a estado de disco/archivo/directorio.
PROMPT	Símbolo del sistema del sistema	Cambiar el símbolo del sistema.
DEFRAG	Desfragmentar disco	Compactar archivos fragmentados.
(Nombre del archivo)		Ejecutar el archivo.

(Tabla 14.4)

Algunos comandos MS-DOS de usuarios comunes. Los comandos pueden escribirse en letras mayúsculas o minúsculas, aunque en este texto sólo usamos mayúsculas por consistencia en la notación. Consulte documentación técnica para ortografía y sintaxis correctas para su sistema.

Cuando el usuario oprime la tecla Enter, el shell denominado COMMAND.COM interpreta el comando y llama a la siguiente rutina de nivel inferior para satisfacer la solicitud.

Comandos de usuario incluyen algunos o todos los siguientes elementos en este orden:

```
command    source-file    destination-file    switches
```

El comando es cualquier comando MS-DOS legal. El archivo fuente y el archivo destino se incluyen cuando es válido y, dependiendo de la unidad y el directorio actuales, podría ser necesario incluir la ruta completa del archivo. Los cambios comienzan con un *slash*(es decir: /P/V/F) y son opcionales; proporcionan detalles específicos sobre cómo debe efectuarse el comando. La mayor parte de los comandos requieren un espacio entre cada uno de sus elementos.

Los comandos son ejecutados por el archivo COMMAND.COM, que forma parte de MS-DOS, como se muestra en la figura 14.2. Como ya mencionamos, cuando COMMAND.COM se carga durante el inicio del sistema, una de sus secciones se almacena en la sección baja de la memoria: esta es la porción residente del código. Contiene el intérprete de comandos y las rutinas necesarias para atender un programa activo. Además, contiene las rutinas necesarias para procesar CTRL-C, CTRL-BREAK y los errores críticos.

El código transitorio, la segunda sección de COMMAND.COM, se almacena en las direcciones más altas de la memoria y pueden sobreescribirse por programas de aplicación si necesitan usar su espacio de la memoria. Después, cuando termina el programa, la porción residente de COMMAND.COM verifica si el código transitorio sigue intacto. En caso negativo, carga una nueva copia.

A medida que un usuario escribe un comando, cada caracter se almacena en la memoria y aparece en la pantalla. Cuando se oprime la tecla Enter, el sistema operativo transfiere el control a la porción del intérprete de comandos de COMMAND.COM, que entonces tiene acceso a la rutina que lleva a cabo la solicitud o muestra un mensaje de error. Si la rutina reside en la memoria, entonces el control se le proporciona de manera directa. Si la rutina reside en el almacenamiento secundario, se carga en la memoria y después se le proporciona el control.

Aunque no es posible describir todos los comandos que hay en MS-DOS, merece la pena destacar algunas características para mostrar la flexibilidad de este sistema operativo.

Archivos por lotes

Al crear archivos por lotes personalizados, los usuarios pueden ejecutar rápidamente combinaciones de comandos DOS para configurar sus sistemas, realizar tareas de rutina, o facilitar que usuarios no técnicos puedan ejecutar software.

Por ejemplo, si un usuario verifica rutinariamente la fecha y hora del sistema, carga un controlador de dispositivos para un ratón, se desplaza a cierto subdirectorio y carga un pro-

grama denominado MAIL.COM, entonces este programa, denominado START.BAT, realiza cada uno de esos pasos a su vez como se muestra en la figura 14.8.

(Figura 14.8)
Contenido del programa START.BAT.

Para ejecutar este programa, el usuario sólo necesita escribir START en el símbolo del sistema. Para que este programa se ejecute automáticamente cada que se reinicia el sistema, entonces el archivo debe volverse a nombrar AUTOEXEC.BAT y cargarse en el directorio raíz del sistema. Al usar archivos por lotes, cualquier combinación tediosa de teclas puede reducirse a unos cuantos comandos personalizados fáciles de recordar.

Redirección

MS-DOS puede redirigir la salida desde un dispositivo de entrada o salida normal hacia otro. Por ejemplo, el comando DATE envía el resultado directamente a la pantalla pero al usar el símbolo de **redirección** (>), en lugar de lo anterior la salida se redirige a otro dispositivo o archivo.

La sintaxis es:

```
command > destination
```

Por ejemplo, si se desea enviar una lista del directorio a la impresora, debe escribirse DIR > PRN y la lista aparece en la página impresa, en lugar de en la pantalla. En forma semejante, si se desea que la unidad predeterminada sea redirigida a un archivo en el disco en la unidad B, debe escribirse DIR > B:DIRFILE y en la unidad B se crea un nuevo archivo denominado DIRFILE, que contiene una lista del directorio.

Es posible redirigir y agregar nuevos resultados hacia un archivo existente al usar el símbolo de agregar (>>). Por ejemplo, si ya se ha creado el archivo DIRFILE con el comando de redirección y se desea generar una lista del directorio y agregarla en el archivo DIRFILE creado previamente, debe escribirse:

```
DIR >> B:DIRFILE
```

Ahora DIRFILE contiene dos listas del mismo directorio.

La redirección también funciona de manera opuesta. Si se quiere modificar la fuente a un dispositivo o archivo específico, debe usarse el símbolo <. Por ejemplo, suponga que tiene en

desarrollo un programa denominado INVENTRY.EXE que espera datos de entrada desde el teclado, aunque para efectos de prueba y depuración se requiere que acepte datos de entrada desde un archivo de prueba de datos. En este caso debe escribirse lo siguiente:

```
INVENTRY < B:TEST.DAT
```

Filtros

Los **comandos filtro** aceptan datos de entrada desde el dispositivo predeterminado, manipulan los datos de alguna manera y envían los resultados al dispositivo de salida predeterminado. Un filtro de uso común es SORT, que acepta datos de entrada desde el teclado, ordena esos datos y los muestra en la pantalla. Este comando filtro se vuelve aún más útil al usar parámetros de redirección. Por ejemplo, si usted desea ordenar un archivo de datos denominado STD.DAT y almacenarlo en otro denominado SORTSTD.DAT, debe escribir:

```
SORT <STD.DAT> SORTSTD.DAT
```

El archivo almacenado puede estar en orden creciente (numérica o alfabéticamente) empezando con el primer caracter en cada línea del archivo. Si desea que el archivo esté almacenado en orden inverso, debe escribir:

```
SORT /R  <STD.DAT> SORTSTD.DAT
```

Es posible ordenar el archivo por columnas. Por ejemplo, suponga que un archivo denominado EMPL contiene datos que siguen este formato: los números de identificación empiezan en la columna 1, los números telefónicos empiezan en la columna 6, y los apellidos empiezan en la columna 14. (Una columna se define como caracteres delimitados por uno o más espacios.) Para ordenar el archivo por apellido, el comando sería:

```
SORT /+14  <EMPL.DAT> SORTEMPL.DAT
```

El archivo estaría ordenado en orden creciente por el campo empezando en la columna 14.

Otro filtro común es MORE, que hace que los resultados aparezcan en la pantalla en grupos de 24 líneas y espera hasta que el usuario oprime la tecla Enter antes de mostrar las 24 líneas siguientes.

Tuberías

Una **tubería** puede ocasionar que la salida normal de un comando sea utilizada como entrada normal para otro comando; este símbolo es una barra vertical (|). Es posible ordenar alfabéticamente el directorio y mostrar la lista ordenada en la pantalla al escribir:

```
DIR | SORT
```

También es posible combinar tuberías y otros filtros. Por ejemplo, para mostrar en la pantalla el contenido del archivo INVENTRY.DAT a la vez, el comando sería:

```
TYPE INVENTRY.DAT | MORE
```

El mismo resultado puede obtenerse usando sólo redirección al escribir:

```
MORE < INVENTRY.DAT
```

Es posible ordenar el archivo y mostrar el resultado en la pantalla a la vez al usar tuberías con este comando:

```
DIR | SORT | MORE
```

O bien, el mismo resultado puede obtenerse al usar tanto tuberías como filtros con estos dos comandos:

```
DIR | SORT > SORTFILE

MORE < SORTFILE
```

Comandos adicionales

Tres comandos adicionales que se usan a menudo en MS-DOS son FIND, PRINT y TREE. Observe que estos comandos son "tradicionales" de MS-DOS, y que algunos de los cambios u opciones mencionados aquí pueden no funcionar en emuladores semejantes a Windows DOS.

FIND

FIND es un comando filtro que busca una cadena específica en un archivo o archivos dados y despliega todas las líneas que contienen la cadena de esos archivos. La cadena debe estar entre comillas dobles ("") y debe escribirse exactamente como se busca; las letras mayúsculas y minúsculas se toman según se introducen.

Por ejemplo, el comando para mostrar todas las líneas en el archivo PAYROLL.COB que contienen la cadena AMNT-PAID es:

```
FIND "AMNT-PAID" PAYROLL.COB
```

El comando para contar el número de líneas en el archivo PAYROLL.COB que contiene la cadena AMNT-PAID y mostrar el número en la pantalla es:

```
FIND /C "AMNT-PAID" PAYROLL.COB
```

El comando para mostrar el número de línea relativo, así como la línea en el archivo PAYROLL.COB que contiene la cadena AMNT-PAID es:

```
FIND /N "AMNT-PAID" PAYROLL.COB
```

El comando para mostrar todas las líneas en el archivo PAYROLL.COB que *no contiene* la cadena AMNT-PAID es:

```
FIND /V "AMNT-PAID" PAYROLL.COB
```

El comando para mostrar los nombres de todos los archivos en el disco en la unidad B y que *no contienen* la cadena SYS es:

```
DIR B: | FIND /V "SYS"
```

PRINT

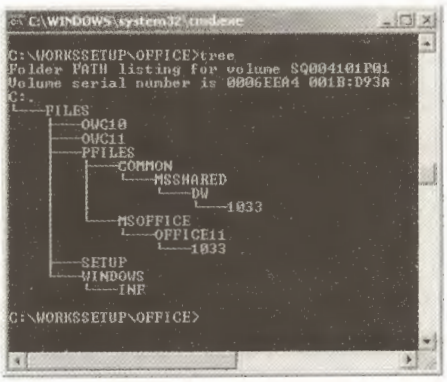
Este comando permite que el usuario establezca una serie de archivos para imprimir mientras libera al COMMAND.COM para aceptar otros comandos. En efecto, se trata de un *spooler*. A medida que la impresora imprime sus archivos, usted puede escribir otros comandos y trabajar en otras aplicaciones. El comando PRINT tiene muchas opciones, pero para usar las dos siguientes, éstas deben proporcionarse la primera vez que se usa el comando PRINT después de arrancar el sistema.

- El comando PRINT/B permite cambiar el tamaño del *búfer* interno. Por defecto, es de 512 bytes, pero aumentar su valor acelera el proceso PRINT.
- El comando PRINT/Q especifica el número de archivos permitidos en la siguiente cola de impresión. El valor mínimo para Q es 4 y el máximo, 32.

TREE

El comando TREE muestra los directorios y subdirectorios en una lista jerárquica y con sangría. También tiene opciones que permiten que el usuario borre archivos mientras se está generando el árbol. El desplegado empieza con el directorio actual o especificado con los subdirectorios con sangría bajo el directorio que los contiene. Por ejemplo, si se emite el comando TREE, la respuesta sería semejante a lo que se muestra en la figura 14.9.

(Figura 14.9)
Resultados muestra del comando TREE.



Para mostrar los nombres de los archivos en cada directorio, debe agregarse el conmutador /F:

```
TREE /F
```

El comando TREE también puede usarse para borrar un archivo que esté duplicado en diferentes directorios. Por ejemplo, para borrar el archivo PAYROLL.COB en cualquier parte del disco, el comando sería:

```
TREE PAYROLL.COB /D /Q
```

El sistema muestra el árbol como se acostumbra pero siempre que encuentra un archivo denominado PAYROLL.COB hace una pausa y pregunta si usted quiere que se borre. Si usted escribe Y, entonces borra el archivo y continúa. Si escribe N, entonces prosigue como antes.

Para efectos ilustrativos, aquí sólo hemos incluido unos cuantos comandos MS-DOS. Para una lista completa de comandos, su sintaxis exacta y más detalles sobre lo que se ha analizado aquí, consulte la página *www.microsoft.com*.

Conclusión

MS-DOS fue escrito para rendir servicio a los usuarios de computadoras personales en la década de 1980, incluyendo las primeras versiones de las computadoras personales de IBM. Como tal, tuvo éxito pero su flexibilidad limitada lo hicieron inutilizable a medida que evolucionaban las computadoras personales.

MS-DOS se recuerda como el primer sistema operativo normal que fue adoptado por muchos fabricantes de máquinas de cálculo personales. Como estándar también atendía, y es atendido, por legiones de grupos de diseño de software.

La debilidad de MS-DOS fue su diseño de sistema de usuario único de tarea única que no podía atender multitarea, redes y otras aplicaciones sofisticadas requeridas por computadoras de todos tamaños. Hoy es una reliquia del pasado, pero su estructura simple e interfaz de usuario lo hicieron una herramienta de aprendizaje accesible para estudiantes de sistemas operativos.

Términos clave

Archivo por lotes: Archivo que incluye una serie de comandos que se ejecutan en secuencia sin ningún dato de entrada de parte del usuario. Contrasta con una sesión interactiva.

Asignación de memoria del primer ajuste: Esquema de asignación de memoria que busca desde el principio del bloque de árbol y escoge para su asignación el primer bloque de memoria suficientemente grande para satisfacer la solicitud. Contrasta con la *asignación de memoria del mejor ajuste*.

BIOS: Acrónimo de *basic input output system* (sistema básico de entrada salida), un conjunto de programas codificados en un chip para cargarse en ROM al inicio.

***Bootstrapping*:** Proceso de arrancar una computadora inactiva al usar un pequeño programa de inicio para cargar otros programas.

Comando filtro: Comando que dirige los datos de entrada desde un dispositivo o archivo, los modifica y después envía el resultado a una impresora o una pantalla.

Compactación: Proceso de reunir fragmentos de espacio de memoria disponible en bloques contiguos al mover programas y datos en la memoria o en almacenamiento secundario de una computadora.

Controlador de dispositivos: Módulo de programa específico que maneja las interrupciones y controla un tipo particular de dispositivo.

Directorio de trabajo: Directorio o subdirectorio que actualmente es el que está usando el directorio principal.

Extensión: Parte del nombre del archivo que indica cuál compilador o paquete de software es necesario para ejecutar los archivos.

Interfaz controlada por comandos: Interfaz que acepta comandos por escrito, una línea a la vez, del usuario. Contrasta con una interfaz controlada por menú.

Manejador de interrupciones: Programa que controla la acción que debe emprender el sistema operativo cuando se interrumpe una secuencia de eventos.

Multitarea: Sinónimo de multiprogramación, una técnica que permite que un solo procesador procese varios programas que residen simultáneamente en la memoria principal e intercala su ejecución al superponer solicitudes de E/S con solicitudes de CPU.

Redirección: Instrucción que dirige al sistema operativo para enviar los resultados de un comando hacia o desde un archivo u otro dispositivo que no sean el teclado o el monitor. En MS-DOS, los símbolos de redirección son < y >.

Ruta: Secuencia de directorios y subdirectorios que deben seguir el sistema operativo para encontrar un archivo específico.

Símbolo del sistema del sistema: Señal desde el sistema operativo que está lista para aceptar un comando del usuario, como C:\> o C:\Documents>.

Tabla de asignación de archivos (FAT): Tabla usada para seguir la pista de segmentos de un archivo.

Tubería: Símbolo que dirige al sistema operativo para diversificar los resultados de un comando, de modo que se vuelvan los datos de entrada de otro comando. En MS-DOS, el símbolo de una tubería es |.

Búsquedas de interés

- Emulador MS-DOS
- Archivo por lotes autoejecutable
- Basic Input/Output System (BIOS)

- Interfaz de usuario controlada por comandos
- Sintaxis de comandos MS-DOS

Ejercicios

Temas de investigación

A. Explore el mundo de la computación al inicio de la década de 1980 e identifique varias razones de la popularidad de MS-DOS en ese momento. Haga una lista de los sistemas operativos competidores y las marcas de computadoras personales que estaban disponibles. Cite sus fuentes.

B. De acuerdo con *www.microsoft.com*, la empresa sigue manteniendo MS-DOS debido a que este sistema operativo está en uso en sitios alrededor del mundo. Dirija su propia investigación para encontrar un sitio en el que aún se esté ejecutando MS-DOS y explique por qué este es el sistema operativo elegido ahí.

Ejercicios

1. Describa el propósito de todas las interfaces de usuario, ya sea guiadas por comandos o por menús.
2. Mencione cinco ventajas que una interfaz de usuario guiada por comandos tiene sobre una interfaz de usuario guiada por menús.
3. ¿Cómo se construye un nombre de archivo legal en MS-DOS? Describa la longitud máxima y los roles de los caracteres especiales, mayúsculas/minúsculas, slashes, etcétera.
4. ¿Cómo se comparan los tamaños de los búfers de un sistema con los sectores de un disco? ¿Cuál es mayor? Explique por qué esto es así.
5. Proporcione ejemplos de los comandos CD, DIR y TREE y explique por qué decidiría usar cada uno.
6. Abra el emulador de MS-DOS de un sistema operativo Windows (quizá bajo el Menú de Accesorios y llamado "Command Prompt"). Cambie a un directorio con varios archivos y subdirectorios y ejecute un comando DIR. ¿Cómo está ordenado el directorio resultante (alfabética, cronológicamente, otra)?
7. Abra el emulador de MS-DOS de un sistema operativo Windows y haga un listado de directorios del directorio raíz (utilice el comando CD\ y luego el comando DIR). Después utilice el sistema operativo Windows y abra la carpeta C. Compare las dos listas y explique en qué son similares y en qué son diferentes.
8. Abra el emulador de MS-DOS de un sistema operativo Windows y ejecute el comando TREE. Explique si tener o no acceso a este comando MS-DOS podría ser valioso para un usuario de Windows.

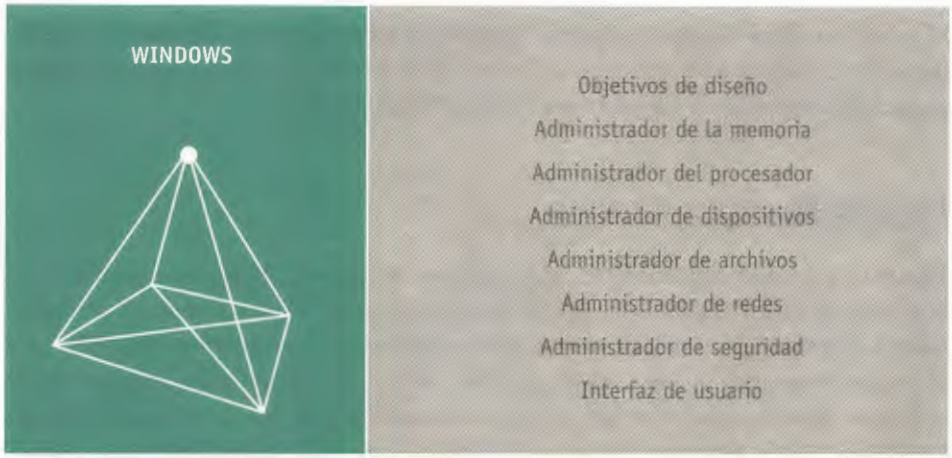
9. Describa el rol de una tabla de asignación de archivos (FAT) y cómo ésta administra los archivos.
10. ¿Cómo trabaja el directorio descrito en este capítulo comparado con el grupo de trabajo detallado en el capítulo 3?

Ejercicios avanzados

11. Si usted fuera a configurar una oficina pequeña con 10 computadoras personales ejecutando sólo MS-DOS, describa cómo podría conectarlas en red y cuántas copias del sistema operativo necesitaría comprar.
12. El comando FORMAT borra cualquier dato existente en el disco al formatearlo o reformatearlo. Describa qué características de seguridad tendría que agregar al sistema para prevenir el uso inadvertido de este comando.
13. Explique por qué es necesaria una rutina de arranque como elemento de todo sistema operativo.
14. Describa cómo MS-DOS ejecuta un arranque en frío y en qué orden este controlador de disco es ingresado. Explique por qué este orden hace o no sentido.
15. Describa cómo agregaría un control de acceso para proteger datos sensibles en una computadora al ejecutar MS-DOS. ¿Puede describir las soluciones de hardware y de software? Describa otras características que usted desearía agregar al sistema para hacerlo más seguro.
16. La rutina de arranque está almacenada en el primer sector del disco del sistema hasta que esta rutina puede ser el primer software cargado en la memoria cuando se enciende la computadora. Conduzca su propia investigación para descubrir los elementos esenciales de la rutina de arranque y explique por qué este software es necesario para "iniciar" el sistema.

Capítulo 15

Sistemas operativos Windows



"Windows tiene esta emocionante posición central, posición que es usada por miles y miles de compañías para elaborar sus productos."

Bill Gates

Objetivos de aprendizaje

Después de completar este capítulo, el lector debe ser capaz de describir:

- Los objetivos de diseño para los sistemas operativos de Windows.
- El rol de MS-DOS en las primeras versiones de Windows.
- El rol del administrador de la memoria y el administrador de la memoria virtual.
- El uso de los administradores de dispositivos, procesador y de red en versiones recientes de Windows.
- Los desafíos para la seguridad del sistema Windows.
- La interfaz de usuario de Windows.

Windows 95 fue el primer sistema operativo de perfil completo vendido por Microsoft Corporation y desde entonces cada versión ha sido un éxito financiero. El sistema operativo de Windows ahora está disponible para ambientes computacionales de todos los tamaños.

Desarrollo de Windows

El primer producto Windows usaba una **interfaz gráfica de usuario (GUI)** como su método primario de comunicación con el usuario y necesitaba un sistema operativo subyacente para traducir las solicitudes de los usuarios en comandos del sistema.

Primeros productos Windows

Windows 1.0, presentado en 1985, funcionaba en microcomputadoras compatibles con el sistema operativo MS-DOS. Es decir, la primera aplicación Windows no era un verdadero sistema operativo. Era simplemente una interfaz entre el sistema operativo MS-DOS real y el usuario. A pesar de que se trata de un producto sencillo (cuando se le compara con los complejos sistemas operativos actuales), éste era notable dado que fue la primer interfaz guiada por menús para computadoras de escritorio que eran compatibles con la computadora personal de IBM (PC).

Windows 1.0 fue seguido por GUIs, cada vez más complicadas diseñadas para trabajar computadoras de escritorio cada vez más poderosas, como se muestra en la tabla 15.1. El primer producto Windows ampliamente adoptado, Windows 3.1, presentaba un aspecto normal, semejante al popularizado por la computadora Macintosh de Apple. Windows 3.1 se volvió el producto de nivel de entrada para instalaciones encontradas en entornos de usuario único o de pequeños negocios.

Año	Producto	Características
1985	Windows 1.0	Primer embarque al menudeo del primer producto Windows; requería MS-DOS.
1990	Windows 3.0	Rendimiento mejorado y facilidad de uso avanzada; requería MS-DOS.
1992	Windows 3.1	Ampliamente adoptada, GUI comercialmente exitosa con más de 1 000 mejoras respecto a Windows 3.0; requería MS-DOS.
1992	Windows para grupos de trabajo	GUI para pequeñas redes; requería MS-DOS.

(Tabla 15.1)
Los primeros productos GUI Microsoft Windows se ejecutaban en "la parte superior del" MS-DOS.

Observe en la tabla 15.1 que Windows para grupos de trabajo fue el primer producto Windows en dar cabida a las necesidades de usuarios de red al incluir programas y características para redes de área local pequeñas. Por ejemplo, un Windows para el sistema de grupos de trabajo puede compartir fácilmente directorios, discos e impresoras entre varias máquinas interconectadas.

Cada producto Windows tiene un número de versión. Por ejemplo, Windows XP es la versión 5.1 y Windows Vista es la versión 6.0 y Windows 7 es la versión 6.1. Para encontrar el número de versión, oprima la tecla con el logo de Windows y la tecla R al mismo tiempo. Luego, escriba winver. Después oprima OK para ver el número de versión.

También permitía intercomunicación personal mediante programas de correo electrónico y de chat. Estaba dirigido a grupos pequeños o medianos de computadoras personales que suelen verse en negocios pequeños o pequeños departamentos de organizaciones más grandes.

Sistemas operativos para usuarios de hogar y profesionales

Antes del lanzamiento del sistema operativo Windows 95, todos los productos Windows estaban construidos para su ejecución en la parte superior del sistema operativo MS-DOS. Es decir, MS-DOS era el verdadero sistema operativo, pero tomaba su dirección del programa Windows que estaba ejecutándose en MS-DOS. Sin embargo, esta técnica por capas demostró ser una desventaja. Aunque ayudó a Windows a ganar cuota de mercado entre los usuarios de MS-DOS, contaba con poca seguridad integrada, no podía ejecutar multitareas y carecía de capacidad de comunicación entre procesos. Además, estaba escrito para trabajar estrechamente con el hardware de la microcomputadora, dificultando el movimiento del sistema operativo hacia otras plataformas.

Para responder a estas necesidades, Microsoft desarrolló y lanzó una sucesión de sistemas operativos Windows (no sólo GUI) para atraer usuarios de hogar y oficina, como se muestra en la tabla 15.2. (En la tabla 15.3 se muestra el desarrollo paralelo de productos de red.)

(Tabla 15.2)
Evolución de sistemas operativos Microsoft Windows clave para uso casero y profesional.

1995	Windows 95	Verdadero sistema operativo diseñado para sustituir a Windows 3.x, Windows para grupos de trabajo y MS-DOS para computadoras de escritorio de usuario único.
1998	Windows 98	Para usuarios de PC. Implementó muchas depuraciones para Windows 95; tenía apoyo de hardware más extendido y era completamente de 32 bits. No está relacionado directamente con Windows NT.
2000	Windows Millennium Edition (ME)	Último sistema operativo Windows construido sobre el código Windows 95.
2001	Windows XP Home	Para usuarios de PC. Sistema operativo de 32 bits construido para suceder a Windows 95 y 98; construido sobre el núcleo Windows NT.
2001	Windows XP Professional	Para usuarios de redes, construido con base en el núcleo Windows NT. La edición profesional estaba disponible en versiones de 32 y 64 bits.
2007	Windows Vista	Complejo sistema operativo con mejoras en el diagnóstico y herramientas de reparación.
2009	Windows 7	Disponible en seis versiones, la mayoría manejando 64 bits. Diseñado para manejar los defectos en la estabilidad y respuesta de Windows Vista.

Mientras Microsoft cortejaba el entorno de hogar y oficina con sistemas operativos de usuario único, la empresa también empezó a desarrollar productos de red más poderosos, empezando con Windows NT ("Nueva Tecnología"). A diferencia de los sistemas operativos de usuario único, Windows NT nunca dependió de MS-DOS en cuanto a soporte.

Sistemas operativos para redes

En otoño de 1988, Microsoft contrató a David Cutler para dirigir el desarrollo del sistema operativo Windows NT. Arquitecto experimentado de sistemas de minicomputadoras, Cutler identificó los requerimientos básicos de mercado para este nuevo producto: portabilidad, capacidades de multiprocesamiento, apoyo informático distribuido, cumplimiento con contratación pública y certificación gubernamental de seguridad. El producto terminado fue presentado en 1993, y evolucionó como se muestra en la tabla 15.3.

(Tabla 15.3)
Evolución de sistemas operativos de red Microsoft Windows. Todos evolucionaron a partir de Windows NT.

1993	Windows NT Advanced Server versión 3.1	Primera versión de NT; presentó el verdadero sistema operativo cliente/servidor con soporte para Intel, RISC y sistemas de multiprocesamiento.
1994	Windows NT Server versión 3.5	Presentó el paquete de aplicaciones BackOffice; requería 4 MB menos de RAM y ofrecía enlaces más estrechos para redes NetWare y UNIX a través de la pila mejorada TCP/IP.
1996	Windows NT Server versión 4.0	Agregó la interfaz conocida de Windows 95, incluía soporte para DCOM, y soporte integrado para correo electrónico y conectividad a internet.
1999	Windows 2000 Server	Introdujo servicios de directorio estilo X.500. Seguridad Kerberos y el modelo de objetos de componentes distribuidos (DCOM).
2003	Windows Server 2003	Disponible en Standard Edition, Web Edition, Enterprise Edition y Datacenter Edition, este sistema operativo fue diseñado como una plataforma servidora por iniciativa de Microsoft.NET.
2008	Windows Server 2008	Consumo reducido de energía, capacidad de visualización incrementada, soporta más de 64 núcleos.
2009	Windows Server 2008 R2	Actualizado para el sistema operativo Windows NT Server.

En 1999, Microsoft cambió el nombre del sistema operativo de Windows NT a Windows 2000, que también estaba disponible en cuatro paquetes: Windows 2000 Professional, Windows 2000 Server, Windows 2000 Advanced Server y Windows 2000 Datacenter Server. Datacenter Server era un nuevo producto diseñado para grandes almacenes de datos y otras aplicaciones de negocios con grandes cantidades de datos con soporte de hasta 64 GB de memoria física. En forma semejante, Windows Server 2003 también fue lanzado con estos mismos cuatro paquetes más una Web Edition.

Windows Server 2008 Versión 2 fue lanzado en 2009 para coincidir con el lanzamiento de Windows 7 y ofrecía soporte mejorado para múltiples núcleos, arriba de 64, consumo de energía reducido y un incremento en las capacidades de virtualización.

El resto de nuestro análisis de Windows se enfoca en las versiones de redes de este sistema operativo.

Objetivos de diseño

Para que el sistema operativo cumpliera sus requerimientos de mercado, desde el principio fue necesario integrar ciertas características como seguridad. En consecuencia, los diseñadores de Windows ensamblaron un conjunto de objetivos de diseño del software para facilitar la toma de decisiones a medida que evolucionaba el proceso de codificación. Por ejemplo, si dos opciones de diseño eran opuestas, se acudía a los objetivos de diseño para determinar cuál era la mejor.

Cuando se diseñaron los sistemas operativos de redes Windows, experimentaron la influencia de varios modelos de sistemas operativos, al usar marcos de referencia ya existentes a la vez que introducían nuevas características. Usan un modelo basado en objetos para administrar los recursos del sistema operativo y asignarlos a usuarios de manera consistente. Usa multiprocesamiento simétrico (SMP) para lograr rendimiento máximo de computadoras con multiprocesador.

Para dar cabida a varias necesidades de su comunidad de usuarios, así como para optimizar los recursos, el equipo de Windows identificó cinco objetivos de diseño: extensibilidad, portabilidad, confiabilidad, compatibilidad y rendimiento, objetivos que Microsoft cumplió con varios niveles de éxito.

Extensibilidad

Al saber que los sistemas operativos deben cambiar con el tiempo a fin de atender nuevos dispositivos de hardware o nuevas tecnologías de software, el equipo de diseño decidió que el sistema operativo debía mejorarse fácilmente. Esta característica se denomina **extensibilidad**. En un esfuerzo para asegurar la integridad del código del sistema operativo Windows, los diseñadores separaron las funciones del sistema operativo en dos grupos: un proceso ejecutivo privilegiado y un conjunto de procesos no privilegiados denominados "subsistemas protegidos". El término "privilegiado" se refiere a un modo de operación del procesador. La mayor parte de los procesadores tiene un modo privilegiado (donde se permiten todas las instrucciones de máquina y la memoria es accesible) y un modo no privilegiado (donde no se permiten ciertas instrucciones y la memoria del sistema no es accesible). En la terminología de Windows, el modo privilegiado del procesador se denomina modo de **núcleo** y el modo no privilegiado del procesador, **modo de usuario**.

Normalmente, los sistemas operativos se ejecutan únicamente en el modo de núcleo y un programa de aplicación se ejecuta en modo de usuario, excepto cuando llaman servicios del sistema operativo. En Windows, los subsistemas protegidos se ejecutan en modo de usuario como si fuesen aplicaciones, lo cual permite que los subsistemas protegidos puedan modificarse o agregarse sin afectar la integridad del proceso ejecutivo.

Además de los subsistemas protegidos, los diseñadores de Windows incluyeron cuatro características para asegurar la extensibilidad:

- Una estructura modular, de modo que al proceso ejecutivo pueden agregarse nuevos componentes.
- Un grupo de tipos de datos abstractos llamados objetos que son manipulados por un conjunto especial de servicios que permite la administración directa de los recursos del sistema.
- Una llamada de procedimiento remoto que permite que una aplicación llame servicios remotos sin importar su ubicación en la red.

Portabilidad

La **portabilidad** se refiere a la capacidad del sistema operativo para operar en máquinas diferentes que usan procesadores o configuraciones diferentes con una cantidad mínima de recodificación. Para alcanzar este objetivo, el equipo de desarrollo del sistema Windows siguió cuatro directrices. Primero, se escribió en un lenguaje estandarizado de alto nivel en todas las máquinas. Segundo, el sistema dio cabida al hardware al que se esperaba portar (32 bits, 64 bits, etc.). Tercero, el código que interactuaba directamente con el hardware debía minimizarse para reducir errores de incompatibilidad. Cuarto, todo el código dependiente del hardware debía aislarse en módulos que pudieran modificarse fácilmente cada que se portara el sistema operativo.

Windows está escrito para facilitar la portabilidad a máquinas que usan direcciones lineales de 32 o 64 bits y cuentan con capacidades de memoria virtual. Los sistemas operativos Windows han compartido las siguientes características:

- El código es modular. Es decir, el código que debe tener acceso a estructuras de datos y registros dependientes del procesador está contenido en pequeños módulos que pueden sustituirse por módulos semejantes para procesadores diferentes.
- Mucho de Windows está escrito en C, un lenguaje de programación estandarizado y fácilmente disponible. La componente gráfica y algunas porciones de la interfaz de red están escritas en C++. El código en lenguaje ensamblador (que no suele ser portable) se usa sólo para aquellas partes del sistema que deben comunicarse directamente con el hardware.
- Windows contiene una capa de abstracción (HAL), una biblioteca de enlace dinámico que proporciona aislamiento de dependencias del hardware proporcionadas por diferentes vendedores. HAL abstrae el hardware, como memorias caché, con una capa de software de bajo nivel de modo que no es necesario cambiar el código de nivel superior cuando se mueve de una plataforma a otra.

Confiabilidad

La **confiabilidad** se refiere a la fortaleza de un sistema; es decir, su previsibilidad de respuesta a condiciones de error, incluso los provocados por fallas en el hardware. También se refiere a la

capacidad del sistema operativo para protegerse a sí mismo y a sus usuarios de daño accidental o deliberado por programas del usuario.

La manipulación de excepciones estructurada constituye una forma para capturar condiciones de error y responderles de manera uniforme. Cada que ocurre un evento así, el sistema operativo o el procesador emite una llamada de excepción, que automáticamente invoca el código de manipulación de excepciones idóneo para manejar la condición, asegurando que no se produce ningún daño ni a los programas del usuario ni al sistema. Además, las siguientes características fortalecen al sistema:

- Un diseño modular que divide el ejecutivo en componentes individuales del sistema que interactúan entre sí a través de interfaces de programación específicas. Por ejemplo, si es necesario sustituir el administrador de la memoria por uno nuevo, entonces el nuevo usa las mismas interfaces.
- Un sistema de archivos denominado **NTFS** (**sistema de archivos NT**), que puede recuperarse de todos los tipos de error, incluyendo aquellos que ocurren en sectores críticos del disco. Para asegurar su recuperabilidad, NTFS usa almacenamiento redundante y un esquema basado en transacciones para almacenar datos.
- Una arquitectura de seguridad que proporciona una variedad de mecanismos de seguridad, como inicio de sesión de usuario, cuotas de recursos y protección de objetos.
- Una estrategia de memoria virtual que proporciona a todos los programas un gran conjunto de direcciones de memoria y evita que un usuario lea o modifique la memoria que está ocupada por otro usuario, a menos que ambos la compartan de manera explícita.

Compatibilidad

La **compatibilidad** suele referirse a la capacidad de un sistema operativo para ejecutar programas escritos para otros sistemas operativos o para versiones anteriores del mismo sistema. Sin embargo, para Windows, la compatibilidad es un tema más complicado.

Por medio del uso de subsistemas protegidos, Windows proporciona entornos de ejecución para aplicaciones que son diferentes a su interfaz de programación primaria; la interfaz de programación de aplicaciones (API) Win32. Cuando se ejecutan en procesadores Intel, los subsistemas protegidos proporcionan compatibilidad binaria con aplicaciones Microsoft existentes. Windows también proporciona compatibilidad a nivel de fuente con aplicaciones POSIX que se adhieren a las interfaces del sistema operativo POSIX definidas por IEEE. (**POSIX** es *Portable Operating System Interface*: Interfaz para sistemas operativos migrables basados en UNIX, una API del sistema operativo que define la forma de invocar servicios a través de un paquete de software. POSIX fue desarrollada por la IEEE a fin de incrementar la portabilidad del software de aplicación [IEEE, 2004].)

Además de compatibilidad con interfaces de programación, versiones recientes de Windows también atienden sistemas de archivos existentes, incluyendo la tabla de asignación de archivos MS-DOS (FAT), el sistema de archivos CD-ROM (CDFS) y el sistema de archivos NT (NTFS).

Windows contiene integrada la verificación de hardware y software importante. Es decir, los procedimientos de configuración de arranque incluyen un modo de verificación que examina el hardware y el software del sistema para ver si hay problemas potenciales y produce un reporte donde los enumera. El procedimiento se detiene cuando no puede encontrar controladores de dispositivos críticos, como controladores de disco duro, extensiones de bus y otros artículos que algunas veces son necesarios para una actualización exitosa.

Rendimiento

El sistema operativo debe responder rápidamente a aplicaciones limitadas por la CPU. Para lograr lo anterior, Windows ha incorporado las siguientes características:

- Llamadas al sistema, fallos de página y otros procesos cruciales se diseñan para responder de manera rápida.
- Un mecanismo denominado llamada de procedimientos locales (LPC) está incorporado al sistema operativo, de modo que la comunicación entre los subsistemas protegidos es rápida y no restringe el rendimiento.
- Elementos críticos del software de red de Windows se han incorporado en la porción privilegiada del sistema operativo para mejorar el rendimiento. Además, estas componentes pueden cargarse y descargarse del sistema en forma dinámica en caso de ser necesario.

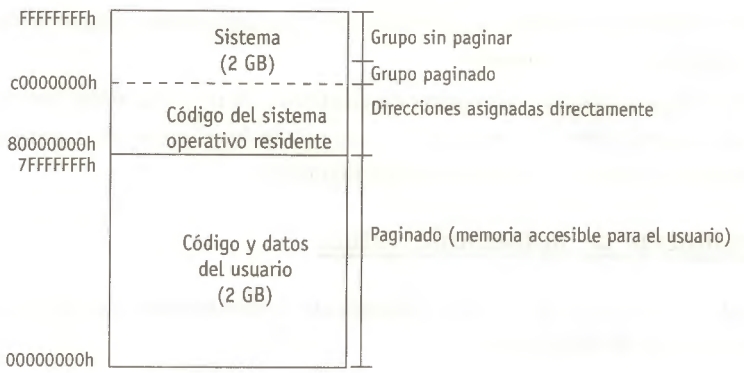
Es decir, la respuesta de algunos sistemas operativos de Windows es ralentizada según como fueron instaladas las aplicaciones y el equipo fue utilizado en el tiempo. Siempre y cuando estas aplicaciones sean desinstaladas, el rendimiento permanecerá lento y éste no regresará al punto de referencia de cuando la computadora era nueva.

Administración de la memoria

Todo sistema operativo usa su propia visión de la memoria física y requiere que sus programas de aplicación tengan acceso a la memoria de ciertas maneras. En el ejemplo que se muestra en la figura 15.1, cada espacio de dirección virtual del proceso es de 4 GB, donde 2 GB se han asignado a almacenamiento del programa y almacenamiento del sistema. Una vez que la memoria física se llena, el administrador de la memoria virtual pagina algo del contenido de la memoria del disco, liberando memoria física para otros procesos.

El desafío para todos los sistemas operativos Windows, especialmente los que se ejecutan en una red, es ejecutar programas de aplicación escritos para Windows o POSIX sin que ninguno provoque la caída del otro en la memoria. Cada uno de los subsistemas del entorno Windows proporciona una visión de la memoria que coincide con lo que esperan sus aplicaciones. El proceso de ejecución tiene su propia estructura de la memoria, a la que el subsistema tiene acceso al llamar los servicios inherentes del sistema operativo.

En versiones recientes de Windows, el sistema operativo reside en la memoria virtual alta y el código y los datos del usuario residen en la memoria virtual baja, como se muestra en la figu-



(Figura 15.1)

Disposición de la memoria Windows. Este es un sistema de memoria virtual basado en direcciones de 32 bits en un espacio de direcciones lineal. Las versiones de 64 bits usan un modelo similar pero en una escala más grande con 8TB pora el usuario y 8TB para el núcleo.

ra 15.1. Un proceso de usuario no puede leer o escribir directamente en la memoria del sistema. Toda la memoria accesible para el usuario puede paginarse hacia el disco, así como el segmento de memoria del sistema puede etiquetarse como *grupo paginado*. Sin embargo, el segmento de memoria del sistema etiquetado como *grupo sin paginar* nunca se pagina hacia el disco porque se usa para almacenar objetos críticos, como el código que hace la paginación, así como estructuras de datos fundamentales.

Características del modo de usuario

El administrador de memoria virtual (VM) permite que los subsistemas en modo de usuario compartan la memoria de manera eficiente y proporciona un conjunto de servicios nativos que un proceso puede usar para administrar su memoria virtual en las siguientes formas:

- Asignar memoria en dos etapas: primero al reservar memoria y luego comprometiendo memoria según sea necesario. Este procedimiento en dos pasos permite que un proceso reserve una gran sección de memoria virtual sin estar a cargo de ésta hasta que es realmente necesaria.
- Proporciona protección de lectura/escritura para la memoria virtual, lo cual permite que los procesos compartan memoria cuando sea necesario.
- Bloqueo de páginas virtuales en la memoria física. Esto asegura que una página crítica no sea eliminada de la memoria mientras un proceso la está usando. Por ejemplo, una aplicación de base de datos que usa una estructura de árbol para actualizar sus datos puede bloquear la raíz del árbol en la memoria, minimizando así los fallos de página mientras tiene acceso a la base de datos.
- Recuperar información sobre páginas virtuales.
- Proteger páginas virtuales. Cada conjunto de páginas tiene asociado un conjunto de marcas que determina los tipos de acceso permitidos en modo de usuario. Además, Windows proporciona protección de la memoria basada en objetos. En consecuencia, cada que un proceso abre un objeto sección, un bloque de memoria que puede ser compartido por

dos o más procesos, el monitor de referencia de seguridad verifica si el proceso está autorizado para tener acceso al objeto.

- Reescribir páginas virtuales en la memoria en el disco. Si una aplicación modifica una página, el administrador de la memoria virtual escribe los cambios de regreso en el archivo durante sus operaciones normales de paginación.

Implementación de la memoria virtual

El administrador de la memoria virtual depende de la administración del espacio de direcciones y de técnicas de paginación.

Administración del espacio de direcciones

Como se muestra en la figura 15.1, la mitad superior del espacio de direcciones sólo es accesible para procesos en modo de núcleo. El código en la parte inferior de esta sección, código y datos del núcleo, jamás se pagina fuera de la memoria. Además, las direcciones en este intervalo son traducidas por el hardware, proporcionando acceso a datos extremadamente rápido. En consecuencia, la parte inferior del código del sistema operativo residente es usada por secciones del núcleo que requieren rendimiento máximo, como el código que despacha unidades de ejecución, denominadas "hilos de ejecución", en un procesador.

Cuando los usuarios crean un nuevo proceso, pueden especificar que el administrador de memoria virtual inicie su espacio de direcciones virtual al duplicar el espacio de direcciones virtuales de otro proceso. Esto permite que los subsistemas del entorno presenten sus procesos cliente con las vistas de memoria que no corresponden al espacio de direcciones virtuales de un proceso nativo.

Paginación

El paginador es la parte del administrador de la memoria virtual que transfiere páginas entre marcos de páginas en la memoria y almacenamiento de disco. Como tal, es una combinación complicada de políticas de software y mecanismos de hardware. Las políticas de software determinan *cuándo* traer una página hacia la memoria y *dónde* colocarla. Los mecanismos de hardware incluyen la manera exacta en que el administrador de la memoria virtual traduce direcciones virtuales en direcciones físicas.

Debido a que las características del hardware de cada sistema afectan directamente el éxito del administrador de la memoria virtual, la implementación de la memoria virtual varía de un procesador a otro. En consecuencia, esta porción del sistema operativo no es portable y es necesario modificarla para cada plataforma de hardware nueva. Para una transición fácil, Windows mantiene pequeño y bien aislado este código. El chip del procesador que manipula la traducción de direcciones y el manejo de excepciones busca en cada dirección generada por un programa y la traduce en una dirección física. Si la página que contiene la dirección no está en la memoria, entonces el hardware genera un fallo de página y emite una llamada al paginador. El *búfer look-aside* de traducción (TLB) es un arreglo del hardware de memoria asociativa usado por el procesador para acelerar el acceso a la memoria. Como las páginas son llevadas dentro de la memoria por el administrador de la memoria virtual VM, crea entradas para ellas en el TLB. Si una dirección virtual no está en el TLB, aún puede

estar en la memoria. En ese caso, para encontrar la dirección virtual se usa software, en vez de hardware, lo cual resulta en tiempos de acceso más lentos.

Las políticas de paginación en un sistema de memoria virtual determinan *cómo* y *cuándo* se lleva a cabo la paginación y están compuestas por políticas de búsqueda, colocación y sustitución.

- La **política de búsqueda** determina cuándo el paginador copia una página del disco hacia la memoria. El administrador de la memoria virtual usa un algoritmo de paginación por demanda con localidad de referencia, denominada "clustering", para cargar páginas en la memoria. Esta estrategia trata de minimizar el número de fallos de página que encuentra un proceso.
- La **política de colocación** es el conjunto de reglas que determina dónde la página virtual se carga en la memoria. Si la memoria no está llena, el administrador de la memoria virtual escoge el primer marco de página de una lista de marcos de página libres. Esta lista se denomina base de datos del marco de página, y es un arreglo de entradas numeradas desde cero hasta $n \times 1$, donde n es igual al número de marcos de página de memoria en el sistema. Cada entrada contiene información sobre el marco de página correspondiente, que puede estar en uno de seis estados en cualquier momento dado: válido, cero, libre, en espera, modificado o malo. Los marcos de página válidos y modificados son los que se están usando en ese momento. Los estados cero, libre o en espera representan marcos de página disponibles; los marcos malos no pueden usarse.

  De los marcos de página disponibles, la base de datos del marco de página enlaza entre sí a aquellos que están en el mismo estado, creando así cinco listas homogéneas por separado. Siempre que el número de páginas cero, libre y en espera llegan a un mínimo preestablecido, el proceso de escritura de la página modificada se activa para escribir el contenido de las páginas modificadas hacia el disco y los enlaza con la lista en espera. Por otra parte, si la lista de páginas modificadas se vuelve demasiado corta, el administrador de la memoria virtual encoge el conjunto de trabajo de cada proceso a su tamaño mínimo de conjunto de trabajo y agrega las nuevas páginas liberadas a las listas modificadas o en espera para su reutilización.
- La **política de sustitución** determina cuál página virtual es necesario quitar de la memoria a fin de hacer espacio para una nueva página. De las políticas de sustitución consideradas en el capítulo 3, el administrador de la memoria virtual usa una política de sustitución FIFO local y sigue la pista de las páginas que actualmente están en la memoria para cada proceso: el conjunto de trabajo del proceso. El algoritmo FIFO es local para cada proceso, de modo que cuando ocurre un fallo de página, sólo pueden liberarse los marcos de página pertenecientes a un proceso. Cuando se crea cada proceso se le asigna un tamaño mínimo de conjunto de trabajo, que es el número de páginas que se garantiza que tiene el proceso en la memoria cuando se está ejecutando. Si la memoria está muy llena, el administrador de la memoria virtual permite que el proceso tenga las páginas que necesita hasta el máximo de su conjunto de trabajo. Si el proceso requiere aún más páginas, el administrador de la memoria virtual quita una de las páginas del proceso para cada nueva falla de página generada por el proceso.

Decimos que las páginas son "eliminadas" para indicar que estas páginas ya no están disponibles en la memoria. Sin embargo, estas páginas no son realmente eliminadas, pero se marcaron para borrar y después son remplazadas por los itinerarios de página.

Ciertas partes del administrador de la memoria virtual dependen del procesador que ejecuta el sistema operativo y debe modificarse para cada plataforma. Las características específicas de la plataforma incluyen entradas de la tabla de páginas, tamaño de página, protección basada en páginas y traducción de direcciones virtuales.

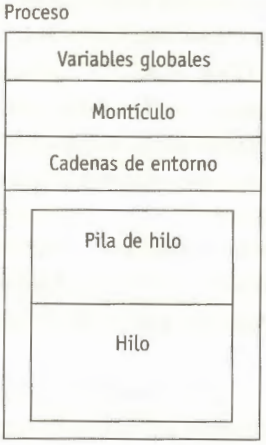
Administración del procesador

En general, un proceso es la combinación de un programa ejecutable, un área privada de la memoria y recursos del sistema asignados por el sistema operativo a medida que se ejecuta el programa. Sin embargo, un proceso requiere una cuarta componente antes de poder realizar cualquier trabajo: por lo menos un hilo de ejecución. Un hilo es la entidad dentro de un proceso que el núcleo planifica para su ejecución; puede igualarse aproximadamente a una tarea. El uso de múltiples hilos, también denominado multihilos, permite que un programador descomponga un proceso único en varios segmentos ejecutables y también aproveche la potencia extra de la CPU disponible en computadoras con múltiples procesadores. Windows Server 2008 Versión 2 puede coordinar el procesamiento entre 64 núcleos.

Windows es un sistema operativo multihilos multitarea preferente. Por defecto, un proceso contiene un hilo, que está compuesto por lo siguiente:

- Un identificador único.
- El contenido de un conjunto volátil de registros que indican el estado del procesador.
- Dos pilas usadas durante la ejecución del hilo.
- Un área de almacenamiento privada usada por subsistemas y bibliotecas de enlace dinámico.

Estas componentes se denominan contexto del hilo; los datos reales que forman este contexto varían de un procesador a otro. El núcleo planifica hilos para ejecución en un procesador. Por ejemplo, cuando se usa el ratón para hacer doble clic en un icono en el administrador de programas, el sistema operativo crea un proceso, y ese proceso tiene un hilo que ejecuta el código del icono. El proceso es como un contenedor para las variables globales, las cadenas de entorno, el montículo que pertenece a la aplicación, y el hilo. El hilo es lo que en realidad ejecuta el código. La figura 15.2 muestra un diagrama de un proceso con un hilo único.



(Figura 15.2)
Unitarea en Windows. Es así como un proceso con un hilo único es planificado para ejecución sobre un sistema con un procesador único.

Para sistemas con múltiples procesadores, un proceso puede tener tantos hilos como CPU haya disponible. La sobrecarga en que incurre un hilo es mínima. En algunos casos, resulta conveniente dividir una sola aplicación en multihilos porque entonces resulta mucho más fácil entender todo el programa. La creación de hilos no es tan complicada como parece. Aunque cada hilo tiene su propia pila, todos los hilos que pertenecen a un proceso comparten sus variables globales, montículo y cadenas de entorno, como se muestra en la figura 15.3.

(Figura 15.3)
Multitarea usando multihilos. Es así como un proceso con cuatro hilos puede planificarse para ejecución sobre un sistema con cuatro procesadores.

Los multihilos pueden presentar problemas porque es posible que varios hilos diferentes modifiquen las mismas variables globales independientemente entre sí. Para impedir esto, los sistemas operativos Windows incluyen mecanismos de sincronización que otorgan acceso exclusivo a variables globales a medida que se ejecutan estos procesos multihilos.

Por ejemplo, suponga que el usuario está modificando una aplicación de base de datos. Cuando el usuario introduce una serie de registros en la base de datos, el cursor cambia hacia una combinación de reloj de arena y apuntador en forma de flecha, indicando que ese hilo está escribiendo el último registro en el disco mientras otro hilo está aceptando nuevos datos. En consecuencia, incluso a medida que el procesamiento se está llevando a cabo, el usuario puede realizar otras tareas. El concepto de E/S superpuesta está ocurriendo ahora en el extremo del usuario, así como en el extremo de la computadora.

Multihilos es ventajoso cuando se realizan búsquedas en bases de datos porque los datos se recuperan más rápido cuando el sistema tiene varios hilos de ejecución que buscan un arreglo de manera simultánea, especialmente si cada hilo tiene su propia CPU. Los programas escritos para aprovechar estas características deben diseñarse con extremo cuidado para minimizar contención, como cuando dos CPU tratan de acceder a la misma ubicación de la memoria al mismo tiempo, o cuando dos hilos compiten por recursos únicos compartidos, como un disco duro.

Las aplicaciones cliente/servidor tienden a ser intensivas de la CPU para el servidor porque, aunque las consultas sobre la base de datos se reciben como solicitudes desde una computadora cliente, la consulta real es administrada por el procesador del servidor. Un ambiente de multiprocesamiento Windows, puede satisfacer estos requerimientos al asignar recursos adicionales de CPU.

Administración de dispositivos

El sistema de E/S debe dar cabida a las necesidades de dispositivos existentes; desde un simple ratón y teclado hasta impresoras, pantallas, unidades de disco, unidades CD-ROM, dispositivos multimedia y redes. Además, debe considerar tecnologías de almacenamiento y entrada futuras. El sistema de E/S proporciona una interfaz uniforme de alto nivel para operaciones de E/S a nivel de ejecución y elimina la necesidad de que las aplicaciones tomen en cuenta las diferencias entre dispositivos físicos. Protege al resto del sistema operativo de los detalles de manipulación de dispositivos, minimizando y aislando así el código dependiente de hardware.

El sistema de E/S en Windows está diseñado para proporcionar lo siguiente:

- Sistemas de archivos múltiples instalables incluyendo FAT, los sistemas de archivos CD-ROM y NTFS.
- Servicios para hacer el desarrollo dispositivo-controlador lo más fácil posible y trabajable sobre sistemas multiprocesamiento.
- Capacidad para que los administradores del sistema agreguen o quiten controladores del sistema en forma dinámica.
- Rápido procesamiento de E/S y permitir que los controladores puedan ser escritos en lenguaje de alto nivel.
- Capacidades de asignación de archivos de E/S para activación de imágenes, almacenamiento de archivos en memoria caché y uso de aplicaciones.

El sistema de E/S es controlado por paquetes. Es decir, toda solicitud de E/S es representada por un paquete de solicitud de E/S (IRP) a medida que se mueve de una componente del sistema de E/S a otra. UN IRP es una estructura de datos que controla la forma en que la operación de E/S se procesa en cada paso. El administrador de E/S crea un IRP que representa cada operación de E/S, pasa el IRP al controlador idóneo y dispone del paquete una vez que la operación está completa. Por otra parte, cuando un controlador recibe el IRP, lleva a cabo la operación especificada y luego lo envía de regreso al administrador de E/S o lo pasa a través de éste hacia otro dispositivo para un procesamiento adicional.

Además de crear y disponer de los IRP, el administrador de E/S proporciona código, común a diferentes controladores, que llama para efectuar su procesamiento de E/S. También administra *búfers* para solicitudes de E/S, proporciona soporte de tiempo de descanso para controladores y registra cuáles sistemas de archivos están cargados en el sistema operativo. Proporciona facilidades flexibles de E/S que permiten a los subsistemas como POSIX implementar sus respectivas interfaces de programación de aplicaciones. Finalmente, el administrador de E/S permite que los controladores de dispositivos y sistemas de archivos, que percibe como controladores de dispositivos, se carguen dinámicamente con base en las necesidades del usuario.

Para asegurarse que el sistema operativo trabaja con una amplia gama de periféricos de hardware, Windows proporciona un modelo independiente de dispositivos para servicios de E/S. Este modelo aprovecha un concepto denominado "controlador de dispositivos de varias capas" que no se encuentra en los sistemas operativos, como los controladores de dis-

positivos monolíticos MS-DOS. Estos controladores de varias capas cuentan con un conjunto de servicios grande y complicado que es comprendido por una capa intermedia del sistema operativo.

Cada controlador de dispositivos consta de un conjunto normal de rutinas, que incluyen lo siguiente:

- Rutina de inicio, que crea objetos del sistema usados por el administrador de E/S para reconocer y tener acceso al controlador.
- Expedición de rutina, que son funciones realizadas por el controlador, como READ o WRITE. Esto lo usa el administrador de E/S para comunicarse con el controlador cuando éste genera un IRP después de una solicitud de E/S.
- Rutina de inicio de E/S, usada por el controlador para iniciar la transferencia de datos hacia o desde un dispositivo.
- Rutina de terminación, usada para notificar a un controlador que un controlador de nivel inferior ha terminado de procesar un IRP.
- Rutina de descarga, que libera cualquier recurso del sistema usado por el controlador, de modo que el administrador de E/S puede eliminarlo de la memoria.
- Registro de errores de rutina, usado cuando ocurren errores de hardware inesperados como un sector dañado en un disco; la información se pasa al administrador de E/S, que escribe toda esta información en un archivo de registro de errores.

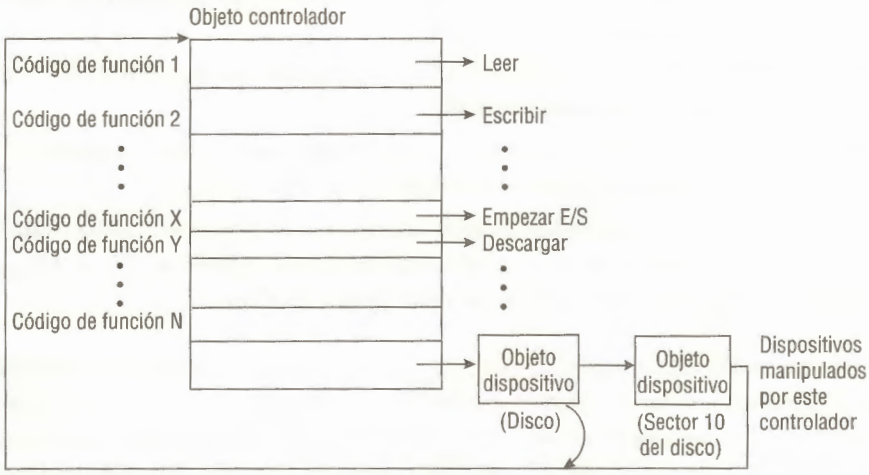
Cuando un proceso requiere tener acceso a un archivo, el administrador de E/S debe determinar a partir del nombre del archivo del objeto qué controlador llamar para procesar la solicitud, y debe poder ubicar esta información la próxima vez que un proceso use el mismo archivo. Esto se logra con un objeto controlador, que representa un controlador individual en el sistema y un objeto dispositivo, que representa un dispositivo físico, lógico o virtual en el sistema y describe sus características.

El administrador de E/S crea un objeto controlador cuando un controlador se carga en el sistema y después llama la rutina de inicio del controlador, que registra los puntos de entrada del controlador en el objeto controlador y crea un objeto dispositivo por cada dispositivo a manipular por este controlador. En la tabla 15.4 se presenta un ejemplo de cómo una instrucción de aplicación resulta en acceso al disco, y en la figura 15.4 se ilustra gráficamente el ejemplo.

(Tabla 15.4)
Ejemplo que muestra cómo se crea un objeto dispositivo a partir de una instrucción para leer un archivo. La instrucción real se traduce como se ilustra en la figura 15.4.

Evento	Resultado
Instrucción: READ "MYFILE.TXT"	READ = FUNCTION CODE 1 "MYFILE.TXT" = DISK SECTOR 10
Acciones:	1. Acceder a DRIVER OBJECT (1)
	2. Activar la rutina READ
	3. Acceder a DISK SECTOR 10

La figura 15.4 ilustra cómo el último objeto dispositivo apunta hacia su objeto controlador, indicando al administrador de E/S cuál rutina del controlador llamar cuando reciba una solicitud de E/S. Funciona como sigue: cuando un proceso solicita acceso a un archivo, usa un nombre de archivo, que incluye el objeto dispositivo donde está almacenado el archivo. Cuando se abre el archivo, el administrador de E/S crea un objeto archivo y luego regresa un identificador de archivo al proceso. Cada que el proceso usa el identificador de archivo, el administrador de E/S puede encontrar de inmediato el objeto dispositivo, que apunta al objeto controlador que representa el controlador que atiende al dispositivo. Al usar el código de función proporcionado en la solicitud original, el administrador de E/S indexa el objeto controlador y activa la rutina idónea porque cada código de función corresponde a un punto de entrada de la rutina del controlador.



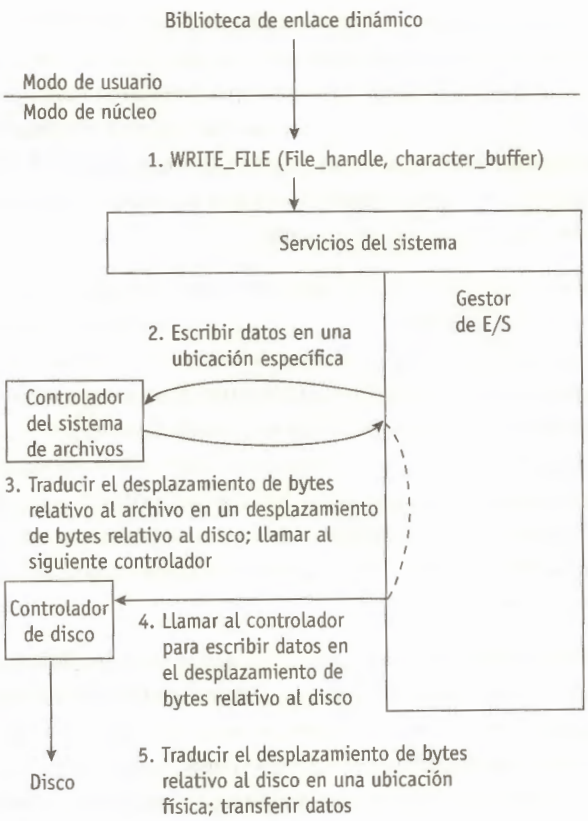
(Figura 15.4)

El objeto controlador de la tabla 15.4 está conectado a varios objetos dispositivo. El último objeto dispositivo apunta de regreso hacia el objeto controlador.

Un objeto controlador puede estar conectado a múltiples objetos dispositivos. La lista de objetos dispositivo representa sus dispositivos físico, lógico y virtual controlados por el controlador. Por ejemplo, cada sector de un disco duro tiene un objeto dispositivo con información específica del sector. Sin embargo, el mismo controlador de disco duro se usa para tener acceso a todos los sectores. Cuando un controlador se descarga del sistema, el administrador de E/S usa la cola de objetos dispositivo para determinar cuáles dispositivos serán afectados por la eliminación del controlador.

El uso de objetos para seguir la pista de información sobre controladores libera al administrador de E/S de tener que conocer los detalles sobre controladores individuales; simplemente sigue a un apuntador para ubicar a un controlador. Esto proporciona portabilidad y permite que nuevos controladores sean cargados fácilmente. Otra ventaja de representar dispositivos y controladores con objetos diferentes es que así es más fácil asignar controladores para controlar dispositivos adicionales o diferentes si cambia la configuración del sistema.

La figura 15.5 muestra cómo el administrador de E/S interactúa con un controlador de dispositivos por capas para escribir datos hacia un archivo en un disco duro al seguir en orden los pasos siguientes:



(Figura 15.5)

Detalles de las capas de un controlador de un sistema de archivos y un controlador de disco mostrados primero en la figura 15.4. Estos son los cinco pasos que se llevan a cabo cuando el administrador de E/S necesita tener acceso al dispositivo de almacenamiento secundario para satisfacer el comando del usuario mostrado aquí con el número 1.

1. Una aplicación emite un comando para escribir en un archivo de disco en un desplazamiento de bytes específico dentro del archivo.
2. El administrador de E/S pasa el identificador de archivo al controlador del sistema de archivos.
3. El administrador de E/S traduce el desplazamiento de bytes relativo al archivo en un desplazamiento de bytes relativo al disco y llama al siguiente controlador.
4. El código de función y el desplazamiento de bytes relativo al disco pasan al controlador de disco.
5. El desplazamiento de bytes relativo al disco se traduce en una ubicación física y se transfieren los datos.

El proceso es semejante al análisis en el capítulo 8 sobre los niveles en la administración del sistema de archivos.

El administrador de E/S ignora todo sobre el sistema de archivos. El proceso descrito en este ejemplo funciona exactamente como si un controlador NTFS se sustituye por un controlador FAT, un controlador del sistema de archivos UNIX, un controlador del sistema de archivos Macintosh o cualquier otro controlador.

Debe tenerse en mente que cuando el administrador de E/S pasa solicitudes por información de un lado a otro hay una sobrecarga implicada. Así, para dispositivos simples, como puertos de impresión en serie y en paralelo, el sistema operativo proporciona un enfoque de controlador de dispositivos de capa única donde el administrador de E/S puede comunicarse con el controlador de dispositivos, que, a su vez, regresa directamente la información. Pero para dispositivos más complicados como discos duros que dependen de un sistema de archivos, una mejor opción es un enfoque con varias capas.

Otra característica del controlador de dispositivos única en los sistemas operativos Windows recientes es que casi todas las operaciones de E/S de bajo nivel son asíncronas. Esto significa que cuando una aplicación emite una solicitud de E/S no tiene que esperar que los datos se transfieran, sino que puede continuar realizando otros trabajos mientras se lleva a cabo la transferencia de datos. Las operaciones de E/S asíncronas deben especificarse por el proceso cuando abre un identificador de archivo. Durante las operaciones asíncronas, el proceso debe tener cuidado en no acceder a ningún dato de la operación de E/S hasta que el controlador de dispositivos ha terminado la transferencia de datos. La E/S asíncrona es de utilidad para operaciones que requieren bastante tiempo para completarse o para las cuales el tiempo de completamiento es variable.

Por ejemplo, el tiempo necesario para enlistar los archivos en un directorio varía según el número de archivos. Puesto que Windows es un sistema operativo multitarea preferente que puede ejecutar muchas tareas al mismo tiempo, es vital que el sistema operativo no desperdicie tiempo esperando satisfacer una solicitud si puede ocuparse en hacer otra cosa. Las diversas capas en el sistema operativo usan multitarea y multihilo preferente para realizar más trabajo en el mismo tiempo.

Administración de archivos

Normalmente, un sistema operativo está asociado con la estructura de archivos particular que usa para dispositivos de almacenamiento masivo, como discos duros. En consecuencia, se habla de un sistema de archivos UNIX (inodos) o de un sistema de archivos MS-DOS (FAT). Aunque hay un sistema de archivos NT residente (NTFS), versiones actuales de Windows están diseñadas para ser independientes del sistema de archivos en los que operan.

El concepto primordial de manipulación de archivos en versiones recientes de Windows, que se introdujo primero en UNIX, es el archivo virtual, es decir, cualquier fuente o destino de E/S se trata como si fuera un archivo. En Windows, los programas realizan E/S en archivos virtuales, manipulándolos al usar identificadores de archivos. Aunque no es un concepto nuevo, en Windows un identificador de archivo en realidad se refiere a un objeto archivo ejecutivo que representa todos los recursos y destinos de E/S. Los procesos llaman a servicios de objetos archivo nativos como los que se requieren para leer o escribir en un archivo. El administrador de E/S dirige estas solicitudes de archivos virtuales a archivos reales, directorios de archivos, dispositivos físicos o a cualquier otro destino atendido por el sistema. Los objetos archivo poseen nombres jerárquicos, están protegidos por seguridad basada en objetos, atienden sincronización y son manipulados por servicios objeto.

Cuando un proceso abre un archivo, proporciona el nombre del archivo y el tipo de acceso requerido. Esta solicitud se mueve hacia un subsistema de entorno que a su vez llama un servicio del sistema. El administrador de objetos inicia la búsqueda de un nombre de objeto y envía el control hacia el administrador de E/S para encontrar el objeto archivo. El administrador de E/S verifica el subsistema de seguridad para determinar si otorga o no el acceso. El administrador de E/S también usa el objeto archivo para determinar si se han solicitado operaciones de E/S asíncronas.

La creación de objetos archivo ayuda a cerrar la brecha entre las características de los dispositivos físicos y las estructuras de directorio, las estructuras del sistema de archivos y los formatos de los datos. Los objetos archivo cuentan con una representación basada en la memoria de recursos físicos compartibles. Cuando se abre un archivo, el administrador de E/S regresa un identificador a un objeto archivo. El administrador de objetos trata los objetos archivo como a todos los otros objetos hasta que llega el instante de escribir hacia, o leer desde, un dispositivo, en cuyo momento el administrador de objetos llama al administrador de E/S por ayuda para tener acceso al dispositivo. La figura 15.6 ilustra el contenido de objetos archivo y los servicios que operan sobre ellos. La tabla 15.5 describe en detalle los atributos del cuerpo del objeto.

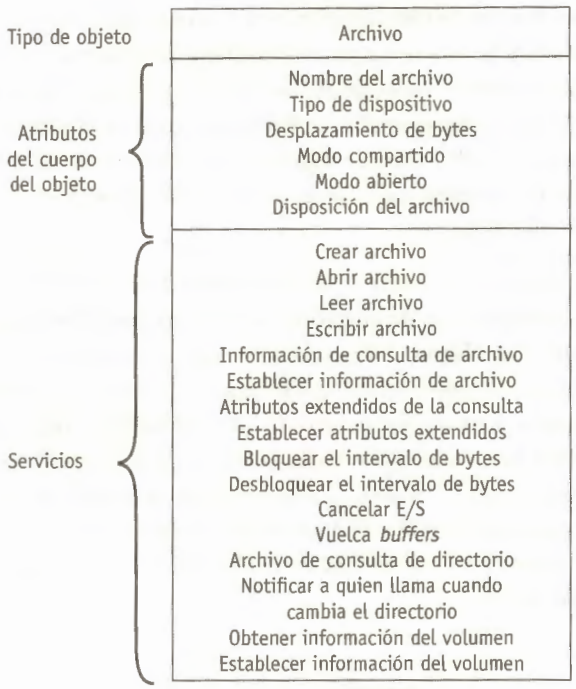
(Tabla 15.5)

Descripción de los atributos mostrados en la figura 15.6.

Atributo	Propósito
Nombre del archivo	Identifica el archivo físico al que se refiere el objeto archivo
Tipo de dispositivo	Indica el tipo de dispositivo en que reside el archivo
Desplazamiento de bytes	Identifica la ubicación actual en el archivo (válido sólo para E/S síncrona)
Modo compartido	Indica si otras llamadas pueden abrir el archivo para lectura, escritura o borrar operaciones mientras quien llama lo está usando
Modo abierto	Indica si la E/S es síncrona o asíncrona, está en memoria caché o no, si es secuencial o aleatoria, etcétera
Disposición del archivo	Indica si hay que borrar el archivo después de cerrarlo

Hagamos una distinción entre un objeto archivo, una representación basada en la memoria de un recurso compartible que contiene sólo datos únicos a un identificador de objetos, y el campo en sí, que contiene los datos a compartir. Cada que un proceso abre un identificador, se crea un nuevo objeto archivo con un nuevo conjunto de atributos específicos del identificador. Por ejemplo, el atributo desplazamiento de bytes se refiere a la ubicación en el archivo donde ocurrirá la siguiente operación READ o WRITE usando ese identificador. Puede ser de utilidad para considerar que los objetos atributo son específicos de un identificador único.

Aunque un identificador de archivo es único para un proceso, el recurso físico no lo es. En consecuencia, los procesos deben sincronizar su acceso a archivos, directorios y dispositivos compartibles. Por ejemplo, si un proceso está escribiendo sobre un archivo, debe especificar acceso exclusivo de escritura o bloquear porciones del archivo mientras escribe sobre ellas, a fin de impedir que otros procesos escriban en ese archivo al mismo tiempo.



(Figura 15.6)

Ilustración de un objeto archivo, sus atributos y los servicios que operan sobre él. Los atributos se explican en la tabla 15.5.

Un archivo asignado en memoria es una característica importante del sistema de E/S y se logra mediante la cooperación del sistema de E/S y el administrador de la memoria virtual. A nivel del sistema operativo, la asignación de archivos suele usarse para guardar archivos en caché, cargarlos y ejecutar programas ejecutables. El administrador de la memoria virtual permite que a los procesos de usuario se les asignen capacidades a través de servicios nativos. Los archivos asignados en memoria aprovechan las capacidades de la memoria virtual al permitir que una aplicación abra un archivo de tamaño arbitrario y lo trate como un arreglo contiguo único de ubicaciones en la memoria sin guardar datos en almacenamiento intermedio o realizar E/S de disco.

Por ejemplo, un archivo de 100 MB puede abrirse y tratarse como un arreglo en un sistema con sólo 20 MB de memoria. En cualquier momento, sólo una porción de los datos del archivo está presente físicamente en la memoria; el resto está paginado fuera del disco. Cuando la aplicación solicita datos que en ese momento no están almacenados en la memoria, el administrador de la memoria virtual usa su mecanismo de paginación para cargar la página correcta desde el archivo en disco. Cuando la aplicación escribe hacia su espacio de memoria virtual, el administrador de la memoria escribe los cambios de regreso en el archivo como parte de la paginación normal. Debido a que el administrador de la memoria virtual optimiza sus accesos al disco, las aplicaciones que son de la envolvente pueden acelerar su ejecución al usar asignación de E/S; escribir en la memoria es más rápido que hacerlo en un dispositivo de almacenamiento secundario.

Una componente del sistema de E/S llamada **administrador de caché** usa E/S asignada para administrar su caché basada en la memoria. El caché se agranda o reduce dinámicamente dependiendo de la cantidad de memoria disponible. Al usar estrategias normales del conjunto de trabajo, el administrador de la memoria virtual agranda el tamaño del caché cuando hay memoria disponible para dar cabida a las necesidades de la aplicación, y reduce su tamaño

cuando necesita páginas libres. El administrador del caché aprovecha el sistema de paginación del administrador de la memoria virtual, evitando la duplicación de esfuerzo.

El sistema de administración de archivos atiende nombres de archivo largos que pueden incluir espacios y caracteres especiales. En consecuencia, los usuarios pueden nombrar un archivo como "*Calificaciones de los estudiantes de primavera de 2005*" en lugar de algo críptico como *P05EST.CAL*. Debido a que el uso de nombres largos originaría problemas de compatibilidad con sistemas operativos más antiguos, el sistema de archivos convierte de manera automática nombres largos al nombre de archivo normal de ocho caracteres y la extensión de tres caracteres requeridos por MS-DOS y las aplicaciones Windows de 16 bits. El administrador de archivos hace lo anterior al mantener una tabla que enumera cada nombre largo de archivo y lo relaciona con el nombre corto del archivo correspondiente.

Administración de redes

En los sistemas operativos Windows, las redes forman parte integral del sistema operativo ejecutivo, proporcionando servicios como cuentas de usuario, seguridad de los recursos y mecanismos usados para implementar comunicación entre computadoras, como con tuberías nombradas y procesadores de mensajes. Las **tuberías nombradas** cuentan con una interfaz de alto nivel para pasar datos entre dos procesos sin importar sus ubicaciones. Los **procesadores de mensajes** cuentan con mecanismos de comunicación uno a muchos y muchos a uno útiles para transmitir mensajes a cualquier número de procesos.

Para ver estadísticos del servidor, oprima juntas la tecla con el logo de Windows y la letra *R*. Luego, escriba CMD **para abrir la ventana de comandos.** Luego escriba net statistics server **y oprima la tecla Enter. Para ver estadísticos de la estación de trabajo, desde la ventana de** comandos escriba net statistics workstation.

Microsoft Networks, informalmente conocido como "MS-NET", se volvió el modelo para el administrador de redes NT. Tres componentes MS-NET, el redirector, el bloque servidor de mensajes (SMB) protocolo y el servidor de red, fueron remodelados ampliamente e incorporados en sistemas operativos Windows posteriores.

El redirector, codificado en lenguaje de programación C, se implementa como un controlador del sistema de archivos cargable y depende de la arquitectura del software. Su función es dirigir una solicitud de E/S desde un usuario o una aplicación para eliminar el servidor remoto que tiene el archivo o el recurso idóneo para satisfacer la solicitud. Una red puede incorporar múltiples redirectores, cada uno de los cuales dirige solicitudes de E/S a sistemas de archivos o dispositivos remotos. Una solicitud remota de E/S típica puede resultar en la siguiente secuencia:

1. El software en modo de usuario emite una solicitud remota de E/S al llamar a servicios locales de E/S.
2. Luego de algo de procesamiento inicial, el administrador de E/S crea un paquete de solicitudes de E/S (IRP) y pasa la solicitud al redirector Windows, que envía el IRP a los controladores de transporte.
3. Finalmente, los controladores de transporte procesan la solicitud y la colocan en la red. Cuando la solicitud llega a su destino se observa la secuencia inversa.

El protocolo SMB es una especificación del alto nivel para formatear mensajes a enviar a través de la red y los correlaciona con la capa de aplicación (capa 7) y de la capa de presentación (capa 6) del modelo OSI descrito en el capítulo 9. Una API denominada interfaz

NetBIOS se usa para pasar solicitudes de E/S en el formato SMB a una computadora remota. Tanto los protocolos SMB como la API NetBIOS fueron adoptados en varios productos de red antes de aparecer en Windows.

Los sistemas operativos Windows Server están escritos en C para efectos de compatibilidad completa con los protocolos existentes MS-NET y SMB administrador de LAN, están implementados como controladores del sistema de archivos cargables y no dependen de la arquitectura del hardware en que se ejecuta el sistema operativo.

Servicios de directorio

La base de datos del **directorio activo**, almacena muchos tipos de información y funciona como un servicio de directorio de propósito general para una red heterogénea.

Microsoft construyó Active Directory completamente alrededor del **servicio de nombre de dominio** o **sistema de nombre de dominio** (**DNS**) y el **protocolo ligero de acceso a directorios** (**LDAP**). DNS es el servicio de nombre replicado jerárquico sobre el que se basa internet. Sin embargo, aunque DNS es el protocolo que constituye la columna vertebral de una de las redes de datos más grandes del mundo, no cuenta con suficiente flexibilidad para actuar como un directorio de empresa en sí. Es decir, DNS es primordialmente para asignar nombres de máquina a direcciones IP, lo cual no es suficiente para un servicio completo de directorio, que debe poder asignar nombres de objetos arbitrarios (como máquinas y aplicaciones) a cualquier tipo de información sobre estos objetos.

Active Directory agrupa máquinas en unidades administradoras denominadas dominios, cada una de las cuales obtiene un nombre de dominio DNS (como pitt.edu). Cada dominio debe tener por lo menos un controlador de dominio, que es una máquina que ejecuta el servidor de directorio activo.

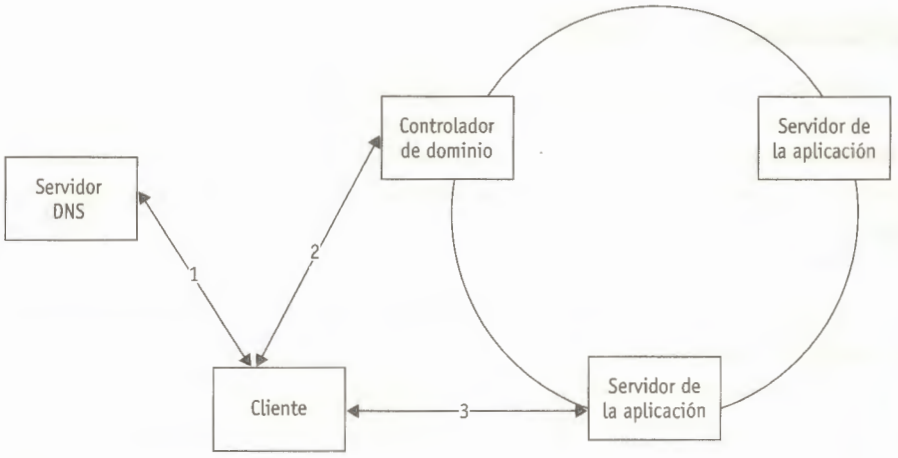
Para tolerancia a fallos y rendimiento mejorados, un dominio puede tener más de un controlador de dominio, donde cada uno cuenta con una copia completa de la base de datos del directorio de ese dominio.

Versiones actuales del sistema operativo de red Windows eliminan la distinción entre controladores de dominio primarios y controladores de dominio de respaldo, haciendo más fácil de administrar la red porque no tiene múltiples jerarquías. Los clientes del directorio activo usan las normas DNS y los protocolos LDAP para localizar objetos en la red. Como se muestra en la figura 15.7, así es como funciona.

1. Un cliente que necesita buscar un nombre en Active Directory primero debe pasar la parte DNS del nombre a un servidor normal DNS. El servidor DNS regresa la dirección de red al controlador de dominio responsable de ese nombre.
2. Después, el cliente usa LDAP para solicitar al controlador de dominio que encuentre la dirección del sistema que tiene el servicio que necesita el cliente.
3. Finalmente, el cliente establece una conexión directa con el servicio solicitado usando el protocolo correcto requerido por ese servicio.

(Figura 15.7)

Los clientes del directorio activo usan las normas DNS y protocolos LDAP para localizar objetos en la red.



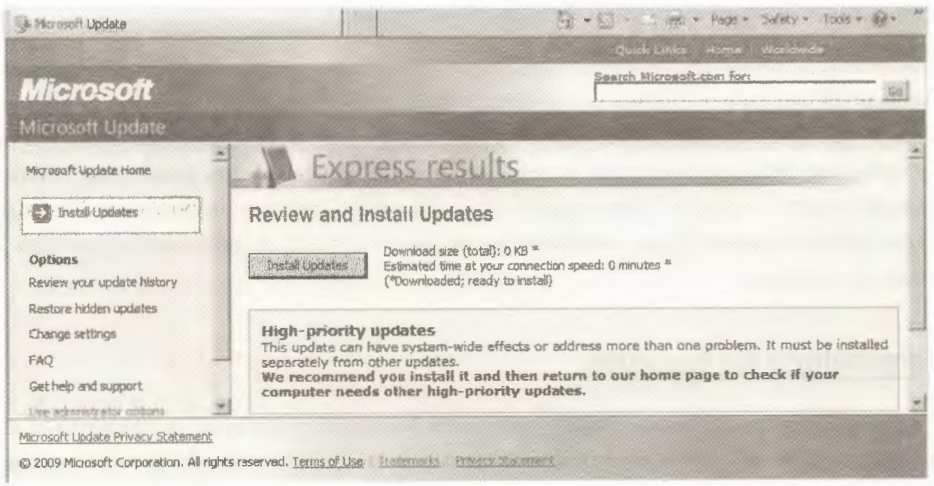
Administración de seguridad

Los sistemas operativos Windows cuentan con un modelo de seguridad basado en objetos. Es decir, un objeto seguridad puede representar cualquier recurso en el sistema; un archivo, dispositivo, proceso, programa o usuario. Esto permite a los administradores del sistema otorgar acceso a seguridad preciso para especificar objetos en el sistema a la vez que les permite monitorear y registrar la forma en que usan los objetos.

Una de las preocupaciones más importantes sobre los sistemas operativos Windows en general es la necesidad de administración de parches agresiva para combatir los diversos virus y gusanos que atacan esos sistemas. Las actualizaciones están disponibles en *www.microsoft.com*, como se muestra en la figura 15.8

(Figura 15.8)

Actualizaciones de las sistemas operativos están disponibles en línea.



Fundamentos de seguridad

El Departamento de Defensa de Estados Unidos ha identificado y clasificado en siete niveles de seguridad ciertas características que hacen seguro un sistema operativo. Las primeras versiones de Windows tenían por objetivo el nivel Clase C2 con un plan para evolucionar al nivel Clase B2; un nivel más restringido de seguridad donde al usuario debe asignarse una autorización de nivel de seguridad específica que no permite otorgar acceso a usuarios de nivel inferior a recursos protegidos.

Para cumplir con el nivel de seguridad C2, los sistemas operativos Windows ahora incluyen las siguientes características:

- Una facilidad de inicio de sesión segura que requiere que los usuarios se identifiquen al introducir un identificador de inicio de sesión único y una contraseña antes que se les permita tener acceso al sistema.
- Control de acceso discrecional que permite al propietario de un recurso determinar quién más puede tener acceso al recurso y qué pueden hacerlo.
- Capacidad de auditoría para detectar y registrar eventos importantes relacionados con la seguridad o cualquier intento por crear, tener acceso o borrar recursos del sistema.
- Protección de la memoria para evitar que cualquier persona lea información escrita por otra persona después que una estructura de datos ha sido liberada de regreso al sistema operativo.

La administración de una contraseña es la primera capa de seguridad.

La segunda capa de seguridad trata directamente con seguridad de acceso a archivos. A este nivel, el usuario puede crear un archivo y establecer varias combinaciones de personas para tener acceso al archivo porque el sistema operativo hace distinciones entre propietarios y grupos. El creador de un archivo es su propietario. El propietario puede designar un conjunto de usuarios como pertenecientes al grupo y permitir que todos los miembros del grupo tengan acceso a ese archivo. Al revés, el propietario puede impedir que algunos de los miembros tengan acceso al archivo.

Además de determinar a quién se le permite tener acceso al archivo, los usuarios pueden decidir qué tipo de operaciones se permite que una persona realice sobre un archivo. Por ejemplo, una persona puede tener acceso de sólo lectura, mientras otra puede tener privilegios de lectura y escritura. Como medida final, el sistema operativo proporciona al usuario capacidades de auditoría que automáticamente siguen la pista de quién usa los archivos y cómo se usan éstos.

Terminología de seguridad

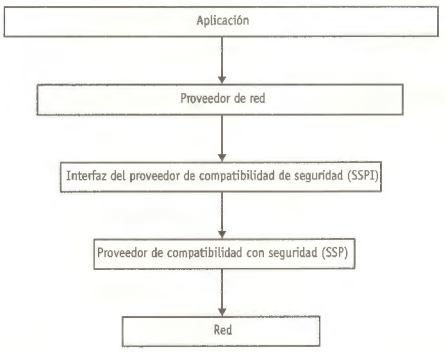
La seguridad integrada para sistemas operativos Windows recientes es un elemento necesario para administradores de servidores de red y redes. Su directorio de servicios permite que los usuarios encuentren lo que necesitan y un protocolo de comunicación permite que los usuarios interactúen en éste. Sin embargo, debido a que no todo mundo debe poder en-

contrar o interactuar con todo lo que hay en la red, el control de acceso es el trabajo de los servicios de seguridad distribuidos.

La seguridad distribuida efectiva requiere un mecanismo de autenticación que permita al usuario probar su identidad a un servidor. Después, el cliente necesita proporcionar información de autorización que el servidor usa para determinar cuáles derechos de acceso específicos han sido otorgados al cliente. Finalmente, necesita proporcionar integridad de datos usando una variedad de métodos que van desde una comprobación de suma cifrada para todos los datos transmitidos hasta datos completamente cifrados y transmitidos.

Los sistemas operativos Windows recientes proporcionan lo anterior con seguridad **Kerberos**, como se describió en el capítulo 11. Kerberos proporciona autenticación, integridad de datos y privacidad de datos. Además, proporciona autenticación mutua, lo cual significa que tanto el cliente como el servidor pueden verificar la identidad del otro. (Otros sistemas de seguridad sólo requieren que los clientes demuestren su identidad. Los servidores se autentican de manera automática.)

Cada dominio cuenta con su propio servidor Kerberos, que comparte la base de datos usada por Active Directory. Esto significa que el servidor Kerberos se debe ejecutar sobre la máquina del controlador de dominio y, como el servidor Active Directory, puede replicarse dentro de un dominio, todos los usuarios que desean tener acceso seguro a servicios remotos deben iniciar la sesión en un servidor Kerberos. La figura 15.9 muestra la ruta seguida por una solicitud desde una aplicación a un servicio proporcionado sobre la red.



(Figura 15.9)

Las solicitudes desde una aplicación fluyen a través de una serie de proveedores de seguridad, así como las respuestas desde la red de regreso a la aplicación.

Una sesión de inicio exitosa regresa una **concesión de entradas ticket** al usuario, que puede devolverse al servidor Kerberos para solicitar boletos para servidores de aplicaciones específicos.

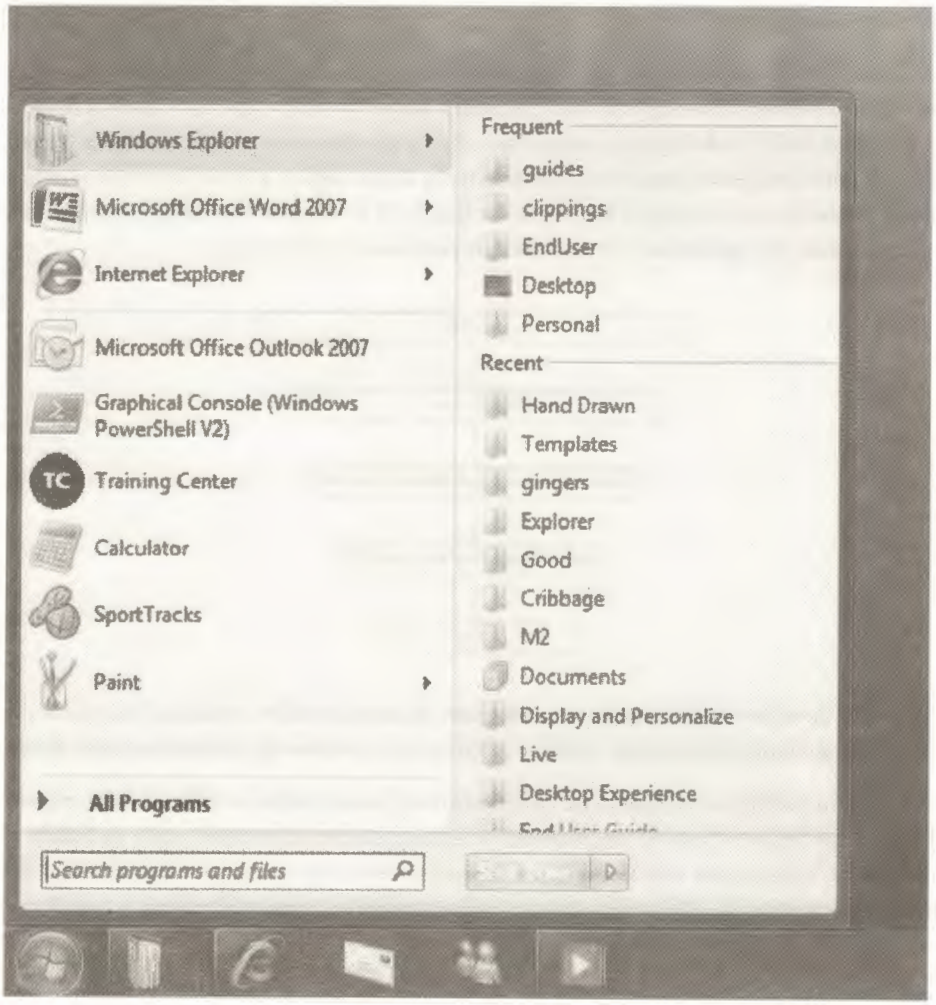
Si el servidor Kerberos determina que un usuario está presentando un boleto válido, regresa el boleto solicitado al usuario sin plantear ninguna pregunta. El usuario envía su boleto al servidor de aplicaciones remoto, que puede examinarlo para comprobar la identidad del usuario y autenticarlo. Todos estos boletos están cifrados en formas diferentes, y para llevar a cabo el cifrado se usan varias llaves.

Implementaciones diferentes de Kerberos envían información de autorización distinta. Microsoft ha implementado el protocolo normal Kerberos para hacer el producto más compatible con otras implementaciones de Kerberos.

Diferentes protocolos de seguridad pueden tener API muy diferentes, ocasionando problemas para aplicaciones que pudieran tener que usar más de uno de ellos. Microsoft ha abordado este problema al separar los usuarios de servicios de seguridad distribuidos de sus proveedores, permitiendo atender muchas opciones sin originar una complejidad inservible.

Interfaz de usuario

Aunque una descripción detallada de las herramientas presentadas en el escritorio rebasa el alcance de este capítulo, analizaremos brevemente el menú de inicio porque se trata de la aplicación clave del escritorio Windows. La figura 15.10 muestra un menú de inicio típico.



(Figura 15.10)

Un menú de inicio típico Windows divide las funciones en grupos lógicos y enumera las aplicaciones más recientemente usadas.

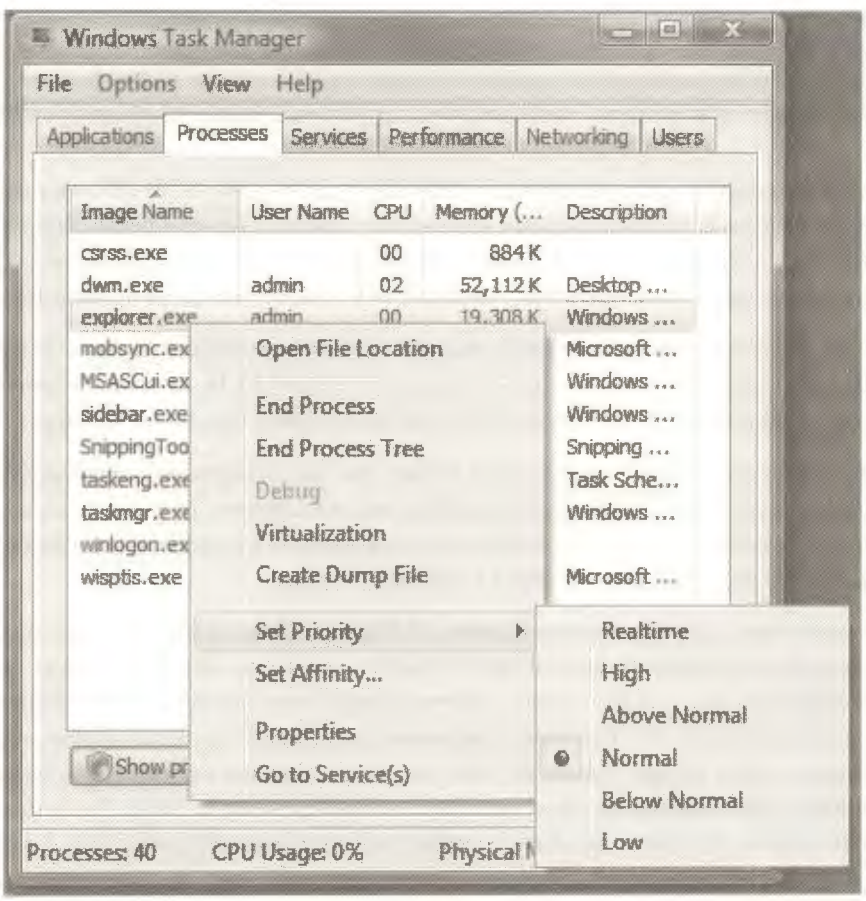
El menú de inicio organiza los archivos y los programas en grupos lógicos. A partir de ahí los usuarios efectúan funciones comunes incluyendo las siguientes:

- *All Programs* va a una lista de muchas aplicaciones disponibles. Las aplicaciones mostradas en la Tabla 15.10 fueron usadas recientemente. Para abrir una de nuevo, se hace clic en el icono.
- *Frequent* y *Recent* muestran aplicaciones y carpetas que son usadas de manera frecuente o reciente.
- *Search* inicia una rutina de búsqueda.
- *Shut Down* con opciones para hibernar o apagar la computadora.

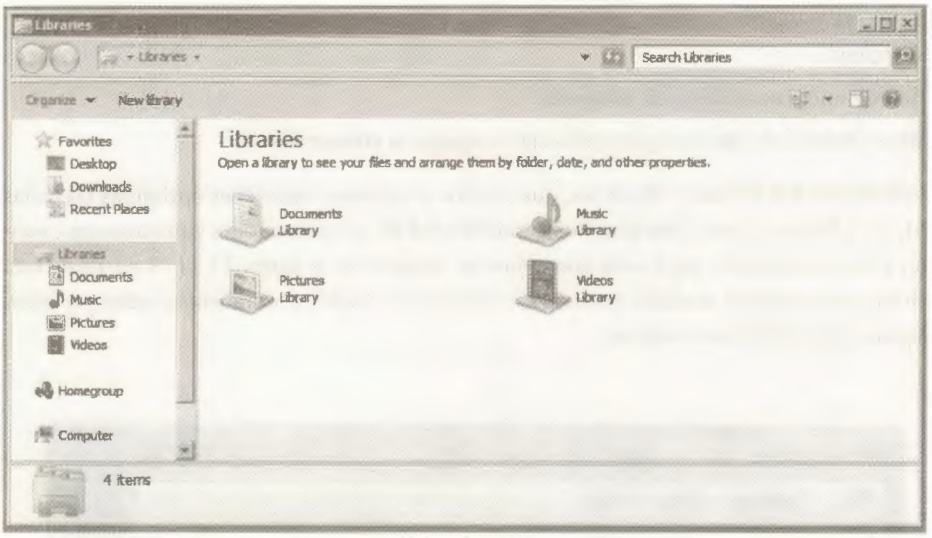
El administrador de tareas Windows, que se abre al oprimir y mantener oprimidas las teclas Ctrl, Alt y Delete, ofrece a los usuarios la posibilidad de ver aplicaciones y procesos en ejecución y fijar prioridades para cada uno, como se muestra en la figura 15.11. A partir de esta ventana, los usuarios también pueden ver información sobre rendimiento, redes y a otros usuarios registrados en el sistema.

(Figura 15.11)

Administración de prioridad usando el administrador de tareas.



Un programa de utilidad normal denominado explorador de Windows (que no debe confundirse con el navegador de la web denominado explorador de internet) contiene herramientas para exhibir directorios y archivos, así como una herramienta para encontrar archivos, como se muestra en la figura 15.12.



(Figura 15.12)
Explorador de Windows es una herramienta de administración de archivos que muestra directorios (carpetas).

Para sistemas de red hay herramientas a fin de ayudar a que los usuarios identifiquen y tengan acceso a recursos de red como carpetas, impresoras y conexiones a otros nodos. Para identificarlas, vaya a Network y Sharing Center, haga clic en View Computers and Devices y luego haga clic en la opción para asignar una unidad de red, como se muestra en la figura 15.13.

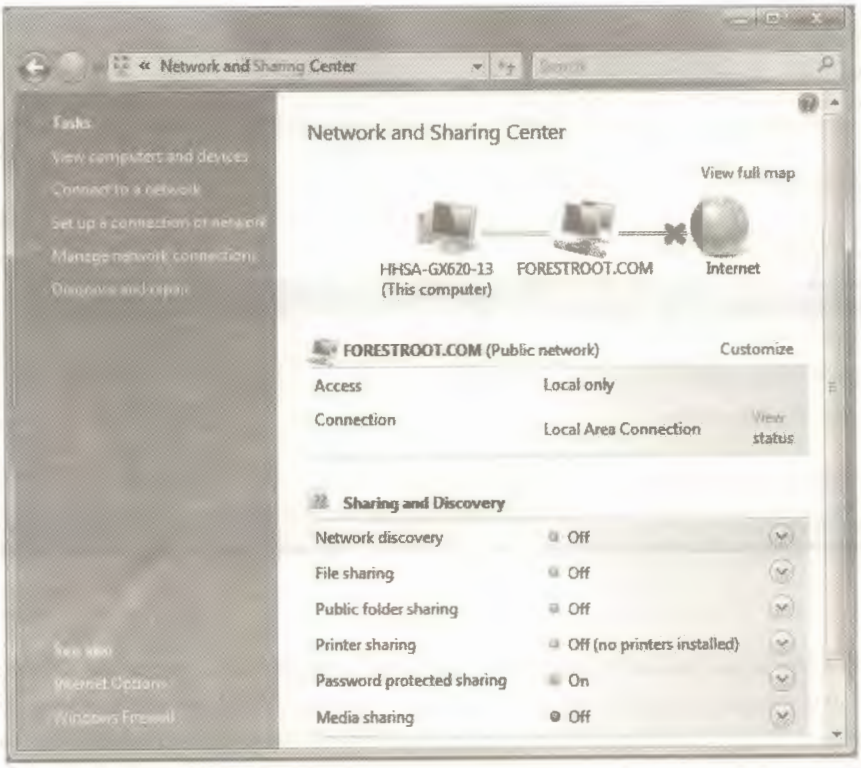
Una interfaz de comandos que semeja la usada por MS-DOS está disponible desde la mayor parte de los escritorios Windows, como se muestra en la figura 15.14. Al usar esta característica es posible intentar comandos MS-DOS desde una máquina ejecutando Windows.

Para usuarios que son más rápidos con el teclado que con un dispositivo apuntador, Windows cuenta con muchos atajos de teclado. Para una guía, observe los atajos de teclado en los menús desplegables, como el que se muestra en la figura 15.15, que identifica a ALT+TAB como el atajo de teclado para cambiar a la siguiente ventana.

Una característica útil de versiones recientes de Windows es su alojamiento para usuarios que trabajan en idiomas distintos al inglés. Windows cuenta con métodos y fuentes de entrada integrados para muchos idiomas, incluyendo lenguajes en bits dobles como el japonés. Durante la instalación, el administrador del sistema puede escoger uno o varios idiomas para el sistema, incluso agregar soporte de otro idioma para personas específicas. Por ejemplo, un usuario puede trabajar en chino mientras otro puede hacerlo en hindi. Aún mejor, los propios recursos del sistema también se vuelven políglotos, lo cual significa que el sistema operativo cambia sus cuadros de diálogo, desplegados y menús para atender el idioma preferido del usuario.

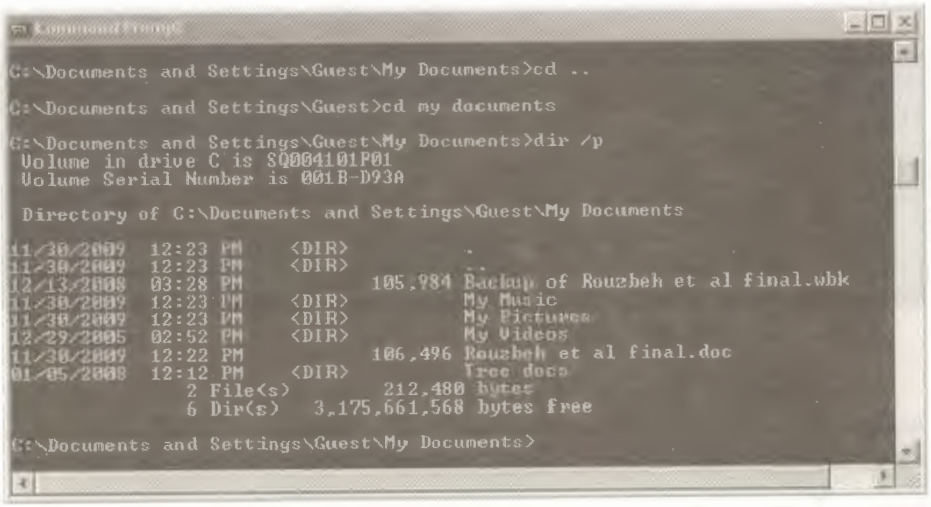
(Figura 15.13)

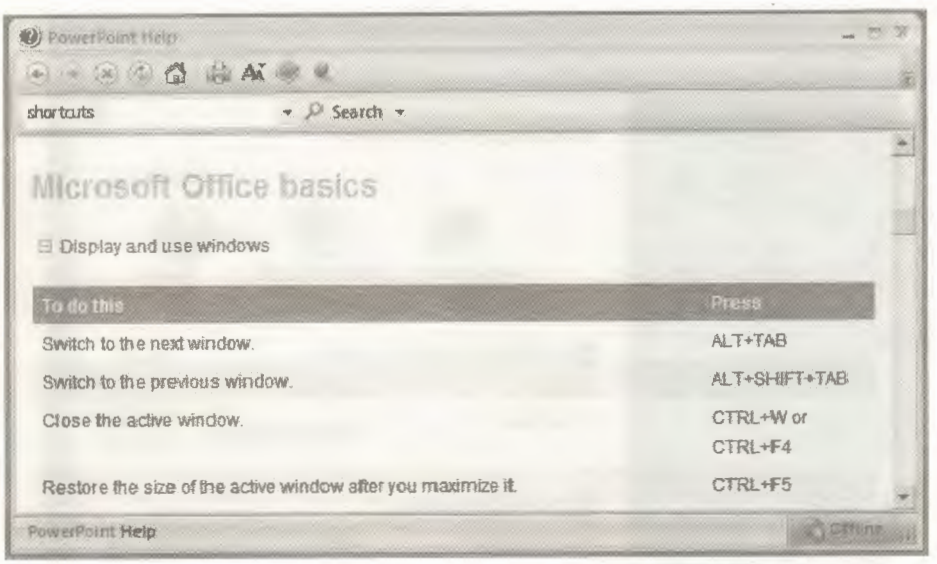
Los administradores de un sistema en una red pueden asignar una unidad de red para identificar recursos disponibles.



(Figura 15.14)

Ventana de comandos que permite a los usuarios ejecutar muchos de los comandos MS-DOS.





(Figura 15.15)

Los atajos de teclado se muestran a la derecha cerca del contenido del menú.

Para usuarios que requieren opciones de accesibilidad mejorada o que tienen dificultades para usar un teclado normal, pero requieren funcionalidad, Windows ofrece un teclado en pantalla, como se muestra en la figura 15.16. Esta herramienta y otras (amplificadora, narradora, identificación de voz y más) pueden encontrarse desde el botón Start, bajo Accessories, Ease of Access.

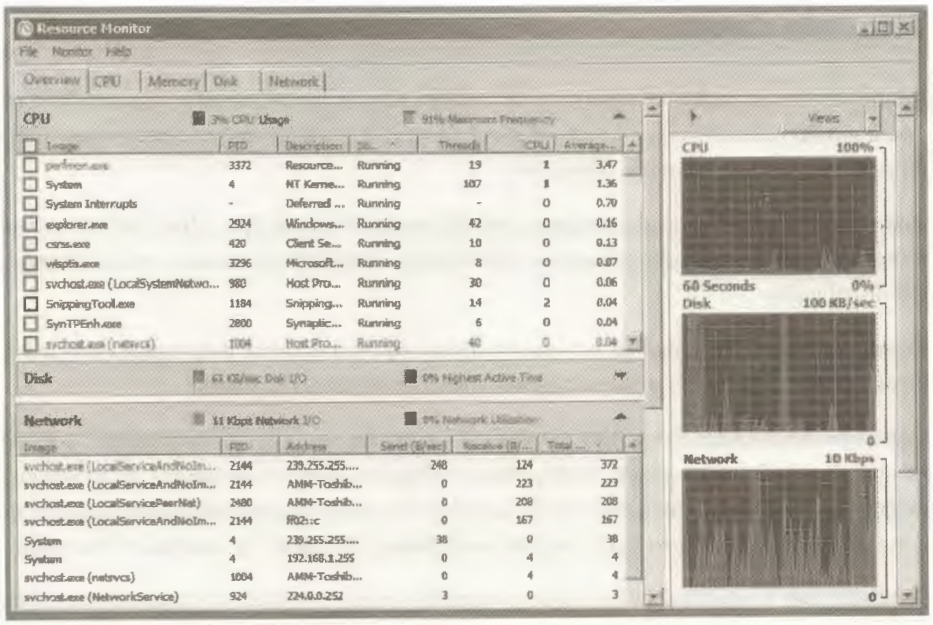
(Figura 15.16)

Desde la carpeta Accesories, herramientas de accesibilidad, como un teclado en pantalla, están disponibles para proporcionar herramientas de interfaz con el usuario mejoradas.

Detalles sobre el uso del hardware y software del sistema pueden encontrarse desde el monitor de recursos, como se muestra en la figura 15.17.

(Figura 15.17)

El monitor de recursos, disponible desde el panel de control, puede proporcionar estadísticos en ejecución sobre el uso de los recursos del sistema.



Conclusión

Lo que comenzó como un sistema operativo de microcomputadora, ha crecido para incluir un complejo software multiplataforma que puede ser utilizado para hacer funcionar sistemas de cómputo de todos los tamaños. El éxito comercial de Windows es incuestionable y sus productos continúan evolucionando en complejidad y alcance para cubrir un gran número de mercados mundiales.

Los productos Windows son ubicuos, incluyendo Windows Embebido, Windows Automotive y Windows Mobile, el nombre de unas cuantas de las muchas versiones especializadas de este sistema operativo. Microsoft ofrece asistencia técnica para sistemas operativos no muy vendidos, como Windows NT y por supuesto MS-DOS.

Una palabra de advertencia: la vulnerabilidad de la seguridad de los sistemas operativos de Windows, los hacen blancos populares para programadores de códigos maliciosos. Ya sea que estas vulnerabilidades se deban a su enorme cuota de mercado (haciéndolo enormemente atractivo) o a los errores en los códigos por parte de Microsoft, el resultado es el mismo: existe una constante necesidad para todo administrador de sistemas y propietarios de una computadora, de mantener proactivamente todos los sistemas de Windows tan seguros como sea posible por medio del control vigilante de acceso y la administración de parches.

Términos clave

Administrador de caché: Componente del sistema d e E/S que administra la parte de la memoria virtual conocida como caché. El caché se amplía o reduce dinámicamente dependiendo de la cantidad de memoria disponible.

Active Directory: Servicio de directorio de Microsoft Windows que ofrece administración centralizada de servicios de aplicación, autenticación y registro de usuarios para sistemas de red distribuidos.

Compatibilidad: Capacidad de un sistema operativo para ejecutar programas escritos para otros sistemas operativos o para versiones anteriores del mismo sistema.

Concesión de entradas ticket: Boleto "virtual" otorgado por un servidor Kerberos que indica que al usuario poseedor del boleto puede otorgarse acceso a servidores de aplicaciones específicas. El usuario envía este boleto cifrado al servidor de las aplicaciones remoto, que entonces puede examinarlo para verificar la identidad del usuario y autenticar al usuario.

Confiabilidad: Capacidad de un sistema operativo para responder de manera predecible a condiciones de error, incluso aquellas ocasionadas por fallas de hardware; o la capacidad de un sistema operativo para protegerse activamente a sí mismo y a sus usuarios de daño accidental o deliberado por programas del usuario.

Extensibilidad: Uno de los objetivos de diseño de un sistema operativo, que permite su fácil mejoramiento a medida que cambian los requerimientos del mercado.

Interfaz gráfica de usuario (GUI): Interfaz de usuario que permite al usuario activar comandos del sistema operativo al hacer clic en iconos o menús del escritorio usando un dispositivo apuntador como un ratón o una pantalla táctil. Las GUI evolucionaron a partir de interfaces de usuario controladas por comandos.

Kerberos: Sistema de autenticación del MIT que permite a los administradores de red administrar y realizar la autenticación del usuario a nivel de red.

Modo de núcleo: Nombre dado para identificar los procesos a los que se ha otorgado acceso privilegiado al procesador. En consecuencia, se permiten todas las instrucciones de máquina y la memoria del sistema es accesible. Contrasta con el modo de usuario, más restrictivo.

Modo de usuario: Nombre dado para indicar los procesos a los que no se ha otorgado acceso privilegiado al procesador. En consecuencia, no se permiten ciertas instrucciones y la memoria del sistema no es accesible. Contrasta con el modo de núcleo menos restrictivo.

Política de búsqueda: Reglas usadas por el administrador de la memoria virtual para determinar cuándo debe copiarse una página del disco a la memoria.

Política de colocación: Reglas usadas por el administrador de la memoria virtual para determinar dónde debe cargarse la página virtual en la memoria.

Política de sustitución: Reglas usadas por el administrador de la memoria virtual para determinar cuál página virtual debe eliminarse de la memoria para hacer espacio para una nueva.

Portabilidad: Capacidad para mover todo un sistema operativo a una máquina basada en un procesador o configuración diferente con la menor cantidad de recodificación posible.

POSIX: *Portable Operating System Interface* para UNIX; una aplicación del programa de interfaz del sistema operativo desarrollada por la IEEE para incrementar la portabilidad del software de aplicación.

Procesadores de mensajes: Interfaz de software de red de alto nivel para pasar datos entre procesos en un mecanismo de comunicación uno a muchos y muchos a uno. Son de utilidad para transmitir mensajes a cualquier número de procesos.

Protocolo ligero de acceso a directorios (LDAP): Protocolo que define un método para crear directorios de recursos en una red que pueden buscarse. Se denomina "ligero" porque es una versión simplificada y habilitada de TCP/IP del protocolo de directorio estilo X.500.

Servicio de nombre de dominio o sistema de nombre de dominio (DNS): Servicio de consulta de datos de propósito general, distribuido y replicado. Su función principal consiste en la resolución de direcciones de internet con base en nombres de dominio totalmente calificados como .com (de entidad comercial) o .edu (de institución educativa).

Sistema de archivos NT (NTFS): Sistema de archivos introducido con Windows NT que ofrece servicios de administración de archivos, como administración de permisos, compresión, registro de transacciones y la capacidad para crear un volumen único que genera dos o más discos físicos.

Tuberías nombradas: Interfaz de software de alto nivel hacia NetBIOS, que representa la porción de hardware en aplicaciones de red como objetos abstractos. Las tuberías nombradas se representan como objetos archivo en Windows NT y versiones posteriores y operan bajo el mismo mecanismo de seguridad que otros objetos ejecutivos.

Búsquedas de interés

- Sistema de archivos Windows
- Sistema operativo Windows embebido
- Windows contra Macintosh
- Punto de referencia de Windows
- Administración de parches en Windows

Ejercicios

Temas de investigación

A. Consulte literatura actual para conocer el estado actual de las normas IEEE POSIX y determine si la versión de Windows en la computadora que usted usa está actualizada al 100% en su compatibilidad con POSIX. Explique el significado de esta compatibilidad y por qué piensa que algunos sistemas operativos populares no son compatibles.

B. Algunos productos de Windows no permiten el uso de caracteres internacionales en el nombre de usuario o en la contraseña. Estos caracteres pueden ser parte de un alfabeto internacional o caracteres asiáticos. Investigue los caracteres que están permitidos en las versiones más recientes de Windows y cite sus fuentes. Describa las ventajas de los sistemas operativos al limitar el grupo de caracteres para los nombres de usuario y contraseñas, y si puede o no sugerir una alternativa.

Ejercicios

1. Si usted buscara agregar estos cuatro archivos a un directorio de Windows (octubre.doc, OCTubre.doc, OCTUBRE.doc y OcTUBre.doc), ¿cuántos archivos pueden desplegarse: uno, dos, tres o cuatro? Explique por qué esto es así. ¿Piensa usted que la respuesta es la misma para todos los sistemas operativos? ¿Por qué sí o por qué no?
2. Explique la importancia del monitoreo del rendimiento del sistema y por qué Windows hace esta información disponible para el usuario.
3. En algunos sistemas operativos de Windows, el archivo de paginación es un archivo oculto en el disco duro de la computadora y su memoria virtual es la combinación del archivo de paginación y la memoria física del sistema. (Esto se denomina pagefile.sys y el tamaño asignado es de 1.5 veces la memoria RAM total del sistema.) Describa cómo el tamaño virtual de la memoria podría tener un efecto en el rendimiento del sistema.
4. Si el archivo de paginación se encuentra en donde es menos probable que ocurra la fragmentación, el rendimiento puede ser mejorado. ¿Verdadero o falso? Explique.
5. Cuando Windows es desplegado en un entorno multilingüe, los administradores encuentran que algunos leguajes requieren más espacio de almacenamiento en el disco duro que otros. En su opinión, ¿por qué es este el caso?
6. La versión 7 de Windows de 64 bits puede ejecutarse en todas las aplicaciones de 32 bits con la ayuda de un emulador, pero no soporta aplicaciones de 16 bits. ¿Puede usted imaginar una circunstancia en la que alguno podría necesitar soporte para una aplicación de 16 bits? Descríbala.
7. Una característica de Windows 7 es la autenticación de Kerberos. Describa el rol del vale de concesión del boleto para autenticar usuarios para el acceso a la red.
8. Describa el papel del Active Directory (directorio activo) en los sistemas operativos de Windows recientes.
9. El sistema E/S se basa en un paquete de solicitud E/S. Explique el papel de este paquete, cuándo se pasa y a dónde va antes de la eliminación.

Ejercicios avanzados

10. Identifique al menos cinco de los principales tipos de amenazas para sistemas que ejecutan Windows y las políticas que los administradores del sistema deben tomar para proteger el sistema de accesos no autorizados. Compare los problemas prácticos cuando

equilibra la necesidad de accesibilidad con la de restricción en el acceso y sugiera la primera acción que usted tomaría para asegurar una computadora Windows o red, si administrara una.

11. Windows embebido es un sistema operativo que está planeado para ejecutarse en tiempo real. Describa la diferencia entre un sistema de tiempo real duro y un sistema de tiempo real suave y describa los puntos más destacables que usted considere como los más importantes de cada tipo de sistema.

12-14. Para cada una de estas preguntas, refiérase a la tabla 15.6 (adaptada de *www.microsoft.com*), la cual muestra cómo las estructuras de memoria de 64 bits del sistema operativo Windows, utilizan un microprocesador Intel de 64 bits en comparación con los 32 bits máximos de los sistemas operativos Windows previos.

(Tabla 15.6)

Especificaciones de Windows para sistemas de 32 y 64 bits adaptado de www.microsoft.com.

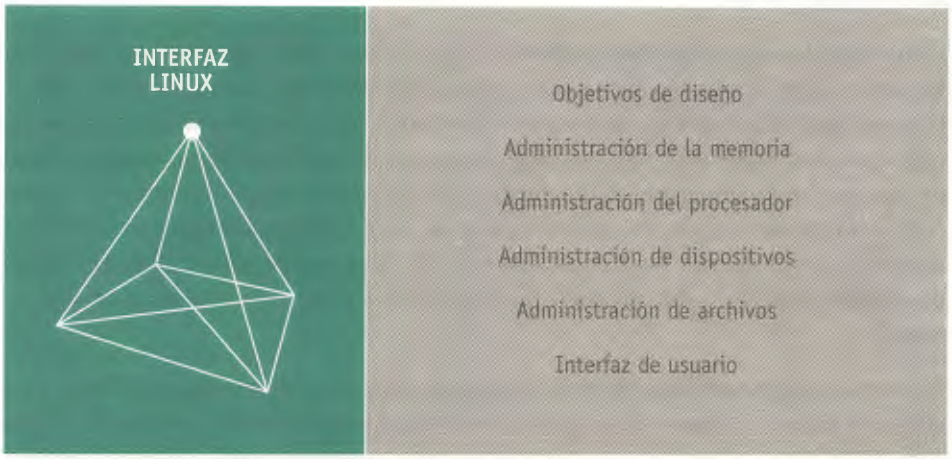
Componente	32-bit	64-bit
Memoria virtual	4 GB	16 TB
Tamaño del archivo de paginación	16 TB	256 TB
Caché del sistema	1 GB	1 TB
Hiperespacio	4 MB	8 GB
Combinación de páginas	470 MB	128 GB
Sistema PTE	660 MB	128 GB

12. El hiperespacio es utilizado para asignar grupos de páginas de trabajo para los procesos del sistema, asignar otras páginas físicas temporales y otras tareas. Al incrementar este espacio de 4 MB a 8 GB en un sistema de 64 bits, el hiperespacio ayuda a Windows a ejecutarse más rápido. En su opinión, explique por qué esto es así y describa otras mejoras en el rendimiento que aumentaron el hiperespacio que puede tener sobre el rendimiento del sistema. ¿Puede usted cuantificar el incremento en la velocidad de la información mostrada aquí? Explique su respuesta.

13. La memoria paginada es la parte de la memoria virtual, creada durante la inicialización del sistema, que puede ser paginada dentro o fuera del grupo de trabajo del proceso del sistema y es utilizada por los componentes del modo de núcleo para asignar memoria en el sistema. Si los sistemas con un procesador tienen dos memorias paginadas y con procesadores múltiples tienen cuatro, explique por qué el tener más de una memoria paginada reduce la frecuencia del código de bloqueo del sistema en llamados simultáneos para rutinas agrupadas.

14. Un sistema PTE es una agrupación de las entradas de tabla de las páginas del sistema que son utilizadas para mapear las páginas del mismo tales como espacio E/S, núcleos apilados y listas de descripción de memoria. Los programas de 32 bits utilizan un modelo de 4 GB y asignan la mitad (2 GB) al usuario y la otra mitad al núcleo. Los programas de 64 bits utilizan un modelo similar pero a una escala mayor con 8 TB para el usuario y 8 TB para el núcleo. Dada esta estructura, calcule cuántos exabytes puede dirigir un apuntador de 64 bits (un exabyte equivale a mil millones de gigabytes).

Capítulo 16 Sistema operativo Linux



"Estoy haciendo un sistema operativo (libre)..."

Linus Torvalds

Objetivos de aprendizaje

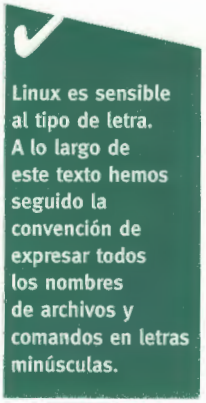
Después de completar este capítulo, el lector debe ser capaz de describir:

- Los objetivos de diseño del sistema operativo.
- La flexibilidad ofrecida al usar archivos para manipular dispositivos.
- Las diferencias entre interfaz basada en comandos e interfaz basada en menús.
- Los roles de los administradores de la memoria, de dispositivos, de archivos, del procesador y de red.
- Algunas fortalezas y debilidades de Linux.

Linux no es UNIX. Linux se basaba en una versión de UNIX pero capitalizó las lecciones aprendidas durante los 20 años previos de desarrollo de UNIX. Linux posee características únicas que lo diferencian de sus predecesores y lo constituyen como una fuerza global en el desarrollo de sistemas operativos. Lo que es más, Linux no sólo es poderoso, sino que es gratis.

Visión general

Linux es totalmente compatible con POSIX (POSIX se analizará dentro de poco) y es portable con versiones disponibles para su ejecución en teléfonos celulares, supercomputadoras y la mayor parte de los sistemas de cómputo correspondientes. A diferencia de otros sistemas operativos descritos en este libro, su código fuente está disponible de forma gratuita, lo cual permite que los programadores lo configuren para ejecutar cualquier dispositivo y cumpla cualquier especificación. La inclusión frecuente de varias GUI de escritorio poderosas sigue atrayendo a los usuarios. También es altamente modular, permitiendo la carga y descarga de múltiples módulos sobre demanda, haciéndolo un sistema operativo técnicamente robusto.



Linux es un programa fuente abierto, lo que significa que este código fuente está abierto para cualquier mejora. Si alguien envía un programa mejor o una secuencia de códigos a Linus Torvalds, el autor de Linux, y si es aceptado como una mejora universal al sistema operativo, entonces el nuevo código es agregado a la siguiente versión poniéndolo disponible para el mundo informático. Las actualizaciones se programan cada seis meses. De este modo, Linux está bajo constante desarrollo por incontables colaboradores alrededor del mundo, muchos de los cuales no se reunirán jamás. El nombre de Linux permanece como una marca registrada de Linus Torvalds.

Historia

Linus Torvalds, deseaba crear un sistema operativo para maximizar las limitadas capacidades del microprocesador Intel 80386. Empezó con MINIX (un sistema UNIX miniatura desarrollado principalmente por Andrew Tanenbaum) y volvió a escribir ciertas partes a fin de agregar más funcionalidad. Una vez que obtuvo un sistema operativo que funcionaba, anunció su logro sobre un grupo de usuarios de internet con este mensaje:

```
Hello everybody out there using minix.I'm doing a (free)
operating system (just a hobby, won't be big and profes-
sional like gnu) for 386(486)AT clones.
```

Era agosto de 1991, y Torvalds era un estudiante de 21 años en la Universidad de Helsinki, Finlandia. (El nombre Linux es una contracción de Linus y UNIX, y cuando se pronuncia éste rima con "mimics".) Este nuevo sistema operativo, originalmente creado para su ejecución en una pequeña microcomputadora, fue construido con bastante flexibilidad y muchas de las funciones encontradas en sistemas operativos comerciales costosos. En efecto, Linux trajo la velocidad, eficiencia y flexibilidad de UNIX a un entorno de computadoras de escritorio.

El primer sistema operativo Linux requería comandos escritos y algunas veces crípticos. Ahora, los usuarios pueden introducir comandos usando ya sea una **interfaz basada en comandos** (modo terminal) o una **interfaz gráfica de usuario** (GUI), ampliando considerablemente la usabilidad del sistema operativo. Después, en este capítulo se analizan las interfaces gráficas de usuario.

El primer partidario corporativo importante de Linux fue Red Hat Linux, el distribuidor mundial líder de Linux hasta 2003. En septiembre de ese año, la empresa dividió sus esfuerzos en dos direcciones, el Fedora Project para alentar la continuación del desarrollo de fuente abierta del **núcleo** Linux, y Red Hat Enterprise Linux (RHEL) para cumplir las necesidades crecientes de organizaciones que deseaban pagar por un sistema operativo a nivel de la empresa y por soporte técnico dedicado.

Como se muestra en la tabla 16.1, el Fedora Project emite actualizaciones gratuitas para el público aproximadamente cada seis meses. Hay muchas otras distribuciones conocidas de Linux, incluyendo Mandriva, Debian y SUSE.

(Tabla 16.1)

Las versiones más importantes de Linux Red Hat, Inc. RHL es un acrónimo de Red Hat Linux. RHEL es un acrónimo de Red Hat Enterprise Linux. Fedora es una marca registrada de Red Hat, Inc.

Año	Versión	Características
1994	Versiones Beta	Primer producto Red Hat Linux disponible para el público en una serie de Versiones beta.
1995	RHL 1.0	Primera versión no beta de Red Hat Linux.
1995	RHL 2.0	Escrita en Perl para fácil desarrollo.
1996	RHL 3.0.3	La primera versión multiarquitectura aproximadamente concurrente; atendía la plataforma Digital Alpha.
1996	RHL 4.0	Basada en el núcleo 20.18 y primera versión que incluyó documentación gratuita disponible en forma electrónica.
1997	RHL 5.0	Nombrada Producto del Año InfoWorld en 1997.
1999	RHL 6.0	Integró la GUI de escritorio GNOME.
2000	RHL 7.0	Primera versión que soporta Red Hat fuera de la caja.
2001	RHL 7.0.90	Introdujo el núcleo 2.4.
2002	RHEL 2.1 AS (Servidor avanzado)	Lanzamiento de Red Hat Enterprise Linux, el primer producto comercial informático, basado en RHL 7.2.
2002	RHL 8.0	Diseñado para suministrar una visión unificada a través de los escritorios RHL y RHEL.
2003	RHL 9	Primera versión en incluir soporte para Native POSIX Thread Library (NPTL).
2003	RHEL 3	Primer producto Red Hat fabricado para ejecutarse en arquitecturas con 7 chips (por Intel, AMD e IBM).

Año	Versión	Características
2003	Fedora Core 1	Producto basado en RHL 9 para usuarios individuales; creado por el Fedora Project en cooperación con Red Hat.
2004	Fedora Core 2	Introdujo Seguridad Mejorada de Linux (SELinux), una implementación de control de acceso obligatorio (Mandatory Access Control; MAC) en el núcleo.
2004	Fedora Core 3	Atendía el núcleo Linux 2.6.9, actualizado como SELinux, y atiende las últimas interfaces gráficas de usuario conocidas más recientes, incluyendo KDE y Gnome.
2005	RHEL 4	Red Hat Enterprise Linux basada en RHL 7.2.
2006	Fedora Core 5 & 6	Atendía tecnología de máquina virtual.
2007	Fedora 7	Nueva denominación (se eliminó Core). Permitía personalización. Accesibilidad ampliada por contribuyentes del Fedora Project.
2007	RHEL 5	Desempeño mejorado, seguridad y flexibilidad, con almacenamiento virtual.
2009	Fedora 11	Rápido arranque desde el encendido hasta la operación completa del sistema. Maneja archivos de más de 16TB.

Debido a que Linux está escrito y se distribuye bajo la Licencia pública general GNU, su código fuente se distribuye y pone a disposición del público de manera gratuita. Al momento de escribir esto, la licencia pública general GNU es la versión 3. Se permite que cualquier persona copie y distribuya copias verbatim de la licencia, pero no se permiten modificaciones. Puede encontrarse en: *www.gnu.org/licenses/gpl.html*.

Objetivos de diseño

Linux tiene tres objetivos de diseño: modularidad, simplicidad y portabilidad (personalizadas en su mascota, mostrada en la figura 16.1). Para lograr estos objetivos, los administra-

(Figura 16.1)

La mascota Linux evolucionó a partir de discusiones con Linus Torvalds, quien dijo: "Ok, por lo tanto deberíamos pensar en un pingüino adorable de peluche sentado tras haberse atiborrado de arenque." Más información acerca del pingüino puede encontrase en www.linux.org

dores de Linux tienen acceso a numerosas utilidades normales, lo cual elimina la necesidad de escribir un código especial. Muchas de estas utilidades pueden usarse en combinación con las otras, de modo que el usuario puede escoger y combinar utilidades idóneas para realizar tareas específicas. Como se muestra en la tabla 16.2, Linux permite numerosas funciones.

(Tabla 16.2)

Linux atiende una amplia gama de funciones del sistema.

Función	Propósito
Múltiples procesos y múltiples procesadores	Linux puede ejecutar más de un programa o procesos a la vez y puede administrar numerosos procesadores.
Múltiples plataformas	Aunque originalmente fue desarrollado para su ejecución en procesadores Intel para microcomputadoras, ahora puede operar en casi cualquier plataforma.
Múltiples usuarios	Linux permite que múltiples usuarios trabajen en la misma máquina al mismo tiempo.
Comunicaciones interprocesos	Atiende tuberías, sockets, etcétera.
Administración de la terminal	Su administración de la terminal se ajusta a las normas POSIX, y también atiende seudo terminales, así como sistemas de control de procesos.
Dispositivos periféricos	Linux atiende una amplia gama de dispositivos, incluyendo tarjetas de sonido, interfaces gráficas, redes, SCSI, USB, etcétera.
Caché del búfer	Linux atiende un área de la memoria reservada para el búfer de entrada y salida de diferentes procesos.
Gestión de la memoria por paginación por demanda	Linux carga páginas en la memoria sólo cuando son necesarias.
Bibliotecas dinámicas y compartidas	Bibliotecas dinámicas se cargan sólo cuando son necesarias, y su código se comparte si varias aplicaciones las están usando.
Particiones de disco	Linux permite particiones de archivos y particiones de disco con diferentes formatos de archivo.
Protocolo de red	Atiende TCP/IP y otros protocolos de red.

Linux se ajusta a las especificaciones de **Interfaz para sistemas operativos migrables basados en UNIX (POSIX®)**, una marca registrada del IEEE. POSIX es una norma del IEEE que define las interfaces del sistema operativo para mejorar la portabilidad de programas de un sistema operativo a otro (IEEE, 2004).

Administración de la memoria

Cuando Linux asigna espacio de memoria, asigna 1 GB de memoria de orden alto al núcleo y 3 GB de memoria a los procesos en ejecución. Este espacio de direcciones de 3 GB se divide entre código de proceso, datos de proceso, datos de la biblioteca compartida usada por el proceso y la pila usada por el proceso.

Cuando se inicia la ejecución de un proceso, sus segmentos tienen un tamaño fijo pero hay casos en que un proceso debe manipular variables de número y tamaño desconocidos. En consecuencia, Linux tiene llamadas al sistema que cambian el tamaño del segmento de datos de proceso, ya sea ampliándolo para dar cabida a valores de datos extra o reduciéndolo cuando ya no son necesarios ciertos valores ubicados en el extremo del segmento de datos.

Linux ofrece protección de la memoria basada en el tipo de información almacenada en cada región perteneciente al espacio de direcciones del proceso. Si un proceso modifica autorización de acceso asignada a una región de la memoria, el núcleo cambia la información de protección asignada a las páginas de la memoria correspondientes.

Cuando un proceso solicita páginas, Linux las carga en la memoria. Cuando el núcleo necesita el espacio de la memoria, las páginas se liberan usando el algoritmo de la menos usada recientemente (LRU). Linux mantiene en la memoria un área administrada dinámicamente, un caché de páginas, donde se insertan nuevas páginas solicitadas por los procesos, y desde donde se borran páginas cuando ya no son necesarias. Si alguna página marcada para ser borrada ha sido modificada, se vuelve a escribir en el disco; una página correspondiente a un archivo asignado a la memoria se vuelve a escribir en el archivo y una página correspondiente a los datos se guarda en un dispositivo de intercambio. El dispositivo de intercambio puede ser una partición en el disco o puede ser un archivo normal. Linux muestra flexibilidad agregada con los dispositivos de intercambio ya que, de ser necesario, Linux puede desactivarlos sin tener que volver a arrancar el sistema. Cuando esto se lleva a cabo, todas las páginas guardadas en ese dispositivo vuelven a cargarse en la memoria.

Para seguir la pista de las páginas libres y ocupadas, Linux usa un sistema de tablas de páginas. Con ciertas arquitecturas de chips, el acceso a la memoria se lleva a cabo usando segmentos.

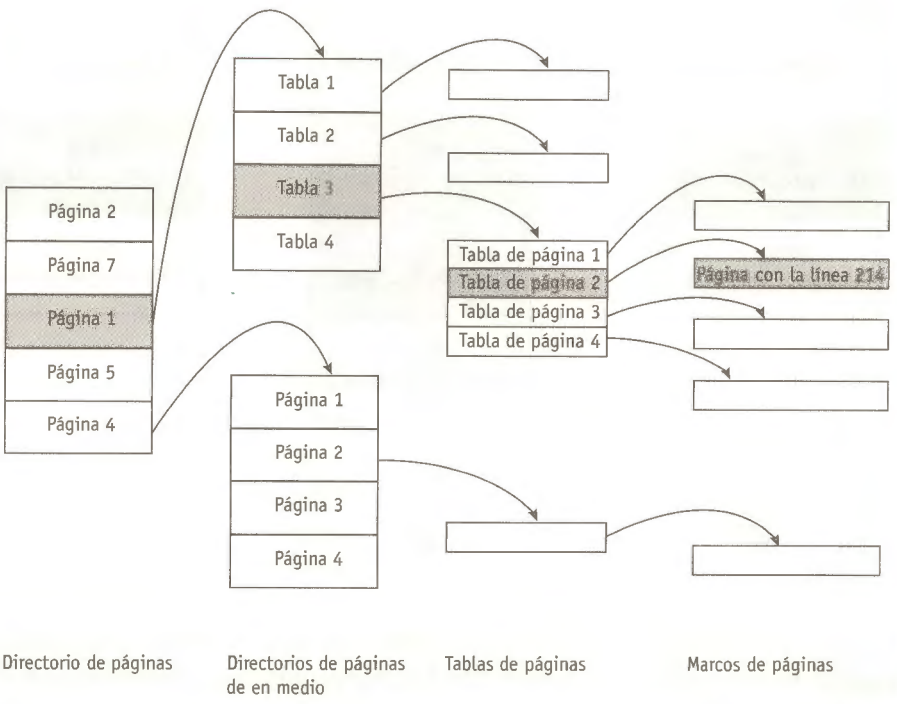
La memoria virtual en Linux se administra usando una jerarquía de tabla multinivel, que da cabida a arquitecturas de 64 y 32 bits. La tabla 16.3 muestra la manera en que cada memoria virtual consta de cuatro campos, que son usados por el administrador de la memoria para ubicar la instrucción o los datos solicitados.

Directorio principal	Directorio de en medio	Directorio de la tabla de páginas	Marco de página
Página 1	Tabla 3	Tabla de página 2	Ubicación de la línea 214.

(Tabla 16.3)

Los cuatro campos que constituyen la dirección virtual para la línea 214 en la figura 16.2.

Cada página tiene su propia entrada en el directorio principal, que tiene apuntadores a cada directorio de en medio de la página. Un directorio de en medio de una página contiene apuntadores a sus directorios de tabla de páginas correspondientes. A su vez, cada directorio de tabla de páginas tiene apuntadores al marco de página actual, como se muestra en la figura 16.2. Finalmente, el campo de desplazamiento de página se usa para ubicar la instrucción a los datos con la página solicitada (en este ejemplo es la línea 214).

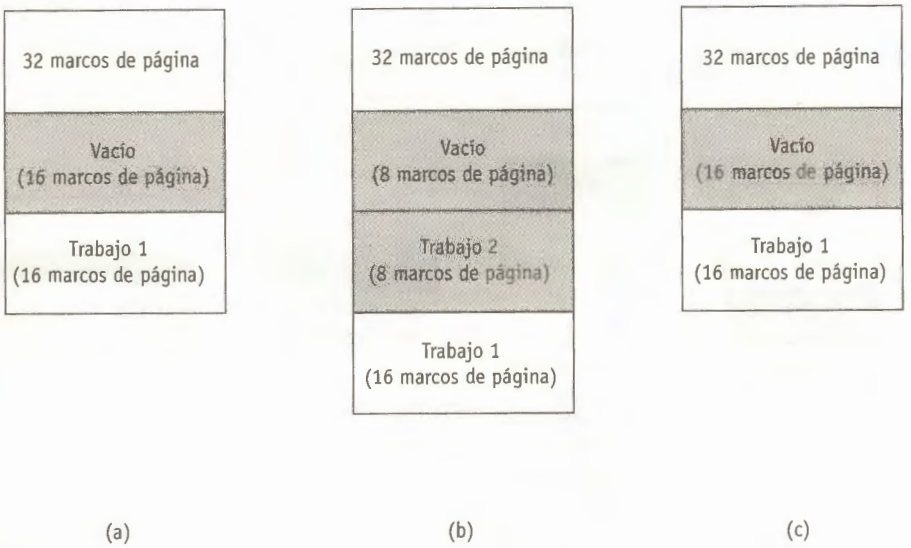


(Figura 16.2)
La administración de la memoria virtual usa tres niveles de tablas (directorios principal, de en medio y de tabla de páginas) para localizar el marco de página con la instrucción o datos solicitados dentro de un trabajo.

En Linux, la memoria virtual se implementa a través de paginación por demanda. Es posible configurar hasta un total de 256 MB de memoria utilizable en marcos de páginas del mismo tamaño, que pueden agruparse para proporcionar más espacio contiguo a un trabajo. El proceso de agrupar y separar se denomina **algoritmo amistoso**, que funciona como sigue.

Considere el caso en que la memoria principal consta de 64 marcos de página y el Trabajo 1 solicita 15 marcos de página. El algoritmo amistoso primero redondea la solicitud a la siguiente potencia de 2 (en este caso, 15 se redondea a 16, que es 2^4). Luego, el grupo de 64 marcos de página se divide en dos grupos de 32, y luego la sección inferior se divide a la mitad. Ahora hay un grupo de 16 marcos de página que pueden satisfacer la solicitud, de modo que las 16 páginas del trabajo se copian en marcos como se muestra en la figura 16.3*a*.

Cuando el siguiente trabajo, el Trabajo 2, solicita 8 marcos de página, el segundo grupo de 16 marcos de página se divide en 2 y la mitad inferior con 8 marcos de página se otorga al Trabajo 2, como se muestra en la figura 16.3*b*. Más tarde, cuando el Trabajo 2 libera sus marcos de página, éstos se combinan con los 8 marcos de página superiores para formar un grupo de 16 marcos de página, como se muestra en la figura 16.3*c*.



(Figura 16.3)

La memoria principal se divide para dar cabida a trabajos de tamaños diferentes. En (a) el grupo original de 32 marcos de página se divide para satisfacer la solicitud del trabajo 1 para 16 marcos de página. En (b) otro grupo de 16 marcos de página se divide para dar cabida al trabajo 2, que requiere 16 marcos de página. En (c), después que termina el trabajo 2, cada uno de los dos grupos de 8 marcos de página se recombina en un grupo de 16, mientras continúa el procesamiento del trabajo 1.

El algoritmo de remplazo de páginas es una versión ampliada de la **política de remplazo de páginas clock** analizado en el capítulo 3. En lugar de usar un bit de referencia único, Linux usa un byte de 8 bits para seguir la pista de la actividad de una página, lo que se denomina su edad. Cada que se hace referencia a una página, se incrementa esta variable edad. Detrás del escenario, a intervalos específicos, el administrador de la memoria verifica cada una de las variables de edad y disminuye su valor por uno. Como resultado, si a una página no se hace referencia a menudo, entonces su variable de edad desciende a cero y la página se vuelve candidato para remplazo si es necesario un intercambio de página. Por otra parte, una página que se usa a menudo tiene un alto valor de edad y constituye un buen candidato para intercambio. En consecuencia, puede verse que Linux usa una forma de la política de remplazo de la menos usada recientemente (LFU, por sus siglas en inglés).

Administración del procesador

Linux usa el mismo procedimiento de diseño padre-hijo encontrado en UNIX y descrito en el capítulo 13, pero también atiende el concepto de "personalidad" para permitir la ejecución de procesos provenientes de otros sistemas operativos. Esto significa que a cada proceso se asigna un dominio de ejecución que especifica la forma en que se llevan a cabo las llamadas al sistema y la forma en que se envían los mensajes a los procesos.

Organización de la tabla de procesos

A cada proceso se hace referencia sólo por medio de un descriptor, que contiene aproximadamente 70 campos descriptores que describen los atributos del proceso junto con la información necesaria para administrar el proceso. El núcleo asigna de manera dinámica estos

descriptores cuando el proceso empieza su ejecución. Todos los descriptores de procesos están organizados en una lista de enlace doble, y los descriptores de los procesos que están listos o en ejecución se colocan en otra lista de enlace doble con los campos indicando "siguiente ejecución" o "previamente ejecutado". Hay varias instrucciones macro usadas en el planificador para administrar y actualizar estas listas descriptoras de procesos según sea necesario.

Sincronización de procesos

Linux proporciona colas de espera y semáforos para permitir que dos procesos se sincronicen entre sí. Una cola de espera es una lista circular de descriptores de procesos. Los semáforos, descritos en el capítulo 6, se usan para resolver problemas de exclusión mutua y los problemas de consumidores y productores. En Linux, la estructura de semáforos contiene tres campos: el contador de semáforo, el número de procesos en espera y la lista de los procesos que están esperando el semáforo. El contador de semáforo sólo puede contener valores binarios, excepto cuando están disponibles varias unidades de un recurso, y luego el contador de semáforo asume el valor de las unidades a las que se está accediendo de manera concurrente.

Administración de procesos

El planificador Linux explora la lista de procesos que están en el estado LISTO y, usando criterios predefinidos, escoge cuáles procesos ejecutar. El planificador tiene tres tipos de planificación diferentes: dos para procesos en tiempo real y uno para procesos normales. La combinación de tipos (mostrada en la tabla 16.4) y la prioridad son usadas por el planificador para determinar la política de planificación usada en los procesos en la cola LISTO.

(Tabla 16.4)
Tres tipos de proceso con tres niveles de prioridad diferentes.

Nombre	Nivel de prioridad	Tipo de proceso	Política de planificación
SCHED_FIFO	Prioridad máxima	Para procesos no preferentes en tiempo real	Sólo primero en entrar, primero en salir
SCHED_RR	Prioridad media	Para procesos preferentes en tiempo real	Round robin y prioridad
SCHED_OTHER	Prioridad mínima	Para procesos normales	Sólo prioridad

De entre los procesos con la prioridad máxima (SCHED_FIFO), el planificador escoge el proceso con la prioridad máxima y lo ejecuta usando el algoritmo primero en entrar, primero en salir. Este proceso suele no ser preferente y se ejecuta hasta su terminación a menos que ocurra una de las siguientes situaciones:

- El proceso entra en un estado de ESPERA (en espera de E/S, o algún otro evento, para terminar).
- El proceso renuncia al procesador voluntariamente, en cuyo caso el proceso se mueve hacia un estado de ESPERA y se ejecutan otros procesos.

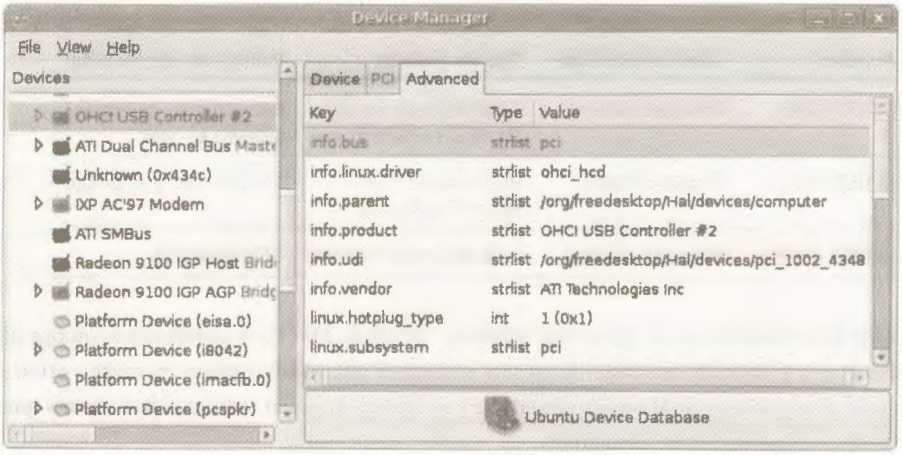
El planificador procede a ejecutar procesos con prioridad inferior sólo cuando se completan los procesos FIFO.

Cuando el planificador ejecuta un proceso del segundo tipo (SCHED_RR), escoge aquellos de este grupo con la prioridad máxima y usa un algoritmo *round robin* con un cuanto de tiempo pequeño. Luego, cuando expira el cuanto de tiempo, es posible escoger y ejecutar otros procesos (como un tipo FIFO u otro tipo de RR con una prioridad más alta) antes que se permita la ejecución del primer proceso hasta su terminación.

El tercer tipo de proceso (SCHED_OTHER), tiene la prioridad mínima y sólo se ejecuta cuando en la cola LISTO no hay otros procesos. De entre estos procesos, el planificador escoge los procesos en orden después de considerar sus prioridades dinámicas (que son fijadas por el usuario que usa llamadas al sistema y por un factor calculado por el sistema). De entre los procesos SCHED_OTHER, las prioridades de todos los procesos que son **limitados por la CPU** disminuye durante la ejecución; en consecuencia, pueden asumir una prioridad inferior a la de los procesos que no se están ejecutando o de aquéllos cuya prioridad no ha disminuido.

Administración de dispositivos

Linux es **independiente de dispositivos**, lo cual mejora su portabilidad de un sistema a otro. Los **controladores de dispositivos** supervisan la transmisión de datos entre la memoria principal y la unidad periférica. A los dispositivos se asigna no sólo un nombre sino también descriptores que identifican aún más cada dispositivo y se almacenan en el directorio de dispositivos, como se muestra en la figura 16.4.



(Figura 16.4)
A los detalles sobre cada dispositivo puede accederse mediante el administrador de dispositivos.

Clasificaciones de dispositivos

Linux identifica cada dispositivo mediante un número menor y un número mayor.

- El *número menor de dispositivo* se pasa al controlador de dispositivos como un argumento y se usa para tener acceso a uno de varios dispositivos físicos idénticos.
- El *número mayor de dispositivo* se usa como un índice para que el arreglo tenga acceso al código idóneo para un número de controlador de dispositivo específico.

Cada *clase* tiene una tabla de configuración que contiene un arreglo de puntos de entrada hacia los controladores de dispositivos. Esta tabla es la única conexión entre el código del sistema y los controladores de dispositivos y es una característica importante del sistema operativo porque permite que los programadores del sistema elaboren nuevos controladores de dispositivos rápidamente para dar cabida a sistemas configurados de manera diferente.

Versiones estándar de Linux a menudo proporcionan una extensa colección de controladores de dispositivos comunes, pero si el sistema de cómputo incluyera hardware o periféricos que no estén en la lista estándar, sus controladores de dispositivos pueden recuperarse desde otra fuente e instalarse por separado. De manera alternativa, un programador hábil puede escribir un controlador de dispositivos e instalarlo para su uso.

Para los sistemas operativos Linux hay disponibles numerosos controladores de dispositivos a bajo o ningún costo. Para más información, consulte la página *www.linux.org*.

Controladores de dispositivos

Linux atiende las clases normales de dispositivos introducidos por UNIX. Además, Linux permite que nuevas clases de dispositivos atiendan nueva tecnología. Las clases de dispositivos no rígidas son por naturaleza; los programadores pueden escoger crear grandes y complejos controladores de dispositivos para llevar a cabo múltiples funciones, pero no se recomienda este tipo de programación por dos razones: 1) el código puede compartirse entre usuarios Linux y cada vez hay más demanda por varios controladores de dispositivos que por uno solo complicado, y 2) el código modular está más capacitado para portar los objetivos de Linux de escalabilidad y extensibilidad del sistema. En consecuencia, se insta a los programadores a que escriban controladores de dispositivos que maximicen la capacidad del sistema para usar de manera efectiva los dispositivos; ni más ni menos.

Una característica notable de Linux es su capacidad para aceptar nuevos controladores de dispositivos al vuelo, mientras el sistema está en marcha y funcionando. Esto significa que los administradores pueden otorgar más funcionalidad al núcleo al cargar y probar nuevos controladores sin tener que volver a arrancar cada vez para reconfigurar el núcleo. Para comprender mejor el siguiente análisis, por favor recuerde que en Linux los dispositivos se tratan de la misma forma en que se tratan otros archivos.

Abrir y liberar

Los módulos pueden cerrarse sin haber liberado nunca un dispositivo. Si ocurre esto, el módulo *no* se libera.

Dos funciones comunes de los controladores de los dispositivos Linux son *abrir* y *liberar*, que esencialmente asignan y liberan el dispositivo idóneo. Por ejemplo, la operación abrir un dispositivo debe realizar las siguientes funciones:

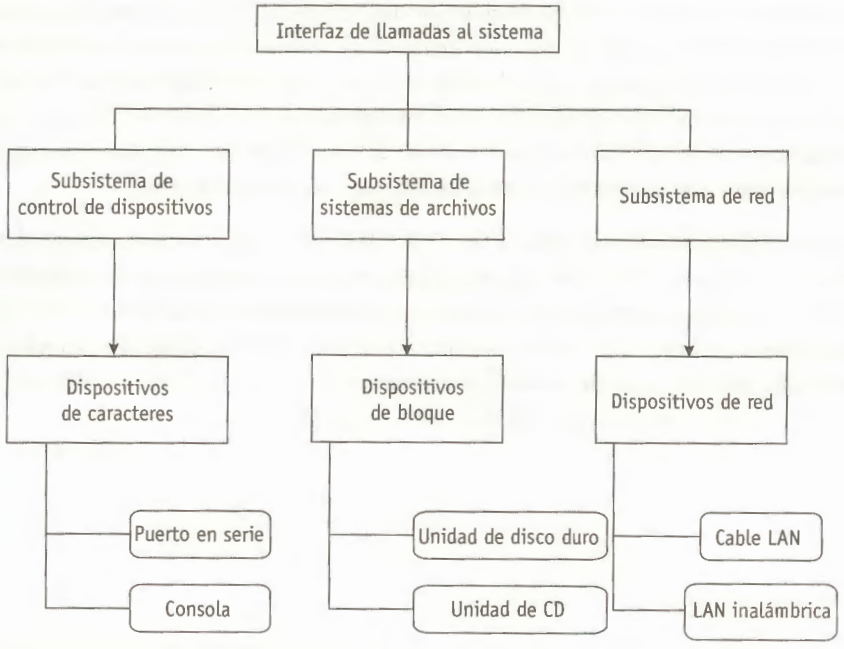
- Verificar que el dispositivo esté disponible y en orden de trabajo.
- Incrementar el contador de uso para el dispositivo en uno, de modo que el subsistema sepa que no es posible cargar el módulo sino hasta que su archivo está cerrado correctamente.
- Iniciar el dispositivo de modo que datos anteriores se eliminen y el dispositivo esté listo para aceptar nuevos datos.
- Identificar el número menor y actualizar el apuntador idóneo en caso de ser necesario.
- Asignar cualquier estructura de datos idónea.

En forma semejante, la función *liberar* (llamada device_close o device_release) lleva a cabo las siguientes funciones:

- Liberar cualquier recurso que esté asignado con la función *abrir*.
- Cerrar el dispositivo.
- Reducir el contador de uso por uno, de modo que el dispositivo pueda liberarse a otro módulo.

Clases de dispositivos

Las tres clases de dispositivos normales que atiende Linux son los dispositivos de caracteres, los dispositivos de bloque y los dispositivos de interfaces de red, como se muestra en la figura 16.5.



(Figura 16.5)
Este ejemplo de los tres tipos esenciales de controladores de dispositivos muestran cómo los controladores de dispositivos reciben dirección desde diferentes subsistemas de Linux.

Dispositivos de caracteres

Los dispositivos de caracteres (también denominados *char devices*) son aquellos a los que puede tenerse acceso como un flujo de bytes, como puerto o monitor de comunicación, o algún otro dispositivo alimentado por flujo de bytes. Como mínimo, los controladores para estos dispositivos suelen implementar las llamadas al sistema abrir, cerrar, leer y escribir, aunque a menudo se agregan llamadas adicionales. A los dispositivos de caracteres puede accederse mediante nodos del sistema de archivos y, desde un punto de vista funcional, estos dispositivos se ven como un área de datos normal. Sus controladores se tratan de la misma manera que los archivos normales, con la excepción de que los controladores de los dispositivos de caracteres son canales de datos que deben accederse secuencialmente.

Dispositivos de bloque

Los dispositivos de bloque son semejantes a los dispositivos de caracteres, excepto que pueden alojar un sistema de archivos, como un disco duro. (Los dispositivos de caracteres no pueden alojar un sistema de archivos.) Como los dispositivos de caracteres, a los dispositivos de bloque puede accederse mediante nodos del sistema de archivos en el directorio/dev, pero estos dispositivos se transfieren en bloques de datos. A diferencia de la mayor parte de los sistemas UNIX, los datos en un sistema Linux pueden transferirse en bloques de cualquier tamaño, desde unos cuantos bytes hasta muchos. Como los controladores de los dispositivos de caracteres, los controladores de los dispositivos de bloque se ven como archivos normales con la excepción de que los controladores de bloques pueden tener acceso a un sistema de archivos en conexión con el dispositivo, algo que no es posible con los dispositivos de caracteres.

Interfaces de red

Las interfaces de red son diferentes a los dispositivos de caracteres y a los dispositivos de bloque porque su función consiste en enviar y recibir paquetes de información según dirige el subsistema del núcleo de red. Así, en lugar de llamadas *leer* y *escribir*, las funciones de los dispositivos de red se relacionan con la transmisión de paquetes.

Cada dispositivo del sistema es manipulado por un controlador de dispositivos que está, a su vez, bajo la dirección de un subsistema de Linux.

Administración de archivos

Estructuras de datos

Todos los archivos Linux están organizados en directorios conectados entre sí en una estructura semejante a árbol. Linux especifica cinco tipos de archivos, como se muestra en la tabla 16.5.

Tipo de archivo	Funciones del archivo
Directorio	Archivo que contiene listas de nombres de archivos.
Archivo normal	Archivo que contiene datos o programas pertenecientes a usuarios.
Enlace simbólico	Archivo que contiene la ruta de otro archivo al que se está enlazando. (Este enlace no es estrictamente directo. Más bien, se trata de información sobre cómo localizar un archivo específico y enlazarlo incluso si está en directorios de usuarios diferentes. Esto es algo que no puede hacerse con enlaces duros.)
Archivo especial	Archivo que está asignado a un controlador de dispositivos ubicado en el núcleo. Cuando se tiene acceso a este tipo de archivo, el dispositivo físico asociado con el archivo se activa y pone en servicio.
Tubería nombrada	Archivo que se usa como canal de comunicación entre varios procesos para intercambiar datos. La creación de una tubería nombrada es lo mismo que la creación de cualquier archivo.

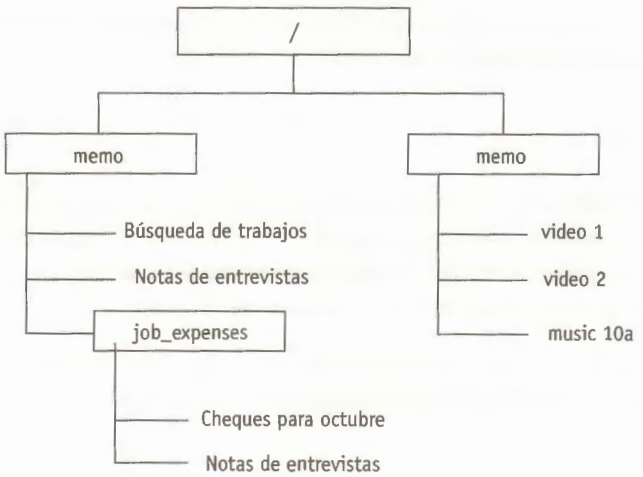
(Tabla 16.5)

El tipo de archivo indica la forma en que debe usarse cada archivo.

Convenciones de nomenclatura de archivos

Los nombres de archivo son sensibles al tipo de letra, de modo que Linux reconoce letras mayúsculas y minúsculas en aquéllos. Por ejemplo, cada uno de los siguientes nombres de archivo son reconocibles como cuatro archivos diferentes alojados en un solo directorio: FIREWALL, firewall, Firewall y fireWALL.

Aunque algunos sistemas operativos usan una barra diagonal invertida (\) para separar nombres de carpetas, Linux usa una barra diagonal (/).



(Figura 16.6)

Ejemplo de jerarquía de archivos. La barra diagonal (/) representa el directorio raíz.

Los nombres de archivo pueden ser hasta de 255 caracteres de largo y contener caracteres alfabéticos, guiones bajos y números. Los sufijos de archivo (semejantes a las extensiones analizadas en el capítulo 8) son opcionales. Los nombres de archivo pueden incluir un espacio, aunque esto puede ocasionar complicaciones si se están ejecutando programas desde la línea de comandos porque un programa denominado *notas de entrevistas* puede considerarse como un comando para ejecutar dos archivos: *notas* y *entrevistas*. Para evitar confusiones, ambas palabras deben escribirse entre comillas: "*notas de entrevistas*". (Esto es importante cuando Linux se usa en modo de terminal por medio de su shell intérprete de comandos.

Los nombres de archivo que empiezan con uno o dos puntos se consideran archivos ocultos y no se enumeran con los comandos ls o ls –l.

Desde una GUI de escritorio Linux, los usuarios escogen nombres de una lista, de modo que rara vez es necesario escribir el nombre del archivo.)

Para copiar el archivo denominado **cheques para octubre**, ilustrado en la figura 16.5, el usuario puede escribir desde cualquiera otra carpeta:

```
cp /memo/job_expenses/cheques para octubre
```

La primera barra diagonal indica que ésta es una ruta absoluta que empieza en el directorio raíz. Si el archivo que se está buscando está en un directorio local, puede usarse una ruta relativa; una que no empiece en el directorio raíz. Dos ejemplos de rutas relativas a partir de la figura 16.6 son:

```
Job_expenses/cheques para octubre
memo/music 10a
```

Para todas las rutas son válidas algunas reglas:

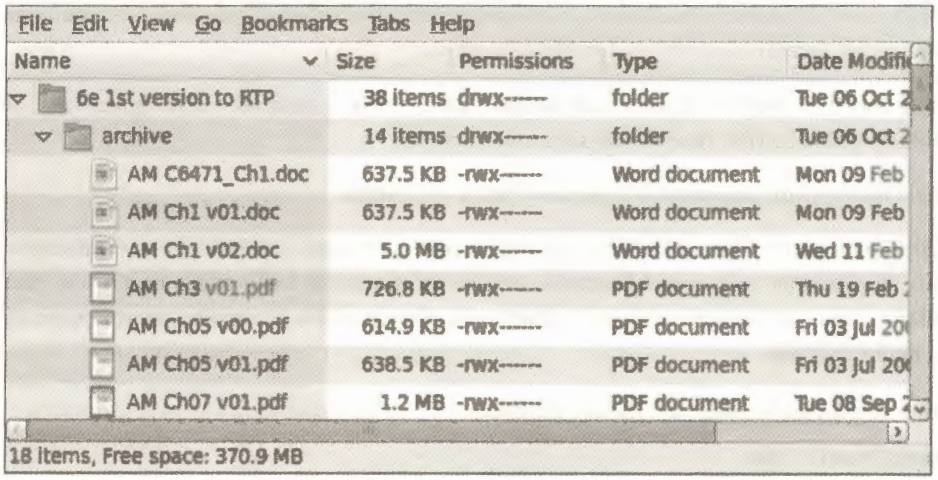
1. Si la ruta empieza con una barra diagonal, la ruta empieza en el directorio raíz.
2. Una ruta puede ser un nombre o una lista de nombres separados por barras diagonales. El último nombre en la lista es el nombre del archivo solicitado. Todos los nombres que preceden al nombre del archivo deben ser nombres de directorio.
3. Al usar dos puntos (..) en una ruta se efectúa un movimiento hacia arriba en la jerarquía (más cerca de la raíz). Esta es la única forma para ascender en la jerarquía; todas las demás rutas descienden por el árbol.

Estructuras de datos

Para permitir que los procesos tengan acceso a archivos de manera consistente, el núcleo tiene una capa de software que mantiene una interfaz entre llamadas al sistema relacionadas con archivo y el código de administración de archivos. Esta capa se denomina sistema de archivos virtual (VFS). Cualquier llamada al sistema iniciada por el proceso solicitando archivos se dirige al VFS, que efectúa operaciones de archivos independientes del formato del sistema de archivos implicado. Luego, el VFS redirige la solicitud al módulo que administra el archivo.

Listas de directorios

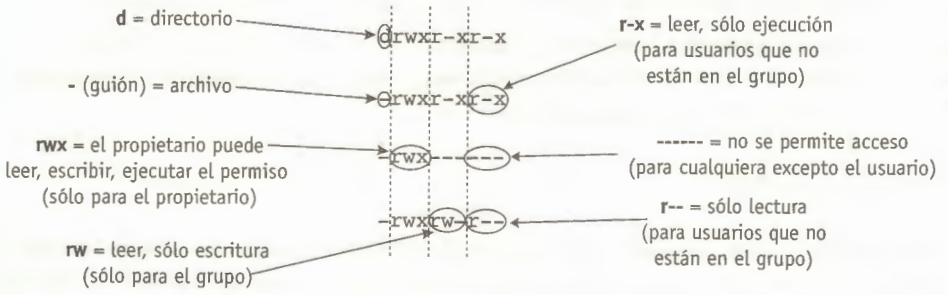
Aunque las listas de directorios pueden crearse desde el modo terminal usando comandos escritos (**ls** o **ls -1**), la forma más fácil de enumerar archivos en directorios es desde el escritorio de la GUI. Un listado típico muestra el nombre del archivo o directorio, su tamaño, la fecha y hora de modificación. Debe añadirse información sobre permisos al archivo, como se muestra en la figura 16.7, desde la opción View de la barra de menús.



(Figura 16.7)

Lista muestra de archivos almacenados en un directorio, incluyendo permisos para archivos.

La columna *Permissions* muestra un código con el tipo de archivo y privilegios de acceso, como se muestra en la figura 16.8. Para comprender el tipo específico de acceso concedido, observe el orden de las letras en esta columna. (Esta misma información se muestra en el listado de directorios generado al usar el comando *listing* en el modo terminal.)



(Figura 16.8)

Descripción gráfica de una lista de los permisos a archivos y directorios.

El primer caracter en la columna describe la naturaleza de la entrada de la carpeta.

- El guión (-) indica un archivo.
- d indica un archivo del directorio.
- l indica un enlace.
- b indica un archivo especial de bloque.
- c indica un archivo especial de caracter.

Los tres caracteres siguientes (**rwx**) muestran los privilegios de acceso otorgados al *propietario* del archivo:

- r indica acceso para **lectura.**
- w indica acceso para **escritura.**
- x indica acceso para **ejecución.**

En forma semejante, los tres caracteres siguientes describen los privilegios de acceso concedidos a otros miembros del *grupo de usuarios*. (Un "grupo" se define como un conjunto de usuarios, excluyendo al propietario, que tienen algo en común: el mismo proyecto, el mismo

curso, el mismo departamento, etcétera.) En consecuencia, **rwx** para los caracteres 5-7 significa que los usuarios del grupo también pueden leer, escribir y/o ejecutar ese archivo, y un guión (-) indica que se ha denegado el acceso para esa operación.

Finalmente, los tres últimos caracteres describen los privilegios de acceso concedidos a "otros", definidos como usuarios en general (aunque excluyendo al propietario y a los miembros del grupo del propietario). El grupo amplio de usuarios del sistema algunas veces se denomina "mundo".

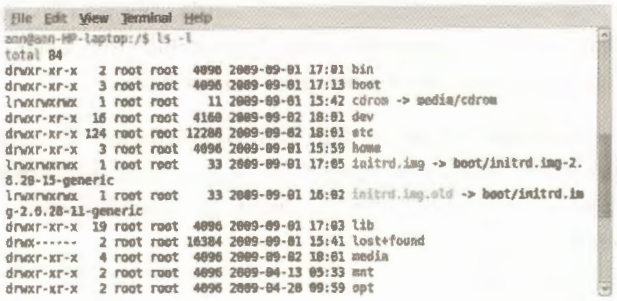
Interfaz de usuario

Las primeras versiones de Linux requerían comandos escritos y amplio conocimiento de comandos válidos, como se muestra en la tabla 16.6. Aunque la mayor parte de las versiones actuales incluyen escritorios de GUI poderosas e intuitivas, que se describirán dentro de poco, que permiten incluso a los programadores novatos navegar exitosamente en el sistema operativo, es importante darse cuenta que los usuarios pueden seguir usando el modo terminal para escribir comandos que son muy semejantes a los que se usan para UNIX, los cuales pueden ser muy útiles para migrar de un sistema operativo controlados por comandos.

(Tabla 16.6)

Comandos muestra del usuario, que pueden abreviarse y deben escribirse con el tipo de letra correcto (por lo general, minúsculas). Muchos comandos pueden combinarse en una sola línea para potencia y flexibilidad adicionales. Compruebe la documentación técnica para su sistema respecto a ortografía y sintaxis correctas.

Comando	Significa	Acción a realizar
(nombre de archivo)	Ejecutar archivo	Correr/ejecutar el archivo con ese nombre.
ls	Enlistar directorio	Mostrar una lista de los nombres de archivo en el directorio.
ls -l	Lista larga	Mostrar una lista exhaustiva del directorio.
ls /bin	Lista/papelera del directorio	Mostrar una lista de comandos válidos.
cd	Cambiar directorio	Cambiar el directorio de trabajo.
chmod	Cambiar permisos	Cambiar permisos en un archivo o directorio de trabajo.
cp	Copiar	Copiar un archivo en otro archivo o directorio.
mv	Mover	Mover un archivo o directorio.
more	Mostrar más	Escribir el contenido del archivo en la pantalla.
lpr	Imprimir	Imprimir un archivo.
date	Fecha	Mostrar fecha y hora.
mkdir	Hacer un directorio	Hacer un nuevo directorio.
grep	Expresión global regular/imprimir	Encontrar una cadena específica en un archivo.
cat	Concatenar o catenar	Concatenar los archivos e imprimir el archivo resultante.
diff	Diferente	Comparar dos archivos.
pwd	Imprimir el directorio de trabajo	Imprimir el nombre del directorio de trabajo.



(Figura 16.9)

En el modo Terminal, los usuarios pueden ejecutar el sistema operativo utilizando comandos en lugar de un menú guiado GUI.

Interfaces basadas en órdenes

La sintaxis general para comandos escritos es la siguiente:

```
command arguments filename
```

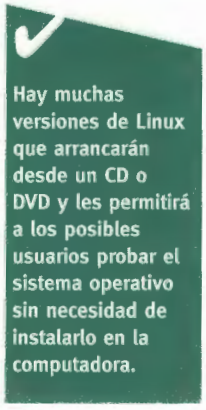
- El `command` es cualquier comando legal del sistema operativo.
- Los `arguments` se requieren para algunos comandos y son opcionales para otros.
- El `filename` puede ser el nombre de un archivo y puede incluir una ruta relativa o absoluta.

Los comandos se interpretan y ejecutan por el shell (como el Bash shell), uno de los dos programas más usados. Shell se conoce técnicamente como el intérprete de comandos, pero no sólo es un intérprete de comandos interactivo; también constituye la clave para la coordinación y combinación de los programas del sistema.

Interfaces gráficas de usuario

La mayor parte de los sistemas operativos Linux son lanzados con múltiples interfaces gráficas de usuario (a menudo gratis), lo cual permite que los usuarios extremos escojan la GUI que mejor se ajuste a sus necesidades o a las de la organización. De hecho, en ciertos entornos, es posible usar interfaces gráficas de usuario diferentes en el mismo sistema. Esta flexibilidad ha estimulado la cada vez más amplia aceptación de Linux y ha ayudado a que se vuelva aún más competitivo.

Además de las interfaces gráficas de usuario, muchas versiones de Linux también vienen equipadas con sofisticados procesadores de palabras compatibles con Windows y hojas de cálculo y aplicaciones de presentaciones; algunas sin costo. Estas herramientas de software hacen posible que los usuarios de Linux lean y escriban documentos que son generados, o leídos, por colegas que usan software registrado de distribuidores de sistemas operativos de la competencia. Debido a que los programas de la competencia pueden costar mucho dinero, la disponibilidad de estas aplicaciones accesibles es un factor que ha estimulado la popularidad de Linux.

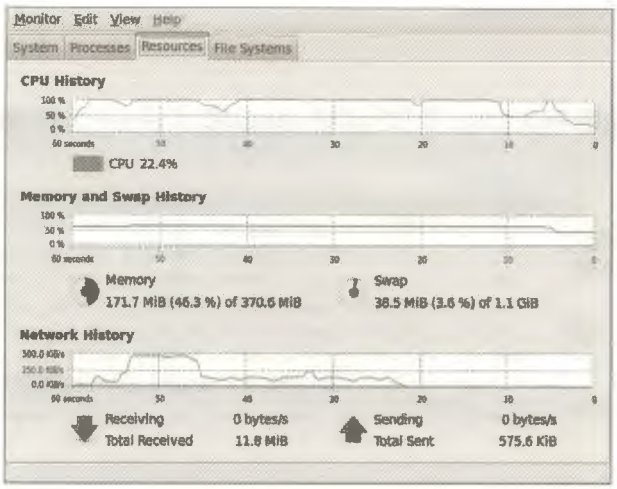


Monitor de sistema

La información sobre el estado del sistema está disponible cuando se usa la ventana System Monitor, mostrada en la figura 16.10, donde se ve la historia inmediata de la CPU, la memoria y el uso de la red. Otra información disponible desde la ventana incluye sistemas de archivos atendidos e información sobre procesos en ejecución en ese momento.

(Figura 16.10)

El monitoreo del sistema muestra la información histórica acerca de la CPU, la memoria y uso de la red.



Configuración de servicios

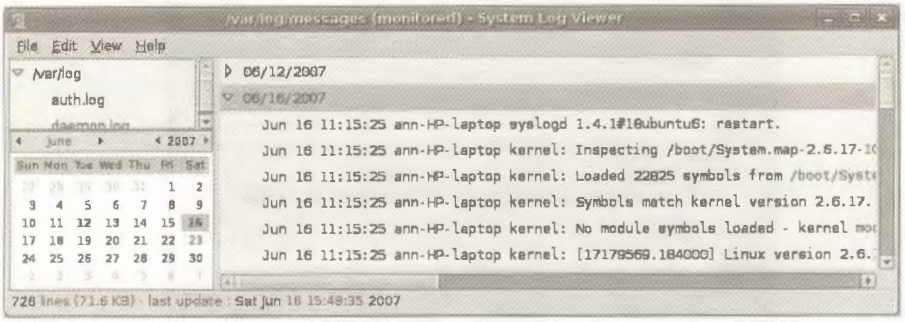
Dependiendo de la distribución de Linux, los administradores pueden implementar varios servicios para ayudar a administrar el sistema. En la figura 16.11 se presenta una muestra de los servicios, aunque las opciones pueden variar de un sistema a otro. Para detalles específicos, consulte la documentación para su sistema.

(Figura 16.11)

En la Ventana de configuración de servicios, hay muchas aplicaciones disponibles para su activación.

Sistema de registros

Los administradores usan el sistema de registros para contar con una descripción detallada de la actividad del sistema. Estos registros constituyen una fuente invaluable para que los administradores sigan la pista del curso de un malfuncionamiento del sistema, fallo de cortafuegos, dispositivos inhabilitados, y más. Estos archivos de registro para algunos sistemas operativos Linux pueden encontrarse en el directorio **/var/log**. En la figura 16.12 se presenta un System Log Viewer muestra.



(Figura 16.12)
Muestra de System Log Viewer.

Hay numerosos archivos de registros disponibles para su revisión (sólo para quien tenga acceso raíz) si se usa un editor de texto simple. En la tabla 16.7 se muestran algunos archivos de registros típicos.

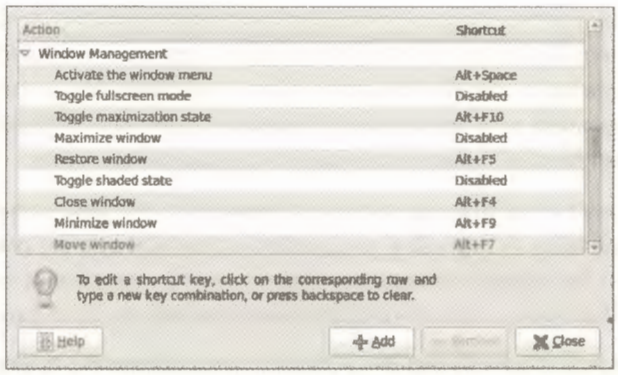
boot.log	Almacena mensajes de cuáles sistemas han empezado y se han apagado exitosamente, así como de los que han fallado.
dmesg	Lista de mensajes creada por el núcleo cuando se inicia el sistema.
maillog	Almacena las direcciones que reciben y envían mensajes por correo electrónico a fin de detectar el mal uso del sistema de correo electrónico.
secure	Contiene listas de todos los intentos por entrar al sistema, incluyendo fecha, hora y duración de cada intento por tener acceso.
xferlog	Enumera el estado de los archivos que han sido transferidos usando un servicio ftp.

(Tabla 16.7)
Archivos de registros típicos Linux. Para detalles específicos de su sistema, consulte la documentación correspondiente.

Atajos de teclado

Para permitir que los usuarios cambien fácilmente de una tarea a otra, Linux atiende atajos de teclado (mostrado en la figura 16.13), muchos de los cuales son idénticos a los de uso común en sistemas operativos Windows, facilitando la transición de un sistema operativo a otro. Por ejemplo, CTRL-V es una forma rápida de emitir un comando PEGAR en Linux, UNIX y Windows.

(Figura 16.13)
Atajos de teclado.

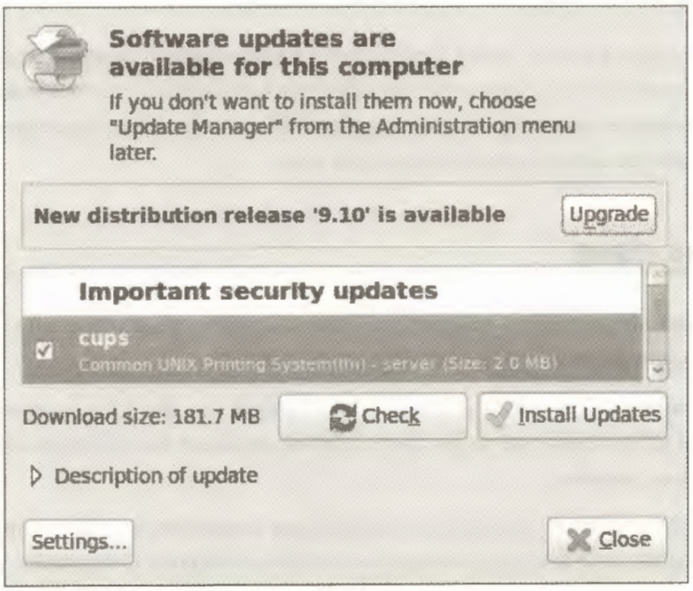


Aquí sólo hemos contemplado una pequeña muestra de las muchas características disponibles en un escritorio Linux típico. Tal vez su sistema tenga ventanas, menús, herramientas y opciones diferentes. Para consultar detalles sobre su sistema operativo Linux, vea por favor el menú de ayuda que suele estar disponible desde el escritorio o desde el menú de su sistema.

Administración de sistemas

Todos los sistemas operativos Linux son parchados entre el lanzamiento de una versión y otra. Estos parches pueden ser descargados por solicitud o los usuarios pueden configurar el sistema y revisar las actualizaciones disponibles, como se muestra en la figura 16.14. La **administración de parches** está diseñada para reemplazar o cambiar los códigos que actua-

(Figura 16.14)
Ubuntu Linux permite a los usuarios verificar actualizaciones de software disponibles.



lizan el software. Las tres razones principales que motivan parchar un sistema operativo son: una gran necesidad de precauciones de seguridad contra los constantes cambios en las amenazas del sistema; la necesidad de asegurar la compatibilidad del sistema con las regulaciones gubernamentales para resguardo de la privacidad y la responsabilidad financiera; y la necesidad de mantener los sistemas funcionando con una eficiencia pico.

Cada administrador de sistemas, sin importar el tamaño de éste, debe permanecer consciente de las vulnerabilidades de seguridad que pueden ser tratadas con los parches adecuados. Después de todo, los intrusos del sistema están en busca de estas mismas vulnerabilidades y señalan como objetivos los dispositivos informáticos que aún no están parchados.

Cuando un parche se vuelve disponible, la primera tarea del usuario es identificar la importancia del parche. Si éste es importante, se debe aplicar inmediatamente. Si el parche no es importante por su naturaleza, su instalación podría demorarse hasta que comience el ciclo regular del mismo. Los ciclos de los parches se analizaron con detalle en el capítulo 12.

Conclusión

Lo que empezó como un esfuerzo individual para obtener más poder de un chip de microcomputadora en la década de 1990 evolucionó hacia un sistema operativo más poderoso y flexible que puede ejecutarse en supercomputadoras, teléfonos celulares y muchos otros dispositivos. Linux disfruta de una popularidad sin par entre programadores, quienes contribuyen a mejorar y superar el conjunto normal del código. Además, debido a que hay una amplia gama de aplicaciones disponibles a costo mínimo y fácil de instalar, Linux ha encontrado una aceptación cada vez más grande entre los usuarios con experiencia de programación mínima. Para aplicaciones en grandes organizaciones, los productos comerciales Linux están disponibles por completo con extenso soporte técnico y ayuda para el usuario.

Linux se caracteriza por su poder, flexibilidad y mantenimiento constante por legiones de programadores a la vez que mantiene una adherencia cuidadosa a normas industriales. Está en vías de demostrar que se trata de un jugador viable en el mercado y se espera que su popularidad aumente durante mucho tiempo por venir.

Términos clave

Administración de parches: Instalación periódica de parches al software para realizar reparaciones y mantener actualizado el sistema operativo.

Algoritmo amistoso: Técnica de asignación de memoria que divide la memoria en mitades para intentar proporcionar un mejor ajuste a fin de satisfacer las solicitudes de memoria de la mejor manera posible.

Argumento: En un sistema operativo controlado por comandos, un valor u opinión colocado en el comando, que modifica la forma en que debe efectuarse el comando.

Comando: Directriz para un programa de computadora que actúa como intérprete de algún tipo para realizar una acción específica.

Controlador de dispositivo: Módulo de programa específico del dispositivo que manipula las interrupciones y controla un tipo particular de dispositivo.

Directorio: Unidad de almacenamiento lógico que contiene archivos.

Independiente del dispositivo: Programas que pueden trabajar en una gran variedad de computadoras y de dispositivos.

Interfaz basada en comandos: Interfaz que acepta comandos escritos, una línea a la vez, de parte del usuario. También se denomina interfaz de línea de comandos y contrasta con una interfaz basada en menús.

Interfaz basada en menús: Interfaz que acepta instrucciones que el usuario escoge de un menú de opciones válidas. También se denomina interfaz gráfica de usuario y contrasta con una interfaz basada en comandos.

Interfaz gráfica de usuario (GUI): Permite que el usuario active comandos del sistema operativo al hacer clic en iconos o símbolos usando un dispositivo apuntador como un ratón. También se denomina interfaz basada en menús.

Interfaz para sistemas operativos migrables basados en UNIX (POSIX): Conjunto de normas IEEE que define las normas de las interfaces del usuario y de programación para sistemas operativos de modo que los desarrolladores puedan portar programas de un sistema operativo a otro.

Limitado por CPU: Trabajo que realiza bastante procesamiento sin detenerse antes de emitir una interrupción. Uno de estos trabajos puede amarrar la CPU durante largos periodos.

Núcleo: Parte del sistema operativo que reside en la memoria principal todo el tiempo y que realiza las tareas más esenciales, como gestionar la memoria y manipular la entrada y salida de disco.

Política de remplazo de páginas de reloj: Variante de la política LRU que elimina de la memoria principal las páginas que muestran la menor cantidad de actividad durante ciclos de reloj recientes.

Búsquedas de interés

- Núcleo Linux
- Fuente de software abierto
- Controladores de dispositivos Linux
- Linux embebido
- Linux para supercomputadoras
- Linux contra UNIX

Ejercicios

Temas de investigación.

A. Investigue las similitudes y las diferencias entre Linux y UNIX. Liste al menos cinco diferencias principales entre los dos sistemas operativos y cite sus fuentes. Diga qué sistema operativo preferiría usted y por qué.

B. Investigue la siguiente declaración "Fuente de software abierto no es software gratuito." Explique si esta declaración es cierta o no y describa las confusiones más comunes acerca de un software abierto. Cite sus fuentes.

Ejercicios.

1. Si usted quisiera agregar estos cuatro archivos a un directorio de Linux (octubre.doc, OCTubre.doc, OCTUBRE.doc y OcTUBre.doc), ¿cuántos archivos nuevos podrían desplegarse: uno, dos, tres o cuatro? Explique por qué esto es así. ¿Piensa usted que la respuesta es la misma para todos los sistemas operativos? ¿Por qué si o por qué no?
2. Linux trata todos los dispositivos como archivos. Explique por qué esta característica agrega flexibilidad a este sistema operativo.
3. En Linux, los dispositivos son identificados por un número mayor o menor de dispositivo. Liste al menos tres tipos de dispositivos que caigan en cada categoría y describa las diferencias entre cada una de las dos categorías.
4. Explique por qué Linux pone el monitoreo del rendimiento del sistema disponible para el usuario.
5. Examinando los permisos de cada uno de los siguientes archivos, identifique si se trata de un archivo o un directorio y describa el acceso permitido para el mundo, usuario y grupo:
   a. `-rwx---r-x`
   b. `drwx------`
   c. `-rwxrwxr--`
   d. `dr-x---r-x`
   e. `-rwx---rwx`
6. Linux utiliza un algoritmo LRU para administración de memoria. Suponga que existe otro algoritmo para el reemplazo de páginas denominado de uso no frecuente (NFU, por sus siglas en inglés) que otorga a cada página su propio contador que se incrementa con cada ciclo del reloj. De esta manera, cada contador rastrea la frecuencia de uso de la página y la página con un menor conteo es cambiada o eliminada cuando es necesario el paginado. En su opinión, ¿cómo se compararían estos dos algoritmos (LRU y NFU)? Explique uno que pueda trabajar mejor bajo un uso normal y defina cómo percibe usted "uso normal".

7. Existen muchas razones por las que el administrador del sistema puede restringir el acceso a las áreas de memoria. Proporcione tres razones que usted crea importantes y clasifíquelas por orden de importancia.
8. Algunas versiones Linux colocan un control de acceso a la información entre las entradas de tabla de página. Explique por qué (o por qué no) esto podría ser un modo eficiente de controlar el acceso a los archivos o directorios.
9. Con relación a la memoria virtual, decida si la siguiente declaración es cierta o falsa: Si el archivo de paginación está ubicado en donde es menos probable que ocurra la fragmentación, puede mejorarse el rendimiento. Explique su respuesta.

Ejercicios avanzados

10. Haga una comparación entre bloque, caracter y dispositivos de red y cómo son manipulados de forma diferente por el administrador de dispositivos de Linux.
11. Describa la circunstancia por la cual un módulo puede cerrarse pero no ser liberado. ¿Qué efecto tiene esta situación en todo el rendimiento del sistema? Mencione los pasos que seguiría para abordar la situación.
12. La seguridad mejorada de Linux (SELinux, por sus siglas en inglés) fue diseñado y desarrollado por un equipo de la agencia nacional de seguridad de Estados Unidos y la industria privada. El sistema operativo resultante, el cual inicia con una serie de parches de seguridad, ha sido incluido en el núcleo de Linux en la versión 2.6. Explique por qué piensa que Linux fue elegido como plataforma base.
13. Existen varias formas de administrar dispositivos. La forma tradicional reconoce los dispositivos del sistema en el orden en el que son detectados por el sistema operativo. Otra es la administración dinámica de dispositivos, que los llama por la creación y eliminación de archivos de dispositivo en el orden en el que el usuario agrega o borra dispositivos. Compare los dos métodos e indique cuál considera que es más efectivo y explique por qué.
14. La administración de dispositivos también incluye la coordinación con la abstracción de capas de hardware (HAL, por sus siglas en inglés). Diga qué dispositivos son administrados por el demonio HAL y cómo comparte actividades con el administrador de dispositivos de Linux.

Apéndice A Código ACM de ética y conducta profesional

Las secciones siguientes son un extracto del Código de ética y conducta profesional adoptado por el Association for Computing Machinery Council el 16 de octubre de 1992. Aquí se reproducen con autorización. Para ver el texto completo, consulte la página *www.acm.org/about/code-of-ethics*.

Nota: Estos imperativos se expresan en forma general para recalcar los principios éticos válidos para ética informática y se deducen a partir de principios éticos más generales.

Preámbulo

Compromiso con la conducta ética profesional que se espera de todos los miembros (los miembros con voto, los miembros asociados y los miembros estudiantes) de la Association for Computing Machinery (ACM).

Este código, que consta de 24 imperativos formulados como declaraciones de la responsabilidad personal, identifica los elementos de ese compromiso. Contiene muchas, pero no todas, de las cuestiones a las que probablemente se enfrenten los profesionales. La Sección 1 describe consideraciones éticas fundamentales, mientras que la Sección 2 aborda consideraciones adicionales más específicas de la conducta profesional. Las declaraciones en la Sección 3 se refieren más específicamente a personas que tienen un papel de liderazgo, ya sea en el lugar de trabajo o en calidad de voluntarios, con organizaciones como ACM. Los principios relativos al cumplimiento de este código aparecen en la Sección 4.

Sección 1. IMPERATIVOS MORALES GENERALES

Como miembro de la ACM yo...

1.1 Contribuir a la sociedad y el bienestar humano

Este principio relativo a la calidad de vida de todas las personas afirma la obligación de proteger los derechos humanos fundamentales y respetar la diversidad de todas las culturas. Un objetivo esencial de los profesionales de la informática es minimizar las consecuencias negativas de los sistemas de cómputo, así como las amenazas a la salud y la seguridad. Al diseñar o implementar sistemas, los profesionales de la informática deben intentar garantizar que los productos de sus esfuerzos se utilicen de manera socialmente responsable, que cumplan las necesidades sociales, y eviten efectos perjudiciales para la salud y el bienestar.

Además de un entorno social seguro, el bienestar humano incluye un entorno natural seguro. En consecuencia, los profesionales de la informática que diseñan y desarrollan sistemas deben estar alerta sobre, y deben crear conciencia en los demás, cualquier daño potencial al medio ambiente local o global.

1.2 Evitar el daño a los demás

"Daño" significa una lesión o consecuencias negativas, como pérdida indeseable de información, pérdida de la propiedad, daños a la propiedad o impactos ambientales no deseados. Este principio prohíbe el uso de tecnología informática de manera que resulte en un daño a cualquiera de los siguientes: usuarios, público en general, empleados, empleadores. Acciones perjudiciales incluyen la destrucción o modificación intencionales de archivos y programas, lo que conduce a una grave pérdida de recursos o gastos innecesarios de recursos humanos, como el tiempo y el esfuerzo requeridos para purgar los sistemas de "virus informáticos".

Acciones bien intencionadas, incluyendo aquellas que cumplen sus obligaciones asignadas, pueden conducir a daño en forma inesperada. En ese caso la persona o las personas responsables están obligadas a deshacer o mitigar las consecuencias negativas tanto como sea posible. Una forma de evitar un daño no intencional es considerar cuidadosamente los impactos potenciales en todos los afectados por las decisiones tomadas durante el diseño y la implementación.

Para minimizar la posibilidad de dañar indirectamente a otros, los profesionales de la informática deben minimizar el mal funcionamiento siguiendo las normas generalmente aceptadas para diseñar y probar el sistema. Además, a menudo es necesario valorar las consecuencias sociales de los sistemas para proyectar la probabilidad de un perjuicio grave a los demás. Si las características del sistema se tergiversan a los usuarios, compañeros de trabajo, o supervisores, el profesional de la informática es responsable de cualquier daño resultante.

En el entorno de trabajo, el profesional de la informática tiene la obligación adicional de informar sobre cualquier signo de peligro del sistema que pudiera resultar en daños personales o sociales graves. Si sus superiores no actúan para reducir o mitigar tales peligros, puede ser necesario "denunciar" para ayudar a corregir el problema o reducir el riesgo. Sin embargo, el hecho de reportar violaciones de manera caprichosa o equivocada puede, de facto ser perjudicial. Antes de reportar violaciones es necesario valorar exhaustivamente todos los aspectos pertinentes del incidente. En particular, la valoración del riesgo y la responsabilidad debe ser creíble. Se sugiere buscar asesoramiento de otros profesionales de la informática. Véase el principio 2.5 respecto a las evaluaciones a fondo.

1.3 Sea honesto y confiable

La honestidad es una componente esencial de la confianza. Sin confianza, una organización no puede funcionar con eficacia. Un profesional de la informática honesto no hace afirmaciones deliberadamente falsas o engañosas acerca de un sistema o su diseño, sino que proporciona información completa de todas las limitaciones y los problemas pertinentes del sistema.

Un profesional de la informática tiene la obligación de ser honesto acerca de sus propias calificaciones y sobre cualquier circunstancia que pueda originar conflictos de intereses.

1.4 Sea justo y emprenda acciones para no discriminar

Los valores de igualdad, tolerancia, respeto por los demás, así como los principios de justicia equitativa, rigen este imperativo. La discriminación por motivos de raza, sexo, religión, edad, discapacidad, origen nacional u otros factores es una violación explícita de la política de la ACM y no será tolerada.

Las desigualdades entre los diferentes grupos de personas pueden resultar del uso o mal uso de información y tecnología. En una sociedad justa, todos los individuos tienen las mismas oportunidades de participar en, o beneficiarse de, la utilización de los recursos informáticos sin distinción de raza, sexo, religión, edad, discapacidad, nacionalidad u otros factores semejantes. Sin embargo, estos ideales no justifican el uso no autorizado de los recursos informáticos ni proporcionan una base adecuada para la violación de cualquier otro imperativo ético de este código.

1.5 Derechos de propiedad de honor incluyendo derechos de autor y de patentes

Violar derechos de autor, patentes, secretos comerciales y los términos de los acuerdos de licencia está prohibido por ley en la mayor parte de las circunstancias. Aun cuando el software no esté tan protegido, estas violaciones son contrarias a la conducta profesional. Copiar software sólo debe hacerse con la debida autorización. La duplicación no autorizada de materiales no debe condonarse.

1.6 Dar crédito idóneo a la propiedad intelectual

Los profesionales de la informática están obligados a proteger la integridad de la propiedad intelectual. En concreto, no se debe tomar el crédito de las ideas o el trabajo de otra persona, incluso en casos en que el trabajo no esté explícitamente protegido por derechos de autor, patentes, etcétera.

1.7 Respetar la privacidad de los demás

La informática y la tecnología de las comunicaciones permiten reunir e intercambiar información personal en una escala sin precedentes en la historia de la civilización. Así, hay un mayor potencial para violar la privacidad de las personas y los grupos. Es responsabilidad de los profesionales mantener la privacidad e integridad de los datos que describen a las personas. Esto incluye tomar precauciones para garantizar la precisión de los datos, así como la protección contra el acceso no autorizado o la divulgación accidental a personas inadecuadas. Además, deben establecerse procedimientos para permitir que las personas revisen sus registros y corrijan imprecisiones.

Este imperativo implica que en un sistema sólo se reúna la cantidad necesaria de información personal, que la retención y los plazos de eliminación de esa información estén claramente definidos y ejecutados, y que la información personal reunida para un propósito específico no se utilice para otros fines sin el consentimiento de la(s) persona(s). Estos principios se aplican a las comunicaciones electrónicas, incluyendo correo electrónico, y prohíben procedimientos electrónicos de captura o control de datos del usuario, incluyendo mensajes, sin el permiso de los usuarios o autorización de buena fe respecto a la operación y mantenimiento del sistema. Los datos del usuario observado durante las tareas normales de operación y mantenimiento del sistema deben tratarse con estricta confidencialidad, excepto en los casos en que constituyen evidencia de violación de la ley, de reglamentos de la organización o de este código. En estos casos, la naturaleza o el contenido de esa información deben ser revelados únicamente a las autoridades correspondientes.

1.8 Confidencialidad del honor

El principio de honestidad se extiende a las cuestiones de confidencialidad de la información cada vez que se ha hecho una promesa explícita de honrar la confidencialidad o, implícitamente, cuando información privada que no está directamente relacionada con el desempeño se vuelve disponible. La preocupación ética es la de respetar todas las obligaciones de confidencialidad a empleadores, clientes y usuarios a menos que estén exentos de tales obligaciones por requisitos legales u otros principios de este código.

Glosario

Abrazo de la muerte: Sinónimo pintoresco de *bloqueo mutuo*.

Acceso directo a la memoria (DMA): Técnica de E/S que permite que una unidad de control tenga acceso a la memoria principal directamente y transfiera datos sin intervención de la CPU.

Acceso Múltiple por Detección de Portadora con Detección de Colisiones (CSMA/CD): Método usado para evitar la transmisión de colisiones en medios compartidos como redes. Suele usarse para detectar la transmisión de colisiones y requiere que las estaciones afectadas dejen de transmitir de inmediato y lo intenten de manera aleatoria después.

Acceso múltiple por detección de portadora con evasión de colisiones (CSMA/CA): Método usado para evitar la transmisión de colisiones en medios compartidos como redes. Suele evitar colisiones al requerir la adquisición de tokens.

Advanced Research Projects Agency network (Arpanet): Red pionera a larga distancia fundada por ARPA (actualmente DARPA). Constituyó la base de las primeras investigaciones de red, así como un pilar durante el desarrollo de Internet. ARPAnet consistía de varias computadoras de conmutación de paquetes interconectadas por líneas arrendadas.

Administrador de archivos: Sección del sistema operativo responsable del control del uso de los archivos. Sigue la pista de todos los archivos en el sistema, incluyendo archivos de datos, ensambladores, compiladores y programas de aplicación. Al usar políticas de acceso predeterminadas, aplica las restricciones de acceso a cada archivo.

Administrador de caché: Componente del sistema de E/S que administra la parte de la memoria virtual conocida como caché. El caché se amplía o reduce dinámicamente dependiendo de la cantidad de memoria disponible.

Administrador de dispositivos: Sección del sistema operativo responsable del control del uso de los dispositivos. Supervisa cada dispositivo, canal y unidad de control, y escoge la manera más eficiente para asignar a todos los dispositivos del sistema.

Administrador de la memoria: Sección del sistema operativo responsable de controlar el uso de la memoria. Verifica la validez de cada solicitud por espacio de memoria y, en caso de ser una solicitud legal, asigna la cantidad necesaria para ejecutar el trabajo.

Administrador de parches: Aplicación rigurosa de parches de software para hacer reparaciones y mantener el software del sistema operativo hasta la última versión estándar.

Administrador de red: Sección del sistema operativo responsable de controlar el acceso a y el uso de los recursos de red.

Administrador del procesador: Composición de dos subadministradores: el planificador de trabajos y el planificador de procesos. Decide la manera de asignar la CPU, monitorea si está ejecutando un proceso o en espera, y controla la entrada de trabajos para asegurar uso equilibrado de los recursos.

Algoritmo: Conjunto de instrucciones paso por paso usadas para resolver un problema particular. Puede plantearse en cualquier forma, como fórmulas matemáticas, diagramas, idiomas naturales o lenguajes de programación.

Algoritmo de Dijkstra: Algoritmo de teoría de gráficas que ha sido usado en varios enlaces de protocolo de encaminamiento de estado. Permite que un encaminador entre en un interfuncionamiento y encuentre la mejor ruta hacia cada destino.

Algoritmo de pila: Algoritmo para el que puede demostrarse que el conjunto de páginas en la memoria para n marcos de páginas siempre es un subconjunto del conjunto de páginas que estarían en la memoria con $n + 1$ marcos de páginas. En consecuencia, incrementar el número de marcos de páginas no provoca la anomalía de Belady.

Algoritmo de planificación de procesos: Algoritmo usado por el planificador de trabajos para asignar la CPU y mover los trabajos a través del sistema.

Algoritmo de planificación: *Véase Algoritmo de planificación de procesos.*

Algoritmo del elevador: *Véase LOOK.*

Algoritmo hashing: Conjunto de instrucciones usadas para realizar una trasformación clave donde un campo clave del registro determina su ubicación. *Véase también Dirección lógica.*

Almacenamiento: Sitio donde se almacenan los datos en el sistema de cómputo. El almacenamiento primario es la memoria principal. El almacenamiento secundario lo constituyen medios no volátiles, como discos y memoria flash.

Almacenamiento contiguo: Tipo de almacenamiento de archivos donde toda la información se almacena en ubicaciones adyacentes en un medio de almacenamiento.

Almacenamiento indexado: Manera en que el gestor de archivos asigna físicamente espacio a un archivo organizado en forma indexada secuencial.

Almacenamiento no contiguo: Tipo de almacenamiento de archivos donde la información se almacena en ubicaciones no adyacentes en un medio de almacenamiento. A los registros de datos puede tenerse acceso directamente al calcular sus direcciones relativas.

Almacenamiento primario: *Véase Memoria principal.*

Almacenar y enviar: Modo operacional de red donde los mensajes se reciben en su totalidad antes de ser transmitidos a su destino, o a su siguiente salto en la ruta hacia su destino.

Anfitrión: (1) Término de Internet para un nodo de red capaz de comunicarse al nivel de la capa de aplicación. Cada anfitrión Internet tiene una dirección IP única. (2) Computadora conectada a una red con un programa o archivos de datos que los ponen a disposición de otras computadoras en la red.

Anomalía Belady: Apunta hacia el hecho de que, para algunos algoritmos de paginación, la tasa de fallos de página puede aumentar a medida que crece el número de marcos de páginas asignado. También se denomina *anomalía FIFO.*

Anomalía FIFO: Circunstancia inusual donde agregar más páginas con marcos provoca un incremento en las interrupciones de página cuando se usa una política de remplazo de páginas FIFO. También se denomina *anomalía Belady.*

Aplazamiento indefinido: Significa que la ejecución de un trabajo se aplaza indefinidamente porque es anticipado de manera repetida, de modo que otros trabajos pueden procesarse.

Apuntador: Una dirección u otro indicador de ubicación.

Árbol-B: Caso especial de una estructura de árbol binario usado para localizar y recuperar registros almacenados en archivos de disco. Las calificaciones impuestas a una estructura de árbol B reducen la cantidad de tiempo necesaria para recorrer un árbol B, lo cual lo hace una organización ideal de archivos para archivos grandes.

Archivo: Grupo de registros relacionados que contiene información a usar por programas de aplicación específicos para generar reportes.

Archivo con acceso directo: *Véase Organización de archivos directa.*

Archivo de datos: Un archivo que contiene sólo datos.

Archivo de programa: Archivo que contiene instrucciones para la computadora.

Archivo invertido: Archivo generado a partir de bases de datos del documento completas. Cada registro en un archivo invertido contiene un tema clave y los números del documento donde se encuentra el tema. El índice de un libro es un archivo invertido.

Argumento: En un sistema operativo controlado por comandos, un valor u opción colocado en el comando, que modifica la manera en que debe efectuarse el comando.

ARPAnet: *Véase Advanced Research Projects Agency network.*

Asignación de la memoria del mejor ajuste: Esquema de asignación de la memoria principal que considera todos los bloques libres y selecciona para asignación el que dé por resultado la menor cantidad de espacio desperdiciado. Contrasta con la *asignación de la memoria del primer ajuste.*

Asignación de memoria paginada: Esquema de asignación de la memoria basado en el concepto de dividir un trabajo de un usuario en secciones del mismo tamaño a fin de permitir almacenamiento no contiguo de programas durante la ejecución. Se implementó para incrementar aún más el nivel de multiprogramación. Contrasta con *asignación de memoria segmentada.*

Asignación de memoria segmentada: Esquema de asignación de la memoria basado en el concepto de dividir un trabajo de un usuario en agrupamientos lógicos de código para permitir almacenamiento no contiguo durante la ejecución. Contrasta con *asignación de memoria paginada.*

Asignación de memoria segmentada/paginación por demanda: Esquema de asignación de la memoria basado en el concepto de dividir el trabajo de un usuario en agrupamientos lógicos de código y cargarlos en la memoria según sea necesario.

Ataque de denegación de servicio (DoS): Ataque a una red que imposibilita la ejecución de funciones para las que estaba diseñada. Esto puede hacerse al inundar al servidor con solicitudes o información sin sentido.

Ataque de diccionario: Técnica mediante la cual un intruso trata de adivinar contraseñas de usuarios al intentar palabras encontradas en un diccionario.

Autenticación: Medios por los cuales un sistema verifica que el intento individual por tener acceso al sistema está autorizado para hacerlo. La protección de contraseñas es una técnica de autenticación.

Base de datos: Grupo de archivos relacionados interconectados a varios niveles para proporcionar a los usuarios flexibilidad de acceso a los datos almacenados.

Biometría: Ciencia y tecnología de identificar usuarios autorizados con base en sus características biológicas.

BIOS: Acrónimo de basic input output system (sistema básico de entrada salida), un conjunto de programas hard-coded en un chip para cargarse en ROM al inicio.

Bit de paridad: Bit extra agregado a un carácter, palabra o alguna otra unidad de datos que se usa para verificación de errores. Se habilita en 0 o 1 de modo que la suma de los bits uno en la unidad de datos siempre sea par, para paridad par, o impar para paridad impar, según la lógica del sistema.

Bloque de control de hilos (TCB): Estructura de datos que contiene información sobre el estado y las características actuales de un hilo.

Bloque de control de procesos (PCB): Estructura de datos que contiene información sobre el estado y las características actuales de un proceso. Todo proceso cuenta con un PCB.

Bloque de índice: Estructura de datos usada para asignación de almacenamiento indexado. Contiene las direcciones de cada sector del disco usado por ese archivo.

Bloquear: Técnica usada para garantizar la integridad de los datos almacenados y de E/S que agrupa los registros individuales en un bloque que se almacena y recupera como una unidad. El tamaño del bloque se establece a menudo para aprovechar la tasa de transferencia.

Bloquear: Técnica usada para garantizar la integridad de los datos en una base de datos a través de la cual el usuario bloquea a todos los otros usuarios mientras trabaja con la base de datos.

Bloqueo activo: Sistema bloqueado donde dos o más procesos bloquean continuamente el avance del otro sin hacer ningún avance en sí. Es semejante a un bloqueo mutuo excepto que ninguno de los procesos está bloqueado o en espera; ambos se encuentran en un estado de cambio continuo.

Bloqueo mutuo: Problema que ocurre cuando los recursos necesarios por algunos trabajos para terminar su ejecución están retenidos por otros trabajos que, a su vez, esperan que otros recursos estén disponibles. El bloqueo mutuo está completo si el resto del sistema llega a una situación de espera como resultado de la retención que los procesos tienen sobre la asignación de recursos. También se denomina *abrazo de la muerte*.

Bomba de tiempo: Virus con un activador vinculado con una fecha dada: año, mes, día u hora que ocasiona su ejecución.

Bomba lógica: Virus con un activador, normalmente un evento, que ocasiona su ejecución.

Bootstrapping: Proceso de arrancar una computadora inactiva al usar un pequeño programa de inicio para cargar otros programas.

Brecha entre bloques (IBG): Espacio sin usar entre bloques de registros en una cinta magnética.

Brecha entre registros (IRG): Espacio sin usar entre los registros de una cinta magnética. Facilita las operaciones de inicio/detención de la cinta.

Búfer doble: Técnica usada para acelerar E/S donde dos buffers están presentes en la memoria principal, canales y unidades de control.

Búfers: Áreas de almacenamiento temporal que residen en la memoria principal, canales y unidades de control. Se usan para almacenar datos leídos desde un dispositivo de entrada antes de ser solicitados por el procesador y para almacenar datos a escribir en un dispositivo de salida.

Bus: (1) Canal físico que enlaza las componentes del hardware y permite la transferencia de datos y señales eléctrica; o bien, (2) enlace de comunicación compartida a la cual pueden conectarse múltiples nodos.

Caballo de Troya: Programa de computadora malicioso con efectos colaterales no intencionales de parte del usuario que ejecuta el programa.

Cabeza de lectura/escritura: Pequeño electroimán usado para leer o escribir datos en un medio de almacenamiento magnético, como un disco o una cinta.

Campo: Grupo de bytes relacionados que el usuario puede identificar con un nombre, tipo y tamaño. Un registro está compuesto por campos.

Campo clave: (1) Un único campo o la combinación de campos en un registro que únicamente identifica a ese registro. (2) El campo que determina la posición de un registro en una secuencia ordenada.

Canal: *Véase Programa de E/S.*

Canal de E/S: Unidad programable especializada ubicada entre la CPU y las unidades de control. Su trabajo consiste en sincronizar la rápida velocidad de la CPU con la lenta velocidad del dispositivo de E/S y viceversa, haciendo posible la superposición de operaciones de E/S con operaciones de la CPU. Los canales de E/S proporcionan una ruta para la transmisión de datos entre las unidades de control y la memoria principal, y controlan esa transmisión.

Capacidad: Máximo nivel de rendimiento de cualquiera de los componentes del sistema.

CD-R: Medio de almacenamiento en disco compacto que puede leerse muchas veces aunque no puede escribirse a la vez.

CD-ROM: Memoria de sólo lectura en disco compacto; medio de almacenamiento de acceso óptico que puede almacenar datos incluyendo gráficos, audio y video. Puesto que es de sólo lectura, no es posible modificar el contenido del disco.

Ciclo de realimentación: Mecanismo para monitorear la utilización de los recursos del sistema de modo que sea posible hacer ajustes.

Ciclo de realimentación negativa: Mecanismo para monitorear los recursos del sistema y, cuando está demasiado congestionado, para indicar al administrador idóneo que aminore la tasa de llegada de los procesos.

Ciclo de realimentación positiva: Mecanismo usado para monitorear el sistema. Cuando el sistema está subutilizado, la realimentación ocasiona el incremento de la tasa de llegada.

Ciclo de reloj: Lapso entre dos tics del reloj del sistema de cómputo.

Cilindro: Para un disco o paquete de discos, es cuando dos o más cabezas de lectura/escritura se posicionan en la misma pista, en la misma posición relativa, sobre sus superficies respectivas.

Cinta magnética: Medio de almacenamiento secundario lineal que fue desarrollado por primera vez para los primeros sistemas informáticos. Sólo permite la recuperación y el almacenamiento secuenciales de registros.

Cliente: Un nodo usuario que solicita y usa varios servicios de la red. Una estación de trabajo que solicita el contenido de un servidor de archivos es un cliente de un servidor de archivos.

C-LOOK: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que es una optimización de C-SCAN.

COBEGIN: Se usa con COEND para indicar a un compilador de multiprocesamiento el inicio de una sección donde las instrucciones pueden procesarse de manera concurrente.

Código compartible: Código ejecutable en el sistema operativo que puede compartirse por varios procesos.

Código Hamming: Código para detectar y corregir errores que mejora bastante la confiabilidad de los datos. Se llama así en honor del matemático Richard Hamming.

Código reentrante: Código que puede usarse por dos o más procesos el mismo tiempo; cada uno comparte la misma copia del código ejecutable pero tiene áreas de datos por separado.

COEND: Usado junto con COBEGIN para indicar a un compilador de multiprocesamiento el fin de una sección donde las instrucciones pueden ser procesadas de manera concurrente.

Cola: Lista enlazada de bloques de control de procesos que indica el orden en que serán atendidos los trabajos o procesos.

Colas de **multinivel:** Esquema de planificación de procesos (usado con otros algoritmos de planificación) que agrupa los trabajos según una característica común. Luego, el procesador se asigna para servir a los trabajos en estas colas en una forma predeterminada.

Colisión: Cuando un algoritmo hashing genera la misma dirección lógica para dos registros con llaves únicas.

Comando filtro: Comando que dirige los datos de entrada desde un dispositivo o archivo, los modifica y luego envía el resultado a una impresora o una pantalla.

Competencia: Problema de sincronización entre dos procesos que compiten por el mismo recurso. En algunos casos puede resultar en corrupción de datos porque no es posible controlar el orden en que serán terminados los trabajos.

Compactación: Proceso de recolección de fragmentos de espacio de memoria disponible en bloques contiguos al mover los programas y los datos en la memoria de una computadora o en almacenamiento secundario. También se denomina *recolección de basura.*

Compartición de recursos: Uso de un recurso por dos o más procesos al mismo tiempo o en momentos diferentes.

Compatibilidad: Capacidad de un sistema operativo para ejecutar programas escritos para otros sistemas operativos o para versiones anteriores del mismo sistema.

Compilador: Programa de computadora que traduce programas un lenguaje de programación de alto nivel (como FORTRAN, COBOL, Pascal, o Ada) a lenguaje de máquina.

Compresión: *Véase Compresión de datos.*

Compresión de datos: Procedimiento usado para reducir la cantidad de espacio requerido para almacenar datos al reducir, codificar o abreviar términos o caracteres repetitivos.

Computadora central: Nombre histórico dado a un sistema de cómputo grande caracterizado por su gran tamaño, alto costo y elevado rendimiento.

Computadora de gama media: Sistema de cómputo de tamaño pequeño a mediano desarrollado para satisfacer las necesidades de instituciones más pequeñas. Originalmente fue desarrollada para sitios con sólo una docena de usuarios. También se denomina minicomputadora.

Concesión de entradas Ticket: Boleto "virtual" otorgado por un servidor Kerberos que indica que al usuario poseedor del boleto puede otorgarse acceso a servidores de aplicaciones específicas. El usuario envía este boleto cifrado al servidor de aplicaciones remoto, que entonces puede examinarlo para verificar la identidad del usuario y autenticar al usuario.

Confiabilidad: (1) Norma que mide la probabilidad de que una unidad no falle durante un periodo de tiempo dado. Es una función del tiempo medio entre fallas; (2) Capacidad de un sistema operativo para responder de manera predecible a condiciones de error, incluso los errores provocados por fallos del hardware; (3) Capacidad de un sistema operativo para protegerse a sí mismo y a sus usuarios de manera activa de daño accidental o deliberado originado por programas del usuario.

Configuración débilmente acoplada: Configuración de multiprocesamiento donde cada procesador tiene una copia del sistema operativo y controla sus propios recursos.

Configuración maestro/esclavo: Configuración de multiprocesamiento asimétrico que consta de un sistema de procesador único conectado a procesadores "esclavos", cada uno de los cuales es administrado por el procesador "amo", que proporciona las funciones y los trabajos de planificación.

Configuración simétrica: Configuración multiprocesamiento donde el planificador del procesador está descentralizado y cada procesador es del mismo tipo. En un área común de la memoria se almacenan una copia única del sistema operativo y una tabla global que enumera cada proceso y su situación, de modo que cada procesador tiene acceso a esta copia. Cada procesador usa el mismo algoritmo de planificación para escoger el siguiente proceso que ejecutará a continuación.

Conjunto de trabajo: Colección de páginas que deben mantenerse en la memoria principal para cada proceso activo en un entorno de memoria virtual.

Conmutación de circuitos: Modelo de comunicación donde una ruta de comunicación dedicada se establece entre dos anfitriones sobre los que se desplazan todos los mensajes. El sistema telefónico es un ejemplo de red de conmutación de circuitos.

Conmutación de paquetes: Modelo de comunicación donde los mensajes se encaminan individualmente entre anfitriones, sin ninguna ruta de comunicación establecida de antemano.

Contención: Situación que se presenta en recursos compartidos donde fuentes de múltiples datos compiten por tener acceso al recurso.

Contraseña: Método de control de acceso definido por el usuario. Normalmente una palabra o cadena de caracteres que un usuario debe especificar en orden para que se le permita iniciar una sesión en un sistema informático.

Control de acceso: El control del acceso del usuario a una red o a un sistema de cómputo.

Control de versión: Rastreo y actualización de alguna versión específica de una pieza de hardware o software.

Controlador de dispositivo: Módulo de programa específico del dispositivo que manipula las interrupciones y controla un tipo particular de dispositivo.

Controlador de tráfico de E/S: Uno de los módulos del subsistema de E/S que supervisa el estado de cada dispositivo, unidad de control y canal.

Controlador del bus universal en serie (USB): Interfaz entre el sistema operativo, los controladores de dispositivos y las aplicaciones, que lee y escribe desde y hacia dispositivos conectados a la computadora mediante el puerto USB. Cada puerto USB puede alojar hasta 127 dispositivos diferentes.

Cortafuegos: Conjunto de hardware y software que disfraza la dirección de la red interna de una computadora o red para controlar la forma en que los clientes desde fuera pueden tener acceso a los servidores internos de la organización.

CPU: *Véase Unidad central de procesamiento.*

Cracker: Persona que intenta acceder a sistemas de cómputo sin autorización. Estas personas suelen ser "maliciosas", en oposición a los "hackers", y cuentan con varios medios a su disposición para irrumpir en un sistema.

Criptografía: Ciencia de la codificación de un mensaje o texto para que un usuario no autorizado no pueda leerlo.

C-SCAN: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de acceso. Es una abreviatura de circular-SCAN.

CSMA/CA: *Véase Acceso Múltiple por Detección de Portadora con Evasión de Colisiones.*

CSMA/CD: *Véase Acceso Múltiple por Detección de Portadora con Detección de Colisiones.*

Cuanto de tiempo: Periodo de tiempo asignado a un proceso. Cuando expira, el proceso es apropiado y al proceso se asigna otro cuanto de tiempo para uso futuro.

DASD: *Véase Dispositivo de almacenamiento con acceso directo.*

Demora de rotación: Sinónimo de *tiempo de búsqueda.*

Depresiones: Pequeñas depresiones sobre la capa reflectora de un CD o un DVD. Cada depresión se interpreta como un 0. Contrasta con las *salientes*, que se interpretan como unos.

Desasignación o liberación: Proceso de liberar un recurso asignado, ya sea espacio de memoria, un dispositivo, un archivo o una CPU.

Descriptor de archivos: Información que se mantiene en el directorio para describir un archivo o la extensión de un archivo. Contiene el nombre del archivo, su ubicación y atributos.

Descriptor de seguridad: Estructura de datos Windows añadida a un objeto que protege al objeto de acceso no autorizado. Contiene una lista de control de acceso y controla auditorías.

Desplazamiento: En un entorno de asignación de la memoria paginado o segmentado, es la diferencia entre una dirección relativa de una página y la dirección verdadera en lenguaje de máquina. Se usa para localizar una instrucción o valores de datos dentro de su marco de página. También se denomina *offset.*

Detección: Proceso de examinar un estado de un sistema operativo para determinar si existe un bloqueo.

Dirección: Número que designa una ubicación particular de la memoria.

Dirección del byte actual (CBA): Dirección del último byte leído. Es usada por el administrador de archivos para tener acceso a registros en almacenamiento secundario y es necesario actualizarlo cada que se accede a un registro, como cuando se ejecuta el comando READ.

Dirección lógica: El resultado de una transformación clave. *Véase también Algoritmo hashing.*

Dirección relativa: En un entorno de organización directa, indica la posición de un registro respecto al inicio del archivo.

Directorio: Área de almacenamiento lógico que contiene archivos.

Directorio activo: Servicio de directorio de Microsoft Windows que ofrece administración centralizada de servicios de aplicación, autenticación y registro de usuarios para sistemas de red distribuidos.

Directorio actual: el directorio o subdirectorio en que está trabajando al usuario.

Directorio de trabajo: El directorio o subdirectorio donde trabaja actualmente el usuario.

Directorio maestro de archivos (MFD): Archivo almacenado inmediatamente después del descriptor de volumen. Enumera los nombres y las características de cada archivo contenido en ese volumen.

Directorio raíz: (1) Para un disco, se trata del directorio al que se tiene acceso por defecto cuando se arranca la computadora; o (2) Para una estructura de directorios jerárquica, es el primer directorio al que tiene acceso un usuario.

Disco: Medio de almacenamiento óptico como CD o DVD.

Disco compacto: *Véase CD.*

Disco duro: Dispositivo de almacenamiento secundario con acceso directo para sistemas de computadoras personales. Suele ser un dispositivo no removible de alta densidad.

Disco óptico: Dispositivo de almacenamiento secundario donde la información se almacena en forma de pequeños huecos denominados "depresiones" dispuesto a lo largo de una pista espiral (en vez de una pista concéntrica como para un disco magnético). Los datos se leen al enfocar un haz láser sobre la pista.

Disponibilidad: Herramienta para la medición de recursos que indica la probabilidad de que el recurso esté listo cuando lo necesite un usuario. Es afectada por el tiempo medio entre fallas y el tiempo medio de reparación.

Dispositivo: Unidad periférica de una computadora, como impresora, plotter, unidad de cinta, unidad de disco o terminal.

Dispositivo compartido: Dispositivo que puede asignarse a varios procesos activos el mismo tiempo.

Dispositivo de almacenamiento con acceso aleatorio: *Véase Dispositivo de almacenamiento con acceso directo.*

Dispositivo de almacenamiento con acceso directo (DASD): Cualquier dispositivo de almacenamiento secundario que puede leerse o escribirse directamente desde o hacia un sitio específico. Algunas veces se denomina *dispositivo de almacenamiento con acceso aleatorio*. Contrasta con un *dispositivo de almacenamiento con acceso secuencial.*

Dispositivo de E/S: Cualquier unidad periférica que permite la comunicación con la CPU por usuarios o programas, incluyendo terminales, impresoras en línea, plotters, lectoras de tarjetas, unidades de cinta y dispositivos de almacenamiento con acceso directo.

Dispositivo dedicado: Dispositivo que puede asignarse sólo a un trabajo a la vez; sirve a ese trabajo durante todo el tiempo que el trabajo está activo.

Dispositivo virtual: Dispositivo dedicado que ha sido transformado en un dispositivo compartido por medio de técnicas de spooling.

Disquete: Disco flexible removible para almacenamiento secundario con acceso directo a bajo costo.

División: Conjunto de franjas consecutivas a través de discos; las divisiones contienen bits de datos y algunas veces bits de paridad, dependiendo del nivel RAID.

DNS: *Véase Sistema de nombre de dominio.*

DO/S: *Véase Sistema operativo distribuido.*

DO/S basado en objetos: Una visión de sistemas operativos distribuidos donde cada unidad de hardware está agrupada con su software operacional requerido, constituyendo un objeto discreto que se manipula como una entidad.

DO/S basado en procesos: Visión de sistemas operativos distribuidos que abarca todos los recursos y procesos del sistema. La administración de procesos se proporciona mediante el uso de procesos cliente/servidor.

DVD: Digital Video Disc (video disco digital), medio de almacenamiento óptico con acceso directo que puede almacenar hasta 17 gigabytes, suficiente para guardar una película de larga duración.

E/S: *Véase entrada/salida.*

Encaminador: Dispositivo que dirige el tráfico entre redes. La decisión de encaminamiento se basa en información de capas de red y en tablas de encaminamiento, construidas a menudo por protocolos de encaminamiento.

Encaminamiento: Proceso de selección de la interfaz idónea y el siguiente salto pare un paquete que se está enviando.

Encriptado: Traducción de un mensaje o datos de su forma original a una forma codificada, escondiendo así su significado y haciéndolo ininteligible sin la llave para descifrarlo. Se usa para mejorar la seguridad del sistema y para protección de datos.

En EJECUCIÓN (En EJECUCIÓN): Condición de un trabajo que significa que el trabajo está ejecutándose.

Enlace: Término genérico para cualquier medio de comunicaciones al que está conectado un nodo de red.

Ensamblador: Programa de computación que traduce programas en lenguaje ensamblador a lenguaje de máquina.

Entrada nula: Entrada vacía en una lista. Asume varios significados con base en la aplicación de la lista.

Envejecimiento: Política usada para asegurar que los trabajos que han estado en el sistema durante bastante tiempo en las colas de nivel inferior terminen por completar su ejecución.

Espera: Estado de un trabajo que significa que el trabajo no puede continuar sino hasta que se asigne un recurso específico o haya terminado una operación de E/S.

Espera activa: Método por el cual los procesos, esperando la ocurrencia de un evento, prueban de manera continua si la condición ha cambiado y permanece en ciclos improductivos consumidores de recursos.

Espera circular: Una de cuatro condiciones para que ocurran bloqueos, a través de la cual cada proceso implicado está esperando por un recurso retenido por otro proceso; cada proceso está bloqueado y no puede continuar, resultando en un bloqueo mutuo.

Espera natural: Término común usado para identificar una solicitud de E/S de un programa en un entorno multiprogramación, que origina que un proceso espere "naturalmente" antes de reanudar su ejecución.

Esquema de asignación: Proceso de asignación de recursos específicos a un trabajo a fin de poder ejecutarlo.

Estación: Cualquier dispositivo que puede recibir y transmitir mensajes en una red.

Estación local: Nodo de red al que está conectado un usuario.

Estación remota: Nodo en el extremo distante de una conexión de red.

Estado de trabajo: Computadora de escritorio conectada a una red de área local que sirve como punto de acceso a esa red.

Estado del proceso: Información almacenada en el PCB del trabajo que indica la condición actual del proceso en ejecución.

Estado inseguro: Situación en que el sistema tiene muy pocos recursos disponibles para garantizar la terminación de por lo menos un trabajo en ejecución en el sistema. Puede conducir a bloqueo mutuo.

Estado seguro: Situación en que el sistema cuenta con suficientes recursos para garantizar la terminación de por lo menos un trabajo en ejecución en el sistema.

Estados de un proceso: Condición de un trabajo a medida que se mueve por el sistema desde el principio hasta el fin de su ejecución: RETENIDO, READY, EN EJECUCIÓN, ESPERA, TERMINADO.

Estrategia de búsqueda: Política predeterminada usada por el manejador de dispositivos de E/S para optimizar los tiempos de acceso.

Estrategias de búsqueda: Algoritmos usados para optimizar el tiempo de búsqueda en dispositivos de almacenamiento con acceso directo. Véase también *Ordenamiento de rotación.*

Ethernet: Norma de 10 megabits, 100 megabits, 1 gigabit o más para LAN, inicialmente desarrollada por Xerox y después por Digital Equipment Corporation, Intel y Xerox. Todos los anfitriones están conectados a un cable coaxial donde compiten por tener acceso a redes usando CSMA/CD.

Ética: Reglas o normas de comportamiento que se espera sigan las personas pertenecientes a la comunidad informática, a fin de demostrar los principios del bien y el mal.

Evasión: Estrategia para evitar bloqueos mutuos. Es una estrategia dinámica que intenta asegurar que los recursos nunca se asignen de manera que pongan un sistema en un estado inseguro.

Exclusión mutua: Una de cuatro condiciones para bloqueo mutuo donde sólo a un proceso se permite tener acceso a un recurso. Suele abreviarse a "mutex" en algoritmos que describen sincronización entre procesos.

Extensibilidad: Uno de los objetivos de diseño de un sistema operativo, que permite su fácil mejoramiento a medida que cambian los requerimientos del mercado.

Extensión: En algunos sistemas operativos, es la parte del nombre del archivo que indica el compilador o paquete de software necesario para ejecutar los archivos. En UNIX y Linux es opcional y se denomina *sufijo*.

Extensiones: Cualquier registro restante y todas las otras adiciones al archivo, que están almacenadas en otras secciones del disco. Las extensiones del archivo están enlazadas con apuntadores.

Fallo de página: Tipo de interrupción del hardware provocada por una referencia a una página que no está en la memoria. El efecto consiste en sacar de la memoria una página y llevarla a almacenamiento secundario, de modo que sea posible llevar otra página a la memoria.

FCFS: *Véase Primero en llegar, primero en servirse.*

FIFO: *Véase Primero en entrar, primero en salir.*

Firmware: Instrucciones o datos de software que están almacenados de manera fija o "en firme", que suelen implementarse en la *memoria de sólo lectura* (ROM). El firmware está integrado a la computadora para que el usuario comprenda de manera más simple el funcionamiento de ésta.

Flop: Medida de velocidad de procesamiento, que significa operaciones de punto flotante por segundo. *Véase Megaflop, gigaflop, teraflop.*

Fragmentación: Condición en la memoria principal donde existe espacio de memoria desperdiciado dentro de particiones, denominada *fragmentación interna*, o entre particiones, denominada *fragmentación externa.*

Fragmentación externa: Situación en que la asignación dinámica de memoria crea fragmentos no utilizables de memoria libre entre bloques de memoria ocupada o asignada. Contrasta con *fragmentación interna.*

Fragmentación interna: Situación en la que una partición fija sólo es usada parcialmente por el programa. El espacio restante dentro de la partición no está disponible para ningún otro trabajo, por lo que se desperdicia. Contrasta con *fragmentación externa.*

Ftp: Nombre del programa invocado por un usuario para ejecutar el Protocolo de transferencia de archivos.

Ftp anónimo: Uso del protocolo de transferencia de archivos que permite al usuario recuperar documentos, archivos, programas y otros datos desde cualquier parte en Internet

sin necesidad de establecer una identificación y una contraseña del usuario. Al usar la identificación especial "anónimo", al usuario de la red se permite evitar controles de seguridad y archivos accesibles al público en el sistema remoto.

Gigabit: Medida de la velocidad de transmisión de datos igual a 1 073 741 824 bits por segundo.

Gigabyte (GB): Unidad de memoria o espacio de almacenamiento igual a 1 073 741 824 bytes o 2^{30} bytes. Un gigabyte es aproximadamente igual a mil millones de bites.

Gigaflop: Prueba comparativa usada para medir la velocidad de procesamiento. Un gigaflop es igual a mil millones de operaciones de punto flotante por segundo.

Gráficas dirigidas: Modelo gráfico que representa varios estados de asignaciones de recursos. Constan de procesos y recursos conectados por líneas dirigidas (líneas con flechas direccionales).

Groupware: Aplicaciones de software que atienden trabajo cooperativo a través de una red. Los sistemas de groupware deben atender comunicación entre usuarios y procesamiento de información. Por ejemplo, un sistema que cuenta con un editor compartido no sólo debe atender la corrección colectiva de documentos, sino también debates entre los participantes sobre lo que debe corregirse y por qué.

Grupo: Propiedad de los sistemas operativos que permite a los administradores del sistema crear conjuntos de usuarios que comparten los mismos privilegios. Un grupo puede compartir archivos o programas sin permitir a todos los usuarios del sistema acceso a estos recursos.

Gusano: Programa de computadora que se replica a sí mismo y se autopropaga en la memoria principal. Los gusanos, en oposición a los virus, se generan en entornos de red.

Hacker: Persona que disfruta al tener un entendimiento íntimo del modo de funcionamiento de un sistema operativo; en particular computadoras y redes informáticas. A menudo el término se malinterpreta en un contexto peyorativo, donde el término correcto debe ser "cracker".

Hardware: La máquina física y sus componentes, incluyendo la memoria principal, dispositivos de E/S, canales de E/S, dispositivos de almacenamiento con acceso directo y la unidad central de procesamiento.

Hilo: Porción de un programa que puede funcionar independientemente de otras partes. Los programas de aplicación multihilos pueden tener varios hilos en ejecución al mismo tiempo con la misma prioridad o con prioridades distintas.

Husmeador: *Véase Husmeador de paquetes.*

Husmeador de paquetes: Software que intercepta paquetes de datos en texto plano y busca en ellos información, como contraseñas.

Identificación de procesos: Identificador único de procesos proporcionado por el usuario y un apuntador que lo conecta con su descriptor, que está almacenado en el PCB.

Inanición: Resultado de la asignación conservadora de recursos donde se previene la ejecución de un solo trabajo porque se mantiene esperando recursos que nunca se vuelven disponibles. Es un caso extremo de *aplazamiento indefinido*.

Independientes del dispositivo: Programas que pueden trabajar en una variedad de computadoras y con una variedad de dispositivos.

Ingeniería social: Técnica con la que los intrusos del sistema obtienen acceso a información sobre un usuario legítimo para conocer contraseñas activas, algunas veces mediante llamadas al usuario y presentándose como un técnico del sistema.

Inicio de sesión remoto: Capacidad para operar en una computadora remota usando un protocolo sobre una red informática como si estuviese conectada localmente.

Inicio en caliente: Característica que permite que el sistema de E/S recupere operaciones de E/S que estaban efectuándose al momento de ocurrir una falla de suministro eléctrico.

Intentos repetidos: Conjetura repetida de una contraseña de usuario hecha por un usuario no autorizado. Se trata de un método usado para entrar de manera ilegal a sistemas que dependen de contraseñas.

Intercambio de páginas: Proceso de sacar una página de la memoria principal y llevarla a almacenamiento secundario, de modo que en su lugar sea posible introducir otra página en la memoria.

Interfaz controlada por comandos: Interfaz que acepta comandos por escrito, una línea a la vez, del usuario. Contrasta con una interfaz controlada por menú.

Interfaz controlada por menú: Interfaz que acepta instrucciones que los usuarios escogen de un menú de opciones válidas. Contrasta con una *interfaz controlada por comandos*.

Interfaz gráfica de usuario (GUI): Interfaz de usuario que permite al usuario activar comandos del sistema operativo al hacer clic en iconos o menúes del escritorio usando un dispositivo apuntador como un ratón.

Interfaz NetBIOS: Interfaz de programación que permite que las solicitudes de E/S sean enviadas hacia y recibidas desde una computadora remota. Oculta el hardware de red de las aplicaciones.

Internet: La más grande colección de redes interconectadas con encaminadores. Es una jerarquía de tres niveles compuesta por redes troncales (por ejemplo, NSFNET), redes regionales (por ejemplo, NEARnet) y redes locales. Internet es una interred de multiprotocolos.

Interrupción: Señal del hardware que suspende la ejecución de un programa y activa la ejecución de un programa especial denominado manejador de interrupciones. Rompe el flujo normal del programa en ejecución.

Interrupciones externas: Interrupciones que ocurren fuera del flujo normal de la ejecución de un programa. Se usan en políticas de planificación preferenciales para asegurar un uso justo de la CPU en entornos de multiprogramación.

Interrupciones internas: También se denominan interrupciones "síncronas", y ocurren como resultado directo de la operación aritmética o instrucción que se está procesando en ese momento. Contrastan con las *interrupciones externas*.

Interrupciones síncronas: Sinónimo de *interrupciones internas*.

Intervención de la línea telefónica: Violación a la seguridad del sistema donde usuarios no autorizados monitorean o modifican la transmisión de un usuario.

IP: *Véase Protocolo de Internet.*

ISO: *Véase Organización Internacional para la Estandarización.*

Java: Lenguaje de programación multi-plataforma desarrollado por Sun Microsystems, bastante parecido a C++ y que funciona en cualquier computadora capaz de ejecutar el intérprete Java.

Kerberos: Sistema de autenticación desarrollado en el MIT que permite a los administradores de la red administrar la autenticación del usuario a nivel de red.

Kilobyte (K): Unidad de espacio de memoria o almacenamiento igual a 1 024 bytes o 2^{10} bytes.

LAN: *Véase Red de área local.*

Lectores y escritores: Problema que surge cuando dos tipos de procesos requieren tener acceso a un recurso compartido como un archivo o una base de datos. Es necesario controlar su acceso para preservar la integridad de los datos.

Lenguaje de control de trabajos (JCL): Lenguaje de comandos usado en varios sistemas de cómputo para dirigir al sistema operativo en la ejecución de sus funciones al identificar los recursos requeridos para ejecutar un trabajo. El JCL ayuda que el sistema operativo coordine y administre mejor los recursos del sistema.

Lenguaje de programación C: Lenguaje de programación de propósito general desarrollado por D. M. Ritchie. Combina declaraciones de alto nivel con controles de máquina de bajo nivel para generar software que sea fácil de usar y altamente eficiente.

Lenguaje ensamblador: Lenguaje de programación que permite a los usuarios escribir programas usando instrucciones nemotécnicas que pueden traducirse por un ensamblador. Se considera un lenguaje de programación de bajo nivel y es muy dependiente de la computadora.

LFU: *Véase Menos frecuentemente utilizada.*

Limitado por E/S: Trabajo que requiere un gran número de operaciones de entrada/salida, resultando en mucho más tiempo libre para la CPU. Contrasta con *limitado por la CPU.*

Limitado por CPU: Trabajo que requiere un gran número de procesamiento sin detención antes de emitir una interrupción. Un trabajo CPU de la envolvente puede amarrar la CPU durante largos periodos. Contrasta con *trabajo limitado por* E/S.

Limitado por CPU: Trabajo que requiere un gran número de procesamiento sin detención antes de emitir una interrupción. Un trabajo CPU de la envolvente puede amarrar la CPU durante largos periodos. Contrasta con *trabajo limitado por* CPU.

Línea arrendada: Circuito telefónico dedicado por el que suscriptor paga una tarifa mensual, sin importar el uso real.

Lista de capacidad: Método de control de acceso que enumera cada usuario, los archivos a los que tiene acceso cada uno y el tipo de acceso que se permite a estos archivos.

Lista de control de acceso: Método de control de acceso que enumera cada archivo, los nombres de los usuarios a quienes se permite tener acceso al archivo, y el tipo de acceso que se permite a cada uno.

Localidad de referencia: Comportamiento observado en muchos programas de ejecución donde las ubicaciones de la memoria referidas recientemente, y las que están próximas, tienen la probabilidad de ser referidas en el futuro próximo.

LOOK: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de acceso. Algunas veces se denomina algoritmo del elevador.

LRU: *Véase Menos usada recientemente.*

Marcos de página: Secciones individuales de la memoria principal, de tamaño uniforme en las que es posible cargar una sola página.

MAN: *Véase Red de área metropolitana.*

Manejador de dispositivos de E/S: Módulo que procesa las interrupciones de E/S, maneja las condiciones de error y proporciona algoritmos de planificación detallados que son extremadamente dependientes del dispositivo. Cada tipo de dispositivo de E/S cuenta con su propio algoritmo manejador de dispositivos.

Manejador de fallo de página: Parte del administrador de la memoria que determina si en la memoria hay páginas con marcos vacíos de modo que la página solicitada pueda copiarse de inmediato desde el almacenamiento secundario, o determina cuál página debe intercambiarse si todas las páginas con marcos están ocupadas.

Manejador de interrupciones: Programa que controla cuál acción debe emprender el sistema operativo cuando se interrumpe una secuencia de eventos.

Marcos de página: Secciones individuales de la memoria principal, de tamaño uniforme en las que es posible cargar una sola página.

Más usada recientemente (MRU): Algoritmo de eliminación de páginas que elimina de la memoria la página más usada recientemente.

Matriz de control de acceso: Método de control de acceso que usa una matriz con cada archivo (enlistado en renglones) y cada usuario (enlistado en columnas) y el tipo de acceso que se permite a cada usuario para cada archivo, registrado en la celda en la intersección de ese renglón y esa columna.

Medio con acceso secuencial: Cualquier medio que almacene registros sólo de manera secuencial, uno después de otro, como una cinta magnética. Contrasta con *acceso directo*.

Megabyte: Unidad de memoria o espacio de almacenamiento igual a 1 048 576 bytes o 2^{20} bytes.

Megaflop: Prueba comparativa usada para medir la velocidad de procesamiento. Un megaflop es igual a un millón de operaciones de punto flotante por segundo.

Megahertz (MHz): Medida de velocidad usada para comparar la velocidad del reloj de las computadoras. Un megahertz es igual a 1 millón de ciclos eléctricos por segundo.

Memoria asociativa: Denominación dada a varios registros, asignados a cada proceso activo, cuyo contenido asocia varios de los segmentos del proceso y números de página con sus direcciones en la memoria principal.

Memoria caché: Pequeña memoria rápida usada para mantener datos selectos y proporcionar acceso más rápido que sería imposible en otro caso.

Memoria de acceso aleatorio (RAM): *Véase Memoria principal.*

Memoria de sólo lectura (ROM): Tipo de almacenamiento primario donde el fabricante almacena una vez los programas y los datos, que luego se recuperan tantas veces como sea necesario. ROM no permite el almacenamiento de nuevos programas o datos.

Memoria flash: Tipo de memoria no volátil usada como dispositivo de almacenamiento secundario que puede borrarse y reprogramarse en bloques de datos.

Memoria interna: *Véase memoria principal.*

Memoria principal: Unidad de la memoria que trabaja directamente con la CPU y donde deben residir los datos e instrucciones para su procesamiento. También se denomina *memoria de acceso aleatorio* (*RAM*), *almacenamiento primario* o *memoria interna*.

Memoria virtual: Técnica que permite la ejecución de programas aún cuando no estén almacenados completamente en la memoria. Proporciona al usuario la ilusión de que una gran cantidad de memoria virtual esta disponible cuando, de hecho, no es así.

Menos frecuentemente utilizada (LFU): Algoritmo que elimina de la memoria la página menos frecuentemente utilizada.

Microcomputadora: Computadora pequeña equipada con todo el hardware y software necesarios para realizar una o más tareas.

Minicomputadora: *Véase Computadora de gama media.*

MIPS: Medida de la velocidad del procesador que representa millones de instrucciones por segundo. Un sistema central que funciona a 100 Mips puede ejecutar 100 000 000 de instrucciones por segundo.

Modelo de Referencia de Interconexión de Sistemas Abiertos (OSI): Estructura conceptual de siete capas diseñada para describir arquitecturas de redes informáticas y las formas en que los datos pasan a través de ellas. Ese modelo fue desarrollado por la ISO en 1978 para definir claramente las interfaces y los protocolos para redes Multivendor, y proporcionar a los usuarios de estas redes directrices conceptuales en la construcción de tales redes.

Modelo de Referencia OSI: *Véase Modelo de referencia de interconexión de sistemas abiertos.*

Modelo de referencia TCP/IP: Acrónimo común para el paquete de protocolos de la capa de transporte y la capa de aplicación que operan sobre el protocolo Internet.

Modem: Dispositivo electrónico que conecta una terminal o una computadora con una línea de comunicación. La palabra proviene de MOdulation/DEModulation, que es lo que hace el modem: convierte un patrón binario generado por una terminal o computadora en señales eléctricas que pueden transmitirse sobre líneas de comunicación, y luego las reconvierte en patrones binarios cuando llegan a sus destinos.

Modo de núcleo: Nombre otorgado para indicar qué procesos tienen acceso privilegiado al procesador. Por tanto, todas las instrucciones de máquina son permitidas y el sistema de memoria es accesible. Contrasta con el más restrictivo *modo de usuario.*

Modo de usuario: Nombre dado para indicar los procesos a los que no se ha otorgado acceso privilegiado al procesador. En consecuencia, no se permiten ciertas instrucciones y la memoria del sistema no es accesible. Contrasta con el menos restrictivo *modo de núcleo.*

Módulo: Sección lógica de un programa. Un programa puede dividirse en un número de módulos lógicos independientes que varios programadores pueden escribir y probar.

Módulo de asignación: Sección del administrador de archivos responsable de seguir la pista de áreas sin utilizar en cada dispositivo de almacenamiento.

Módulo de interfaz de dispositivo: Transforma el número de bloque proporcionado por el sistema de archivos físico en la combinación real cilindro/superficie/registro necesaria para recuperar la información de un dispositivo de almacenamiento secundario específico.

Módulo de verificación de control de acceso: Sección del administrador de archivos que verifica a cuáles usuarios se permite realizar qué operaciones con cada archivo.

MTBF: *Véase Tiempo medio entre fallas.*

MTTR: *Véase Tiempo medio de reparación.*

Multiprocesamiento: Cuando dos o más CPU comparten la misma memoria principal, la mayor parte de los dispositivos de E/S y las mismas rutinas del programa de control. Las CPU atienden el mismo flujo de trabajo y ejecutan programas de procesamiento distintos de manera concurrente.

Multiprogramación: Técnica que permite que un solo procesador procese varios programas que residen simultáneamente en la memoria principal e intercale su ejecución al superponer solicitudes de E/S con solicitudes de la CPU. También se denomina *programación concurrente* o *multitarea.*

Multiprogramación activa: Término usado para indicar que el sistema operativo tiene más control sobre las interrupciones; diseñado para distribuir de manera justa la utilización de la CPU sobre varios programas residentes. Contrasta con la *multiprogramación pasiva.*

Multiprogramación pasiva: Término usado para indicar que el sistema operativo no tiene control sobre la cantidad de tiempo que la CPU se asigna a cada trabajo, sino que espera que cada trabajo termine una secuencia de ejecución antes de emitir una interrupción que libera a la CPU, poniéndola a disposición de otros trabajos. Contrasta con la *multiprogramación activa*.

Multitarea: Sinónimo de multiprogramación.

Mutex: Condición que especifica que sólo un proceso puede actualizar (modificar) un recurso compartido a la vez para asegurar operación y resultados correctos.

Nivel de núcleo: En un sistema operativo distribuido basado en objetos, proporciona los mecanismos básicos para construir dinámicamente el sistema operativo al crear, administrar, planificar, sincronizar y borrar objetos.

No apropiatividad: Una de cuatro condiciones para bloqueo mutuo donde se permite que un proceso retenga recursos mientras esté esperando por otros recursos para terminar su ejecución.

Nombre absoluto del archivo: Nombre de un archivo, otorgado por el usuario, precedido por el directorio (o directorios) donde se encuentra el archivo y, cuando es necesario, la etiqueta del dispositivo específico.

Nombre completo del archivo: *Véase Nombre absoluto del archivo.*

Nombre relativo del archivo: Nombre y extensión de un archivo proporcionados por el usuario. Contrasta con *nombre absoluto del archivo*.

NOS: *Véase Sistema operativo de red.*

N-step SCAN: Variante de la estrategia de planificación SCAN para dispositivos de almacenamiento con acceso directo que se usa para optimizar los tiempos de acceso.

Núcleo: Parte de procesamiento de un chip de la CPU constituido por la unidad de control y la unidad aritmética lógica. El núcleo no incluye el caché.

Objeto: Cualquiera de las muchas entidades que constituyen un sistema de cómputo, como CPU, terminales, unidades de disco, archivos o bases de datos. Cada objeto se llama por un nombre único y cuenta con un conjunto de operaciones que pueden realizarse en él.

Offset: En un entorno de asignación de la memoria paginado o segmentado, es la diferencia entre una dirección de una página y la dirección verdadera en lenguaje de máquina. Se usa para localizar una instrucción o valores de datos dentro de su marco de página. También se denomina *desplazamiento*.

Ojear: Violación del sistema de seguridad donde a usuarios no autorizados se permite buscar en directorios o archivos en almacenamiento secundario información que no tienen el privilegio de leer.

Orden de las operaciones: Convención algebraica que dicta el orden en que se calculan los elementos de una fórmula.

Ordenamiento de rotación: Algoritmo usado para reordenar solicitudes por registros dentro de pistas para optimizar el tiempo de búsqueda.

Organización de archivos directa: Archivos almacenados en un dispositivo de almacenamiento directo y organizados para proporcionar a los usuarios flexibilidad para tener acceso aleatorio a cualquier registro sin importar su posición en el archivo.

Organización indexada secuencial de registros: Forma para organizar datos en un dispositivo de almacenamiento con acceso directo. El objetivo de crear un índice es mostrar donde se almacenan los registros de datos. Cualquier registro de datos puede recuperarse al consultar primero el índice.

Organización Internacional para la Estandarización (ISO): Organización voluntaria no comercial fundada en 1946 responsable de crear normas internacionales en muchas áreas, incluyendo computadoras y comunicación. Sus miembros son las organizaciones nacionales de normas de los 89 países miembros, incluyendo ANSI para Estados Unidos.

Organización secuencial de registros: Organización de registros en una secuencia particular. Los registros en un archivo secuencial deben procesarse uno después de otro.

Orientado a objetos: Filosofía de programación donde los programas constan de módulos independientes reutilizables denominados objetos, cada uno de los cuales realiza una función específica, aunque están clasificados en clases de objetos que comparten la misma función.

OSPF: *Véase Primero la ruta abierta más corta.*

P: Operación realizada en un semáforo, que puede ocasionar que el proceso que llama espere. Proviene de la palabra holandesa *proberen*, que significa "probar", y forma parte de las operaciones P y V para probar e incrementar.

Página: Sección de tamaño fijo de un trabajo de un usuario que corresponde a marcos de página en la memoria principal.

Paginación por demanda: Esquema de asignación de la memoria que carga una página del programa en la memoria cuando es necesaria para procesamiento.

Palabra de bloqueo: Secuencia de letras y/o números que se proporcionan a los usuarios para evitar manipulación peligrosa no autorizada de sus archivos. La palabra de bloqueo sirve como una "contraseña" secreta en el sentido de que el sistema niega el acceso al archivo protegido a menos que el usuario proporcione la palabra de bloqueo correcta al acceder al archivo.

Palabra de control: Contraseña dada a un archivo por su creador.

Palabra de estado del canal (CSW): Estructura de datos que contiene información que indica la condición del canal, incluyendo tres bits para las tres componentes del subsistema de E/S: uno para el canal, otro para la unidad de control y otro para el dispositivo.

Palabra de estado del proceso (PSW): Información almacenada en un registro especial de la CPU, que incluye el conteo de instrucción actual y el contenido del registro. Cuando no está en ejecución -sino que en la cola RETENIDO, LISTO o en ESPERA- se guarda en el PCB del trabajo.

Paquete: Unidad de datos enviados a través de una red. Paquete es un término genérico usado para describir unidades de datos en todas las capas de la pila de protocolos, pero se usa más correctamente para describir unidades de datos de aplicación.

Paquete de discos: Pila removible de discos montados sobre un eje central común con espacios entre cada par de platos, de modo que las cabezas de lectura/escritura pueden moverse entre éstos.

Paquete de servicios: Término usado por algunos vendedores para describir una actualización del software del usuario para reparar los problemas existentes y/o entregar mejoras.

Par (hardware): Nodo en una red que está al mismo nivel que otros nodos sobre esa red. Por ejemplo, todos los nodos en una red de área local son pares.

Par (software): Proceso que se está comunicando con otro proceso residente en la misma capa en la pila de protocolos sobre otro nodo. Por ejemplo, si los procesos son de aplicación, se dice que son pares en la capa de aplicación.

Paralelismo explícito: Tipo de programación concurrente que requiere que el programador especifique de manera explícita cuáles instrucciones pueden ejecutarse en paralelo. Contrasta con *paralelismo implícito*.

Paralelismo implícito: Tipo de programación concurrente donde el compilador detecta automáticamente cuáles instrucciones pueden hacerse en paralelo. Contrasta con *paralelismo explícito*.

Parche: Software ejecutable que repara errores u omisiones en otro programa o parte de software.

Partición: Sección de almacenamiento en disco duro de tamaño arbitrario. Las particiones pueden ser estáticas o dinámicas.

Particiones dinámicas reubicables: Esquema de asignación de la memoria donde el sistema reubica programas en la memoria para reunir todos los bloques vacíos y compactarlos a fin de obtener un bloque de memoria suficientemente grande para acomodar algunos o todos los trabajos que están en espera de memoria.

Particiones dinámicas: Esquema de asignación de memoria donde a los trabajos se otorga tanta memoria como solicitan cuando se cargan para su procesamiento, creando así sus propias particiones en la memoria principal. Contrasta con *particiones estáticas* o *particiones fijas*.

Particiones estáticas. Sinónimo de *particiones fijas*.

Particiones fijas: Esquema de asignación de memoria donde la memoria principal se divide en secciones, con porciones asignadas a cada usuario. Contrasta con *particiones dinámicas*.

Paso a través de la seguridad: Usado para realizar actividades de validación remota en Windows 95. La información de inicio de sesión se pasa hacia al protocolo de red idóneo para procesamiento que permite a Windows 95 usar hardware y software de red existentes con toda la seguridad integrada en esos servidores de red externos.

Paso de trabajo: Unidad de trabajo ejecutado secuencialmente por el sistema operativo para satisfacer la solicitud completa del usuario. Un ejemplo común de tres pasos de trabajo son la compilación, el enlazamiento y la ejecución de un programa del usuario.

PCB: *Véase Bloque de control de procesos.*

Pila: Lista secuencial que se mantiene en la memoria principal. El contenido de la pila se recupera desde la parte superior usando un algoritmo último en entrar, primero en salir (LIFO).

Pista: Ruta a lo largo de la cual se graban datos en un medio magnético como cinta o disco.

Planificación de prioridad: Política (o algoritmo) de planificación de trabajos no expropiativo que permite la ejecución de trabajos con prioridad alta antes que la de trabajos de prioridad baja.

Planificador de alto nivel: Sinónimo de *planificador de trabajos*.

Planificador de bajo nivel: Sinónimo de *planificador de procesos*.

Planificador de E/S: Uno de los módulos del subsistema de E/S que asigna los dispositivos, las unidades de control y los canales.

Planificador de nivel medio: Planificador usado por el administrador del procesador para administrar procesos que han sido interrumpidos debido a que han excedido su rebanada de tiempo de CPU asignada. Se usa en algunos entornos altamente interactivos.

Planificador de procesos: Planificador de bajo nivel del administrador del procesador que establece el orden en que los procesos en la cola LISTO serán atendidos por la CPU.

Planificador de trabajos: El planificador de alto nivel del administrador del procesador que selecciona trabajos de una cola de trabajos de entrada con base en las características de cada trabajo. El objetivo del planificador de trabajos es colocar en una secuencia

los trabajos en la lista READY de modo que los recursos del sistema se usen de manera eficiente.

Política de búsqueda: Reglas usadas por el administrador de la memoria virtual para determinar cuándo debe copiarse una página del disco a la memoria.

Política de colocación: Reglas usadas por el administrador de la memoria virtual para determinar dónde debe cargarse la página virtual en la memoria.

Política de planificación de procesos: Cualquier política usada por el administrador del procesador para seleccionar el orden en que serán ejecutados los trabajos de entrada.

Política de planificación no expropiativa: Estrategia de planificación de trabajos que funciona sin interrupciones externas, de modo que una vez que un trabajo captura al procesador y empieza su ejecución, permanece en el estado en FUNCIONAMIENTO sin interrupción hasta que emite una solicitud de E/S o ha terminado.

Política de planificación expropiativa: Cualquier estrategia de planificación de procesos que, basada en políticas predeterminadas, interrumpe el procesamiento de un trabajo y transfiere la CPU a otro trabajo. Se usa ampliamente en entornos de tiempo compartido.

Política de reloj: Variante de la política LRU que elimina de la memoria principal las páginas que muestran la menor cantidad de actividad durante ciclos de reloj recientes.

Política de remplazo de páginas: Algoritmo usado por sistemas con memoria virtual para decidir cuál página o segmento eliminar de la memoria principal cuando se necesita una página con marcos y la memoria está llena. Dos ejemplos son FIFO y LRU.

Política de remplazo: Reglas usadas por el administrador de la memoria virtual para decidir cuál página virtual debe eliminarse de la memoria a fin de hacer espacio para una página nueva.

Política menos usada recientemente (LRU): Política de remplazo de páginas que elimina de la memoria principal las páginas que muestran la menor cantidad de actividad reciente. Se basa en el supuesto de que estas páginas son las menos probables de usarse de nuevo en el futuro inmediato.

Política primero en entrar, primero en salir (FIFO): Política de remplazo de páginas que elimina de la memoria principal las páginas que llegaron primero. Se basa en el supuesto de que estas páginas son las que tienen menor probabilidad de ser usadas de nuevo en un futuro próximo.

Polling: Mecanismo de software usado para probar la bandera (dispositivo de marcado), que indica cuándo está disponible un dispositivo, una unidad de control o una ruta.

Portabilidad: Capacidad para mover todo un sistema operativo a una máquina basada en un procesador o configuración diferente con la menor cantidad de recodificación posible.

POSIX: Interfaz para sistemas operativos migrables basados en UNIX, conjunto de normas del IEEE que definen las interfaces de usuario y programación normales para sistemas operativos, de modo que los desarrolladores puedan portar programas de un sistema operativo a otro.

Prevención: Estrategia de diseño para un sistema operativo donde los recursos son administrados de modo que algunas de las condiciones necesarias para bloqueo no se cumplan.

Primera generación (1940-1945): Época de las primeras computadoras, caracterizadas por su uso de tubos de vacío y tamaño físico muy grande.

Primero el tiempo de búsqueda más corto (SSTF): Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de acce-

so. Las solicitudes por pistas se ordenan de modo que la más próxima a la pista activa actual se satisface primero y las más alejadas se hacen esperar.

Primero el trabajo más corto (SJF): *Véase Siguiente trabajo más corto primero (SJN).*

Primero en llegar, primero en servirse (FCFS): (1) el algoritmo de planificación más simple para dispositivos de almacenamiento con acceso directo que satisface solicitudes de pistas en el orden en que se reciben; (2) política (o algoritmo) de planificación de procesos no expropiativo que maneja trabajos según su momento de llegada; el primer trabajo en la lista READY es el primero que procesa la CPU.

Primero la ruta abierta más corta (OSPF): Protocolo diseñado para usar en redes del Protocolo de Internet (IP), que se ocupa de seguir la pista del estado operacional de todas las interfaces de red. Cualquier cambio al estado de una interfaz activa un mensaje de actualización.

Primitivas: Mecanismos del sistema operativo bien definidos, predecibles y de bajo nivel que permiten que las componentes de alto nivel del sistema operativo ejecuten sus funciones sin considerar manipulación directa del hardware.

Procesador: (1) Sinónimo de la CPU (unidad central de procesamiento), o (2) cualquier componente en un sistema de cómputo capaz de realizar una secuencia de actividades. Controla la interpretación y ejecución de instrucciones.

Procesador de mensajes: Interfaz de software de alto nivel para pasar datos entre procesos en un mecanismo uno a muchos y muchos a uno. Son útiles para transmitir mensajes a cualquier número de procesos.

Procesamiento concurrente: Ejecución por un sólo procesador de un conjunto de procesos de modo que parece que están sucediendo al mismo tiempo. También se denomina *multiprocesamiento*.

Procesamiento distribuido: Método de procesamiento de datos donde los archivos se almacenan en muchas ubicaciones diferentes y donde el procesamiento se lleva a cabo en sitios diferentes.

Procesamiento paralelo: Proceso de operar dos o más CPU en paralelo; es decir, que más de una CPU ejecute instrucciones de manera simultánea.

Proceso: Instancia de ejecución de un programa que es identificable y controlable por el sistema operativo.

Proceso hijo: En sistemas operativos UNIX y Linux, los procesos subordinados que son controlados por un proceso padre.

Proceso padre: En sistemas operativos semejantes UNIX y Linux, un trabajo que controla uno o más procesos hijo, que están subordinados a aquél.

Proceso servidor: Unidad lógica compuesta por uno o más controladores de dispositivos, un administrador de dispositivos y un módulo servidor de red necesarios para controlar grupos o dispositivos semejantes, como impresoras o unidades de disco, en un entorno de sistema operativo distribuido basado en procesos.

Productores y consumidores: Problema clásico donde un proceso produce datos que serán consumidos, o usados, por otro proceso. Exhibe la necesidad de cooperación de procesos.

Programa: Secuencia de instrucciones que proporciona una solución a un problema y dirige las acciones de la computadora. En un entorno de sistemas operativos, puede igualarse con un trabajo.

Programa del canal: *Véase Programa de canal de E/S.*

Programa del canal de E/S: Programa que controla los canales. Cada programa de canal especifica la acción a realizar por los dispositivos y controla la transmisión de datos entre la memoria principal y las unidades de control.

Programación concurrente: Técnica de programación que permite que un sólo procesador ejecute simultáneamente conjuntos de instrucciones. También se denomina *multiprocesamiento* o *multitarea.*

Prompt del sistema: Señal desde el sistema operativo de que está listo para aceptar un comando de usuario, como C:\>.

Propietario: Uno de los tres tipos de usuario a los que se permite tener acceso a un archivo. El propietario es quien crea originalmente el archivo. Los otros dos tipos son "grupo" y "los demás", también denominado "mundo" en otros sistemas.

Protocolo: Conjunto de reglas para controlar el flujo de mensajes a través de una red.

Protocolo de encaminamiento de información (RIP): Protocolo de encaminamiento usado por IP. Se basa en un algoritmo vector distancia.

Protocolo de Internet (IP): Protocolo de la capa de red usado para encaminar datos de una red a otra. Fue desarrollado por el Departamento de Defensa de Estados Unidos.

Protocolo de transferencia de archivos (ftp): Protocolo que permite a un usuario en un anfitrión tenga acceso y transfiera archivos hacia o desde otro anfitrión sobre una red TCP/IP.

Protocolo ligero de acceso a directorios (LDAP): Protocolo que define un método para crear directorios de recursos en una red que pueden buscarse. Se denomina "ligero" porque es una versión simplificada y habilitada de TCP/IP del protocolo de directorio estilo X.500.

Pruebas comparativas: Herramienta de medición usada para medir y evaluar objetivamente el rendimiento de un sistema al ejecutar un conjunto de trabajos representativos del trabajo que normalmente realiza un sistema informático.

PSW: *Véase Palabra de estado del proceso.*

Puente: Dispositivo de enlace de datos usado para interconectar múltiples redes usando el mismo protocolo. Un puente se usa para crear una red extendida de modo que varias redes personales parezcan formar parte de una red más grande.

Puerta de enlace: Dispositivo o programa de comunicación que pasa datos entre redes que tienen funciones semejantes pero diferentes protocolos. Una puerta de enlace se usa para crear una red extendida de modo que varias redes individuales parecen formar parte de una red más grande.

RAID: Conjunto redundante de discos independientes. Grupo de discos duros controlados de modo que aceleran el acceso a dispositivos de almacenamiento secundario y ayudan a la recuperación de datos.

READY: Situación de un trabajo que significa que el trabajo está listo para su ejecución, aunque está esperando a la CPU.

Rebanada de tiempo: Sinónimo de *cuanto de tiempo.*

Recuperación: (1) Cuando se detecta un bloqueo, los pasos que deben emprenderse para deshacer el bloqueo al romper el círculo de procesos en espera. (2) Cuando un sistema es asaltado, los pasos que deben emprenderse para recuperar la operabilidad del sistema y, en el mejor caso, recuperar cualquier dato perdido.

Red: Conjunto de sistemas de cómputo y dispositivos periféricos interconectados que intercambian información entre sí.

Red de área amplia (WAN): Red usualmente construida con líneas de larga distancia punto a punto, que cubre una gran área geográfica.

Red de área local (LAN): Red de datos cuyo propósito es servir a un área de sólo unos cuantos kilómetros cuadrados o menos.

Red de área metropolitana (MAN): Red de datos cuyo objetivo es dar servicio a un área aproximadamente igual a la de una gran ciudad.

Redirección: Símbolo que instruye al sistema operativo para enviar los resultados de un comando hacia o desde un archivo u otro dispositivo que no sean el teclado o el monitor.

Región crítica: Partes de un programa que deben completar su ejecución antes que otros procesos puedan tener acceso a los recursos que se están usando. Se denomina región crítica porque su ejecución debe manejarse como una unidad.

Registro: Grupo de archivos relacionados tratados como una unidad. Un archivo es un grupo de registros relacionados.

Registro: Unidad de almacenamiento del hardware usada en la CPU para almacenamiento temporal de un solo dato.

Registro de límites: Registro usado para almacenar la ubicación más alta en la memoria legalmente accesible por cada programa. Contrasta con *registro de reubicación*.

Registro de longitud fija: Registro que siempre contiene el mismo número de caracteres. Contrasta con *registro de longitud variable*.

Registro de longitud variable: Registro que no es de longitud uniforme, no deja espacio de almacenamiento vacío y no trunca ningún carácter, eliminando así las dos desventajas de los registros de longitud fija. Contrasta con *registro de longitud fija*.

Registro de reubicación: Registro que contiene el valor que debe sumarse a cada dirección referida en un programa, de modo que pueda acceder a las direcciones correctas en la memoria después de la reubicación. En caso de que el programa no haya sido reubicado, el valor almacenado en el registro de reubicación del programa es cero. Contrasta con *registro de límites*.

Rendimiento: Capacidad de un sistema operativo para proporcionar a los usuarios tiempos de respuesta aceptables bajo cargas pesadas y aplicaciones limitadas por CPU, como paquetes de gráficas y financieros, que requieren procesamiento rápido.

Resolución de la dirección: Proceso de cambiar la dirección de una instrucción o un dato a la dirección en la memoria principal en que debe cargarse o reubicarse.

Respaldo: Proceso de hacer copias de archivos para su almacenamiento a largo plazo en el sistema.

Retención de recursos: Una de cuatro condiciones para bloqueo mutuo donde cada proceso rehúsa liberar los recursos que posee hasta que se completa su ejecución, aún cuando no los esté usando porque está esperando otros recursos. Es lo contrario de *compartición de recursos*.

RETENIDO: Uno de los estados de proceso. Se asigna a procesos que están en espera de entrar a la cola READY.

Reubicación: (1) Proceso de mover un programa de un área de la memoria a otra, o (2) proceso de ajustar referencias a direcciones en un programa, ya sea por medio del software o

del hardware, para permitir que el programa se ejecute correctamente cuando se cargue en diferentes secciones de la memoria.

RIP: *Véase Protocolo de encaminamiento de información.*

Round robin: Política (o algoritmo) de planificación de procesos preferencial que asigna a cada trabajo una unidad de tiempo de procesamiento a su vez para asegurar que la CPU sea compartida equitativamente entre todos los procesos activos y ninguno la monopolice. Se usa ampliamente en sistemas interactivos.

Ruta: (1) Secuencia de encaminadores y vínculos que un paquete pasa en su camino desde lafuente hasta el nodo destino; (2) Secuencia de directorios y subdirectorios que debe seguir el sistema operativo para encontrar un archivo específico.

Salientes: Áreas de superficie planas sobre la capa reflejante de un CD o un DVD. Cada depresión se interpreta como un 1. Contrasta con las *depresiones*, que se interpretan como un 0.

Salto: Nodo de red a través del que pasa un paquete sobre la ruta entre la fuente del paquete y los nodos destino.

SCAN: Estrategia de planificación para dispositivos de almacenamiento con acceso directo que se usa para optimizar el tiempo de acceso. Las variantes más comunes son N-step SCAN y *C-SCAN*.

Script: Serie de comandos ejecutables escritos en texto plano y ejecutados por el sistema operativo en secuencia como un procedimiento.

Sector: División en una pista de un disco. Algunas veces se denominan "bloque". Las pistas se dividen en sectores durante el proceso de formateo.

Segmento: Sección de tamaño variable de un trabajo de un usuario que contiene un agrupamiento lógico de código. Contrasta con *página*.

Segunda generación (1955-1965): Segunda época del desarrollo tecnológico de las computadoras, cuando el transistor sustituyó al tubo de vacío. Las computadoras eran más pequeñas y rápidas y contaban con una mayor capacidad de almacenamiento que las computadoras de primera generación, y fueron desarrolladas para satisfacer las necesidades del mercado de negocios.

Semáforo: Tipo de datos compartidos que puede contener valores binarios o no negativos y se usa para proporcionar exclusión mutua.

Servidor: Nodo que proporciona a los clientes varios servicios de red, como recuperación de archivos, impresión o servicios de acceso a bases de datos.

Servidor de archivos: Nodo de red dedicado que proporciona almacenamiento de datos masivos para otros nodos en la red.

Servidor proxy: Servidor posicionado entre una red interna y una red externa o en Internet para mostrar todas las solicitudes de información e impedir acceso no autorizado a recursos de la red.

Siguiente trabajo más corto primero (SJN): Política (o algoritmo) de planificación no expropiativo que escoge el trabajo en espera con el tiempo de ciclo de CPU más corto. También se denomina *trabajo más corto primero*.

Sincronización de procesos: (1) La necesidad de que los algoritmos resuelvan conflictos entre procesadores en un entorno multiprocesamiento; o (2) la necesidad de asegurar que los eventos ocurran en el orden idóneo incluso si han sido efectuados por varios procesos.

Sistema de archivos NT (NTFS): Sistema de archivos introducido con Windows NT que ofrece servicios de administración de archivos, como administración de permisos, compresión,

registro de transacciones y la capacidad para crear un volumen único que genera dos o más discos físicos.

Sistema de nombre de dominio (DNS): Servicio de consulta de datos replicados, distribuido, de propósito general. Su función principal es la resolución de direcciones de Internet con base en nombres de dominio totalmente calificados, como .com (para una entidad comercial) o .edu (para una institución educativa).

Sistema de tiempo compartido: Sistema que permite que cada usuario interactúe directamente con el sistema operativo mediante comandos introducidos desde un teclado. También se denomina *sistema interactivo*.

Sistema en tiempo real: Sistema de cómputo usado en entornos en tiempo crítico que requieren tiempos de respuesta garantizados, como sistemas de navegación, sistemas de transporte rápido y sistemas de control industrial.

Sistema híbrido: Sistema de cómputo que atiende procesos por lotes e interactivos. Parece interactivo porque usuarios individuales pueden tener acceso al sistema mediante terminales y obtener respuestas rápidas, pero acepta y ejecuta programas por lotes en el fondo cuando la carga interactiva es ligera.

Sistema informático embebido: Sistema de cómputo dedicado que a menudo reside en un sistema físico más grande, como un avión de propulsión a chorro o un barco. Debe ser pequeño y rápido y trabajar con restricciones en tiempo real, ejecución a prueba de fallos y dispositivos de E/S no estándares. En algunos casos debe poder administrar actividades concurrentes, lo cual requiere procesamiento paralelo.

Sistema interactivo: Sistema que permite que cada usuario interactúe directamente con el sistema operativo mediante comandos introducidos desde un teclado. También se denomina *sistema de tiempo compartido*.

Sistema monoprogramación: Sistema de cómputo de usuario único.

Sistema operativo: Software que administra todos los recursos de un sistema de cómputo.

Sistema operativo de red (NOS): Software que administra los recursos de una red para un nodo en la red y puede proporcionar seguridad y control de acceso. Estos recursos pueden no incluir correo electrónico, servidores de archivos y servidores de impresión. *Véase también Sistema operativo distribuido.*

Sistema operativo distribuido (DO/S): Sistema operativo que proporciona el control para un sistema de cómputo (dos o más computadoras interconectadas para un propósito específico) distribuido, permitiendo tener acceso a sus recursos en forma unificada. *Véase también Sistema operativo de red.*

Sistema por lotes: Tipo de sistema desarrollado para las primeras computadoras, que usaba tarjetas perforadas o cinta como entrada. Cada trabajo se introducía ensamblando las tarjetas en un "mazo" y varios trabajos se agrupaban, o "acumulaban por lotes" antes de enviarse a la lectora de tarjetas.

Sitio: Ubicación específica sobre una red, que contiene uno o más sistemas de cómputo.

Situación del proceso: Información almacenada en el PCB del trabajo que indica la posición actual del trabajo y los recursos responsables de esa situación.

SJF: *Véase Primero el trabajo más corto.*

SJN: *Véase Siguiente trabajo más corto siguiente.*

Socket: Interfaces de comunicación abstracta que permiten que las aplicaciones se comuniquen a la vez que ocultan las comunicaciones reales desde las aplicaciones.

Software: Colección de programas usados para realizar ciertas tareas. El software se clasifica en tres grandes categorías: programas del sistema operativo, compiladores y ensambladores y programas de aplicación.

Software antivirus: Software diseñado para detectar y recuperarse de ataques de virus y gusanos. Suele ser parte de un paquete de software de protección del sistema.

Software pirata: Software obtenido ilegalmente.

spin lock: Mecanismo de sincronización Windows usado por el núcleo y partes del ejecutivo que garantiza acceso mutuamente excluyente a una estructura de datos de un sistema global por medio de múltiples procesadores.

Spoofing: Creación de direcciones IP falsas en los encabezados de paquetes de datos enviados por Internet, algunas veces con la intención de obtener acceso cuando de otra forma no sería otorgado.

Spooling: Técnica desarrollada para acelerar la E/S al recolectar en un archivo de disco ya sea entrada recibida desde dispositivos de entrada lentos o salida dirigida a dispositivos de salida lentos, como impresoras. Spooling minimiza la espera realizada por los procesos que realizan la E/S.

SRT: *Véase Tiempo más corto restante.*

SSTF: *Véase Primero el tiempo de búsqueda más corto.*

Subdirectorio: Directorio creado por el usuario dentro de los límites de un directorio existente. Algunos sistemas operativos lo denominan carpeta.

Subrutina: También se denomina "subprograma"; segmento de un programa capaz de efectuar una función específica. Las subrutinas pueden reducir el tiempo de programación cuando una función específica se requiere en más de un punto en un programa.

Subsistema: *Véase Subsistema de E/S.*

Subsistema de E/S: Colección de módulos dentro del sistema operativo que controla todas las solicitudes de E/S.

Sufijo: *Véase extensión.*

Supercomputadoras: Las más avanzadas y más rápidas computadoras que se produjeron, usadas para realizar cálculos complicados a la velocidad máxima permitida por la tecnología actual.

Superponer: Técnica usada para incrementar el tamaño aparente de la memoria principal. Esto se logra al mantener en la memoria principal sólo los programas o datos que están activos actualmente; el resto se mantiene en almacenamiento secundario. Ocurre superposición cuando segmentos de un programa se transfieren de almacenamiento secundario a la memoria principal para su ejecución, de modo que dos o más segmentos ocupan las mismas ubicaciones de almacenamiento en momentos diferentes.

Supervivencia del sistema: Capacidad de un sistema para lograr su misión, de manera oportuna, en presencia de ataques, fallos o accidentes.

Suplantación: En Windows, la capacidad de que un hilo en un proceso asuma la identidad de seguridad en un hilo en otro proceso y realice operaciones en nombre de ese hilo. Usado por subsistemas de entorno y servicios de red cuando se tiene acceso a recursos remotos para aplicaciones del cliente.

Suplantación de identidad: Técnica usada para engañar a los consumidores para que revelen información personal, al presentarse como una entidad legítima.

Tabla de asignación de archivos (FAT): Tabla usada para seguir la pista de segmentos no contiguos de un archivo.

Tabla de asignación de memoria (MEMORIAT): Tabla en la memoria principal que contiene tantas entradas como marcos de página y enumera la ubicación y el estado libre/ocupado de cada una.

Tabla de trabajo (JT): Tabla en la memoria principal que contiene dos entradas para cada trabajo activo: el tamaño del trabajo y la ubicación en la memoria donde está almacenada la tabla de mapa. Se usa para esquemas de asignación de memoria por paginación.

Tabla del mapa de página (PMT): Tabla en la memoria principal con la información vital para cada página, incluyendo el número de página y su dirección en la memoria de la página con marcos correspondiente.

Tabla del mapa de segmentos (SMT): Tabla en la memoria principal con la información vital para cada segmento, incluyendo el número de segmento y la dirección correspondiente en la memoria.

Tarea: (1) Término usado para describir un proceso, o (2) unidad básica de lenguajes de programación concurrentes que define una secuencia de instrucciones que pueden ejecutarse en paralelo con otras unidades semejantes.

Tarjeta inteligente: Pequeño dispositivo del tamaño de una tarjeta de crédito que usa tecnología de encriptado para controlar el acceso a computadoras y redes informáticas. Cada tarjeta inteligente tiene su propio identificador personal, que sólo conoce el usuario, así como su propia contraseña almacenada y codificada.

Tarjetas de control: Tarjetas que definen la naturaleza exacta de cada programa y sus requerimientos. Contiene información que dirige al sistema operativo para realizar funciones específicas, como iniciar la ejecución de un trabajo particular. *Véase Lenguaje de control de trabajos.*

Tasa de transferencia: Tasa a que se transfieren los datos desde medios con acceso secuencial. Para cinta magnética, es igual al producto de la densidad de la cinta y su velocidad de transporte.

Terabyte (TB): Unidad de memoria o espacio de almacenamiento igual a 1 099 511 627 776 bytes o 2^{40} bytes. Un terabyte es aproximadamente igual a un quintillón de bytes.

Teraflop: Prueba comparativa usada para medir la velocidad de procesamiento. Un teraflop es igual a un quintillón de operaciones de punto flotante por segundo.

Tercera generación: Época del desarrollo de las computadoras que comenzó a mediados de la década de 1960 e introdujo circuitos integrados y miniaturización de componentes para sustituir a los transistores, reducir costos, trabajar más rápido e incrementar la confiabilidad.

TERMINADO: Estado de trabajo que significa que la ejecución del trabajo se ha completado.

Test and set: Instrucción de máquina indivisible conocida simplemente como "TS", que es ejecutada en un solo ciclo de máquina y fue presentada por primera vez por IBM para sus computadoras multiprocesamiento System 360/370 para determinar si el procesador estaba disponible.

Texto plano: En criptografía, un método de transmisión de datos sin encriptado, en texto que es legible para quien lo lea.

Thrashing: Fenómeno en un sistema con memoria virtual donde una cantidad excesiva de intercambio de páginas de un lado a otro entre la memoria principal y el almacenamiento secundario resulta en gastos generales más elevados y poco trabajo útil.

Tiempo de acceso: Tiempo requerido para posicionar la cabeza de lectura/escritura en la pista idónea a partir del momento en que se emite la solicitud de E/S.

Tiempo de acceso: Tiempo total requerido para tener acceso a datos en almacenamiento secundario. Para un dispositivo de almacenamiento con acceso directo con cabezas móviles de lectura/escritura, es la suma del tiempo de búsqueda (movimiento del brazo) y el tiempo de transferencia (transferencia de datos).

Tiempo de búsqueda: Tiempo necesario para girar el disco desde el momento en que se emite un comando de E/S hasta que el registro solicitado se mueve bajo la cabeza de lectura/escritura. También se denomina *demora de rotación*.

Tiempo de conexión: En tiempo compartido, cantidad de tiempo en que un usuario está conectado a un sistema de cómputo. Suele medirse por el tiempo transcurrido entre el inicio y la salida del sistema.

Tiempo de espera: Cantidad de tiempo que un proceso pasa esperando recursos, especialmente dispositivos de E/S. Afecta el volumen de trabajo y la utilización.

Tiempo de respuesta: Medida de la eficiencia de un sistema interactivo que sigue la pista de la velocidad con que el sistema responde a un comando de usuario.

Tiempo de respuesta: Medida de la eficiencia de un sistema que sigue la pista del tiempo requerido para ejecutar un trabajo y regresar el resultado al usuario.

Tiempo de transferencia: Tiempo necesario para la transferencia de datos entre almacenamiento secundario y la memoria principal.

Tiempo más corto restante (SRT): Política (o algoritmo) de planificación expropiativa semejante al algoritmo SJN que asigna el procesador al trabajo más próximo a completarse.

Tiempo medio de reparación (MTTR): Herramienta para la medición de recursos; tiempo medio necesario para reparar una unidad y volver a ponerla en servicio.

Tiempo medio entre fallas (MTBF): Herramienta para la medición de recursos; tiempo medio que una unidad es operacional antes de descomponerse.

Token: Patrón único de bits que todas las estaciones en la LAN reconocen como un indicador de "permiso para transmitir".

Token bus: Tipo de red de área local con nodos conectados a un cable común usando un protocolo CSMA/CA.

Token de acceso: Objeto que identifica de manera única a un usuario que se ha registrado. Un token de acceso se agrega a cada proceso que pertenece al usuario. Contiene la identificación de seguridad del usuario, los nombres del grupo al que pertenece el usuario, cualquier privilegio que disfruta el usuario, el propietario predeterminado de cualquier objeto que crean los proceso del usuario y la lista de control de acceso predeterminada a aplicar a cualquier objeto que crean los procesos del usuario.

Token ring: Tipo de red de área local con estaciones conectadas en una red de anillo. Cada estación pasa de manera constante un token hacia el otro. Sólo la estación con el token puede enviar un mensaje.

Topología de anillo: Topología de red donde cada nodo está conectado a dos nodos adyacentes. Las redes de anillo cuentan con la ventaja de que no requieren encaminamiento porque todos los paquetes se pasan simplemente a un vecino corriente arriba del nodo.

Topología de bus: Arquitectura de red para conectar elementos entre sí a lo largo de una sola línea.

Topología en estrella: Topología de red donde múltiples nodos de red están conectados a través de un nodo central único. El nodo central es un dispositivo que administra la red. Esta topología presenta la desventaja de depender de un nodo central, cuya falla podría descomponer la red.

Topología híbrida: Arquitectura de red que combina otros tipos de topologías de red, como de árbol y estrella, para dar cabida a características de operación particulares o volúmenes de tráfico.

Trabajo: Unidad de trabajo presentada por un usuario a un sistema operativo.

Trampa: Punto de entrada no especificado y sin documentar al sistema. Representa un riesgo importante para la seguridad.

Troyano: Programa de computadora malicioso con efectos colaterales no intencionales de parte del usuario que ejecuta el programa. También llamado *caballo de Troya.*

Tubería: Símbolo que dirige al sistema operativo para diversificar los resultados de un comando, de modo que se vuelvan los datos de entrada de otro comando.

Tuberías nombradas: Interfaz de software de alto nivel hacia NetBIOS, que representa el hardware en aplicaciones de red como objetos abstractos. Las tuberías nombradas se representan como objetos archivo en Windows y operan bajo los mismos mecanismos de seguridad que otros objetos ejecutivos.

Unidad aritmética lógica: Circuito de alta velocidad de la CPU que es parte del núcleo del procesador que realiza todos los cálculos y comparaciones.

Unidad central de procesamiento (CPU): Componente con los circuitos, los "chips", para controlar la interpretación y ejecución de las instrucciones. Esencialmente, control la operación de todo el sistema de cómputo. Todas las referencias a almacenamiento, manipulación de datos y operaciones de E/S son iniciadas o realizadas por la CPU.

Unidad de control: *Véase Unidad de control de E/S.*

Unidad de control de E/S: Unidad de hardware que contiene las componentes electrónicas comunes a un tipo de dispositivo de E/S, como una unidad de disco. Se usa para controlar la operación de varios dispositivos de E/S del mismo tipo.

Unidad de disco magneto-óptico (MO): Unidad de almacenamiento de datos que usa un haz láser para leer y/o escribir información grabada sobre discos magneto-ópticos.

Unidad de disco óptico: Unidad que usa un haz láser para lee y/o escribir información grabada en discos ópticos compactos.

Usuario: Cualquiera que requiera los servicios de un sistema de cómputo.

Utilización de recursos: Medida de cuánto contribuye cada unidad a la operación global del sistema. Suele proporcionarse como un porcentaje del tiempo que un recurso se usa realmente.

V: Operación realizada en un semáforo, que puede ocasionar que el proceso que llama continúe. Proviene de la palabra holandesa *verhogen*, que significa "incrementar", y forma parte de las operaciones P y V para probar e incrementar.

Velocidad de transporte: Velocidad que debe alcanzar la cinta magnética antes que los datos sean escritos en esta o leídos desde esta. Una velocidad de transporte típica es 200 pulgadas por segundo.

Verificación: Proceso para asegurarse que una solicitud de acceso es válida.

Víctima: Trabajo prescindible que se escoge para su eliminación en un sistema bloqueado a fin de proveer más recursos a los trabajos en espera y resolver el bloqueo mutuo.

Virtualización: Creación de una versión virtual de hardware o software. La virtualización del sistema operativo permite que una sola CPU ejecute imágenes de varios sistemas operativos al mismo tiempo.

Virus: Programa que se replica a sí mismo en un sistema de cómputo, incluyendo los que están en almacenamiento secundario, que se comparten entre otros sistemas de cómputo.

Volumen: Cualquier unidad de almacenamiento secundario, como discos duros, paquetes de discos, CD, DVD, discos removibles o cintas. Cuando un volumen contiene varios archivos, se denomina "volumen de varios archivos". Cuando un archivo es extremadamente grande y está contenido en varios volúmenes, se denomina "archivos en varios volúmenes".

Volumen de trabajo: Medida compuesta de la eficiencia de un sistema que cuenta el número de trabajos atendidos en una unidad de tiempo dada.

WAIT and SIGNAL: Modificación del mecanismo de sincronización test and set, diseñada para eliminar la espera activa.

Bibliografía

ACM, (1999). *Ada 95: the language for a complex world.* http://www.acm.org/sigada/ada_95/.

Anderson, R. E. (1991). Código de ética y conducta profesional de ACM: *Communications of the ACM. 35* (5), 94-99.

Barnes, J. G. P. (1980). Revisión de Ada. *Software Practice and Experience, 10,* 85187.

Belady, L. A., Nelson, R. A. y Shelder, G. S. (Junio, 1969). Una anomalía en características espacio-tiempo de ciertos programas que se ejecutan en un entorno de paginación. *CACM, 12*(6), 349-353.

Ben-Ari, M. (1982). *Principles of concurrent programming.* Englewood Cliffs, NJ: Prentice-Hall.

Bhargava, R. (1995). *Open VMS: architecture, use, and migration.* Nueva York: McGraw-Hill.

Bic, L., y Shaw, A. C. (1988). *The logical design of operating systems* (2a. ed.). Englewood Cliffs, NJ: Prentice-Hall.

Bic, L. y Shaw, A. C. (2003). *Operating system principles.* Upper Saddle River, NJ: Pearson Education, Inc.

Bourne, S. R. (1987). *The UNIX system V environment.* Reading, MA: Addison-Wesley.

Brain, M. (1994). *Win32 system services. The heart of Windows NT.* Englewood Cliffs, NJ: Prentice-Hall.

Brinch Hansen, P. (1973). *Operating system principles.* Englewood Cliffs, NJ: Prentice-Hall.

Calingaert, P. (1982). *Operating system elements: A user perspective.* Englewood Cliffs, NJ: Prentice-Hall.

Card, R., Dumas, E. y Mevel, F. (1997). *The Linux kernel book, translated by Chris Skrimshire.* Nueva York, NY: John Wiley & Sons.

Christian, K. (1983). *The UNIX operating system. Nueva* York: Wiley.

Columbus, L. y Simpson, N. (1995). *Windows NT for the technical professional.* Santa Fe: OnWord Press.

Courtois, P. J., Heymans, F. y Parnas, D. L. (Octubre, 1971). Control concurrente con lectores y escritores. *CACM, 14*(10), 667-668.

CSO Online (Mayo, 2004). 2004 E-Crime Watch survey www.csoonline.com.

Custer, H. (1993). *Inside Windows NT.* Redmond, WA: Microsoft Press.

Davis, W. S. y Rajkumar, T.M. (2001). *Operating systems—a systematic view* (5a. ed.). Reading, MA: Addison-Wesley.

Deitel, H., Deitel, P. y Choffnes, D. (2004). *Operating systems* (3a. ed.). Upper Saddle River, NJ: Pearson Education, Inc.

Denning, D. E. (1999). *Information warfare and security*. Reading, MA: Addison-Wesley.

Dettmann, T. R. (1988). *DOS programmer's reference*. Indianapolis, IN: Que Corporation.

Dickie, M. (1994). *Routing in today's internetworks. The routing protocols of IP, DECnet, NetWare, and AppleTalk*. Nueva York: Van Nostrand Reinhold.

Dijkstra, E. W. (1965). *Cooperating sequential processes*. Technical Report EWD-123, Technological University, Eindhoven, Países bajos. Reproducido en Genuys (1968), 43-112.

Dijkstra, E. W. (Mayo, 1968). *The structure of the T.H.E. multiprogramming system. CACM, 11*(5), 341-346.

Dougherty, E. R. y Laplante, P. S. (1995), *Introduction to real-time imaging, Understanding Science & Technology Series*. Nueva York: IEEE Press.

Dukkipati, N., Ganjali, Y., Zhang-Shen, R. (2005). *Typical versus Worst Case Design in Networking*, Fourth Workshop on Hot Topics in Networks (HotNets-IV), College Park.

Finkel, R. (1986). *An operating systems vade mecum*. Englewood Cliffs, NJ: Prentice-Hall.

Fitzgerald, J. (1993). *Business data communications. Basic concepts, security, and design* (4a. ed.). Nueva York: Wiley.

Forinash, D. E. (1987). *An investigation of Ada run-times supportive of real-time multiprocessor systems*. Tesis de maestría sin publicar, Department of Statistics & Computer Science, West Virginia University, Morgantown.

Fortier, P. J. (1986). *Design of distributed operating systems*. Nueva York: McGraw-Hill.

Frank, L. R. (Ed.). (2001). Quotationary. Nueva York: Random House.

Gal, E. y Toledo, S. (2005) *Algorithms and data structures for flash memories*. (Author Abstract). En ACM Computing Surveys, 37, 138(26).

Gollmann, D. (1999). *Computer security*. Chichester, Inglaterra: Wiley.

Gosling, J. y McGilton, H. (1996). *The Java language environment: Contents*. Sun Microsystems, Inc., http://java.sun.com/docs/white/langenv/

Habermann, A. N. (1976). *Introduction to operating system design*. Chicago: Science Research Associates.

Harvey, M. S. y Szubowicz, L. S. (1996). Extending OpenVMS for 64-bit addressable virtual memory. *Digital Technical Journal. 8*(2), 57-70.

Havender, J. W. (1968). Avoiding deadlocks in multitasking systems. *IBMSJ, 7*(2), 74-84.

Haviland, K. y Salama, B. (1987). *UNIX system programming*. Reading, MA: Addison-Wesley.

Hoare, C. A. R. (Octubre, 1974). Monitors: An operating system structuring concept. *CACM, 17*(10), 549-557. Erratum in (1975, February), *CACM, 18*(2), 95.

Holt, R. C. (Septiembre 1972). Some deadlock properties of computer systems. *ACM Computing Surveys, 4*(3), 179-196.

Howell, M. A. y Palmer, J.M. (1996). Integrating the Spiralog file system into the OpenVMS operating system. *Digital Technical Journal. 8*(2), 46-56.

Hugo, I. (1993). *Practical open systems. A guide for managers* (2a. ed.). Oxford, Inglaterra: NCC Blackwell Ltd.

Horvath, D. B. (1998). *UNIX for the mainframer* (2a. ed.). Upper Saddle River, NJ: Prentice-Hall PTR.

IEEE. (2004). *Standard for Information Technology- Portable Operating System Interface (POSIX) 1003.1-2001/Cor 2-2004*. IEEE Computer Society.

Intel. (1999). *What is Moore's Law?* http://www.pentium.de/intel/museum/25anniv/ hof/ moore.htm.

Johnson, J. E. y Laing, W.A. (1996). Overview of the Spiralog file system. *Digital Technical Journal. 8*(2), 5-14.

Lane, M. G. y Mooney, J. D. (1988). *A practical approach to operating systems*. Boston: Boyd & Fraser.

Lehnert, W. G. (1998). *Internet 101, a beginner's guide to the Internet and the World Wide Web*. Reading, MA: Addison-Wesley Longman.

Leonard, T. E. (Ed.). (1987). *VAX architecture reference manual*. Bedford, MA: Digital Equipment Corporation.

Lewis, T. (Agosto, 1999). Mainframes are dead, long live mainframes. *Computer, 32*(8), 102-104.

Linger, R.C. et al. (2002). Life-cycle models for survivable systems. (CMU/SEI-2002-TR-026). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University.

MacLennan, B. J. (1987). *Principles of programming languages: design, evaluation, and implementation* (2a. ed.). Nueva York: Holt, Rinehart & Winston.

Madnick, S. E. y Donovan, J. J. (1974). *Operating systems*. Nueva York: McGraw-Hill.

Negus, C. et al. (2007). *Linux Bible 2007 Edition*. Indianapolis, IN: Wiley Publishing.

Nemeth, E., Snyder, G., Seebass, S. y Hein, T. R. (1995). *UNIX system administration handbook* (2a. ed.). Upper Saddle River, NJ: Prentice-Hall.

Nickel, W. E. (1978, Noviembre). Determining network effectiveness. *Mini-Micro Systems, 11*.

Noel, K.L. y Karkhamis, N.Y. (1997). OpenVMS Alpha 64-bit very large memory design. *Digital Technical Journal. 9*(4), 33-48.

Otellini, P. (2006). Paul Otellini Keynote. http://www.intel.com/pressroom/kits/ events/ idffall_2006/pdf/idf 09-26-06 paul otellini keynote transcript.pdf

Pase, D. M y Eckl, M. A. (2005). *A comparison of single-core and dual-core Opteron processor performance for HPC*. http://www-03.ibm.com/servers/eserver/opteron/ pdf/ IBM_dualcore.pdf.

Patil, S. S. (Febrero, 1971). *Limitations and capabilities of Dijkstra's semaphore primitive for coordination among processes*. M.I.T. Proj. MAC Computational Structures Group Memo 57.

Petersen, R. (1998). *Linux: the complete reference* (2a. ed.). Berkeley, CA: Osborne/ McGraw-Hill.

Pfaffenberger, B. (Ed.). (2001). *Webster's new world computer dictionary* (9a. ed.). Nueva York: Hungry Minds.

Pinkert, J. R. y Wear, L. L. (1989). *Operating systems concepts, policies, and mechanisms*. Englewood Cliffs, NJ: Prentice-Hall.

Pouzin, L. y Zimmermann, H. (Noviembre, 1978). A tutorial on protocols. *Proceedings of the IEEE. 66*(11), 1346-1370.

Prasad, N. S. (1989). *IBM mainframes: Architecture and design.* Nueva York: McGraw-Hill.

Ritchie, D. M. y Thompson, K. (Julio-Agosto, 1978). The UNIX time-sharing system. *The Bell Systems Technical Journal, 57*(6), 1905-1929.

Rubini, A. y Corbet, J. (2001) *Linux Device Drivers* (2a. ed.). Sebastopol, CA: O'Reilly.

Ruley, J. D., et al. (1994). *Networking Windows NT.* Nueva York: Wiley.

Seyer, M. D. y Mills, W. J. (1986). *DOS:UNIX systems becoming a super user.* Englewood Cliffs, NJ: Prentice-Hall.

Shelly, G. B. y Cashman, T. J. (1984). *Computer fundamentals for an information age.* Brea, CA: Anaheim Publishing Company.

Silberschatz, A. y Galvin, P. B. (1994). *Operating systems concepts* (4a. ed.). Reading, MA: Addison-Wesley.

Shoch, J. F. y Hupp, J. A. (1982, Marzo). The "worm" programs—early experience with a distributed computation. *Communications of the ACM, 25*(3), 172-180.

Spencer, D. D. (1983). *The illustrated computer dictionary* (Ed. rev.). Columbus, OH: Charles E. Merrill.

Stair, R.M. & Reynolds, G.W. (1999) *Principles of Information Systems* (4a. ed.). Cambridge, MA: Course Technology-ITP.

Stallings, W. (1994). *Data and computer communications* (4a. ed.). Nueva York: Macmillan.

Stallings, W. (2001). *Operating systems: internals and design principles* (4a. ed.). Upper Saddle River, NJ: Prentice-Hall.

Tanenbaum, A. S. (1987). *Operating systems: Design and implementation.* Englewood Cliffs, NJ: Prentice-Hall.

Tanenbaum, A. S. (1992). *Modern operating systems.* Englewood Cliffs, NJ: Prentice-Hall.

Teorey, T. J. y Pinkerton, T. B. (Marzo, 1972). A comparative analysis of disk scheduling policies. *CACM, 15*(3), 177-184.

Thompson, K. (Julio-Agosto, 1978). UNIX implementation. *The Bell Systems Technical Journal, 57*(6), 1905-1929.

Trobel, S. y Uhl, T. (1996). *The Linux universe, installation and configuration* (2a. ed.). Nueva York, NY: Springer-Verlag.

U.S. Department of Defense. (1982). *Ada Programming Language.* (MIL-STD-1815), Washington, D.C.

Wall, K., Watson, M. y Whitis, M. (1999). *Linux programming unleashed.* Indianapolis, IN: Sams Publishing.

White, R. (2004). *How computers work, 7th edition.* Indianapolis, IN: Que.

White, R. (2006). *How computers work, 8th edition.* Indianapolis, IN: Que.

Yourdon, E. (1972). *Design of on-line computer systems.* Englewood Cliffs, NJ: Prentice-Hall.

Índice analítico

A

acceso
  de usuario, 274
    revocamiento de, 359
  derechos de, 86, 273, 326, 358
  lectura/escritura de, 322
  métodos de, 267, 302
  protocolos de, 302
  secuencial, 208, 212, 217–218, 239, 262, 267–268
    medio de, 205
  tiempo de, 216–219
ACM (Asociación para maquinaria de cómputo), 366
adelantamiento, 149, 154, 226
administración
  de archivos, 25, 417, 422
    Linux, 511–515
    MS-DOS, 437, 447–452
    relativo, 256–258
    sistema de, 247–277, 331
    UNIX, 417–423
    Windows, 480–483
  de dispositivos
    Linux, 508–511
    MS-DOS, 446–447
    UNIX, 414–416
    Windows, 476–480
  de seguridad
    Windows, 485–488
gráfica dirigida, 142, 150–152, 157, 333
algoritmo
  de carga de trabajo, 33
  de desasignación, 44–47
  de evasión, 157–158
  de hash, 260–262
  de mejor ajuste, 42–43
  de planeación de dispositivos, 227, 271
  de primer ajuste, 40–41
  de reordenamiento, 376, 378
  de transferencia de memoria principal, 96
  del banquero, 155, 157
  hashing, 261–262
  no reemplazable, 121, 124
  SJN, 120–122
Allen, Paul, 436
almacenamiento
  acceso
    directo, 204–215
    secuencial, 205, 207
  contiguo, 263
  disco
    magnético, 208
    óptico, 211
  dispositivos, 9, 272, 331
  espacio, 250, 259, 264, 266, 274, 276, 351
  indexado, 265
  secundario, 20, 205
anfitrión
  de redes, 286
  remoto, 284, 308
  servidor, 285–286
apuntador, 80, 114, 193, 261–262, 264–266, 269
archivos secuenciales, indexados, 262, 267, 269
arquitectura de red, 305
asignación de memoria, 25
  dinámica, 38, 40, 48
  esquemas de, 32–34, 48, 53, 90, 98, 375
  paginación de, 64–69
  primer ajuste de, 38–39, 41, 43
  segmentada, 86–87
almacenamiento en disco magnético de cabeza fija, 208–209, 216, 218
autenticación, 356–357

B

bases de datos, 281
  bloqueos, 145–147
  administrador de archivos, 254
Benchmarks, 389–390
  tiempo de acceso, 208, 217–218
Berners-Lee, Tim, 21
bit
  de referencia, 80–84, 90
  modificado, 82–83
bloque de control de tareas, 193
bloqueo, 17, 143, 192, 207, 217–218, 331–332, 356, 376
  activo, 140, 149
  mutuo, 140–161, 177–179, 181, 205, 333
    estrategias de, 153, 155–159
    evasión de, 155
    modelado de, 150–151
    prevención de, 153, 333
    recuperación de, 159
    resolución de, 331, 333
bloques
  de memoria, 40–48, 52
    tamaño, 40–46, 64
  libres, 44–47, 53

bomba
de tiempo, 353
lógica, 346, 353
ciclo
de realimentación
negativo, 383
positivo, 384
infinito, 18, 118
buffering, 18
búfers, 17–18, 147, 184–185, 207, 213, 224–225, 271, 376
circulares, 224
bus, topología de, 286, 289–292, 301–305

C

caché, 94–98, 213
niveles, 95
tamaño, 98
tiempo de acceso, 98
cambios de contexto, 122–126, 180
campos clave, 260
capa de red, 296, 306–307, 309–310
capacidad, 9, 11, 95, 119, 213, 380
cartucho de memoria. *Véase* memoria flash
competencia, 108, 144–145, 177, 347
CBA (dirección del byte actual), 267–269, 271
CD media, 213–215, 253–254
cilindro, 209–210, 230–232, 252–253, 260, 271
cinta magnética, 206
clave
privada, 360
pública, 360
cliente, 10–11, 191, 285–286, 336, 357–358
C-LOOK, 226, 229–230
COBEGIN, 185–188
código
Hamming, 236
reentrante, 93
COEND, 185, 187–188
cola LISTO, 111–125, 130, 191, 193, 328, 392
colas, 115
alta, 128–129
circular, 80
de multinivel, 127
distribuidas, bus dual (DQDB), 301, 304
espera, 40–42, 53
múltiple, 122
prioridad, 127–128
trabajos entrantes, 110
comando(s)
grep (UNIX), 428–429
Linux, 515–519
UNIX, 424–431
compactación, 48–53, 89, 264–265, 322
compresión de datos, 214
configuraciones simétricas, 177
conmutación
de circuitos, 298–301
páginas, 73–79, 83, 380
política, 24
superposiciones, 93
trabajos, 85
consumidores y productores, 183
contabilidad, 114–115, 309, 391
contraseñas, 17, 251, 338, 345–355, 358, 360–365, 391
encriptado, 357, 362–363
administración, 346, 352, 359, 361, 363, 365
gráficas, 364, 363–365
ingeniería social, 365
control
de acceso, 272–275, 301, 356
de concurrencia, 331
controladores de dispositivos, 205, 231, 252, 328–329, 414–416, 476–480
cortafuegos (firewall), 354, 356–357, 385
CPU (unidad central de procesamiento), 8, 26
criptografía, 357, 360
C-SCAN, 226, 229–230, 240
CSMA (acceso múltiple por detección de portadora), 301–302
CSMA/CD, 302–303
CSW (palabra de estado de canal), 225, 242
cuanto de tiempo, 124–130
variable, 128
cuellos de botella, 385

D

DASD *Véase* dispositivos de almacenamiento de acceso directo
páginación por demanda, 71–76, 85, 87, 89, 92–93
desasignación de memoria, 44–47, 321
desfragmentación, *Véase* compactación
desplazamiento (Offset), 67, 88–90, 271
Dijkstra, Edsger, 155, 161, 181, 201, 297–298
dirección
absoluta, 93
bloque de memoria, 41, 43
física, 260
Internet, 295
lógica, 260–261
registro, 122, 263
relativa, 261
resolución, 71, 296, 307
direccionamiento de memoria, 70

directorio
activo, 483–484
actual, 258, 330
de trabajo, 258, 448, 515
maestro de archivos. *Véase* MFD
disco
Blu-ray, 213–215
magnético, 116, 150, 208, 211, 216–217, 252, 253, 260, 330–333, 377–379
óptico, 22, 145, 208, 211–215, 239, 252–254, 292, 326, 350
disponibilidad, 113, 142, 172, 250–251, 318, 328, 353
estadísticas, 380, 382
dispositivos
de acceso directo a la memoria (DASDs), 204–215, 260
dedicados, 145–146, 154, 156, 204
USB, 205, 215, 239, 253, 284
DNS (servidor de nombres de dominio), 223–224, 295–296
DO/S basado en
objetos, 324 – 326, 330, 335
procesos, 324, 326–331, 334, 336–337
DVD media, 22, 208, 212–215, 253–254

E

eliminación de fallos, 15–16
encaminadores, 296–298, 300
encriptado, 361–363, 370
entrada nula, 44, 46–48
envejecimiento, 122, 127, 129, 163
espera
activa, 180
circular, 149, 151, 154, 157–159, 333
natural, 117–118, 121, 148
esquema de direcciones, 88
estados, 113
de espera, 116, 161
LISTO, 325, 507
no seguro, 156, 162, 167
seguro, 155–157
estrategias de búsqueda, 218, 226–229, 239
estructura de datos
Linux, 507–508
UNIX, 420–422
ethernet, 314
ética, 368–369
evasión de un bloqueo mutuo, 153, 155, 333
exclusión mutua, 149, 154, 182–184, 186
extensión, 256, 442

F

fallo de página, 76, 81, 84–86, 93, 113, 322, 384, 391
manejador, 74, 113
FCFS (primer llegado, primer servido), 113, 118–120, 226–228, 230, 240
FCFS, algoritmo, 118–120, 112, 126, 228
FIFO (primero en entrar, primero en salir), 76–81, 324
fragmentación
externa, 36, 38, 40, 44, 51, 65, 71, 89, 93–94
interna, 36, 38, 40, 65, 71, 93–94
FTP (protocolo de transferencia de archivos), 338, 345

G

Gates, Bill, 435–436, 463
GUI (interfaz gráfica de usuario), 195, 258
Linux, 515–520
MS-DOS, 452–458
UNIX, 423–431
Windows, 488–492
gusanos, 348, 352–356, 385

H

herramientas de medición, 380
hilos
planeador, 24, 108, 174
control de bloques, 193
múltiple, 24–25, 108, 191
programación concurrente y, 190–191, 193, 195
Windows, 474–475
sistema, 346, 354–356
Hooper, Grace, 15
hueco
interbloque, 207
interregistro, 206

I

IBG, 207
IEEE (Instituto de ingenieros eléctricos y en electrónica), 293–294, 366
inanición, 140–141, 161–163
independencia de dispositivos, 253
información de ruteo, 296–297
ingeniería social, 365
interfaz
de usuario, 4, 7–8, 285, 308
gráfica de usuario (GUI), 195, 258, 464
interrupciones, 78–79, 93, 117, 129–130, 176–178, 223–224, 325, 329
manejador de, 130, 223, 445–446
intervenciones de la línea, 348–349

J

Java, 196–198, 199
JT (tabla de trabajo), 66, 72, 90

K

kerberos, 357–358, 487

L

LAN (Redes de área local), 292
Lenguaje ADA, 193
ley de Moore, 11–12
LFU, (menos usada frecuentemente), 81
LIFO(último en llegar–primero en salir), 324
Linux, 8, 11, 14, 258, 275, 337, 351, 362, 499–520
lista
  de memoria, 38, 40, 47
  libre, 38–39, 41–43, 45–48, 51
  LISTO, 178–179
  de capacidades, 274–275, 326, 336
LOOK, 244
LRU (menos usada recientemente), 76–83, 91, 97, 408
  algoritmo, 82, 84
  políticas, 79, 81, 83–84
LTR (menor tiempo restante), 323

M

Macintosh OS, 10–11, 258, 337, 408, 416, 421, 425, 431
  lanzamientos, 399–403
macrovirus, 353
maestro/esclavo, 175
mapa de tabla de páginas. *Véase* PMT
marcos de página, 64–70, 74–82, 86, 89, 93
  direccionamiento de memoria, 66
  número de, 69–70, 73
  tamaño de, 70
  vacío, 64–65, 74, 76
medios, almacenamiento, 205, 207
mejor ajuste, 38, 40–43
asignación de memoria, 38
memoria
  acceso aleatorio, 6, 9, 32, 215
  asignación, 19, 98, 443
  asociativa, 91–92
  compactación, 53
  de acceso directo DMA, 223
  disponible, 36, 77, 85, 89, 92
  espacio disponible, 32
  flash, 9, 72, 205, 208, 215–216, 239
  administración, 32–99
  núcleo, 32
  principal
    circuito, 32
    grande, 11
    transferencias, 96
  tamaño, 40, 86
  virtual, 19, 24, 72, 86, 92–94, 114, 190, 322, 375
    administración, 94
    Linux, 504–506
    sistemas, 154, 384
    tamaño de, 375
    UNIX, 407
    visión general, 19
    Windows, 472 473
MFD (Directorio maestro de archivos), 254–256
MMT (Tabla de asignación de memoria), 66, 72, 75, 87, 89, 93
modelo de referencia OSI, 305, 309–310
módulo de verificación de control de acceso, 269, 271–273
Moore, Gordon, 12
MS-DOS, 435–465
  emulador, 490–491
  lanzamientos, 398–399, 437
MTBF (Tiempo medio entre fallas), 381–382
MTTR (Tiempo medio de reparación), 221, 351, 381–382
multihilos, 25, 399, 403
multiprocesadores, 174, 179, 187
multiprocesamiento, 20
  configuración de, 175
  sistemas de, 172–173
multiprogramación, 24, 94, 108
  sistemas de, 18, 113, 375
mutex, 182, 184–185, 196

N

negación del servicio, 348, 354
nombres de archivos, 251–252, 254–256, 264
  absoluto, 256
  completo, 256
  extensión, 255–257, 337–338
  Linux, 512–513, 516
  MS-DOS, 437, 447–448
  relativo, 256–258
  UNIX, 418–419
  Windows, 481
normas
  de protocolos de transporte (TPS), 305, 307, 309
  inalámbricas del IEEE, 294
NOS sistemas operativos de red, 20, 284, 318–321, 336–337
N-step SCAN, 226, 229, 240, 299–300
núcleo, 14, 25, 190, 321, 323–324, 326–327
  Linux, 501–502
  MS-DOS, 438–439, 441
  UNIX, 408

O

orden de las operaciones, 187
OSPF (primero la ruta abierta más corta), 297–298

P

página
  algoritmos de borrado, 81
  interruptor, 74, 78–80, 86, 91

política de reemplazo, 76–77, 79,, 81–85
solicitudes, 78–80, 91
tamaño, 64–65, 68–71, 76
paginado, 71, 82, 86, 89, 93–94, 97, 213, 472–473
paquetes, 288, 298–303, 306–308, 356, 360–361
conmutación, 299, 302
de tres bytes, 302
paralelismo
explícito, 199
implícito, 188
parche, 344, 351, 385–388
ciclos, 386, 388
despliegue, 386–387
administración, 385, 387
pruebas, 387
paridad, 206, 234, 236–238, 334
bit, 205, 236
franja, 234, 237–238
particiones, 22, 33–34, 36–38, 40, 51
dinámicas, 36, 86–87
reubicables, 48–49, 51, 53
fijas, 34–36, 38
libres, 36, 38, 45–46
reubicables, 65
tamaño, 34–36
PCB (bloques de control de procesos), 114–116, 124–125, 161, 180, 193, 225–226, 321–324
pista, 148–149, 205–218, 227–232, 353, 377–378
disco óptico, 213–215
espiral, 211, 214
solicitudes, 210, 227–229
PMT (mapa de tabla de páginas), 93
POSIX, 403, 406, 469–470, 476, 500–503
prioridad, 109, 160, 205
administración, 489
planeación, 113, 507–508
problema
de los filósofos que cenan, 161
de los fumadores, 201
del barbero durmiente, 201
procesador(es)
de núcleo múltiple, 23, 110, 174–175
débilmente acoplados, 284
maestro, 176
procesamiento
concurrente, 171–196, 202, 222, 334
de consultas, 331
distribuido, 6, 9, 20, 29, 108, 318
paralelo, 172–174, 183, 188
por lotes, 12–13, 110, 127
procesos
activos, 124, 155, 322, 325,
bloque de control de. *Véase* PCB
bloqueo, 152, 182
mutuo de, 160, 187–188
cooperación, 183, 185
espera, 178–180, 333
estado, 113–114
hijo, 411, 414
identificación, 114, 193
padre, 411–414
planeación de, 25, 110–111
políticas de planeación de, 116–117
sincronización de, 140, 173, 178, 411
solicitud de, 145, 153
tabla de, 409–411
productores y consumidores, 183, 185
programación concurrente, 172
promedio, 119–120, 123, 125
mínimo, 120
protocolo(s), 147, 288–309, 336, 356–357
de control de transmisión (TCP), 308–310
de Internet (IP), 310, 356, 361
DQDB, 306–307
CSMA/CA, 302
prueba y orden, 179, 327
puentes, 294, 313
puerta(s)
de enlace, 287
traseras, 348–349

R

RAID, 232–240
controlador, 233, 236–237
niveles, 233–239
raíz, 256, 258, 352
RAM (memoria de acceso aleatorio), 6, 9, 32, 215, 441
rebanada de tiempo, 19, 94, 124–125
recuperación, bloqueo mutuo, 159, 331–333, 344–346
redes
de área local. *Véase* LAN
distribuidas, 324–334
encriptado, 359
inalámbrica, 294, 360
LAN, 292–294
LAN, 292–296, 331, 337, 345
MAN, 293, 304–305
protocolos de, 147, 228–297, 302–309, 336, 356–357
WAN, 292–293
región crítica, 179–183, 185
registro(s)
asociativo, 91
confiabilidad de, 12, 22, 176, 221–222, 233, 284–286, 293, 300, 380–382
de longitud
fija, 259, 268–269
variable, 259, 268
de propósito general, 95
formato de, 259–260
liberación, sistemas operativos, 398–400
límites, 51–52

Linux, 501–502
MS-DOS, 452–453
reubicación, 49–53
UNIX, 424–425
Windows, 464–466
relación de aciertos, 97–98
remplazo de páginas Clock, 100, 517
rendimiento, 163, 324, 374–375, 380
RIP (Protocolo de encaminamiento de información), 297
Ritchie, Dennis, 402, 404–405
rotación
demora de, 216, 218, 231, 235, 253
ordenamiento de, 230, 232
Round robin, 113, 124–126, 154, 301, 323
algoritmo, 125–126
administración del procesador, 301
ruta, 95, 221–229, 255–256, 287, 297–300, 334, 350

S

Salientes, disco óptico, 213
SCAN, 226, 228–230, 232, 240
algoritmos de planeación, 21, 109, 112, 120, 127, 131, 177, 226, 230, 323
política, 7, 109, 117, 119, 127, 325
sector
de arranque, 351
dirección, 115, 266
disco, 64, 211–213, 230–232, 252–254, 265–266, 353
pista, 211
segmentación, 86–89, 93–94
asignación de paginación por demanda, 89, 91
segmentos, 33, 86–93, 233
seguridad, sistema, 344–365
semáforos, 180–181
servidores, 10, 22, 285–286, 319, 328–329, 336, 354, 357–358, 361, 379
procesos, 322, 329
proxy, 356–357
SIGNAL y WAIT, 179–180
Sistema III de UNIX, 398
sistema(s)
de red, 5, 20, 85, 284–286, 334, 337
distribuidos, 284, 321, 325, 327–328, 333, 344
embebidos, 13, 26
en tiempo real, 12–13, 142, 235
administrador, 122, 344–346, 350, 362, 366
híbridos, 13
operativo de red. *Véase* NOS
operativos distribuidos (DO/S), 21, 284, 318–334
rendimiento, 53, 110, 119, 378–382, 385
recursos, 16, 320, 336, 347
seguridad, 272, 344, 347, 355, 361
SJN (siguiente trabajo más corto o trabajo más corto primero), 113, 120, 122–224, 227–228
SMT (tabla de mapa de segmentos), 86–88, 90–93
sobrecarga, 53–54, 65, 90, 97, 125–126, 222–224, 239–240, 376
software
antivirus, 355–356
colaborativo, 341
solicitudes de E/S, 111–112, 117–119, 376
Spoofing, 345, 360–361
Spooling, 18, 146–147, 328
SRT (tiempo más corto restante), 113, 122, 124
SSTF (primero el tiempo de búsqueda más corto), 226–230, 240
subdirectorios, 254–255, 258, 338
subsistema de E/S, 219–225, 233, 239
superficie del disco, 209–211, 232, 239, 252, 260. *Véase* Pista
superposición, 19, 33, 92–93
supervivencia, 344–345

T

tabla de usuario, 409–411
TCP/IP, 305, 308–310
tecnología WiMAX, 294
fork, 411–414
teoría de localidad, 79, 84, 96–97
Thompson, Ken, 402, 404–405
tiempo
de búsqueda, 190, 216, 218, 226–229, 232, 235
de respuesta, 12, 36, 117–120, 125–126, 143, 191, 226, 240, 260, 376, 380–381
medio
de reparación. *Véase* MTTR
entre fallas. *Véase* MTBF
token, 286, 303–304
topología(s), 286, 291–292
de anillo, 289–291
de árbol, 290, 292
de bus, 271, 286, 289–292, 301, 303–305
de red, 286–291, 295, 334
en estrella, 286–287, 289, 291–292
híbrida, 291–292
lógica, 286
token
bus, 303
ring, 286, 303–304
Torvalds, Linus, 499–500, 502
trabajo(s)
activo, 66, 112, 154, 157, 380
inanición, 163
limitados por CPU, 112, 117, 127–129

lista de, 36, 39–40, 51, 178
planificación del, 6, 110–111, 113–115, 117, 122, 177, 383–384
por lotes, 117, 120, 127, 381
prioridad, 128, 160
secuencia, 119–120, 123, 125
transferencia
tasa de, 207, 217
tiempo de, 216, 218
transportadora de acceso múltiple. *Véase CSMA*
trashing (hiperpaginación), 75–77, 94, 380
Troyano, 349, 352–356

U

UNIX, 10–11, 14, 258, 275, 336–338, 362, 399–432
lanzamientos, 398–400
listado de directorios, 270
usuarios autorizados, 344, 348, 364

V

velocidad
angular constante (CAV), 211
lineal constante (CLV), 211
víctima de bloqueo mutuo, 159–160, 333
virus, 345, 348–352, 354–356, 385
software de protección, 322
volumen, 253–255

W

WAIT y SIGNAL, 180
WAN (Redes de área amplia), 292–293
Windows, 11, 14, 172, 256–258, 336, 463–492
lanzamientos, 399–400
WLAN (Redes de área local inalámbricas), 292–294

Esta obra se terminó de imprimir en
los talleres de Grupo Artgraph, S.A. de C.V.
Av. Peñuelas No. 15-D Col. San Pedrito Peñuelas
C.P. 76148 Querétaro, Qro.
Se imprimieron 4000 ejemplares
Diciembre de 2010

Con el mismo estilo de escritura sencillo y claro que ha hecho exitosas las ediciones anteriores, *Sistemas operativos, Sexta edición,* proporciona la mezcla ideal de operar la teoría y práctica. La cobertura incluye los fundamentos de sistemas operativos: lo que son, lo que hacen, cómo funcionan, cómo pueden evaluarse y cómo se comparan entre sí. La primera parte describe la gestión de memoria, procesadores, dispositivos, archivos y redes, así como la seguridad del sistema, ética y gestión del sistema global. La segunda parte del libro se centra en cuatro sistemas operativos específicos (UNIX, MS-DOS, Windows y Linux) y cómo se aplican a la teoría explicada en la primera parte del texto. Las actualizaciones incluyen debates de las nuevas tecnologías que afectan al diseño del sistema operativo, incluidos chips de varios núcleos y virtualización, haciendo de esta edición un recurso actual y pertinente.

Características

- Describe conceptos técnicos de una manera no-para-técnica que permite a los estudiantes comprender las complejidades de sistemas independientes y la creación de redes.
- Contiene temas de investigación en cada capítulo para alentar a los estudiantes a explorar acontecimientos de actualidad sobre temas de rápida solución.
- Contiene numerosos diagramas, ejemplos, ilustraciones y notas en un formato de dos colores.

CENGAGE Learning

http://latinoamerica.cengage.com

ISBN-13: 978-607481485-9
ISBN-10: 607481485-6

9 786074 814859